

/rha/app/courses/do180-4.18/

Chapter 1. Introduction to Kubernetes and OpenShift

Containers and Kubernetes

Quiz: Containers and Kubernetes

Red Hat OpenShift Components and Editions

Quiz: Red Hat OpenShift Components and Editions

Navigate the OpenShift Web Console

Guided Exercise: Navigate the OpenShift Web Console

Monitor an OpenShift Cluster

Guided Exercise: Monitor an OpenShift Cluster

Lab: Introduction to Kubernetes and OpenShift

Quiz: Introduction to Kubernetes and OpenShift

Summary

Abstract

Goal	Identify the main Kubernetes cluster services and OpenShift platform services and monitor them by using the web console.
Sections	• Containers and Kubernetes (and Matching Quiz) • Red Hat OpenShift Components and Editions (and Matching Quiz) • Navigate the OpenShift Web Console (and Guided Exercise) • Monitor an OpenShift Cluster (and Guided Exercise)
Lab	• Introduction to Kubernetes and OpenShift

Containers and Kubernetes

Objectives

- Describe the main characteristics of containers and Kubernetes.

Introduction to Containers and Kubernetes

Red Hat OpenShift Container Platform (RHOCP) is a complete platform to run applications in clusters of servers. OpenShift is based on existing technologies such as *containers* and Kubernetes. Containers provide a way to package applications and their dependencies that can be executed on any system with a container runtime.

Kubernetes provides many features to build clusters of servers that run containerized applications. However, Kubernetes does not intend to provide a complete solution, but rather provides extension points so system administrators can complete Kubernetes. OpenShift builds on the extension points of Kubernetes to provide a complete platform.

Containers Overview

A *container* is a process that runs an application independently from other containers on the host. Containers are created from a *container image*, which includes all the runtime dependencies of the application. Containers can then be executed on any host, without requiring the installation of any dependency on the host, and without conflicts between the dependencies of different containers.

Figure 1.1: Applications in containers versus on the host operating system

Containers use Linux kernel features, such as namespaces and control groups (cgroups). For example, containers use cgroups for resource management, such as CPU time allocation and system memory. Namespaces isolate a container's processes and resources from other containers and the host system.

The Open Container Initiative (OCI) maintains standards about containers, container images, and container runtimes. Because most container engine implementations are designed to conform to the OCI specifications, applications that are packaged according to the specification can run on any conforming platform.

Kubernetes Overview

Kubernetes is a platform that simplifies the deployment, management, and scaling of containerized applications.

You can run containers on any Linux system by using tools such as Podman. Further work is required to create containerized services that, for example, can run across a cluster for redundancy.

Kubernetes creates a cluster that runs applications across multiple nodes. If a node fails, then Kubernetes can restart an application in a different node.

Kubernetes uses a declarative configuration model. Kubernetes administrators write a definition of the workloads to execute in the cluster, and Kubernetes ensures that the running workloads match the definition. For example, an administrator defines several workloads. Each workload defines the amount of required memory. Kubernetes then creates the necessary containers in different nodes, to meet the memory requirements.

Kubernetes defines workloads in terms of *Pods*. A pod is one or multiple containers that run in the same cluster node. Pods with multiple containers are useful in certain situations, when two containers must run in the same cluster node to share some resource. For example, a job is a workload that runs a task in a pod until completion.

Kubernetes Features

Kubernetes offers the following features on top of a container infrastructure:

Service discovery and load balancing

Distributing applications over multiple nodes can complicate communication between applications.

Kubernetes automatically configures networking and provides a DNS service for pods. With these features, pods can communicate with services from other pods transparently across nodes by using only hostnames instead of IP addresses. Multiple pods can back a service for performance and reliability. For example, Kubernetes can evenly split incoming requests to an NGINX web server by considering the availability of the NGINX pods.

Horizontal scaling

Kubernetes can monitor the load on a service, and create or delete pods to adapt to the load. With some configurations, Kubernetes can also provision nodes dynamically to accommodate cluster load.

Self-healing

If applications declare health check procedures, then Kubernetes can monitor, restart, and reschedule failing or unavailable applications. Self-healing protects applications both from internal failure (the application stops unexpectedly) or external failure (the node that runs the application becomes unavailable).

Automated rollout

Kubernetes can gradually roll out updates to your application's containers. If something goes wrong during the rollout, then Kubernetes can roll back to the previous version of the deployment. Kubernetes routes requests to the rolled out version of the application, and deletes pods from the previous version when the rollout completes.

Secrets and configuration management

You can manage the configuration settings and secrets of your applications without requiring changes to containers. Application secrets can be usernames, passwords, and service endpoints, or any configuration setting that must be kept private.

NOTE

Kubernetes does not encrypt secrets.

Operators

Operators are packaged Kubernetes applications that can manage Kubernetes workloads in a declarative manner. For example, an operator can define a database server resource. If the user creates a database resource, then the operator creates the necessary workloads to deploy the database, configure replication, and back up automatically.

Kubernetes Architectural Concepts

Kubernetes uses several servers (also called *nodes*) to ensure the resilience and scalability of the applications that it manages. These nodes can be physical or virtual machines. The nodes come in two types, each of which provides a different aspect of the cluster operations.

Control plane nodes provide global cluster coordination for scheduling the deployed workloads. These nodes also manage the state of the cluster configuration in response to cluster events and requests.

Compute plane nodes run the user workloads.

Figure 1.2: Kubernetes components

Although a server can act as both a control plane node and a compute node, the two roles are usually separated for increased stability, security, and manageability. Kubernetes clusters can range from small single-node clusters, to large clusters with up to 5,000 nodes.

The Kubernetes API and Configuration Model

Kubernetes provides a model to define the workloads to run on a cluster.

Administrators define workloads in terms of resources.

All resource types work in the same way, with a uniform API. Tools such as the `kubectl` command can manage resources of all types, even custom resource types.

Kubernetes can import and export resources as text files. By working with resource definitions in text formats, administrators can describe their workloads instead of performing the right sequence of operations to create them. This approach is called *declarative resource management*.

Figure 1.3: Declarative resource management

Declarative resource management can reduce the work to create workloads and improve maintainability. Additionally, when using text files to describe resources,

administrators can use generic tools such as version control systems to gain further benefits such as change tracking.

To support declarative resource management, Kubernetes uses *controllers* that continuously track the state of the cluster and perform the necessary steps to keep the cluster in the intended state. This process means that changes to resources often require some time to be effective. However, Kubernetes can automatically apply complex changes. For example, if you increase the RAM requirements of an application that cannot be satisfied on the current node, then Kubernetes can move the application to a node with sufficient RAM. Kubernetes redirects traffic to the new instance only when the movement is complete.

Kubernetes also supports *namespaces*. Administrators can create namespaces, and most resources must be created inside a namespace. Besides helping organize large amounts of resources, namespaces provide the foundations for features such as resource access. Administrators can define permissions for namespaces, to allow specific users to view or modify resources.

Quiz: Containers and Kubernetes

Match the following items to their counterparts in the table.

• Concept	Description
Service discovery and load balancing	Automatic network configuration for transparent cluster communication
Horizontal scaling	Automatic change in the number of replicas of an application to respond to load
Self-healing	Automatic restart of failing or unavailable applications
Automated rollout	Management of instances of different versions of an application

• Concept	Description
Secrets and configuration management	Configuration of applications externally from their container images
Declarative resource management	Reconciliation of a description of the intended state of a cluster
Container images	Application packaging format that is compatible with container runtimes

Red Hat OpenShift Components and Editions

Objectives

- Describe the relationship between OpenShift, Kubernetes, and other Open Source projects, and list key features of Red Hat OpenShift products and editions.

Introduction to Red Hat OpenShift

Kubernetes provides many features to run container workloads on clusters. However, for some features, Kubernetes provides only the building blocks to implement them, because different environments might need different solutions. Kubernetes administrators can select an existing solution or implement their own solution to fit their specific requirements.

OpenShift uses the extensibility of Kubernetes to build a complete solution by adding the following features to a Kubernetes cluster:

Integrated developer workflow

When running applications on Kubernetes, you need to build and store container images for the applications. OpenShift integrates a built-in container registry, CI/CD pipelines, and S2I, a tool to build artifacts from source repositories to container images.

Observability

To achieve the intended reliability, performance, and availability of applications, cluster administrators might need additional tools to prevent and solve issues. OpenShift includes monitoring and logging services for both your applications and the cluster.

Server management

Kubernetes requires an operating system to run on that must be installed, configured, and maintained.

OpenShift provides installation and update procedures for many scenarios.

Additionally, hosts in a cluster use Red Hat Enterprise Linux CoreOS (RHEL CoreOS) as the underlying operating system. RHEL CoreOS is an immutable operating system that is optimized for running containerized applications. RHEL CoreOS uses the same kernel and packages as Red Hat Enterprise Linux. OpenShift also provides features to manage RHEL CoreOS following the Kubernetes configuration model.

OpenShift also brings unified tools and a graphical web console to manage all the different capabilities, and additional enhancements such as improved security measures.

This diagram shows many of the projects that provide the various functions within each OpenShift cluster:

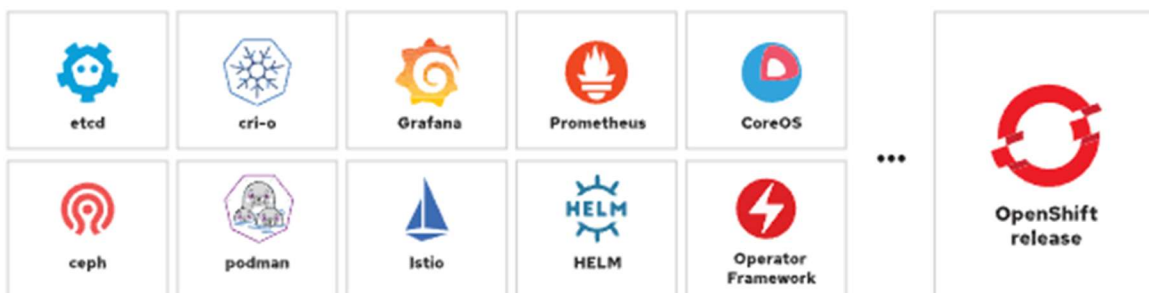


Figure 1.4: Open source projects in an OpenShift release

Because OpenShift features build on Kubernetes extensibility, administrators can often operate these features by using their existing Kubernetes knowledge. Besides

Kubernetes knowledge, OpenShift administrators can use most existing products that are designed for Kubernetes.

Red Hat OpenShift Editions

When you initially explore OpenShift, using Red Hat OpenShift Local is a viable approach that deploys a cluster on a local computer for testing and exploration. Red Hat also provides a Developer Sandbox where you get 30 days of free access to a shared OpenShift cluster. These options provide access to a cluster and support testing and exploration as you consider adopting OpenShift, but are not suitable environments for production deployments.

When you are ready to adopt Red Hat OpenShift for production workloads, various editions are available to suit any business requirement for the cluster deployment.

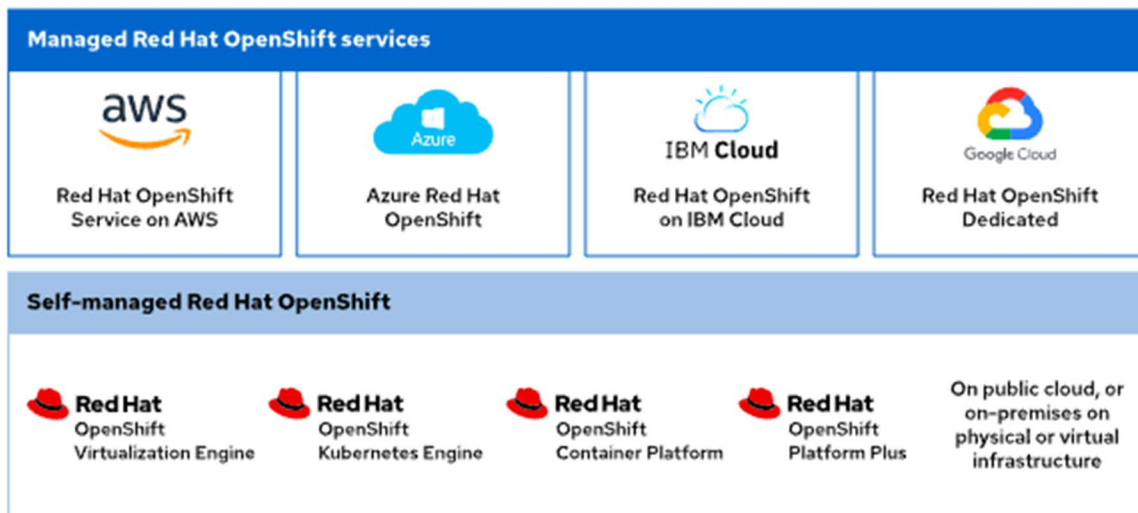


Figure 1.5: Product editions

Public cloud partners, such as Amazon Web Services, Microsoft Azure, IBM Cloud, and Google Cloud each provide quick access to an on-demand Red Hat OpenShift deployment. These managed deployments offer quick access to a cluster on infrastructure that you can rely on from a Red Hat trusted cloud provider.

You can also deploy a Red Hat OpenShift cluster by using the available installers on physical or virtual infrastructure, either on-premise or in a public cloud. These self-managed offerings are available in several forms.

The choice of deployment methods depends on many factors. When using the managed services, more responsibilities are delegated to Red Hat and the cloud provider. Because the installation process integrates with the cloud provider, the managed service creates and manages all necessary cloud resources.

When using the installers, you can still delegate hardware management to a cloud provider, or use your own. However, you manage the rest of the solution. With self-managed editions, you have greater control and flexibility, but you also take on greater responsibilities over the service.

For example, with managed services, the Red Hat Site Reliability Engineering teams update the clusters and remedy update issues (although you still participate in scheduling the updates). On self-managed editions, you update the clusters and remedy update issues. On the other hand, on self-managed editions you have complete control over aspects such as authentication, when managed editions might restrict some options.

Each managed edition documents the responsibilities for customers, Red Hat, and the cloud provider.

Additionally, to assist in managing clusters, the Red Hat Insights Advisor is available from the Red Hat Hybrid Cloud Console. The Insights Advisor helps administrators to identify and remediate cluster issues by analyzing data that the Insights Operator provides. The data from the operator is uploaded to the Red Hat Hybrid Cloud Console, where you further inspect the recommendations and their impact on the cluster.

The contents of this course apply to both managed services and self-managed editions.

Red Hat OpenShift Virtualization Engine provides the virtualization functionality found across all OpenShift editions to deploy, manage, and scale virtual machines (VMs) exclusively. This solution offers a focused approach to virtual machine management and includes key capabilities for VM support including a dedicated virtualization admin console, Red Hat OpenShift GitOps, user workload monitoring, and platform logging.

Red Hat OpenShift Kubernetes Engine includes the latest version of the Kubernetes platform with the additional security hardening and enterprise stability that Red Hat is famous for delivering. This deployment runs on the Red Hat Enterprise

Linux CoreOS immutable container operating system, by using Red Hat OpenShift Virtualization for virtual machine management, and provides an administrator console to aid in operational support.

Red Hat OpenShift Container Platform builds on the features of the OpenShift Kubernetes Engine to include additional cluster manageability, security, stability, and ease of application development for businesses. Additional features of this tier include a developer console, as well as log management, cost management, and metering information. This offering adds Red Hat OpenShift Serverless (Knative), Red Hat OpenShift Service Mesh (Istio), Red Hat OpenShift Pipelines (Tekton), and Red Hat OpenShift GitOps (Argo CD) to the deployment.

Red Hat OpenShift Platform Plus expands further on the offering to deliver the most valuable and robust available features. This offering includes Red Hat Advanced Cluster Management for Kubernetes, Red Hat Advanced Cluster Security for Kubernetes, and the Red Hat Quay private registry platform. For the most complete and full-featured container experience, Red Hat OpenShift Platform Plus bundles all the necessary tools for a complete development and administrative approach to containerized application platform management.

All OpenShift editions use the same code. Most content in this course applies to all OpenShift editions.

Quiz: Red Hat OpenShift Components and Editions

Match the following items to their counterparts in the table.

• Concept	Description
Kubernetes	An application platform that is based on containers that does not provide features such as building applications
Red Hat OpenShift Container Platform	A complete application platform that builds on the extensibility of Kubernetes
Red Hat OpenShift	An extended offering that includes a container registry

• Concept	Description
Platform Plus	
Red Hat OpenShift Local	A system to deploy OpenShift to a local computer for testing and exploration
OpenShift managed services	OpenShift offerings where Red Hat and cloud providers assume responsibilities
OpenShift self-managed offerings	OpenShift offerings where you have greater control, flexibility, and responsibilities
Red Hat Enterprise Linux CoreOS	An immutable operating system that is optimized for running containerized applications

Navigate the OpenShift Web Console

Objectives

- Navigate the OpenShift web console to identify running applications and cluster services.

Overview of the Red Hat OpenShift Web Console

The Red Hat OpenShift Web Console provides a graphical user interface to perform many administrative tasks for managing a cluster. The web console uses the Kubernetes APIs and OpenShift extension APIs to deliver a robust graphical experience. The menus, tasks, and features within the web console are always available by using the CLI. The web console provides ease of access and management for the complex tasks that cluster administration requires.

Kubernetes provides a web-based dashboard, which is not deployed by default within a cluster. The Kubernetes dashboard provides minimal security permissions, and accepts only token-based authentication. This dashboard also requires a proxy setup that limits access to the web console from only the system terminal that creates the proxy. By contrast with the stated limitations of the Kubernetes web console, OpenShift includes a fuller-featured web console.

The OpenShift web console is not related to the Kubernetes dashboard, but is a separate tool for managing OpenShift clusters. Additionally, operators can extend the web console features and functions to include more menus, views, and forms to aid in cluster administration.

Accessing the OpenShift Web Console

You access the web console by any modern web browser. The web console URL is generally configurable, and you can discover the address for your cluster web console by using the `oc` command-line interface (CLI). From a terminal, you must first authenticate to the cluster via the CLI by using the `oc login -u <USERNAME> -p <PASSWORD> <API_ENDPOINT>:<PORT>` command:

```
[user@host ~]$ oc login -u developer -p developer https://api.ocp4.example.com:6443
Login successful.
```

...output omitted...

Then, you execute the `oc whoami --show-console` command to retrieve the web console URL:

```
[user@host ~]$ oc whoami --show-console
https://console-openshift-console.apps.ocp4.example.com
```

Lastly, use a web browser to go to the URL, which displays the authentication page:

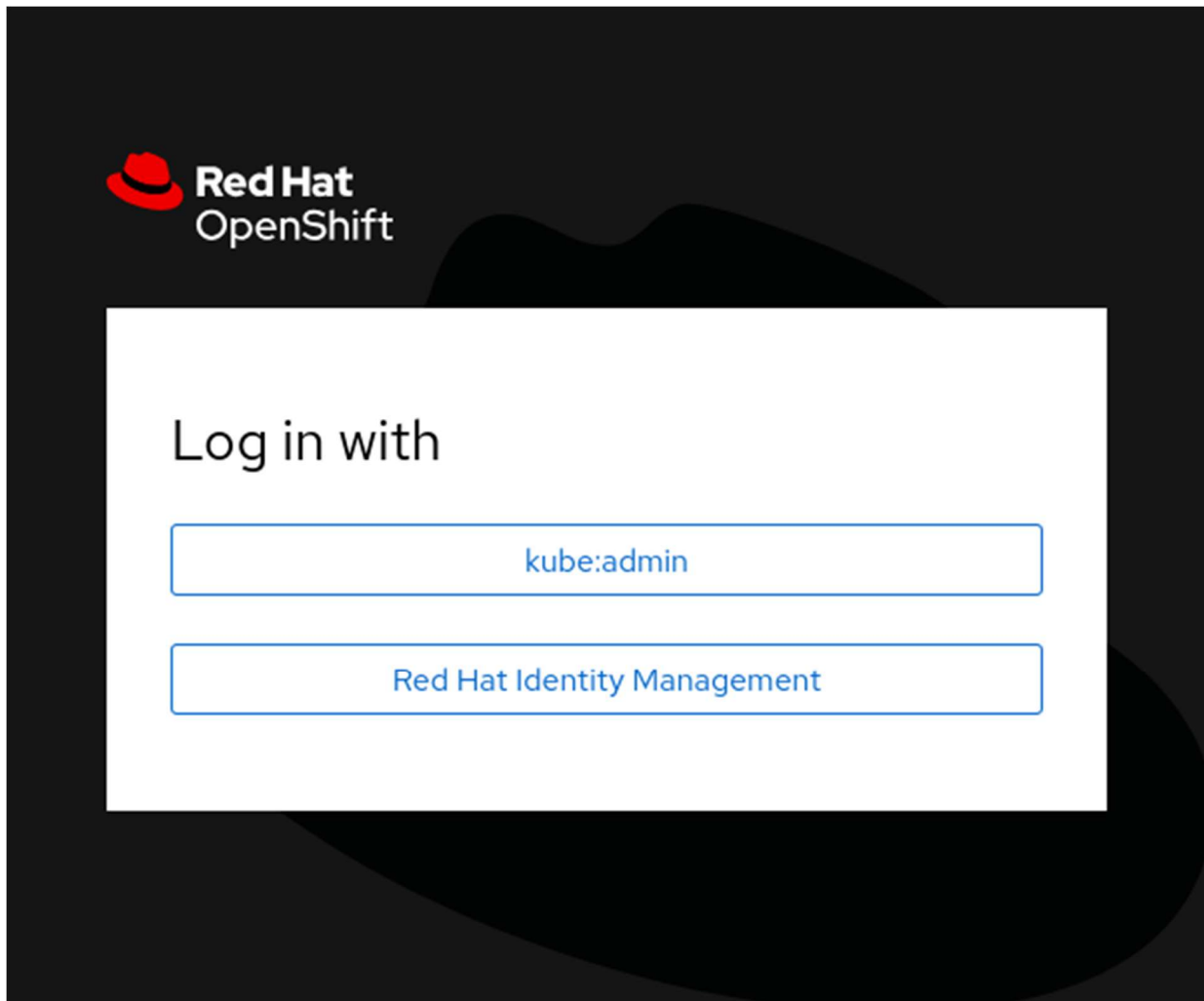


Figure 1.6: The OpenShift authentication page

Using the credentials for your cluster access brings you to the home page for the web console.

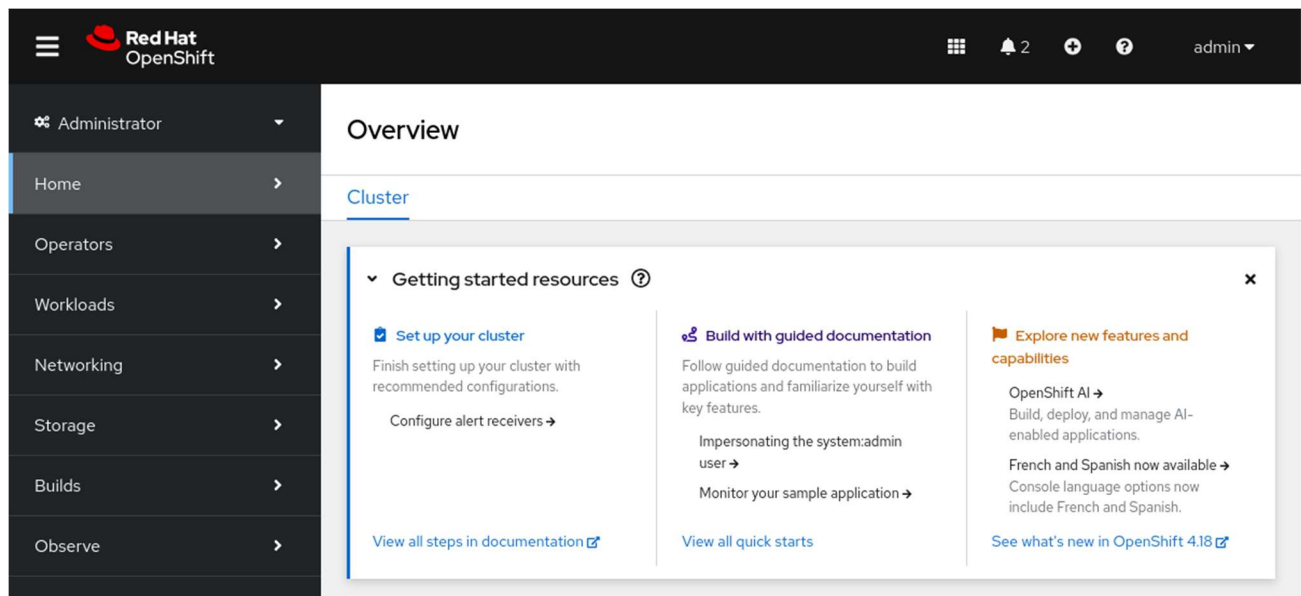


Figure 1.7: The OpenShift home page

Web Console Perspectives

The OpenShift web console provides the Administrator and Developer console perspectives. The sidebar menu layout and the features that it displays differ between using the two console perspectives. The first item on the console sidebar menu is the perspective switcher, to switch between the Administrator and the Developer perspectives.

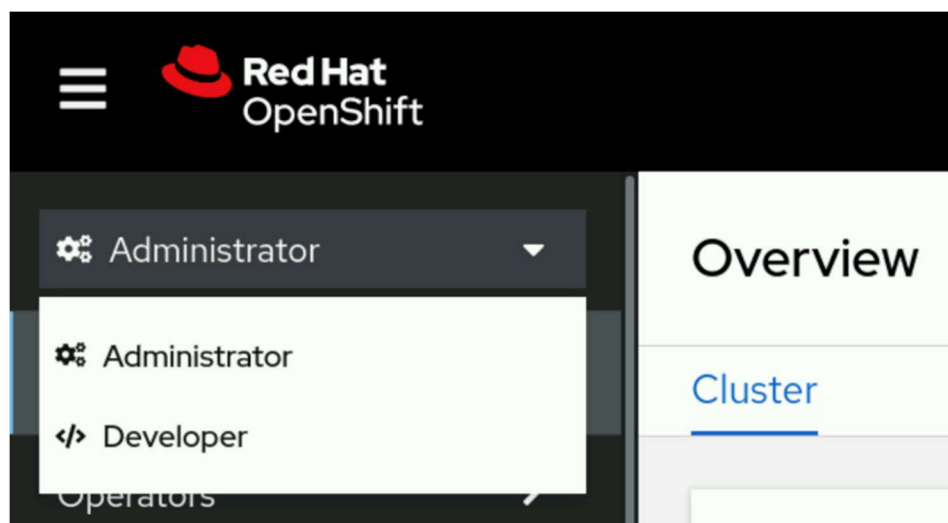


Figure 1.8: The OpenShift web console perspective switcher

Each perspective presents the user with different menu categories and pages that cater to the needs of the two separate personas. The Administrator perspective focuses on cluster configuration, deployments, and operations of the cluster and running workloads. The Developer perspective pages focus on creating and running applications.

NOTE

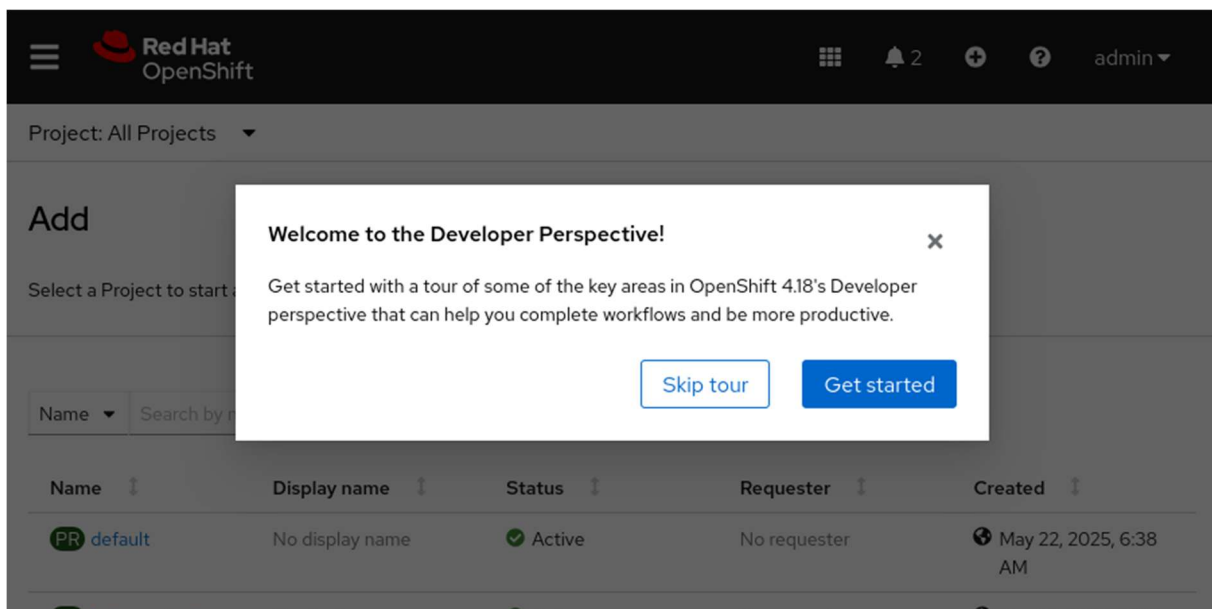
In clusters with the Red Hat OpenShift Virtualization operator deployed, a Virtualization perspective is available. The Virtualization perspective pages focus on creating and managing virtual machines.

OpenShift Web Console Layout

From the web console home page, the primary navigation method is through the sidebar. The sidebar organizes cluster functions and administration into several major categories. By selecting a category from the sidebar, the category expands to reveal the various areas that each provide specific cluster information, configuration, or functionality.

NOTE

An initial log in to the web console presents the option for a short informational tour. Click **Skip Tour** if you prefer to dismiss the tour option at this time.



By default, the console displays the **Home** → **Overview** page, which provides a quick glimpse of initial cluster configurations, documentation, and general cluster status. Go to **Home** → **Projects** to list all projects in the cluster that are available to the credentials in use.

You might initially peruse the **Operators** → **OperatorHub** page, which provides access to the collection of operators that are available for your cluster.

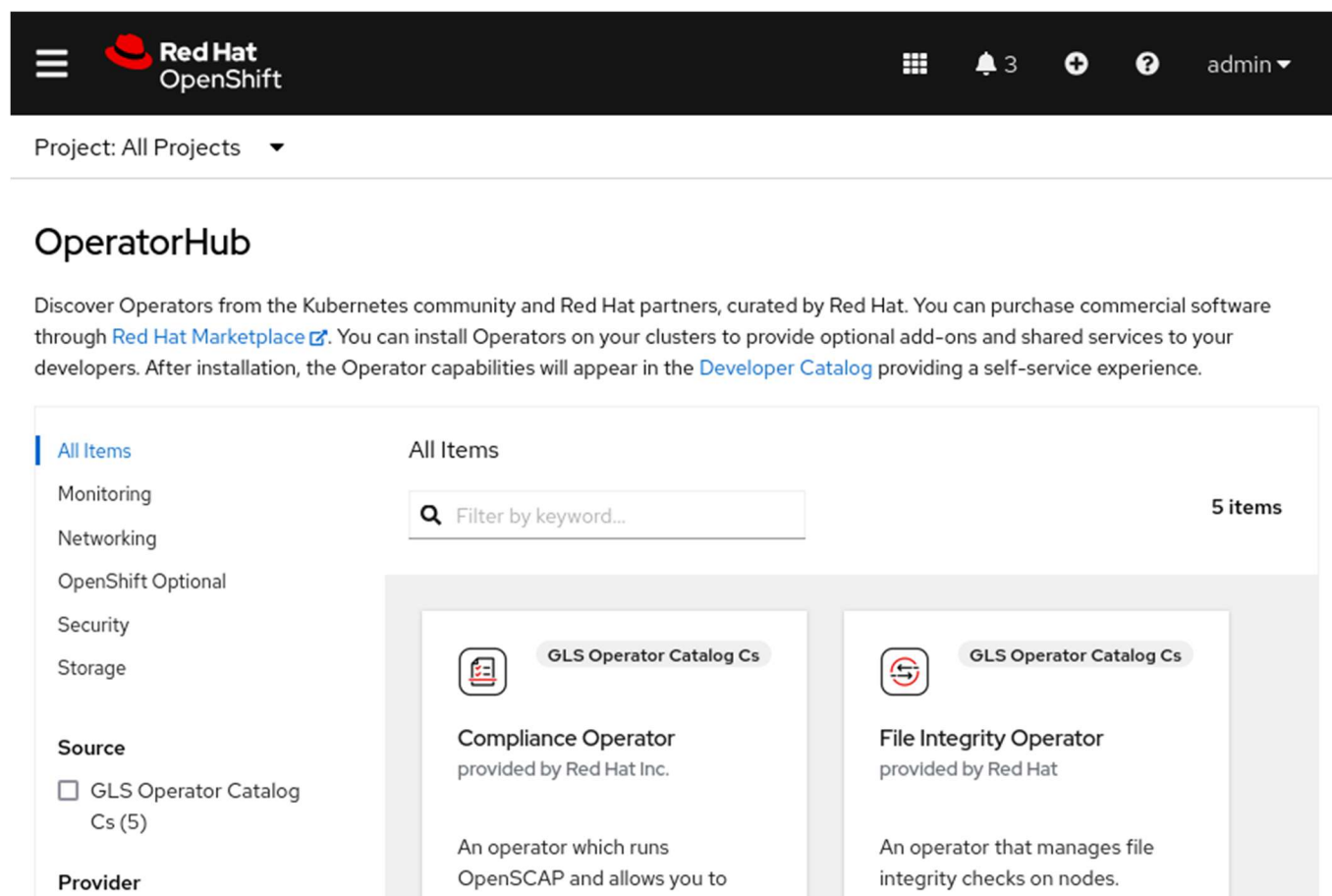


Figure 1.10: The OpenShift OperatorHub

By adding operators to the cluster, you can extend the features and functions that your OpenShift cluster provides. Use the search filter to find the available operators to enhance the cluster and to supply the OpenShift aspects that you require.

By clicking the link on the operator Hub page, you can peruse the Developer Catalog.

Project: All Projects ▾

OperatorHub

Discover Operators from the Kubernetes community and Red Hat partners, curated by Red Hat. You can purchase commercial software through [Red Hat Marketplace](#). You can install Operators on your clusters to provide optional add-ons and shared services to your developers. After installation, the Operator capabilities will appear in the [Developer Catalog](#) providing a self-service experience.

All Items

Monitoring

Networking

OpenShift Optional

Security

Storage

Source

☐ GLS Operator Catalog Cs (5)

Provider

All Items

 Filter by keyword...

5 items



GLS Operator Catalog Cs

Compliance Operator

provided by Red Hat Inc.

An operator which runs OpenSCAP and allows you to



GLS Operator Catalog Cs

File Integrity Operator

provided by Red Hat

An operator that manages file integrity checks on nodes.

Select any project, or use the search filter to find a specific project, to visit the Developer Catalog for that project, where shared applications, services, event sources, or source-to-image builders are available.

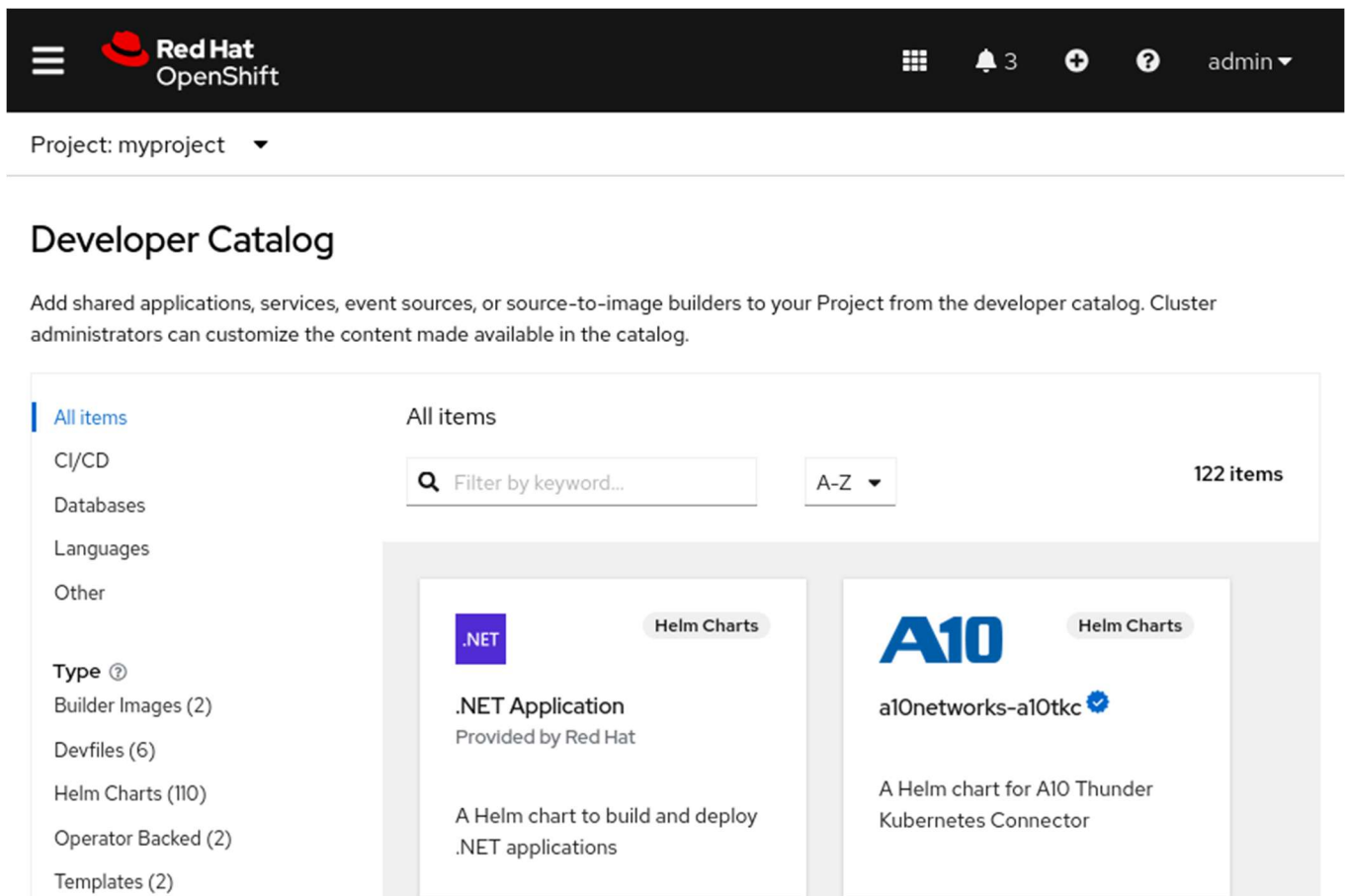


Figure 1.12: The Developer Catalog

After finding the preferred additions for a project, a cluster administrator can further customize the content that the catalog provides. By adding the necessary features to a project from this approach, developers can customize features to provide an ideal application deployment.

Red Hat OpenShift Key Concepts

When you navigate the OpenShift web console, it is useful to know some introductory OpenShift, Kubernetes, and container terminology. The following list includes some basic concepts that can help you to navigate the OpenShift web console.

- **Pods:** The smallest unit of a Kubernetes-managed containerized application. A pod consists of one or more containers.
- **Deployments:** The operational unit that provides granular management of a running application.

- Projects: A Kubernetes namespace with additional annotations that provide multitenancy scoping for applications.
- Routes: Networking configuration to expose your applications and services to resources outside the cluster.
- Operators: Packaged Kubernetes applications that extend cluster functions.

These concepts are covered in more detail throughout the course. You can find these concepts throughout the web console as you explore the features of an OpenShift cluster from the graphical environment.

Guided Exercise: Navigate the OpenShift Web Console

Access an OpenShift cluster by using its web console and review pages to identify key OpenShift cluster services.

Outcomes

- Explore the features and components of Red Hat OpenShift by using the web console.
- Create a sample application by using the Developer perspective in the web console.
- Switch to the Administrator perspective and examine the resources that are created for the sample application.
- Use the web console to describe the cluster nodes, networking, storage, and authentication.
- View the default cluster operators, pods, deployments, and services.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the cluster is validated for the exercise.

```
[student@workstation ~]$ lab start intro-navigate
```

Instructions

1. As the `developer` user, locate and go to the Red Hat OpenShift web console.

1. Use the terminal to log in to the OpenShift cluster as the developer user with the developer password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \  
3. https://api.ocp4.example.com:6443  
4.
```

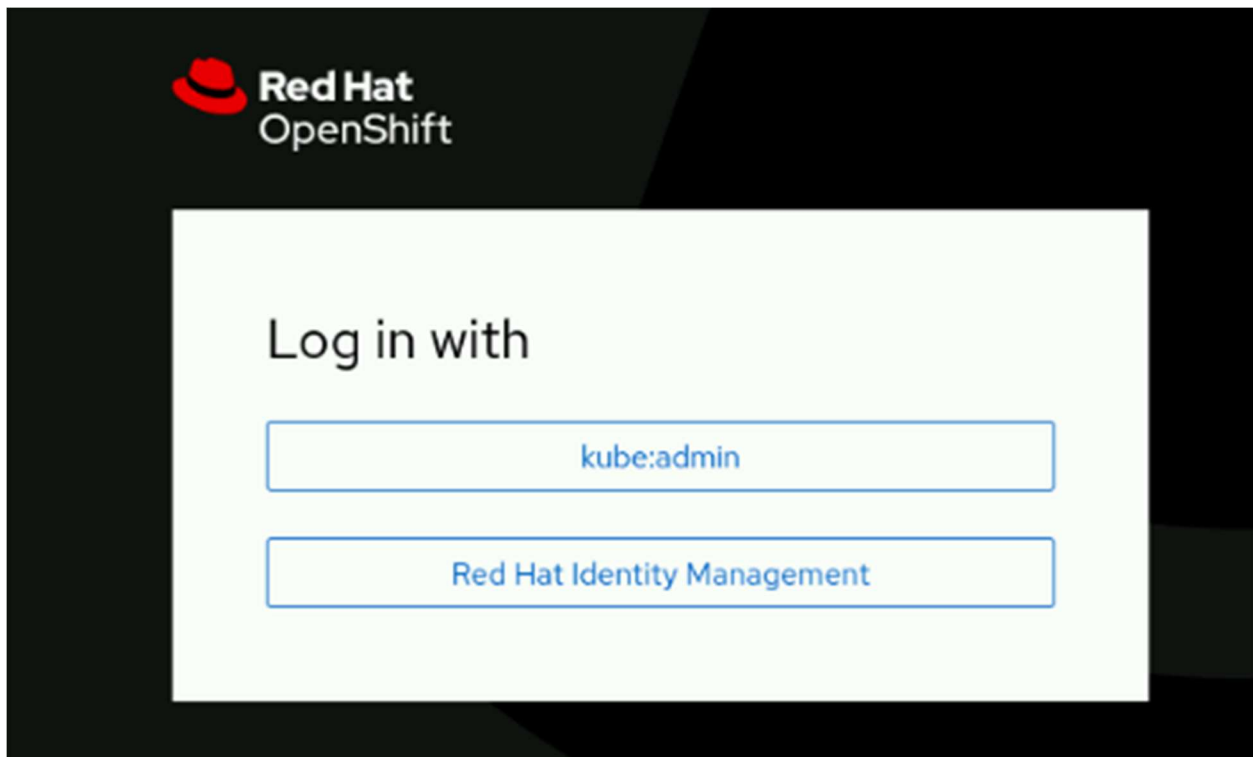
...output omitted...

5. Identify the URL for the OpenShift web console.

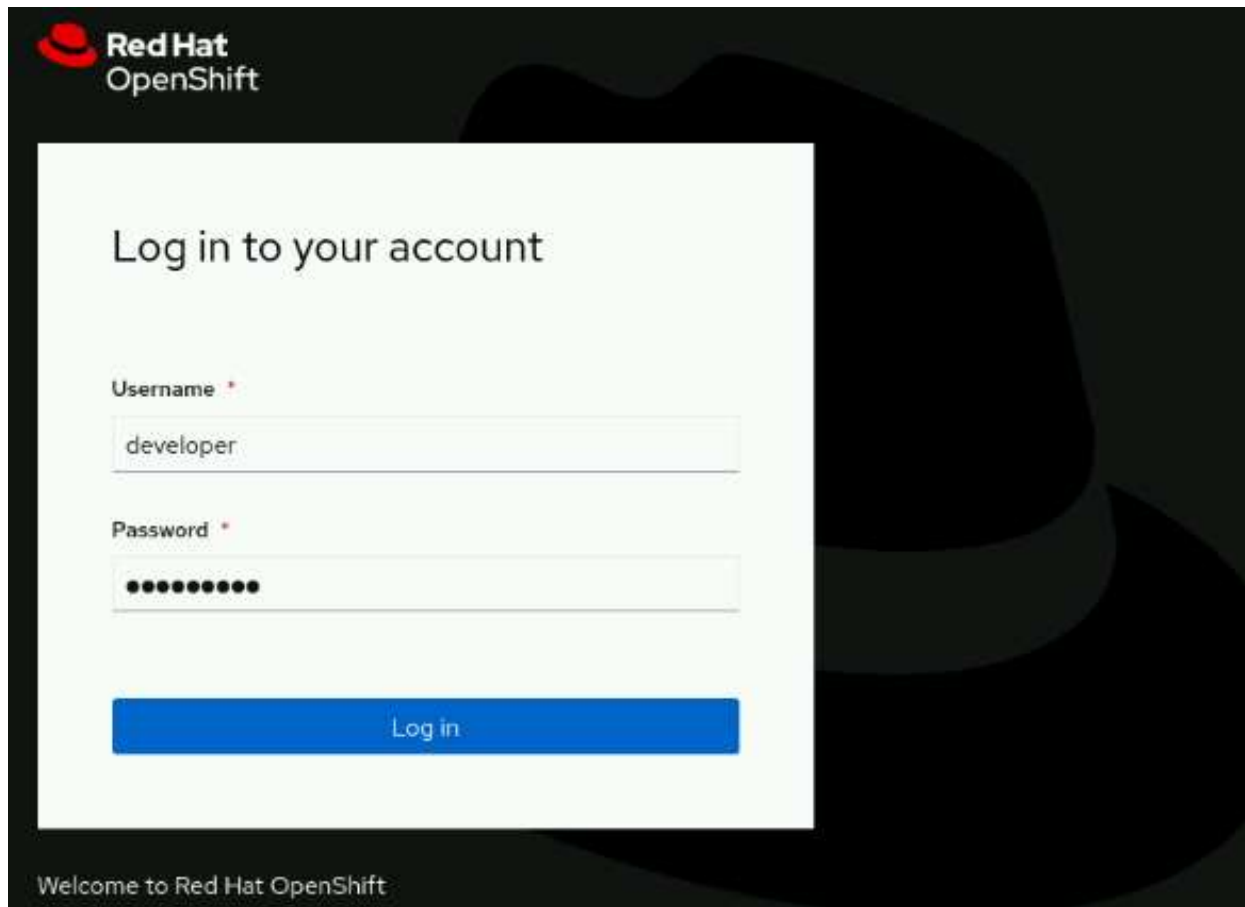
```
6. [student@workstation ~]$ oc whoami --show-console
```

<https://console-openshift-console.apps.ocp4.example.com>

7. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>.

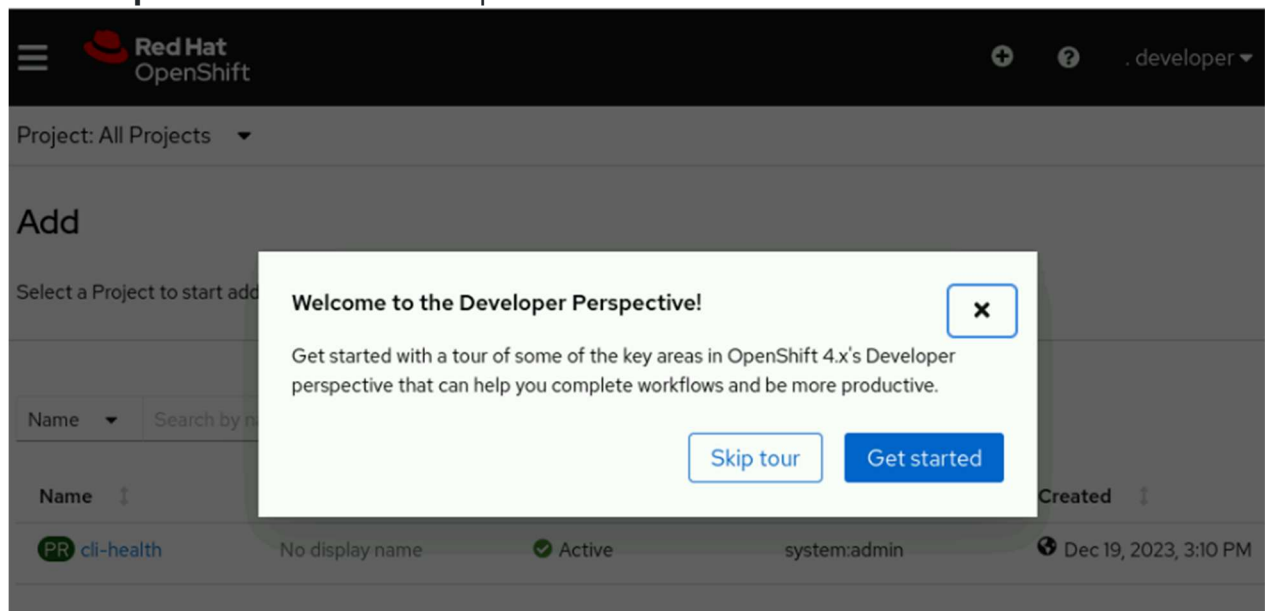


2. Log in to the OpenShift web console as the developer user.
1. Click **Red Hat Identity Management** and log in as the developer user with the developer password.



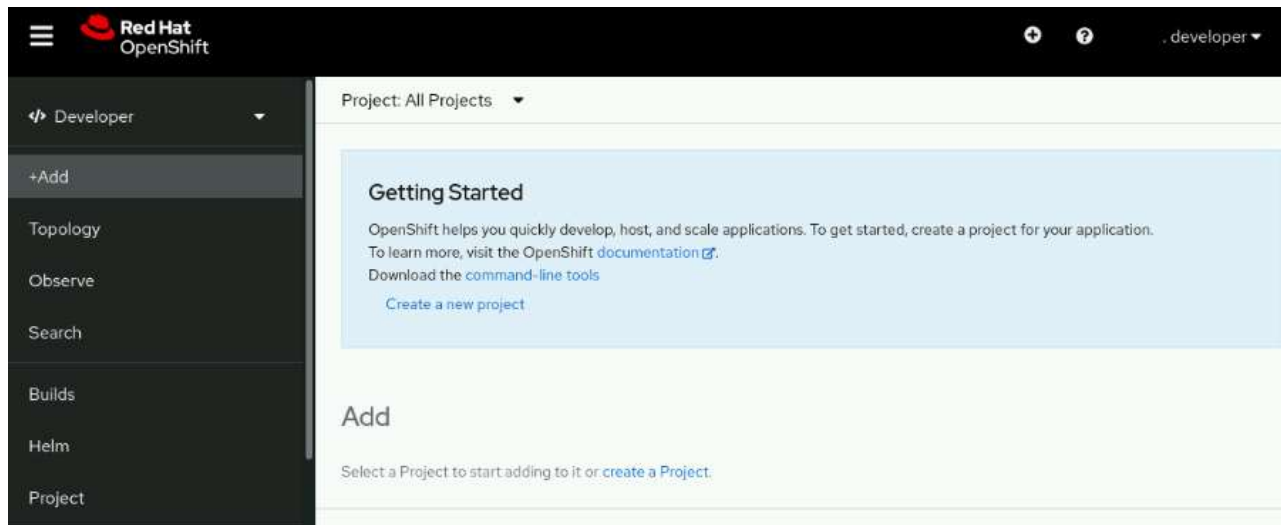
2. NOTE

3. Click **Skip Tour** to dismiss the option to view a short tour on the first visit.



- 4.

3. Use the Developer perspective of the web console to create your first project.
1. From the Getting Started section in the **+Add** page, click **Create a new project** to open the Create Project wizard.



2. Create a project named intro-navigate by using the wizard. Use intro-navigate for the display name, and add a brief description of the project.

Create Project

An OpenShift project is an alternative representation of a Kubernetes namespace.

[Learn more about working with projects](#) 

Name * 

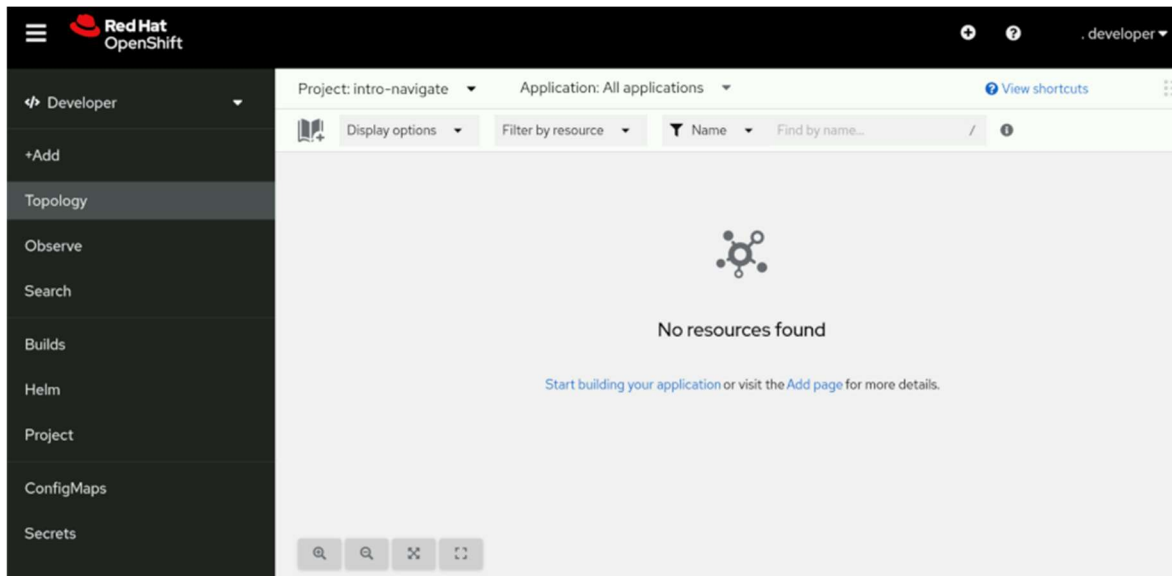
Display name

Description

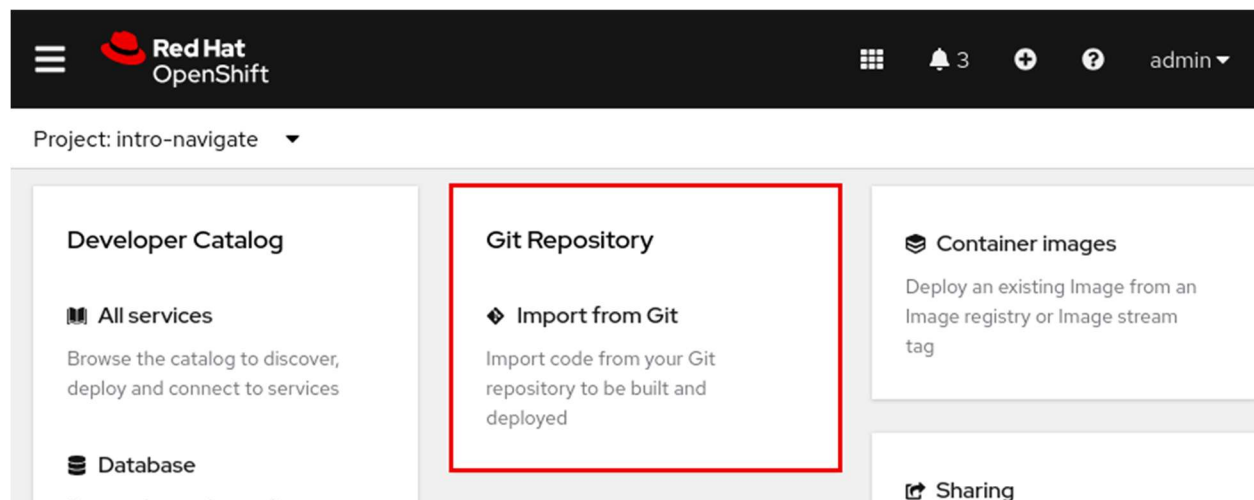
Cancel

Create

3. Click **Create** to create the project.
4. Click **Topology** to view the project.



4. Deploy a sample application in the project.
1. Select the **+Add** page. Select the **Import from Git** panel.



2. Enter <https://git.ocp4.example.com/developer/intro-navigate.git> into the Git Repo URL field and select GitLab for the Git Type field. Click **Create** to start the deployment.

 **Red Hat**
OpenShift

  developer ▾

Project: intro-navigate ▾ Application: All applications ▾

Import from Git

Git

Git Repo URL *

https://git.ocp4.example.com/developer/intro-navigate.git

✓

Validated

Git type *

 GitHub

 **GitLab**

 Bitbucket

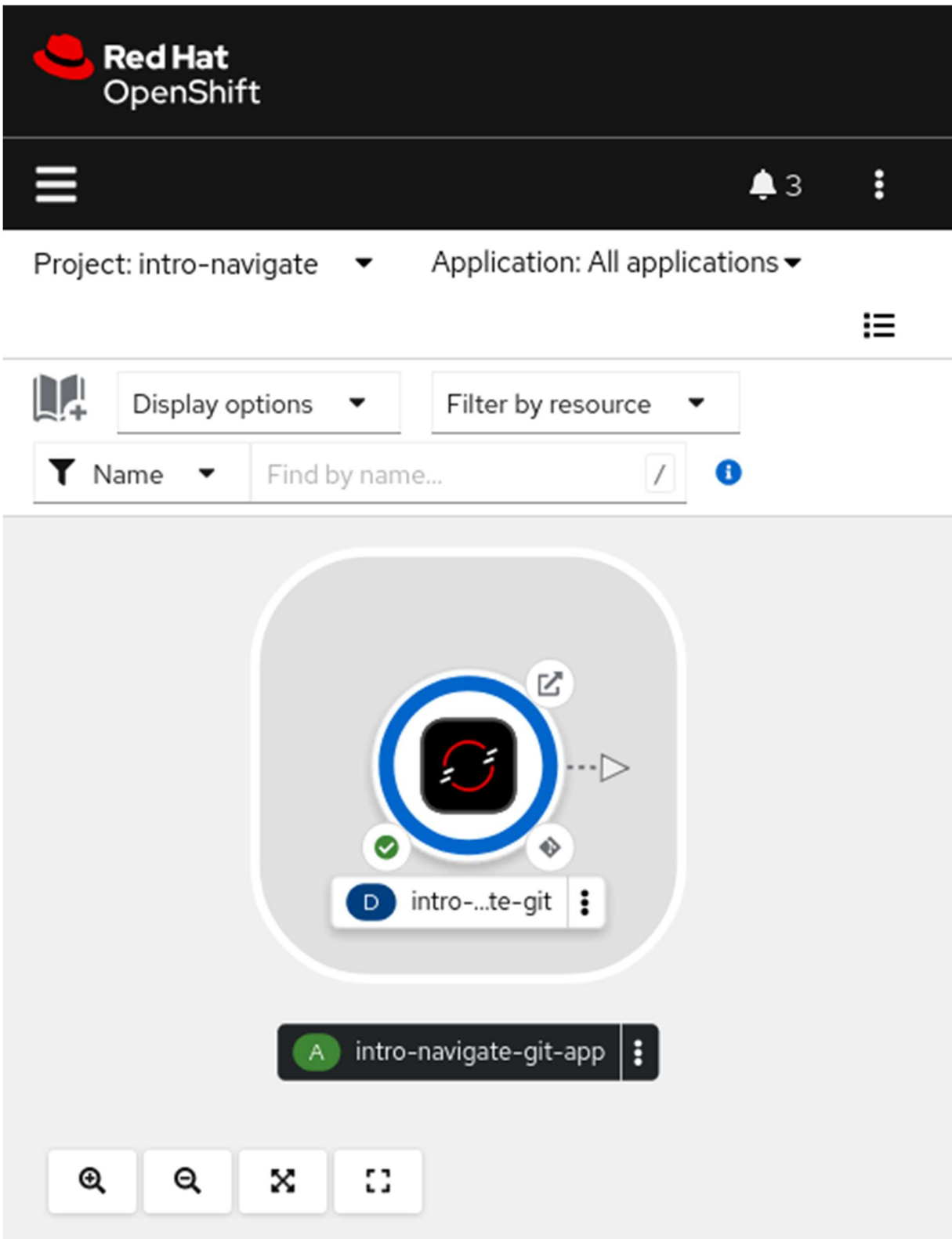
 Gitea

 Other

Create

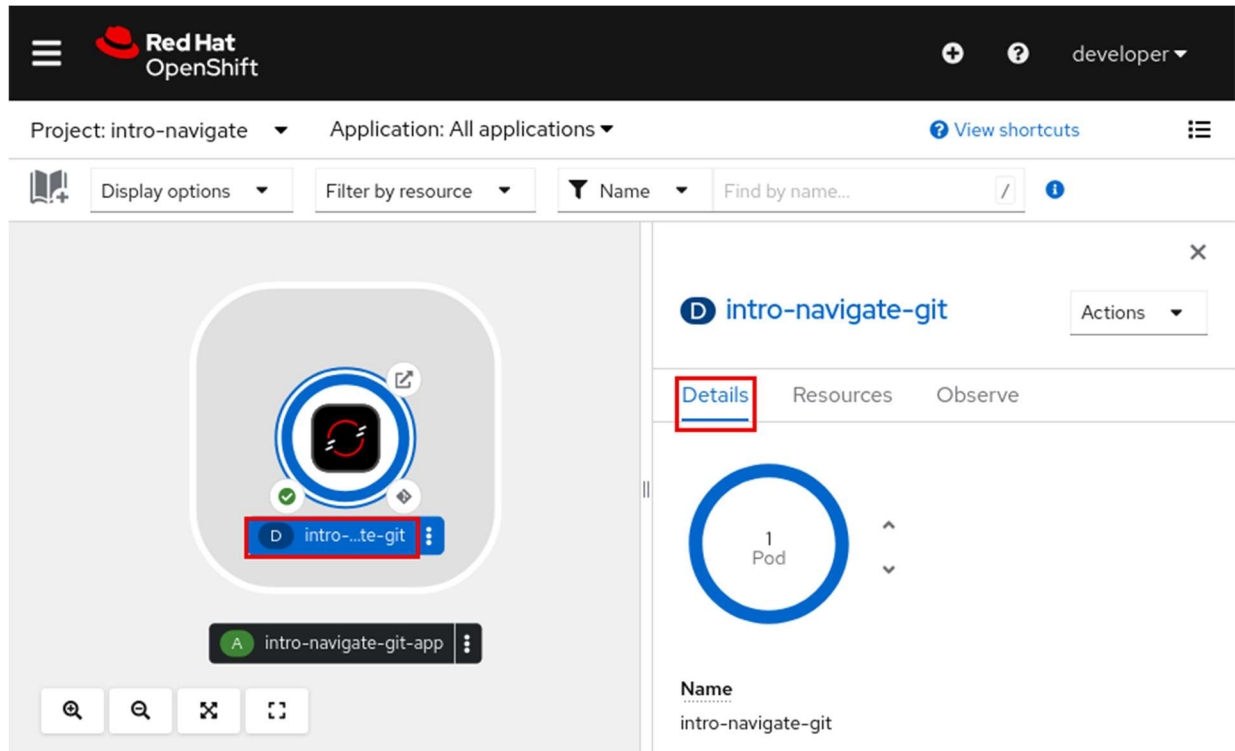
Cancel

3. The Topology page opens and displays the intro-navigate-git-app application deployment.

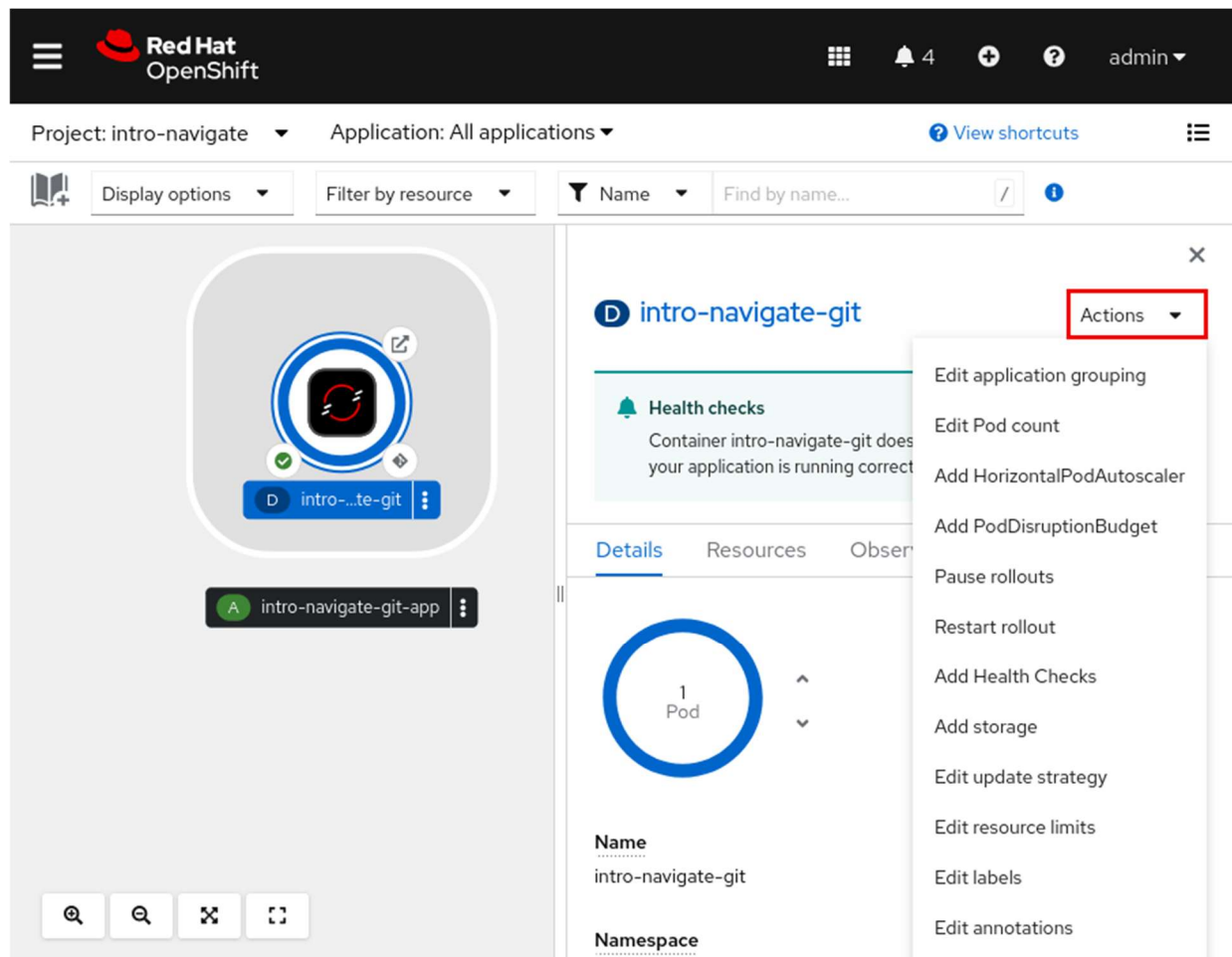


5. View the deployment details for the intro-navigate-git-app application.

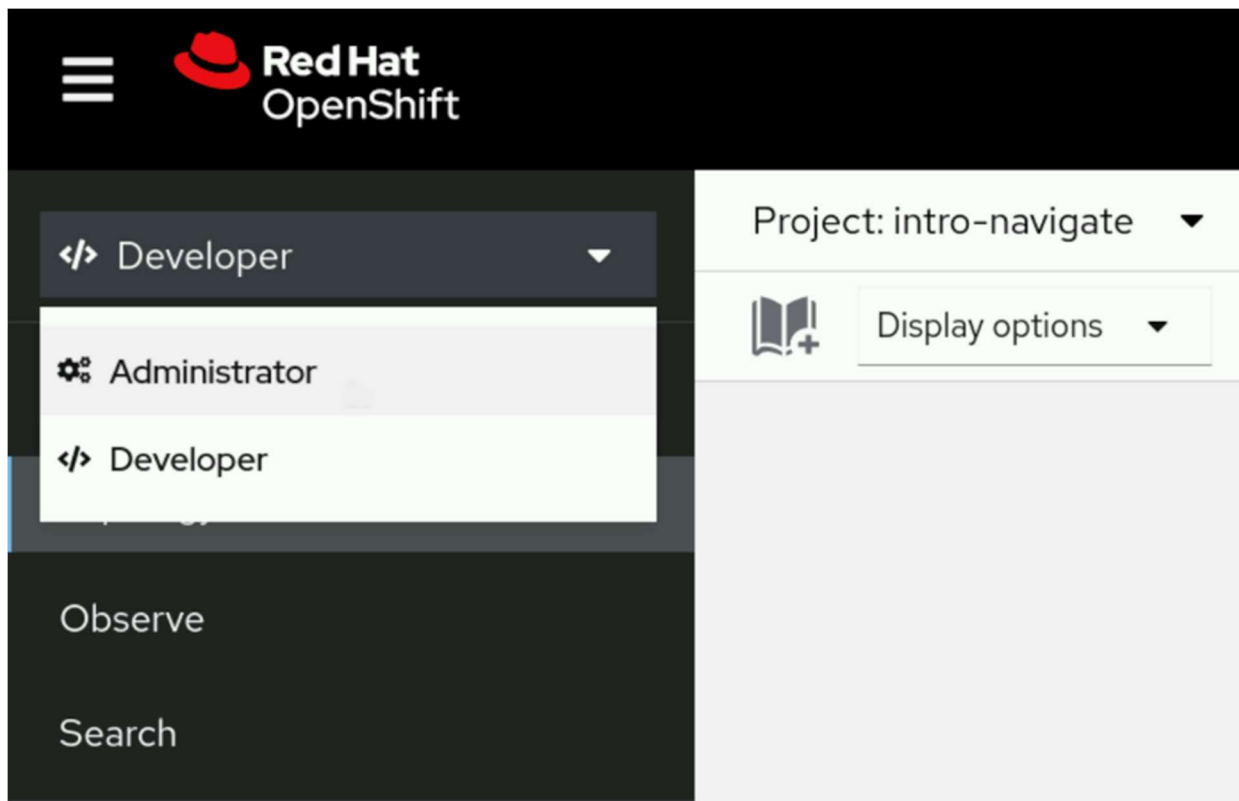
1. Click **D** to display the deployment in the right panel. Click **Details** in the panel. Wait, for several minutes if needed, for the **Details** page to show one pod deployed.



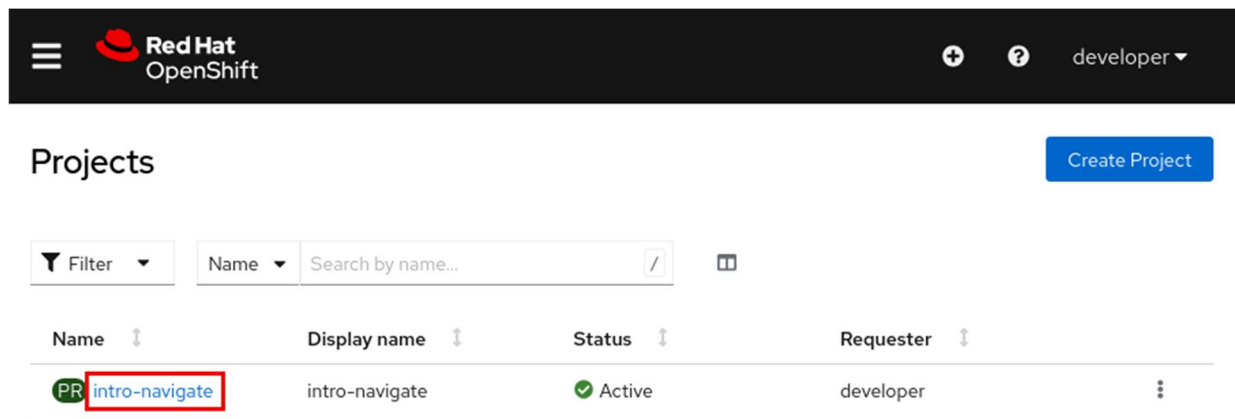
2. Select the **Actions** list to view the available controls for the deployment.



6. Switch into the Administrator perspective and inspect the deployment.
1. From the OpenShift web console, locate the left panel. If you do not see the left panel, then click the main menu icon at the upper left of the web console. Click **Developer** and then click **Administrator** to change to the Administrator perspective. The web console changes to the new perspective and exposes additional information through the sidebar.



2. Select the **Home** → **Projects** page from the left panel. Select the **intro-navigate** project to open the Project Details page.



3. This page includes a general overview of the project, such as the project status and resource usage details.

NOTE

You can ignore image stream import warnings because the image streams are not available in the classroom cluster.

7. View the intro-navigate pods and deployment.
1. From the OpenShift web console menu, go to **Workloads** → **Pods** to view the intro-navigate pods.



developer

Project: intro-navigate

Pods

Create Pod

Filter

Name

Search by name...

Name	Status	Ready	Restarts	
 intro-navigate-git-1-build	 Completed	0/1	0	
 intro-navigate-git-5b9f95f4f4-92gq5	 Running	1/1	0	

2. Go to **Workloads** → **Deployments** to view the list of deployments in the project.



developer

Project: intro-navigate

Deployments

Create Deployment

Name

Search by name...

Name	Status	
 intro-navigate-git	1 of 1 pods	

3. Click the **intro-navigate-git** link to view the deployment details.

Red Hat OpenShift

developer

Project: intro-navigate

Deployments > Deployment details

D intro-navigate-git

Actions

Details

Metrics

YAML

ReplicaSets

Pods

Environment

Events

Deployment details

1 Pod

Name

intro-navigate-git

Update strategy

RollingUpdate

Namespace

NS intro-navigate

Max unavailable

25% of 1 pod

8. View the intro-navigate service and route.
 1. Go to **Networking** → **Services** and click **intro-navigate-git** to view the details of the intro-navigate service.

Red Hat OpenShift

developer

Project: intro-navigate

Services

Create Service

Name

Search by name...

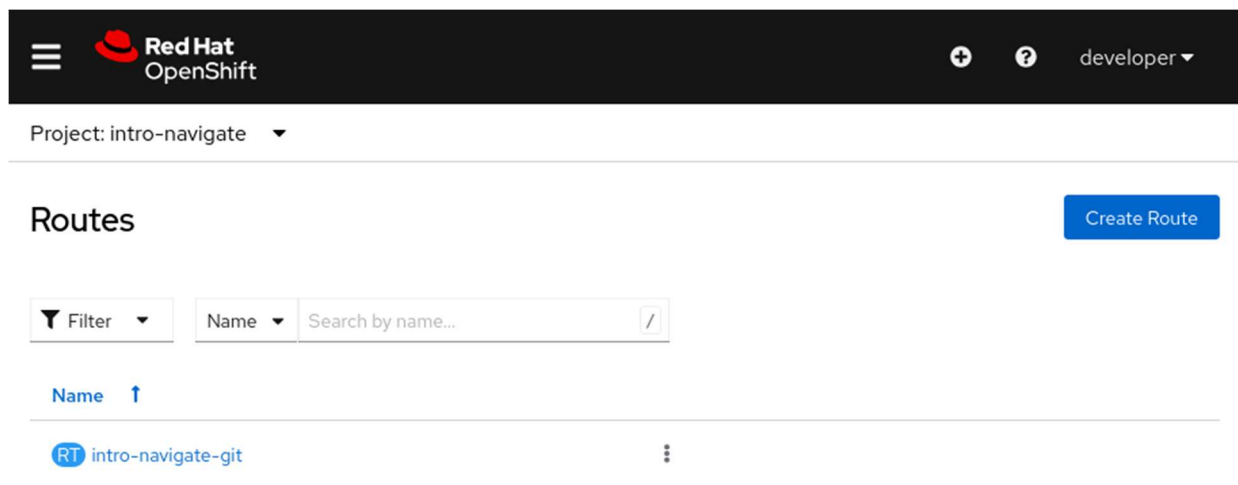
/

Name

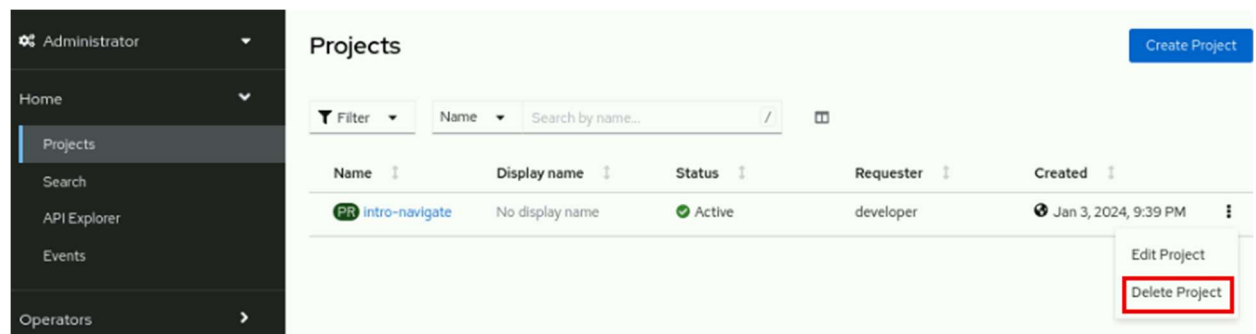
↑

S intro-navigate-git

2. Go to **Networking** → **Routes** and click **intro-navigate-git** to view the details of the intro-navigate route.



9. Delete the project and log out of the web console.
1. From the OpenShift web console menu, go to **Home** → **Projects**. Select **Delete Project** from the context menu for the intro-navigate project.



2. Enter the project name in the text field and then select **Delete**.

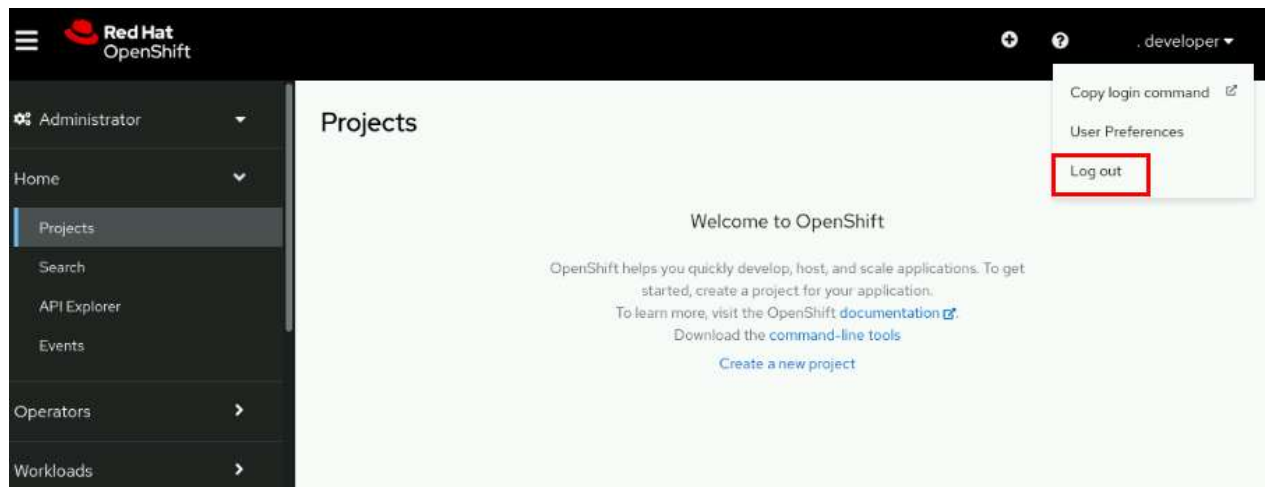
⚠ Delete Project?

This action cannot be undone. It will destroy all pods, services and other objects in the namespace **intro-navigate**.

Confirm deletion by typing **intro-navigate** below:

CancelDelete

3. Log out of the web console. From the OpenShift web console right panel, click **developer** and then select **Log out** from the account menu.

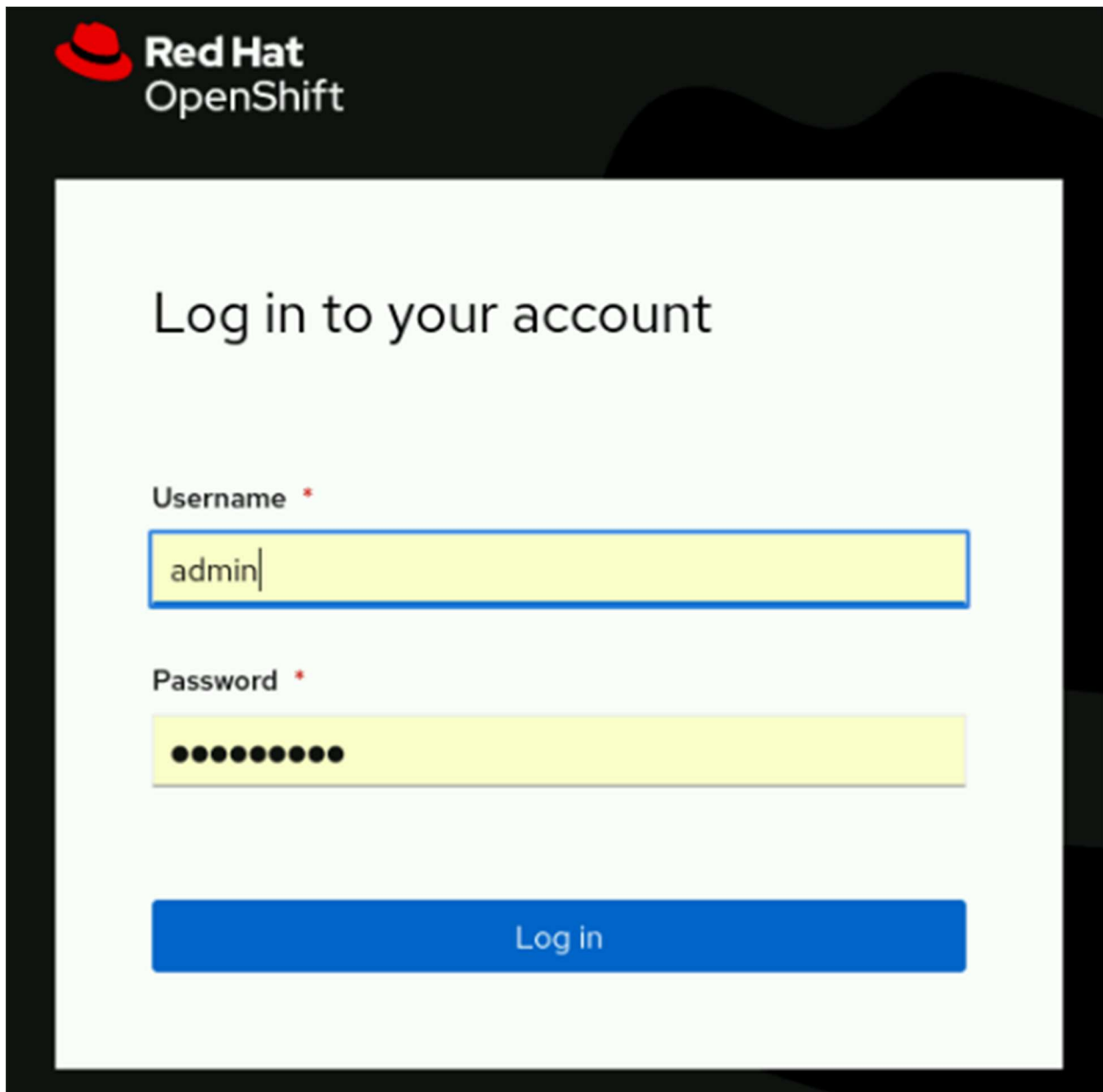


10. Log in to the OpenShift web console as the `admin` user to inspect additional cluster details.

NOTE

When you use a cluster administrator account, you can browse the cluster components, but do not alter or remove any components.

1. Log in to the web console. Select **Red Hat Identity Management** and then enter the admin username and the redhatocp password.

The image shows the Red Hat OpenShift login page. At the top left is the Red Hat logo (a red fedora) and the text "Red Hat OpenShift". The main heading is "Log in to your account". Below this are two input fields. The first is labeled "Username *" and contains the text "admin". The second is labeled "Password *" and contains ten black dots. Below the password field is a blue button with the text "Log in".

Red Hat OpenShift

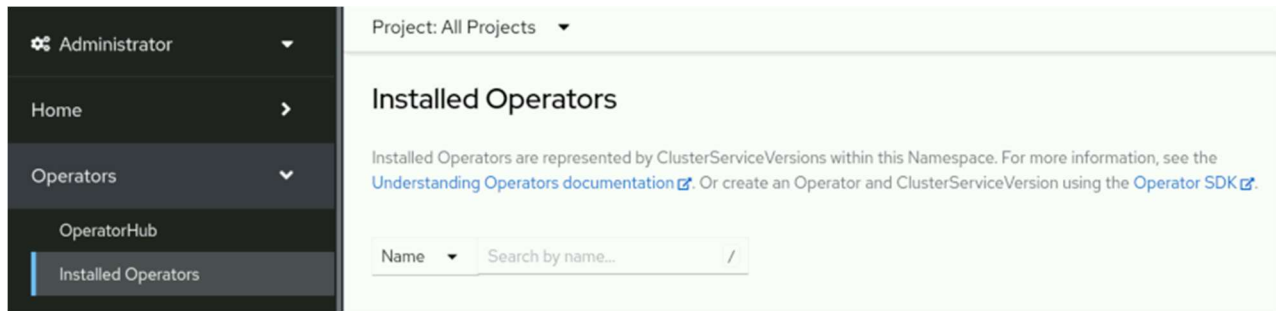
Log in to your account

Username *

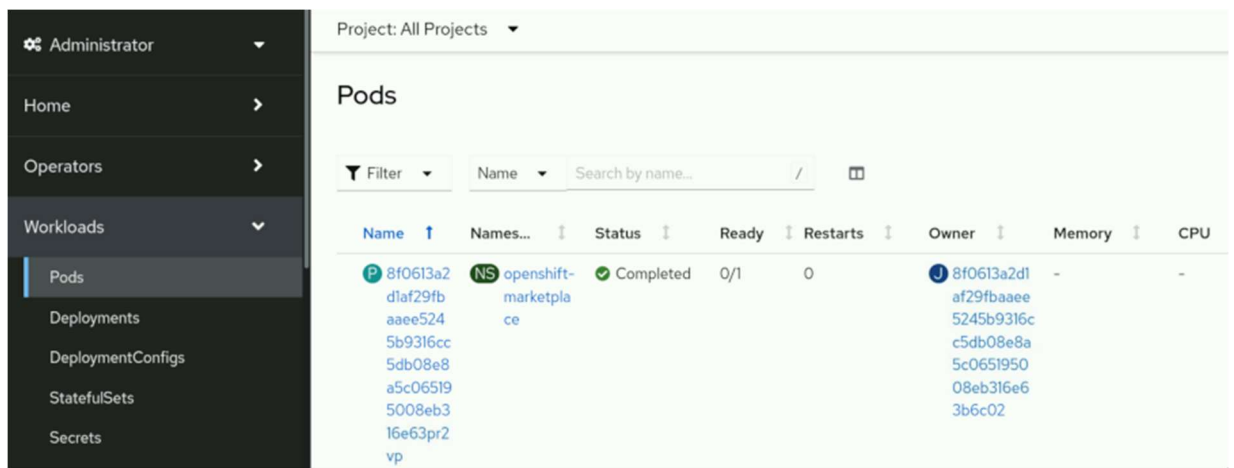
Password *

Log in

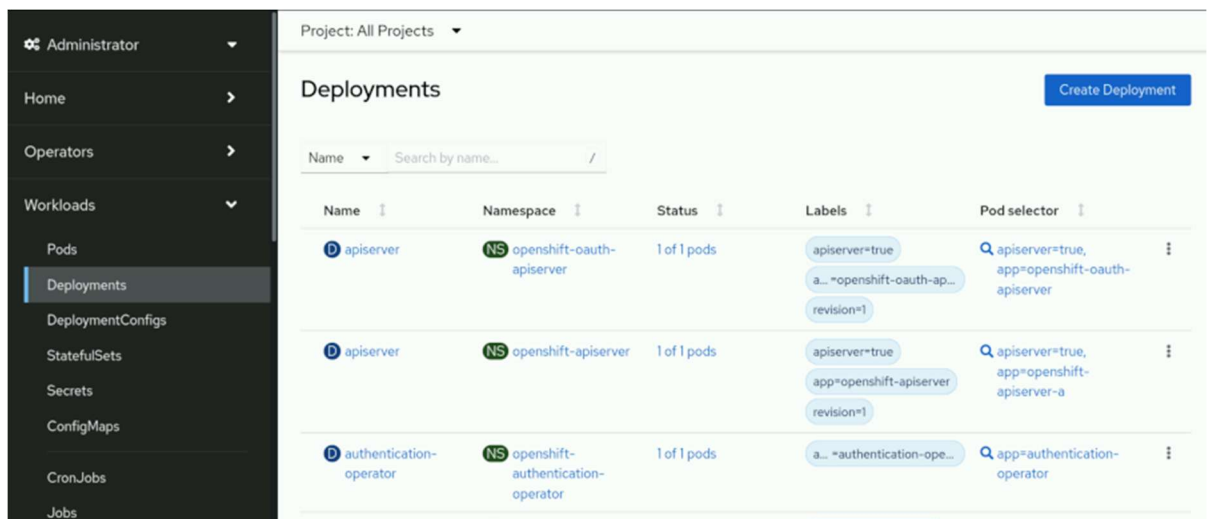
2. From the OpenShift web console menu, go to **Operators** → **Installed Operators**. Each operator provides a specific function for the cluster. Select an individual operator to display its details.



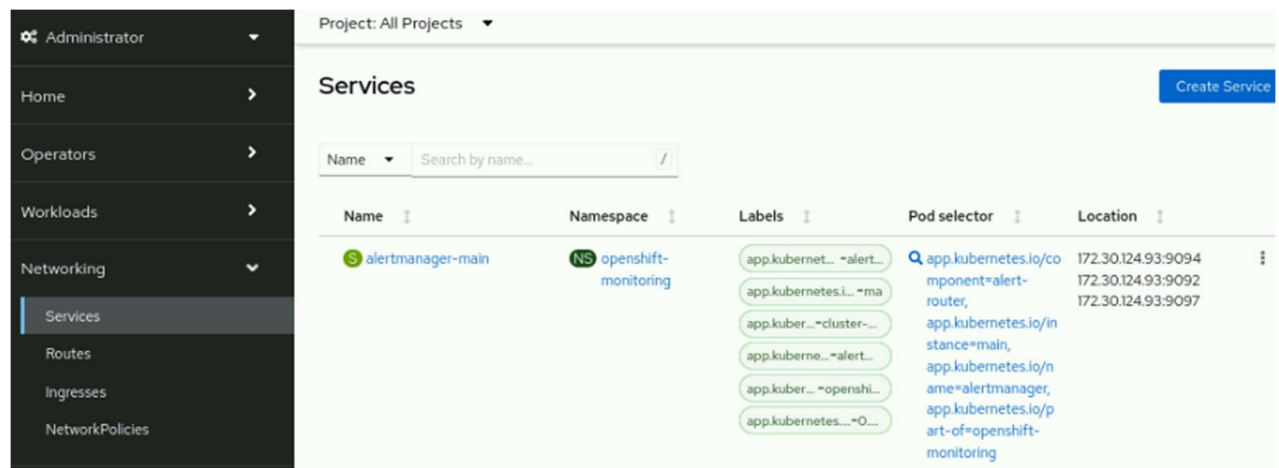
- Go to **Workloads** → **Pods** to view the list of all pods in the cluster. The search bar at the top can narrow down the list of pods. Select an individual pod to display its details.



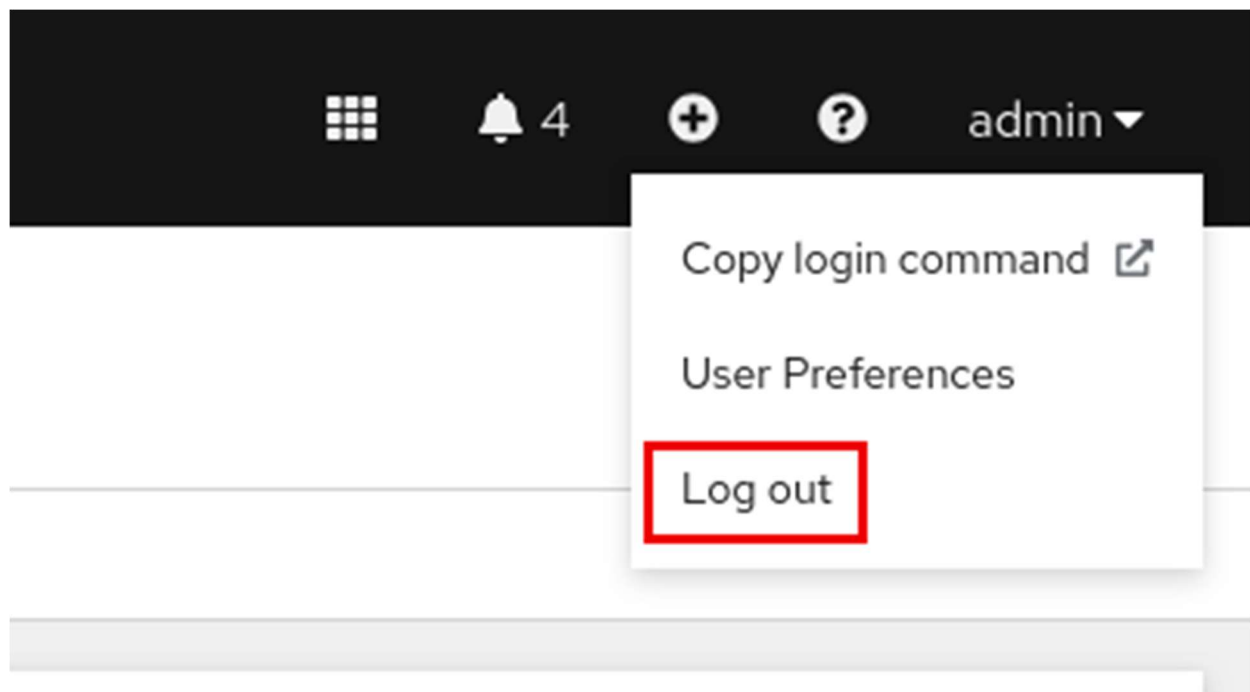
- Go to **Workloads** → **Deployments** to view the list of all deployments in the cluster. Select an individual deployment to display its details.



5. Go to **Networking** → **Services** to view the list of all services in the cluster. Select an individual service to display its details.



11. Log out of the web console.
1. Log out of the web console. From the OpenShift web console right panel, click **admin** and select **Log out** from the account menu.



Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish intro-navigate
```

Monitor an OpenShift Cluster

Objectives

- Navigate the Events, Compute, and Observe panels of the OpenShift web console to assess the overall state of a cluster.

Overview of Nodes, Machines, and Machine Configurations

In Kubernetes, a *node* is any single system in the cluster where pods can run. These systems are any of the bare metal, virtual, or cloud computers that are members of the cluster. Nodes run the necessary services to communicate within the cluster, and receive control plane operational requests. When you deploy a pod, an available node is tasked with satisfying the request.

Whereas the *node* and *machine* terms are often interchangeable, Red Hat OpenShift Container Platform (RHOCP) uses the *machine* term more specifically. In OpenShift, a *machine* is the resource that describes a cluster node. Using a machine resource is particularly valuable when using public cloud providers to provision infrastructure.

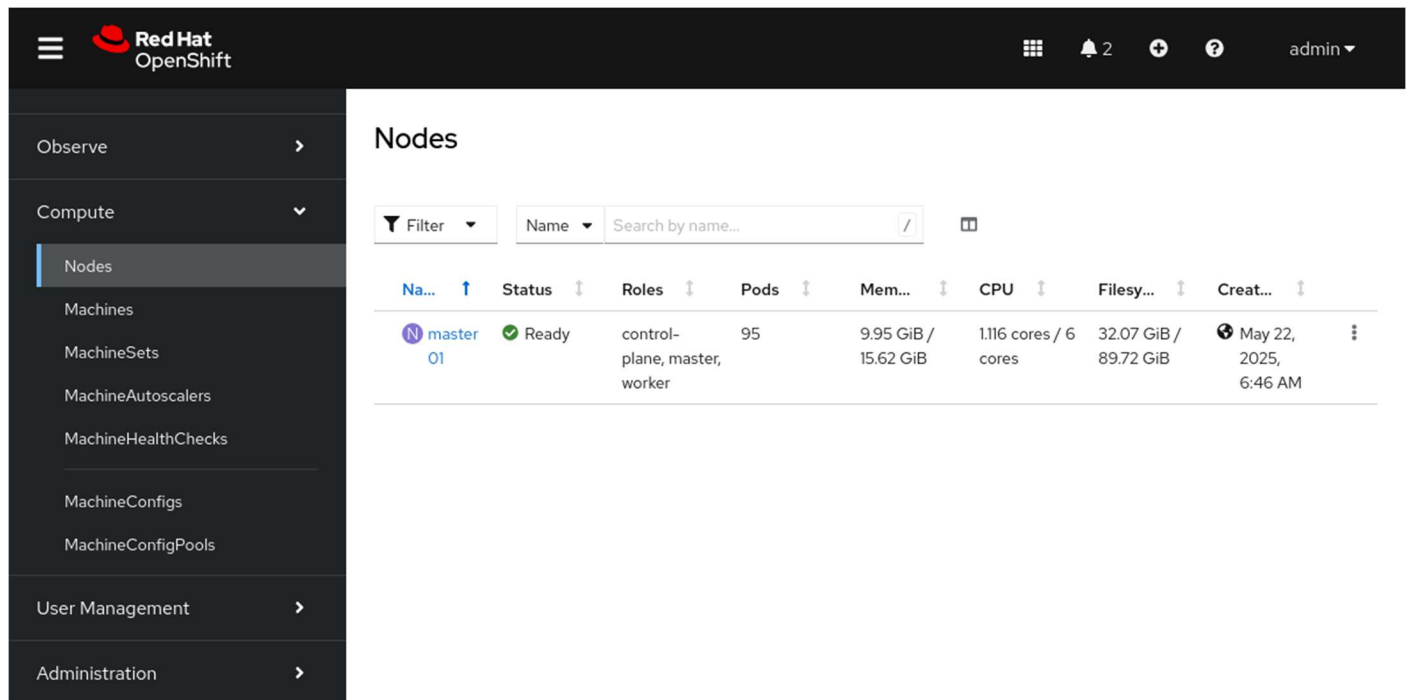
A `MachineConfig` resource defines the initial state and any changes to files, services, operating system updates, and critical OpenShift service versions for the `kubelet` and `cri-o` services. OpenShift relies on the Machine Config Operator (MCO) to maintain the operating systems and configuration of the cluster machines. The MCO is a cluster-level operator that ensures the correct configuration of each machine. This operator also performs routine administrative tasks, such as system updates. This operator uses the machine definitions in a `MachineConfig` resource to continually validate and remediate the state of cluster machines to the intended state. After a `MachineConfig` change, the MCO orchestrates the execution of the changes for all affected nodes.

NOTE

The orchestration of MachineConfig changes through the MCO is prioritized alphabetically by zone, by using the `topology.kubernetes.io/zone` node label.

Identifying Errors from Nodes

Administrators routinely view the logs and connect to the nodes in the cluster by using a terminal. This technique is necessary to manage a cluster and to remediate issues that arise. From the web console, go to **Compute** → **Nodes** to view the list of all nodes in the cluster.



The screenshot shows the Red Hat OpenShift web console interface. On the left is a sidebar with navigation options: Observe, Compute (expanded), Nodes (selected), Machines, MachineSets, MachineAutoscalers, MachineHealthChecks, MachineConfigs, MachineConfigPools, User Management, and Administration. The main panel is titled 'Nodes' and contains a table of cluster nodes. The table has columns for Name, Status, Roles, Pods, Memory, CPU, Filesystem, and Creation time. One node is listed: 'master-01' with a status of 'Ready' and roles of 'control-plane, master, worker'.

Name	Status	Roles	Pods	Mem...	CPU	Filesy...	Creat...
master-01	Ready	control-plane, master, worker	95	9.95 GiB / 15.62 GiB	1.116 cores / 6 cores	32.07 GiB / 89.72 GiB	May 22, 2025, 6:46 AM

Figure 1.40: Node list in the web console

Click a node's name to go to the overview page for the node. On the node overview page, you can view the node logs or connect to the node by using the terminal.

Red Hat

OpenShift

2

admin

Nodes > Node details

N master01

Ready

Actions

Overview

Details

YAML

Pods

Logs

Events

Terminal

The log is abridged due to length.

To view unabridged log content, [open the raw file in another window](#).

journal

Filter by unit

Q

Search

☐ Wrap lines

1	Jun 12 21:30:25.929854 master01 crio[2795]: time="2025-06-12 21:30:25.929334020Z" level=info msg=
2	Jun 12 21:30:25.929854 master01 crio[2795]: time="2025-06-12 21:30:25.929522209Z" level=info msg=
3	Jun 12 21:30:25.941898 master01 crio[2795]: time="2025-06-12 21:30:25.935341939Z" level=info msg=
4	Jun 12 21:30:25.941898 master01 crio[2795]: time="2025-06-12 21:30:25.935395625Z" level=info msg=
5	Jun 12 21:30:25.941898 master01 crio[2795]: time="2025-06-12 21:30:25.935542827Z" level=info msg=
6	Jun 12 21:30:25.941898 master01 crio[2795]: time="2025-06-12 21:30:25.935625467Z" level=info msg=
7	Jun 12 21:30:25.949972 master01 crio[2795]: time="2025-06-12 21:30:25.949917987Z" level=info msg=
8	Jun 12 21:30:25.957243 master01 kubenswrapper[2859]: I0612 21:30:25.955338 2859 kubelet.go:2453]
9	Jun 12 21:30:25.957243 master01 kubenswrapper[2859]: I0612 21:30:25.955383 2859 kubelet.go:2542]
10	Jun 12 21:30:25.965636 master01 kubenswrapper[2859]: I0612 21:30:25.965584 2859 kubelet.go:2453]
11	Jun 12 21:30:25.965810 master01 kubenswrapper[2859]: I0612 21:30:25.965651 2859 kubelet.go:2453]
12	Jun 12 21:30:25.971980 master01 kubenswrapper[2859]: I0612 21:30:25.971939 2859 kubelet.go:2453]

Figure 1.41: Logs page in the web console

From the previous page in the web console, view the node logs and investigate the system information to aid troubleshooting and remediation for node issues.

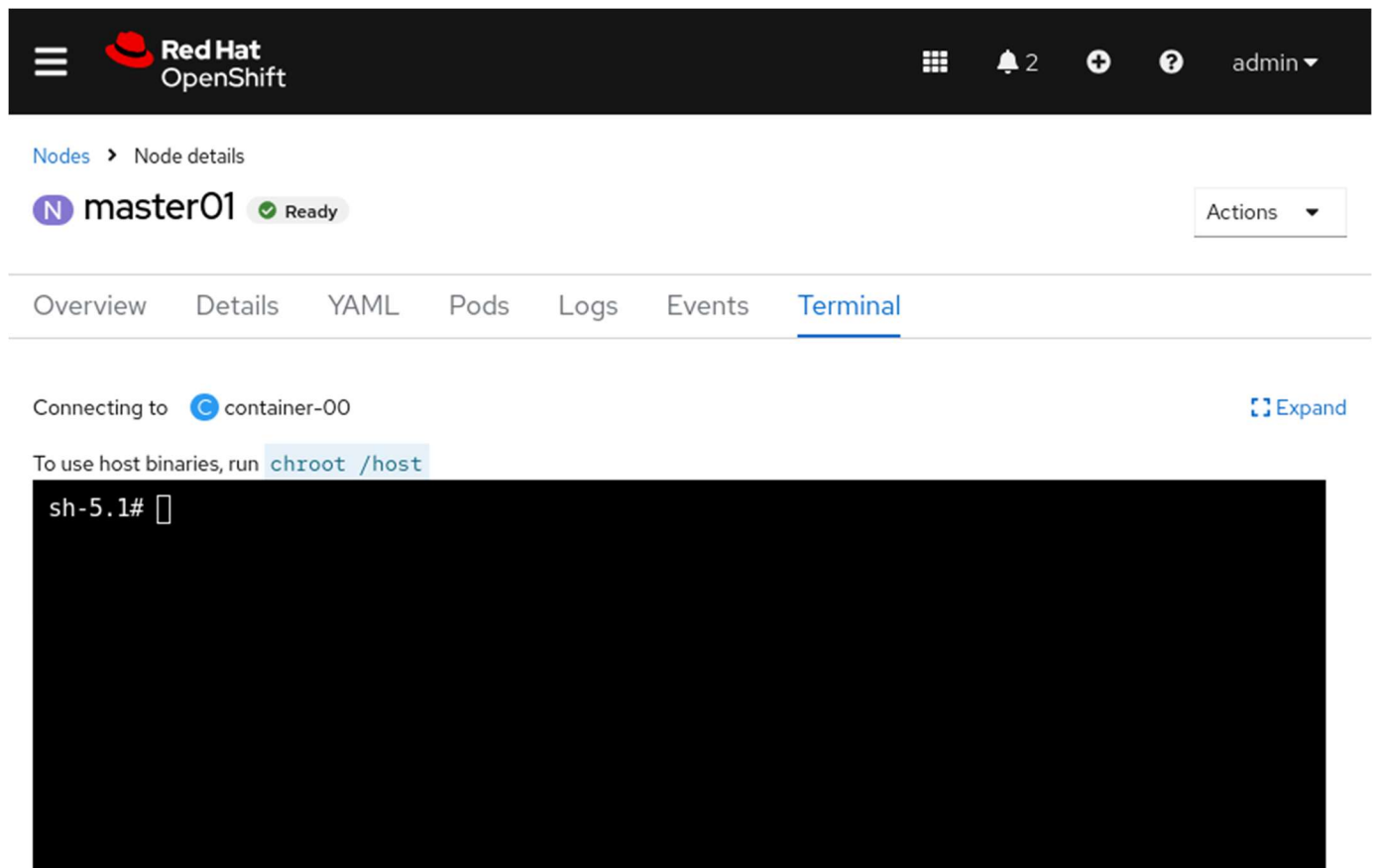


Figure 1.42: Terminal shell in the web console

The preceding page shows the web console terminal that is connected to the cluster node. From this tab, you can access the debug pod and use the commands from the host binaries to view the status of the node's services. An OpenShift node debug pod is an interface to a container that runs on the node.

Although making changes directly on the cluster node from the terminal is not recommended, it is common practice to connect to the cluster node for diagnostic investigation and remediation. From this terminal, you can use the same binaries that are available within the cluster node itself.

Additionally, the tabs on the node overview page show metrics, events, and the node's YAML definition file.

Accessing Pod Logs

Administrators often peruse pod logs to assess the health of a deployed pod or to troubleshoot pod deployment issues. Go to the **Workloads** → **Pods** page to view the list of all pods in the cluster.

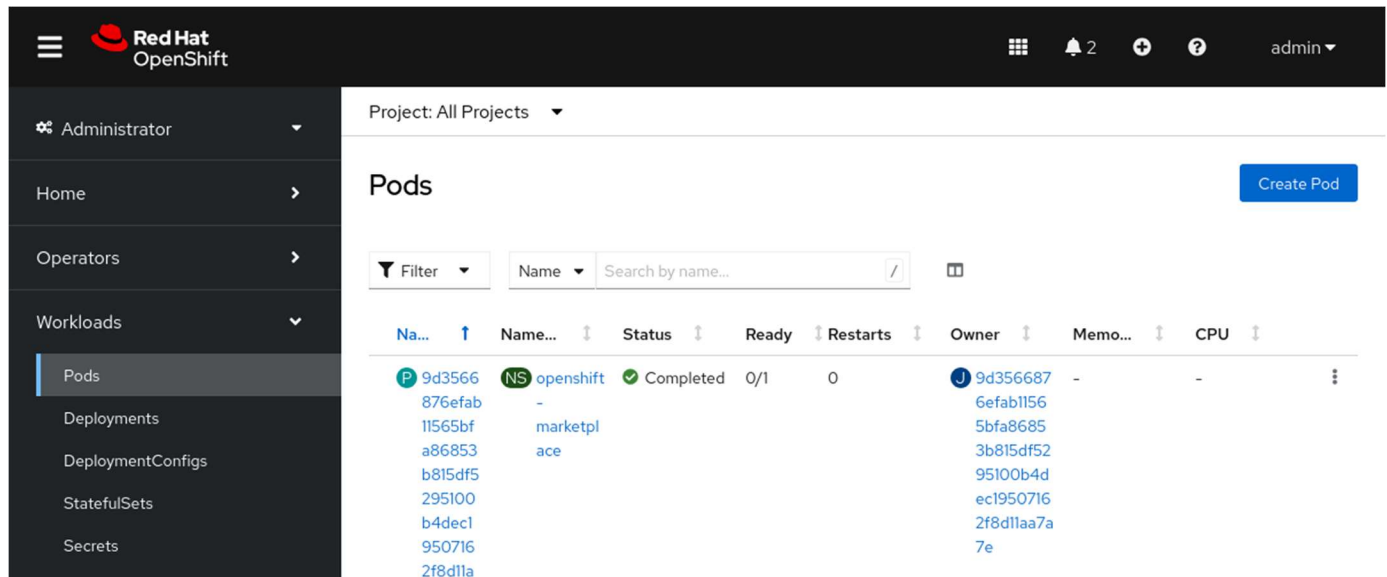


Figure 1.43: Pods page in the web console

You can filter and order pods by project and by other fields. To view the pod details page, click a pod name in the list.

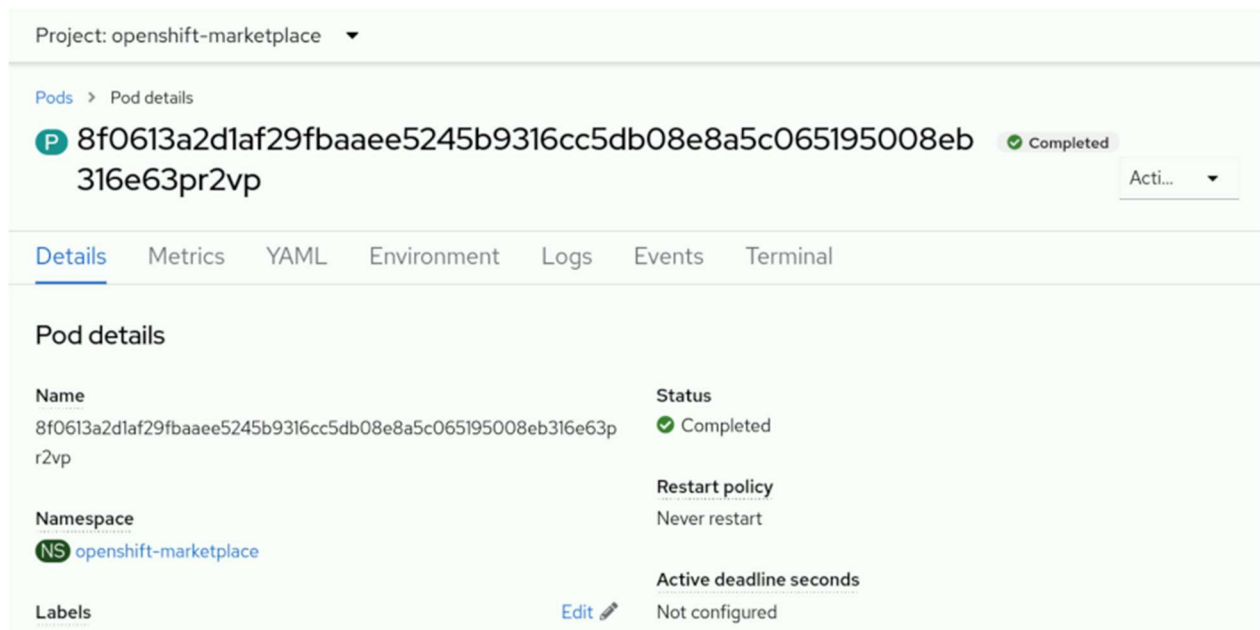


Figure 1.44: Pod details page

The pod details page contains links to pod metrics, environment variables, logs, events, a terminal, and the pod's YAML definition. The pod logs are available on the **Pods** → **Logs** page and provide information about the pod status.

The **Pods** → **Terminal** page opens a shell connection to the pod for inspection and issue remediation. Although it is not recommended to alter a running pod, the terminal is useful for diagnosing and remediating pod issues. To fix a pod, update the pod configuration to reflect the necessary changes, and redeploy the pod.

Red Hat OpenShift Container Platform Metrics and Alerts

In an RHOCP cluster, HTTP service endpoints provide data metrics that are collected to provide information for monitoring cluster and application performance. These metrics are authored at the application level for each service by using the client libraries that are provided by Prometheus, an open source monitoring and alerting toolkit. Metrics data is available from the service `/metrics` endpoint. You can use the data for creating monitors to alert based on degradation of the service. Monitors are processes that continuously assess the value for a specific metric and provide alerts that are based on a predefined condition, to signal a degradation in the service or a performance issue. Authoring a `ServiceMonitor` resource defines how a specific service uses the metrics to define a monitor and the alerting values. The same approach is available for monitoring pods by defining a `PodMonitor` resource that uses the metrics that are gathered from the pod.

Depending on the monitor definitions, alerting is then available based on the metric that is polled and the defined success criteria. The monitor continuously compares the gathered metric, and creates an alert when the success criteria are no longer met. As an example, a web service monitor polls on the listening port, port 80, and alerts only if the response from that port becomes invalid.

From the web console, go to **Observe** → **Metrics** to visualize gathered metrics by using a Grafana-based data query utility. On this page, users can submit queries to build data graphs and dashboards, which administrators can view to gather valuable statistics for the cluster and applications.

For configured monitors, visit **Observe** → **Alerting** to view firing alerts, and filter on the alert severity to view those alerts that need remediation. Alerting data is a key component to help administrators to deliver cluster and application accessibility and functions.

Kubernetes Events

Administrators are typically familiar with the contents of log files for services, whereas logs tend to be highly detailed and granular. *Events* provide a high-level abstraction to log files and to provide information about more significant changes. Events are useful in understanding the performance and behavior of the cluster, nodes, projects, or pods, at a glance. Events provide details to understand general performance and to highlight meaningful issues. Logs provide a deeper level of detail for remediating specific issues.

The **Home** → **Events** page shows the events for all projects or for a specific project. You can further filter and search events.

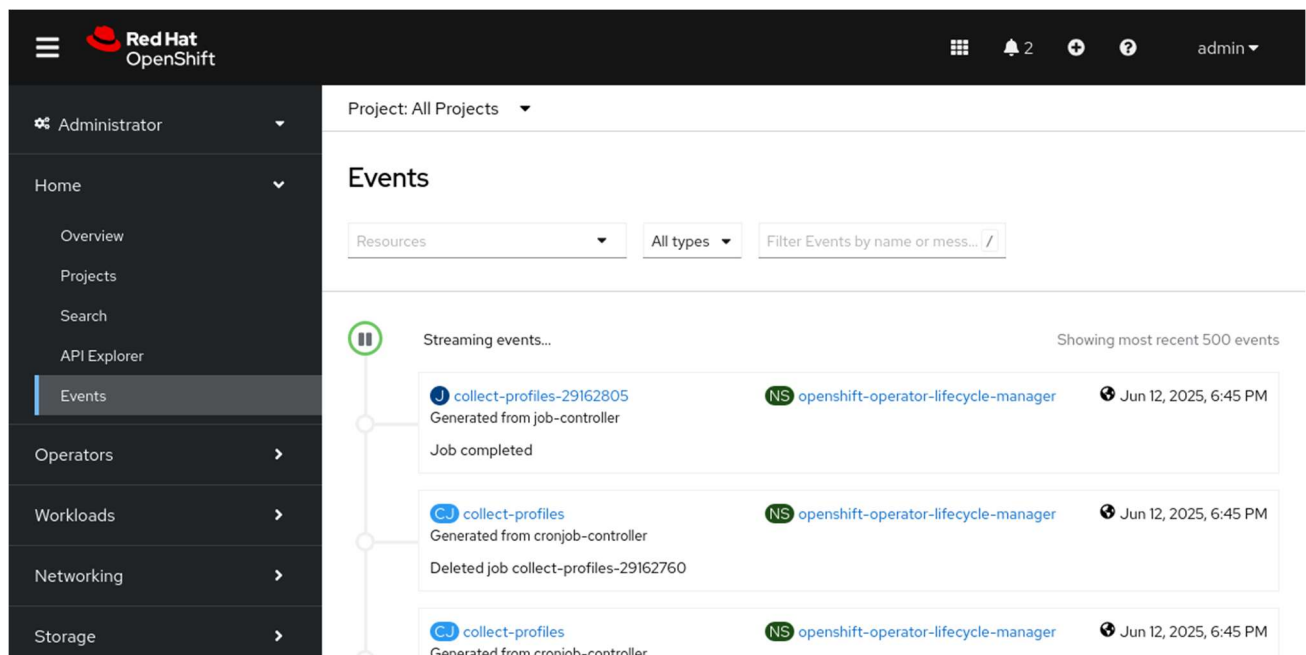


Figure 1.45: RHOCP Events console

Red Hat OpenShift Container Platform API Explorer

Starting from version 4, RHOCP includes the API Explorer feature, for users to view the catalog of Kubernetes resource types that are available within the cluster. By navigating to **Home** → **API Explorer**, you can view and explore the details for resources. Such details include the description, schema, and other metadata for the resource. This feature is helpful for all users, and especially for new administrators.

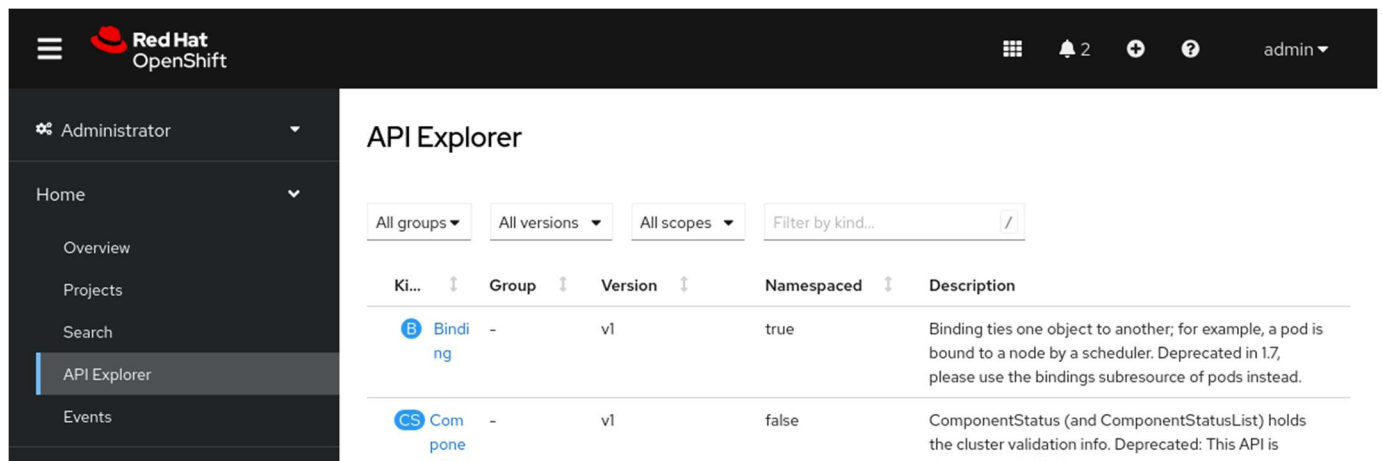


Figure 1.46: The RHOCP API Explorer

Guided Exercise: Monitor an OpenShift Cluster

Assess the overall status of an OpenShift cluster by using the web console, and identify projects and pods of core architectural components of Kubernetes and OpenShift.

Outcomes

- Explore and show the monitoring features and components.
- Explore the Overview page to inspect the cluster status.
- Use a terminal connection to the `master01` node to view the `crio` and `kubelet` services.
- Explore the Monitoring page, alert rule configurations, and the `etcd` service dashboard.
- Explore the events page, and filter events by resource name, type, and message.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the cluster is prepared for the exercise.

```
[student@workstation ~]$ lab start intro-monitor
```

Instructions

1. As the developer user, locate and then go to the Red Hat OpenShift web console.
1. Use the terminal to log in to the OpenShift cluster as the developer user with the developer password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \  
3. https://api.ocp4.example.com:6443
```

...output omitted...

4. Identify the URL for the OpenShift web console.

```
5. [student@workstation ~]$ oc whoami --show-console
```

https://console-openshift-console.apps.ocp4.example.com

6. Open a web browser and go to https://console-openshift-console.apps.ocp4.example.com. Either type the URL in a web browser, or right-click and select **Open Link** from the terminal.



Log in with

kube:admin

Red Hat Identity Management

2. Log in to the OpenShift web console as the `admin` user.
1. Click **Red Hat Identity Management** and log in as the `admin` user with the `redhatocp` password.

Log in to your account

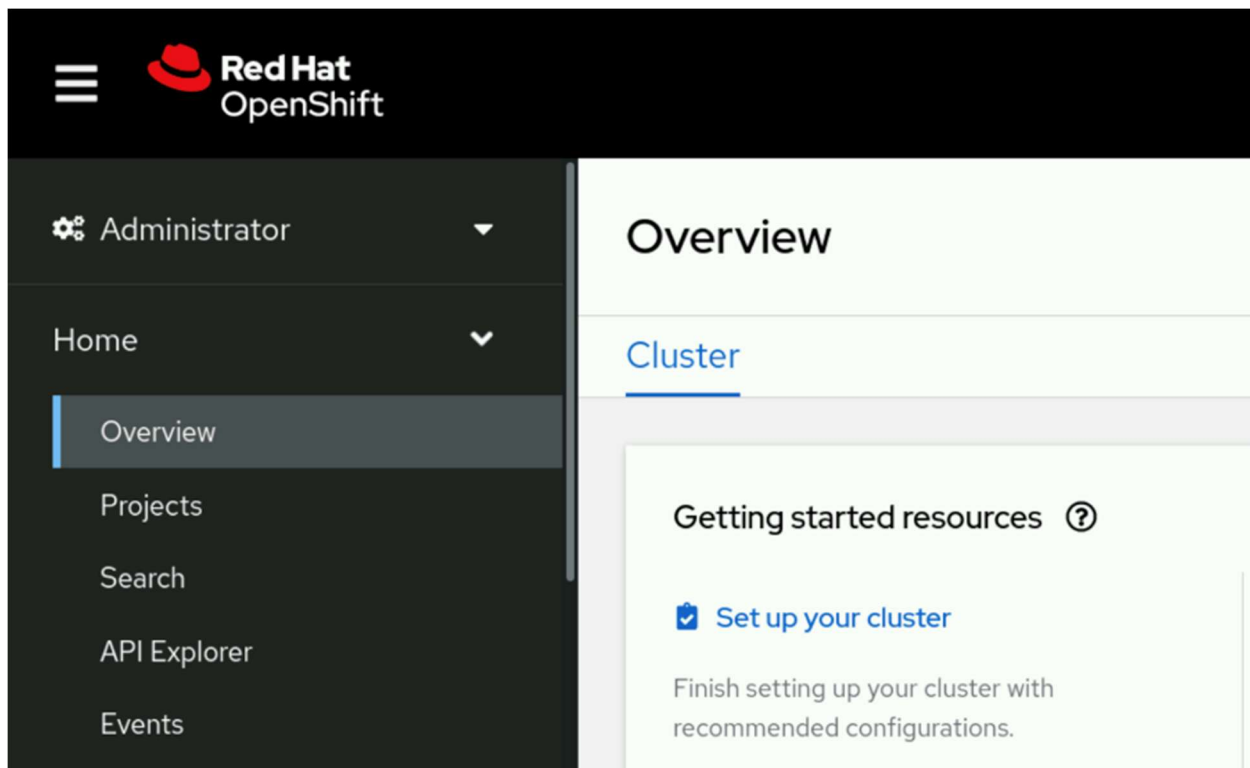
Username *

Password *

Log in

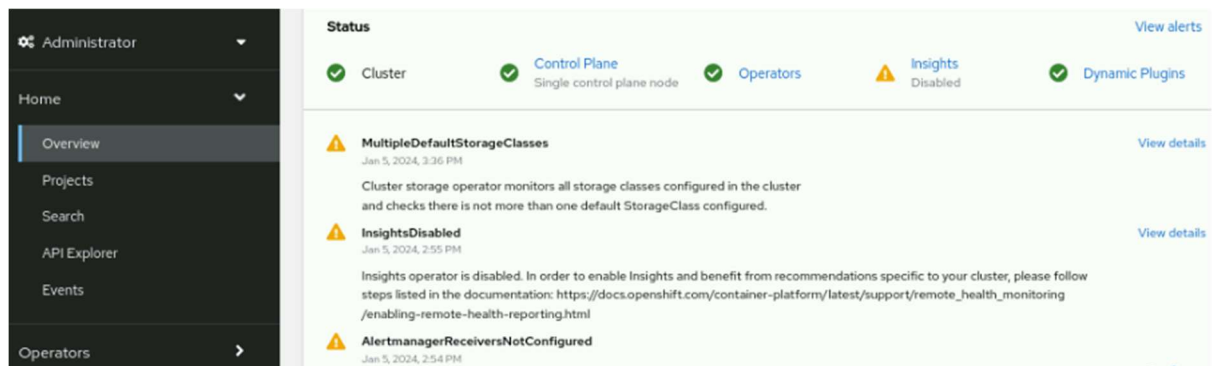
3. View the cluster health and overall status.
1. Review the **Cluster Overview** page.

If you do not see this page after a successful login, then locate the left panel from the OpenShift web console. If you do not see the left panel, then click the main menu icon at the upper left of the web console. Go to **Home** → **Overview** to view general cluster information.



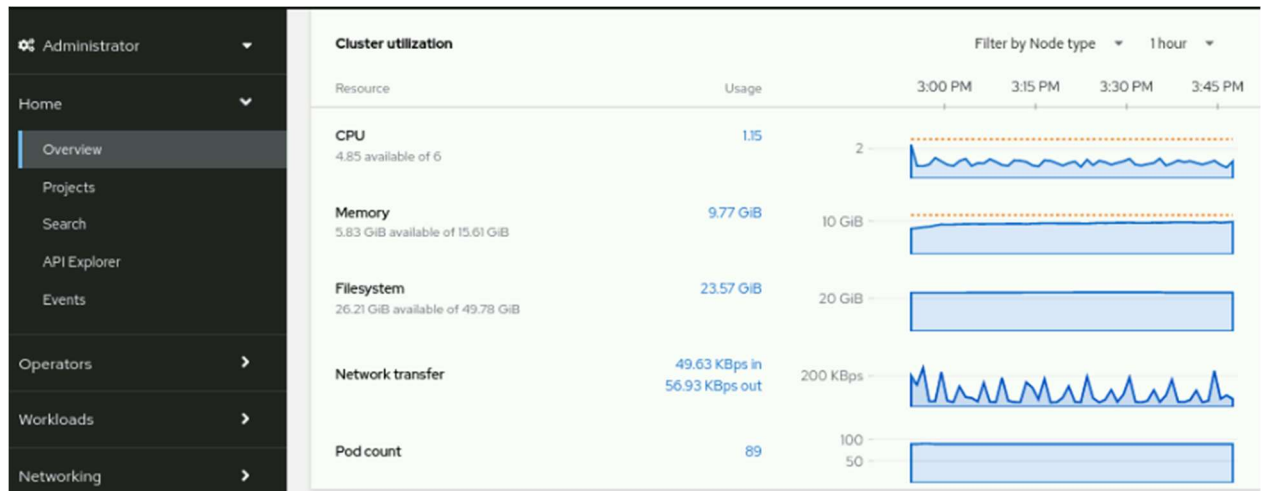
The **Overview** section contains links to helpful documentation and an initial cluster configuration walkthrough.

2. Scroll down to view the **Status** section, which provides a summary of cluster performance and health.



Many of the headings are links to sections with more detailed cluster information.

3. Continue scrolling to view the **Cluster utilization** section, which contains metrics and graphs that show resource consumption.



- Continue scrolling to view the **Details** section, including information such as the cluster API address, cluster ID, and Red Hat OpenShift version.

Details [View settings](#)

Cluster API address
https://api.ocp4.example.com:6443

Cluster ID
6c8c6eed-26ed-4911-9df9-b081404842c8
[OpenShift Cluster Manager](#)

Infrastructure provider
None

OpenShift version
4.18.6

Update channel
Not available

Control plane high availability
No (single control plane node)

- Scroll to the **Cluster Inventory** section, which contains links to the Nodes, Pods, StorageClasses, and PersistentVolumeClaim pages.

Cluster inventory

[1 Node](#)

[115 Pods](#)

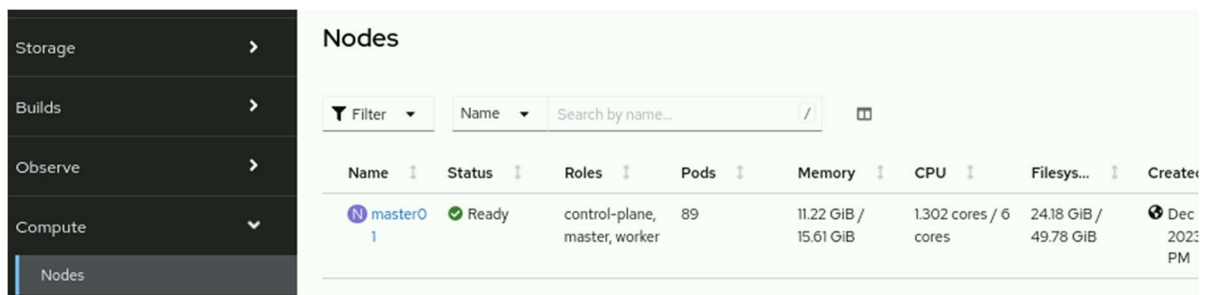
[2 StorageClasses](#)

[1 PersistentVolumeClaim](#)

- The last part of the page contains the **Activity** section, which lists ongoing activities and recent events for the cluster.



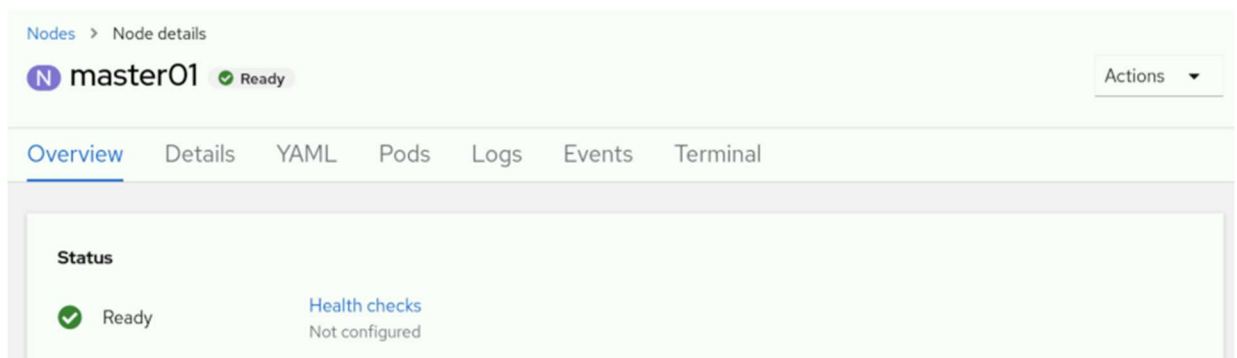
4. Use the OpenShift web console to access the terminal of a cluster node.
From the terminal, determine the status of the kubelet node agent service and the CRI-O container runtime interface service.
1. Go to **Compute** → **Nodes** to view the machine that provides the cluster resources.



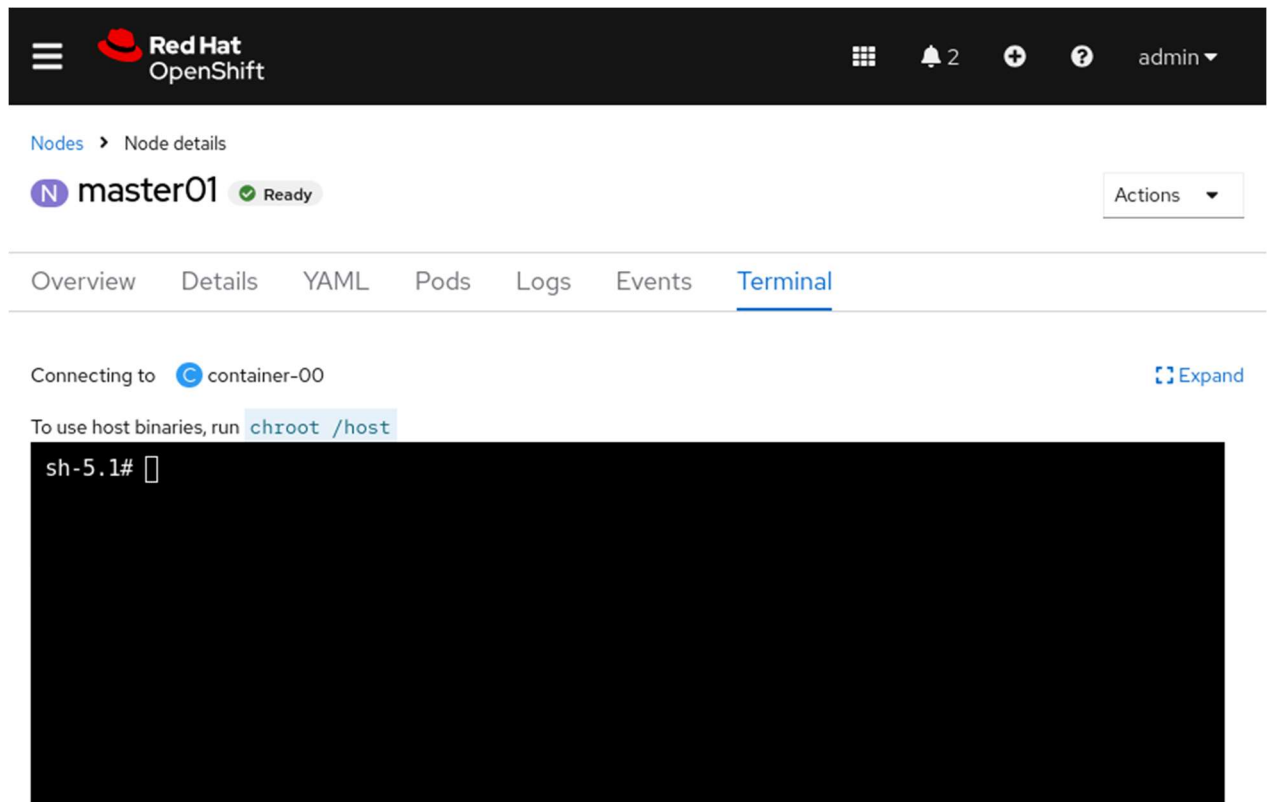
NOTE

The classroom cluster runs on a single node named `master01`, which serves as the control and data planes for the cluster, and is intended for training purposes. A production cluster uses multiple nodes to ensure stability and to provide a highly available architecture.

2. Click the **master01** link to view the details of the cluster node.



3. Click the **Terminal** tab to connect to a shell on the master01 node.



The screenshot shows the Red Hat OpenShift console interface. At the top is a dark header with the Red Hat logo, navigation icons, a notification bell with '2', a help icon, and a user profile 'admin'. Below the header, the breadcrumb 'Nodes > Node details' is visible. The main content area shows the 'master01' node with a 'Ready' status. A horizontal tab bar includes 'Overview', 'Details', 'YAML', 'Pods', 'Logs', 'Events', and 'Terminal', with 'Terminal' being the active tab. Below the tabs, it says 'Connecting to container-00' with an 'Expand' link. A hint states 'To use host binaries, run `chroot /host`'. The terminal window itself is black with the prompt 'sh-5.1#' and a cursor.

With the interactive shell on this page, you can run commands directly on the cluster node.

4. Run the `chroot /host` command to enable host binaries on the node.



 3 admin ▾

Nodes > Node details

N master01

✓ Ready

Actions ▾

OverviewDetailsYAMLPodsLogsEventsTerminal

Connecting to  container-00  Expand

To use host binaries, run `chroot /host`

```
sh-5.1# chroot /host
sh-5.1#
```

5. View the status of the kubelet node agent service by running the `systemctl status kubelet` command.



 3 admin ▾

Nodes > Node details

N master01

Ready

Actions ▾

OverviewDetailsYAMLPodsLogsEventsTerminal

Connecting to container-00

Expand

To use host binaries, run `chroot /host`

```
● kubelet.service - Kubernetes Kubelet
   Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/kubelet.service.d
            └─01-kubens.conf, 10-mco-default-madv.conf, 20-logging.conf, 20-nodenet.conf
   Active: active (running) since Thu 2025-06-12 21:28:17 UTC; 1h 41min ago
     Main PID: 2859 (kubelet)
        Tasks: 25 (limit: 101735)
       Memory: 728.6M
          CPU: 13min 32.123s
      CGroup: /system.slice/kubelet.service
              └─2859 /usr/bin/kubelet --config=/etc/kubernetes/kubelet.conf --bootstrap-kub>
```

lines 1-11

Press **q** to exit the command and to return to the terminal prompt.

6. View the status of the CRI-O container runtime interface service by running the `systemctl status crio` command.



 3 admin ▾

Nodes > Node details

N master01

Ready

Actions ▾

OverviewDetailsYAMLPodsLogsEventsTerminal

Connecting to container-00

Expand

To use host binaries, run `chroot /host`

```
● crio.service - Container Runtime Interface for OCI (CRI-O)
   Loaded: loaded (/usr/lib/systemd/system/crio.service; disabled; preset: disabled)
   Drop-In: /etc/systemd/system/crio.service.d
            └─01-kubens.conf, 05-mco-ordering.conf, 10-mco-default-madv.conf, 10-mco-prof
   Active: active (running) since Thu 2025-06-12 21:28:14 UTC; 1h 42min ago
     Docs: https://github.com/cri-o/cri-o
    Main PID: 2795 (crio)
      Tasks: 51
     Memory: 556.7M
        CPU: 2min 53.582s
    CGroup: /system.slice/crio.service

lines 1-11
```

Press **q** to exit the command and to return to the terminal prompt.

5. Inspect the cluster monitoring and alert rule configurations.
1. From the OpenShift web console menu, go to **Observe** → **Alerting** to view cluster alert information.

The screenshot shows the Red Hat OpenShift Alerting interface. The left sidebar contains navigation links: Workloads, Networking, Storage, Builds, Observe (expanded), Alerting (selected), Metrics, Dashboards, Targets, and Compute. The main panel is titled 'Alerting' and has tabs for Alerts, Silences, and Alerting rules. The 'Alerts' tab is active. It features a filter section with a 'Filter' dropdown, a 'Name' dropdown, and a search box. Below this, there are filter tags for 'Source' (Platform), 'Alert State' (Firing), and a 'Clear all filters' link. An 'Export as CSV' button is also present. The table below lists active alerts:

Name ↑	Severity ↓	State ↓	Source ↓
AL AlertmanagerReceiversNotConfigured Alerts are not configured to be sent to a notification system, meaning that you...	Warning	Firing Since Jun 12, 2025, 5:31 PM	Platform
AL HighOverallControlPlaneMemory Given three control plane nodes, the	Warning	Firing Since	Platform

2. Select the **Alerting rules** tab to view the various alert definitions.

The screenshot shows the Red Hat OpenShift Alerting interface with the 'Alerting rules' tab selected. The left sidebar is the same as in the previous image. The main panel is titled 'Alerting' and has tabs for Alerts, Silences, and Alerting rules. The 'Alerting rules' tab is active. It features the same filter section as the 'Alerts' tab. Below the filters, there is a table listing alerting rules:

Name ↑	Severity ↓	Alert state ↓	Source ↓
AR AlertmanagerClusterDown	Warning	-	Platform
AR AlertmanagerClusterFailedToSendAlerts	Warning	-	Platform
AR AlertmanagerConfigInconsistent	Warning	-	Platform
AR AlertmanagerFailedReload	Critical	-	Platform
AR AlertmanagerFailedToSendAlerts	Warning	-	Platform

3. Filter the alerting rules by name and search for the storageClasses term.

Alerting

Alerts Silences Alerting rules

 Filter ▾

Name ▾

storageClasses /

2 filters applied [Clear all filters](#)

Name ↑	Severity ↓	Alert state ↓	Source ↓
 MultipleDefaultStorageClasses	 Warning	 1 Pending	Platform

4. Select the warning alert that is labeled `MultipleDefaultStorageClasses` to view the details of the alerting rule. Inspect the **Description** and **Expression** definition for the rule.

Alerting

Alerts Silences Alerting rules

Filter ▾ Name ▾ storageClasses /

2 filters applied [Clear all filters](#)

Name ↑	Severity ↓	Alert state ↓	Source ↓
AR MultipleDefaultStorageClasses	 Warning	 1 Pending	Platform

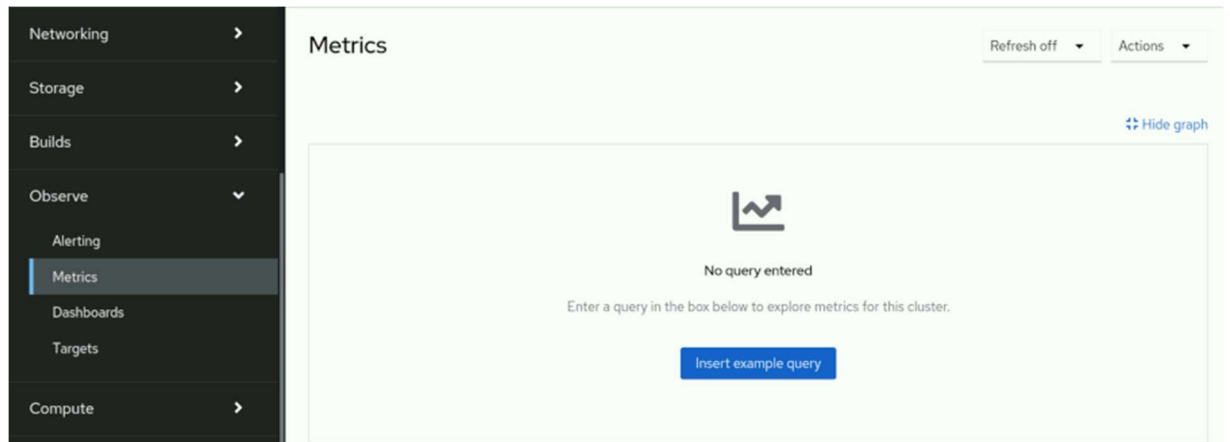
Alerting rules > Alerting rule details

AR MultipleDefaultStorageClasses 

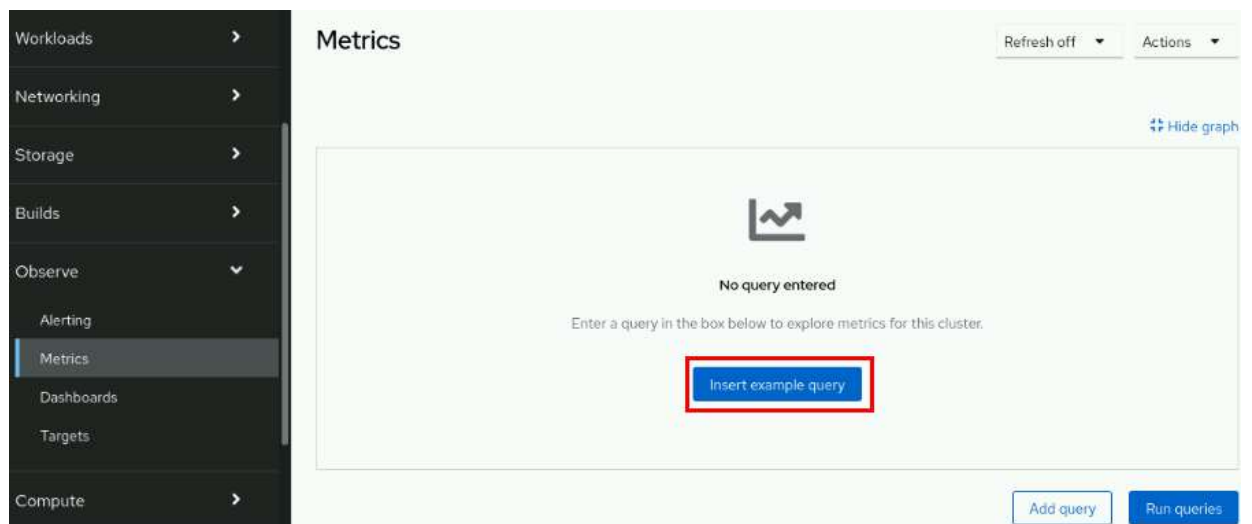
Alerting rule details

Name MultipleDefaultStorageClasses	Source Platform
Severity  Warning	For 10m
Description Cluster storage operator monitors all storage classes configured in the cluster and checks there is not more than one default StorageClass configured.	Expression <code>max_over_time(default_storage_class_count[5m]) > 1</code>
Summary More than one default StorageClass detected.	

6. Inspect cluster metrics and execute an example query.
1. Go to **Observe** → **Metrics** to open the cluster metrics utility.



2. Click **Insert example query** to populate the metrics graph with sample data.

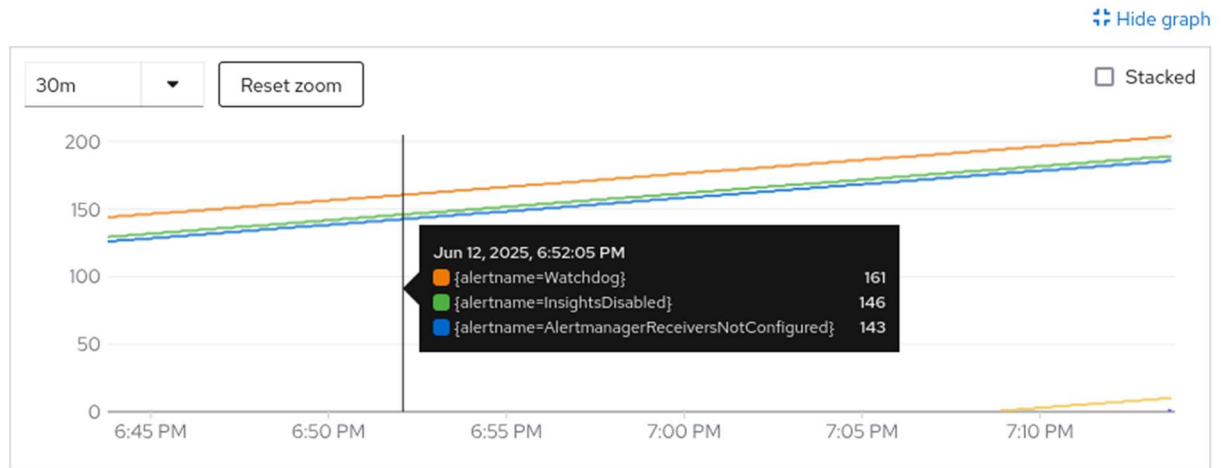


3. From the graph, hover over any point on the timeline to view the detailed data points.

Metrics

30 seconds ▾

Actions ▾



Queries

Select query ▾

Add query

Run queries

7. View the cluster events log from the web console.

1. Go to **Home** → **Events** to open the cluster events log.

Red Hat OpenShift

admin ▾

Administrator ▾

Home ▾

Overview

Projects

Search

API Explorer

Events

Operators >

Workloads >

Networking >

Storage >

Project: All Projects ▾

Events

Resources ▾

All types ▾

Filter Events by name or mess... /

Streaming events...

Showing most recent 500 events

machine-config-operator

Generated from machine-config-operator

NS openshift-machine-config-operator

Updated ConfigMap/kube-rbac-proxy -n openshift-machine-config-operator: cause by changes in data.config-file.yaml

Jun 12, 2025, 7:14 PM

machine-config-operator

Generated from machine-config-operator

NS openshift-machine-config-operator

Updated ConfigMap/kube-rbac-proxy -n openshift-machine-config-operator: cause by changes in data.config-file.yaml

Jun 12, 2025, 7:11 PM

machine-config-operator

Generated from machine-config-operator

NS openshift-machine-config-operator

Updated ConfigMap/kube-rbac-proxy -n openshift-machine-config-operator: cause by changes in data.config-file.yaml

Jun 12, 2025, 7:08 PM

NOTE

The event log updates every 15 minutes and can require additional time to populate entries.

2. Scroll down to view a chronologically ordered stream that contains cluster events.

  4 admin ▾		
Project: All Projects ▾		
 collect-profiles-29162835 Generated from job-controller Job completed	 openshift-operator-lifecycle-manager	 Jun 12, 2025, 7:15 PM
 collect-profiles Generated from cronjob-controller Deleted job collect-profiles-29162790	 openshift-operator-lifecycle-manager	 Jun 12, 2025, 7:15 PM
 collect-profiles Generated from cronjob-controller Saw completed job: collect-profiles-29162835, condition: Complete	 openshift-operator-lifecycle-manager	 Jun 12, 2025, 7:15 PM 2 times in the last 0 minutes
 collect-profiles-29162835-hqkp7 Generated from kubelet on master01 Created container: collect-profiles	 openshift-operator-lifecycle-manager	 Jun 12, 2025, 7:15 PM
 collect-profiles-29162835-hqkp7 Generated from kubelet on master01	 openshift-operator-lifecycle-manager	 Jun 12, 2025, 7:15 PM

NOTE

Select an event to open the `Details` page of the related resource.

8. Filter the events by resource name, type, or message.
1. From the **Resources** drop-down, use the search bar to filter for the pod term, and select the box labeled **Pod** to display events that relate to that resource.

Red Hat OpenShift

Project: All Projects

Events

pod x

All types

Filter Events by name or mess... /

- ☐ HPA HorizontalPodAutoscaler
- ☒ P Pod
- ☐ PDB PodDisruptionBudget
- ☐ PM PodMetrics

Showing 342 events

- Continue to refine the filter by selecting **Normal** from the types drop-down.

Red Hat OpenShift

Project: All Projects

Events

pod x

All types

Filter Events by name or mess... /

Resource P Pod x

Streaming events...

Normal

Warning

Showing 342 events

- Filter the results by using the **Message** text field. Enter the started container text to retrieve the matching events.

The screenshot shows the Red Hat OpenShift web console interface. At the top, there's a navigation bar with the Red Hat logo, a hamburger menu, and user information (admin). Below the navigation bar, the 'Project: All Projects' dropdown is visible. The main section is titled 'Events'. It features a filter bar with 'pod' selected in the resource dropdown, 'Normal' in the severity dropdown, and 'started container' in the event type dropdown (highlighted with a red box). Below the filter bar, there's a 'Resource' section showing 'Pod' with a close button. The events list shows two events, both with a green 'P' icon, generated from kubelet on master01, and both showing 'Started container collect-profiles'. The events are timestamped 'Jun 12, 2025, 7:15 PM' and 'Jun 12, 2025, 7:00 PM'. A 'Streaming events...' button is visible on the left, and 'Showing 48 events' is on the right.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish intro-monitor
```

Lab: Introduction to Kubernetes and OpenShift

Find essential information about your OpenShift cluster by navigating its web console.

Outcomes

Navigate the Red Hat OpenShift Container Platform web console to find various information items and configuration details.

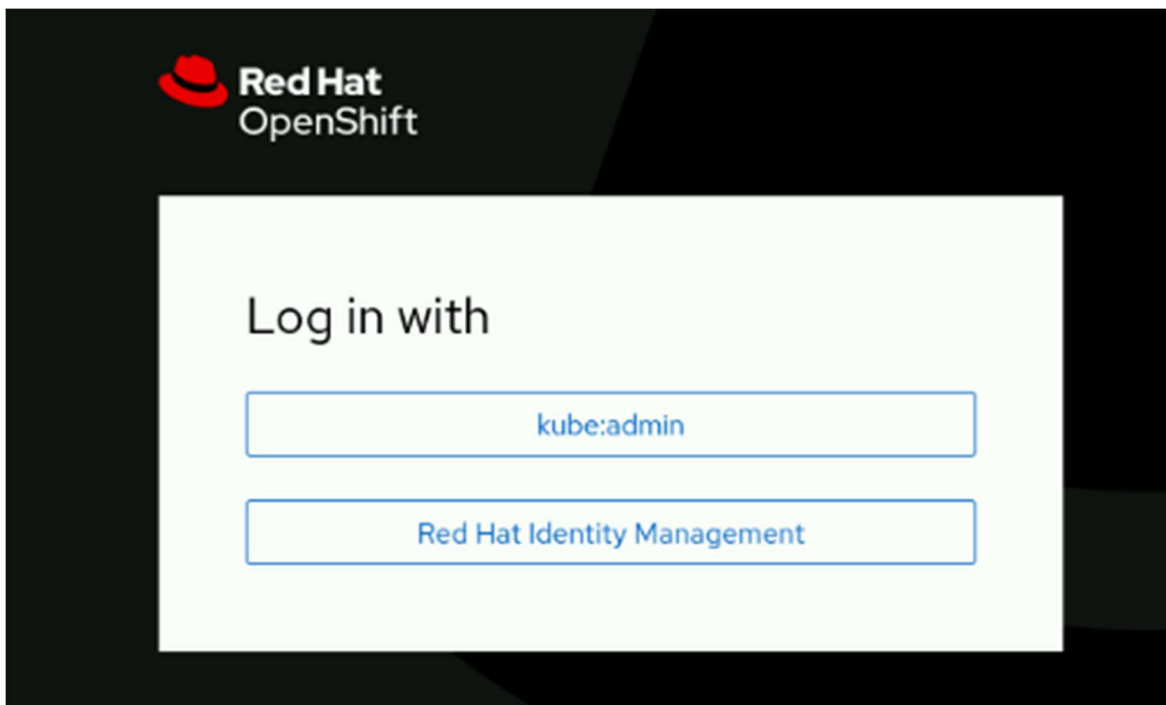
As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the Red Hat OpenShift Container Platform is deployed and ready for the lab.

```
[student@workstation ~]$ lab start intro-review
```

Instructions

1. Log in to the Red Hat OpenShift Container Platform web console, with Red Hat Identity Management as the admin user with the redhatocp password, and review the answers for the quiz in the section that follows.
1. Use a browser to view the login page at the <https://console-openshift-console.apps.ocp4.example.com> address.



2. Click Red Hat Identity Management, and supply the admin username and the redhatocp password, and then click Log in to access the home page.

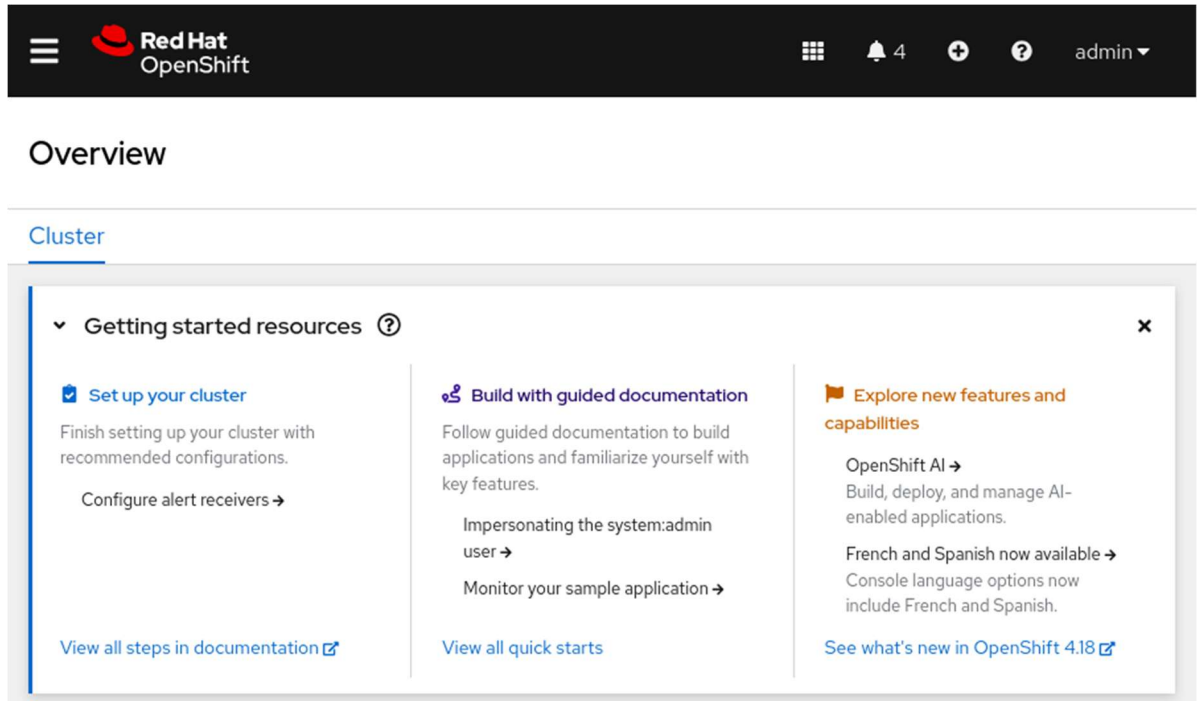
Log in to your account

Username *

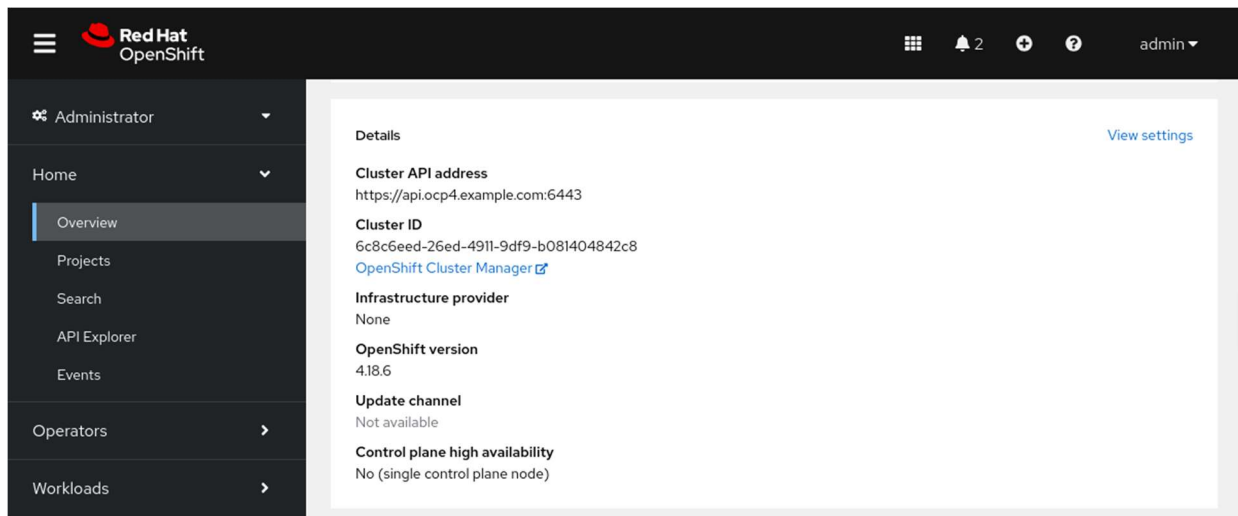
Password *

Log in

2. View the cluster version on the Overview page for the cluster.
1. From the **Home** → **Overview** page, scroll down to view the cluster details.



2. Locate the OpenShift version in the Details section.



3. View the available alert severity types within the filters on the Alerting page.

1. Go to the **Observe** → **Alerting** page.

Red Hat
OpenShift

3

admin

Alerting

Alerts Silences Alerting rules

Filter

Name

Search by name...

Export as CSV

Name ↑	Severity ↑	State ↑	Source ↑
AL AlertmanagerReceiversNotConfigured Alerts are not configured to be sent to a notification system, meaning that you ma...	Warning	Firing Since Jun 16, 2025, 1:42 PM	Platform
AL HighOverallControlPlaneMemory Given three control plane nodes, the overall memory utilization may only be...	Warning	Pending Since Jun 16, 2025, 1:53 PM	Platform

- Click the Filter drop-down to view the available severity options.

Red Hat
OpenShift

3

admin

Alerting

Alerts Silences Alerting rules

Filter

Name

Search by name...

Alert State

☐ Firing 4

☐ Pending 1

☐ Silenced 0

Severity

☐ Critical 1

☐ Warning 2

☐ Info 1

☐ None 1

- View the labels for the thanos-querier route.

1. Go to the **Networking** → **Routes** page.

Red Hat OpenShift

Project: All Projects

Routes

Create Route

Filter Name Search by name...

Name	Namespa...	Status	Location	Service
RT alertmanager-main	NS openshift-monitoring	Accepted	https://alertmanager-main-openshift-monitoring.apps.ocp4.example.com/api	S alertmanager-main
RT canary	NS openshift-ingress-canary	Accepted	https://canary-openshift-ingress-canary.apps.ocp4.example.com	S ingress-canary

2. Type the **thanos** keyword in the text search field.

Routes

Create Route

Filter Name

Name: thanos X Clear all filters

Name	Namespace	Status	Location	Service
RT thanos-querier	NS openshift-monitoring	Accepted	https://thanos-querier-openshift-monitoring.apps.ocp4.example.com/api	S thanos-querier

3. Select the **thanos-querier** route in the Name column.

Routes					Create Route
Filter	Name	thanos			
Name	thanos	Clear all filters			
Name	Namespace	Status	Location	Service	
RT thanos-querier	NS openshift-monitoring	Accepted	https://thanos-querier-openshift-monitoring.apps.ocp4.example.com/api	S thanos-querier	

4. Scroll down on the thanos-querier route details page to view the labels.

Red Hat

OpenShift

3

admin

Project: openshift-monitoring

Route details

Name

thanos-querier

Namespace

NS openshift-monitoring

Labels

app.kubernetes.io/component=query-layer

app.kubernetes.io/instance=thanos-querier

app.kubernetes.io/managed-by=cluster-monitoring-operator

app.kubernetes.io/name=thanos-query

app.kubernetes.io/part-of=openshift-monitoring

app.kubernetes.io/version=0.36.1

Location

https://thanos-querier-openshift-monitoring.apps.ocp4.example.com/api

Status

Accepted

Host

thanos-querier-openshift-monitoring.apps.ocp4.example.com

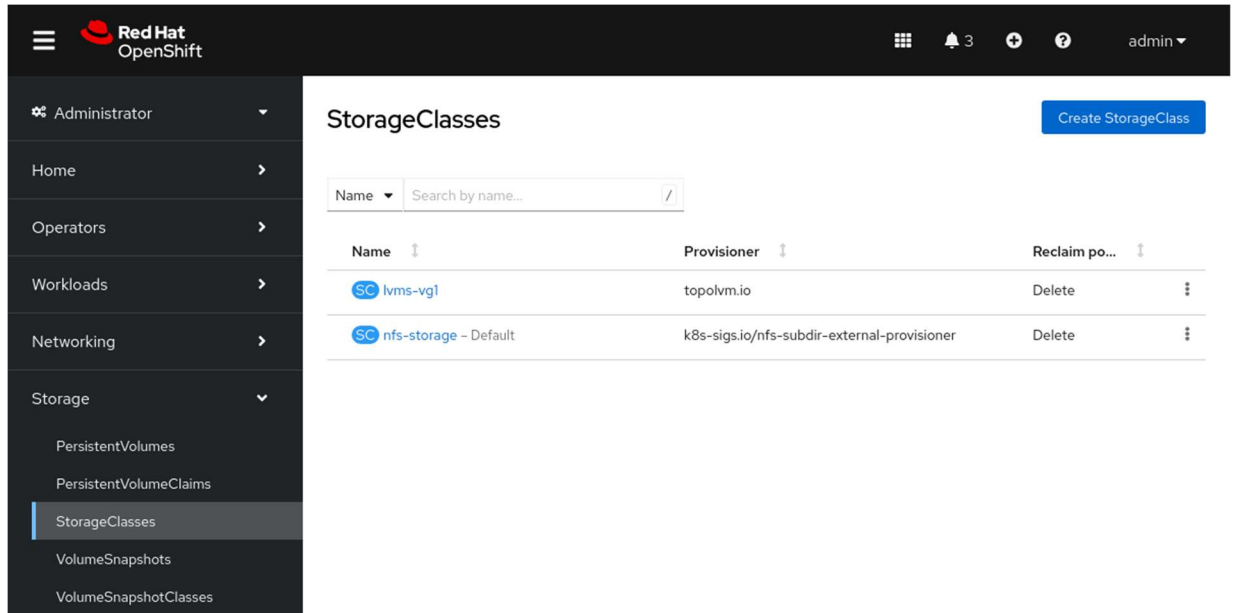
Path

/api

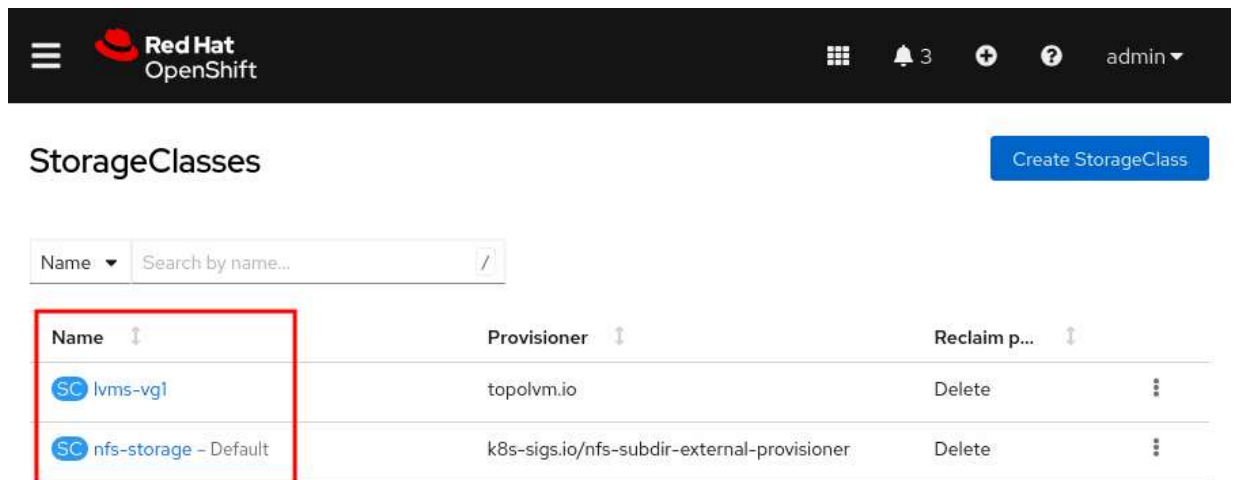
Router canonical hostname

router-default.apps.ocp4.example.com

5. View the available storage classes in the cluster.
1. Go to the **Storage** → **StorageClasses** page.



2. View the available storage classes in the cluster.



6. View the installed operators for the cluster.

1. Go to the **Operators** → **Installed Operators** page.

Red Hat OpenShift
 3 admin ▾

Administrator ▾
Home >
Operators ▾
OperatorHub
Installed Operators
Workloads >
Networking >
Storage >
Builds >
Observe >

Project: All Projects ▾

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name ▾ /

Name	Namespace	Managed Namespaces	Status	Provided APIs
 LVM Storage 4.18.1 provided by Red Hat	NS openshift-storage	NS openshift-storage	✓ Succeeded Up to date	LVMCluster
 MetalLB Operator 4.18.0-202505081435 provided by Red Hat	NS metallb-system	All Namespaces	✓ Succeeded Up to date	BGPPeer BFDProfile BGPAdvertisement

Red Hat

OpenShift

3

admin

Project: All Projects

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name

Search by name...

Name	Namespace	Managed Namespaces
 LVM Storage 4.18.1 provided by Red Hat	 openshift-storage	 openshift-storage
 MetalLB Operator 4.18.0-202505081435 provided by Red Hat	 metallb-system	All Namespaces
 Package Server 0.0.1-snapshot provided by Red Hat	 openshift-operator-lifecycle-manager	 openshift-operator-lifecycle-manager

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish intro-review
```

Quiz: Introduction to Kubernetes and OpenShift

Introduction to Kubernetes and OpenShift

Choose the correct answers to the following questions:

1.

2.

1. What is the installed version for the OpenShift cluster?

A ☐ 3.9.1

B ☐ 4.3.2

C ☐ 4.18.6

D ☐ 5.4.3

3.

4.

2. Which three severity types are available for the alerts in the

cluster?
(Choose
three.)

A ☐

Warning

B ☐

Firing

C ☐

Info

D ☐

Urgent

E ☐

Critical

F ☐

Oops

5.

6.

3. Which three labels are
on the thanos-
querier route in
the openshift-
monitoring namespace?
(Choose three.)

A ☐

app.kubernetes.io/component=query-layer

B ☐

app.kubernetes.io/instance=thanos-querier

C ☐

app.kubernetes.io/part-of=openshift-
storage

D ☐

app.kubernetes.io/version=0.36.1

7.

8.

4. Which two objects are
listed as
the StorageClasses objects

for the cluster? (Choose two.)

A ☐ ceph-storage

B ☒ nfs-storage

C ☐ k8s-lvm-vg1

D ☐ local-volume

E ☒ lvms-vg1

9.

10.

5. When **All Projects** is selected at the top, which three operators are installed in the cluster? (Choose three.)

A ☐ MongoDB Operator

B ☒ MetalLB Operator

C ☐ Red Hat Fuse

D ☒ LVM Storage

E ☒ Package Server



Summary

- A *container* is an encapsulated process that includes the required runtime dependencies for an application to run.
- Containerization addresses the application development challenges around code portability, to aid in consistently running an application from diverse environments.
- Containerization also aims to modularize applications to improve development and maintenance on the various components of the application.
- When running containers at scale, it becomes challenging to configure and deliver high availability applications and to set up networking without a container platform, such as Kubernetes.
- Pods are the smallest organizational unit for a containerized application in a Kubernetes cluster.
- Red Hat OpenShift Container Platform (RHOCP) adds enterprise-class functions to the Kubernetes container platform to deliver the wider business needs.
- Most administrative tasks that cluster administrators and developers perform are available through the RHOCP web console.
- Logs, metrics, alerts, terminal connections to the nodes and pods in the cluster, and many other features are available through the RHOCP web console.

Chapter 2. Kubernetes and OpenShift Command-line Interfaces and APIs

The Kubernetes and OpenShift Command-line Interfaces

Guided Exercise: The Kubernetes and OpenShift Command-line Interfaces

Inspect Kubernetes Resources

Guided Exercise: Inspect Kubernetes Resources

Assess the Health of an OpenShift Cluster

Guided Exercise: Assess the Health of an OpenShift Cluster

Lab: Kubernetes and OpenShift Command-line Interfaces and APIs

Quiz: Kubernetes and OpenShift Command-line Interfaces and APIs

Summary

Abstract

Goal	Access an OpenShift cluster by using the command line and query its Kubernetes API resources to assess the health of a cluster.
Sections	• The Kubernetes and OpenShift Command-line Interfaces (and Guided Exercise) • Inspect Kubernetes Resources (and Guided Exercise) • Assess the Health of an OpenShift Cluster (and Guided Exercise)
Lab	• Kubernetes and OpenShift Command-line Interfaces and APIs

The Kubernetes and OpenShift Command-line Interfaces

Objectives

- Access an OpenShift cluster by using the Kubernetes and OpenShift command-line interfaces.

The Kubernetes Command-line Interface

You can manage an OpenShift cluster from the web console or by using the `kubectl` or `oc` command-line interfaces (CLI). The `kubectl` commands are native to Kubernetes, and are a thin wrapper over the Kubernetes API. The OpenShift `oc` commands are a superset of the `kubectl` commands, and add commands for the OpenShift-specific features. In this course, examples of both the `kubectl` and the `oc` commands are shown, to highlight the differences between the commands.

With the `oc` command, you can create applications and manage Red Hat OpenShift Container Platform (RHOCP) projects from a terminal. The OpenShift CLI is ideal in the following situations:

- Working directly with project source code
- Scripting OpenShift Container Platform operations
- Managing projects that are restricted by bandwidth
- When the web console is unavailable
- Working with OpenShift resources, such as routes and projects

Kubernetes Command-line Tool

The `oc` CLI installation also includes an installation of the `kubectl` CLI, which is the recommended method for installing the `kubectl` CLI for OpenShift users.

You can also install the `kubectl` CLI independently of the `oc` CLI. You must use a `kubectl` CLI version that is within one minor version difference of your cluster. For example, a `v1.30` client can communicate with `v1.29`, `v1.30`, and `v1.31` control planes. Using the latest compatible version of the `kubectl` CLI helps to avoid unforeseen issues.

To perform a manual installation of the `kubectl` binary for a Linux installation, you must first download the latest release by using the `curl` command.

```
[user@host ~]$ curl -LO "https://dl.k8s.io/release/$(curl -L \
-s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Then, you must download the `kubect1` checksum file and validate the `kubect1` binary against the checksum file.

```
[user@host ~]$ curl -LO "https://dl.k8s.io/$(curl -L \
-s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1.sha256"
[user@host ~]$ echo "$(cat kubect1.sha256)  kubect1" | sha256sum --check
kubect1: OK
```

If the check fails, then the `sha256sum` command exits with nonzero status, and prints a `kubect1: FAILED` message.

You can then install the `kubect1` CLI.

```
[user@host ~]$ sudo install -o root -g root -m 0755 kubect1 \
/usr/local/bin/kubect1
```

NOTE

If you do not have root access on the target system, then you can still install the `kubect1` CLI to the `~/local/bin` directory. For more information, refer to <https://kubernetes.io/docs/tasks/tools/install-kubect1-linux/>.

Finally, use the `kubect1 version` command to verify the installed version. This command prints the client and server versions. Use the `--client` option to view the client version only.

```
[user@host ~]$ kubect1 version --client
Client Version: v1.31.1
Kustomize Version: v5.4.2
```

Alternatively, a distribution that is based on Red Hat Enterprise Linux (RHEL) can install the `kubect1` CLI with the following command:

```
[user@host ~]$ cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.31/rpm/
```

```
enabled=1
pgpcheck=1
pgpkey=https://pkgs.k8s.io/core:/stable:/v1.31/rpm/repodata/repomd.xml.key
EOF
[user@host ~]$ sudo yum install -y kubectl
```

To view a list of the available kubectl commands, use the `kubectl --help` command.

```
[user@host ~]$ kubectl --help
kubectl controls the Kubernetes cluster manager.

Find more information at:
https://kubernetes.io/docs/reference/kubectl/

Basic Commands (Beginner):
  create      Create a resource from a file or from stdin
  expose      Take a replication controller, service, deployment or pod and
expose it as a new Kubernetes Service
  run         Run a particular image on the cluster
  set         Set specific features on objects

Basic Commands (Intermediate):
...output omitted...
```

You can also use the `--help` option with any command to view detailed information about the command, including its purpose, examples, available subcommands, and options.

For example, the following command provides information about the `kubectl create` command and its usage.

```
[user@host ~]$ kubectl create --help
Create a resource from a file or from stdin.

JSON and YAML formats are accepted.
```

Examples:

```
# Create a pod using the data in pod.json
kubectl create -f ./pod.json

# Create a pod based on the JSON passed into stdin
cat pod.json | kubectl create -f -

# Edit the data in registry.yaml in JSON then create the resource using the edited
data
kubectl create -f registry.yaml --edit -o json
```

Available Commands:

```
clusterrole          Create a cluster role
clusterrolebinding   Create a cluster role binding for a particular cluster ro
le
...output omitted...
```

Kubernetes uses many resource components to support applications. The `kubectl explain` command provides detailed information about the attributes of a given resource. For example, use the following command to learn more about the attributes of a `pod` resource.

```
[user@host ~]$ kubectl explain pod
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
    APIVersion defines the versioned schema of this representation of an
    object.
```

...output omitted...

Refer to the Kubernetes documentation at <https://kubernetes.io/docs/reference/generated/kubect/kubectl-commands/> for further information about the `kubect1` commands.

The OpenShift Command-line Interface

The main method of interacting with an RHOCP cluster is by using the `oc` command.

You can download the `oc` CLI from the OpenShift web console to ensure that the CLI tools are compatible with the RHOCP cluster. From the OpenShift web console, go to **Help** → **Command Line tools**. The **Help** menu is represented by a ? icon. The web console provides several installation options for the `oc` client, such as downloads for the following operating systems:

- x86_64 Windows, Mac, and Linux systems
- ARM 64 Linux and Mac systems
- Linux for IBM Z, IBM Power, and little endian



Command Line Tools

[Copy login command](#)

oc - OpenShift Command Line Interface (CLI)

With the OpenShift command line interface, you can create applications and manage OpenShift projects from a terminal.

The `oc` binary offers the same capabilities as the `kubectl` binary, but it is further extended to natively support OpenShift Container Platform features.

- [Download oc for Linux for x86_64](#)
- [Download oc for Mac for x86_64](#)
- [Download oc for Windows for x86_64](#)
- [Download oc for Linux for ARM 64](#)
- [Download oc for Mac for ARM 64](#)
- [Download oc for Linux for IBM Power, little endian](#)
- [Download oc for Linux for IBM Z](#)
- [LICENSE](#)

The basic usage of the `oc` command is through its subcommands in the following syntax:

```
[user@host ~]$ oc command
```

Because the `oc` CLI is a superset of the `kubectl` CLI, the `version`, `--help`, and `explain` commands work the same for both CLIs. However, the `oc` CLI includes additional commands that are not included in the `kubectl` CLI, such as the `oc login` and `oc new-project` commands.

Developers who are familiar with Kubernetes can use the `kubectl` utility to manage a RHOCP cluster. This course uses the `oc` command-line utility, to take advantage of additional RHOCP features. The `oc` commands manage resources that are exclusive to RHOCP, such as projects, routes, and image streams.

Authentication Methods

Before you can interact with your RHOCP cluster, you must authenticate your requests. The authentication layer identifies the user that is associated with requests to the RHOCP API. After authentication, the authorization layer uses information about the requesting user to determine whether the request is allowed.

A user in OpenShift is an entity that can send requests to the RHOCP API. An RHOCP user object represents an actor that can be granted permissions in the system by adding roles to the user or to the user's groups. Typically, this entity represents the account of a developer or an administrator.

Several types of users can exist.

Regular users

Most interactive RHOCP users are represented by this user type. An RHOCP user object represents a regular user.

System users

Infrastructure uses system users to interact with the API securely. Some system users are automatically created, including the cluster administrator,

with access to everything. By default, unauthenticated requests use an anonymous system user.

Service accounts

`ServiceAccount` objects represent accounts that allow applications to access the API without sharing regular user credentials. RHOCP creates service accounts automatically when a project is created. Project administrators can create additional service accounts to define access to the contents of each project.

Each user must authenticate to access a cluster. After authentication, the policy determines what the user is authorized to do.

NOTE

Authentication and authorization are covered in greater detail in the *Red Hat OpenShift Administration II: Operating a Production Kubernetes Cluster* (DO280) course.

Use the `oc login` command to authenticate your requests. The `oc login` command provides role-based authentication and authorization that protects the RHOCP cluster from unauthorized access.

The `oc login` command supports several authentication methods.

Username and Password Authentication

The syntax for username and password authentication is as follows:

```
[user@host ~]$ oc login -u username api-url
```

Then, the `oc login` command prompts for the password.

For example, in this course, you can use the following command:

```
[user@host ~]$ oc login -u admin https://api.ocp4.example.com:6443
Console URL: https://api.ocp4.example.com:6443/console
Authentication required for https://api.ocp4.example.com:6443 (openshift)
Username: admin
```

```
Password: redhatocp
Login successful.
...output omitted...
```

Token-based Authentication

The RHOCP control plane includes a built-in OAuth server. To authenticate to the API, users obtain OAuth access tokens. Token authentication is the recommended method for production environments, automation, and CI/CD pipelines to improve security and to eliminate storing passwords in scripts. You can use token-based authentication to log in to the cluster, as follows:

```
[user@host ~]$ oc login --token=your-token --server=cluster-url
```

To retrieve a token, open a web browser and go to `https://oauth-openshift.apps.cluster-url/oauth/token/request`. Then, click **Display token** to display the login command.

Your API token is

```
sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4
```

Log in with this token

```
oc login --token=sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4
--server=https://api.ocp4.example.com:6443
```

Use this token directly against the API

```
curl -H "Authorization: Bearer sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4"
"https://api.ocp4.example.com:6443/apis/user.openshift.io/v1/users/~"
```

[Request another token](#)

Web Authentication

You can also use the `--web` option to log in to the cluster by using a web browser, as follows:

```
[user@host ~]$ oc login --web api-url
```


This command opens a browser window to the OpenShift authentication page, to log in with your username and password, or other authentication method. Then, the command automatically returns the authentication token to the command line, to complete the login process.

IMPORTANT

For simplicity, this course uses the username and password authentication method by providing the password as plain text, as follows:

```
[user@host ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
```

However, this method is not secure, and Red Hat does not recommend it for production environments. Thus, in production environments use a more secure method, such as token-based authentication.

Managing Resources at the Command Line

After authenticating to the RHOCP cluster, you can create a project with the `oc new-project` command. Projects provide isolation between your application resources. Projects are Kubernetes namespaces with additional annotations that provide multitenancy scoping for applications.

```
[user@host ~]$ oc new-project myapp
```

Several essential commands can manage RHOCP and Kubernetes resources, as described here. Unless otherwise specified, the following commands are compatible with both the `oc` and `kubectl` CLIs.

Some commands require a user with cluster administrator access. The following list includes several useful `oc` commands for cluster administrators.

oc cluster-info

The `cluster-info` command prints the address of the control plane and other cluster services. The `oc cluster-info dump` command expands the output to include helpful details for debugging cluster problems.

```
[user@host ~]$ oc cluster-info
Kubernetes control plane is running at https://api.ocp4.example.com:6443
...output omitted...
```

oc api-versions

The structure of cluster resources has a corresponding API version, which the `oc api-versions` command displays. The command prints the supported API versions on the server, in the form of "group/version".

In the following example, the group is `admissionregistration.k8s.io` and the version is `v1`:

```
[user@host ~]$ oc api-versions
admissionregistration.k8s.io/v1
...output omitted...
```

oc get clusteroperator

The cluster operators that Red Hat ships serve as the architectural foundation for RHOCP. RHOCP installs cluster operators by default. Use the `oc get clusteroperator` command to see a list of the cluster operators:

```
[user@host ~]$ oc get clusteroperator
```

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE ...
authentication	4.18.6	True	False	False	18d
baremetal	4.18.6	True	False	False	18d

```
...output omitted...
```

Other useful commands are available to both regular and administrator users:

oc get

Use the `get` command to retrieve information about resources in the selected project. Generally, this command shows only the most important characteristics of the resources, and omits detailed information.

The `oc get RESOURCE_TYPE` command displays a summary of all resources of the specified type.

For example, the following command returns the list of the `pod` resources in the current project:

```
[user@host ~]$ oc get pod
```

NAME	READY	STATUS	RESTARTS	AGE
quotes-api-6c9f758574-nk8kd	1/1	Running	0	39m
quotes-ui-d7d457674-rbk17	1/1	Running	0	67s

You can use the `oc get RESOURCE_TYPE RESOURCE_NAME` command to export a resource definition. Typical use cases include creating a backup or modifying a definition. The `-o yaml` option prints the object representation in YAML format. You can change to JSON format by providing a `-o json` option.

oc get all

Use the `oc get all` command to retrieve a summary of the most important resources in a project. This command iterates through the major resource types for the current project, and prints a summary of their information:

```
[user@host ~]$ oc get all
```

...output omitted...

NAME	READY	STATUS	RESTARTS	AGE
pod/mysql-6bb757c595-qj5ng	1/1	Running	0	16s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/mysql	ClusterIP	172.30.173.61	<none>	3306/TCP, 33060/TCP

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/mysql	1/1	1	1	2m21s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/mysql-55dc44b4c9	0	0	0	2m21s

replicaset.apps/mysql-6bb757c595	1	1	1	16s
replicaset.apps/mysql-7d9b6b5996	0	0	0	2m21s

NAME	UPDATED	IMAGE REPOSITORY
imagestream.image.openshift.io/mysql		image-registry.../mysql-openshift/mysql
latest	2 minutes ago	

oc describe

If the summaries from the `get` command are insufficient, then you can use the `oc describe RESOURCE_TYPE RESOURCE_NAME` command to retrieve additional information. Unlike the `get` command, you can use the `describe` command to iterate through all the different resources by type. Although most major resources can be described, this function is not available across all resources. The following example demonstrates describing a pod resource:

```
[user@host ~]$ oc describe pod mysql-6bb757c595-qj5ng
Name:          mysql-6bb757c595-qj5ng
Namespace:     mysql-openshift
Priority:       0
Service Account: default
Node:          master01/192.168.50.10
Start Time:    Mon, 16 Jun 2025 16:07:00 -0400
Labels:        db=mysql
               deployment=mysql
...output omitted...
Status:        Running
SeccompProfile: RuntimeDefault
IP:            10.129.0.85
...output omitted...
```

oc explain

To learn about the fields of an API resource object, use the `oc explain` command. This command describes the purpose and the fields that are associated with each supported API resource. You can also use this

command to print the documentation of a specific field of a resource. Fields are identified via a JSONPath identifier. The following example prints the documentation for the `.spec.containers.resources` field of the `pod` resource type:

```
[user@host ~]$ oc explain pods.spec.containers.resources
KIND:      Pod
VERSION:   v1

FIELD: resources <ResourceRequirements>

DESCRIPTION:
    Compute Resources required by this container. Cannot be updated. More info:
    https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

    ResourceRequirements describes the compute resource requirements.

FIELDS:
    claims          <[]ResourceClaim>
        Claims lists the names of resources, defined in spec.resourceClaims, that
        are used by this container.

        This is an alpha field and requires enabling the DynamicResourceAllocation
        feature gate.

        This field is immutable. It can only be set for containers.

    limits <map[string]Quantity>
        Limits describes the maximum amount of compute resources allowed. More
        info:
        https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
```

```
requests      <map[string]Quantity>

Requests describes the minimum amount of compute resources required. If
Requests is omitted for a container, it defaults to Limits if that is
explicitly specified, otherwise to an implementation-defined value. Reque
sts
cannot exceed Limits. More info:
https://kubernetes.io/docs/concepts/configuration/manage-resources-contai
ners/
```

Add the `--recursive` flag to display all fields of a resource without descriptions. Information about each field is retrieved from the server in OpenAPI format.

oc create

Use the `create` command to create a RHOCP resource in the current project. This command creates resources from a resource definition. Typically, this command is paired with the `oc get RESOURCE_TYPE RESOURCE_NAME -o yaml` command for editing definitions. Developers commonly use the `-f` flag to indicate the file that contains the JSON or YAML representation of an RHOCP resource.

For example, to create resources from the `pod.yaml` file, use the following command:

```
[user@host ~]$ oc create -f pod.yaml
pod/quotes-pod created
```

RHOCP resources in the YAML format are discussed later.

oc status

The `oc status` command provides a high-level overview of the current project. The command shows services, deployments, build configurations, and active deployments. Information about any misconfigured components is also shown. The `--suggest` option shows additional details for any identified issues.

oc delete

Use the `delete` command to delete an existing RHOCP resource from the current project. You must specify the resource type and the resource name.

For example, to delete the `quotes-ui` pod, use the following command:

```
[user@host ~]$ oc delete pod quotes-ui  
pod/quotes-ui deleted
```

A fundamental understanding of the RHOCP architecture is needed here, because deleting managed resources, such as pods, results in the automatic creation of new instances of those resources. When a project is deleted, it deletes all the resources and applications within it.

Each of these commands is executed in the current selected project. To execute commands in a different project, you must include the `--namespace` or `-n` options.

```
[user@host ~]$ oc get pods -n openshift-apiserver
```

NAME	READY	STATUS	RESTARTS	AGE
apiserver-68c9485699-ndqlc	2/2	Running	2	1

Guided Exercise: The Kubernetes and OpenShift Command-line Interfaces

Access an OpenShift cluster by using the command-line to get information about cluster services and nodes.

Outcomes

- Use the OpenShift web console to locate the installation file for the `oc` OpenShift command-line interface.
- Get and use a token from the web console to access the cluster from the command line.
- Identify key differences between the `kubectl` and `oc` command-line tools.
- Identify the main components of OpenShift and Kubernetes.

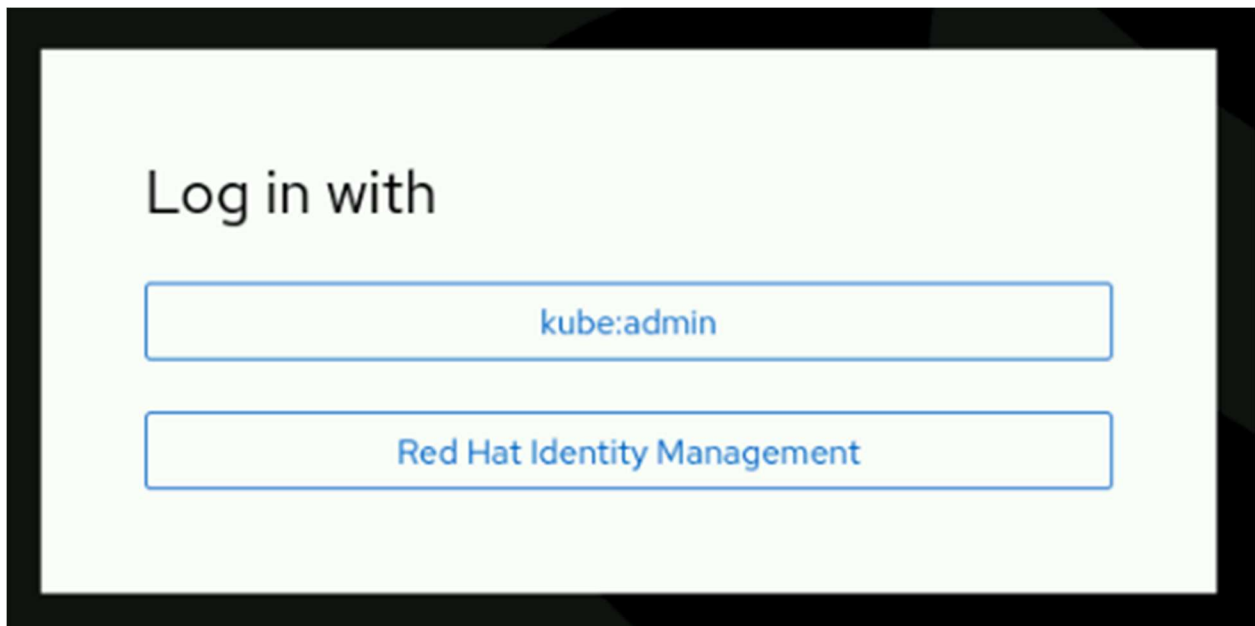
As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise.

```
[student@workstation ~]$ lab start cli-interfaces
```

Instructions

1. Log in to the OpenShift web console as the developer user. Locate the installation file for the `oc` OpenShift command-line interface (CLI).
1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>.
2. Click **Red Hat Identity Management** and log in as the developer user with the developer password.



3. Locate the installation file for the `oc` CLI. From the OpenShift web console, select **Help** → **Command line tools**. The **Help** menu is represented by a ? icon.

Command Line Tools

[Copy login command](#)

oc - OpenShift Command Line Interface (CLI)

With the OpenShift command line interface, you can create applications and manage OpenShift projects from a terminal.

The `oc` binary offers the same capabilities as the `kubectl` binary, but it is further extended to natively support OpenShift Container Platform features.

- [Download oc for Linux for x86_64](#)
 - [Download oc for Mac for x86_64](#)
 - [Download oc for Windows for x86_64](#)
 - [Download oc for Linux for ARM 64](#)
 - [Download oc for Mac for ARM 64](#)
 - [Download oc for Linux for IBM Power, little endian](#)
 - [Download oc for Linux for IBM Z](#)
 - [LICENSE](#)
-

The `oc` binary is available for multiple operating systems and architectures. For each operating system and architecture, the `oc` binary also includes the `kubectl` binary.

NOTE

You do not need to download or install the `oc` and `kubectl` binaries, which are already installed on the workstation machine.

2. Download an authorization token from the web console. Then, use the token and the `oc` command to log in to the OpenShift cluster.
1. From the **Command Line Tools** page, click the **Copy login command** link.
2. The link opens a login page. Click **Red Hat Identity Management** and log in as the `developer` user with the `developer` password.
3. A web page is displayed. Click the **Display token** link to show your API token and the login command.

Your API token is

sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4

Log in with this token

```
oc login --token=sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4
--server=https://api.ocp4.example.com:6443
```

Use this token directly against the API

```
curl -H "Authorization: Bearer sha256~nJg06D38lkgTfN9K2Ib7cTvBs3dvVKQfnrXrm5oBiU4"
"https://api.ocp4.example.com:6443/apis/user.openshift.io/v1/users/~"
```

[Request another token](#)

4. Copy the `oc login` command to your clipboard. Open a terminal window and then use the copied command to log in to the cluster with your token.

5. [student@workstation ~]\$ `oc login --token=sha256-fypX...0t6A \`
6. `--server=https://api.ocp4.example.com:6443`
7. Logged into "https://api.ocp4.example.com:6443" as "developer" using the token provided.

...output omitted...

3. Compare the available commands for the `kubectl` and `oc` commands.
1. Use the `help` command to list and review the available commands for the `kubectl` command.

2. [student@workstation ~]\$ `kubectl help`
3. `kubectl` controls the Kubernetes cluster manager.
- 4.
5. Find more information at: <https://kubernetes.io/docs/reference/kubectl/>
- 6.
7. Basic Commands (Beginner):
8. `create` Create a resource from a file or from stdin
9. `expose` Take a replication controller, service, deployment or pod and expose it as a new Kubernetes service

- 10. `run` Run a particular image on the cluster
- 11. `set` Set specific features on objects
- 12.
- 13. Basic Commands (Intermediate):
- 14. `explain` Get documentation for a resource
- 15. `get` Display one or many resources
- 16. `edit` Edit a resource on the server
- 17. `delete` Delete resources by file names, stdin, resources and name s, or by resources and label selector

...output omitted....

Notice that the `kubect1` command does not provide a `login` command.

- 18. Examine the available subcommands and options for the `kubect1` `create` command by using the `--help` option.

- 19. `[student@workstation ~]$ kubect1 create --help`
- 20. Create a resource from a file or from stdin.
- 21.
- 22. JSON and YAML formats are accepted.
- 23.
- 24. Examples:
- 25. `# Create a pod using the data in pod.json`
- 26. `kubect1 create -f ./pod.json`
- 27. *...output omitted....*
- 28. Available Commands:
- 29. `clusterrole` Create a cluster role
- 30. `clusterrolebinding` Create a cluster role binding for a particular cluster role
- 31. `configmap` Create a config map from a local file, directory or literal value
- 32. `cronjob` Create a cron job with the specified name
- 33. `deployment` Create a deployment with the specified name
- 34. *...output omitted...*
- 35. Options:

```
36.      --allow-missing-template-keys=true:
37.          If true, ignore any errors in templates when a field or map key is m
issing in the template. Only applies to
38.          goyaml and jsonpath output formats.
39.
40.      --dry-run='none':
41.          Must be "none", "server", or "client". If client strategy, only prin
t the object that would be sent, without
42.          sending it. If server strategy, submit server-side request without p
ersisting the resource.
43....output omitted....
44.Usage:
45.  kubectl create -f FILENAME [options]
46.
47.Use "kubectl create <command> --help" for more information about a given co
mmand.
```

Use "kubectl options" for a list of global command-line options (applies to all commands).

You can use the --help option with any kubectl command. The --help option provides information about a command, including the available subcommands and options, and the command syntax.

48. List and review the available commands for the oc binary by using the help command.

```
49. [student@workstation ~]$ oc help
50. OpenShift Client
51.
52. This client helps you develop, build, deploy, and run your applications on
any
53. OpenShift or Kubernetes cluster. It also includes the administrative
54. commands for managing a cluster under the 'adm' subcommand.
55.
56. Basic Commands:
57.  login                Log in to a server
58.  new-project          Request a new project
```

59.	<code>new-app</code>	Create a new application
60.	<code>status</code>	Show an overview of the current project
61.	<code>project</code>	Switch to another project
62.	<code>projects</code>	Display existing projects
63.	<code>explain</code>	Get documentation for a resource

...output omitted....

The `oc` command supports the same capabilities as the `kubectl` command. The `oc` command provides additional commands to natively support an OpenShift cluster. For example, the `new-project` command creates a project, which is a Kubernetes namespace, in the OpenShift cluster. The `new-app` command is unique to the `oc` command. It creates applications by using existing source code or prebuilt images.

64. Use the `--help` option with the `oc create` command to view the available subcommands and options.

```
65. [student@workstation ~]$ oc create --help
66. Create a resource from a file or from stdin.
67.
68. JSON and YAML formats are accepted.
69.
70. Examples:
71. # Create a pod using the data in pod.json
72. oc create -f ./pod.json
73. ...output omitted...
74.
75. Available Commands:
76. build                Create a new build
77. clusterresourcequota Create a cluster resource quota
78. clusterrole          Create a cluster role
79. clusterrolebinding   Create a cluster role binding for a particular cluster role
80. configmap            Create a config map from a local file, directory or literal value
81. cronjob              Create a cron job with the specified name
```

```
82. deployment          Create a deployment with the specified name
83....output omitted....
84.Options:
85.    --allow-missing-template-keys=true:
86.        If true, ignore any errors in templates when a field or map key is missing in the template. Only applies to
87.        goyaml and jsonpath output formats.
88.
89.    --dry-run='none':
90.        Must be "none", "server", or "client". If client strategy, only print the object that would be sent, without
91.        sending it. If server strategy, submit server-side request without persisting the resource.
92....output omitted...
93.Usage:
94.  oc create -f FILENAME [options]
```

```
....output omitted....
```

The `oc create` command includes the same subcommands and options as the `kubectl create` command, and provides additional subcommands for OpenShift resources. For example, you can use the `oc create` command to create OpenShift resources such as a deployment, a route, and an image stream.

4. Use the `oc login` command with the `--web` option to log in as the admin user with `redhatocp` as the password. Identify the components and Kubernetes resources of an OpenShift cluster by using the terminal. Regular cluster users, such as the `developer` user, cannot list resources at a cluster scope. Unless otherwise noted, all commands are available for the `oc` and `kubectl` commands.
1. In a terminal, use the `oc login --web` command.

```
[student@workstation ~]$ oc login --web
```

2. In the browser tab that opens, click **Red Hat Identity Management** and log in as the admin user with `redhatocp` as the password. You get the access

token received successfully; please return to your terminal message.
Close the browser tab.

3. Return to the terminal window. Identify the cluster version with the version command.

```
4. [student@workstation ~]$ oc version
5. Client Version: 4.18.6
6. Kustomize Version: v5.4.2
7. Server Version: 4.18.6
```

```
Kubernetes Version: v1.31.6
```

8. Use the cluster-info command to identify the URL for the Kubernetes control plane.

```
9. [student@workstation ~]$ oc cluster-info
10. Kubernetes control plane is running at https://api.ocp4.example.com:6443
11.
```

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

12. Identify the supported API versions by using the api-versions command.

```
13. [student@workstation ~]$ oc api-versions
14. admissionregistration.k8s.io/v1
15. admissionregistration.k8s.io/v1beta1
16. apiextensions.k8s.io/v1
17. apiregistration.k8s.io/v1
18. apiserver.openshift.io/v1
19. apps.openshift.io/v1
20. apps/v1
```

...output omitted....

21. List cluster operators with the get clusteroperator command.

```
22. [student@workstation ~]$ oc get clusteroperator
```

23. NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE
...					
24. authentication	4.18.6	True	False	False	5d5h
25. baremetal	4.18.6	True	False	False	41d
26. cloud-controller-manager	4.18.6	True	False	False	41d
27. cloud-credential	4.18.6	True	False	False	41d
28. cluster-autoscaler	4.18.6	True	False	False	41d
29. config-operator	4.18.6	True	False	False	41d
30. console	4.18.6	True	False	False	29d
31. control-plane-machine-set	4.18.6	True	False	False	41d
32. csi-snapshot-controller	4.18.6	True	False	False	41d
33. dns	4.18.6	True	False	False	32m
34. etcd	4.18.6	True	False	False	41d
35. image-registry	4.18.6	True	False	False	41d
36. ingress	4.18.6	True	False	False	41d

...output omitted...

37. Use the `get` command to list pods in the `openshift-api` project. Specify the project with the `-n` option.

```
38. [student@workstation ~]$ oc get pods -n openshift-apiserver
```

39. NAME	READY	STATUS	RESTARTS	AGE
----------	-------	--------	----------	-----

apiserver-68c9485699-ndqlc	2/2	Running	6	18d
----------------------------	-----	---------	---	-----

40. Use the `oc status` command to retrieve the status of resources in the `openshift-authentication` project.

```
41. [student@workstation ~]$ oc status -n openshift-authentication
```

```
42. Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14+, unavailable in v4.10000+
```

```
43. In project openshift-authentication on server https://api.ocp4.example.com:6443
```

```
44.
```

```
45. https://oauth-openshift.apps.ocp4.example.com (passthrough) to pod port 6443 (svc/oauth-openshift)
```



```
46. deployment/oauth-openshift deploys quay.io/openshift-release-dev/ocp-v4.0
-art-dev@sha256:64e6...de42
47. deployment #7 running for 2 weeks - 1 pod
48. deployment #6 deployed 2 weeks ago
49. deployment #4 deployed 2 weeks ago
50. deployment #5 deployed 2 weeks ago
51. deployment #3 deployed 2 weeks ago
52. deployment #2 deployed 2 weeks ago
53. deployment #1 deployed 2 weeks ago
```

...output omitted...

54. Use the `explain` command to list the description and available fields for services resources.

```
55. [student@workstation ~]$ oc explain services
56. KIND:      Service
57. VERSION:   v1
58.
59. DESCRIPTION:
60.     Service is a named abstraction of software service (for example, mysql
    )
61.     consisting of local port (for example 3306) that the proxy listens on,
    and
62.     the selector that determines which pods will answer requests sent thro
    ugh
63.     the proxy.
64.
65. FIELDS:
66.     apiVersion    <string>
67.     APIVersion defines the versioned schema of this representation of an
68.     object. Servers should convert recognized schemas to the latest intern
    al
69.     value, and may reject unrecognized values.
```

...output omitted...

70. Use the `get` command to list cluster nodes.

```
71. [student@workstation ~]$ oc get nodes
```

72. NAME	STATUS	ROLES	AGE	VERSION
master01	Ready	control-plane,master,worker	18d	v1.31.6

A single node exists in the cluster.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-interfaces
```

Inspect Kubernetes Resources

Objectives

- Query, format, and filter attributes of Kubernetes resources.

Kubernetes and OpenShift Resources

Kubernetes uses API resource objects to represent the intended state of everything in the cluster. All administrative tasks require creating, viewing, and changing the API resources. Use the `oc api-resources` command to view the Kubernetes resources.

NOTE

The `oc` commands in the examples are identical to the equivalent `kubectl` commands.

```
[user@host ~]$ oc api-resources
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
componentstatuses	cs	v1	false	ComponentStatus

configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
...output omitted...				
daemonsets	ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true	Deployment
replicasets	rs	apps/v1	true	ReplicaSet
statefulsets	sts	apps/v1	true	StatefulSet
...output omitted...				

The `SHORTNAME` for a component helps to minimize typing long CLI commands. For example, you can use `oc get cm` instead of `oc get configmaps`.

The `APIVERSION` column divides the objects into API groups. The column uses the `<API-Group>/<API-Version>` format. The API-Group object is blank for Kubernetes core resource objects.

Many Kubernetes resources exist within the context of a Kubernetes namespace. Kubernetes namespaces and OpenShift projects are broadly similar. A one-to-one relationship always exists between a namespace and an OpenShift project.

The `KIND` is the formal Kubernetes resource schema type.

The `oc api-resources` command can further filter the output with options that operate on the data.

Table 2.1. The `api-resources` Command Options

Option Example	Description
<code>--namespaced=true</code>	If false, return non-namespaced resources, otherwise return namespaced resources
<code>--api-group apps</code>	Limit to resources in the specified API group. Use <code>--api-group=''</code> to show core resources
<code>--sort-by name</code>	If non-empty, sort the list of resources by using the specified field. The field can be <code>name</code> , <code>age</code> , or <code>cpu</code>

For example, use the following `oc api-resources` command to see all the namespaced resources in the `apps` API group, sorted by name.

```
[user@host ~]$ oc api-resources --namespaced=true --api-group apps --sort-by name
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
controllerrevisions		apps/v1	true	ControllerRevision
daemonsets	ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true	Deployment
replicasets	rs	apps/v1	true	ReplicaSet
statefulsets	sts	apps/v1	true	StatefulSet

Each resource contains fields that identify the resource or that describe the intended configuration of the resource. Use the `oc explain` command to get information about valid fields for an object. For example, execute the `oc explain pod` command to get information about possible pod object fields.

```
[user@host ~]$ oc explain pod
```

```
KIND:      Pod
```

```
VERSION:   v1
```

DESCRIPTION:

Pod is a collection of containers that can run on a host. This resource is created by clients and scheduled onto hosts.

FIELDS:

apiVersion <string>

APIVersion defines the versioned schema of this representation of an

...

kind <string>

Kind is a string value representing the REST resource this object

...

metadata <ObjectMeta>

Standard object's metadata. More info:

...

```
spec <PodSpec>
```

Specification of the desired behavior of the pod. More info:

...output omitted...

Every Kubernetes resource contains the `kind`, `apiVersion`, `spec`, and `status` fields. However, when you create an object definition, you do not need to provide the `status` field. Instead, Kubernetes generates the `status` field, and it lists information such as runtime status and readiness. The `status` field is useful for troubleshooting an error or for verifying the current state of a resource.

You can use the YAML path to a field and dot-notation to get information about a particular field. For example, the following `oc explain` command shows details for the `pod.spec` fields.

```
[user@host ~]$ oc explain pod.spec
```

KIND: Pod

VERSION: v1

FIELD: spec <PodSpec>

DESCRIPTION:

Specification of the desired behavior of the pod. More info:

<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>

PodSpec is a description of a pod.

FIELDS:

activeDeadlineSeconds <integer>

Optional duration in seconds the pod may be active on the node relative to

...output omitted...

The following Kubernetes main resource types can be created and configured by using a YAML or a JSON manifest file, or by using OpenShift management tools:

Pods (pod)

Represent a collection of containers that share resources, such as IP addresses and persistent storage volumes. It is the primary unit of work for Kubernetes.

Services (svc)

Define a single IP/port combination that provides access to a pool of pods. By default, services connect clients to pods in a round-robin fashion.

ReplicaSet (rs)

Ensure that a specified number of pod replicas are running at any given time.

Persistent Volumes (pv)

Define storage areas for Kubernetes pods to use.

Persistent Volume Claims (pvc)

Represent a request for storage by a pod. PVCs link a PV to a pod so that its containers can use the provisioned storage, usually by mounting the storage into the container's file system.

ConfigMaps (cm) and Secrets

Contain a set of keys and values that other resources can use. Configuration maps and secrets centralize configuration values for several resources. Secrets differ from configuration maps in that the values of Secrets are always encoded (not encrypted), and their access is restricted to authorized users.

Deployment (deploy)

A deployment represents a set of containers that are included in a pod, and the deployment strategies to use. A deployment object contains the configuration to apply to all containers of each pod replica, such as the base image, tags, storage definitions, and the commands to execute when the containers start. Although Kubernetes replicas can be created stand-alone in OpenShift, they are usually created by higher-level resources such as deployment controllers.

Red Hat OpenShift Container Platform (RHOCP) adds the following main resource types to Kubernetes:

Build (build)

A *build* is the generic process of taking an input source, such as the source code of an application, and producing a usable resource as the output, such as a runnable image.

BuildConfig (bc)

A *BuildConfig* object defines a build process. A *BuildConfig* object requires at least the source input of the build, the strategy that defines how to build the input, and where to store the output of the process.

Routes

Represent a DNS hostname that the OpenShift router recognizes as an ingress point for applications and microservices.

Structure of Resources

Almost every Kubernetes object includes two nested object fields that govern the object's configuration: the object spec and the object status. The spec object describes the intended state of the resource, and the status object describes the current state. You specify the spec section of the resource when you create the object. Kubernetes controllers continuously update the status of the object throughout the existence of the object. The Kubernetes control plane continuously and actively manages every object's actual state to match the desired state you supplied.

The status field uses a collection of condition resource objects with the following fields.

Table 2.2. Condition Resource Fields

Field	Example
Type	ContainersReady
Status	False

Field	Example
Reason	RequirementsNotMet
Message	2/3 containers are running
LastTransitionTime	2023-03-07T18:05:28Z

For example, in Kubernetes, a Deployment object can represent an application that is running on your cluster. When you create a Deployment object, you might configure the deployment spec object to specify that you want three replicas of the application to be running. Kubernetes reads the deployment spec object and starts three instances of your chosen application, and updates the status field to match your spec object. If any of those instances fails, then Kubernetes responds to the difference between the spec and status objects by making a correction, in this case to start a replacement instance.

Other common fields provide base information in addition to the spec and status fields of a Kubernetes object.

Table 2.3. API Resource Fields

Field	Description
apiVersion	Identifier of the object schema version.
kind	Schema identifier.
metadata.name	Creates a label with a name key that other resources in Kubernetes can use to identify the object.
metadata.namespace	The namespace, or the RHOCP project where the resource is.
metadata.labels	Key-value pairs that can connect identifying metadata with Kubernetes objects.

Resources in Kubernetes consist of multiple objects. These objects define the intended state of a resource. When you create or modify an object, you make a persistent record of the intended state. Kubernetes reads the object and modifies the current state accordingly.

All RHOCP and Kubernetes objects can be represented as a JSON or YAML structure. Consider the following pod object in the YAML format:

```
apiVersion: v1

kind: Pod

metadata:

  name: wildfly

  namespace: my_app

  labels:

    name: wildfly

spec:
  containers:
    - resources:
        limits:
          cpu: 0.5

        image: quay.io/example/todojee:v1

        name: wildfly

        ports:

          - containerPort: 8080
            name: wildfly

        env:
          - name: MYSQL_DATABASE
            value: items
          - name: MYSQL_USER
            value: user1
```

```

      - name: MYSQL_PASSWORD
        value: mypa55
...object omitted...

status:
  conditions:
  - lastProbeTime: null
    lastTransitionTime: "2023-08-19T12:59:22Z"
    status: "True"
    type: PodScheduled

```

Identifier of the object schema version.

Schema identifier. In this example, the object conforms to the pod schema.

Metadata for a given resource, such as annotations, labels, name, and namespace.

A unique name for a pod in Kubernetes that enables administrators to run commands on it.

The namespace, or the RHOC project that the resource resides in.

Creates a label with a name key that other resources in Kubernetes, usually a service, can use to find it.

Defines the pod object configuration, or the intended state of the resource.

Defines the container image name.

Name of the container inside a pod. Container names are important for oc commands when a pod contains multiple containers.

A container-dependent attribute to identify the port that the container uses.

Defines a collection of environment variables.

Current state of the object. Kubernetes provides this field, which lists information such as runtime status, readiness, and container images.

Labels are key-value pairs that you define in the `.metadata.labels` object path, for example:

```

kind: Pod
apiVersion: v1
metadata:
  name: example-pod

```

```
labels:
  app: example-pod
  group: developers
...object omitted...
```

The preceding example contains the `app=example-pod` and `group=developers` labels. Developers often use labels to target a set of objects by using the `-l` or the `--selector` option. For example, the following `oc get` command lists pods that contain the `group=developers` label:

```
[user@host ~]$ oc get pod --selector group=developers
NAME                                READY   STATUS    RESTARTS   AGE
example-pod-6c9f758574-7fhg        1/1     Running   5           11d
```

Command Outputs

The `kubectl` and `oc` CLI commands provide many output formatting options. By default, many commands display a small subset of the most useful fields for the given resource type in a tabular output. Many commands support a `-o wide` option that shows additional fields.

Table 2.4. Tabular Fields

<code>oc get pods</code>	<code>oc get pods -o wide</code>
NAME	NAME
READY	READY
STATUS	STATUS
RESTARTS	RESTARTS
AGE	AGE
	IP
	NODE
	NOMINATED NODE
	READINESS GATES

To view all the fields that are associated with a resource, the `describe` subcommand shows a detailed description of the selected resource and related resources. You can select a single object by name, or all objects of that type, or provide a name prefix, or a label selector.

For example, the following command first looks for an exact match on the `TYPE` object and the `NAME-PREFIX` object. If no such resource exists, then the command outputs details for every resource of that type with a name with a `NAME_PREFIX` prefix.

```
[user@host ~]$ oc describe TYPE NAME-PREFIX
```

The `describe` subcommand provides detailed human-readable output. However, the format of the `describe` output might change between versions, and thus is not recommended for script development. Any scripts that rely on the output of the `describe` subcommand might break after a version update.

Kubernetes provides YAML and JSON-formatted output options that are suitable for parsing or scripting.

YAML Output

The `-o yaml` option provides a YAML-formatted output that is parsable and still human-readable.

```
[user@host ~]$ oc get pods -o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: Pod
  metadata:
    annotations:
...object omitted...
```

NOTE

The reference documentation provides a more detailed introduction to YAML.

You can use any tool that can process YAML documents to filter the YAML output for your chosen field. For example, you can use the `yq` tool at <https://mikefarah.gitbook.io/yq/> to process YAML and JSON files.

The `yq` processor uses a dot notation to separate field names in a query. The following example pipes the YAML output to the `yq` command to parse the `podIP` field.

```
[user@host ~]$ oc get pods -o yaml | yq '.items[0].status.podIP'
10.8.0.60
```

The `[0]` in the example specifies the first index in the `items` array.

NOTE

Another tool named `yq` is at <https://kislyuk.github.io/yq/>. The two `yq` tools are not compatible; commands that are designed for one of them do not work with the other.

JSON Output

Kubernetes uses the JSON format internally to process resource objects. Use the `-o json` option to view a resource in the JSON format.

```
[user@host ~]$ oc get pods -o json
{
  "apiVersion": "v1",
  "items": [
    {
      "apiVersion": "v1",
      "kind": "Pod",
      "metadata": {
        "annotations": {
```

You can use other tools to process JSON documents, such as the `jq` tool at <https://jqlang.github.io/jq/>. Similar to the `yc` processor, use the `jq` processor and dot notation on the fields to query specific information from the JSON-formatted output.

```
[user@host ~]$ oc get pods -o json | jq '.items[0].status.podIP'
"10.8.0.60"
```

Alternatively, the example might have used `.items[].status.podIP` for the query string. The empty brackets instruct the `jq` tool to query all items.

Custom Output

Kubernetes provides a custom output format that combines the convenience of extracting data via `jq` styled queries with a tabular output format. Use the `-o custom-columns` option with comma-separated *<column name> : <jq query string>* pairs.

```
[user@host ~]$ oc get pods \
-o custom-columns=PodName:".metadata.name",\
ContainerName:".spec.containers[].name",\
Phase:".status.phase",\
IP:".status.podIP",\
Ports:".spec.containers[].ports[].containerPort"
PodName          ContainerName    Phase    IP          Ports
myapp-77fb5cd997-xplhz  myapp           Running  10.8.0.60   <none>
```

Kubernetes also supports the use of JSONPath expressions. JSONPath is a query language for JSON. JSONPath expressions refer to a JSON data structure; they filter and extract formatted fields from JSON objects.

In the following example, the JSONPath expression uses the `range` operator to iterate over the list of pods to extract the name of the pod, its IP address, and the assigned ports.

```
[user@host ~]$ oc get pods \
-o jsonpath='{range .items[]}{ "Pod Name: "}{.metadata.name}
```

```
{"IP: "}{.status.podIP}
{"Ports: "}{.spec.containers[].ports[].containerPort}{"\n"}{end}'
Pod Name: myapp-77fb5cd997-xplhz
IP: 10.8.0.60
Ports:
```

You can customize the format of the output with Go templates, which the Go programming language uses. Use the `-o go-template` option followed by a Go template, where Go expressions are inside double braces, `{{ }}`.

```
[user@host ~]$ oc get pods \
-o go-template='{{range .items}}{{.metadata.name}}{{"\n"}}{{end}}'
myapp-77fb5cd997-xplhz
```

Guided Exercise: Inspect Kubernetes Resources

Verify the state of an OpenShift cluster by querying its recognized resource types, their schemas, and extracting information from Kubernetes resources that are related to OpenShift cluster services.

Outcomes

- List and explain the supported API resources for a cluster.
- Identify resources from specific API groups.
- Format command outputs in the YAML and JSON formats.
- Use filters to parse command outputs.
- Use JSONPath and custom columns to extract information from resources.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise. It also creates a `myapp` application in the `cli-resources` project.

```
[student@workstation ~]$ lab start cli-resources
```

Instructions

1. Log in to the OpenShift cluster as the developer user with the developer password. Select the cli-resources project.
1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Set the cli-resources project as the active project.

```
6. [student@workstation ~]$ oc project cli-resources
```

...output omitted...

2. List the available cluster resource types with the api-resources command. Then, use filters to list namespaced and non-namespaced resources.
1. List the available resource types with the api-resources command.

```
2. [student@workstation ~]$ oc api-resources
3. NAME                                SHORTNAMES  APIVERSION  NAMESPACED  KIND
4. bindings                            v1          true        Binding
5. componentstatuses                  cs          v1          false        ComponentStat
6. configmaps                         cm          v1          true         ConfigMap
7. endpoints                         ep          v1          true         Endpoints
8. events                             ev          v1          true         Event
9. limitranges                       limits      v1          true         LimitRange
10. namespaces                        ns          v1          false        Namespace
11. nodes                             no          v1          false        Node
12. persistentvolumeclaims            pvc         v1          true         PersistentVo
13. persistentvolumes                 pv          v1          false        PersistentVo
14. pods                              po          v1          true         Pod
15. ...output omitted...
16. controllerrevisions               apps/v1     true        ControllerRe
```


17. daemonsets	ds	apps/v1	true	DaemonSet
18. ...output omitted...				
19. cronjobs	cj	batch/v1	true	CronJob
20. jobs		batch/v1	true	Job

...output omitted...

The `api-resources` command prints the supported API resources, including resource names, available shortnames, and the API versions.

You can use the `APIVERSIONS` field to determine which API group provides the resource. The field lists the group followed by the API version of the resource. For example, the `jobs` resource type is provided by the `batch` API group, and `v1` is the API version of the resource.

21. Use the `--namespaced` option to limit the output of the `api-resources` command to namespaced resources.

Then, determine the number of available namespaced resources. Use the `-o name` option to list the resource names, and then pipe the output to the `wc -l` command.

```
[student@workstation ~]$ oc api-resources --namespaced
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
Pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota

```

secrets                                v1            true          Secret
...output omitted...
[student@workstation ~]$ oc api-resources --namespaced -o name | wc -l
122

```

The cluster has 122 namespaced cluster resource types, such as the pods, deployments, and services resources.

22. Limit the output of the api-resources command to non-namespaced resources.

Then, determine the number of available non-namespaced resources. To list the resource names, use the `-o name` option and then pipe the output to the `wc -l` command.

```

[student@workstation ~]$ oc api-resources --namespaced=false
NAME                                SHORTNAMES     APIVERSION
...
componentstatuses                  cs             v1
...
namespaces                         ns             v1
...
nodes                              no             v1
...
persistentvolumes                  pv             v1
...
mutatingwebhookconfigurations      admissionregistration.k8s.io/
v1 ...
validatingwebhookconfigurations    admissionregistration.k8s.io/
v1 ...
customresourcedefinitions          crd,crds       apiextensions.k8s.io/v1
...
...output omitted...
[student@workstation ~]$ oc api-resources --namespaced=false -o name | wc -l
129

```

The cluster has 129 non-namespaced cluster resource types, such as the nodes, images, and project resources.

3. Identify and explain the available cluster resource types that the core API group provides. Then, describe a resource from the core API group in the `cli-resources` project.
1. Filter the output of the `api-resources` command to show only resources from the core API group. Use the `--api-group` option and set `'` as the value.

```
2. [student@workstation ~]$ oc api-resources --api-group ''
```

3. NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
4. bindings		v1	true	Binding
5. componentstatuses	cs	v1	false	ComponentSta tus
6. configmaps	cm	v1	true	ConfigMap
7. endpoints	ep	v1	true	Endpoints
8. events	ev	v1	true	Event
9. limitranges	limits	v1	true	LimitRange
10. namespaces	ns	v1	false	Namespace
11. nodes	no	v1	false	Node
12. persistentvolumeclaims	pvc	v1	true	PersistentV olumeClaim
13. persistentvolumes	pv	v1	false	PersistentV olume
14. pods	po	v1	true	Pod
15. podtemplates		v1	true	PodTemplate
16. replicationcontrollers	rc	v1	true	Replication Controller
17. resourcequotas	quota	v1	true	ResourceQuo ta
18. secrets		v1	true	Secret
19. serviceaccounts	sa	v1	true	ServiceAcco unt

services	svc	v1	true	Service
----------	-----	----	------	---------

The core API group provides several resource types, such as nodes, events, and pods.

20. Use the `explain` command to list a description and the available fields for the `pods` resource type.

```
21. [student@workstation ~]$ oc explain pods
22. KIND:      Pod
23. VERSION:   v1
24.
25. DESCRIPTION:
26.     Pod is a collection of containers that can run on a host. This resource is
27.     created by clients and scheduled onto hosts.
28.
29. FIELDS:
30.     apiVersion    <string>
31.     APIVersion defines the versioned schema of this representation of an
32.     object. Servers should convert recognized schemas to the latest internal
33.     value, and may reject unrecognized values. More info:
```

...output omitted...

34. List all pods in the `cli-resources` project.

```
35.[student@workstation ~]$ oc get pods
```

36.NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

```
myapp-54fcdcd9d7-2h5vx    1/1      Running    0          4m25s
```

A single pod exists in the `cli-resources` project. The pod name might differ in your output.

37. Use the `describe` command to view the configuration and events for the pod. Specify the pod name from the previous step.

```
38.[student@workstation ~]$ oc describe pod myapp-54fcdcd9d7-2h5vx
39.Name:                myapp-54fcdcd9d7-2h5vx
40.Namespace:           cli-resources
41....output omitted...
```

```

42. Status:           Running
43. IP:               10.8.0.127
44. IPs:
45.   IP:             10.8.0.127
46. Controlled By:    ReplicaSet/myapp-54fcdcd9d7
47. Containers:
48.   myapp:
49.     Container ID:   cri-o://e0da...669d
50.     Image:          registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
51.     Image ID:       registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:91a
                        d...fd83
52. ...output omitted...
53. Events:
54.   Type      Reason             Age   From              Message
55.   ----      -
56.   Normal    Scheduled          10m   default-scheduler Successfully assigned c
cli-resources/myapp-54fcdcd9d7-2h5vx to master01

```

```
...output omitted...
```

57. Retrieve the details of the pod in a structured format. Use the `get` command and specify the output as the YAML format. Compare the results of the `describe` command versus the `get` command.

```

58. [student@workstation ~]$ oc get pod myapp-54fcdcd9d7-2h5vx -o yaml
59. apiVersion: v1
60. kind: Pod
61. metadata:
62.   annotations:
63. ...output omitted...
64.   labels:
65.     app: myapp
66.     pod-template-hash: 54fcdcd9d7
67.   name: myapp-54fcdcd9d7-2h5vx
68.   namespace: cli-resources

```

```

69. ...output omitted...
70. spec:
71.   containers:
72.   - image: registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
73.     imagePullPolicy: Always
74.     name: myapp
75.     resources: {}
76.     securityContext:

```

```
...output omitted...
```

Using a structured format with the `get` command provides more details about a resource than the `describe` command.

4. Identify and explain the available cluster resource types that the Kubernetes `apps` API group provides. Then, describe a resource from the `apps` API group in the `cli-resources` project.

1. List the resource types that the `apps` API group provides.

```

2. [student@workstation ~]$ oc api-resources --api-group apps
3. NAME                                SHORTNAMES  APIVERSION  NAMESPACED  KIND
4. controllerrevisions                 apps/v1     true        ControllerRevis
   ion
5. daemonsets                          ds          apps/v1     true        DaemonSet
6. deployments                         deploy      apps/v1     true        Deployment
7. replicaset                          rs          apps/v1     true        ReplicaSet

```

```
statefulsets      sts          apps/v1     true        StatefulSet
```

8. Use the `explain` command to list a description and fields for the `deployments` resource type.

```

9. [student@workstation ~]$ oc explain deployments
10. GROUP:      apps
11. KIND:       Deployment
12. VERSION:    apps/v1
13.

```

14. DESCRIPTION:

15. Deployment enables declarative updates for Pods and ReplicaSets.

16.

17. FIELDS:

18. apiVersion <string>

19. APIVersion defines the versioned schema of this representation of an

20. object. Servers should convert recognized schemas to the latest internal

21. value, and may reject unrecognized values. More info:

...output omitted...

22. Use the `get` command to identify any deployment resources in the `cli-resources` project.

23. [student@workstation ~]\$ `oc get deploy`

24. NAME	READY	UP-TO-DATE	AVAILABLE	AGE
----------	-------	------------	-----------	-----

myapp	1/1	1	1	25m
-------	-----	---	---	-----

25. The `myapp` deployment exists in the `cli-resources` project. Use the `get` command and the `-o wide` option to identify the container name and the container image in the deployment.

26. [student@workstation ~]\$ `oc get deploy myapp -o wide`

27. NAME	... CONTAINERS	IMAGES	SE
----------	----------------	--------	----

myapp	... myapp	registry.ocp4.example.com:8443/ubi8/httpd-24:1-215	app=myapp
-------	-----------	--	-----------

The `myapp` deployment uses the `registry.ocp4.example.com:8443/ubi8/httpd-24:1-215` container image for the `myapp` container.

28. Describe the `myapp` deployment to view more details about the resource.

29. [student@workstation ~]\$ `oc describe deployment myapp`

30. Name: myapp

```

31.Namespace:                cli-resources
32.CreationTimestamp:         Tue, 23 Sep 2025 18:41:39 -0500
33.Labels:                    my-app
34.Annotations:               deployment.kubernetes.io/revision: 1
35.Selector:                  app=myapp
36.Replicas:                   1 desired | 1 updated | 1 total | 1 available | 0 u
    navailable
37.StrategyType:              RollingUpdate
38.MinReadySeconds:           0
39.RollingUpdateStrategy:     25% max unavailable, 25% max surge
40.Pod Template:
41.  Labels:  app=myapp
42.  Containers:
43.    myapp:
44.      Image:      registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
45.      Port:       <none>
46.      Host Port:  <none>
47.      Environment: <none>
48.      Mounts:      <none>
49.      Volumes:      <none>
50.      Node-Selectors: <none>
51.      Tolerations:  <none>
52.Conditions:
53.  Type          Status  Reason
54.  ----          -
55.  Available      True    MinimumReplicasAvailable
56.  Progressing    True    NewReplicaSetAvailable
57.OldReplicaSets: <none>
58.NewReplicaSet:  myapp-54fcdcd9d7 (1/1 replicas created)
59.Events:
60.  Type          Reason          Age    From          Message
61.  ----          -

```



```
Normal ScalingReplicaSet 30m deployment-controller Scaled up replica
set myapp-54fcdcd9d7 to 1
```

5. Identify and explain the available cluster resource types that the OpenShift configuration API group provides. Then, describe a resource from the OpenShift configuration API group.
1. List the resource types that the OpenShift configuration API group provides.

```
2. [student@workstation ~]$ oc api-resources --api-group config.openshift.io
```

3. NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
4. apiservers		config.openshift.io/v1	false	APIServer
5. authentications		config.openshift.io/v1	false	Authentication
6. builds		config.openshift.io/v1	false	Build
7. clusteroperators	co	config.openshift.io/v1	false	ClusterOperator
8. clusterversions		config.openshift.io/v1	false	ClusterVersion
9. consoles		config.openshift.io/v1	false	Console
10. dnses		config.openshift.io/v1	false	DNS
11. featuregates		config.openshift.io/v1	false	FeatureGate
12. imagecontentpolicies		config.openshift.io/v1	false	ImageContentPo
13. imagedigestmirrorsets	idms	config.openshift.io/v1	false	ImageDigestMir
14. images		config.openshift.io/v1	false	Image
15. imagetagmirrorsets	itms	config.openshift.io/v1	false	ImageTagMirror
16. infrastructures		config.openshift.io/v1	false	Infrastructure
17. ingresses		config.openshift.io/v1	false	Ingress
18. networks		config.openshift.io/v1	false	Network
19. nodes		config.openshift.io/v1	false	Node
20. oauths		config.openshift.io/v1	false	OAuth
21. operatorhubs		config.openshift.io/v1	false	OperatorHub
22. projects		config.openshift.io/v1	false	Project
23. proxies		config.openshift.io/v1	false	Proxy

schedulers		config.openshift.io/v1	false	Scheduler
------------	--	------------------------	-------	-----------

The config.openshift.io API group provides multiple, non-namespaced resource types.

24. Use the `explain` command to list a description and fields for the `projects` resource type.

```
25. [student@workstation ~]$ oc explain projects
26. GROUP:      project.openshift.io
27. KIND:       Project
28. VERSION:    v1
29.
30. DESCRIPTION:
31.     Projects are the unit of isolation and collaboration in OpenShift. A
32.     project has one or more members, a quota on the resources that the pro
    ject
33.     may consume, and the security controls on the resources in the project
    .
34.     Within a project, members may have different roles - project administr
    ators
35.     can set membership, editors can create and manage the resources, and
36.     viewers can see but not access running containers. In a normal cluster
37.     project administrators are not able to alter their quotas - that is
38.     restricted to cluster administrators.
39.
40.     Listing or watching projects will return only projects the user has th
    e
41.     reader role on.
```

...output omitted...

42. Describe the `cli-resources` project.

```
43. [student@workstation ~]$ oc describe project cli-resources
44. Name:                cli-resources
45. Created:              10 minutes ago
46. Labels:              kubernetes.io/metadata.name=cli-resources
47.                     pod-security.kubernetes.io/audit=restricted
48.                     pod-security.kubernetes.io/audit-version=latest
```

```
49.          pod-security.kubernetes.io/warn=restricted
50.          pod-security.kubernetes.io/warn-version=latest
51. Annotations:  openshift.io/description=
52.              openshift.io/display-name=
53.              openshift.io/requester=system:admin
54.              openshift.io/sa.scc.mcs=s0:c26,c25
55.              openshift.io/sa.scc.supplemental-groups=1000710000/10000
56.              openshift.io/sa.scc.uid-range=1000710000/10000
57. Display Name:  <none>
58. Description:   <none>
59. Status:        Active
60. Node Selector: <none>
61. Quota:         <none>
```

```
Resource limits: <none>
```

62. Retrieve more details of the `cli-resources` project. Use the `get` command, and format the output to use JSON.

```
63. [student@workstation ~]$ oc get project cli-resources -o json
64. {
65.   "apiVersion": "project.openshift.io/v1",
66.   "kind": "Project",
67.   "metadata": {
68.     ...output omitted...
69.     "labels": {
70.       "kubernetes.io/metadata.name": "cli-resources",
71.       "pod-security.kubernetes.io/audit": "restricted",
72.       "pod-security.kubernetes.io/audit-version": "latest",
73.       "pod-security.kubernetes.io/warn": "restricted",
74.       "pod-security.kubernetes.io/warn-version": "latest"
75.     },
76.     "name": "cli-resources",
77.     "resourceVersion": "705313",
78.     "uid": "53cbbe45-31ea-4b41-93a9-4ba5c2c4c1f3"
```

```

79.     },
80. ...output omitted...
81.     "status": {
82.         "phase": "Active"
83.     }

```

```
}

```

The `get` command provides additional details, such as the `kind` and `apiVersion` attributes, of the project resource.

6. Verify the cluster status by inspecting cluster services. Format command outputs by using filters.
1. Retrieve the list of pods for the Etcd operator. The Etcd operator is available in the `openshift-etcd` namespace. Specify the namespace with the `--namespace` or `-n` option.

```
2. [student@workstation ~]$ oc get pods -n openshift-etcd

```

```
Error from server (Forbidden): pods is forbidden: User "developer" cannot list resource "pods" in API group "" in the namespace "openshift-etcd"

```

The `developer` user cannot access resources in the `openshift-etcd` namespace. Regular cluster users, such as the `developer` user, cannot query resources in the `openshift-` namespaces.

Log in as the `admin` user with the `redhatocp` password. Then, retrieve the list of pods in the `openshift-etcd` namespace.

```

[student@workstation ~]$ oc login -u admin -p redhatocp
Login successful
...output omitted...
[student@workstation ~]$ oc get pods -n openshift-etcd

```

NAME	READY	STATUS	RESTARTS	AGE
etcd-master01	5/5	Running	60	124d
installer-1-master01	0/1	Completed	0	124d
installer-2-master01	0/1	Completed	0	124d

3. Retrieve the image of the etcd-master01 pod in the openshift-etcd namespace. Use filters to limit the output to the .spec.containers attribute of the pod to get the first element. Compare the outputs of the JSONPath, the jq filters, and the Go template format.

```
4. [student@workstation ~]$ oc get pods etcd-master01 -n openshift-etcd \
5. -o=jsonpath='{.spec.containers[0].image}{"\n"}'
```

```
quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:24d9...ifca
[student@workstation ~]$ oc get pods -n openshift-etcd etcd-master01 \
-o json | jq .spec.containers[0].image
"quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:24d9...ifca"
[student@workstation ~]$ oc get pods -n openshift-etcd etcd-master01 \
-o go-template='{{(index .spec.containers 0).image}}'
quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:24d9b9d9d7fadacbc505c
849a1e4b390b2f0fcd452ad851b7cce21e8cf
```

6. Retrieve the condition status of the prometheus-k8s-0 pod in the openshift-monitoring namespace. Configure the output to use the YAML format, and then filter the output with the yq filter.

```
7. [student@workstation ~]$ oc get pods -n openshift-monitoring prometheus-k8s-
0 \
8. -o yaml | yq '.status.conditions'
9. - lastProbeTime: null
10. lastTransitionTime: "2025-09-23T13:30:26Z"
11. status: "True"
12. type: PodReadyToStartContainers
13. - lastProbeTime: null
14. lastTransitionTime: "2025-09-22T14:16:09Z"
15. status: "True"
16. type: Initialized
17. - lastProbeTime: null
18. lastTransitionTime: "2025-09-23T13:31:18Z"
19. status: "True"
20. type: Ready
21. - lastProbeTime: null
```

```

22. lastTransitionTime: "2025-09-23T13:31:18Z"
23. status: "True"
24. type: ContainersReady
25.- lastProbeTime: null
26. lastTransitionTime: "2025-05-22T11:13:27Z"
27. status: "True"

```

```
type: PodScheduled
```

28. Use the `get` command to retrieve detailed information for the pods in the `openshift-storage` namespace. Use the YAML format and custom columns to filter the output according to the following table:

Column title	Object
PodName	metadata.name
ContainerName	spec.containers[].name
Phase	status.phase
IP	status.podIP
Ports	spec.containers[].ports[].containerPort

```

29. [student@workstation ~]$ oc get pods -n openshift-storage \
30.   -o custom-columns=PodName:".metadata.name",\
31.   ContainerName:"spec.containers[].name",\
32.   Phase:"status.phase",\
33.   IP:"status.podIP",\
34.   Ports:"spec.containers[].ports[].containerPort"
35. PodName                                ContainerName    Phase    IP            Ports
36. lvms-operator-7fcd897cb-...            manager          Running  10.8.0.97     9443
37. vg-manager-z8g5k                       vg-manager       Running  10.8.0.101    8081

```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-resources
```

Assess the Health of an OpenShift Cluster

Objectives

- Query the health of essential cluster services and components.

Query Operator Conditions

Operators are important components of Red Hat OpenShift Container Platform (RHOCP). Operators automate the required tasks to maintain a healthy RHOCP cluster that would otherwise require human intervention. Operators are the preferred method of packaging, deploying, and managing services on the control plane.

Operators integrate with Kubernetes APIs and CLI tools such as `kubectl` and `oc` commands. Operators provide the means of monitoring applications, performing health checks, managing over-the-air (OTA) updates, and ensuring that applications remain in your specified state.

Because CRI-O and the Kubelet run on every node, almost every other cluster function can be managed on the control plane by using Operators. Components that are added to the control plane by using operators include critical networking and credential services.

Operators in RHOCP are managed by two different systems, depending on the purpose of the operator.

Cluster Version Operator (CVO)

Cluster operators perform cluster functions. These operators are installed by default, and the CVO manages them.

Cluster operators use a Kubernetes kind value of `clusteroperators`, and thus can be queried via `oc` or `kubectl` commands. As a user with the `cluster-admin` role, use the `oc get clusteroperators` command to list all the cluster operators.

```
[user@host ~]$ oc get clusteroperators
```

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SIN
authentication	4.18.6	True	False	False	3d1
h					
baremetal	4.18.6	True	False	False	38d
cloud-controller-manager	4.18.6	True	False	False	38d
cloud-credential	4.18.6	True	False	False	38d
cluster-autoscaler	4.18.6	True	False	False	38d
config-operator	4.18.6	True	False	False	38d
console	4.18.6	True	False	False	38d

...output omitted...

For more details about a cluster operator, use the `describe clusteroperators operator-name` command to view the field values that are associated with the operator, including the current status of the operator. The `describe` command provides a human-readable output format for a resource. As such, the output format might change with an RHOCP version update.

For an output format that is less likely to change with a version update, use one of the `-o` output options of the `get` command. For example, use the following `oc get clusteroperators dns -o yaml` command for the YAML-formatted output details for the `dns` operator.

```
[user@host ~]$ oc get clusteroperators dns -o yaml
```

```
apiVersion: config.openshift.io/v1
kind: ClusterOperator
metadata:
  annotations:
    ...output omitted...
status:
```



```

conditions:
- lastTransitionTime: "2025-07-14T13:55:21Z"
  message: DNS "default" is available.
  reason: AsExpected
  status: "True"
  type: Available
...output omitted...
relatedObjects:
- group: ""
  name: openshift-dns-operator
  resource: namespaces
...output omitted...
versions:
- name: operator
  version: 4.18.6
...output omitted...

```

Operator Lifecycle Manager (OLM) Operators

Optional add-on operators that the OLM manages can be made accessible for users to run in their applications.

As a user with the `cluster-admin` role, use the `get operators` command to list all the add-on operators.

```

[user@host ~]$ oc get operators

```

NAME	AGE
lvms-operator.openshift-storage	44d
metallb-operator.metallb-system	44d

You can likewise use the `describe` and `get` commands to query details about the fields that are associated with the add-on operators.

Operators use one or more pods to provide cluster services. You can find the namespaces for these pods under the `relatedObjects` section of the detailed output for the operator. As a user with a `cluster-admin` role, use the `-n namespace` option on

the `get pod` command to view the pods. For example, use the following `get pods` command to retrieve the list of pods in the `openshift-dns-operator` namespace.

```
[user@host ~]$ oc get pods -n openshift-dns-operator
```

NAME	READY	STATUS	RESTARTS	AGE
dns-operator-74bb989655-vgm7t	2/2	Running	18	56d

Use the `-o yaml` or `-o json` output formats to view or analyze more details about the pods. The resource conditions, which are found in the status for the resource, track the current state of the resource object. The following example uses the `jq` processor to extract the status values from the JSON output details for the `dns` pod.

```
[user@host ~]$ oc get pod -n openshift-dns-operator \
dns-operator-74bb989655-vgm7t -o json | jq .status
```

```
{
  "conditions": [
    {
      "lastProbeTime": null,
      "lastTransitionTime": "2025-07-14T16:04:38Z",
      "status": "True",
      "type": "PodReadyToStartContainers"
    },
    ...output omitted...
  ]
}
```

In addition to listing the pods of a namespace, you can also use the `--show-labels` option of the `get` command to print the labels used by the pods. The following example retrieves the pods and their labels in the `openshift-etcd` namespace.

```
[user@host ~]$ oc get pods -n openshift-etcd --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
etcd-master01	5/5	Running	50	56d	app=etcd,etcd=true,k8s-app=etcd,revision=2
installer-1-master01	0/1	Completed	0	56d	app=installer

installer-2-master01	0/1	Completed	0	56d	app=installer
----------------------	-----	-----------	---	-----	---------------

Examining Cluster Metrics

Another way to gauge the health of an RHOC cluster is to examine the compute resource usage of cluster nodes and pods. The `oc adm top` command provides this information. For example, to list the total memory and CPU usage of all pods in the cluster, you can use the `--sum` option with the command to print the sum of the resource usage.

```
[user@host ~]$ oc adm top pods -A --sum
```

NAMESPACE	NAME	CPU(cores)	MEMORY(bytes)
metallb-system	controller-...-fxhh4	2m	43Mi
metallb-system	metallb-...-t9pgn	1m	19Mi
...output omitted...			
openshift-storage	topolvm-node-f9rpf	7m	47Mi
openshift-storage	vg-manager-q6zhf	1m	21Mi
		-----	-----
		3121m	9123Mi

The `-A` option shows pods from all namespaces. Use the `-n namespace` option to filter the results to show the pods in a single namespace. Use the `--containers` option to display the resource usage of containers within a pod. For example, use the following command to list the resource usage of the containers in the `etcd-master01` pod in the `openshift-etcd` namespace.

```
[user@host ~]$ oc adm top pods etcd-master01 -n openshift-etcd --containers
```

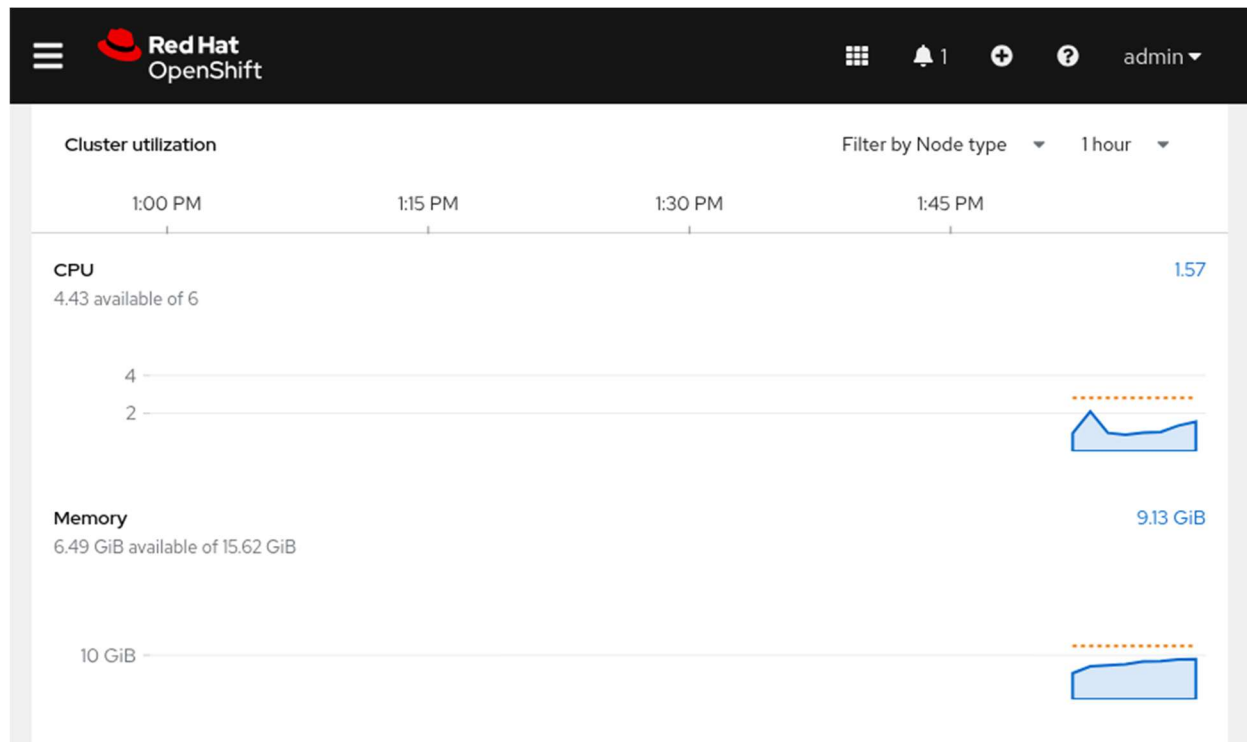
POD	NAME	CPU(cores)	MEMORY(bytes)
etcd-master01	etcd	72m	334Mi
etcd-master01	etcd-metrics	8m	31Mi
etcd-master01	etcd-readyz	3m	46Mi
etcd-master01	etcd-rev	1m	33Mi
etcd-master01	etcdctl	0m	0Mi

Viewing Cluster Metrics

The OpenShift web console incorporates graphs to visualize cluster and resource analytics. Cluster administrators and users with either the `view` or the `cluster-monitoring-view` cluster role can access the **Home** → **Overview** page. The Overview page displays a collection of cluster-wide metrics, and provides a high-level view of the overall health of the cluster.

The Overview page displays the following metrics:

- Current cluster capacity based on CPU, memory, storage, and network usage
- A time-series graph of total CPU, memory, and disk usage
- The ability to display the top consumers of CPU, memory, and storage

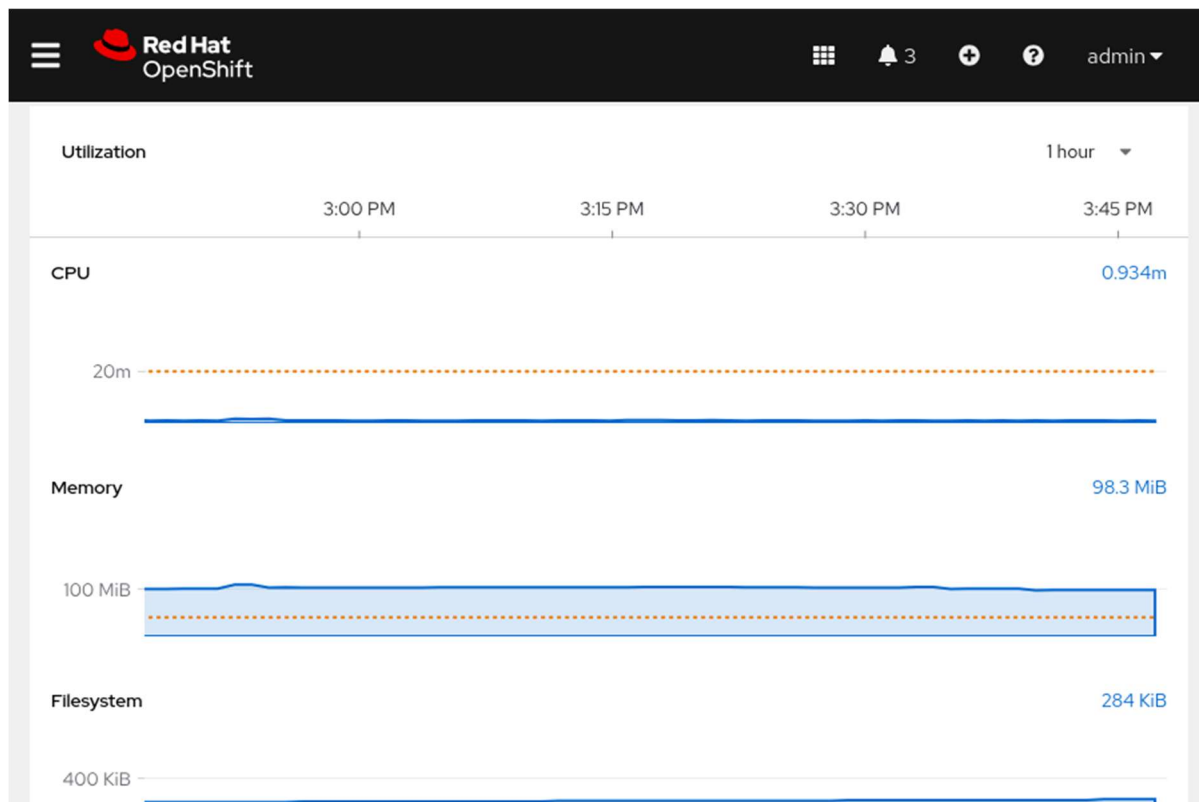


For any of the listed resources in the **Cluster Utilization** section, administrators can click the link for current resource usage. The link displays a window with a breakdown of top consumers for that resource. Top consumers can be sorted by project, by pod, or by node. The list of top consumers can be useful for identifying problematic pods or nodes. For example, a pod with an unexpected memory leak might appear at the top of the list.

Viewing Project Metrics

The **Project Details** page displays metrics that provide an overview of the resources that are used within the scope of a specific project.

The **Utilization** section displays usage information about resources, such as CPU and memory, along with the ability to display the top consumers for each resource.



All metrics are pulled from Prometheus. Click any graph to go to the **Metrics** page. View the executed query, and inspect the data further.

If a resource quota is created for the project, then the current project request and limits appear on the **Project Details** page.

Viewing Resource Metrics

When troubleshooting, it is often useful to view metrics at a smaller granularity than for the entire cluster or project. The **Metrics** page displays time-series graphs of the CPU, memory, and file-system usage for a specific pod. A sudden change in

these critical metrics, such as a CPU spike caused by high load, is visible on this page.

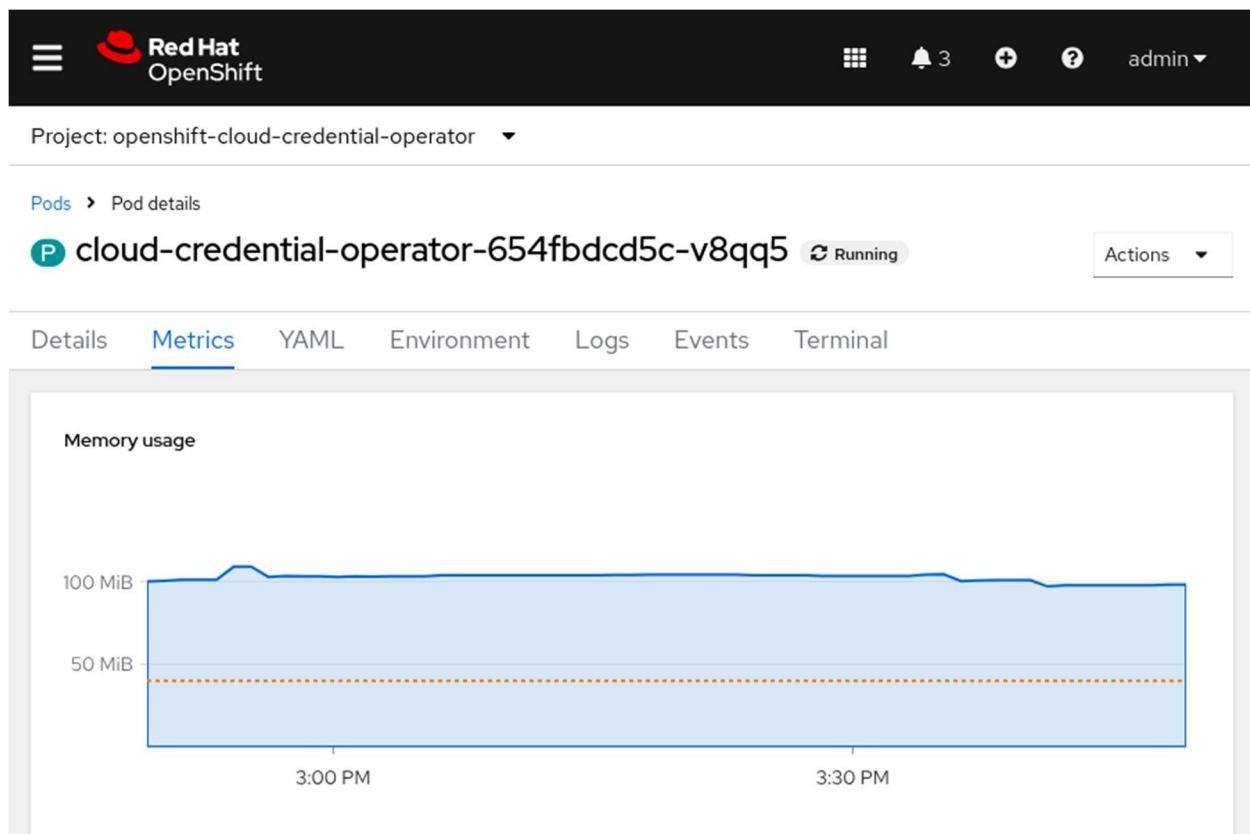


Figure 2.8: Time-series graphs showing various metrics for a pod

Performing Prometheus Queries in the Web Console

The Prometheus UI is a feature-rich tool for visualizing metrics and configuring alerts. The OpenShift web console provides an interface for executing Prometheus queries directly from the web console.

To perform a query, go to **Observe** → **Metrics**, enter a Prometheus Query Language expression in the text field, and click **Run Queries**. The results of the query are displayed as a time-series graph:

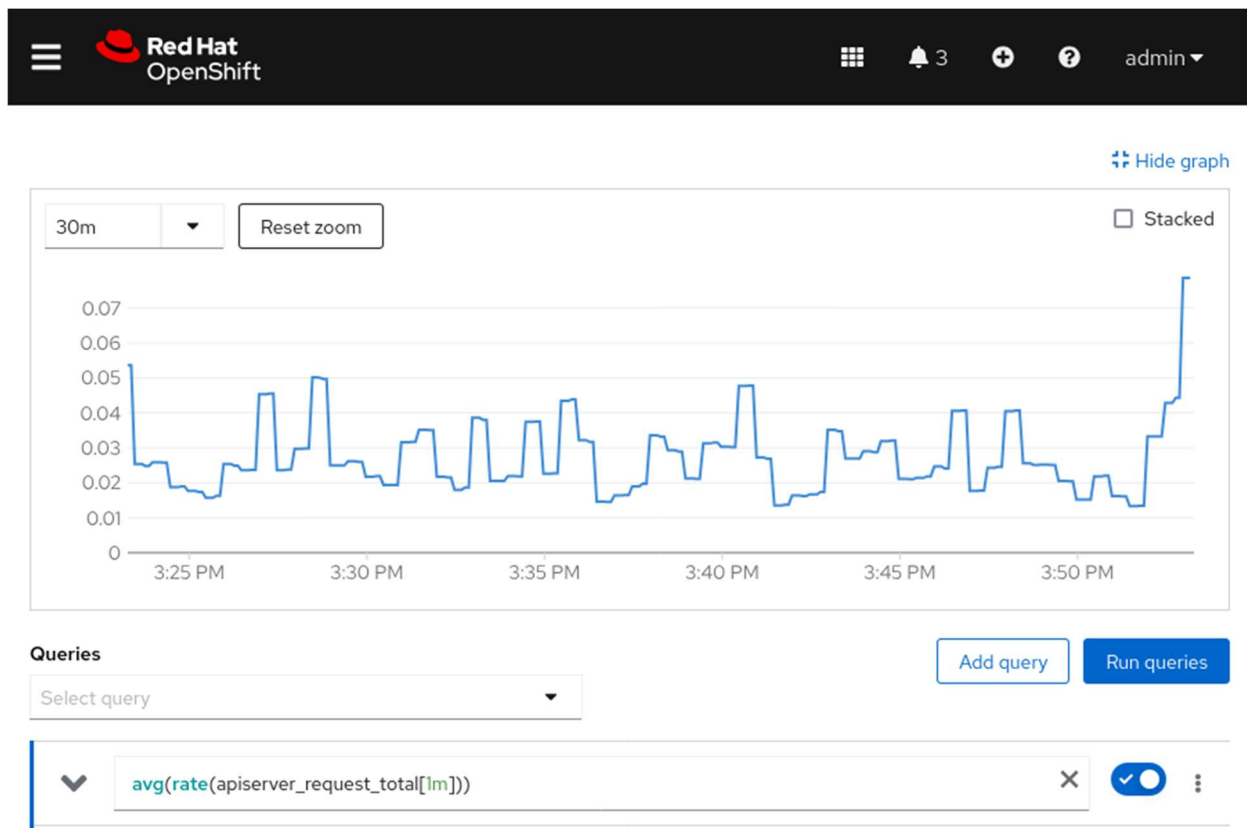


Figure 2.9: Using a Prometheus query to display a time-series graph

NOTE

The Prometheus Query Language is not discussed in detail in this course. Refer to the references section for a link to the official documentation.

Query Cluster Events and Alerts

Some developers consider OpenShift logs to be too low-level, thus making troubleshooting difficult. Fortunately, RHOCP provides a high-level logging and auditing facility called *events*. Kubernetes generates event objects in response to state changes in cluster objects, such as nodes, pods, and containers. Events signal significant actions, such as starting a container or destroying a pod.

To read events, use the `get events` command. The command lists the events for the current RHOCP project (namespace). You can display the events for a different project by adding the `-n namespace` option to the command. To list the events for all the projects, use the `-A` (or `--all-namespaces`) option.

NOTE

To sort the events by time, add the `--sort-by .metadata.creationTimestamp` option to the `oc get events` command.

The following `get events` command prints events in the `openshift-kube-controller-manager` namespace.

```
[user@host ~]$ oc get events -n openshift-kube-controller-manager
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
75d	Normal	LeaderElection	lease/cert-recovery-controller-lock	master01_
...				

...output omitted...

You can use the `describe pod pod-name` command to further narrow the results to a single pod. Refer to the `Events` field from the output of the `oc describe pod pod-name` command to review events for the specified pod.

Kubernetes Alerts

RHOCP includes a monitoring stack, which is based on the Prometheus open source project. The monitoring stack is configured to monitor the core RHOCP cluster components, by default. You can optionally configure the monitoring stack to also monitor user projects.

The components of the monitoring stack are installed in the `openshift-monitoring` namespace. Use the following `get all` command to display a list of all resources, their status, and their types in the `openshift-monitoring` namespace.

```
[user@host ~]$ oc get all -n openshift-monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
pod/alertmanager-main-0	6/6	Running	48	75d
pod/cluster-monitoring-operator-55b4dbf86-mb4xd	1/1	Running	8	75d
pod/kube-state-metrics-75455b796c-8q28d	3/3	Running	27	75d
pod/prometheus-k8s-0	6/6	Running	48	75d

...output omitted...

The Prometheus Operator in the openshift-monitoring namespace creates, configures, and manages the Prometheus and Alertmanager instances. The prometheus-k8s-0 entry is the Prometheus pod. A cluster administrator can use the following command to get the alerts from the Prometheus API.

```
[user@host ~]$ oc -n openshift-monitoring exec -c prometheus \
prometheus-k8s-0 -- curl -s 'http://localhost:9090/api/v1/alerts' | jq
...output omitted...
```

An Alertmanager pod in the openshift-monitoring namespace receives alerts from Prometheus. Alertmanager uses alert receivers to send alerts to external notification systems. The alertmanager-main-0 pod is the Alertmanager for the cluster. Use the curl command to retrieve the fired alerts from the Alertmanager API.

```
[user@host ~]$ oc -n openshift-monitoring exec -c alertmanager \
alertmanager-main-0 -- curl -s 'http://localhost:9093/api/v2/alerts' | jq
...output omitted...
```

Check Node Status

RHOC clusters can have several components, including at least one control plane and at least one compute node. The two components can occupy a single node. The following oc command, or the matching kubectl command, can display the overall health of all cluster nodes.

```
[user@host ~]$ oc cluster-info
```

The oc cluster-info output is high-level, and can verify that the cluster nodes are running. For a more detailed view into the cluster nodes, use the get nodes command.

```
[user@host ~]$ oc get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master01	Ready	control-plane,master,worker	75d	v1.31.6

The example shows a single `master01` node with multiple roles. The `STATUS` value of `Ready` means that this node is healthy and can accept new pods. A `STATUS` value of `NotReady` means that a condition triggered the `NotReady` status and the node is not accepting new pods.

As with any other RHOCP resource, you can drill down into further details of the node resource with the `describe node node-name` command. For parsable output of the same information, use the `-o json` or the `-o yaml` output options with the `get node node-name` command. For more information about using and parsing these output formats, see the section called “Inspect Kubernetes Resources”.

The output of the `get nodes node-name` command with the `-o json` or `-o yaml` option is long. The following examples use the `-jsonpath` option or the `jq` processor to parse the `get node node-name` command output.

```
[user@host ~]$ oc get node master01 -o jsonpath=\
*'{\"Allocatable:\n\"}{.status.allocatable}{\"\n\n\"}
{\"Capacity:\n\"}{.status.capacity}{\"\n\n\"}'

Allocatable:
{\"cpu\":\"5500m\", \"ephemeral-storage\":\"75691125429\", \"hugepages-1Gi\":\"0\",
\"hugepages-2Mi\":\"0\", \"memory\":\"15225452Ki\", \"pods\":\"250\"}

Capacity:
{\"cpu\":\"6\", \"ephemeral-storage\":\"83295212Ki\", \"hugepages-1Gi\":\"0\",
\"hugepages-2Mi\":\"0\", \"memory\":\"16376428Ki\", \"pods\":\"250\"}
```

The JSONPath expression in the previous command extracts the allocatable and capacity measures for the `master01` node. These measures help to understand the available resources on a node.

View the status object of a node to understand the current health of the node.

```
[user@host ~]$ oc get node master01 -o json | jq '.status.conditions'
[
  {
    \"lastHeartbeatTime\": \"2025-08-05T15:32:36Z\",
```

```
"lastTransitionTime": "2025-05-22T12:12:24Z",
"message": "kubelet has sufficient memory available",
"reason": "KubeletHasSufficientMemory",
"status": "False",

"type": "MemoryPressure"
},
{
  "lastHeartbeatTime": "2025-08-05T15:32:36Z",
  "lastTransitionTime": "2025-05-22T12:12:24Z",
  "message": "kubelet has no disk pressure",
  "reason": "KubeletHasNoDiskPressure",
  "status": "False",

  "type": "DiskPressure"
},
{
  "lastHeartbeatTime": "2025-08-05T15:32:36Z",
  "lastTransitionTime": "2025-05-22T12:12:24Z",
  "message": "kubelet has sufficient PID available",
  "reason": "KubeletHasSufficientPID",
  "status": "False",

  "type": "PIDPressure"
},
{
  "lastHeartbeatTime": "2025-08-05T15:32:36Z",
  "lastTransitionTime": "2025-08-05T14:41:42Z",
  "message": "kubelet is posting ready status",
  "reason": "KubeletReady",
  "status": "True",

  "type": "Ready"
}
```

If the status of the `MemoryPressure` condition is true, then the node is low on memory.

If the status of the `DiskPressure` condition is true, then the disk capacity of the node is low.

If the status of the `PIDPressure` condition is true, then too many processes are running on the node.

If the status of the `Ready` condition is false, then the node is not healthy and is not accepting pods.

More conditions indicate other potential problems with a node.

Table 2.5. Possible Node Conditions

Condition	Description
<code>OutOfDisk</code>	If true, then the node has insufficient free space on the node for adding new pods.
<code>NetworkUnavailable</code>	If true, then the network for the node is not correctly configured.
<code>NotReady</code>	If true, then one of the underlying components, such as the container runtime or network, is not working.
<code>SchedulingDisabled</code>	Pods cannot be scheduled for placement on the node.

To gain deeper insight into a given node, you can view the logs of processes that run on the node. A cluster administrator can use the `oc adm node-logs` command to view node logs. Node logs might contain sensitive output, and thus are limited to privileged node administrators. Use `oc adm node-logs node-name` to filter the logs to a single node.

The `oc adm node-logs` command has other options to further filter the results.

Table 2.6. Filters for `oc adm node-logs`

Option Example	Description
<code>--role master</code>	Use the <code>--role</code> option to filter the output to nodes with a specified role.
<code>-u kubelet</code>	The <code>-u</code> option filters the output to a specified unit.
<code>--path=cron</code>	The <code>--path</code> option filters the output to a specific process under the <code>/var/logs</code> directory.

Option Example	Description
<code>--tail 1</code>	Use <code>--tail x</code> to limit output to the last x log entries.

Use `oc adm node-logs --help` for a complete list of command options.

For example, to retrieve the most recent log entry for the `crio` service on the `master01` node, you can use the following command.

```
[user@host ~]$ oc adm node-logs master01 -u crio --tail 1
-- Logs begin at Thu 2023-02-09 21:19:09 UTC, end at Fri 2023-03-17 15:11:43 UTC. --
Aug 05 15:16:09.519642 master01 crio[2783]: time="2025-08-05 15:30:09.519474755Z" level=info msg="Stopped pod sandbox..."
...output omitted...
```

When you create a pod with the CLI, the `oc` or `kubectl` command is sent to the `apiserver` service, which then validates the command. The `scheduler` service reads the YAML or JSON pod definition, and then assigns pods to compute nodes. Each compute node runs a `kubelet` service that converts the pod manifest to one or more containers in the CRI-O container runtime.

Each compute node must have an active `kubelet` service and an active `crio` service. To verify the health of these services, first start a debug session on the node by using the `debug` command.

```
[user@host ~]$ oc debug node/node-name
```

Replace the `node-name` value with the name of your node.

NOTE

The `debug` command is covered in greater detail in a later section.

Within the debug session, change to the `/host` root directory so that you can run binaries in the host's executable path.

```
sh-4.4# chroot /host
```

Then, use the `systemctl is-active` calls to confirm that the services are active.

```
sh-4.4# for SERVICES in kubelet crio; do echo ---- $SERVICES ---- ;
systemctl is-active $SERVICES ; echo ""; done
---- kubelet ----
active

---- crio ----
active
```

For more details about the status of a service, use the `systemctl status` command.

```
sh-4.4# systemctl status kubelet
• kubelet.service - Kubernetes Kubelet
   Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: disabled)
   Drop-In: /etc/systemd/system/kubelet.service.d
   ...output omitted...
```

Check Pod Status

With RHOCP, you can view logs in running containers and pods to ease troubleshooting. When a container starts, RHOCP redirects the container's standard output and standard error to a disk in the container's ephemeral storage. With this redirect, you can view the container logs by using `logs` commands, even after the container stops. However, the pod hosting the container must still exist.

In RHOCP, the following command returns the output for a container within a pod:

```
[user@host ~]$ oc logs pod-name -c container-name
```

Replace *pod-name* with the name of the target pod, and replace *container-name* with the name of the target container. The `-c container-name` argument is optional, if the pod has only one container. You must use the `-c container-name` argument to connect to a

specific container in a multicontainer pod. Otherwise, the command defaults to the only running container and returns the output.

When debugging images and setup problems, it is useful to get an exact copy of a running pod configuration, and then troubleshoot it with a shell. If a pod is failing or does not include a shell, then the `rsh` and `exec` commands might not work. To resolve this issue, the `debug` command creates a copy of the specified pod and starts a shell in that pod.

By default, the `debug` command starts a shell inside the first container of the referenced pod. The debug pod is a copy of your source pod, with some additional modifications. For example, the pod labels are removed. The executed command is also changed to the `/bin/sh` command for Linux containers, or the `'cmd.exe'` executable for Windows containers. Additionally, readiness and liveness probes are disabled.

A common problem for containers in pods is security policies that prohibit a container from running as a root user. You can use the `debug` command to test running a pod as a non-root user by using the `--as-user` option. You can also run a non-root pod as the root user with the `--as-root` option.

With the `debug` command, you can invoke other types of objects besides pods. For example, you can use any controller resource that creates a pod, such as a deployment, a build, or a job. The `debug` command also works with nodes, and with resources that can create pods, such as image stream tags. You can also use the `--image=IMAGE` option of the `debug` command to start a shell session by using a specified image.

If you do not include a resource type and name, then the `debug` command starts a shell session into a pod by using the OpenShift tools image.

```
[user@host ~]$ oc debug
```

The next example tests running a job pod as a non-root user.

```
[user@host ~]$ oc debug job/test --as-user=1000000
```

The following example creates a debug session for a node.

```
[user@host ~]$ oc debug node/master01
Starting pod/master01-debug-gdmkp ...
To use host binaries, run chroot /host
Pod IP: 192.168.50.10
If you don't see a command prompt, try pressing enter.
sh-5.1# chroot /host
sh-5.1#
```

The debug pod is deleted when the remote command completes, or when the user interrupts the shell.

Collect Information for Support Requests

When opening a support case, it is helpful to provide debugging information about your cluster to Red Hat Support. It is recommended that you provide the following information:

- Data gathered by using the `oc adm must-gather` command as a cluster administrator
- The unique cluster ID

The `oc adm must-gather` command collects resource definitions and service logs from your cluster that are most likely needed for debugging issues. This command creates a pod in a temporary namespace on your cluster, and the pod then gathers and downloads debugging information. By default, the `oc adm must-gather` command uses the default plug-in image, and writes into the `./must-gather.local` directory on your local system. To write to a specific local directory, you can also use the `--dest-dir` option, such as in the following example:

```
[user@host ~]$ oc adm must-gather --dest-dir /home/student/must-gather
```

Then, create a compressed archive file from the `must-gather` directory. For example, on a Linux-based system, you can run the following command:

```
[user@host ~]$ tar cvaf mustgather.tar must-gather/
```

Replace `must-gather/` with the actual directory path.

Then, attach the compressed archive file to your support case in the Red Hat Customer Portal.

Similar to the `oc adm must-gather` command, the `oc adm inspect` command gathers information on a specified resource. For example, the following command collects debugging data for the `openshift-apiserver` and `kube-apiserver` cluster operators.

```
[user@host ~]$ oc adm inspect clusteroperator/openshift-apiserver \
clusteroperator/kube-apiserver
```

The `oc adm inspect` command can also use the `--dest-dir` option to specify a local directory to write the gathered information. The command shows all logs by default. Use the `--since` option to filter the results to logs that are later than a relative duration, such as 5s, 2m, or 3h.

```
[user@host ~]$ oc adm inspect clusteroperator/openshift-apiserver --since 10m
```

Guided Exercise: Assess the Health of an OpenShift Cluster

Verify the health of an OpenShift cluster by querying the status of its cluster operators, nodes, pods, and systemd services. Also verify cluster events and alerts.

Outcomes

- View the status and get information about cluster operators.
- Retrieve information about cluster pods and nodes.
- Retrieve the status of a node's systemd services.
- View cluster events and alerts.
- Retrieve debugging information for the cluster.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that all resources are available for this exercise.

```
[student@workstation ~]$ lab start cli-health
```

Instructions

1. Retrieve the status and view information about cluster operators.
1. Log in to the OpenShift cluster as the admin user with the redhatocp password.

```
2. [student@workstation ~]$ oc login -u admin -p redhatocp \
3. https://api.ocp4.example.com:6443
4. Login successful
```

...output omitted...

5. List the operators that users installed in the OpenShift cluster.

```
6. [student@workstation ~]$ oc get operators
7. NAME                                AGE
8. lvms-operator.openshift-storage    69d
```

```
metallb-operator.metallb-system    69d
```

9. List the cluster operators that are installed by default in the OpenShift cluster.

```
10. [student@workstation ~]$ oc get clusteroperators
11. NAME                                VERSION   AVAILABLE   PROGRESSING   DEGRADED
    ...
12. authentication                      4.18.6    True        False         False
    ...
13. baremetal                          4.18.6    True        False         False
    ...
14. cloud-controller-manager           4.18.6    True        False         False
    ...
15. cloud-credential                   4.18.6    True        False         False
    ...
16. cluster-autoscaler                 4.18.6    True        False         False
    ...
17. config-operator                    4.18.6    True        False         False
    ...
18. console                            4.18.6    True        False         False
    ...
19. control-plane-machine-set          4.18.6    True        False         False
    ...
```

20. csi-snapshot-controller	4.18.6	True	False	False
...				
21. dns	4.18.6	True	False	False
...				
22. etcd	4.18.6	True	False	False
...				

...output omitted...

23. Use the `describe` command to view detailed information about the `openshift-apiserver` cluster operator, such as related objects, events, and version.

```

24. [student@workstation ~]$ oc describe clusteroperators openshift-apiserver
25. Name:          openshift-apiserver
26. Namespace:
27. Labels:        <none>
28. Annotations:   exclude.release.openshift.io/internal-openshift-hosted: true
29.                include.release.openshift.io/self-managed-high-availability:
    true
30.                include.release.openshift.io/single-node-developer: true
31. API Version:   config.openshift.io/v1
32. Kind:          ClusterOperator
33. Metadata:
34. ...output omitted...
35. Spec:
36. Status:
37.  Conditions:
38.    Last Transition Time:  2025-05-22T10:49:18Z
39.    Message:              All is well
40.    Reason:               AsExpected
41.    Status:               False
42.    Type:                 Degraded
43. ...output omitted...
44.  Extension:             <nil>
45.  Related Objects:

```



```
openshift-apiserver-operator-85fc9669d-g58s1    1/1    Running    12
81d
[student@workstation ~]$ oc get pod -n openshift-apiserver-operator \
openshift-apiserver-operator-85fc9669d-g58s1 \
-o json | jq .status
{
  "conditions": [
...output omitted...
    {
      "lastProbeTime": null,
      "lastTransitionTime": "2025-08-11T17:52:20Z",
      "status": "True",
      "type": "Ready"
    },
...output omitted...
  ],
  "containerStatuses": [
    {
...output omitted...
      "name": "openshift-apiserver-operator",
      "ready": true,
      "restartCount": 12,
      "started": true,
      "state": {
        "running": {
          "startedAt": "2025-08-11T17:52:19Z"
        }
      }
    }
  ],
  "hostIP": "192.168.50.10",
...output omitted...
  "phase": "Running",
  "podIP": "10.8.0.17",
```

```
...output omitted...  
}
```

2. Retrieve the status, resource consumption, and events of cluster pods.
1. List the memory and CPU usage of all pods in the cluster. Use the --sum option to print the sum of the resource usage. The resource usage on your system probably differs.

```
2. [student@workstation ~]$ oc adm top pods -A --sum
```

3. NAMESPACE	NAME	CPU(cores)	MEMORY(bytes)
--------------	------	------------	---------------

```
4. ...output omitted...
```

5. metallb-system	metallb-operator-controller-manager-...	2m	58Mi
-------------------	---	----	------

6. metallb-system	metallb-operator-webhook-server-...	1m	7Mi
-------------------	-------------------------------------	----	-----

7. metallb-system	speaker-nrvz4	4m	8Mi
-------------------	---------------	----	-----

```
8. ...output omitted...
```

8982Mi	505m
--------	------

9. List the pods and their labels in the openshift-etcd namespace.

```
10. [student@workstation ~]$ oc get pods -n openshift-etcd --show-labels
```

11. NAME	READY	STATUS	RESTARTS	AGE	LABELS
12. etcd-master01	5/5	Running	55	81d	app=etcd,etcd=t
13. installer-1-master01	0/1	Completed	0	81d	app=installer

installer-2-master01	0/1	Completed	0	81d	app=installer
----------------------	-----	-----------	---	-----	---------------

14. List the resource usage of the containers in the etcd-master01 pod in the openshift-etcd namespace. The resource usage on your system probably differs.

```
15. [student@workstation ~]$ oc adm top pods etcd-master01 \
```

```
16. -n openshift-etcd --containers
```

17. POD	NAME	CPU(cores)	MEMORY(bytes)
---------	------	------------	---------------

18. etcd-master01	etcd	44m	269Mi
19. etcd-master01	etcd-metrics	8m	30Mi
20. etcd-master01	etcd-readyz	4m	47Mi
21. etcd-master01	etcd-rev	1m	44Mi

etcd-master01	etcdctl	0m	0Mi
---------------	---------	----	-----

22. Display a list of all resources, their status, and their types in the openshift-monitoring namespace.

23. [student@workstation ~]\$ **oc get all -n openshift-monitoring**

24. Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14+, unavailable in v4.10.0+

25. NAME	READY	STATUS
...		
26. pod/alertmanager-main-0	6/6	Running
g ...		
27. pod/cluster-monitoring-operator-56b769b58f-dtmqj	1/1	Running
g ...		
28. pod/kube-state-metrics-75455b796c-8q28d	3/3	Running
g ...		

29. ...output omitted...

30. NAME	TYPE	CLUSTER-IP
...		
31. service/alertmanager-main	ClusterIP	172.30.90.122
...		
32. service/alertmanager-operated	ClusterIP	None
...		
33. service/cluster-monitoring-operator	ClusterIP	None
...		
34. service/kube-state-metrics	ClusterIP	None
...		

...output omitted...

35. View the logs of the alertmanager-main-0 pod in the openshift-monitoring namespace. The logs might differ on your system.

36. [student@workstation ~]\$ **oc logs alertmanager-main-0 -n openshift-monitoring**

37. ...output omitted...

38. ts=2025-08-11T17:50:17.481Z caller=coordinator.go:113 level=info component=configuration msg="Loading configuration file" file=/etc/alertmanager/config_out/alertmanager.env.yaml

39. ts=2025-08-11T17:50:17.483Z caller=coordinator.go:126 level=info component=configuration msg="Completed loading of configuration file" file=/etc/alertmanager/config_out/alertmanager.env.yaml

...output omitted...

40. Retrieve the events for the openshift-kube-controller-manager namespace.

41. [student@workstation ~]\$ **oc get events -n openshift-kube-controller-manager**

42. LAST SEEN	TYPE	REASON	OBJECT
...			
43. 169m	Normal	LeaderElection	lease/cert-recovery-controller-lock
...			
44. 169m k...	Normal	LeaderElection	lease/cluster-policy-controller-lock
45. 172m 1...	Normal	Pulled	pod/kube-controller-manager-master01

...output omitted...

3. Retrieve information about cluster nodes.

1. View the status of the nodes in the cluster.

2. [student@workstation ~]\$ **oc get nodes**

3. NAME	STATUS	ROLES	AGE	VERSION
---------	--------	-------	-----	---------

master01	Ready	control-plane,master,worker	81d	v1.31.6
----------	-------	-----------------------------	-----	---------

4. Retrieve the resource consumption of the master01 node. The resource usage on your system probably differs.

5. [student@workstation ~]\$ **oc adm top node**

6. NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
---------	------------	------	---------------	---------

master01	719m	13%	12069Mi	81%
----------	------	-----	---------	-----

7. Use a JSONPath filter to determine the capacity and allocatable CPU for the master01 node. The values might differ on your system.

```
8. [student@workstation ~]$ oc get node master01 -o jsonpath=\
9. 'Allocatable: {.status.allocatable.cpu}{"\n"}'\
10. 'Capacity: {.status.capacity.cpu}{"\n"}'\
11. Allocatable: 5500m
```

Capacity: 6

12. Determine the number of allocatable pods for the node.

```
13. [student@workstation ~]$ oc get node master01 -o jsonpath=\
14. 'Allocatable Pods: {.status.allocatable.pods}{"\n"}'
```

Allocatable Pods: 250

15. Use the describe command to view the events, resource requests, and resource limits for the node. The output might differ on your system.

```
16. [student@workstation ~]$ oc describe node master01
17. ...output omitted...
18. Allocated resources:
19.   (Total limits may be over 100 percent, i.e., overcommitted.)
20. Resource           Requests          Limits
21.  -----
22.  cpu                 2809m (51%)      10m (0%)
23.  memory              12359Mi (83%)    0 (0%)
24.  ephemeral-storage   0 (0%)           0 (0%)
25.  hugepages-1Gi       0 (0%)           0 (0%)
26.  hugepages-2Mi       0 (0%)           0 (0%)
```

Events: <none>

4. Retrieve the logs and status of the systemd services on the master01 node.

1. Display the logs of the node. Filter the logs to show the most recent log for the crio service. The logs might differ on your system.

```
2. [student@workstation ~]$ oc adm node-logs master01 -u crio --tail 1
```

3. -- Logs begin at Thu 2025-05-22 10 21:19:09 UTC, end at Thu 2025-05-22 10 16:57:00 UTC. --
4. May 22 10:46:12.108535 master01 systemd[1]: Starting Container Runtime Interface for OCI (CRI-O)...
5. May 22 10:46:12.655846 master01 crio[2698]: time="2025-05-22 10:46:12.655729 195Z" level=info msg="Updating config from single file: /etc/crio/crio.conf"

...output omitted...

6. Display the two most recent log entries of the kubelet service on the node. The logs might differ on your system.

7. [student@workstation ~]\$ **oc adm node-logs master01 -u kubelet --tail 2**
8. -- Logs begin at Thu 2025-05-22 10 21:19:09 UTC, end at Thu 2025-05-22 10 16:57:00 UTC. --
9. Aug 11 20:45:05.086582 master01 kubenswrapper[3028]: I0811 20:45:05.086523 3028 kubelet_volumes.go:163] "Cleaned up orphaned pod volumes dir" podUID="a1c8c4ef-7604-426f-bc88-682cc4fb0d5e" path="/var/lib/kubelet/pods/a1c8c4ef-7604-426f-bc88-682cc4fb0d5e/volumes"

Aug 11 20:45:21.367632 master01 kubenswrapper[3028]: I0811 20:45:21.367580 3028 scope.go:117] "RemoveContainer" containerID="72ef...8dd0"

10. Create a debug session for the node. Then, use the chroot /host command to access the host binaries.

11. [student@workstation ~]\$ **oc debug node/master01**
12. Starting pod/master01-debug-945fs ...
13. To use host binaries, run `chroot /host`
14. Pod IP: 192.168.50.10
15. If you don't see a command prompt, try pressing enter.
16. sh-5.1# **chroot /host**

sh-5.1#

17. Verify the status of the kubelet service.

18. sh-5.1# **systemctl status kubelet**
19. ● kubelet.service - Kubernetes Kubelet

```
20. Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; preset: disabled)
21. Drop-In: /etc/systemd/system/kubelet.service.d
22.          └─01-kubens.conf, 10-mco-default-madv.conf, 20-logging.conf, 20-nodenet.conf
23. Active: active (running) since Mon 2025-08-11 17:47:14 UTC; 3h 10min ago
24. Main PID: 3028 (kubelet)
25.   Tasks: 35 (limit: 127707)
26.  Memory: 779.7M
27.    CPU: 27min 9.454s
```

...output omitted...

Press **Ctrl+C** to quit the command.

28. Confirm that the `crio` service is active.

```
29. sh-5.1# systemctl is-active crio
```

active

30. Exit the debug pod.

```
31. sh-5.1# exit
32. exit
33. sh-5.1# exit
34. exit
35.
```

Removing debug pod ...

5. Retrieve debugging information for the cluster.

1. Retrieve debugging information of the cluster by using the `oc adm must-gather` command. Specify the `/home/student/must-gather` directory as the destination directory. This command might take several minutes to complete.

```
2. [student@workstation ~]$ oc adm must-gather --dest-dir /home/student/must-gather
```

3. [must-gather] OUT 2025-08-12T16:37:58.163228054Z Using must-gather plug-in image: quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:a81f...b12a
4. When opening a support case, bugzilla, or issue please include the following summary data along with any other requested information:
5. ClusterID: 6c8c6eed-26ed-4911-9df9-b081404842c8
6. ClientVersion: 4.18.6
7. ClusterVersion: Stable at "4.18.6"
8. ClusterOperators:

All healthy and stable

9. Verify that the debugging information exists in the destination directory.

10. [student@workstation ~]\$ ls -l ~/must-gather
11. event-filter.html
12. quay-io-openshift-release-dev-ocp-v4-0-art-dev-sha256...
13. must-gather.logs

timestamp

14. List the last five kubelet service logs, and confirm that an error occurred. Replace quay-io... with the generated directory name.

15. [student@workstation ~]\$ tail -5 ~/must-gather/\
16. quay-io-openshift-release-dev-ocp-v4-0-art-dev-sha256...\gather.logs
17. 2025-08-14T22:11:20.510531824Z Gathering data for ns/openshift-cluster-csi-drivers...
18. 2025-08-14T22:11:21.166365975Z Wrote inspect data to must-gather.
19. 2025-08-14T22:11:21.166420106Z **error: inspection completed with the errors occurred while gathering data:**
20. 2025-08-14T22:11:21.166420106Z [skipping gathering secrets/support due to error: secrets "support" not found, skipping gathering endpoints/host-etcd-2 due to error: endpoints "host-etcd-2" not found]

2025-08-14T22:11:21.352314360Z error: the server doesn't have a resource type "clusters"

21. Generate debugging information for the openshift-apiserver cluster operator. Specify the /home/student/inspect directory as the destination directory. Limit the debugging information to the last five minutes.

```
22. [student@workstation ~]$ oc adm inspect clusteroperator/openshift-apiserver \
23.   --dest-dir /home/student/inspect --since 5m
24. Gathering data for ns/openshift-config...
25. Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14+, una
    vailable in v4.10000+
26. Gathering data for ns/openshift-config-managed...
27. Gathering data for ns/openshift-apiserver-operator...
28. Gathering data for ns/openshift-apiserver...
29. Gathering data for ns/openshift-etcd-operator...
30. Wrote inspect data to /home/student/inspect.
```

...output omitted...

31. Verify that the debugging information exists in the destination directory.

```
32. [student@workstation ~]$ ls -l inspect/
33. cluster-scoped-resources
34. event-filter.html
35. namespaces
```

timestamp

36. Review the cluster.yaml file from the ~/inspect/cluster-scoped-resources/operator.openshift.io/openshiftapiservers directory.

```
37. [student@workstation ~]$ cat \
38. ~/inspect/cluster-scoped-resources/operator.openshift.io/\
39. openshiftapiservers/cluster.yaml
40. apiVersion: operator.openshift.io/v1
41. kind: OpenShiftAPIServer
42. metadata:
43.   annotations:
44.     include.release.openshift.io/hypershift: "true"
```

```
45.   include.release.openshift.io/ibm-cloud-managed: "true"
46.   include.release.openshift.io/self-managed-high-availability: "true"
47.   include.release.openshift.io/single-node-developer: "true"
48.   release.openshift.io/create-only: "true"
49.   creationTimestamp: "2025-05-22T10:40:34Z"
50.   generation: 3
51.   managedFields:
52.     - apiVersion: operator.openshift.io/v1
53.       fieldsType: FieldsV1
54.       fieldsV1:
55.         f:status:
56.           f:conditions:
```

...output omitted...

57.Delete the debugging information from your system.

```
[student@workstation ~]$ rm -rf must-gather inspect
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-health
```

Lab: Kubernetes and OpenShift Command-line Interfaces and APIs

Find detailed information about your OpenShift cluster and assess its health by querying its Kubernetes resources.

Outcomes

- Use the command line to retrieve information about the cluster resources.
- Identify cluster operators and API resources.
- List the available namespaced resources.
- Identify the resources that belong to the core API group.
- List the resource types that the `oauth.openshift.io` API group provides.
- List the resource usage of containers in a pod.
- Use the JSONPath filter to get the number of allocatable pods and compute resources for a node.
- List the memory and CPU usage of all pods in the cluster.
- Use `jq` filters to retrieve the conditions status of a pod.
- View cluster events and alerts.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

```
[student@workstation ~]$ lab start cli-review
```

Instructions

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

Log in to the OpenShift cluster as the `developer` user with the `developer` password. Use the `cli-review` project for your work.

1. Log in to the OpenShift cluster and create the `cli-review` project.
1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
```

...output omitted...

4. Create the `cli-review` project.

```
5. [student@workstation ~]$ oc new-project cli-review
6. Now using project "cli-review" on server "https://api.ocp4.example.com:6443"
.
```

...output omitted...

2. Use the `oc` command to list the following information for the cluster:

- Retrieve the cluster version.
- Identify the supported API versions.
- Identify the fields for the `pod.spec.securityContext` object.

4. Identify the cluster version.

```
5. [student@workstation ~]$ oc version
```

```
6. Client Version: 4.18.6
```

```
7. Kustomize Version: v5.4.2
```

```
Kubernetes Version: v1.31.6
```

8. Identify the supported API versions.

```
9. [student@workstation ~]$ oc api-versions
```

```
10. admissionregistration.k8s.io/v1
```

```
11. admissionregistration.k8s.io/v1beta1
```

```
12. apiextensions.k8s.io/v1
```

```
13. apiregistration.k8s.io/v1
```

```
14. apiserver.openshift.io/v1
```

```
15. apps.openshift.io/v1
```

```
16. apps/v1
```

```
...output omitted...
```

17. Identify the fields for the `pod.spec.securityContext` object.

```
18. [student@workstation ~]$ oc explain pod.spec.securityContext
```

```
19. KIND:      Pod
```

```
20. VERSION:   v1
```

```
21.
```

```
22. FIELD: securityContext <PodSecurityContext>
```

```
23.
```

```
24. DESCRIPTION:
```

```
...output omitted...
```


3. From the terminal, log in to the OpenShift cluster as the `admin` user with the `redhatocp` password. Then, use the command line to identify the following cluster resources:
 - List the cluster operators.
 - Identify the available namespaced resources.
 - Identify the resources that belong to the core API group.
 - List the resource types that the `oauth.openshift.io` API group provides.
 - List the events in the `openshift-kube-controller-manager` namespace.
5. Log in to the OpenShift cluster.

```
6. [student@workstation ~]$ oc login -u admin -p redhatocp \
7. https://api.ocp4.example.com:6443
```

...output omitted...

8. List the cluster operators.

```
9. [student@workstation ~]$ oc get clusteroperators
```

10. NAME SINCE	VERSION	AVAILABLE	PROGRESSING	DEGRADED
11. authentication 12h	4.18.6	True	False	False
12. baremetal 31d	4.18.6	True	False	False
13. cloud-controller-manager 31d	4.18.6	True	False	False
14. cloud-credential 31d	4.18.6	True	False	False
15. cluster-autoscaler 31d	4.18.6	True	False	False
16. config-operator 31d	4.18.6	True	False	False
17. console 31d	4.18.6	True	False	False

...output omitted...

18. List the available namespaced resources.

```
19. [student@workstation ~]$ oc api-resources --namespaced
```

20. NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
----------	------------	------------	------------	------

21. bindings		v1	true	Binding
22. configmaps	cm	v1	true	ConfigMap
23. endpoints	ep	v1	true	Endpoints
24. events	ev	v1	true	Event
25. limitranges	limits	v1	true	LimitRange
26. persistentvolumeclaims	pvc	v1	true	PersistentVolum
eClaim				
27. pods	po	v1	true	Pod

...output omitted...

28. Identify the resources that belong to the core API group.

29. [student@workstation ~]\$ oc api-resources --api-group ''				
30. NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
31. bindings		v1	true	Binding
32. componentstatuses	cs	v1	false	ComponentSt
atus				
33. configmaps	cm	v1	true	ConfigMap
34. endpoints	ep	v1	true	Endpoints
35. events	ev	v1	true	Event
36. limitranges	limits	v1	true	LimitRange
37. namespaces	ns	v1	false	Namespace
38. nodes	no	v1	false	Node

...output omitted...

39. List the resource types that the oauth.openshift.io API group provides.

40. [student@workstation ~]\$ oc api-resources --api-group oauth.openshift.io				
41. NAME	...	APIVERSION	NAMESPACED	KIND
42. oauthaccess tokens	oauth.openshift.io/v1	false		OAuthAccessToken
43. oauthauthorization tokens	oauth.openshift.io/v1	false		OAuthAuthorizati
...				

...output omitted...

44. Retrieve the events for the openshift-kube-controller-manager namespace.

```
45. [student@workstation ~]$ oc get events -n openshift-kube-controller-manager
```

```
46. LAST SEEN TYPE      REASON              OBJECT
    ...
```

...output omitted...

4. Identify the following information about the cluster services and its nodes:

- Retrieve the conditions status of the etcd-master01 pod in the openshift-etcd namespace by using jq filters to limit the output.
- List the compute resource usage of the containers in the etcd-master01 pod in the openshift-etcd namespace.
- Get the number of allocatable pods for the master01 node by using a JSONPath filter.
- List the memory and CPU usage of all pods in the cluster.
- Retrieve the compute resource consumption of the master01 node.
- Retrieve the capacity and allocatable CPU for the master01 node by using a JSONPath filter.

6. Retrieve the conditions status of the etcd-master01 pod in the openshift-etcd namespace. Use jq filters to limit the output to the .status.conditions attribute of the pod.

```
7. [student@workstation ~]$ oc get pods etcd-master01 -n openshift-etcd \
```

```
8.  -o json | jq .status.conditions
```

```
9. [
```

```
10.  {
```

```
11.    "lastProbeTime": null,
```

```
12.    "lastTransitionTime": "2023-03-12T16:40:35Z",
```

```
13.    "status": "True",
```

```
14.    "type": "Initialized"
```

```
15.  },
```

```
16.  {
```

```
17.    "lastProbeTime": null,
```

```
18.    "lastTransitionTime": "2023-03-12T16:40:47Z",
```

```
19.    "status": "True",
```

```
20.    "type": "Ready"
```

```
21.  },
```

```

22.  {
23.    "lastProbeTime": null,
24.    "lastTransitionTime": "2023-03-12T16:40:47Z",
25.    "status": "True",
26.    "type": "ContainersReady"
27.  },
28.  {
29.    "lastProbeTime": null,
30.    "lastTransitionTime": "2023-03-12T16:40:23Z",
31.    "status": "True",
32.    "type": "PodScheduled"
33.  }

```

```
]

```

34. List the resource usage of the containers in the `etcd-master01` pod in the `openshift-etcd` namespace.

```

35. [student@workstation ~]$ oc adm top pods etcd-master01 \
36.   -n openshift-etcd --containers
37. POD                NAME                CPU(cores)  MEMORY(bytes)
38. etcd-master01      etcd                101m       323Mi
39. etcd-master01      etcd-metrics        8m         32Mi
40. etcd-master01      etcd-readyz         3m         46Mi
41. etcd-master01      etcd-rev            1m         33Mi

```

```
etcd-master01  etcdctl          0m          0Mi

```

42. Use a JSONPath filter to determine the number of allocatable pods for the `master01` node.

```

43. [student@workstation ~]$ oc get node master01 \
44.   -o jsonpath='{.status.allocatable.pods}{"\n"}'

```

```
250

```

45. List the memory and CPU usage of all pods in the cluster. Use the `--sum` option to print the sum of the resource usage. The resource usage on your system probably differs.

```
46. [student@workstation ~]$ oc adm top pods -A --sum
```

```
47. NAMESPACE          NAME                                CPU(cores)  MEMORY
    (bytes)
48. metallb-system     controller-5f6dfd8c4f-ddr8v        0m          56Mi
49. metallb-system     metallb-operator-controller-manager-... 0m          50Mi
50. metallb-system     metallb-operator-webhook-server-...    0m          26Mi
51. metallb-system     speaker-2dds4                      9m          210Mi
52. ...output omitted...
53.
    --
```

```

                                         386m      11929
Mi
```

54. Retrieve the resource consumption of the `master01` node.

```
55. [student@workstation ~]$ oc adm top node
```

```
56. NAME          CPU(cores)  CPU%  MEMORY(bytes)  MEMORY%
```

```
master01    1199m      15%   12555Mi       66%
```

57. Use a JSONPath filter to determine the capacity and allocatable CPU for the `master01` node.

```
58. [student@workstation ~]$ oc get node master01 -o jsonpath=\
```

```
59. 'Allocatable: {.status.allocatable.cpu}{"\n"}'\
```

```
60. 'Capacity: {.status.capacity.cpu}{"\n"}'
```

```
61. Allocatable: 7500m
```

```
Capacity: 8
```

5. Retrieve debugging information for the cluster. Specify the `/home/student/D0180/labs/cli-review/debugging` directory as the destination directory.

Then, generate debugging information for the kube-apiserver cluster operator. Specify the /home/student/D0180/labs/cli-review/inspect directory as the destination directory. Limit the debugging information to the last five minutes.

0. Retrieve debugging information for the cluster. Save the output to the /home/student/D0180/labs/cli-review/debugging directory.

```
1. [student@workstation ~]$ oc adm must-gather \  
2.   --dest-dir /home/student/D0180/labs/cli-review/debugging  
3. [must-gather      ] OUT Using must-gather plug-in image: quay.io/openshift-r  
   ...  
4. ....output omitted...  
5. Reprinting Cluster State:  
6. When opening a support case, bugzilla, or issue please include the following  
   ...  
7. ClusterID: 6c8c6eed-26ed-4911-9df9-b081404842c8  
8. ClientVersion: 4.18.6  
9. ClusterVersion: Stable at "4.18.6"  
10. ClusterOperators:
```

All healthy and stable

11. Generate debugging information for the kube-apiserver cluster operator. Save the output to the /home/student/D0180/labs/cli-review/inspect directory, and limit the debugging information to the last five minutes.

```
12. [student@workstation ~]$ oc adm inspect clusteroperator kube-apiserver \  
13.   --dest-dir /home/student/D0180/labs/cli-review/inspect --since 5m  
14. Gathering data for ns/metallb-system...  
15. ....output omitted...
```

Wrote inspect data to /home/student/D0180/labs/cli-review/inspect.

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade cli-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-review
```

Summary

- You can use either the web console or the `kubectl` or `oc` commands to manage the RHOCP cluster.
- An RHOCP cluster can be managed from the web console or by using the `kubectl` or `oc` command-line interfaces (CLI).
- Use the `--help` option on any command to view detailed information about the command.
- Projects provide isolation between your application resources.
- Token authentication is the only guaranteed method to work with any RHOCP cluster, because enterprise SSO might replace the login form of the web console.
- All administrative tasks require creating, viewing, and changing the API resources.
- Kubernetes provides YAML- and JSON-formatted output options, which are ideal for parsing or scripting.
- Operators provide the means of monitoring applications, performing health checks, managing over-the-air (OTA) updates, and ensuring that applications remain in your specified state.
- The RHOCP web console incorporates useful graphs to visualize cluster and resource analytics.
- The RHOCP web console provides an interface for executing Prometheus queries, visualizing metrics, and configuring alerts.
- The monitoring stack is based on the Prometheus project, and it is configured to monitor the core RHOCP cluster components, by default.
- RHOCP provides the ability to view logs in running containers and pods to ease troubleshooting.

- You can use the `oc adm must-gather` command to collect resource definitions and service logs from your cluster.

Chapter 3. Run Applications as Containers and Pods

Create Linux Containers and Kubernetes Pods

Guided Exercise: Create Linux Containers and Kubernetes Pods

Find and Inspect Container Images

Guided Exercise: Find and Inspect Container Images

Troubleshoot Containers and Pods

Guided Exercise: Troubleshoot Containers and Pods

Lab: Run Applications as Containers and Pods

Quiz: Run Applications as Containers and Pods

Summary

Abstract

Goal	Run and troubleshoot containerized applications as unmanaged Kubernetes pods.
Sections	• Create Linux Containers and Kubernetes Pods (and Guided Exercise) • Find and Inspect Container Images (and Guided Exercise) • Troubleshoot Containers and Pods (and Guided Exercise)
Lab	• Run Applications as Containers and Pods

Create Linux Containers and Kubernetes Pods

Objectives

- Run containers inside pods and identify the host OS processes and namespaces that the containers use.

Running Containers in Pods

Kubernetes and Red Hat OpenShift offer many ways to create containers in pods. You can use one such way, the `run` command, with the `kubect1` or `oc` CLI to create and deploy an application in a pod from a container image. A *container image* contains immutable data that defines an application and its libraries.

NOTE

Container images are discussed in more detail elsewhere in the course.

Use the `run` command to create single pods for interactive debugging or temporary tasks. For enterprise applications, using declarative YAML manifests with workload resources such as `Deployment` resources is the recommended practice for ensuring reproducibility and manageability. The `run` command uses the following syntax:

```
oc run RESOURCE/NAME --image IMAGE [options]
```

For example, the following command deploys an Apache HTTPD application in a pod named `web-server` that uses the `registry.access.redhat.com/ubi10/httpd-24` container image.

```
[user@host ~]$ oc run web-server \  
--image registry.access.redhat.com/ubi10/httpd-24
```

You can use several options and flags with the `run` command. The `--command` option executes a custom command and its arguments in a container, rather than the default command that is defined in the container image. You must follow the `--command` option with a double dash (`--`) to separate the custom command and its arguments from the `run` command options. The following syntax is used with the `--command` option:

```
oc run RESOURCE/NAME --image IMAGE --command -- cmd arg1 ... argN
```

You can also use the double dash option to provide custom arguments to a default command in the container image.

```
oc run RESOURCE/NAME --image IMAGE -- arg1 arg2 ... argN
```

To start an interactive session with a container in a pod, include the `-it` options before the pod name. The `-i` option tells Kubernetes to keep open the standard input (`stdin`) on the container in the pod. The `-t` option tells Kubernetes to open a TTY session for the container in the pod. You can use the `-it` options to start an interactive, remote shell in a container. From the remote shell, you can then execute additional commands in the container. The following example starts an interactive remote shell, `/bin/bash`, in the default container in the `my-app` pod.

```
[user@host ~]$ oc run -it my-app --image registry.access.redhat.com/ubi10/ubi \
--command -- /bin/bash
```

If you don't see a command prompt, try pressing enter.

```
bash-5.1$
```

NOTE

Unless you include the `--namespace` or `-n` options, the `run` command creates containers in pods in the current selected project.

You can define a restart policy for a pod by using the `--restart` option. A pod restart policy determines how the cluster responds when containers in that pod exit. The accepted values are `Always`, `OnFailure`, or `Never`.

Policy	Description
Always	The cluster continuously tries to restart a container that exits, whether it exited successfully or with an error. This policy is the default.
OnFailure	The cluster restarts a container only if it exits with an error.
Never	The cluster does not try to restart exited or failed containers. Instead, the pods immediately fail and exit.

The following example command executes the `date` command in the container of the pod named `my-app`, redirects the `date` command output to the terminal, and defines `Never` as the pod restart policy.

```
[user@host ~]$ oc run -it my-app --image registry.access.redhat.com/ubi10/ubi \
```

```
--restart Never --command -- date
```

```
Mon Feb 20 22:36:55 UTC 2023
```

To automatically delete a pod after it exits, include the `--rm` option with the `run` command.

```
[user@host ~]$ oc run -it my-app --rm \
--image registry.access.redhat.com/ubi10/ubi --restart Never --command -- date
```

```
Mon Feb 20 22:38:50 UTC 2023
```

```
pod "my-app" deleted
```

For some containerized applications, you might need to specify environment variables for the application to work. To specify an environment variable and its value, include the `--env=` option with the `run` command.

```
[user@host ~]$ oc run mysql --image registry.redhat.io/rhel9/mysql-80 \
--env MYSQL_ROOT_PASSWORD=myP@$$123
```

```
pod/mysql created
```

User and Group IDs Assignment

When a project is created, Red Hat OpenShift adds annotations to the project that determine the user ID (UID) range and supplemental group ID (GID) for pods and their containers in the project. You can retrieve the annotations with the `oc describe project project-name` command.

```
[user@host ~]$ oc describe project my-app
```

```
Name: my-app
```

```
...output omitted...
```

```
Annotations: openshift.io/description=
```

```
openshift.io/display-name=
```

```
openshift.io/requester=developer
```

```
openshift.io/sa.scc.mcs=s0:c27,c4
```

```
openshift.io/sa.scc.supplemental-groups=1000710000/10000
```

```
openshift.io/sa.scc.uid-range=1000710000/10000
```

...output omitted...

The values for `supplemental-groups` and `uid-range` use the `base/size` format. In the `1000710000/10000` example, the `1000710000` number is the starting ID of the block that is allocated to this project. The `10000` number is the size of the block. Therefore, the project can assign up to 10,000 unique UIDs and GIDs to containers that are running within it, starting from the base ID of `1000710000`.

With Red Hat OpenShift default security policies, regular cluster users cannot choose the `USER` or UIDs for their containers. When a regular cluster user creates a pod, Red Hat OpenShift ignores the `USER` instruction in the container image. Instead, Red Hat OpenShift assigns a random UID to the container's primary process from the `uid-range` that is defined in the project's annotations.

The primary group ID (GID) of the process is always `0`, which is the `root` group. Because the process runs with a non-root UID but belongs to the `root` group, any files or directories that the container writes to must be owned by the `root` group and have group write permissions.

This feature is essential for security. Although the process in the container belongs to the `root` group, its high-numbered, non-root UID makes it an unprivileged account.

In contrast, when a cluster administrator creates a pod, the `USER` instruction in the container image is processed. For example, if the `USER` instruction for the container image is set to `0`, then the process in the container runs as the `root` privileged account, with a UID of `0`. Executing a container as a privileged account is a security risk. A privileged account in a container has unrestricted access to the container's host system. Unrestricted access means that the container could modify or delete system files, install software, or otherwise compromise its host. Red Hat therefore recommends that you run containers as `rootless`, or as an unprivileged user with only the necessary privileges for the container to run. By assigning a distinct block of UIDs to each project, Red Hat OpenShift ensures that containers in different projects do not run with the same UID, which is a critical security practice.

Pod Security

The Kubernetes Pod Security Admission (PSA) controller issues a warning when a pod is created without a defined security context. Red Hat OpenShift Container

Platform 4.18 integrates the PSA controller, which enforces security profiles (privileged, baseline, restricted) at the namespace level. Security contexts grant or deny OS-level privileges to pods.

Red Hat OpenShift uses the Security Context Constraints (SCC) controller to provide safe defaults for pod security. SCCs grant pods the necessary permissions to meet the requirements of the enforced PSA profile. You can ignore pod security warnings in these course exercises. SCC controllers are discussed in more detail in the *Red Hat OpenShift Administration II: Operating a Production Kubernetes Cluster* (DO280) course.

Execute Commands in Running Containers

To execute a command in a running container in a pod, you can use the `exec` command with the `oc` CLI. The `exec` command uses the following syntax:

```
oc exec RESOURCE/NAME -- COMMAND [args...] [options]
```

The output of the executed command is sent to your terminal. In the following example, the `exec` command executes the `date` command in the `my-app` pod.

```
[user@host ~]$ oc exec my-app -- date  
Tue Feb 21 20:43:53 UTC 2023
```

The specified command is executed in the first container of a pod. For multicontainer pods, include the `-c` or `--container=` options to specify which container is used to execute the command. The following example executes the `date` command in a container named `ruby-container` in the `my-app` pod.

```
[user@host ~]$ oc exec my-app -c ruby-container -- date  
Tue Feb 21 20:46:50 UTC 2023
```

The `exec` command also accepts the `-i` and `-t` options to create an interactive session with a container in a pod. In the following example, Kubernetes sends `stdin` to the `bash` shell in the `ruby-container` container from the `my-app` pod, and sends `stdout` and `stderr` from the `bash` shell back to the terminal.

```
[user@host ~]$ oc exec my-app -c ruby-container -it -- bash -il
```

```
[1000780000@ruby-container /]$
```

In the previous example, a raw terminal is opened in the `ruby-container` container. From this interactive session, you can execute additional commands in the container. To terminate the interactive session, you must execute the `exit` command in the raw terminal.

```
[user@host ~]$ oc exec my-app -c ruby-container -it -- bash -il
[1000780000@ruby-container /]$ date
Tue Feb 21 21:16:00 UTC 2023
[1000780000@ruby-container] exit
```

Container Logs

Container logs are the standard output (`stdout`) and standard error (`stderr`) output of a container. You can retrieve logs with the `logs pod/pod-name` command that the `oc` CLI provides. The command includes the following options:

Option	Description
<code>-l</code> or <code>--selector=' '</code>	Filter objects based on the specified <code>key:value</code> label constraint.
<code>--tail=</code>	Specify the number of lines of recent log files to display; the default value is <code>-1</code> with no selectors, which displays all log lines.
<code>-c</code> or <code>--container=</code>	Print the logs of a specific container in a multicontainer pod. If this option is omitted, then logs are shown from the first container that is defined in the pod or from the container that the <code>kubectl.kubernetes.io/default-container</code> annotation on the pod specifies.
<code>-f</code> or <code>--follow</code>	Follow, or stream, logs for a container.
<code>-p</code> or <code>--previous=true</code>	Print the logs of a previous container instance in the pod, if it exists. This option is helpful for troubleshooting a pod that failed to start, because it prints the logs of the last attempt.

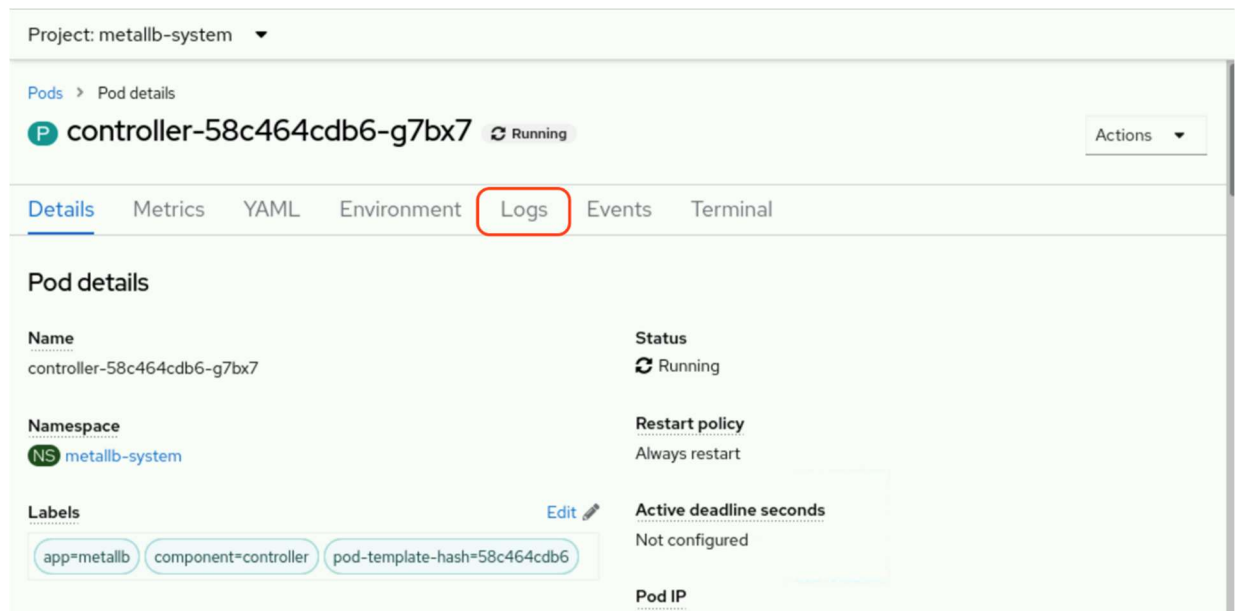
The following example restricts the `oc logs` command output to the 10 most recent log lines:

```
[user@host ~]$ oc logs postgresql-1-jw89j --tail=10
done
server stopped
Starting server...
2023-01-04 22:00:16.945 UTC [1] LOG:  starting PostgreSQL 12.11 on x86_64-redhat-linu
x-gnu, compiled by gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-10), 64-bit
2023-01-04 22:00:16.946 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2023-01-04 22:00:16.946 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2023-01-04 22:00:16.953 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.
s.PGSQL.5432"
2023-01-04 22:00:16.960 UTC [1] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2023-01-04 22:00:16.968 UTC [1] LOG:  redirecting log output to logging collector pro
cess
2023-01-04 22:00:16.968 UTC [1] HINT:  Future log output will appear in directory "lo
g".
```

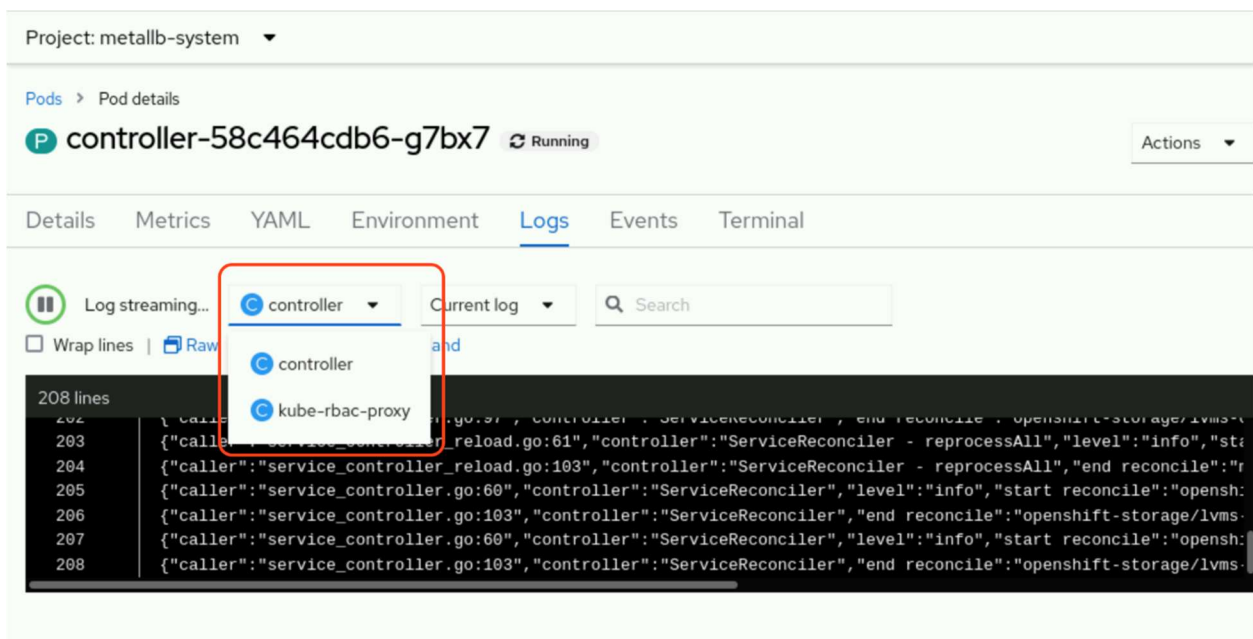
You can also use the `attach pod-name -c container-name -it` command to connect to and start an interactive session on a running container in a pod. The `-c container-name` option is required for multicontainer pods. If the container name is omitted, then Kubernetes uses the `kubect1.kubernetes.io/default-container` annotation on the pod to select the container. Otherwise, the first container in the pod is chosen. You can use the interactive session to retrieve application log files and to troubleshoot application issues.

```
[user@host ~]$ oc attach my-app -it
If you don't see a command prompt, try pressing enter.
bash-4.4$
```

You can also retrieve logs from the web console by clicking the **Logs** tab of any pod.



If you have more than one container, then you can change between them to list the logs of each one.



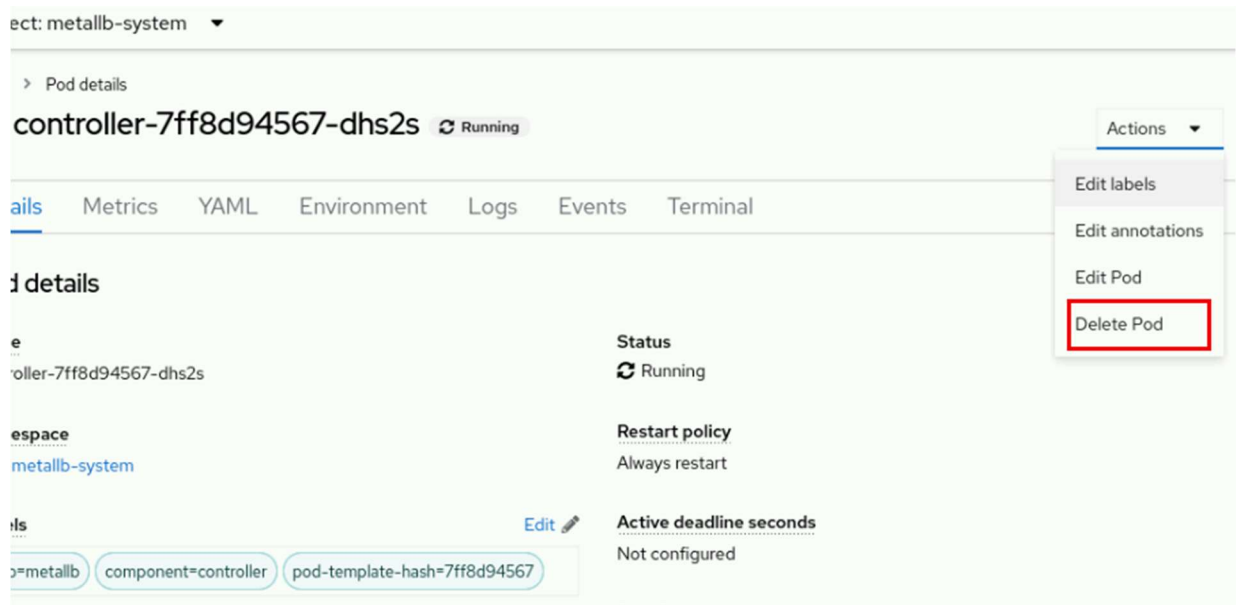
Deleting Resources

You can delete Kubernetes resources, such as pod resources, with the `delete` command. The `delete` command can delete resources by resource type and name, resource type and label, standard input (`stdin`), and with JSON or YAML-

formatted files. The command accepts only one argument type at a time. For example, you can supply the resource type and name as a command argument.

```
[user@host ~]$ oc delete pod php-app
```

You can also delete pods from the web console by clicking **Actions** and then **Delete Pod** in the pod's principal menu.



To select resources based on labels, you can include the `-l` option and the `key:value` label as a command argument.

```
[user@host ~]$ oc delete pod -l app=my-app
pod "php-app" deleted
pod "mysql-db" deleted
```

You can also provide the resource type and a JSON or YAML-formatted file that specifies the name of the resource. To use a file, you must include the `-f` option and provide the full path to the JSON or YAML-formatted file.

```
[user@host ~]$ oc delete pod -f ~/php-app.json
pod "php-app" deleted
```

You can also use `stdin` and a JSON or YAML-formatted file that includes the resource type and resource name with the `delete` command.

```
[user@host ~]$ cat ~/php-app.json | oc delete -f -  
pod "php-app" deleted
```

Pods support graceful termination, which means that pods try to terminate their processes first before Kubernetes forcibly terminates the pods. To change the time period before a pod is forcibly terminated, you can include the `--grace-period` flag and a time period in seconds in your `delete` command. For example, to change the grace period to 10 seconds, use the following command:

```
[user@host ~]$ oc delete pod php-app --grace-period=10
```

To shut down the pod immediately, set the grace period to 1 second. You can also use the `--now` flag to set the grace period to 1 second.

```
[user@host ~]$ oc delete pod php-app --now
```

You can also forcibly delete a pod with the `--force` option. If you forcibly delete a pod, Kubernetes does not wait for a confirmation that the pod's processes ended, which can leave the pod's processes running until its node detects the deletion. Therefore, forcibly deleting a pod could result in inconsistency or data loss. Forcibly delete pods only if you are sure that the pod's processes are terminated.

```
[user@host ~]$ oc delete pod php-app --force
```

To delete all pods in a project, you can include the `--all` option.

```
[user@host ~]$ oc delete pods --all  
pod "php-app" deleted  
pod "mysql-db" deleted
```

Likewise, you can delete a project and its resources with the `oc delete project project-name` command.

```
[user@host ~]$ oc delete project my-app
```

```
project.project.openshift.io "my-app" deleted
```

The CRI-O Container Engine

A container engine is required to run containers. Worker and control plane nodes in a Red Hat OpenShift Container Platform cluster use the CRI-O container engine to run containers. Unlike tools such as Podman or Docker, the CRI-O container engine is a runtime that is designed and optimized specifically for running containers in a Kubernetes cluster. Because CRI-O meets the Kubernetes Container Runtime Interface (CRI) standards, the container engine can integrate with other Kubernetes and Red Hat OpenShift tools, such as networking and storage plug-ins.

NOTE

For more information about the Kubernetes Container Runtime Interface (CRI) standards, refer to the *CRI-API* repository at <https://github.com/kubernetes/cri-api>.

CRI-O provides a command-line interface to manage containers with the `crictl` command. The `crictl` command includes several subcommands to help you to manage containers. The following subcommands are commonly used with the `crictl` command:

Command	Description
<code>crictl pods</code>	List all pods on a node.
<code>crictl image</code>	List all images on a node.
<code>crictl inspect</code>	Retrieve the status of one or more containers.
<code>crictl exec</code>	Run a command in a running container.
<code>crictl logs</code>	Retrieve the logs of a container.
<code>crictl ps</code>	List running containers on a node.

To manage containers with the `crictl` command, you must first identify the node that is hosting your containers.

```
[user@host ~]$ oc get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
postgresql-1-8lzf2	1/1	Running	0	20m	10.8.0.64	master01
postgresql-1-deploy	0/1	Completed	0	21m	10.8.0.63	master01

```
[user@host ~]$ oc get pod postgresql-1-8lzf2 \
-o jsonpath='{.spec.nodeName}{"\n"}'
master01
```

Next, you must connect to the identified node as a cluster administrator. Cluster administrators can use SSH to connect to a node or create a debug pod for the node. Regular users cannot connect to or create debug pods for cluster nodes. As a cluster administrator, you can create a debug pod for a node with the `oc debug node/node-name` command. Red Hat OpenShift creates the `pod/node-name-debug` pod in your currently selected project and automatically connects you to the pod. You must then enable access host binaries, such as the `crictl` command, with the `chroot /host` command. This command mounts the host's root file system in the `/host` directory within the debug pod shell. By changing the root directory to the `/host` directory, you can run binaries contained in the host's executable path.

```
[user@host ~]$ oc debug node/master01
Starting pod/master01-debug ...
To use host binaries, run chroot /host
Pod IP: 192.168.50.10
If you don't see a command prompt, try pressing enter.
sh-4.4# chroot /host
```

After enabling host binaries, you can use the `crictl` command to manage the containers on the node. For example, you can use the `crictl ps` and `crictl inspect` commands to retrieve the process ID (PID) of a running container. You can then use the PID to retrieve or enter the namespaces within a container, which is useful for troubleshooting application issues. To find the PID of a running container, you must first determine the container's ID. You can use the `crictl ps` command with the `--name` option to filter the command output to a specific container.

```
sh-5.1# crictl ps --name postgresql
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD
-----------	-------	---------	-------	------	---------	--------	-----

```
27943ae4f3024 image...7104 5...ago Running postgresql 0          5768...f015 po...
```

The default output of the `crictl ps` command is a table. You can find the short container ID under the `CONTAINER` column. You can also use the `-o` or `--output` options to specify the format of the `crictl ps` command as JSON or YAML and then parse the output. The parsed output displays the full container ID.

```
sh-5.1# crictl ps --name postgresql -o json | jq .containers[0].id
"2794...29a4"
```

After identifying the container ID, you can use the `crictl inspect` command and the container ID to retrieve the PID of the running container. By default, the `crictl inspect` command displays verbose output. You can use the `-o` or `--output` options to format the command output as JSON, YAML, a table, or as a Go template. If you specify the JSON format, you can then parse the output with the `jq` command. Likewise, you can use the `grep` command to limit the command output.

```
sh-5.1# crictl inspect -o json 27943ae4f3024 | jq .info.pid
43453
sh-5.1# crictl inspect 27943ae4f3024 | grep pid
    "pid": 43453,
...output omitted...
```

Alternatively, use the built-in methods such as the `go-template` option to parse out the ID without using external tools.

```
sh-5.1# crictl inspect --output go-template \
--template '{{.info.pid}}' 27943ae4f3024
43453
```

After determining the PID of a running container, you can use the `lsns -p PID` command to list the system namespaces of a container.

```
sh-5.1# lsns -p 43453
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
4026531835	cgroup		530	1	root	/usr/lib/systemd/systemd --switche...

```

4026531837 user      530      1 root      /usr/lib/systemd/systemd --switche...
4026537853 uts        8 43453 1000690000 postgres
4026537854 ipc        8 43453 1000690000 postgres
4026537856 net        8 43453 1000690000 postgres
4026538013 mnt        8 43453 1000690000 postgres
4026538014 pid        8 43453 1000690000 postgres

```

You can also use the PID of a running container with the `nsenter` command to enter a specific namespace of a running container. For example, you can use the `nsenter` command to execute a command within a specified namespace on a running container. The following example executes the `ps -ef` command within the process namespace of a running container.

```

sh-5.1# nsenter -t 43453 -p -r ps -ef

```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
1000690+	1	0	0	18:49	?	00:00:00	postgres
1000690+	58	1	0	18:49	?	00:00:00	postgres: logger
1000690+	60	1	0	18:49	?	00:00:00	postgres: checkpointer
1000690+	61	1	0	18:49	?	00:00:00	postgres: background writer
1000690+	62	1	0	18:49	?	00:00:00	postgres: walwriter
1000690+	63	1	0	18:49	?	00:00:00	postgres: autovacuum lau...
1000690+	64	1	0	18:49	?	00:00:00	postgres: stats collector
1000690+	65	1	0	18:49	?	00:00:00	postgres: logical replic...
root	7414	0	0	20:14	?	00:00:00	ps -ef

The `-t` option specifies the PID of the running container as the target PID for the `nsenter` command. The `-p` option directs the `nsenter` command to enter the process or `pid` namespace. The `-r` option sets the top-level directory of the process namespace as the root directory, thus enabling commands to execute in the context of the namespace. You can also use the `-a` option to execute a command in all of the container's namespaces.

```

sh-5.1# nsenter -t 43453 -a ps -ef

```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
1000690+	1	0	0	18:49	?	00:00:00	postgres

1000690+	58	1	0	18:49	?	00:00:00	postgres: logger
1000690+	60	1	0	18:49	?	00:00:00	postgres: checkpointer
1000690+	61	1	0	18:49	?	00:00:00	postgres: background writer
1000690+	62	1	0	18:49	?	00:00:00	postgres: walwriter
1000690+	63	1	0	18:49	?	00:00:00	postgres: autovacuum lau...
1000690+	64	1	0	18:49	?	00:00:00	postgres: stats collector
1000690+	65	1	0	18:49	?	00:00:00	postgres: logical replic...
root	10058	0	0	20:45	?	00:00:00	ps -ef

Guided Exercise: Create Linux Containers and Kubernetes Pods

Run a base OS container in a pod and compare the environment inside the container with its host node.

Outcomes

- Create a pod with a single container, and identify the pod and its container within the container engine of an OpenShift node.
- View the logs of a running container.
- Retrieve information inside a container, such as the operating system (OS) release and running processes.
- Identify the process ID (PID) and namespaces for a container.
- Identify the User ID (UID) and supplemental group ID (GID) ranges of a project.
- Compare the namespaces of containers in one pod versus in another pod.
- Inspect a pod with multiple containers, and identify the purpose of each container.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise.

```
[student@workstation ~]$ lab start pods-containers
```

Instructions

1. Log in to the OpenShift cluster and create the pods-containers project.
Determine the UID and GID ranges for pods in the pods-containers project.
1. Log in to the OpenShift cluster as the developer user with the oc command.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful
```

...output omitted...

5. Create the pods-containers project.

```
6. [student@workstation ~]$ oc new-project pods-containers
7. Now using project "pods-containers" on server "https://api.ocp4.example.com:
6443".
```

...output omitted...

8. Identify the UID and GID ranges for pods in the pods-containers project.

```
9. [student@workstation ~]$ oc describe project pods-containers
10. Name: pods-containers
11. Created: 16 seconds ago
12. Labels: kubernetes.io/metadata.name=pods-containers
13. pod-security.kubernetes.io/audit=restricted
14. pod-security.kubernetes.io/audit-version=latest
15. pod-security.kubernetes.io/warn=restricted
16. pod-security.kubernetes.io/warn-version=latest
17. Annotations: openshift.io/description=
18. openshift.io/display-name=
19. openshift.io/requester=developer
20. openshift.io/sa.scc.mcs=s0:c28,c2
21. openshift.io/sa.scc.supplemental-groups=1000760000/1000
    0
22. openshift.io/sa.scc.uid-range=1000760000/10000
23. Display Name: <none>
24. Description: <none>
25. Status: Active
```


26. Node Selector: <none>

27. Quota: <none>

Resource limits: <none>

Your UID and GID range values might differ from the previous output.

2. As the developer user, create a pod called `ubi9-user` from a UBI9 base container image. The image is available in the `registry.ocp4.example.com:8443/ubi9/ubi` container registry. Set the restart policy to `Never` and start an interactive session. Configure the pod to execute the `whoami` and `id` commands to determine the UIDs, supplemental groups, and GIDs of the container user in the pod. Delete the pod afterward.

After the `ubi-user` pod is deleted, log in as the `admin` user and then re-create the `ubi9-user` pod. Retrieve the UIDs and GIDs of the container user. Compare the values to the values of the `ubi9-user` pod that the developer user created.

Afterward, delete the `ubi9-user` pod.

1. Use the `oc run` command to create the `ubi9-user` pod. Configure the pod to execute the `whoami` and `id` commands through an interactive bash shell session.

```
2. [student@workstation ~]$ oc run -it ubi9-user --restart 'Never' \  
3. --image registry.ocp4.example.com:8443/ubi9/ubi \  
4. -- /bin/bash -c "whoami && id" \  
5. 1000760000
```

```
uid=1000760000(1000760000) gid=0(root) groups=0(root),1000760000
```

Your values might differ from the previous output.

Notice that the user in the container has the same UID that is identified in the `pods-containers` project. However, the GID of the user in the container is `0`, which means that the user belongs to the `root` group. Any files and

directories that the container processes might write to must have read and write permissions by `GID=0` and have the `root` group as the owner.

Although the user in the container belongs to the `root` group, a UID value over `1000` means that the user is an unprivileged account. When a regular OpenShift user, such as the `developer` user, creates a pod, the containers within the pod run as unprivileged accounts.

6. Delete the pod.

```
7. [student@workstation ~]$ oc delete pod ubi9-user
```

```
pod "ubi9-user" deleted
```

8. Log in as the admin user with the `redhatocp` password.

```
9. [student@workstation ~]$ oc login -u admin -p redhatocp
```

```
10. Login successful.
```

```
11.
```

```
12. You have access to 71 projects, the list has been suppressed. You can list  
all projects with 'oc projects'
```

```
13.
```

```
Using project "pods-containers".
```

14. Re-create the `ubi9-user` pod as the admin user. Configure the pod to execute the `whoami` and `id` commands through an interactive bash shell session. Compare the values of the UID and GID for the container user to the values of the `ubi9-user` pod that the `developer` user created.

NOTE

It is safe to ignore pod security warnings when using a cluster-admin user that creates unmanaged pods. The `admin` user can create privileged pods that the Security Context Constraints controller does not manage.

```
[student@workstation ~]$ oc run -it ubi9-user --restart 'Never' \  
--image registry.ocp4.example.com:8443/ubi9/ubi \  
-- /bin/bash -c "whoami && id"
```

```
Warning: would violate PodSecurity "restricted:v1.24": allowPrivilegeEscalation != false (container "ubi9-user" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "ubi9-user" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "ubi9-user" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "ubi9-user" must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
```

```
root
```

```
uid=0(root) gid=0(root) groups=0(root)
```

Notice that the value of the UID is 0, which differs from the UID range value of the pod-containers project. The user in the container is the privileged account root user and belongs to the root group. When a cluster administrator creates a pod, the containers within the pod run as a privileged account by default.

15. Delete the ubi9-user pod.

```
16. [student@workstation ~]$ oc delete pod ubi9-user
```

```
pod "ubi9-user" deleted
```

3. As the developer user, use the `oc run` command to create a `ubi9-date` pod from a UBI9 base container image. The image is available in the `registry.ocp4.example.com:8443/ubi9/ubi` container registry. Set the restart policy to `Never`, and configure the pod to execute the `date` command. Retrieve the logs of the `ubi9-date` pod to confirm that the `date` command executed. Delete the pod afterward.

1. Log in as the developer user with the developer password.

```
2. [student@workstation ~]$ oc login -u developer -p developer
```

```
3. Login successful.
```

```
4.
```

```
5. You have one project on this server: "pods-containers"
```

```
6.
```

```
Using project "pods-containers".
```

7. Create a pod called `ubi9-date` that executes the `date` command.

```
8. [student@workstation ~]$ oc run ubi9-date --restart 'Never' \
```

```
9. --image registry.ocp4.example.com:8443/ubi9/ubi -- date
```

```
pod/ubi9-date created
```

10. Wait a few moments for the creation of the pod. Then, retrieve the logs of the ubi9-date pod.

```
11. [student@workstation ~]$ oc logs ubi9-date
```

```
Mon Nov 28 15:02:55 UTC 2022
```

12. Delete the ubi9-date pod.

```
13. [student@workstation ~]$ oc delete pod ubi9-date
```

```
pod "ubi9-date" deleted
```

4. Use the `oc run ubi9-command -it` command to create a ubi9-command pod with the `registry.ocp4.example.com:8443/ubi9/ubi` container image. Add the `/bin/bash` in the `oc run` command to start an interactive shell. Exit the pods and view the logs for the ubi9-command pod with the `oc logs` command. Then, connect to the ubi9-command pod with the `oc attach` command, and issue the following command:

```
while true; do echo $(date); sleep 2; done
```

This command executes the `date` and `sleep` commands to generate output to the console every two seconds. Use the `oc logs` command to retrieve the logs of the ubi9 pod, and confirm that the logs display the executed `date` and `sleep` commands.

1. Create a pod called ubi9-command and start an interactive shell.

```
2. [student@workstation ~]$ oc run ubi9-command -it \
```

```
3. --image registry.ocp4.example.com:8443/ubi9/ubi -- /bin/bash
```

4. If you don't see a command prompt, try pressing enter.

```
bash-5.1$
```

5. Exit the shell session.

```
6. bash-5.1$ exit
```

```
7. exit
```

Session ended, resume using 'oc attach ubi9-command -c ubi9-command -i -t' command when the pod is running

8. Use the `oc logs` command to view the logs of the `ubi9-command` pod.

```
9. [student@workstation ~]$ oc logs ubi9-command
```

```
bash-5.1$ [student@workstation ~]$
```

The pod's command prompt is returned. The `oc logs` command displays the pod's current `stdout` and `stderr` output in the console. Because you disconnected from the interactive session, the pod's current `stdout` is the command prompt, and not the commands that you executed previously.

10. Use the `oc attach` command to connect to the `ubi9-command` pod again. In the shell, execute the `while true; do echo $(date); sleep 2; done` command to continuously generate `stdout` output.

```
11. [student@workstation ~]$ oc attach ubi9-command -it
```

If you don't see a command prompt, try pressing enter.

```
bash-5.1$ while true; do echo $(date); sleep 2; done
```

```
Mon Nov 28 15:15:16 UTC 2022
```

```
Mon Nov 28 15:15:18 UTC 2022
```

```
Mon Nov 28 15:15:20 UTC 2022
```

```
Mon Nov 28 15:15:22 UTC 2022
```

...output omitted...

12. Open another terminal window and view the logs for the `ubi9-command` pod with the `oc logs` command. Limit the log output to the last 10 entries with the `--tail=10` option. Confirm that the logs display the results of the command that you executed in the container.

```
13. [student@workstation ~]$ oc logs ubi9-command --tail=10
```

```
14. Mon Nov 28 15:15:16 UTC 2022
```

```
15. Mon Nov 28 15:15:18 UTC 2022
16. Mon Nov 28 15:15:20 UTC 2022
17. Mon Nov 28 15:15:22 UTC 2022
18. Mon Nov 28 15:15:24 UTC 2022
19. Mon Nov 28 15:15:26 UTC 2022
20. Mon Nov 28 15:15:28 UTC 2022
21. Mon Nov 28 15:15:30 UTC 2022
22. Mon Nov 28 15:15:32 UTC 2022
```

```
Mon Nov 28 15:15:34 UTC 2022
```

5. Identify the name for the container in the `ubi9-command` pod. Identify the process ID (PID) for the container in the `ubi9-command` pod by using a debug pod for the pod's host node. Use the `crictl` command to identify the PID of the container in the `ubi9-command` pod. Then, retrieve the PID of the container in the debug pod.
1. Identify the container name in the `ubi9-command` pod with the `oc get` command. Specify the JSON format for the command output. Parse the JSON output with the `jq` command to retrieve the value of the `.status.containerStatuses[].name` object.

```
2. [student@workstation ~]$ oc get pod ubi9-command -o json | \
3. jq .status.containerStatuses[].name
```

```
"ubi9-command"
```

The `ubi9-command` pod has a single container of the same name.

4. Find the host node for the `ubi9-command` pod. Start a debug pod for the host with the `oc debug` command.

```
5. [student@workstation ~]$ oc get pods ubi9-command -o wide
```

```
6. NAME                READY STATUS  RESTARTS   AGE   IP            NODE      ...
```

```
ubi9-command    1/1    Running 2 (16m ago) 27m  10.8.0.26  master01 ...
```

```
[student@workstation ~]$ oc debug node/master01
```

```
Error from server (Forbidden): nodes "master01" is forbidden: User "developer" cannot get resource "nodes" in API group "" at the cluster scope
```

The debug pod fails because the developer user does not have the required permission to debug a host node.

7. Log in as the admin user with the redhatocp password. Start a debug pod for the host with the `oc debug` command. After connecting to the debug pod, run the `chroot /host` command to use host binaries, such as the `crictl` command-line tool.

```
8. [student@workstation ~]$ oc login -u admin -p redhatocp
9. Login successful.
```

```
...output omitted...
[student@workstation ~]$ oc debug node/master01
Starting pod/master01-debug ...
To use host binaries, run `chroot /host`
Pod IP: 192.168.50.10
If you don't see a command prompt, try pressing enter
sh-4.4# chroot /host
```

10. Use the `crictl ps` command to retrieve the `ubi9-command` container ID. Specify the `ubi9-command` container with the `--name` option and use the JSON output format. Parse the JSON output with the `jq -r` command to get the RAW JSON output. Export the container ID as the `$CID` environment variable.

NOTE

When using `jq` without the `-r` flag, the container ID is wrapped in double quotes, which does not work with `crictl` commands. If the `-r` flag is not used, then you can add `| tr -d '"'` to the end of the command to trim the double quotes.

```
sh-5.1# crictl ps --name ubi9-command -o json | jq -r .containers[0].id
81adbc6222d79ed9ba195af4e9d36309c18bb71bc04b2e8b5612be632220e0d6
sh-5.1# CID=$(crictl ps --name ubi9-command -o json | jq -r .containers[0].id)
sh-5.1# echo $CID
81adbc6222d79ed9ba195af4e9d36309c18bb71bc04b2e8b5612be632220e0d6
```

Your container ID value might differ from the previous output.

11. Use the `crictl inspect` command to find the PID of the `ubi9-command` container. The PID value is in the `.info.pid` object in the `crictl inspect` output. Export the `ubi9-command` container PID as the `$PID` environment variable.

```
12. sh-5.1# crictl inspect $CID | grep pid
13.     "pid": 365297,
14.     "pids": {
15.         "type": "pid"
16. ...output omitted...
17.     }
```

```
...output omitted...
sh-5.1# PID=365297
```

Your PID values might differ from the previous output.

6. Use the `lsns` command to list the system namespaces of the `ubi9-command` container. Confirm that the running processes in the container are isolated to different system namespaces.
1. View the system namespaces of the `ubi9-command` container with the `lsns` command. Specify the PID with the `-p` option and use the `$PID` environment variable. In the resulting table, the `ns` column contains the namespace values for the container.

```
2. sh-5.1# lsns -p $PID
3.      NS TYPE   NPROCS   PID USER      COMMAND
4. 4026531835 cgroup    540      1 root      /usr/lib/systemd/systemd --swit .
   ..
5. 4026531837 user      540      1 root      /usr/lib/systemd/systemd --swit .
   ..
6. 4026536117 uts        1 153168 1000800000 /bin/bash
7. 4026536118 ipc        1 153168 1000800000 /bin/bash
8. 4026536120 net         1 153168 1000800000 /bin/bash
9. 4026537680 mnt         1 153168 1000800000 /bin/bash
```



```
4026537823 pid          1 153168 1000800000 /bin/bash
```

Your namespace values might differ from the previous output.

7. Use the host debug pod to retrieve and compare the operating system (OS) and the GCC support library (libgcc) package version of the ubi9-command container and the host node.

1. Retrieve the OS for the host node with the `cat /etc/redhat-release` command.

```
2. sh-5.1# cat /etc/redhat-release
```

```
Red Hat Enterprise Linux CoreOS release 4.18
```

3. Use the `crictl exec` command and the `$CID` container ID variable to retrieve the OS of the ubi9-command container. Use the `-it` options to create an interactive terminal to execute the `cat /etc/redhat-release` command.

```
4. sh-5.1# crictl exec -it $CID cat /etc/redhat-release
```

```
Red Hat Enterprise Linux release 9.6 (Plow)
```

The ubi9-command container has a different OS from the host node.

5. Use the `rpm -qi libgcc` command to retrieve the libgcc package version of the host node.

```
6. sh-5.1$ rpm -qi libgcc
```

```
7. Name: libgcc
```

```
8. Version: 11.4.1
```

```
...output omitted...
```

9. Use the `crictl exec` command and the `$CID` container ID variable to retrieve the glibc package version of the ubi9-command container. Use the `-it` options to create an interactive terminal to execute the `rpm -qi libgcc` command.

```
10. sh-5.1# crictl exec -it $CID rpm -qi libgcc
```

```
11. Name: libgcc
```

```
12. Version: 11.5.0
```

```
...output omitted...
```

The `ubi9-command` container might have a different version of the `libgcc` package from its host.

8. Exit the `master01-debug` pod and the `ubi9-command` pod.
1. Exit the `master01-debug` pod. You must issue the `exit` command to end the host binary access. Execute the `exit` command again to exit and remove the `master01-debug` pod.

```
2. sh-5.1# exit
```

```
exit
sh-4.4# exit
exit

Removing debug pod ...
Temporary namespace openshift-debug-bg7kn was removed.
```

3. Return to the terminal window that is connected to the `ubi9-command` pod. Press **Ctrl+C** and then execute the `exit` command. Confirm that the pod is still running.

```
4. ...output omitted...
```

```
5. ^C
```

```
6. bash-5.1$ exit
```

```
7. exit
```

```
Session ended, resume using 'oc attach ubi9-command -c ubi9-command -i -t'
command when the pod is running
```

```
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ubi9-command	1/1	Running	2 (6s ago)	35m

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish pods-containers
```

Find and Inspect Container Images

Objectives

- Find containerized applications in container registries and get information about the runtime parameters of supported and community container images.

Container Image Overview

A *container* is an isolated runtime environment where applications are executed as isolated processes. The isolation of the runtime environment ensures that applications do not interfere with other containers or system processes.

A *container image* contains a packaged version of your application, with all the necessary dependencies for the application to run. Images can exist without containers. However, containers depend on images, because containers use container images to build a runtime environment to execute applications.

Containers can be split into two similar but distinct concepts: *container images* and *container instances*. A *container image* contains immutable data that defines an application and its libraries. You can use container images to create *container instances*, which are running processes that are isolated by a set of kernel namespaces.

You can use each container image many times to create distinct container instances. These replicas can be split across multiple hosts. The application within a container is independent of the host environment.

Container Image Registries

Image registries are services that offer container images to download. Image creators and maintainers can store and distribute container images in a controlled

manner to public or private audiences. Some examples of image registries include Quay.io, Red Hat Registry, Docker Hub, and Amazon ECR.

Red Hat Registry

Red Hat distributes container images through the Red Hat Ecosystem Catalog, <https://catalog.redhat.com/>, which provides a centralized searching utility. You can search the Red Hat Ecosystem Catalog for technical details about container images. The catalog hosts a large set of container images, including from major open source projects, such as Apache, MySQL, and Jenkins. Although many of the container images are publicly available, authentication to the website is required to access the container images that are not available in the public domain.

Because the Red Hat Ecosystem Catalog also contains software products other than container images, go to <https://catalog.redhat.com/software/containers/explore> to search for specific container images.

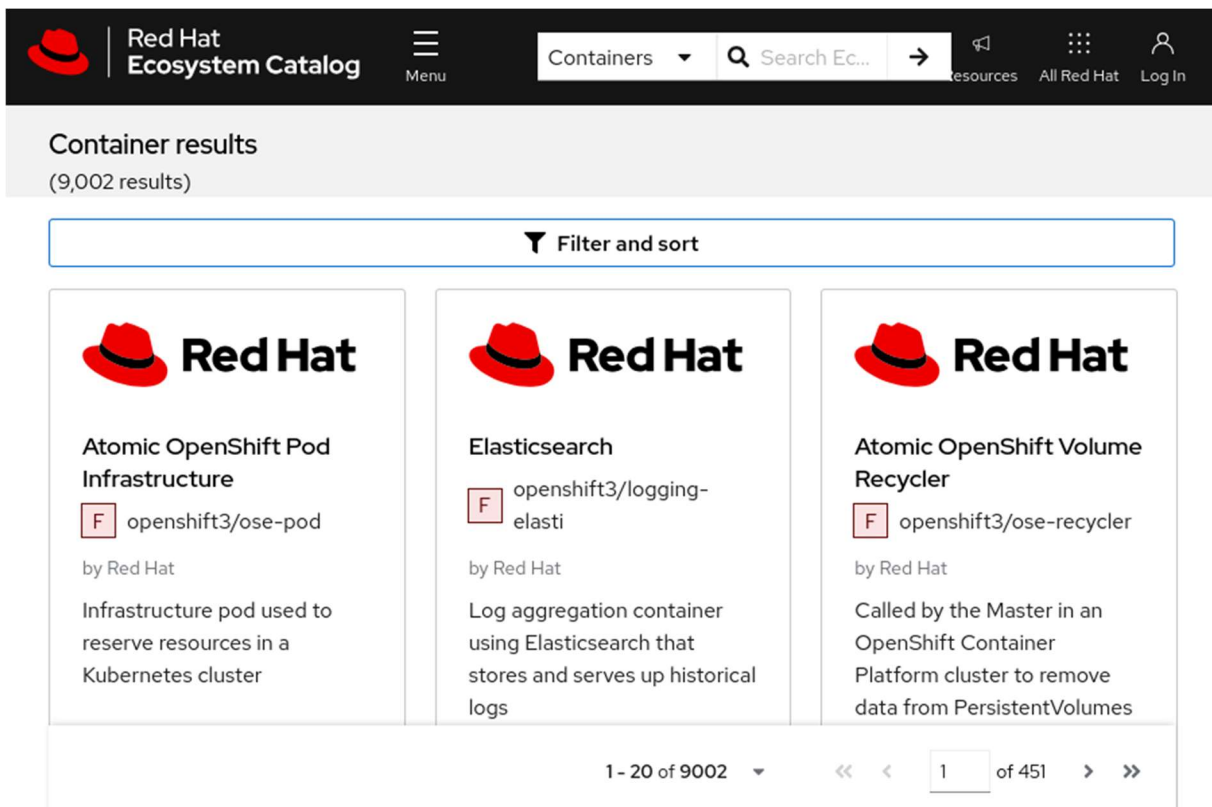


Figure 3.4: Red Hat Ecosystem Catalog

The details page of a container image includes relevant information, such as technical data, the installed packages within the image, or a security scan. You can navigate through these options by using the tabs on the website. You can also change the image version by selecting a specific tag.

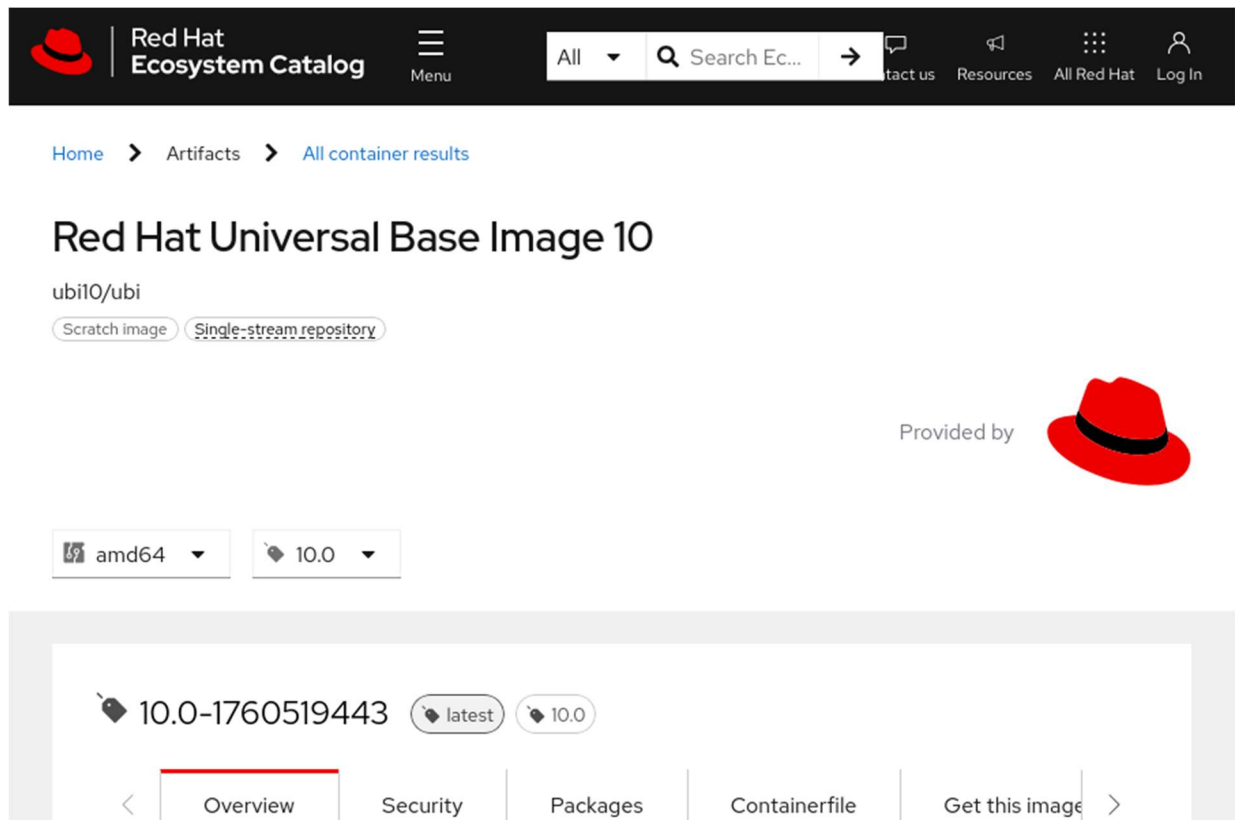


Figure 3.5: The Red Hat Universal Base Image 10 details page

The Red Hat internal security team validates all images in the container catalog. Red Hat rebuilds all components to avoid known security vulnerabilities.

Red Hat container images provide the following benefits:

- **Trusted source:** All container images use sources that Red Hat knows and trusts.
- **Original dependencies:** None of the container packages are altered, and they include only known libraries.
- **Vulnerability-free:** Container images are without known critical vulnerabilities in the platform components or layers.
- **Runtime protection:** All applications in container images run as non-root users to minimize the exposure surface to malicious or faulty applications.

- Red Hat Enterprise Linux (RHEL) compatible: Container images are compatible with all RHEL platforms, from bare metal to cloud.
- Red Hat support: Red Hat commercially supports the container images in the catalog.

NOTE

You must log in to the catalog website with a customer portal account or a Red Hat Developer account to use the stored container images in the registry.

Quay.io

Although the Red Hat Registry stores only images from Red Hat and certified providers, you can store your own images with Quay.io, which is another public image registry that Red Hat sponsors. Although storing public images in Quay is without cost, some options are available only for paying customers. Quay also offers an on-premise version of the product, to set up a localized image registry in your own servers.

Quay.io introduces features such as server-side image building, fine-grained access controls, and automatic scanning of images for known vulnerabilities.

Quay.io offers live images that creators regularly update. Quay.io users can create their namespaces, with fine-grained access control, and publish their created images to that namespace. Container Catalog users seldom push new images, and mostly consume trusted images from the Red Hat team.



Figure 3.6: The Quay.io welcome page

Private Registries

Image creators or maintainers can choose to make their images publicly available. However, other image creators might prefer to keep their images private, for the following reasons:

- Company privacy and protection of secrets
- Legal restrictions and laws
- Avoidance of publishing images in development

In some cases, private images are preferred. Private images are more secure than images in public registries. Private registries give image creators control over image placement, distribution, and usage.

Public Registries

Other public registries, such as Docker Hub and Amazon ECR, are also available for storing, sharing, and consuming container images. These registries can include

official images that the registry owners or the registry community users create and maintain. For example, Docker Hub hosts a Docker Official Image of a WordPress container image. Although the `docker.io/library/wordpress` container image is a Docker Official Image, the container image is not supported by WordPress, Docker, or Red Hat. Instead, the Docker Community, which is a global group of Docker Hub users, supports and maintains the container image. Support for this container image depends on the availability and skills of the Docker Community users.

Consuming container images from public registries brings risks. For example, a container image might include malicious code or vulnerabilities, which can compromise the host system that executes the container image. A host system can also be compromised by public container images, because the images often execute by using the privileged `root` user. Additionally, the software in a container image might not be correctly licensed, or violate licensing terms.

Before you use a container image from a public registry, review and test the container image for your environment. Also ensure that you have the correct permissions to use the software in the container image.

Container Image Identifiers

Several objects provide identifying information about a container image.

Registry

A registry is a content server, such as `registry.access.redhat.com`, to store and share container images. A registry consists of one or more repositories that contain tagged container images.

Name

The name identifies the container image repository, and contains a string of letters, numbers, and special characters. The name refers to the directory or container repository within the container registry where the container image is stored.

For example, consider the fully qualified domain name (FQDN) of the `registry.access.redhat.com/ubi10/httpd-24:1-233` container image. The container image is in the `ubi10/httpd-24` repository in the `registry.access.redhat.com` container registry.

ID or Hash

The ID or hash is the SHA (Secure Hash Algorithm) code to uniquely locate, pull, or verify an image within the registry. The SHA image ID cannot change, and always references the same container image content. The ID or hash is the unique identifier for a container image. For example, the sha256:4186a1ead13fc30796f951694c494e7630b82c320b81e20c020b3b07c888985b image ID always refers to the `registry.access.redhat.com/ubi10/httpd-24:1` container image.

Tag

The tag is a label for a container image in a repository to distinguish the specific image from others, for version control. The tag comes after the image repository name and is delimited by a colon (:).

When an image tag is omitted, the floating tag, `latest`, is used as the default tag. A floating tag is an alias to another tag. In contrast, a fixed tag points to a specific container build. For example, the `registry.access.redhat.com/ubi10/httpd-24:1` container image, `1` is the fixed tag for the image, and corresponds to the floating `latest` flag until a later image is registered.

Container Image Components

A container image is composed of multiple components.

Layers

Container images are created from instructions. Each instruction adds a layer to the container image. Each layer consists of the differences between it and the following layer. The layers are then stacked to create a read-only container image.

Metadata

Metadata includes the instructions and documentation for a container image.

Container Image Instructions and Metadata

Container image layers consist of instructions, or steps, and metadata for building the image. You can override instructions during container creation to adjust the container image according to your needs. Some instructions can affect the running container, and other instructions are for informational purposes only.

The following instructions affect the state of a running container:

ENV

Defines the available environment variables in the container. A container image might include multiple `ENV` instructions. Any container can recognize additional environment variables that are not listed in its metadata.

ARG

Defines build-time variables, typically to make a customizable container build. Developers commonly configure the `ENV` instructions by using the `ARG` instruction. `ARG` is useful for preserving the build-time variables for run time.

USER

Defines the active user in the container. Later instructions run as this user. It is a good practice to define a user other than `root` for security purposes. OpenShift does not honor the user in a container image, for regular cluster users. Only cluster administrators can run containers (pods) with their chosen user IDs (UIDs) and group IDs (GIDs).

ENTRYPOINT

Defines the executable to run when the container is started.

CMD

Defines the command to execute when the container is started. This command is passed to the executable that the `ENTRYPOINT` instruction defines. Base images define a default `ENTRYPOINT` executable, which is usually a shell executable, such as `Bash`.

WORKDIR

Defines the current working directory within the container. Later instructions execute within this directory.

Metadata is used for documentation purposes, and does not affect the state of a running container. You can also override the metadata values during container creation.

The following metadata is for information only, and does not affect the state of the running container:

EXPOSE

Specifies the network port that the application binds to within the container. This metadata does not automatically bind the port on the host, and is used only for documentation purposes.

VOLUME

Defines where to store data outside the container. The value shows the path where your container runtime mounts the directory inside the container. More than one path can be defined to create multiple volumes.

LABEL

Defines a key-value pair in the metadata of the image for organization and image selection.

Container engines are not required to honor metadata in a container image, such as `USER` or `EXPOSE`. A container engine can also recognize additional environment variables that are not listed in the container image metadata.

Base Images

A *base image* is the image that your resulting container image is built on. Your chosen base image determines the Linux distribution and any of the following components:

- Package manager
- Init system
- File-system layout
- Preinstalled dependencies and runtimes

The base image can also influence factors such as image size, vendor support, and processor compatibility.

Red Hat provides enterprise-grade container images that are engineered to be the base operating system layer for your containerized applications. These container images are intended as a common starting point for containers, and are known as *universal base images* (UBI). Red Hat UBI container images are *Open Container Initiative* (OCI) compliant images that contain portions of Red Hat Enterprise Linux. UBI container images include a subset of RHEL content. They provide a set of prebuilt runtime languages, such as Python and Node.js, and associated DNF repositories to add application dependencies. UBI-based images can be distributed without cost or restriction. These images can be deployed to both Red Hat and non-Red Hat platforms, and be pushed to your chosen container registry.

A Red Hat subscription is not required to use or distribute UBI-based images. However, Red Hat provides full support for containers that are built on UBI only if the containers are deployed to a Red Hat platform, such as an RHOCP cluster or RHEL.

Red Hat provides four UBI variants: `standard`, `init`, `minimal`, and `micro`. All UBI variants and UBI-based images use RHEL at their core and are available from the Red Hat Container Catalog. The main differences are as follows:

Standard

This image is the primary UBI, which includes DNF, systemd, and utilities such as `gzip` and `tar`.

Init

This image simplifies running multiple applications within a single container by managing them with systemd.

Minimal

This image is smaller than the `init` image and provides nice-to-have features. This image uses the `microdnf` minimal package manager instead of the full-sized version of DNF.

Micro

This image is the smallest available UBI, and includes only the minimum packages. For example, this image does not include a package manager.

Additionally, Red Hat provides language-specific runtime images, such as for python and node.js.

Inspecting and Managing Container Images

Various tools can inspect and manage container images, including the `oc image` command and Skopeo.

Skopeo

Skopeo is a tool to inspect and manage remote container images. With Skopeo, you can copy and synchronize container images from different container registries and repositories. You can copy an image from a remote repository and save it to a local disk. If you have the appropriate repository permissions, then you can delete an image from container registry. You can use Skopeo to inspect the configuration and contents of a container image, and to list the available tags for a container image. Unlike other container image tools, Skopeo can execute without a privileged account such as `root`. Skopeo does not require a running daemon to execute various operations.

Skopeo is executed with the `skopeo` command-line utility, which you can install with various package managers, such as DNF, Brew, or APT. The `skopeo` utility might already be installed on some Linux-based distributions. You can install the `skopeo` utility on Fedora, CentOS Stream 8 and later, and Red Hat Enterprise Linux 8 and later systems, by using the DNF package manager.

```
[user@host ~]$ sudo dnf -y install skopeo
```

The `skopeo` utility is currently not available as a packaged binary for Windows-based systems. However, the `skopeo` utility is available as a container image from the `quay.io/skopeo/stable` container repository. For more information about the Skopeo container image, refer to the `skopeoimage` overview guide in the Skopeo repository at https://github.com/containers/image_build/tree/main/skopeo

You can alternatively build `skopeo` from source code in a container, or build it locally without using a container. Refer to the installation guide in the Skopeo repository

for more information about installing or building Skopeo from source code at <https://github.com/containers/skopeo/blob/main/install.md#container-images>

The skopeo utility provides commands for managing and inspecting container images and container image registries. For container registries that require authentication, you must first log in to a registry before you can execute additional skopeo commands.

```
[user@host ~]$ skopeo login quay.io
```

NOTE

OpenShift clusters are typically configured with registry credentials. When a pod is created from a container image in a remote repository, OpenShift authenticates to the container registry with the configured registry credentials, and then *pulls* the image, to copy from the remote repository. Because OpenShift automatically uses the registry credentials, you typically do not need to manually authenticate to a container registry when you create a pod. By contrast, the `oc image` command and the skopeo utility require you first to log in to a container registry.

After you log in to a container registry (if required), you can execute additional skopeo commands against container images in a repository. When you execute a skopeo command, you must specify the transport and the repository name. A *transport* is the mechanism to transfer container images between locations. Two common transports are `docker` and `dir`. The `docker` transport is used for container registries, and the `dir` transport is used for local directories.

The `oc image` command and other tools default to the `docker` transport, and so you do not need to specify the transport when executing commands. However, the skopeo utility does not define a default transport; you must specify the transport with the container image name. Most skopeo commands use the `skopeo command [command options] transport://IMAGE-NAME` format. For example, the following `skopeo list-tags` command lists all available tags in a `registry.access.redhat.com/ubi10/httpd-24` container repository by using the `docker` transport:

```
[user@host ~]$ skopeo list-tags docker://registry.access.redhat.com/ubi10/httpd-24
{
  "Repository": "registry.access.redhat.com/ubi10/httpd-24",
```

```
"Tags": [  
    "1",  
...output omitted...  
    "latest"  
...output omitted...
```

The skopeo utility includes other commands for container image management.

skopeo inspect

View detailed information for an image name, such as environment variables and available tags. Use the `skopeo inspect [command options] transport://IMAGE-NAME` command format. You can include the `--config` flag to view the configuration, metadata, and history of a container repository. The following example retrieves the configuration information for the `registry.access.redhat.com/ubi10/httpd-24` container repository:

```
[user@host ~]$ skopeo inspect --config docker://registry.access.redhat.com/ubi10/httpd-24  
...output omitted...  
  "config": {  
    "User": "1001",  
    "ExposedPorts": {  
      "8080/tcp": {},  
      "8443/tcp": {}  
    },  
    "Env": [  
...output omitted...  
      "HTTPD_MAIN_CONF_PATH=/etc/httpd/conf",  
      "HTTPD_MAIN_CONF_MODULES_D_PATH=/etc/httpd/conf.modules.d",  
      "HTTPD_MAIN_CONF_D_PATH=/etc/httpd/conf.d",  
      "HTTPD_TLS_CERT_PATH=/etc/httpd/tls",  
      "HTTPD_VAR_RUN=/var/run/httpd",  
      "HTTPD_DATA_PATH=/var/www",  
      "HTTPD_DATA_ORIG_PATH=/var/www",  
      "HTTPD_LOG_PATH=/var/log/httpd"
```

```

    ],
    "Entrypoint": [
        "container-entrypoint"
    ],
    "Cmd": [
        "/usr/bin/run-httpd"
    ],
    "WorkingDir": "/opt/app-root/src",
    ...output omitted...
}
...output omitted...

```

skopeo copy

Copy an image from one location or repository to another. Use the `skopeo copy transport://SOURCE-IMAGE transport://DESTINATION-IMAGE` format. For example, the following command copies the `quay.io/skopeo/stable:latest` container image to the skopeo repository in the `registry.example.com` container registry:

```
[user@host ~]$ skopeo copy docker://quay.io/skopeo/stable:latest \
docker://registry.example.com/skopeo:latest
```

skopeo delete

Delete a container image from a repository. Use the `skopeo delete [command options] transport://IMAGE-NAME` format. The following command deletes the `skopeo:latest` image from the `registry.example.com` container registry:

```
[user@host ~]$ skopeo delete docker://registry.example.com/skopeo:latest
```

skopeo sync

Synchronize one or more images from one location to another. Use this command to copy all container images from a source to a destination. Use the `skopeo sync [command options] --src transport --dest transport SOURCE DESTINATION` format. The following command synchronizes the `registry.access.redhat.com/ubi10/httpd-24` container repository to the `registry.example.com/httpd-24` container repository:


```
[user@host ~]$ skopeo sync --src docker --dest docker \
registry.access.redhat.com/ubi10/httpd-24 registry.example.com/httpd-24
```

Registry Credentials

Some registries require users to authenticate before accessing container images and metadata. For example, Red Hat containers that are based on RHEL typically require authenticated access:

```
[user@host ~]$ skopeo inspect docker://registry.redhat.io/rhel10/httpd-24
FATA[0000] Error parsing image name "docker://registry.redhat.io/rhel10/httpd-24": un
able to retrieve auth token: invalid username/password: unauthorized: Please login to
the Red Hat Registry using your Customer Portal credentials. Further instructions can
be found here: https://access.redhat.com/RegistryAuthentication
```

You might choose a different image that does not require authentication, such as the UBI 10 image:

```
[user@host ~]$ skopeo inspect docker://registry.access.redhat.com/ubi10:latest
{
  "Name": "registry.access.redhat.com/ubi10",
  "Digest": "sha256:b515...",
  "RepoTags": [
    "10.0",
    "10.0-1745487123",
    ...output omitted....
  ]
}
```

Alternatively, you must execute the `skopeo login` command for the registry before you can access protected images.

```
[user@host ~]$ skopeo login registry.redhat.io
Username: YOUR_USER
Password: YOUR_PASSWORD
Login Succeeded!
```

Skopeo stores the credentials in the `${XDG_RUNTIME_DIR}/containers/auth.json` file, where `${XDG_RUNTIME_DIR}` refers to a directory that is specific to the current user. The credentials are encoded in Base64 format:

```
[user@host ~]$ cat ${XDG_RUNTIME_DIR}/containers/auth.json
{
    "auths": {
        "registry.redhat.io": {
            "auth": "dXNlcjpodW50ZXIy"
        }
    }
}

[user@host ~]$ echo -n dXNlcjpodW50ZXIy | base64 -d
user:hunter2
```

NOTE

For security reasons, the `skopeo login` command does not show your password in the interactive session. Although you do not see what you are typing, Skopeo registers every keystroke. After typing your full password in the interactive session, press **Enter** to start the login.

The oc image Command

The OpenShift command-line interface provides the `oc image` command. You can use this command to inspect, configure, or retrieve information about container images.

The `oc image info` command inspects and retrieves information about a container image. You can use the `oc image info` command to identify the ID or hash SHA and to list the image layers of a container image. You can also review container image metadata, such as environment variables, network ports, and commands. If a container image repository provides a container image in multiple architectures, such as AMD64 or ARM64, then you must include the `--filter-by-os` tag. For example, you can execute the following command to retrieve information about the `registry.access.redhat.com/ubi10/httpd-24:10.0` container image that is based on the AMD64 architecture:

```
[user@host ~]$ oc image info registry.access.redhat.com/ubi10/httpd-24 \
--filter-by-os amd64
Name:          registry.access.redhat.com/ubi10/httpd-24:10.0
Digest:        sha256:b765...
...output omitted...
```

The `oc image` command provides more options for managing container images.

oc image append

Use this command to add layers to container images, and then push the container image to a registry.

oc image extract

You can use this command to extract or copy files from a container image to a local disk. Use this command to access the contents of a container image without first running the image as a container. A running container engine is not required.

oc image mirror

Copy, or *mirror*, container images from one container registry or repository to another. For example, you can use this command to mirror container images between public and private registries. You can also use this command to copy a container image from a registry to a disk. The command mirrors the HTTP structure of a container registry to a directory on a disk. The directory on the disk can then be served as a container registry.

Running Containers as Root

Running containers as the root user is a security risk, because an attacker could exploit the application, access the container, and exploit vulnerabilities to escape from the containerized environment and access the host system. Attackers might escape the containerized environment by exploiting bugs and vulnerabilities that are typically in the kernel or the container runtime.

Traditionally, when an attacker gains access to the container file system by using an exploit, the root user inside the container corresponds to the root user on the host system. If an attacker escapes the container isolation, then they have access to

elevated privileges on the host system, which provides a vector of attack that could cause damage.

Containers that do not run as the `root` user might prove unsuitable for use in your application because of the following limitations:

Non-trivial Containerization

Some applications might require the `root` user. Depending on the application architecture, some applications might not be suitable for non-privileged containers, or might require additional experience to containerize.

For example, applications such as HTTPd and Nginx start a bootstrap process and then create a process with a non-privileged user, which interacts with external users. Such applications are non-trivial to containerize for non-privileged use.

Red Hat provides containerized versions of HTTPd and Nginx that do not require `root` privileges for production usage. You can peruse these validated containers in the Red Hat container registry (<https://catalog.redhat.com/software/containers/explore>).

Required Use of Privileged Utilities

Non-privileged containers cannot bind to privileged ports, such as the 80 or 443 ports. Red Hat advises against using privileged ports. Instead, use port forwarding to avoid the use of privileged ports within container definitions.

Similarly, non-privileged containers cannot use the `ping` utility by default, because it requires elevated privileges to establish raw sockets.

Guided Exercise: Find and Inspect Container Images

Use a supported MySQL container image to run server and client pods in Kubernetes; also test two community images and compare their runtime requirements.

Outcomes

- Locate and run container images from a container registry.
- Inspect remote container images and container logs.
- Set environment variables and override entry points for a container.
- Access files and directories within a container.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise.

```
[student@workstation ~]$ lab start pods-images
```

Instructions

1. Log in to the OpenShift cluster and create the `pods-images` project.
1. Log in to the OpenShift cluster as the `developer` user with the `oc` command.

2. `[student@workstation ~]$ oc login -u developer -p developer \`
3. `https://api.ocp4.example.com:6443`

...output omitted...

4. Create the `pods-images` project.

5. `[student@workstation ~]$ oc new-project pods-images`

...output omitted...

2. Authenticate to `registry.ocp4.example.com:8443`, which is the classroom container registry. This private registry hosts certain copies and tags of community images from Docker and Bitnami, as well as some supported images from Red Hat. Use `skopeo` to log in as the `developer` user, and then retrieve a list of available tags for the `registry.ocp4.example.com:8443/redhattraining/docker-nginx` container repository.
1. Use the `skopeo login` command to log in as the `developer` user with the `developer` password.

2. `[student@workstation ~]$ skopeo login registry.ocp4.example.com:8443`
3. Username: `developer`

4. Password: **developer**

Login Succeeded!

5. The classroom registry contains a copy and specific tags of the `docker.io/library/nginx` container repository. Use the `skopeo list-tags` command to retrieve a list of available tags for the `registry.ocp4.example.com:8443/redhattraining/docker-nginx` container repository.

```
6. [student@workstation ~]$ skopeo list-tags \  
7.   docker://registry.ocp4.example.com:8443/redhattraining/docker-nginx  
8. {  
9.   "Repository": "registry.ocp4.example.com:8443/redhattraining/docker-nginx",  
10.  "Tags": [  
11.    "1.23",  
12.    "1.23-alpine",  
13.    "1.23-alpine-perl",  
14.    "1.23-perl",  
15.    "latest"  
16.  ]
```

```
}
```

3. Create a `docker-nginx` pod from the `registry.ocp4.example.com:8443/redhattraining/docker-nginx:1.23` container image. Investigate any pod failures.

1. Use the `oc run` command to create the `docker-nginx` pod.

```
2. [student@workstation ~]$ oc run docker-nginx \  
3.   --image registry.ocp4.example.com:8443/redhattraining/docker-nginx:1.23
```

pod/docker-nginx created

4. After a few moments, verify the status of the `docker-nginx` pod.

```
5. [student@workstation ~]$ oc get pods
```

6. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

docker-nginx	0/1	Error	2 (23s ago)	4s
--------------	-----	-------	-------------	----

```
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
docker-nginx	0/1	CrashLoopBackOff	3 (29s ago)	38s

The docker-nginx pod failed to start.

7. Investigate the pod failure. Retrieve the logs of the docker-nginx pod to identify a possible cause of the pod failure.

```
8. [student@workstation ~]$ oc logs docker-nginx
```

```
9. ...output omitted...
```

```
10. /docker-entrypoint.sh: Configuration complete; ready for start up
```

```
11. 2025/08/24 18:51:45 [warn] 1#1: the "user" directive makes sense only if the master process runs with super-user privileges, ignored in /etc/nginx/nginx.conf:2
```

```
12. nginx: [warn] the "user" directive makes sense only if the master process runs with super-user privileges, ignored in /etc/nginx/nginx.conf:2
```

```
13. 2025/08/24 18:51:45 [emerg] 1#1: mkdir() "/var/cache/nginx/client_temp" failed (13: Permission denied)
```

```
nginx: [emerg] mkdir() "/var/cache/nginx/client_temp" failed (13: Permission denied)
```

The pod failed to start because of permission issues for the nginx directories.

14. Create a debug pod for the docker-nginx pod.

```
15. [student@workstation ~]$ oc debug pod/docker-nginx
```

```
16. Starting pod/docker-nginx-debug-jf8h9 ...
```

```
17. Pod IP: 10.8.0.28
```

```
18. If you don't see a command prompt, try pressing enter.
```

```
19.
```

```
$
```

20. From the debug pod, verify the permissions of the `/etc/nginx` and `/var/cache/nginx` directories.

```
21. $ ls -la /etc/ | grep nginx
```

```
drwxr-xr-x. 3 root root    132 Dev 14 2022 nginx
$ ls -la /var/cache | grep nginx
drwxr-xr-x. 2 root root    6 Dev 13 2022 nginx
```

Only the root user has permission to the `nginx` directories. The pod must therefore run as the privileged root user to work.

22. Retrieve the user ID (UID) of the `docker-nginx` user to determine whether the user is a privileged or unprivileged account. Then, exit the debug pod.

```
23. $ whoami
```

```
1000770000
$ exit
```

```
Removing debug pod ...
```

Your UID value might differ from the previous output.

A UID over 0 means that the container's user is a non-root account. Recall that OpenShift default security policies prevent regular user accounts, such as the `developer` user, from running pods and their containers as privileged accounts.

24. Confirm that the `docker-nginx:1.23` image requires the root privileged account. Use the `oc image info` command to view the configuration for the image.

```
25. [student@workstation ~]$ oc image info \
```

```
26.   registry.ocp4.example.com:8443/redhattraining/docker-nginx:1.23
```

```
27.
```

```
28. Name:           registry.ocp4.example.com:8443/redhattraining/docker-nginx:1.23
```

```
29. Digest:         sha256:d586...82ab
```

```
30. Media Type:     application/vnd.docker.distribution.manifest.v2+json
```



```

31.Created:      2y ago
32.Image Size:   56.89MB in 6 layers
33.Layers:      31.41MB sha256:025c...84bb
34.             25.47MB sha256:ec0f...9437
35.             627B   sha256:cc9f...c9be
36.             958B   sha256:defc...b074
37.             774B   sha256:8855...3b49
38.             1.403kB sha256:f124...c7db
39.OS:          linux
40.Arch:         amd64
41.Entrypoint:   /docker-entrypoint.sh
42.Command:      nginx -g daemon off;
43.Exposes Ports: 80/tcp
44.Environment:  PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
                 :/bin
45.             NGINX_VERSION=1.23.3
46.             NJS_VERSION=0.7.9
47.             PKG_RELEASE=1~bullseye

```

```

Labels:      maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>

```

The image configuration does not define USER metadata, which confirms that the image must run as the root privileged user.

48. The `docker-nginx:1-23` container image must run as the root privileged user. OpenShift security policies prevent regular cluster users, such as the developer user, from running containers as the root user. Delete the `docker-nginx` pod.

```

49. [student@workstation ~]$ oc delete pod docker-nginx

```

```

pod "docker-nginx" deleted

```

4. Create a `bitnami-mysql` pod, which uses a copy of the Bitnami community MySQL image. The image is available in

the registry.ocp4.example.com:8443/redhattraining/bitnami-mysql container repository.

1. A copy and specific tags of the docker.io/bitnami/mysql container repository are hosted in the classroom registry. Use the skopeo list-tags command to identify available tags for the Bitnami MySQL community image in the registry.ocp4.example.com:8443/redhattraining/bitnami-mysql container repository.

```
2. [student@workstation ~]$ skopeo list-tags \  
3.   docker://registry.ocp4.example.com:8443/redhattraining/bitnami-mysql  
4. {  
5.   "Repository": "registry.ocp4.example.com:8443/redhattraining/bitnami-mys  
6.     "Tags": [  
7.       "8.0.28",  
8.       "8.0.29",  
9.       "8.0.30",  
10.      "8.0.31",  
11.      "latest"  
12.    ]
```

```
}
```

13. Retrieve the configuration of the bitnami-mysql:8.0.31 container image. Use the oc image info command to determine whether the image requires a privileged account by inspecting the image configuration for USER metadata.

```
14. [student@workstation ~]$ oc image info \  
15.   registry.ocp4.example.com:8443/redhattraining/bitnami-mysql:8.0.31  
16. Name:           registry.ocp4.example.com:8443/redhattraining/bitnami-mysql:  
17.    8.0.31  
18. Digest:         sha256:2a5e...7725  
19. Media Type:     application/vnd.docker.distribution.manifest.v2+json  
20. Created:        2y ago  
21. Image Size:     154.8MB in 2 layers  
22. Layers:         30.9MB sha256:f8c1...abb5  
                   123.9MB sha256:31da...c470
```

```
23. OS:          linux
24. Arch:        amd64
25. Entrypoint:  /opt/bitnami/scripts/mysql/entrypoint.sh
26. Command:     /opt/bitnami/scripts/mysql/run.sh
27. User:        1001
28. Exposes Ports: 3306/tcp
```

...output omitted...

The image defines the 1001 UID, which means that the image does not require a privileged account.

29. Create the `bitnami-mysql` pod with the `oc run` command. Use the `registry.ocp4.example.com:8443/redhattraining/bitnami-mysql:8.0.31` container image. Then, wait a few moments and then retrieve the pod's status with the `oc get` command.

```
30. [student@workstation ~]$ oc run bitnami-mysql \
31.   --image registry.ocp4.example.com:8443/redhattraining/bitnami-mysql:8.0.31
```

```
pod/bitnami-mysql created
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
bitnami-mysql	0/1	CrashLoopBackoff	2 (16s ago)	42s

The pod failed to start.

32. Examine the logs of the `bitnami-mysql` pod to determine the cause of the failure.

```
33. [student@workstation ~]$ oc logs bitnami-mysql
34. mysql 16:18:00.40
35. mysql 16:18:00.40 Welcome to the Bitnami mysql container
36. mysql 16:18:00.40 Subscribe to project updates by watching https://github.com/bitnami/containers
37. mysql 16:18:00.40 Submit issues and feature requests at https://github.com/bitnami/containers/issues
```

```
38.mysql 16:18:00.40
39.mysql 16:18:00.41 INFO ==> ** Starting MySQL setup **
40.mysql 16:18:00.42 INFO ==> Validating settings in MYSQL_*/MARIADB_* env va
rs
```

```
mysql 16:18:00.42 ERROR ==> The MYSQL_ROOT_PASSWORD environment variable is
empty or not set. Set the environment variable ALLOW_EMPTY_PASSWORD=yes to
allow the container to be started with blank passwords. This is recommended
only for development.
```

The MYSQL_ROOT_PASSWORD environment variable must be set for the pod to start.

41. Delete and then re-create the bitnami-mysql pod. Specify redhat123 as the value for the MYSQL_ROOT_PASSWORD environment variable. After a few moments, verify the status of the pod.

```
42. [student@workstation ~]$ oc delete pod bitnami-mysql
```

```
pod "bitnami-mysql" deleted
[student@workstation ~]$ oc run bitnami-mysql \
  --image registry.ocp4.example.com:8443/redhattraining/bitnami-
mysql:8.0.31 \
  --env MYSQL_ROOT_PASSWORD=redhat123
pod/bitnami-mysql created
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
bitnami-mysql	1/1	Running	0	20s

The bitnami-mysql pod successfully started.

43. Determine the UID of the container user in the bitnami-mysql pod. Compare this value to the UID in the container image and to the UID range of the pods-images project.

```
44. [student@workstation ~]$ oc exec -it bitnami-mysql -- /bin/bash -c "whoami
&& id"
45. 1000770000
```

```
uid=1000770000(1000770000) gid=0(root) groups=0(root),1000770000
```

```
[student@workstation ~]$ oc describe project pods-images
Name:                pods-images
...output omitted...
Annotations:   openshift.io/description=
...output omitted...
               openshift.io/sa.scc.supplemental-groups=1000770000/10000
               openshift.io/sa.scc.uid-range=1000770000/10000
...output omitted...
```

Your values for the UID of the container and the UID range of the project might differ from the previous output.

The container user UID is the same as the specified UID range in the namespace. Notice that the container user UID does not match the 1001 UID of the container image. For a container to use the specified UID of a container image, the pod must be created with a privileged OpenShift user account, such as the admin user.

5. The private classroom registry hosts a copy of a supported MySQL image from Red Hat. Retrieve the list of available tags for the `registry.ocp4.example.com:8443/rhel9/mysql-80` container repository. Compare the `rhel9/mysql-80` container image release version that is associated with each tag.
1. Use the `skopeo list-tags` command to list the available tags for the `rhel9/mysql-80` container image.

```
2. [student@workstation ~]$ skopeo list-tags \
3.   docker://registry.ocp4.example.com:8443/rhel9/mysql-80
4. {
5.   "Repository": "registry.ocp4.example.com:8443/rhel9/mysql-80",
6.   "Tags": [
7.     "1-224",
8.     "1-224-source",
9.     "1-228",
10.    "1-228-source",
11.    "1-237",
12.    "latest",
```

```
13.         "1"
```

```
14.     ]
```

```
}
```

Several tags are available:

- The `latest` and `1` tags are floating tags, which are aliases to other tags, such as the `1-237` tag.
 - The `1-228` and `1-224` tags are fixed tags, which point to a build of a container.
 - The `1-228-source` and `1-224-source` tags are source containers, which provide the necessary sources and license terms to rebuild and distribute the images.
15. Use the `skopeo inspect` command to compare the `rhel9/mysql-80` container image release versions and SHA IDs that are associated with the identified tags.

NOTE

To improve readability, the instructions truncate the SHA-256 strings.

On your system, the commands return the full SHA-256 strings.

```
[student@workstation ~]$ skopeo inspect \
    docker://registry.ocp4.example.com:8443/rhel9/mysql-80:latest
...output omitted...
    "Name": "registry.ocp4.example.com:8443/rhel9/mysql-80",
    "Digest": "sha256:d282...f38f",
...output omitted...
    "Labels":
...output omitted...
        "name": "rhel9/mysql-80",
        "release": "237",
...output omitted...
```

You can also format the output of the `skopeo inspect` command with a Go template. Append the template objects with `\n` to add new lines between the results.

```
[student@workstation ~]$ skopeo inspect --format \
    "Name: {{.Name}}\n Digest: {{.Digest}}\n Release: {{.Labels.release}}" \
    docker://registry.ocp4.example.com:8443/rhel9/mysql-80:latest
Name: registry.ocp4.example.com:8443/rhel9/mysql-80
Digest: sha256:d282...f38f
Release: 237

[student@workstation ~]$ skopeo inspect --format \
    "Name: {{.Name}}\n Digest: {{.Digest}}\n Release: {{.Labels.release}}" \
    docker://registry.ocp4.example.com:8443/rhel9/mysql-80:1
Name: registry.ocp4.example.com:8443/rhel9/mysql-80
Digest: sha256:d282...f38f
Release: 237

[student@workstation ~]$ skopeo inspect --format \
    "Name: {{.Name}}\n Digest: {{.Digest}}\n Release: {{.Labels.release}}" \
    docker://registry.ocp4.example.com:8443/rhel9/mysql-80:1-237
Name: registry.ocp4.example.com:8443/rhel9/mysql-80
Digest: sha256:d282...f38f
Release: 237
```

The `latest`, `1`, and `1-237` tags resolve to the same release versions and SHA IDs. The `latest` and `1` tags are floating tags for the `1-237` fixed tag.

6. The classroom registry hosts a copy and certain tags of the `registry.redhat.io/rhel9/mysql-80` container repository. Use the `oc run` command to create a `rhel9-mysql` pod from the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` container image. Verify the status of the pod and then inspect the container logs for any errors.
1. Create a `rhel9-mysql` pod with the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` container image.

```
2. [student@workstation ~]$ oc run rhel9-mysql \
```

```
3. --image registry.ocp4.example.com:8443/rhel9/mysql-80:1-237
```

```
pod/rhel9-mysql created
```

4. After a few moments, retrieve the pod's status with the `oc get` command.

```
5. [student@workstation ~]$ oc get pods
```

6. NAME	READY	STATUS	RESTARTS	AGE
7. bitnami-mysql	1/1	Running	0	12m

rhel9-mysql	0/1	CrashLoopBackoff	1 (2s ago)	16s
-------------	-----	------------------	------------	-----

The pod failed to start.

8. Retrieve the logs for the `rhel9-mysql` pod to determine why the pod failed.

```
9. [student@workstation ~]$ oc logs rhel9-mysql
```

```
10. => sourcing 20-validate-variables.sh ...
```

11. **You must either specify the following environment variables:**

```
12. MYSQL_USER (regex: '^[a-zA-Z0-9_]+$')
```

```
13. MYSQL_PASSWORD (regex: '^[a-zA-Z0-9_~!@#%&*()-=<>,.?;:|]+$')
```

```
14. MYSQL_DATABASE (regex: '^[a-zA-Z0-9_]+$')
```

15. Or the following environment variable:

```
16. MYSQL_ROOT_PASSWORD (regex: '^[a-zA-Z0-9_~!@#%&*()-=<>,.?;:|]+$')
```

17. Or both.

18. Optional Settings:

```
19. MYSQL_LOWER_CASE_TABLE_NAMES (default: 0)
```

```
...output omitted...
```

The pod failed because the required environment variables were not set for the container.

7. Delete the `rhel9-mysql` pod. Create another `rhel9-mysql` pod and specify the necessary environment variables. Retrieve the status of the pod and inspect the container logs to confirm that the new pod is working.

1. Delete the `rhel9-mysql` pod with the `oc delete` command. Wait for the pod to delete before continuing to the next step.

```
2. [student@workstation ~]$ oc delete pod rhel9-mysql
```

```
pod "rhel9-mysql" deleted
```

3. Create another `rhel9-mysql` pod from the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` container image. Use the `oc run` command with the `--env` option to specify the following environment variables and their values:

Variable	Value
MYSQL_USER	redhat
MYSQL_PASSWORD	redhat123
MYSQL_DATABASE	worldx

```
4. [student@workstation ~]$ oc run rhel9-mysql \  
5.   --image registry.ocp4.example.com:8443/rhel9/mysql-80:1-237 \  
6.   --env MYSQL_USER=redhat \  
7.   --env MYSQL_PASSWORD=redhat123 \  
8.   --env MYSQL_DATABASE=worldx \  
9. pod/rhel9-mysql created
```

10. After a few moments, retrieve the status of the `rhel9-mysql` pod with the `oc get` command. View the container logs to confirm that the database on the `rhel9-mysql` pod is ready to accept connections.

```
11. [student@workstation ~]$ oc get pods  
12. NAME                READY   STATUS    RESTARTS   AGE  
13. bitnami-mysql 1/1     Running   0           10m
```

```
rhel9-mysql 1/1     Running   0           20s  
[student@workstation ~]$ oc logs rhel9-mysql  
...output omitted...
```

```
2025-08-26T20:14:14.333599Z 0 [System] [MY-013576] [Server] X Plugin ready
for connections. Bind-address: '::' port: 33060, socket: /var/lib/mysql/mys
qlx.sock
```

```
2025-08-26T20:14:14.333641Z 0 [System] [MY-013576] [Server] /usr/libexec/my
sqld: ready for connections. Version: '8.0.30' socket: '/var/lib/mysql/mys
ql.sock' port: 3306 Source distribution.
```

The `rhel9-mysql` pod is ready to accept connections.

8. Determine the location of the MySQL database files for the `rhel9-mysql` pod. Confirm that the directory contains the `worldx` database.
1. Use the `oc image` command to inspect the `rhel9/mysql-80:1-237` image in the registry.`ocp4.example.com:8443` classroom registry.

```
2. [student@workstation ~]$ oc image info \
3.   registry.ocp4.example.com:8443/rhel9/mysql-80:1-237
4. Name:          registry.ocp4.example.com:8443/rhel9/mysql-80:1-237
5. ...output omitted...
6. Command:       run-mysqld
7. Working Dir:   /opt/app-root/src
8. User:          27
9. Exposes Ports: 3306/tcp
10. Environment:  container=oci
11.               STI_SCRIPTS_URL=image:///usr/libexec/s2i
12.               STI_SCRIPTS_PATH=/usr/libexec/s2i
13.               APP_ROOT=/opt/app-root
14.               PATH=/opt/app-root/src/bin:/opt/app-root/bin:/usr/local/sbin
:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
15.               PLATFORM=e19
16.               MYSQL_VERSION=8.0
17.               APP_DATA=/opt/app-root/src
```

```
HOME=/var/lib/mysql
```

The container manifest sets the `HOME` environment variable for the container user to the `/var/lib/mysql` directory.

18. Use the `oc exec` command to list the contents of the `/var/lib/mysql` directory.

```
19. [student@workstation ~]$ oc exec -it rhel9-mysql -- ls -la /var/lib/mysql
20. total 12
21. drwxrwxr-x. 1 mysql root          102 Aug 26 16:28 .
22. drwxr-xr-x. 1 root  root           19 Jan 17  2023 ..
23. drwxrwxr-x. 1 mysql root        4096 Aug 26 20:54 data
24. srwxrwxrwx. 1 mysql 1000770000     0 Aug 26 16:28 mysql.sock
25. -rw-----. 1 mysql 1000770000     2 Aug 26 16:28 mysql.sock.lock
26. srwxrwxrwx. 1 mysql 1000770000     0 Aug 26 16:28 mysqlx.sock
```

```
-rw-----. 1 mysql 1000770000     2 Aug 26 16:28 mysqlx.sock.lock
```

A data directory exists in the `/var/lib/mysql` directory.

27. Use the `oc exec` command again to list the contents of the `/var/lib/mysql/data` directory.

```
28. [student@workstation ~]$ oc exec -it rhel9-mysql \
29.  -- ls -la /var/lib/mysql/data | grep worldx
```

```
drwxr-x---. 2 1000770000 root      6 Aug  26 16:28 worldx
```

The `/var/lib/mysql/data` directory contains the `worldx` database with the `worldx` directory.

9. Determine the IP address of the `rhel9-mysql` pod. Next, create another MySQL pod, named `mysqlclient`, to access the `rhel9-mysql` pod. Confirm that the `mysqlclient` pod can view the available databases on the `rhel9-mysql` pod with the `mysqlshow` command.

1. Identify the IP address of the `rhel9-mysql` pod.

```
2. [student@workstation ~]$ MYSQL_IP=$(oc get pods rhel9-mysql -o=jsonpath='{.status.podIP}') && echo $MYSQL_IP
```

```
"10.8.0.69"
```

Note the IP address stored in the `MYSQL_IP` variable. Your IP address might differ from the previous output.

3. Use the `oc run` command to create a pod named `mysqlclient` that uses the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` container image. Set the value of the `MYSQL_ROOT_PASSWORD` environment variable to `redhat123`, and then confirm that the pod is running.

```
4. [student@workstation ~]$ oc run mysqlclient \  
5.   --image registry.ocp4.example.com:8443/rhel9/mysql-80:1-237 \  
6.   --env MYSQL_ROOT_PASSWORD=redhat123
```

```
pod/mysqlclient created  
[student@workstation ~]$ oc get pods  


| NAME          | READY | STATUS  | RESTARTS | AGE |
|---------------|-------|---------|----------|-----|
| bitnami-mysql | 1/1   | Running | 0        | 15m |
| mysqlclient   | 1/1   | Running | 0        | 19s |
| rhel9-mysql   | 1/1   | Running | 0        | 8m  |


```

7. Use the `oc exec` command with the `-it` options to execute the `mysqlshow` command on the `mysqlclient` pod. Connect as the `redhat` user and specify the host as the IP address of the `rhel9-mysql` pod. When prompted, enter `redhat123` for the password, although no characters are shown.

```
8. [student@workstation ~]$ oc exec -it mysqlclient \  
9.   -- mysqlshow -u redhat -p -h $MYSQL_IP  
10. Enter password: redhat123  
11. +-----+  
12. |      Databases      |  
13. +-----+  
14. | information_schema |  
15. | performance_schema |  
16. | worldx             |
```

```
+-----+
```

The `worldx` database on the `rhel9-mysql` pod is accessible to the `mysql-client` pod.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish pods-images
```

Troubleshoot Containers and Pods

Objectives

- Troubleshoot a pod by starting additional processes on its containers, changing their ephemeral file systems, and opening short-lived network tunnels.

Container Troubleshooting Overview

Containers are designed to be immutable and ephemeral. When changes are needed or a new container image is available, you can apply them to running containers without redeployment.

Updating a running container is best reserved for troubleshooting problematic containers. Red Hat does not generally recommend editing a running container to fix errors in a deployment. Changes to a running container are not captured in source control, but help to identify the needed corrections to the source code for the container functions. Capture these container updates in version control after you identify the necessary changes. Then, build a new container image and redeploy the application.

Custom alterations to a running container are incompatible with elegant architecture, reliability, and resilience for the environment.

CLI Troubleshooting Tools

Administrators use various tools to interact with, inspect, and alter running containers. Administrators can use commands such as `oc get` to gather initial details for a specified resource type. Other commands are available for detailed inspection of a resource, or to update a resource in real time.

NOTE

When interacting with the cluster containers, take suitable precautions with actively running components, services, and applications.

Use these tools to validate the functions and environment for a running container:

- `oc describe`: Display the details of a resource.
- `oc edit`: Edit a resource configuration by using the system editor.
- `oc patch`: Update a specific attribute or field for a resource.
- `oc cp`: Copy files and directories to and from containers.
- `oc exec`: Execute a command within a specified container.
- `oc port-forward`: Configure a port forwarder for a specified container.
- `oc logs`: Retrieve the logs for a specified container.
- `oc rsync`: Synchronize files and directories to and from containers.
- `oc rsh`: Start a remote shell within a specified container.

Editing Resources

Troubleshooting and remediation often begin with a phase of inspection and data gathering. When solving issues, the `describe` command can provide helpful details about the running resource, such as the definition of a container and its purpose.

The following example demonstrates use of the `oc describe RESOURCE NAME` command to retrieve information about a pod in the `openshift-dns` namespace:

```
[user@host ~]$ oc describe pod dns-default-1t13h
Name: dns-default-1t13h
Namespace: openshift-dns
Priority: 2000001000
Priority Class Name: system-node-critical
...output omitted...
```

Various CLI tools can apply a change that you determine is needed to a running container. The `edit` command opens the specified resource in the default editor for your environment. This editor is specified by setting either the `KUBE_EDITOR` or the `EDITOR` environment variable, or otherwise with the editor application in your operating system.

The following example demonstrates use of the `oc edit RESOURCE_NAME` command to edit a running container:

```
[user@host ~]$ oc edit pod mongo-app-sw88b

# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: Pod
metadata:
  annotations:
...output omitted...
```

You can also use the `patch` command to update fields of a resource.

The following example uses the `patch` command to update the container image that a pod uses:

```
[user@host ~]$ oc patch pod valid-pod --type='json' \
-p='[{"op": "replace", "path": "/spec/containers/0/image", \
"value": "http://registry.access.redhat.com/ubi8/httpd-24"}]'
```

Copy Files to and from Containers

Administrators can copy files and directories to or from a container to inspect, update, or correct functionality. Adding a configuration file or retrieving an application log are common use cases.

NOTE

To use the `cp` command with the `oc` CLI, the `tar` binary must be present in the container. If the binary is absent, then an error message appears and the operation fails.

The following example demonstrates copying a file from a running container to a local directory by using the `oc cp SOURCE DEST` command:

```
[user@host ~]$ oc cp apache-app-kc82c:/var/www/html/index.html /tmp/index.bak

[user@host ~]$ ls /tmp
index.bak
```

The following example demonstrates use of the `oc cp SOURCE DEST` command to copy a file from a local directory to a directory in a running container:

```
[user@host ~]$ oc cp /tmp/index.html apache-app-kc82c:/var/www/html/

[user@host ~]$ oc exec -it apache-app-kc82c -- ls /var/www/html
index.html
```

NOTE

Targeting a file path within a pod for either the `SOURCE` or `DEST` arguments uses the `pod_name:path` format, and can include the `-c container_name` option to specify a container within the pod. If you omit the `-c container_name` option, then the command targets the first container in the pod.

Additionally, when using the `oc` CLI, file and directory synchronization is available by using the `oc rsync` command.

The following example demonstrates use of the `oc rsync SOURCE_NAME DEST` command to synchronize files from a running container to a local directory.

```
[user@host ~]$ oc rsync apache-app-kc82c:/var/www/ /tmp/web_files
```



```
[user@host ~]$ ls /tmp/web_files  
cgi-bin  
html
```

The `oc rsync` command uses the `rsync` client on your local system to copy changed files to and from a pod container. The `rsync` binary must be available locally and within the container for this approach. If the `rsync` binary is not found, then a `tar` archive is created on the local system and is sent to the container. The container then uses the `tar` utility to extract files from the archive. Without the `rsync` and `tar` binaries, an error message occurs and the `oc rsync` command fails.

NOTE

For Linux-based systems, you can install the `rsync` client and the `tar` utility on a local system by using a package manager, such as DNF. For Windows-based systems, you can install the `cwRsync` client. For more information about the `cwRsync` client, refer to <https://www.itefix.net/cwrsync>.

Remote Container Access

Exposing a network port for a container is routine, especially for containers that provide a service. In a cluster, port forwarding connections are made through the kubelet, which maps a local port on your system to a port on a pod. Configuring port forwarding creates a request through the Kubernetes API, and creates a multiplexed stream, such as HTTP/2, with a `port` header that specifies the target port in the pod. The kubelet delivers the stream data to the target pod and port, and vice versa for egress data from the pod.

When troubleshooting an application that typically runs without a need to connect locally, you can use the port-forwarding function to expose connectivity to the pod for investigation. With this function, an administrator can connect on the new port and inspect the problematic application. After you remediate the issue, the application can be redeployed without the port-forward connection.

To forward a local port to a pod's port, use the following command syntax:

```
[user@host ~]$ oc port-forward RESOURCE EXTERNAL_PORT:CONTAINER_PORT
```

The following example demonstrates the use of the `oc port-forward` command to listen locally on port 8080 and to forward connections to port 80 on the pod:

```
[user@host ~]$ oc port-forward nginx-app-cc78k 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Connect to Running Containers

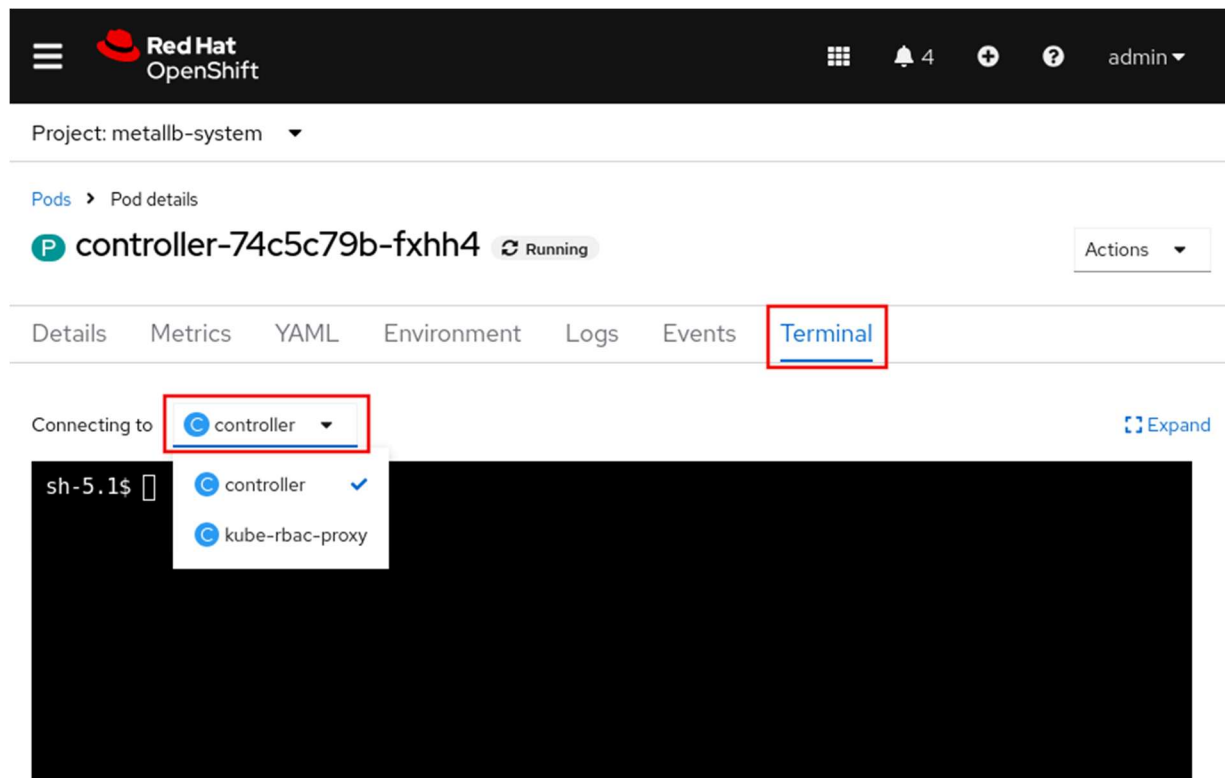
Administrators use CLI tools to connect to a container via a shell for forensic inspections. With this approach, you can connect to, inspect, and run any available commands within the specified container.

The following example demonstrates use of the `oc rsh` *POD_NAME* command to connect to a container via a shell:

```
[user@host ~]$ oc rsh tomcat-app-jw53r
sh-4.4#
```

If you need to connect to a specific container in a pod, then use the `-c container_name` option to specify the container name. If you omit this option, then the command connects to the first container in the pod.

You can also connect to running containers from the RHOC web console. Click **Terminal** from the **Pod details** page, and select the running container from the **Connecting to** menu. If you have more than one container, then you can change between them to connect to the CLI.



Execute Commands in a Container

Passing commands to execute within a container from the CLI is another method for troubleshooting a running container. Use this method to send a command to run within the container, or to connect to the container, when further investigation is necessary.

Use the following command to pass and execute commands in a container:

```
[user@host ~]$ oc exec POD / TYPE/NAME [-c container_name] -- COMMAND [arg1 ... argN]
```

If you omit the `-c container_name` option, then the command targets the first container in the pod.

The following examples demonstrate the use of the `oc exec` command to execute the `ls` command in a container to list the contents of the container's root directory:

```
[user@host ~]$ oc exec -it mariadb-1c78h -- ls /  
bin boot dev etc help.1 home lib lib64 ...
```

```
...output omitted...  
[user@host ~]$ oc exec mariadb-1c78h -- ls /  
bin  
boot  
dev  
etc  
...output omitted...
```

NOTE

It is common to add the `-it` flags to the `oc exec` command. These flags instruct the command to send `STDIN` to the container and `STDOUT/STDERR` back to the terminal. The format of the command output is impacted by the inclusion of the `-it` flags.

Container Events and Logs

Reviewing the historical actions for a container can offer insights into both the lifecycle and health of the deployment. Retrieving the cluster logs provides the chronological details of the container actions. Administrators inspect this log output for information and issues that occur in the running container.

For the following commands, use the `-c container_name` to specify a container in the pod. If you omit this option, then the command targets the first container in the pod.

The following examples demonstrate use of the `oc logs POD_NAME` command to retrieve the logs for a pod:

```
[user@host ~]$ oc logs BIND9-app-rw43j  
Defaulted container "dns" out of: dns  
.:5353  
[INFO] plugin/reload: Running configuration SHA512 = 7c3d...3587  
CoreDNS-1.9.2  
...output omitted...
```

In Kubernetes, an event resource is a report of an event somewhere in the cluster. You can use the `oc get events` command to view pod events in a namespace:

```
[user@host ~]$ oc get events
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
...output omitted...				
21m	Normal	AddedInterface	pod/php-app-5d9b84b588-kzfxd	Add eth0 [10.8.0.93/23] from ovn-kubernetes
21m	Normal	Pulled	pod/php-app-5d9b84b588-kzfxd	Container image "registry.ocp4.example.com:8443/redhattraining/php-webapp:v4" already present on machine
21m	Normal	Created	pod/php-app-5d9b84b588-kzfxd	Created container php-webapp
21m	Normal	Started	pod/php-app-5d9b84b588-kzfxd	Started container php-webapp

Available Linux Commands in Containers

The use of Linux commands for troubleshooting applications can also help with troubleshooting containers. However, when connecting to a container, only the defined tools and applications within the container are available. You can augment the environment inside the container by adding the tools from this section or any other remedial tools to the container image.

Before you add tools to a container image, consider how the tools affect your container image:

- Additional tools increase the size of the image, which might impact container performance.
- Tools might require additional update packages and licensing terms, which can impact the ease of updating and distributing the container image.
- Hackers might exploit tools in the image.

Troubleshooting from Inside the Cluster

Administrators can alternatively author and deploy a container within the cluster for investigation and remediation. By creating a container image that includes the cluster troubleshooting tools, you have a reliable environment to perform these tasks from any computer with access to the cluster. This approach ensures that an administrator always has access to the tools for reliable troubleshooting and remediation of issues.

Additionally, administrators should plan to author a container image that provides the most valuable troubleshooting tools for containerized applications. In this way, you deploy this "toolbox" container to supplement the forensic process and to provide an environment with the required commands and tools for troubleshooting problematic containers. For example, the `toolbox` container can test how resources operate inside a cluster, such as to confirm whether a pod can connect to resources outside the cluster. Regular cluster users can also create a `toolbox` container to help with application troubleshooting. For example, a regular user could run a pod with a MySQL client to connect to another pod that runs a MySQL server.

Although this approach falls outside the focus of this course, because it is more application-level remediation than container-level troubleshooting, it is important to realize that containers have such capacity.

Guided Exercise: Troubleshoot Containers and Pods

Troubleshoot and fix a failed MySQL pod and manually initialize a database with test data.

Outcomes

- Investigate errors with creating a pod.
- View the status, logs, and events for a pod.
- Copy files into a running pod.
- Connect to a running pod by using port forwarding.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster and all exercise resources are available.

```
[student@workstation ~]$ lab start pods-troubleshooting
```

Instructions

1. Log in to the OpenShift cluster and create the `pods-troubleshooting` project.
1. Open a new terminal window and log in to the OpenShift cluster as the `developer` user with `developer` as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
```

3. `https://api.ocp4.example.com:6443`

4. Login successful.

...output omitted...

5. Create the pods-troubleshooting project.

6. `[student@workstation ~]$ oc new-project pods-troubleshooting`

7. Now using project "pods-troubleshooting" on server "`https://api.ocp4.example.com:6443`".

...output omitted...

2. Create the MySQL `mysql-server` pod. Use the `registry.ocp4.example.com:8443/rhel9/mysql-80:1228` container image for the pod. Specify the following environment variables:

Variable	Value
<code>MYSQL_USER</code>	<code>redhat</code>
<code>MYSQL_PASSWORD</code>	<code>redhat123</code>
<code>MYSQL_DATABASE</code>	<code>world</code>

1. Use the `oc run` command to create the `mysql-server` pod. Specify the environment values with the `--env` option.

```
2. [student@workstation ~]$ oc run mysql-server \  
3.   --image registry.ocp4.example.com:8443/rhel9/mysql-80:1228 \  
4.   --env MYSQL_USER=redhat \  
5.   --env MYSQL_PASSWORD=redhat123 \  
6.   --env MYSQL_DATABASE=world
```

`pod/mysql-server` created

- Retrieve the status of the pod by using the `oc get pods` command. Execute the `oc get pods` command a few times to view the status updates for the pod.

```
8. [student@workstation ~]$ oc get pods
```

9. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

mysql-server	0/1	ImagePullBackoff	0	45s
--------------	-----	------------------	---	-----

The pod fails to start.

- Identify the root cause of the pod's failure to start.
- Use the `oc logs` command to retrieve the pod's logs.

```
2. [student@workstation ~]$ oc logs mysql-server
```

```
Error from server (BadRequest): container "mysql-server" in pod "mysql-serv
er" is waiting to start: trying and failing to pull image
```

The logs state that the pod cannot pull the container image.

- Retrieve the events log with the `oc get events` command.

```
4. student@workstation ~]$ oc get events
```

5. LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
6. 33s	Normal	Scheduled	pod/mysql-server	Successfully assigned pods-troubleshooting/mysql-server to master01
7. 31s	Normal	AddedInterface	pod/mysql-server	Add eth0 [10.8.0.68/23] from ovn-kubernetes
8. 16s	Normal	Pulling	pod/mysql-server	Pulling image "registry.ocp4.example.com:8443/rhel9/mysql-80:1228"
9. 16s	Warning	Failed	pod/mysql-server	Failed to pull image "registry.ocp4.example.com:8443/rhel9/mysql-80:1228": initializing source docker://registry.ocp4.example.com:8443/rhel9/mysql-80:1228: reading manifest 1228 in registry.ocp4.example.com:8443/rhel9/mysql-80: manifest unknown
10. 16s	Warning	Failed	pod/mysql-server	Error: ErrImagePull
11. 4s	Normal	BackOff	pod/mysql-server	Back-off pulling image "registry.ocp4.example.com:8443/rhel9/mysql-80:1228"


```
4s      Warning   Failed           pod/mysql-server   Error: ImagePullB
ackOff
```

The output states that the image pull failed because the 1228 manifest is unknown. This failure could mean that the manifest, or image tag, does not exist in the repository.

12. Authenticate to the container repository with the `skopeo login` command as the `developer` user with `developer` as the password.

```
13. [student@workstation ~]$ skopeo login registry.ocp4.example.com:8443
14. Username: developer
15. Password: developer
```

```
Login Succeeded!
```

16. Use the `skopeo inspect` command to retrieve the available manifests in the `registry.ocp4.example.com:8443/rhel9/mysql-80` repository.

```
17. [student@workstation ~]$ skopeo inspect \
18.   docker://registry.ocp4.example.com:8443/rhel9/mysql-80
19. ...output omitted...
20.   "Name": "registry.ocp4.example.com:8443/rhel9/mysql-80",
21.   "Digest": "sha256:5098...72a1",
22.   "RepoTags": [
23.     "1",
24.     "1-224",
25.     "1-224-source",
26.     "1-228",
27.     "1-228-source",
28.     "1-237",
29.     "latest"
30.   ],
```

```
...output omitted...
```

The 1228 manifest, or tag, is not available in the repository, which means that the `registry.ocp4.example.com:8443/rhel9/mysql-80:1228` image does not exist. However, the 1-228 tag does exist.

4. Update the pod's configuration to use the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-228` container image. Confirm that the pod is re-created after editing the resource.
1. Locate the `.spec.containers.image` object. Use the `oc edit` command to update the value to the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-228` container image, and save the change.

```
2. [student@workstation ~]$ oc edit pod/mysql-server
3. ...output omitted...
4. apiVersion: v1
5. kind: Pod
6. metadata:
7. ...output omitted...
8. spec:
9.   containers:
10.    - image: registry.ocp4.example.com:8443/rhel9/mysql-80:1-228
```

```
...output omitted...
```

11. Verify the status of the `mysql-server` pod with the `oc get` command. The pod's status might take a few moments to update after the resource edit. Repeat the `oc get` command until the pod's status changes.

```
12. [student@workstation ~]$ oc get pods
13. NAME                READY   STATUS    RESTARTS   AGE
```

```
mysql-server    1/1     Running   0           10m
```

The `mysql-server` pod successfully pulled the image and created the container. The pod now shows a `Running` status.

5. Copy the `~/D0180/labs/pods-troubleshooting/world_x.sql` file to the `mysql-server` pod. Then, connect to the pod and execute the `world_x.sql` file to populate the `world` database.

1. Use the `oc cp` command to copy the `world_x.sql` file in the `~/D0180/labs/pods-troubleshooting` directory to the `/tmp/` directory on the `mysql-server` pod.

```
2. [student@workstation ~]$ oc cp ~/D0180/labs/pods-troubleshooting/world_x.sql \
```

```
mysql-server:/tmp/
```

3. Confirm that the `world_x.sql` file is accessible within the `mysql-server` pod with the `oc exec` command.

```
4. [student@workstation ~]$ oc exec -it pod/mysql-server -- ls -la /tmp/
```

```
5. total 548
```

```
6. drwxrwxrwx. 1 root      root   25   Aug 15 01:53 .
```

```
7. dr-xr-xr-x. 1 root      root   50   Aug 15 01:50 ..
```

```
-rw-r--r--. 1 1000750000 root  558791 Aug 15 01:53 world_x.sql
```

8. Connect to the `mysql-server` pod with the `oc rsh` command.

```
9. [student@workstation ~]$ oc rsh mysql-server
```

```
sh-5.1$
```

10. Log in to MySQL as the `redhat` user with `redhat123` as the password.

```
11. sh-5.1$ mysql -u redhat -p
```

```
12. Enter password: redhat123
```

```
13. Welcome to the MySQL monitor.  Commands end with ; or \g.
```

```
14. ...output omitted...
```

```
mysql>
```

15. From the MySQL prompt, select the `world` database.

```
16. mysql> USE world
```

```
Database changed
```

17. Source the `world_x.sql` script inside the pod to initialize and populate the `world` database.

```
18. mysql> SOURCE /tmp/world_x.sql
19. Query OK, 1 row affected (0.00 sec)
```

...output omitted...

20. Execute the `SHOW TABLES;` command to confirm that the database now contains tables. Then, exit the database and the pod.

```
21. mysql> SHOW TABLES;
22. -----
23. | Tables_in_test |
24. -----
25. | city           |
26. | country        |
27. | countryinfo    |
28. | countrylanguage |
29. -----
30. 4 rows in set (0.00 sec)
31. mysql> exit
32. Bye
33. sh-5.1$ exit
34. exit
```

[student@workstation ~]\$

6. Configure port forwarding and then use the MySQL client on the terminal to connect to the `world` database on the `mysql-server` pod. Confirm that you can access data within the `world` database from the terminal.

1. From the terminal, use the `oc port-forward` command to forward the 3306 local port to the 3306 port on the `mysql-server` pod.

```
2. [student@workstation ~]$ oc port-forward mysql-server 3306:3306
3. Forwarding from 127.0.0.1:3306 -> 3306
```

Forwarding from [::1]:3306 -> 3306

4. Open another terminal window on the workstation machine. Connect to the `world` database with the local MySQL client on the workstation machine. Log in as the `redhat` user with the `redhat123` password. Specify the host as the localhost `127.0.0.1` IP address and use `3306` as the port.

5. `[student@workstation ~]$ mysql -u redhat -p -h 127.0.0.1 -P 3306`

6. Enter password: **redhat123**

7. Welcome to the MySQL monitor. Commands end with `;` or `\g`.

8. ...*output omitted*...

`mysql>`

9. Select the `world` database.

10. `mysql> USE world`

11. Reading table information for completion of table and columns names

12. You can turn off this feature to get a quicker startup with `-A`

13.

Database changed

14. Use the `SHOW TABLES;` command to list the tables in the `world` database.

15. `mysql> SHOW TABLES;`

16. -----

17. | Tables_in_test |

18. -----

19. | city |

20. | country |

21. | countryinfo |

22. | countrylanguage |

23. -----

4 rows in set (0.01 sec)

24. Confirm that you can retrieve data from the `country` table. Execute the `SELECT COUNT(*) FROM country` command to retrieve the number of countries within the `country` table.

```
25. mysql> SELECT COUNT(*) FROM country;
26. -----
27. | COUNT(*) |
28. -----
29. |      239 |
30. -----
```

```
1 row in set (0.01 sec)
```

31. Exit the database.

```
32. mysql> exit
33. Bye
```

```
[student@workstation ~]$
```

Close the second terminal window.

34. Return to the terminal that is executing the `oc port-forward` command. Press **Ctrl+C** to end the connection.

```
35. [student@workstation ~]$ oc port-forward mysql-server 3306:3306
36. Forwarding from 127.0.0.1:3306 -> 3306
37. Forwarding from [::1]:3306 -> 3306
38. Handling connection for 3306
39. Handling connection for 3306
```

```
^C[student@workstation ~]$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish pods-troubleshooting
```

Lab: Run Applications as Containers and Pods

Run a web server as a pod and insert a debug page that displays diagnostic information.

Outcomes

- Deploy a pod from a container image.
- Retrieve the status and events of a pod.
- Troubleshoot a failed pod.
- Edit pod resources.
- Copy files to a running pod for diagnostic purposes.
- Use port forwarding to connect to a running pod.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that exercise resources are available.

```
[student@workstation ~]$ lab start pods-review
```

Instructions

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

Log in to the OpenShift cluster as the `developer` user with `developer` as the password.

Use the `pods-review` project for your work.

1. Log in to the OpenShift cluster and change to the `pods-review` project.
1. Log in to the OpenShift cluster.

2. `[student@workstation ~]$ oc login -u developer -p developer \`
3. `https://api.ocp4.example.com:6443`

...output omitted...

4. Select the pods-review project.

5. [student@workstation ~]\$ **oc project pods-review**

Already on project "pods-review" on server "https://api.ocp4.example.com:6443".

2. Deploy a pod named webphp that uses the registry.ocp4.example.com:8443/redhattraining/webphp:v1 container image. Determine why the pod fails to start.

1. Deploy a pod named webphp that uses the registry.ocp4.example.com:8443/redhattraining/webphp:v1 container image.

2. [student@workstation ~]\$ **oc run webphp **

3. **--image=registry.ocp4.example.com:8443/redhattraining/webphp:v1**

pod/webphp created

4. After a few moments, observe the status of the webphp pod.

5. [student@workstation ~]\$ **oc get pods**

6. NAME	READY	STATUS	RESTARTS	AGE
7. webphp	0/1	CrashLoopBackOff	1 (4s ago)	7s

8. [student@workstation ~]\$ **oc get pods**

9. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

webphp	0/1	Error	2 (24s ago)	7s
--------	-----	-------	-------------	----

The pod failed to start.

10. Retrieve the cluster events.

11. [student@workstation ~]\$ **oc get events**

12. LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
13. 3m25s	Normal	Scheduled	pod/webphp	Successfully assigned p ods-review/webphp to master01

14. 3m23s	Normal	AddedInterface	pod/webphp	Add eth0 [10.8.0.73/23] from ovn-kubernetes
15. 3m23s	Normal	Pulling	pod/webphp	Pulling image "registry.ocp4.example.com:8443/redhattraining/webphp:v1"
16. 3m15s	Normal	Pulled	pod/webphp	Successfully pulled image "registry.ocp4.example.com:8443/redhattraining/webphp:v1" in 9.432s (9.432s including waiting). Image size: 292899537 bytes.
17. 104s	Normal	Created	pod/webphp	Created container webphp
18. 104s	Normal	Started	pod/webphp	Started container webphp
19. 104s	Normal	Pulled	pod/webphp	Container image "registry.ocp4.example.com:8443/redhattraining/webphp:v1" already present on machine

103s	Warning	BackOff	pod/webphp	Back-off restarting failed container webphp in pod webphp_pods-review(1a1d3055-5250-4a80-9600-9008a5a4b601)
------	---------	---------	------------	---

20. Retrieve the logs for the webphp pod.

```

21. [student@workstation ~]$ oc logs webphp
22. [15-Aug-2025 18:46:56] NOTICE: [pool www] 'user' directive is ignored when FPM is not running as root
23. [15-Aug-2025 18:46:56] NOTICE: [pool www] 'group' directive is ignored when FPM is not running as root
24. [15-Aug-2025 18:46:56] ERROR: unable to bind listening socket for address '/run/php-fpm/www.sock': Permission denied (13)
25. [15-Aug-2025 18:46:56] ERROR: FPM initialization failed
26. AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 10.8.0.78. Set the 'ServerName' directive globally to suppress this message
27. (13)Permission denied: AH00058: Error retrieving pid file run/httpd.pid

```

AH00059: Remove it before continuing if it is corrupted.

The logs indicate permission issues with the /run directory within the pod.

3. Troubleshoot the failed webphp pod by creating a debug pod.

1. Create a debug pod to troubleshoot the failed webphp pod.

```
2. [student@workstation ~]$ oc debug pod/webphp
```

3. Starting pod/webphp-debug-mfbxx ...
4. Pod IP: 10.8.0.87
5. If you don't see a command prompt, try pressing enter.

```
sh-4.4$
```

6. List the contents of the /run directory to retrieve the permissions, owners, and groups.

```
7. sh-4.4$ ls -la /run
8. total 0
9. drwxr-xr-x. 1 root root  42 Aug 15 18:47 .
10. dr-xr-xr-x. 1 root root  17 Aug 15 18:47 ..
11. -rw-r--r--. 1 root root   0 Aug 15 18:47 .containerenv
12. drwx--x---. 3 root apache 26 Dec 20 18:42 httpd
13. drwxr-xr-x. 2 root root   6 Nov  3 11:10 lock
14. drwxr-xr-x. 2 root root   6 Dec 20 18:42 php-fpm
```

```
drwxr-xr-x. 4 root root  80 Aug 15 18:47 secrets
```

The /run/httpd directory grants read, write, and execute permissions to the root user, but does not provide permissions for the root group.

15. Retrieve the UID and GID of the user in the container. Determine whether the user is a privileged user and belongs to the root group.

```
16. sh-4.4$ id
```

```
uid=1000770000(1000680000) gid=0(root) groups=0(root),1000770000
```

Your UID and GID values might differ from the previous output.

The user is an unprivileged, non-root user and belongs to the root group, which does not have access to the /run directory. Therefore, the user in the container cannot access the files and directories that the container processes use, which is required for arbitrarily assigned UIDs.

17. Exit the debug pod.

```
18.sh-4.4$ exit
19.exit
20.
```

```
Removing debug pod ...
```

4. The application developer resolved the identified issue in the `registry.ocp4.example.com:8443/redhattraining/webphp:v2` container image. In a terminal window, edit the `webphp` pod resource to use the `v2` image tag. Retrieve the status of the `webphp` pod. Then, confirm that the user in the container is an unprivileged user and belongs to the `root` group. Confirm that the `root` group permissions are correct for the `/run/httpd` directory.

1. Use the terminal to edit the `webphp` pod resource.

```
[student@workstation ~]$ oc edit pod/webphp
```

2. Update the `.spec.containers.image` object value to use the `:v2` image tag.

```
3....output omitted...
4.spec:
5.  containers:
6.    - image: registry.ocp4.example.com:8443/redhattraining/webphp:v2
7.      imagePullPolicy: IfNotPresent
```

```
...output omitted...
```

8. Verify the status of the `webphp` pod.

```
9.[student@workstation ~]$ oc get pods
```

10.NAME	READY	STATUS	RESTARTS	AGE
webphp	1/1	Running	9 (2m9s ago)	18m

11.Retrieve the UID and GID of the user in the container to confirm that the user is an unprivileged user.

```
12.[student@workstation ~]$ oc exec -it webphp -- id
```

```
uid=1000680000(1000680000) gid=0(root) groups=0(root),1000680000
```

Your UID and GID values might differ from the previous output.

13. Confirm that the permissions for the `/run/httpd` directory are correct.

```
14. [student@workstation ~]$ oc exec -it webphp -- ls -la /run/
15. total 0
16. drwxr-xr-x. 1 root root 70 Dec 20 19:01 .
17. dr-xr-xr-x. 1 root root 39 Dec 20 19:01 ..
18. -rw-r--r--. 1 root root  0 Dec 20 18:45 .containerenv
19. drwxrwx---. 1 root root 41 Dec 20 19:01 httpd
20. drwxr-xr-x. 2 root root  6 Oct 26 11:10 lock
21. drwxrwxr-x. 1 root root 41 Dec 20 19:01 php-fpm
```

```
drwxr-xr-x. 4 root root 80 Dec 20 19:01 secrets
```

5. Connect port 8080 on the workstation machine to port 8080 on the webphp pod. In a new terminal window, retrieve the content of the pod's `127.0.0.1:8080/index.php` web page to confirm that the pod is operational.

NOTE

The terminal window that you connect to the webphp pod must remain open for the remainder of the lab. This connection is necessary for the final lab step and for the `lab grade` command.

1. Connect to port 8080 on the webphp pod.

```
2. [student@workstation ~]$ oc port-forward pod/webphp 8080:8080
3. Forwarding from 127.0.0.1:8080 -> 8080
```

```
Forwarding from [::1]:8080 -> 8080
```

4. Open a second terminal window and then retrieve the `127.0.0.1:8080/index.php` web page on the webphp pod.

```
5. [student@workstation ~]$ curl 127.0.0.1:8080/index.php
```

```
6. <html>
7. <body>
8. Hello, World!
9. </body>
```

```
</html>
```

6. An issue occurs with the PHP application that is running on the `webphp` pod. To debug the issue, the application developer requires diagnostic and configuration information for the PHP instance that is running on the `webphp` pod.

The `~/D0180/labs/pods-review` directory contains a `phpinfo.php` file to generate debugging information for a PHP instance. Copy the `phpinfo.php` file to the `/var/www/html/` directory on the `webphp` pod.

Then, confirm that the PHP debugging information is displayed when accessing the `127.0.0.1:8080/phpinfo.php` from a web browser.

NOTE

After running the `lab grade` command in the second terminal, return to the terminal that is executing the `oc port-forward` command, and press **Ctrl+C** to end the connection.

```
[student@workstation ~]$ oc port-forward pod/webphp 8080:8080
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
^C[student@workstation ~]$
```

1. In the second terminal, copy the `~/D0180/labs/pods-review/phpinfo.php` file to the `webphp` pod as the `/var/www/html/phpinfo.php` file.

```
2. [student@workstation ~]$ oc cp ~/D0180/labs/pods-review/phpinfo.php \
```

```
webphp:/var/www/html/phpinfo.php
```

3. Open a web browser and access the `127.0.0.1:8080/phpinfo.php` web page. Confirm that PHP debugging information is displayed.

PHP Version 7.4.30	
System	Linux webphp 5.14.0-427.61.1.el9_4.x86_64 #1 SMP PREEMPT_DYNAMIC Fri Mar 14 15:21:35 2025 x86_64
Build Date	Jun 7 2022 08:38:19
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype.ini, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/php.d/20-ftp.ini, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-json.ini, /etc/php.d/20-mbstring.ini, /etc/php.d/20-pdo.ini, /etc/php.d/20-phar.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sockets.ini, /etc/php.d/20-sqlite3.ini, /etc/php.d/20-tokenizer.ini, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlwriter.ini, /etc/php.d/20-xsl.ini, /etc/php.d/30-pdo_sqlite.ini, /etc/php.d/30-xmlreader.ini
PHP API	20190902
PHP Extension	20190902
Zend Extension	320190902
Zend Extension Build	API320190902.NTS
PHP Extension Build	API20190902.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled

PHP debugging information

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade pods-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish pods-review
```

Summary

- A container is an encapsulated process that includes the required runtime dependencies for an application to run.
- OpenShift uses Kubernetes to manage pods. Pods consist of one or more containers that share resources, such as selected namespaces and networking, and each pod represents a single application.
- Container images can create container instances, which are executable versions of the image, and include references to networking, disks, and other runtime needs.
- Container image registries, such as Quay.io and the Red Hat Container Catalog, are the preferred way to distribute container images to many users and hosts.
- You can use the `oc image` command and Skopeo to inspect and manage container images.
- Containers are immutable and ephemeral. Thus, avoid updating a running container except for troubleshooting problematic containers.

Chapter 4. Deploy Managed and Networked Applications on Kubernetes

Deploy Applications from an Image and from a Template

Guided Exercise: Deploy Applications from an Image and from a Template

Manage Long-lived and Short-lived Applications by Using the Kubernetes Workload API

Guided Exercise: Manage Long-lived and Short-lived Applications by Using the Kubernetes Workload API

Kubernetes Pod and Service Networks

Guided Exercise: Kubernetes Pod and Service Networks

Scale and Expose Applications to External Access

Guided Exercise: Scale and Expose Applications to External Access

Lab: Deploy Managed and Networked Applications on Kubernetes

Summary

Abstract

Goal	Deploy applications and expose them to network access from inside and outside a Kubernetes cluster.
Sections	• Deploy Applications from an Image and from a Template (and Guided Exercise) • Manage Long-lived and Short-lived Applications by Using the Kubernetes Workload API (and Guided Exercise) • Kubernetes Pod and Service Networks (and Guided Exercise) • Scale and Expose Applications to External Access (and Guided Exercise)
Lab	• Deploy Managed and Networked Applications on Kubernetes

Deploy Applications from an Image and from a Template

Objectives

- Identify the main resources and settings that Kubernetes uses to manage long-lived applications and demonstrate how OpenShift simplifies common application deployment workflows.

Deploying Applications

Microservices and DevOps are growing trends in enterprise software. Containers and Kubernetes gained popularity alongside those trends, and became categories of their own. Container-based infrastructures support most types of traditional and modern applications.

The *application* term can refer to a software system or to a service within it. Given this ambiguity, it is clearer to refer directly to resources, services, and other components.

Resources and Resource Definitions

Kubernetes manages applications, or services, as a loose collection of resources. Resources are configuration pieces for components in the cluster. When you create

a resource, the corresponding component is not created immediately. Instead, the cluster is responsible for eventually creating the component.

A *resource type* represents a specific component type, such as a pod. Kubernetes ships with many default resource types, some of which overlap in function. Red Hat OpenShift Container Platform (RHOCP) includes the default Kubernetes resource types, and provides other resource types of its own. To add resource types, you can create or import CRDs.

Managing Resources

You can add, view, and edit resources in various formats, including YAML and JSON format. Traditionally, YAML is the most common format.

You can delete resources in batch by using label selectors or by deleting the entire project or namespace. For example, the following command deletes only deployments with the `app=my-app` label:

```
[user@host ~]$ oc delete deployment -l app=my-app  
deployment.apps "my-app" deleted
```

Similar to creation, deleting a resource is not immediate, but is instead a request for eventual deletion.

NOTE

Commands that are executed without specifying a namespace are executed in the user's current namespace.

Common Resource Types and Their Uses

The following resources are standard OpenShift resources:

Templates

Similar to projects, templates are an RHOCP addition to Kubernetes. A template is a YAML manifest that contains parameterized definitions of one or more resources. RHOCP provides predefined templates in the `openshift` namespace.

You can process a template into a list of resources by using the `oc process` command, which replaces values and generates resource definitions. The resulting resource definitions create or update resources in the cluster by supplying them to the `oc apply` command.

For example, the following command processes the `mysql-template.yaml` template file and generates four resource definitions:

```
[user@host ~]$ oc process -f mysql-template.yaml -o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: Secret
  ...output omitted...
- apiVersion: v1
  kind: Service
  ...output omitted...
- apiVersion: v1
  kind: PersistentVolumeClaim
  ...output omitted...
- apiVersion: apps/v1
  kind: Deployment
  ...output omitted...
```

The `--parameters` option instead displays the parameters of a template. For example, the following command lists the parameters of the `mysql-template.yaml` file.

```
[user@host ~]$ oc process -f mysql-template.yaml --parameters
NAME                DESCRIPTION ...output omitted...
...output omitted...
MYSQL_USER           Username for MySQL user   ...output omitted...
MYSQL_PASSWORD       Password for the MySQL connection ...output omitted...
MYSQL_ROOT_PASSWORD  Password for the MySQL root user. ...output omitted...
MYSQL_DATABASE       Name of the MySQL database accessed. ...output omitted...
```

VOLUME_CAPACITY

Volume space available for data,

...output omitted...

You can use templates with the `new-app` command from RHOC. In the following example, the `new-app` command uses the `mysql-persistent` template to create a MySQL application and its supporting resources:

```
[user@host ~]$ oc new-app --template mysql-persistent
--> Deploying template "db-app/mysql-persistent" to project db-app
...output omitted...

The following service(s) have been created in your project: mysql.

      Username: userQSL
      Password: pyf0yElPvFWYQQou
      Database Name: sampled
      Connection URL: mysql://mysql:3306/
...output omitted...

* With parameters:
  * Memory Limit=512Mi
  * Namespace=openshift
  * Database Service Name=mysql
  * MySQL Connection Username=userQSL # generated
  * MySQL Connection Password=pyf0yElPvFWYQQou # generated
  * MySQL root user Password=HHbdurqW05gAog2m # generated
  * MySQL Database Name=sampled
  * Volume Capacity=1Gi
  * Version of MySQL Image=8.0-el8

--> Creating resources ...
      secret "mysql" created
      service "mysql" created
      persistentvolumeclaim "mysql" created
      deployment.apps "mysql" created
--> Success
```

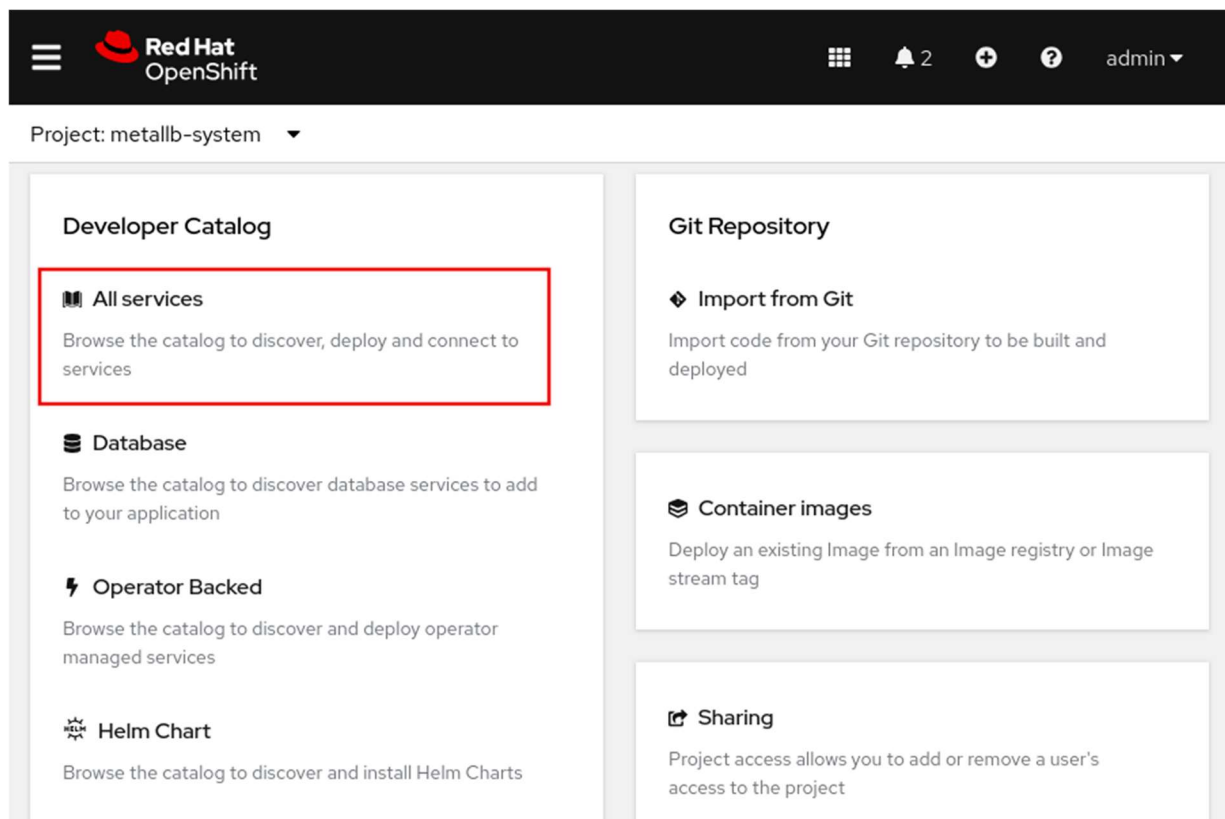
...output omitted...

Several resources are created to meet the requirements of the deployment, including a secret, a service, and a persistent volume claim.

NOTE

You can specify environment variables to be configured in creating your application.

You can also use templates, helm charts, builder images, or devfiles to create applications from the web console by using the **Developer Catalog** page. From the **Developer** perspective, go to the **+Add** menu and click **All Services** in the **Developer Catalog** section to open the catalog.



Then, enter the application name in the filter box to search for an application's template. You can instantiate a template and change the default values, such as the Git repository, the memory limits, and the application version.

Memory Limit *

512Mi

Maximum amount of memory the container can use.

Namespace

openshift

The OpenShift Namespace where the ImageStream resides.

Database Service Name *

postgresql

The name of the OpenShift Service exposed for the database.

PostgreSQL Connection Username

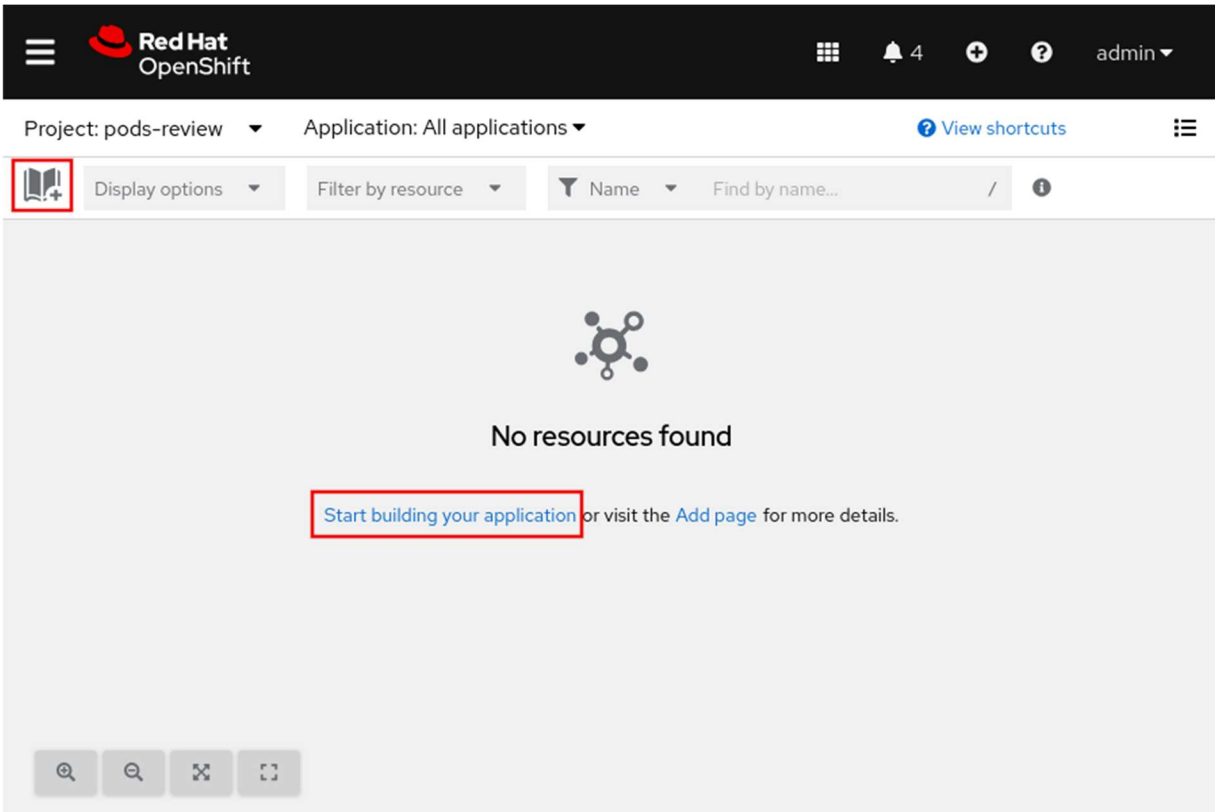
(generated if empty)

Username for PostgreSQL user that will be used for accessing the database.

PostgreSQL Connection Password

(generated if empty)

You can also create an application from the **Topology** menu, and by clicking **Start building application** or clicking the book icon.



Pod

A pod is the smallest compute unit that you can define, deploy, and manage in RHOCP. A pod runs one or more containers that represent a single application. Containers in the pod share resources, such as networking and storage.

The following example shows the definition of a pod:

```
apiVersion: v1

kind: Pod

metadata:
  annotations: {}
  labels:
    deployment: docker-registry-1
  name: registry
  namespace: pod-registries
```

```

spec:
  containers:
  - env:
    - name: OPENSIFT_CA_DATA
      value:
image: openshift/origin-docker-registry:v0.6.2
imagePullPolicy: IfNotPresent
name: registry
ports:
- containerPort: 5000
  protocol: TCP
resources: {}
securityContext: {}
volumeMounts:
- mountPath: /registry
  name: registry-storage
dnsPolicy: ClusterFirst
imagePullSecrets:
- name: default-dockercfg-at06w
restartPolicy: Always
serviceAccount: default
volumes:
- emptyDir: {}
  name: registry-storage

status:
  conditions: {}

```

The resource kind is set to Pod.

Information that describes your application, such as the name, project, attached labels, and annotations.

Section where the application requirements are specified, such as the container name, the container image, environment variables, volume mounts, network configuration, and volumes.

Indicates the last condition of the pod, such as the last probe time, the last transition time, the status setting as true or false, and more.

Deployments

A deployment describes the intended state of a component of the application as a pod template. A deployment manages one or more replica sets.

The following example shows the definition of a deployment:

```
apiVersion: apps/v1

kind: Deployment
metadata:

  name: hello-openshift
spec:

  replicas: 1
  selector:
    matchLabels:
      app: hello-openshift

  template:
    metadata:
      labels:
        app: hello-openshift
    spec:
      containers:

        - name: hello-openshift

          image: openshift/hello-openshift:latest

          ports:
            - containerPort: 80
```


The resource kind is set to Deployment.

Name of the deployment resource.

Number of running instances.

Section to define the metadata, labels, and the container information of the deployment resource.

Name of the container.

Resource of the image for creating the deployment resource.

Port configuration, such as the port number, the name of the port, and the protocol.

Projects

RHOC P adds projects to enhance the function of Kubernetes namespaces. A project is a Kubernetes namespace with additional annotations, and is the primary method for managing access to resources for regular users. Projects can be created from templates and must use RBAC for organization and permission management. Administrators must grant cluster users access to a project. If a cluster user is allowed to create projects, then the user automatically has access to their created projects.

Projects provide logical and organizational isolation to separate your application component resources. Resources in one project can access resources in other projects, but not by default.

The following example shows the definition of a project:

```
apiVersion: project.openshift.io/v1

kind: Project

metadata:

  name: test

spec:

  finalizers:
  - kubernetes
```

The resource kind is set to Project.

Name of the project.

A finalizer is a special metadata key that tells Kubernetes to wait until a specific condition is met before it fully deletes a resource.

Services

You can configure internal pod-to-pod network communication in RHOCp by using a service. Applications send requests to the service name and port. RHOCp provides a virtual network, which reroutes such requests to the pods that the service targets by using labels.

The following example shows the definition of a service:

```
apiVersion: v1

kind: Service

metadata:

  name: docker-registry

  namespace: test

spec:

  selector:

    app: MyApp

  ports:

    - protocol: TCP

      port: 80

      targetPort: 9376
```

The resource kind is set to Service.

Name of the service.

Project name where the service resource exists.

The label selector identifies all pods with the attached `app=MyApp` label and adds the pods to the service endpoints.

Internet protocol is set to TCP.

Port that the service listens on.

Port on the backing pods, which the service forwards connections to.

Persistent Volume Claims

RHOC P uses the Kubernetes persistent volume (PV) framework to enable cluster administrators to provision persistent storage for a cluster. Developers can use persistent volume claims (PVCs) to request PV resources without having specific knowledge of the underlying storage infrastructure. After a PV is bound to a PVC, that PV cannot then be bound to additional PVCs. Because PVCs are namespaced objects and PVs are globally scoped objects, this binding effectively scopes a bound PV to a single namespace until the binding PVC is deleted.

The following example shows the definition of a persistent volume claim:

```
apiVersion: v1

kind: PersistentVolumeClaim

metadata:

  name: mysql-pvc

spec:

  accessModes:

    - ReadWriteOnce

  resources:

    requests:

      storage: 1Gi

  storageClassName: nfs-storage

status: {}
```

The resource kind is set to `PersistentVolumeClaim`.

Name of the service.

The access mode, to define the read/write and mount permissions.

The storage of the PVC.

Name of the StorageClass that the claim requires.

Secrets

Secrets provide a mechanism to hold sensitive information, such as passwords, private source repository credentials, sensitive configuration files, and credentials to an external resource, such as an SSH key or an OAuth token. You can mount secrets into containers by using a volume plug-in. Kubernetes can also use secrets to perform actions, such as on pods, declaring environment variables. Secrets can store any type of data. Kubernetes and OpenShift support different types of secrets, such as service account tokens, SSH keys, and TLS certificates.

The following example shows the definition of a secret:

```
apiVersion: v1

kind: Secret
metadata:

  name: example-secret

  namespace: my-app

type: Opaque

data:
  username: bXl1c2VyCg==
  password: bXlQQDU1Cg==

stringData:
  hostname: myapp.mydomain.com
  secret.properties: |
    property1=valueA
```

```
property2=valueB
```

The resource kind is set to Secret.

Name of the service.

Project name where the service resource exists.

Specifies the type of secret.

Specifies the encoded string and data.

Specifies the decoded string and data.

Managing Resources from the Command Line

Many Kubernetes and RHOCP commands can create and modify cluster resources. Some commands are part of core Kubernetes, whereas others are exclusive additions to RHOCP.

The resource management commands generally fall into one of two categories:

- a. An *imperative* command instructs what the cluster does.
- b. A *declarative* command defines the state that the cluster attempts to match.

Imperative Resource Management

The create command is an imperative way to create resources, and is included in both of the `oc` and `kubectl` commands. For example, the following command creates a deployment named `my-app` that creates pods that are based on the specified image.

```
[user@host ~]$ oc create deployment my-app --image example.com/my-image:dev
deployment.apps/my-app created
```

Use the `set` command to define attributes on a resource, such as environment variables. For example, the following command adds the `TEAM=red` environment variable to the preceding deployment.

```
[user@host ~]$ oc set env deployment/my-app TEAM=red
```

```
deployment.apps/my-app updated
```

Another imperative approach to creating a resource is the `run` command. In the following example, the `run` command creates the `example-pod` pod.

```
[user@host ~]$ oc run example-pod \  
  
--image=registry.access.redhat.com/ubi8/httpd-24 \  
  
--env GREETING='Hello from the awesome container' \  
  
--port 8080  
pod/example-pod created
```

The `pod.metadata.name` definition

The image for the single container in this pod

The environment variable for the single container in this pod

The port metadata definition

The imperative commands are a faster way of creating pods, because such commands do not require a pod object definition. Versioning and incremental changes require declarative manifests.

Generally, developers test a deployment by using imperative commands, and then use the imperative commands to generate the pod object definition. Use the `--dry-run=client` option to avoid creating the object in RHOCP. Additionally, use the `-o yaml` or `-o json` option to configure the definition format.

The following command is an example of generating the YAML definition for the `example-pod` pod:

```
[user@host ~]$ oc run example-pod \  
  
--image=registry.access.redhat.com/ubi8/httpd-24 \  
  
--env GREETING='Hello from the awesome container' \  
  
--port 8080 \  

```

```
--dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: example-pod
  name: example-pod
spec:
  containers:
...output omitted...
```

Managing resources in this way is imperative, because you are instructing the cluster what to do rather than declaring the intended outcomes.

Declarative Resource Management

Using the `create` command does not guarantee that you are creating resources imperatively. For example, providing a manifest declaratively creates the resources that are defined in the YAML file, such as in the following command.

```
[user@host ~]$ oc create -f my-app-deployment.yaml
deployment.apps/my-app created
```

RHOC P adds the `new-app` command, which provides another declarative way to create resources. This command uses heuristics to automatically determine which types of resources to create based on the specified parameters. For example, the following command deploys the `my-app` application by creating several resources, including a deployment resource, from a YAML manifest file:

```
[user@host ~]$ oc new-app --file=./example/my-app.yaml
...output omitted...
--> Creating resources ...
    imagestream.image.openshift.io "my-app" created
    deployment.apps "my-app" created
```

```
services "my-app" created
...output omitted...
```

In both of the preceding `create` and `new-app` examples, you are declaring the intended state of the resources, and so they are declarative.

You can also use the `new-app` command with templates for resource management. A template describes the intended state of resources that must be created for an application to run, such as deployments and services. Supplying a template to the `new-app` command is a form of declarative resource management.

The `new-app` command also includes options, such as the `--param` option, that customize an application deployment declaratively. For example, when the `new-app` is used with a template, you can include the `--param` option to override a parameter value in the template.

```
[user@host ~]$ oc new-app --template mysql-persistent \
--param MYSQL_USER=operator --param MYSQL_PASSWORD=myP@55 \
--param MYSQL_DATABASE=mydata \
--param DATABASE_SERVICE_NAME=db
--> Deploying template "db-app/mysql-persistent" to project db-app
...output omitted...

The following service(s) have been created in your project: db.

    Username: operator
    Password: myP@55
    Database Name: mydata
    Connection URL: mysql://db:3306/
...output omitted...

* With parameters:
  * Memory Limit=512Mi
  * Namespace=openshift
  * Database Service Name=db
  * MySQL Connection Username=operator
  * MySQL Connection Password=myP@55
```



```
* MySQL root user Password=tlH8BThuVgnIrCon # generated
* MySQL Database Name=mydata
* Volume Capacity=1Gi
* Version of MySQL Image=8.0-e18

--> Creating resources ...
    secret "db" created
    service "db" created
    persistentvolumeclaim "db" created
--> Success
...output omitted...
```

Similar to the `create` command, you can use the `new-app` command imperatively. When using a container image with the `new-app` command, you are instructing the cluster what to do, rather than declaring the intended outcomes. For example, the following command deploys the `example.com/my-app:dev` image by creating several resources, including a deployment resource:

```
[user@host ~]$ oc new-app --image example.com/my-app:dev
...output omitted...
--> Creating resources ...
    imagestream.image.openshift.io "my-app" created
    deployment.apps "my-app" created
...output omitted...
```

You can also supply a Git repository to the `new-app` command. The following command creates an application named `httpd24` by using a Git repository:

```
[user@host ~]$ oc new-app https://github.com/apache/httpd.git#2.4.56
...output omitted...
--> Creating resources ...
    imagestream.image.openshift.io "httpd24" created
    deployment.apps "httpd24" created
...output omitted...
```

Retrieving Resource Information

You can supply the `all` argument to commands, to specify a list of common resource types. However, the `all` option does not include every resource type. Instead, `all` is a shorthand form for a predefined subset of types. When you use this command argument, ensure that `all` includes any intended types to address.

You can view detailed information about a resource, such as the defined parameters, by using the `describe` command. For example, RHOCP provides templates in the `openshift` project to use with the `oc new-app` command. The following example command displays detailed information about the `mysql-ephemeral` template:

```
[user@host ~]$ oc describe template mysql-ephemeral -n openshift
Name:          mysql-ephemeral
Namespace:     openshift
...output omitted...
Parameters:
  Name:          MEMORY_LIMIT
  Display Name:   Memory Limit
  Description:    Maximum amount of memory the container can use.
  Required:      true
  Value:         512Mi

  Name:          NAMESPACE
  Display Name:   Namespace
  Description:    The OpenShift Namespace where the ImageStream resides.
  Required:      false
  Value:         openshift
...output omitted...
Objects:
  Secret          ${DATABASE_SERVICE_
NAME}

  Service         ${DATABASE_SERVICE_
NAME}
```

The `oc describe` command cannot generate structured output, such as the YAML or JSON formats. A structured format is required for the `oc describe` command to parse or filter the output with tools such as JSONPath or Go templates." Instead, use the `oc get` command to generate and to parse the structured output of a resource.

Guided Exercise: Deploy Applications from an Image and from a Template

Deploy a database from a container image and from a template by using the OpenShift command-line interface and compare the resources and attributes that each method generates.

Outcomes

- Deploy a database from a container image.
- Deploy a database from a template.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start deploy-newapp
```

Instructions

1. As the developer user, create a project and verify that it is not empty after creation.
1. Log in to the OpenShift cluster as the developer user with developer as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful
```

...output omitted...

5. Create a project named `deploy-newapp`.

6. [student@workstation ~]\$ **oc new-project deploy-newapp**

7. Now using project "deploy-newapp" on server "https://api.ocp4.example.com:6443".

...output omitted...

The new project is automatically selected.

8. Verify that no pod resources exist in the deploy-newapp project.

9. [student@workstation ~]\$ **oc get pods**

No resources found in deploy-newapp namespace.

10. Verify that the new project contains other types of resources, such as service accounts and secrets.

11. [student@workstation ~]\$ **oc get serviceaccounts,secrets**

12. NAME	SECRETS	AGE	
13. serviceaccount/builder	1	20s	
14. serviceaccount/default	1	20s	
15. serviceaccount/deployer	1	20s	
16.			
17. NAME	TYPE	DATA	AGE
18. secret/builder-dockercfg-sczxg	kubernetes.io/dockercfg	1	15s
19. secret/default-dockercfg-gsnqj	kubernetes.io/dockercfg	1	15s

secret/deployer-dockercfg-6f8nm	kubernetes.io/dockercfg	1	15s
---------------------------------	-------------------------	---	-----

2. Create two PostgreSQL instances by using the `oc new-app` command with different options.

1. View the `mysql-persistent` template definition to inspect the resources that it creates. Specify the project that houses the template by using the `-n openshift` option.

2. [student@workstation ~]\$ **oc describe template mysql-persistent -n openshift**

3. Name: mysql-persistent

4. Namespace: openshift

```

5. ...output omitted...
6. Objects:
7.      Secret                                ${DATABASE_SERVICE_NAME}
8.      Service                               ${DATABASE_SERVICE_NAME}
9.      PersistentVolumeClaim                 ${DATABASE_SERVICE_NAME}

```

```

Deployment.apps                                ${DATABASE_SERVICE_NAME}

```

The `objects` attribute specifies several resource definitions that are applied on using the template. These resources include one of each of the following types: secret, service (svc), persistent volume claim (pvc), and deployment.

10. Create an instance by using the `mysql-persistent` template. Specify the application name to `mysql`, and attach a custom `team=red` label to the created resources.

```

11. [student@workstation ~]$ oc new-app --name mysql \
12.   --template mysql-persistent \
13.   -l team=red \
14.   -p MYSQL_USER=developer \
15.   -p MYSQL_PASSWORD=developer
16. ...output omitted...
17. --> Creating resources with label team=red ...
18.      secret "mysql" created
19.      service "mysql" created
20.      persistentvolumeclaim "mysql" created
21.      deployment.apps "mysql" created
22. --> Success

```

```

...output omitted...

```

The template creates resources of the types from the preceding step.

23. Run the `watch` command with the `oc get pods` command, and wait until the pod is running. The name of the pod is different in your cluster.

```
24. [student@workstation ~]$ watch oc get pods
```

```
25. NAME                                READY   STATUS    RESTARTS   AGE
```

```
mysql-8d7d996f-qn798    1/1     Running   0           60s
```

Press **Ctrl+C** to end the watch command after the pod displays Running in the STATUS column.

26. Create an instance by using a container image. Specify a name option and attach a custom team=blue label to the created resources.

```
27. [student@workstation ~]$ oc new-app --name db-image -l team=blue \
```

```
28.   --image registry.ocp4.example.com:8443/rhel9/mysql-80:1 \
```

```
29.   -e MYSQL_USER=developer \
```

```
30.   -e MYSQL_PASSWORD=developer \
```

```
31.   -e MYSQL_ROOT_PASSWORD=redhat
```

```
32. ...output omitted...
```

```
33. --> Creating resources with label team=blue ...
```

```
34.   imagestream.image.openshift.io "db-image" created
```

```
35.   deployment.apps "db-image" created
```

```
36.   service "db-image" created
```

```
37. --> Success
```

```
...output omitted...
```

The command creates predefined resources that are needed to deploy an image. These resource types are image stream (is), deployment, and service (svc). Image streams and services are discussed in more detail elsewhere in the course.

38. Run the watch command with the oc get pods command, and wait until the pods are running. The command lists all pods and the value of the team label for each pod. The names of the pods are different in your cluster.

```
39. [student@workstation ~]$ watch oc get pods -L team
```

```
40. NAME                                READY   STATUS    RESTARTS   AGE   TEAM
```

41. db-image-8d4b97594-6jb85	1/1	Running	0	55s	blue
mysql-8d7d996f-qn798	1/1	Running	0	1m30s	

Press **Ctrl+C** to end the watch command after the pods display Running in the STATUS column.

Without a readinessProbe, the db-image pod shows as ready before the MySQL service is ready for requests. Readiness probes are discussed elsewhere in the course.

Notice that only the db-image pod has a label that contains the word team. Pods that the mysql-persistent template creates do not have the team=red label, because the template does not define this label in its pod specification template.

3. Compare the resources that each image and template method creates.
1. View the template-based pod and observe that it contains a readiness probe.

```

2. [student@workstation ~]$ oc get pods -l deployment=mysql \
3.   -o jsonpath='{.items[0].spec.containers[0].readinessProbe}' | jq
4. {
5.   "exec": {
6.     "command": [
7.       "/bin/sh",
8.       "-i",
9.       "-c",
10.      "MYSQL_PWD=\"${MYSQL_PASSWORD}\" mysqladmin -u ${MYSQL_USER} ping"
11.    ]
12.  },
13.  "failureThreshold": 3,
14.  "initialDelaySeconds": 5,
15.  "periodSeconds": 10,
16.  "successThreshold": 1,
17.  "timeoutSeconds": 1

```

```
}
```

NOTE

The results of the preceding `oc` command are passed to the `jq` command, which formats the JSON output.

18. Observe that the image-based pod does not contain a readiness probe.

```
19. [student@workstation ~]$ oc get pods -l deployment=db-image \
```

```
-o jsonpath='{.items[0].spec.containers[0].readinessProbe}' | jq
```

20. Observe that the template-based pod has a memory resource limit, which restricts allocated memory to the resulting pods. Resource limits are discussed in more detail elsewhere in the course.

```
21. [student@workstation ~]$ oc get pods -l deployment=mysql \
```

```
22.   -o jsonpath='{.items[0].spec.containers[0].resources.limits}' | jq
```

```
23. {
```

```
24.   "memory": "512Mi"
```

```
}
```

25. Observe that the image-based pod has no resource limits.

```
26. [student@workstation ~]$ oc get pods -l deployment=db-image \
```

```
27.   -o jsonpath='{.items[0].spec.containers[0].resources}' | jq
```

```
{}
```

28. Retrieve secrets in the project. Notice that the template produced a secret, whereas the pod that was created with only an image did not produce a secret.

```
29. [student@workstation ~]$ oc get secrets
```

```
30. NAME                                TYPE      DATA   AGE
```

```
31. ...output omitted...
```


mysql	Opaque	4	3m
-------	--------	---	----

4. Explore filtering resources via labels.

1. Observe that when a label is not supplied, all services are shown.

```
2. [student@workstation ~]$ oc get services
```

3. NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
4. db-image	ClusterIP	172.30.38.113	<none>	3306/TCP	1m30s

mysql	ClusterIP	172.30.95.52	<none>	3306/TCP	2m30s
-------	-----------	--------------	--------	----------	-------

5. Observe that when a label is supplied, only the services with the label are shown.

```
6. [student@workstation ~]$ oc get services -l team=red
```

7. NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
---------	------	------------	-------------	---------	-----

mysql	ClusterIP	172.30.95.52	<none>	3306/TCP	2m43s
-------	-----------	--------------	--------	----------	-------

8. Observe that not all resources include a label, such as the pods that are created with the template.

```
9. [student@workstation ~]$ oc get pods -l team=red
```

No resources found in deploy-newapp namespace.

5. Use labels to delete only the resources that are associated with the template-based deployment.

1. Delete only the resources that use the `team=red` label by using the `oc delete all -l` command.

```
2. [student@workstation ~]$ oc delete all -l team=red
```

```
3. service "mysql" deleted
```

```
4. deployment.apps "mysql" deleted
```

...output omitted...

5. Because the `oc delete all -l` command does not delete the secret and PVC resources, delete those resources manually by running the following command:

```
6. [student@workstation ~]$ oc delete secret,pvc -l team=red
7. secret "mysql" deleted
```

```
persistentvolumeclaim "mysql" deleted
```

8. Observe that the resources that the template created are deleted.

```
9. [student@workstation ~]$ oc get secret,svc,pvc,deployment -l team=red
10. ...output omitted...
```

```
No resources found in deploy-newapp namespace.
```

11. Observe that the image-based resources remain unchanged.

```
12. [student@workstation ~]$ oc get is,deployment,svc
13. NAME                                IMAGE REPOSITORY                                ...
14. imagestream.image.openshift.io/db-image  image-registry.openshift... ...
15. NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
16. deployment.apps/db-image             1/1      1              1             46m
17.
18. NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
```

```
service/db-image    ClusterIP    172.30.71.0    <none>         3306/TCP    46m
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish deploy-newapp
```

Guided Exercise: Manage Long-lived and Short-lived Applications by Using the Kubernetes Workload API

Deploy a batch application managed by a job resource and a database server that a deployment resource manages by using the Kubernetes command-line interface.

Outcomes

In this exercise, you deploy a database server and a batch application, both managed by workload resources that the deployment resources manage.

- Create deployments.
- Update environment variables on a pod template.
- Create and run job resources.
- Retrieve the logs and termination status of a job.
- View the pod template of a job resource.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures resource availability for the exercise.

```
[student@workstation ~]$ lab start deploy-workloads
```

Instructions

1. As the developer user, create a MySQL deployment in a new project.
1. Log in as the developer user with the developer password.

2. `[student@workstation ~]$ oc login -u developer -p developer \`
3. `https://api.ocp4.example.com:6443`

...output omitted...

4. Create a project named `deploy-workloads`.

5. `[student@workstation ~]$ oc new-project deploy-workloads`
6. Now using project "deploy-workloads" on server "https://api.ocp4.example.com:6443".

...output omitted...

7. Create a deployment that runs an ephemeral MySQL server.

```
8. [student@workstation ~]$ oc create deployment my-db \
9. --image registry.ocp4.example.com:8443/rhel9/mysql-80:1
```

```
deployment.apps/my-db created
```

10. Retrieve the status of the deployment.

```
11. [student@workstation ~]$ oc get deployments
12. NAME          READY    UP-TO-DATE    AVAILABLE    AGE
```

```
my-db    0/1      1             0            67s
```

The deployment never shows a ready instance.

13. Retrieve the status of the created pod. Your pod name might differ from the output.

```
14. [student@workstation ~]$ oc get pods
15. NAME                                READY   STATUS             RESTARTS   AGE
```

```
my-db-8567b478dd-d28f7    0/1     CrashLoopBackOff   4 (60s ago)  2m35s
```

The pod fails to start and repeatedly crashes. If the pod status shows Pending, allow a few minutes for it to crash before proceeding.

16. Review the logs for the pod to determine why it fails to start.

```
17. [student@workstation ~]$ oc logs deploy/my-db
18. ...output omitted...
19. You must either specify the following environment variables:
20.  MYSQL_USER (regex: '^$')
21.  MYSQL_PASSWORD (regex: '^[a-zA-Z0-9_~!@#$$%^&*()-=<>,.?;:|]$')
22.  MYSQL_DATABASE (regex: '^$')
23. Or the following environment variable:
24.  MYSQL_ROOT_PASSWORD (regex: '^[a-zA-Z0-9_~!@#$$%^&*()-=<>,.?;:|]$')
```

```
...output omitted...
```

Note that the container fails to start due to missing environment variables.

2. Fix the database deployment and verify that the server runs.
1. Set the `MYSQL_USER`, `MYSQL_PASSWORD`, and `MYSQL_DATABASE` environment variables.

```
2. [student@workstation ~]$ oc set env deployment/my-db \  
3.   MYSQL_USER=developer \  
4.   MYSQL_PASSWORD=developer \  
5.   MYSQL_DATABASE=sampled
```

```
deployment.apps/my-db updated
```

6. Retrieve the list of deployments and observe that the `my-db` deployment has a running pod.

```
7. [student@workstation ~]$ oc get deployments  
8. NAME      READY    UP-TO-DATE    AVAILABLE    AGE
```

```
my-db      1/1      1             1            4m50s
```

9. Retrieve the internal IP address of the MySQL pod within the list of all pods.

```
10. [student@workstation ~]$ oc get pods -o wide
```

```
11. NAME                                READY    STATUS    RESTARTS    AGE    IP            ...  
  
my-db-748c97d478-g8xc9    1/1      Running    0           64s    10.8.0.91 ...
```

The `-o wide` option provides additional output, such as IP addresses. Your IP address value might differ from the previous output.

12. Store the IP address of the MySQL pod as a variable for use in the next step.

```
13. [student@workstation ~]$ DB_IP=$(oc get pods my-db-7459cb94-zdb5p \  
-o=jsonpath='{.status.podIP}')
```

14. Verify that the database server runs by executing a query. This command uses the variable that contains the IP address from preceding step.

```
15. [student@workstation ~]$ oc run -it db-test --restart=Never \  
16.   --image registry.ocp4.example.com:8443/rhel9/mysql-80:1 \  
17.   -- mysql sampled -h $DB_IP -u developer --password=developer \  
18.   -e "select 1;"  
19. ...output omitted...  
20. ---  
21. | 1 |  
22. ---  
23. | 1 |
```

```
---
```

3. Delete the database server pod and observe that the deployment causes the pod to be re-created.

1. Delete the existing MySQL pod by using the label that is associated with the deployment.

```
2. [student@workstation ~]$ oc delete pod -l app=my-db
```

```
pod "my-db-84c8995d5-2sssl" deleted
```

3. Retrieve the information for the MySQL pod and observe that it is newly created. Your pod name might differ in your output.

```
4. [student@workstation ~]$ oc get pod -l app=my-db
```

5. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

my-db-fbccb9447-p99jd	1/1	Running	0	6s
-----------------------	-----	---------	---	----

4. Create and apply a job resource that prints the time and date repeatedly.

1. Create a job resource called date-loop that runs a script. Ignore the warning.

```
2. [student@workstation ~]$ oc create job date-loop \  
3.   --image registry.ocp4.example.com:8443/ubi9/ubi \  
4.   -- /bin/bash -c "for i in {1..30}; do date; done"
```

```
job.batch/date-loop created
```

5. Retrieve the job resource to review the pod specification.

```
6. [student@workstation ~]$ oc get job date-loop -o yaml
7. ...output omitted...
8.   spec:
9.     containers:
10.      - command:
11.        - /bin/bash
12.        - -c
13.        - for i in {1..30}; do date; done
14.      image: registry.ocp4.example.com:8443/ubi9/ubi
15.      imagePullPolicy: Always
16.      name: date-loop
17.      resources: {}
18.      terminationMessagePath: /dev/termination-log
19.      terminationMessagePolicy: File
20.      dnsPolicy: ClusterFirst
21.      restartPolicy: Never
22.      schedulerName: default-scheduler
23.      securityContext: {}
24.      terminationGracePeriodSeconds: 30
```

```
...output omitted...
```

The command object, which specifies the defined script to execute within the pod.

Sets the container image for the pod.

Defines the restart policy for the pod. Kubernetes does not restart the job pod after the pod exits.

25. List the jobs to see that the date-loop job completed successfully.

```
26. [student@workstation ~]$ oc get jobs
```

27. NAME	COMPLETIONS	DURATION	AGE
date-loop	1/1	7s	8s

You might need to wait for the script to finish and run the command again.

28. Retrieve the logs for the associated pod. The log values might differ in your output.

```
29. [student@workstation ~]$ oc logs job/date-loop
```

```
30. Fri Nov 18 14:50:56 UTC 2022
```

```
31. Fri Nov 18 14:50:59 UTC 2022
```

...output omitted...

5. Delete the pod for the date-loop job and observe that the pod is not created again.

1. Delete the associated pod.

```
2. [student@workstation ~]$ oc delete pod -l job-name=date-loop
```

pod "date-loop-wvn2q" deleted

3. View the list of pods and observe that the pod is not re-created for the job.

```
4. [student@workstation ~]$ oc get pod -l job-name=date-loop
```

No resources found in deploy-workloads namespace.

5. Verify that the job status is still listed as successfully completed.

```
6. [student@workstation ~]$ oc get job -l job-name=date-loop
```

7. NAME	COMPLETIONS	DURATION	AGE
---------	-------------	----------	-----

date-loop	1/1	7s	7m36s
-----------	-----	----	-------

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish deploy-workloads
```

Kubernetes Pod and Service Networks

Objectives

- Interconnect application pods inside the same cluster by using Kubernetes services.

The Software-defined Network

Kubernetes implements software-defined networking (SDN) through Open Virtual Network (OVN)-Kubernetes to manage the network infrastructure of the cluster. OVN-Kubernetes creates a virtual network that encompasses all cluster nodes. The virtual network enables communication between any container or pod within the cluster. Cluster node processes that Kubernetes pods manage have access to the OVN-Kubernetes network. However, the OVN-Kubernetes network remains inaccessible from outside the cluster or by regular processes on cluster nodes.

With the OVN-Kubernetes model, you can manage network services through the abstraction of multiple networking layers. With OVN-Kubernetes, you can manage network traffic and network resources programmatically to enable organization teams to determine how to expose their application.

The OVN-Kubernetes implementation creates a model that is compatible with traditional networking practices. The model makes pods that are similar to virtual machines in terms of port allocation, IP address leasing, and reservation.

With the OVN-Kubernetes design, you do not need to modify how application components communicate with each other. The OVN-Kubernetes design assists in containerizing legacy applications. If your application consists of multiple services that communicate over the TCP/UDP stack, then this approach remains effective because containers within a pod use the same network stack.

Among the features of OVN-Kubernetes, open standards enable vendors to propose solutions for centralized management, dynamic routing, and tenant isolation.

Kubernetes Networking

Networking in Kubernetes provides a scalable method for communication between containers and offers the following capabilities:

- Highly coupled container-to-container communications
- Pod-to-pod communications
- Pod-to-service communications
- External-to-service communication: Covered in the section called “ Scale and Expose Applications to External Access ”

Kubernetes automatically assigns an IP address to each pod. However, pod IP addresses remain unstable because pods are ephemeral. Pods continuously create and delete themselves across the nodes in the cluster. For example, when you deploy a new version of your application, Kubernetes deletes the existing pods and then deploys new ones.

All containers within a pod share networking resources. The IP address and MAC address that are assigned to the pod are shared among all containers in the pod. Thus, all containers within a pod can access each other's ports via the loopback address, which is `localhost`. Ports that are bound to `localhost` are available to all containers that are running within the pod but not to containers outside it.

By default, pods can communicate with each other even if they run on different cluster nodes or belong to different Kubernetes namespaces. Every pod is assigned an IP address in a flat shared networking namespace that has full communication with other physical computers and containers across the network. All pods are assigned a unique IP address from a Classless Inter-Domain Routing (CIDR) range of host addresses. The shared address range places all pods in the same subnet.

Because all the pods are on the same subnet, pods on all nodes can communicate with pods on any other node without the aid of Network Address Translation (NAT). Kubernetes also provides a service subnet, which links the stable IP address of a service resource to a set of specified pods. The traffic is forwarded in a transparent way to the pods; an agent, which OVN-Kubernetes manages, handles routing rules to direct traffic to pods that match the service resource selectors. Thus, pods

function similarly to virtual machines or to physical hosts regarding port allocation, networking, naming, service discovery, load balancing, application configuration, and migration.

The following illustration gives further insight into how the infrastructure components work along with the pod and service subnets to enable network access between pods inside an OpenShift instance.

The shared networking namespace of pods enables a straightforward communication model. However, the dynamic nature of pods presents a problem. Pods can be added on the fly to handle increased traffic. Likewise, pods can be dynamically scaled down. If a pod fails, then Kubernetes automatically replaces the pod with a new one. These events change pod IP addresses.

Figure 4.8: Problem with direct access to pods

In the diagram, the *Before* side shows a *Front-end* container that is running in a pod with a 10.8.0.1 IP address. The container also shows a *Back-end* container that is running in a pod with a 10.8.0.2 IP address. In this example, an event occurs that causes the *Back-end* container to fail. A pod can fail for many reasons. In response to the failure, Kubernetes creates a pod for the *Back-end* container that uses a new IP address of 10.8.0.4.

From the *After* side of the diagram, the *Front-end* container now has an invalid reference to the *Back-end* container because of the IP address change. Kubernetes resolves this problem with service resources.

Using Services

Containers inside Kubernetes pods must not connect directly to each other's dynamic IP address. Instead, Kubernetes assigns a stable IP address to a service resource that is linked to a set of specified pods. The service acts as a virtual network load balancer for the pods that are linked to it.

If pods are restarted, replicated, or rescheduled to different nodes, then the service endpoints are updated. The updated endpoints provide scalability and fault tolerance for your application. Unlike the IP addresses of pods, the IP addresses of services do not change. Service IP stability provides scalability and fault tolerance for your application.

Figure 4.9: Services resolve pod failure issues

In the diagram, the *Before* side shows that a *Front-end* container references the stable IP address of the *Back-end* service, instead of to the IP address of the pod that is running the *Back-end* container. When the *Back-end* container fails, Kubernetes creates a pod with the *New back-end* container to replace the failed pod. In response to the change, Kubernetes removes the failed pod from the service endpoints. Kubernetes then adds the IP address of the *New back-end* container pod to the service endpoints.

With the service in place, requests from the *Front-end* container to the *Back-end* container continue to function. The service updates dynamically with the IP address change. A service provides a permanent, static IP address for a group of pods that belong to the same deployment or replica set for an application. Until you delete the service, the assigned IP address does not change, and the cluster does not reuse it.

Most real-world applications do not run as a single pod. Applications need to scale horizontally. Multiple pods run the same containers to meet growing user demand. A *Deployment* resource manages multiple pods that execute the same container. A service provides a single IP address for the entire set. The service performs load balancing for client requests among member pods.

With services, containers in one pod can establish network connections to containers in another pod. The pods that the service tracks do not need to exist on the same compute node or in the same namespace or project. Because a service provides a stable IP address for other pods to use, a pod does not need to discover the new IP address of another pod after a restart. The service provides a stable IP address to use, no matter which compute node runs the pod after each restart

The *Service* object provides a stable IP address for the *Client* container on *Node X*. The stable IP address enables the *Client* container to send a request to any of the *API* containers.

Kubernetes uses labels on pods to select those pods that are associated with a service. To include a pod in a service, the pod labels must include each of the selector fields of the service.

Figure 4.11: Service selector match to pod labels

In this example, the selector has a key-value pair of `app: myapp`. Thus, pods with a matching label of `app: myapp` are included in the set that is associated with the

service. The *selector* attribute of a service identifies the set of pods that form the endpoints for the service. Each pod in the set is an endpoint for the service.

To create a service for a deployment, use the `oc expose` command:

```
[user@host ~]$ oc expose deployment/<deployment-name> [--selector <selector>]
[--port <port>][--target-port <target port>][--protocol <protocol>][--name <name>]
```

The `oc expose` command uses the `--selector` option to specify label selectors. When you omit the `--selector` option, the command applies a selector that matches the replication controller or the replica set.

The `--port` option specifies the port that the service listens on. This port is available only to pods within the cluster. If a port value is not provided, then the port is copied from the configuration of the deployment.

The `--target-port` option specifies the name or number of the container port that the service uses to communicate with the pods. If you do not provide a port value, then the port is copied from the configuration of the deployment.

The `--protocol` option determines the network protocol for the service. TCP is used by default.

The `--name` option explicitly names the service. If you do not specify a name, then the service uses the same name as the deployment.

To view the selector that a service uses, use the `-o wide` option with the `oc get` command.

```
[user@host ~]$ oc get service db-pod -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
db-pod	ClusterIP	172.30.108.92	<none>	3306/TCP	108s	app=db-pod

In this example, `db-pod` is the name of the service. Pods must use the `app=db-pod` label to be included in the host list for the `db-pod` service. To see the endpoints that a service uses, use the `oc get endpoints` command.

```
[user@host ~]$ oc get endpoints
```

NAME	ENDPOINTS	AGE
------	-----------	-----

```
db-pod    10.8.0.86:3306,10.8.0.88:3306    27s
```

This example illustrates a service with two pods in the host list. The `oc get endpoints` command returns the service endpoints in the current selected project. Add the name of the service to the command to show only the endpoints of a single service. Use the `--namespace` option to view the endpoints in a different namespace.

Use the `oc describe deployment <deployment name>` command to view the deployment selector.

```
[user@host ~]$ oc describe deployment db-pod
Name:                db-pod
Namespace:           deploy-services
CreationTimestamp:    Wed, 18 Jan 2023 17:46:03 -0500
Labels:              app=db-pod
Annotations:         deployment.kubernetes.io/revision: 2
Selector:            app=db-pod
...output omitted...
```

You can view or parse the selector from the YAML or JSON output for the deployment resource from the `spec.selector.matchLabels` object. In this example, the `-o yaml` option of the `oc get` command returns the selector label that the deployment uses.

```
[user@host ~]$ oc get deployment/<deployment_name> -o yaml
...output omitted...
selector:
  matchLabels:
    app: db-pod
...output omitted...
```

Kubernetes DNS for Service Discovery

Kubernetes uses an internal Domain Name System (DNS) server that the DNS operator deploys. The DNS operator creates a default cluster DNS name, and assigns DNS names to services that you define. The DNS operator implements the

DNS API from the `operator.openshift.io` API group. The operator deploys CoreDNS, creates a service resource for CoreDNS, and configures the `kubelet` component to instruct pods to use the CoreDNS service IP address for name resolution. When a service does not have a cluster IP address, the DNS operator assigns a DNS record that resolves to the set of IP addresses of the pods behind the service.

The DNS server discovers a service from a pod by using the internal DNS server, which is visible only to pods. Each service is dynamically assigned a *Fully Qualified Domain Name* (FQDN) that uses the following format:

```
SVC-NAME.PROJECT-NAME.svc.CLUSTER-DOMAIN
```

When a pod is created, Kubernetes provides the container with a `/etc/resolv.conf` file with similar contents to the following items:

```
[user@host ~]$ cat /etc/resolv.conf
nameserver 172.30.0.10
search deploy-services.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

In this example, `deploy-services` is the project name for the pod, and `cluster.local` is the cluster domain.

The `nameserver` directive provides the IP address of the Kubernetes internal DNS server. The `options ndots` directive specifies the number of dots that must appear in a name to qualify for an initial absolute query. Alternative hostname values are derived by appending values from the `search` directive to the name that you send to the DNS server.

In the `search` directive in this example, the `svc.cluster.local` entry enables any pod to communicate with another pod in the same cluster by using the service name and project name:

```
SVC-NAME.PROJECT-NAME
```

The first entry in the `search` directive enables a pod to use the service name to specify another pod in the same project. In Red Hat OpenShift Container Platform

(RHOCP), a project is also the namespace for the pod. The service name alone is sufficient for pods in the same RHOCP project:

SVC-NAME

Kubernetes Networking Drivers

Container Network Interface (CNI) plug-ins provide a common interface between the network provider and the container runtime. CNI defines the specifications for plug-ins that configure network interfaces inside containers. Plug-ins that are written to the specification enable different network providers to control the RHOCP cluster network.

Red Hat provides the following CNI plug-ins for a RHOCP cluster:

- OVN-Kubernetes: The default plug-in for first-time installations of RHOCP 4.10 and later versions.
- OpenShift SDN: An earlier plug-in from RHOCP 3.x; it is incompatible with some later features of RHOCP 4.x.

Certified CNI-plugins from other vendors are also compatible with an RHOCP cluster.

The SDN uses CNI plug-ins to create Linux namespaces to partition the usage of resources and processes on physical and virtual hosts. With this implementation, containers inside pods can share network resources, such as devices, IP stacks, firewall rules, and routing tables. The SDN allocates a unique routable IP to each pod, so that you can access the pod from any other service in the same network.

In OpenShift 4.18, OVN-Kubernetes serves as the default network provider and OpenShift SDN is deprecated.

OVN-Kubernetes uses OVN to manage the cluster network. A cluster that uses the OVN-Kubernetes plug-in also runs Open vSwitch (OVS) on each node. OVN configures OVS on each node to implement the declared network configuration.

The OpenShift Cluster Network Operator

RHOCP provides a Cluster Network Operator (CNO) that configures OpenShift cluster networking. The CNO serves as an OpenShift cluster operator that loads and

configures CNI plug-ins. As a cluster administrator, execute the following command to observe the status of the CNO:

```
[user@host ~]$ oc get -n openshift-network-operator deployment/network-operator
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
network-operator	1/1	1	1	41d

An administrator configures the CNO at installation time. To view the configuration, use the following command:

```
[user@host ~]$ oc describe network.config/cluster
```

Name: cluster

...output omitted...

Spec:

Cluster Network:

Cidr: 10.128.0.0/14

Host Prefix: 23

External IP:

Policy:

Network Type: OVNKubernetes

Service Network:

172.30.0.0/16

...output omitted...

The Cluster Network CIDR defines the range of IP addresses for all pods in the cluster. The Service Network CIDR defines the range of IP addresses for

all services in
the cluster.

Guided Exercise: Kubernetes Pod and Service Networks

Deploy a database server and access it through a Kubernetes service.

Outcomes

Deploy a database server, and access it indirectly through a Kubernetes service, and also directly pod-to-pod for troubleshooting.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `deploy-services` project and the `/home/student/D0180/labs/deploy-services/resources.txt` file. The `resources.txt` file contains commands that you can copy and paste to use in this exercise.

```
[student@workstation ~]$ lab start deploy-services
```

NOTE

It is safe to ignore pod security warnings for exercises in this course. OpenShift uses the Security Context Constraints controller to provide safe defaults for pod security.

Instructions

1. Log in to the OpenShift cluster as the `developer` user with `developer` as the password. Use the `deploy-services` project.
1. Log in to the OpenShift cluster.

2. `[student@workstation ~]$ oc login -u developer -p developer \`
3. `https://api.ocp4.example.com:6443`
4. Login successful.

```
...output omitted...
```

5. Set the `deploy-services` project as the active project.

```
6. [student@workstation ~]$ oc project deploy-services
```

```
...output omitted...
```

2. Use the `registry.ocp4.example.com:8443/rhel8/mysql-80` container image to create a MySQL deployment named `db-pod`. Add the missing environment variables for the pod to run.

1. Create the `db-pod` deployment.

```
2. [student@workstation ~]$ oc create deployment db-pod --port 3306 \
3.   --image registry.ocp4.example.com:8443/rhel8/mysql-80
```

```
deployment.apps/db-pod created
```

4. Add the environment variables.

```
5. [student@workstation ~]$ oc set env deployment/db-pod \
6.   MYSQL_USER=user1 \
7.   MYSQL_PASSWORD=mypa55w0rd \
8.   MYSQL_DATABASE=items
```

```
deployment.apps/db-pod updated
```

9. Confirm that the pod is running.

```
10. [student@workstation ~]$ oc get pods
```

11. NAME	READY	STATUS	RESTARTS	AGE
----------	-------	--------	----------	-----

db-pod-6ccc485cfc-vrc4r	1/1	Running	0	2m30s
-------------------------	-----	---------	---	-------

Your pod name might differ from the previous output.

3. Expose the `db-pod` deployment to create a ClusterIP service.

1. View the deployment for the pod.

2. [student@workstation ~]\$ **oc get deployment**

3. NAME READY UP-TO-DATE AVAILABLE AGE

db-pod	1/1	1	1	3m36s
--------	-----	---	---	-------

4. Expose the db-pod deployment to create a service.

5. [student@workstation ~]\$ **oc expose deployment/db-pod**

service/db-pod exposed

4. Validate the service. Verify that the service selector matches the pod label. Then, confirm that the db-pod service endpoint matches the pod IP address.

1. Identify the selector for the db-pod service. Use the `oc get service` command with the `-o wide` option.

2. [student@workstation ~]\$ **oc get service db-pod -o wide**

3. NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
 GE SELECTOR

db-pod	ClusterIP	172.30.108.92	<none>	3306/TCP	
108s	app=db-pod				

The selector shows an `app=db-pod` key:value pair.

4. Capture the pod name in a variable.

5. [student@workstation ~]\$ **PODNAME=\$(oc get pods **

-o jsonpath='{.items[0].metadata.name}')

6. Query the label on the pod.

7. [student@workstation ~]\$ **oc get pod \$PODNAME --show-labels**

8. NAME READY STATUS RESTARTS AGE L
 ABELS

db-pod-6ccc485cfc-vrc4r	1/1	Running	0	6m50s	
app=db-pod ...					

The label list includes the `app=db-pod` key-value pair, which matches the service selector.

9. Retrieve the endpoints for the `db-pod` service.

```
10. [student@workstation ~]$ oc get endpoints
```

```
11. NAME           ENDPOINTS           AGE
```

```
db-pod    10.8.0.85:3306    4m38s
```

The endpoint IP is different in your output.

12. Verify that the service endpoint matches the pod IP address.
Use the `oc get pods` command with the `-o wide` option.

```
13. [student@workstation ~]$ oc get pods -o wide
```

```
14. NAME           READY   STATUS    RESTARTS   AGE   I
P           ...
```

```
db-pod-6ccc485cfc-vrc4r    1/1    Running    0           54m    1
0.8.0.85 ...
```

The service endpoint resolves to the pod's IP address.

5. Delete and re-create the `db-pod` deployment. Confirm that the `db-pod` service endpoint automatically resolves to the new pod's IP address.

1. Delete the `db-pod` deployment.

```
2. [student@workstation ~]$ oc delete deployment.apps/db-pod
```

```
deployment.apps "db-pod" deleted
```

3. Verify that the service still exists without the deployment.

```
4. [student@workstation ~]$ oc get service
```

```
5. NAME           TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)    AG
E
```

```
db-pod    ClusterIP    172.30.108.92  <none>        3306/TCP   9
m53s
```

6. Confirm that the service endpoints list is empty.

```
7. [student@workstation ~]$ oc get endpoints
```

```
8. NAME          ENDPOINTS    AGE
```

```
db-pod          <none>      12m
```

9. Re-create the db-pod deployment.

```
10. [student@workstation ~]$ oc create deployment db-pod --port 3306 \
```

```
11. --image registry.ocp4.example.com:8443/rhel8/mysql-80
```

```
deployment.apps/db-pod created
```

12. Add the environment variables.

```
13. [student@workstation ~]$ oc set env deployment/db-pod \
```

```
14.  MYSQL_USER=user1 \
```

```
15.  MYSQL_PASSWORD=mypa55w0rd \
```

```
16.  MYSQL_DATABASE=items
```

```
deployment.apps/db-pod updated
```

17. Confirm that the new pod has the app=db-pod selector.

```
18. [student@workstation ~]$ oc get pods --selector app=db-pod -o wide
```

```
19. NAME                                READY   STATUS    RESTARTS   AGE   IP
    ...
```

```
db-pod-6ccc485cfc-12x    1/1     Running   0           32s   10.8.0.85 ...
```

The pod name might differ, and the pod IP address might also change.

20. Confirm that the endpoints include the new pod.

```
21. [student@workstation ~]$ oc get endpoints
```

22. NAME	ENDPOINTS	AGE
db-pod	10.8.0.85:3306	16m

6. Create a pod to identify the available DNS name assignments for the service.

1. Create a pod named `shell` for troubleshooting. Use the `oc run` command with the `registry.ocp4.example.com:8443/openshift4/network-tools-rhel8` image.

```
2. [student@workstation ~]$ oc run shell -it \
3.   --image registry.ocp4.example.com:8443/openshift4/network-tools-rhel8
4. If you don't see a command prompt, try pressing enter.
```

```
bash-4.4$
```

5. View the `/etc/resolv.conf` file from the `shell` pod to identify the cluster-domain name.

```
6. bash-4.4$ cat /etc/resolv.conf
7. search deploy-services.svc.cluster.local svc.cluster.local ...
8. nameserver 172.30.0.10
```

```
options ndots:5
```

The container uses the `search` directive values as suffixes for DNS queries. The cluster-domain name follows `svc`.

9. Test the available DNS names with the `nc` and `echo` commands.

```
10. bash-4.4$ nc -z db-pod.deploy-services 3306 && \
11.   echo "Connection success to db-pod.deploy-services:3306" || \
12.   echo "Connection failed"
```

```
Connection success to db-pod.deploy-services:3306
```

13. Exit the interactive session.

```
14. bash-4.4$ exit
```

```
Session ended, resume using 'oc attach shell -c shell -i -t' c  
ommand when the pod is running
```

15. Delete the shell pod.

```
16. [student@workstation ~]$ oc delete pod shell
```

```
pod "shell" deleted
```

7. Test pod communications across namespaces with a new project.

1. Create a second namespace with the `oc new-project` command.

```
2. [student@workstation ~]$ oc new-project deploy-services-2
```

3. Now using project "deploy-services-2" on server "https://api.ocp4.example.com:6443".

```
...output omitted...
```

4. Test DNS name access from a pod in the new namespace.

```
5. [student@workstation ~]$ oc run shell -it --rm \  
6.   --image registry.ocp4.example.com:8443/openshift4/network-too  
   ls-rhel8 \  
7.   --restart Never -- nc -z db-pod.deploy-services.svc.cluster.l  
   ocal 3306 && \  
8.   echo "Connection success to db-pod.deploy-services.svc.cluste  
   r.local:3306" || \  
9.   echo "Connection failed"  
10. pod "shell" deleted
```

```
Connection success to db-pod.deploy-services.svc.cluster.local  
:3306
```

11. Return to the deploy-services project.


```
12. [student@workstation ~]$ oc project deploy-services
```

```
Now using project "deploy-services" on server "https://api.ocp4.example.com:6443".
```

8. Use a Kubernetes job to add initialization data to the database.

1. Create a job named `mysql-init` by using the `registry.ocp4.example.com:8443/redhattraining/do180-dbinit:v1` image. This image, which is based on `mysql-80`, includes a script to add initial records.

```
2. [student@workstation ~]$ oc create job mysql-init \  
3.   --image registry.ocp4.example.com:8443/redhattraining/do180-d  
   binit:v1 \  
4.   -- /bin/bash -c "mysql -uuser1 -pmypa55w0rd --protocol tcp \  
5.   -h db-pod -P3306 items </tmp/db-init.sql"
```

```
job.batch/mysql-init created
```

The `-h` option directs the command to the `db-pod` service short name. The `--` option separates `oc` arguments from the pod command. The `/tmp/db-init.sql` file, which is included in the image, contains the following queries:

```
DROP TABLE IF EXISTS `Item`;  
  
CREATE TABLE `Item` (`id` BIGINT not null auto_increment prima  
ry key, `description` VARCHAR(100), `done` BIT);  
  
INSERT INTO `Item` (`id`,`description`,`done`) VALUES (1,'Pick  
up newspaper', 0);  
  
INSERT INTO `Item` (`id`,`description`,`done`) VALUES (2,'Buy  
groceries', 1);
```

6. Confirm the `mysql-init` job status, and wait for its completion.

```
7. [student@workstation ~]$ oc get job
```

8. NAME	STATUS	COMPLETIONS	DURATION	AGE
---------	--------	-------------	----------	-----

mysql-init	Complete	1/1	28s	42s
------------	----------	-----	-----	-----

9. Check the mysql-init pod status.

10. [student@workstation ~]\$ oc get pods				
11. NAME	READY	STATUS	RESTARTS	AGE
E				
12. db-pod-6ccc485cfc-2lk1x	1/1	Running	0	4h
24m				

mysql-init-ln9cg	0/1	Completed	0	23
m				

13. Delete the mysql-init job.

14. [student@workstation ~]\$ oc delete job mysql-init

job.batch "mysql-init" deleted

15. Verify that the mysql-init pod is deleted.

16. [student@workstation ~]\$ oc get pods				
17. NAME	READY	STATUS	RESTARTS	AGE

db-pod-6ccc485cfc-2lk1x	1/1	Running	0	4h2
-------------------------	-----	---------	---	-----

9. Create a query-db pod to query the database service.

1. Create the query-db pod. Use the MySQL client to query the db-pod service.

2. [student@workstation ~]\$ oc run query-db -it --rm \
3. --image registry.ocp4.example.com:8443/redhattraining/do180-d
binit:v1 \
4. --restart Never \
5. -- mysql -uuser1 -pmypa55w0rd --protocol tcp \
6. -h db-pod -P3306 items -e 'select * from Item;'
7. mysql: [Warning] Using a password on the command line interface
can be insecure.
8. +-----+
9. id description done

```

10. +---+-----+-----+
11. |  1 | Pick up newspaper | 0x00 |
12. |  2 | Buy groceries    | 0x01 |
13. +---+-----+-----+

```

```
pod "query-db" deleted
```

10. Use pod-to-pod communication for troubleshooting.

1. Confirm the IP address of the MySQL database pod.

```
2. [student@workstation ~]$ oc get pods -o wide
```

```

3. NAME                                READY  STATUS      RESTARTS   AGE  IP
   ...

```

```

db-pod-6ccc485cfc-2lk1x 1/1    Running    0          4h5   1
0.8.0.69 ...

```

Your pod IP address might differ.

4. Capture the IP address in an environment variable.

```
5. [student@workstation ~]$ POD_IP=$(oc get pod -l app=db-pod \
```

```
-o jsonpath='{.items[0].status.podIP}')
```

6. Create a test pod named shell. Use nc to test the \$POD_IP and port 3306.

```

7. [student@workstation ~]$ oc run shell --env POD_IP=$POD_IP -it
   --rm \
8.   --image registry.ocp4.example.com:8443/openshift4/network-tools-rhel8 \
9.   --restart Never \
10.  -- nc -z $POD_IP 3306 && echo "Connection success to $POD_IP
    :3306" || \
11.  echo "Connection failed"
12. pod "shell" deleted

```

```
Connection success to 10.8.0.69:3306
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish deploy-services
```

Scale and Expose Applications to External Access

Objectives

- Expose applications to clients outside the cluster by using Kubernetes ingress and OpenShift routes.

Outcomes

- Allocate IP addresses for pods and services in a Red Hat OpenShift Container Platform (RHOCP) cluster.
- Configure services to enable communication between pods.
- Expose applications to external networks by using routes and ingress objects.

IP Addresses for Pods and Services

Real-world applications typically require multiple pods to scale horizontally. To meet growing user demand, RHOCP deploys many pods that run the same containers from a single pod resource definition. A `Service` resource defines a single IP and port combination. The resource assigns a stable IP address to a pool of pods. The resource balances client requests across those pods.

RHOCP assigns each service a unique IP address for clients to connect to. The assignment uses a round-robin load-balancing strategy by default. The service IP address originates from an internal OVN-Kubernetes network. The network, which is separate from the pod

network, is accessible only to pods within the cluster. RHOCP adds each pod to match the selector of the service as an endpoint.

NOTE

OVN-Kubernetes, which is the default network provider starting with RHOCP 4.18, manages pod-to-pod and service-to-pod communication. The provider ensures efficient traffic routing within the cluster.

Containers within Kubernetes pods must avoid connecting directly to dynamic IP addresses of other pods. Instead, services link stable OVN-Kubernetes IP addresses to pods to ensure scalability and fault tolerance. When pods restart, replicate, or reschedule to different nodes, RHOCP updates the service endpoints accordingly.

Service Types

RHOCP offers several service types to meet diverse application needs, cluster infrastructure, and security requirements.

ClusterIP

The `clusterIP` type, which is the default unless otherwise specified, assigns a cluster-internal IP address to the service. The assignment makes the service accessible only within the RHOCP cluster.

`clusterIP` services enable pod-to-pod communication. RHOCP allocates IP addresses from a dedicated service network. The network, which is accessible only inside the cluster, supports internal communication. Most applications benefit from this service type because Kubernetes automates management of the service.

LoadBalancer

The `LoadBalancer` type directs RHOCP to provision a load balancer of the cloud provider. The load balancer assigns an externally accessible IP address to the application.

Use caution when deploying `LoadBalancer` services because costs can be high when assigning a load balancer to every application. These services expose applications to external networks. The exposure requires additional security configurations to prevent unauthorized access.

ExternalIP

The `ExternalIP` service redirects traffic from a virtual IP address on a cluster node to a pod. A cluster administrator assigns the virtual IP address to a node. The administrator configures failover to another node if needed.

WARNING

Starting with RHOCP 4.18, `ExternalIP` is a legacy feature due to security concerns. Direct network connections to cluster nodes, which this service requires, conflict with most security policies. Use the `LoadBalancer` or the `NodePort` type unless the `ExternalIP` type meets specific requirements.

RHOCP configures Network Address Translation (NAT) rules to route traffic from the virtual IP address to the pod. Administrators must ensure that external IP addresses route correctly to nodes. Additional security measures protect the cluster from external access.

NodePort

The `NodePort` service exposes a service on a specific port, with a default range of 30000–32767, on the IP address of each cluster node. Each node redirects traffic from this port to the endpoints of the service.

WARNING

`NodePort` services require direct network connections to cluster nodes, which poses a security risk. Implement strict firewall rules and access controls to mitigate risks.

ExternalName

The `ExternalName` service maps a Kubernetes service to an external DNS name that is specified in the `externalName` field. The Kubernetes DNS server returns the `externalName` in a Canonical Name (CNAME) record. The record directs clients to resolve the external name to an IP address.

Using Routes for External Connectivity

RHOCP provides `Route` resources to expose applications to external networks. `Route` resources support HTTP, HTTPS, TCP, and other protocols, and prioritize HTTP and TLS-based applications for external exposure. Non-HTTP applications, such as databases, typically remain internal. `Route` and `Ingress` resources handle ingress traffic in RHOCP.

A `Route` resource assigns a publicly accessible hostname to an application. The OpenShift ingress controller, which is HAProxy-based, redirects traffic from a public IP address to pods. The OpenShift ingress controller is the default, with support for third-party ingress controllers that are deployed in parallel. `Route` resources offer advanced features, which surpass standard Kubernetes `Ingress` capabilities. Features include TLS re-encryption, passthrough, and split traffic for blue-green deployments.

To create a route by using the `oc` CLI, use the following command:

```
[user@host ~]$ oc expose service api-frontend --hostname api.apps.acme.com
```

The `--target-port` option of the `oc expose` command specifies the name or number of the container port that the service uses to communicate with the pods. If you provide no port value, then the port copies from the configuration of the deployment.

Omitting the `--hostname` option prompts RHOCP to generate a hostname in the `<route-name>-<project-name>.<default-domain>` format. For example, a `frontend` route in an `api` project with a wildcard domain of `apps.example.com` becomes `frontend-api.apps.example.com`.

IMPORTANT

The DNS server for the wildcard domain resolves all names to configured IPs. The server does not recognize specific route hostnames. The OpenShift ingress controller treats route hostnames as HTTP virtual hosts. Invalid or non-existent route hostnames trigger an HTTP 503 error.

When creating a route, configure the following settings:

1. **Service Name:** Specifies the service to determine target pods.
2. **Hostname:** Sets the route hostname as a subdomain of the cluster wildcard domain, which RHOCP can autogenerate.
3. **Path:** Defines an optional path for path-based routing.
4. **Target Port:** Matches the `targetPort` in the service, and specifies the listening port of the application.
5. **Encryption Strategy:** Chooses secure (TLS) or insecure routing.

The following example shows a minimal route definition:

```
kind: Route
apiVersion: route.openshift.io/v1
metadata:

  name: a-simple-route

  labels:
    app: API
    name: api-frontend
spec:

  host: api.apps.acme.com
  to:
    kind: Service

    name: api-frontend
```



```
port:
```

```
targetPort: 8443
```

Specifies a unique route name.

Defines selectors for the route.

Sets the hostname, which is a subdomain of the cluster wildcard domain.

Identifies the service to redirect traffic to, by determining target pods.

Maps the router port to the endpoint port of the service.

NOTE

Some ecosystem components integrate with Ingress resources but not with Route resources. Starting with RHOCP 4.18, creating an Ingress object automatically generates a managed Route object, which is deleted when the Ingress object is removed.

To delete a route, use the following command:

```
[user@host ~]$ oc delete route a-simple-route
```

To create a route from the RHOCP web console, go to **Networking** → **Routes**. Click **Create Route**. Customize the name, hostname, path, and target service by using the form or the YAML editor.

Project: metallb-system ▾

Create Route

Routing is a way to make your application publicly visible.

Configure via: ☒ Form view ☐ YAML view

Name *

A unique name for the Route within the project.

Hostname

Public hostname for the Route. If not specified, a hostname is generated.

Path

Using Ingress Objects for External Connectivity

A Kubernetes Ingress resource provides similar functions to RHOCP Route resources, and supports HTTP, HTTPS, Server Name Indication (SNI), and TLS with SNI. The OpenShift ingress controller processes Ingress objects, and enables features such as TLS termination, path redirection, and sticky sessions.

NOTE

Although Ingress resources are standard in Kubernetes, Route resources are preferred for advanced features and native integration with the OpenShift ingress controller.

To create an Ingress object, use the following command:

```
[user@host ~]$ oc create ingress ingr-sakila --rule="ingr-sakila.apps.ocp4.example.com/*=sakila-service:8080"
```

The following example shows a minimal Ingress definition:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
name: frontend
spec:
  rules:
    - host: www.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
  tls:
    - hosts:
        - www.example.com
      secretName: example-com-tls-certificate
```

Specifies a unique name for the Ingress object.

Sets the hostname for inbound traffic.

Identifies the service to redirect traffic to.

Defines the port for the service back end.

Configures TLS for secure paths, which requires a matching host and a certificate secret.

To delete an Ingress object, use the following command:

```
[user@host ~]$ oc delete ingress frontend
```

Sticky Sessions

Sticky sessions ensure that stateful application traffic reaches the same pod for a user session. The OpenShift ingress controller can use cookies to manage session persistence for Route and Ingress resources. The controller selects a pod for a user request. The controller generates a cookie. The controller includes the cookie in the response. The client sends the cookie with subsequent requests, which enables the controller to route traffic to the same pod.

To configure session persistence for an Ingress object by using a cookie, use the following command:

```
[user@host ~]$ oc annotate ingress ingr-example ingress.kubernetes.io/affinity=cookie
```

To configure session persistence for a Route object with a custom cookie named `myapp`, use the following command:

```
[user@host ~]$ oc annotate route route-example router.openshift.io/cookie_name=myapp
```

To capture the route hostname, use the following command:

```
[user@host ~]$ ROUTE_NAME=$(oc get route route-example -o jsonpath='{.spec.host}')
```

To access the route and save the cookie, use the following command:

```
[user@host ~]$ curl $ROUTE_NAME -k -c /tmp/cookie_jar
```

To use the saved cookie for subsequent requests, use the following command:

```
[user@host ~]$ curl $ROUTE_NAME -k -b /tmp/cookie_jar
```

Using the saved cookie ensures that requests reach the same pod.

NOTE

Cookies cannot be set on passthrough routes because the HTTP traffic is encrypted, and the router does not terminate TLS or read the contents of the request.

Load Balance and Scale Applications

Administrators and developers can scale the number of replica pods in a deployment to handle traffic surges or to conserve resources.

To scale a deployment, use the following command:

```
[user@host ~]$ oc scale --replicas=5 deployment/scale
```

The deployment updates the replica set. The replica set creates or deletes pods to match the intended replica count.

Avoid modifying a replica set directly, because the deployment controller manages these changes.

Load Balance Pods

A Kubernetes `Service` resource functions as an internal load balancer, by providing access to a workload via the service name. The service distributes incoming connections across replicated pods. The OpenShift ingress controller uses the selector of the service to identify the service and endpoints. When both the controller and a service provide load balancing, RHOCP relies on the controller to direct traffic to pods. The controller detects changes in IP addresses of the service. The controller updates the configuration. Custom controllers can propagate API object changes to external routing solutions.

The OpenShift ingress controller maps external hostnames. The controller balances service endpoints over protocols that include distinguishing information, such as HTTP host headers.

Guided Exercise: Scale and Expose Applications to External Access

Deploy one web server and access it through a Kubernetes ingress; and deploy another web server and access it through an OpenShift route.

Outcomes

In this exercise, you deploy two web applications to access them through an ingress object and a route, and scale them to verify the load-balance between the pods.

- Deploy two web applications.
- Create a route and an ingress object to access the web applications.
- Enable the sticky sessions for the web applications.
- Scale the web applications to load-balance the service.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible.

```
[student@workstation ~]$ lab start deploy-routes
```

Instructions

1. Create two web application deployments, named `satir-app` and `sakila-app`. Use the `registry.ocp4.example.com:8443/httpd-app:v1` container image for both deployments.
1. Log in to the OpenShift cluster as the `developer` user with `developer` as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful
5. You have one project on this server: "web-applications"
```

...output omitted...

6. Change to the `web-applications` project.

7. [student@workstation ~]\$ **oc project web-applications**

8. Now using project "web-applications" on server "https://api.ocp4.example.com:6443".

...output omitted...

9. Create the satir-app web application deployment by using the registry.ocp4.example.com:8443/redhattraining/do180-httpd-app:v1 container image.

10. [student@workstation ~]\$ **oc create deployment satir-app **

11. **--image registry.ocp4.example.com:8443/redhattraining/do180-httpd-app:v1**

deployment.apps/satir-app created

12. After a few moments, verify that the deployment is successful.

13. [student@workstation ~]\$ **oc get pods**

14. NAME	READY	STATUS	RESTARTS	...
----------	-------	--------	----------	-----

satir-app-787b7d7858-5dfsh	1/1	Running	0	...
----------------------------	-----	---------	---	-----

[student@workstation ~]\$ **oc get deploy/satir-app**

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
satir-app	1/1	1	1	6m39s

15. Create the sakila-app web application deployment by using the registry.ocp4.example.com:8443/redhattraining/do180-httpd-app:v1 image.

16. [student@workstation ~]\$ **oc create deployment sakila-app **

17. **--image registry.ocp4.example.com:8443/redhattraining/do180-httpd-app:v1**

deployment.apps/sakila-app created

18. Wait a few moments and then verify that the deployment is successful.

19. [student@workstation ~]\$ **oc get pods**

20. NAME	READY	STATUS	RESTARTS	...
----------	-------	--------	----------	-----

21. sakila-app-6694...5kpd	1/1	Running	0	...
----------------------------	-----	---------	---	-----

```
satir-app-787b7...dfsh 1/1 Running 0 ...
```

```
[student@workstation ~]$ oc get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
sakila-app	1/1	1	1	41s
satir-app	1/1	1	1	62m

2. Create services for the web application deployments. Then, use the services to create a route for the satir-app application and an ingress object for the sakila-app application.

1. Expose the satir-app deployment. Name the service satir-svc, and specify port 8080 as the port and target port.

2. [student@workstation ~]\$ oc expose deployment satir-app --name satir-svc \
3. --port 8080 --target-port 8080

```
service/satir-svc exposed
```

4. Expose the sakila-app deployment to create the sakila-svc service.

5. [student@workstation ~]\$ oc expose deployment sakila-app --name sakila-svc \
6. --port 8080 --target-port 8080

```
service/sakila-svc exposed
```

7. Verify the status of the services.

8. [student@workstation ~]\$ oc get services

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	...
10. sakila-svc	ClusterIP	172.30.230.41	<none>	8080/TCP	...

satir-svc	ClusterIP	172.30.143.15	<none>	8080/TCP	...
-----------	-----------	---------------	--------	----------	-----

11. Verify the status of the endpoints.

12. [student@workstation ~]\$ oc get endpoints

NAME	ENDPOINTS	AGE
14. sakila-svc	10.8.0.66:8080	25s


```
satir-svc    10.8.0.65:8080    43s
```

15. Verify the status of the pods.

```
16.[student@workstation ~]$ oc get pods -o wide
```

17. NAME	READY	STATUS	RESTARTS	AGE	IP	...
18. sakila-app-6694...5kpd	1/1	Running	0	92s	10.8.0.66	...

```
satir-app-787b7...dfsh 1/1    Running 0          2m49s 10.8.0.65 ...
```

19. Expose the `satir-svc` service to create a route named `satir` for the `satir-app` web application.

```
20. [student@workstation ~]$ oc expose service satir-svc --name satir
```

route.route.openshift.io/satir exposed

```
[student@workstation ~]$ oc get routes
```

NAME	HOST/PORT	...	SERVICES	PORT
...				
satir	satir-web-applications.apps.ocp4.example.com	...	satir-svc	8080
...				

21. Create an ingress object named `ingr-sakila` for the `sakila-svc` service. Configure the `--rule` option with the following values:

Field	Value
Host	ingr-sakila.apps.ocp4.example.com
Service name	sakila-svc
Port number	8080

```
22. [student@workstation ~]$ oc create ingress ingr-sakila \
```

```
23.  --rule "ingr-sakila.apps.ocp4.example.com/*=sakila-svc:8080"
```

24.ingress.networking.k8s.io/ingr-sakila created

```
25.[student@workstation ~]$ oc get ingress
```

26. NAME	... HOSTS	ADDRESS	PORTS ...
----------	-----------	---------	-----------

```
27. ingr-sakila ... ingr-sakila.apps.ocp4.example.com router...com 80 ...
```

28. Confirm that a route exists for the ingr-sakila ingress object.

```
29. [student@workstation ~]$ oc get routes
```

```
30. NAME                                HOST/PORT                                ... SERVICES
    PORT
```

```
31. ingr-sakila... ingr-sakila.apps.ocp4.example.com ... sakila-svc
    <all>
```

```
satir                                satir-web-applications.apps.ocp4.example.com ... satir-svc
8080
```

A specific port is not assigned to routes that ingress objects created. By contrast, a route that an exposed service created is assigned the same ports as the service.

32. Use the curl command to access the ingr-sakila ingress object and the satir route. The output states the name of the pod that is servicing the request.

```
33. [student@workstation ~]$ curl ingr-sakila.apps.ocp4.example.com
```

```
Welcome to Red Hat Training, from sakila-app-66947cdd78-x5kpd
```

```
[student@workstation ~]$ curl satir-web-applications.apps.ocp4.example.com
```

```
Welcome to Red Hat Training, from satir-app-787b7d7858-bdfsh
```

3. Scale the web application deployments to load-balance their services.

Scale the sakila-app deployment to two replicas, and scale the satir-app deployment to three replicas.

1. Scale the sakila-app deployment with two replicas.

```
2. [student@workstation ~]$ oc scale deployment sakila-app --replicas 2
```

```
deployment.apps/sakila-app scaled
```

3. Wait a few moments and then verify the status of the replica pods.

```
4. [student@workstation ~]$ oc get pods
```

```
5. NAME                                READY   STATUS    RESTARTS   ...
```

```
6. sakila-app-6694...5kpd             1/1     Running   0           ...
```

```
7. sakila-app-6694...rfzg 1/1 Running 0 ...
```

```
satir-app-787b...dfsh 1/1 Running 0 ...
```

8. Scale the satir-app deployment with three replicas.

```
9. [student@workstation ~]$ oc scale deployment satir-app --replicas 3
```

```
deployment.apps/satir-app scaled
```

10. Wait a few moments and then verify the status of the replica pods.

```
11. [student@workstation ~]$ oc get pods -o wide
```

12. NAME	READY	STATUS	RESTARTS	...	IP	...
13. sakila-app-6694...5kpd	1/1	Running	0	...	10.8.0.66	...
14. sakila-app-6694...rfzg	1/1	Running	0	...	10.8.0.67	...
15. satir-app-787b...dfsh	1/1	Running	0	...	10.8.0.65	...
16. satir-app-787b...z8xm	1/1	Running	0	...	10.8.0.69	...

```
satir-app-787b...7bhj 1/1 Running 0 ... 10.8.0.70 ...
```

17. Retrieve the service endpoints to confirm that the services are load-balanced between the additional replica pods.

```
18. [student@workstation ~]$ oc get endpoints
```

19. NAME	ENDPOINTS	...
20. sakila-svc	10.8.0.66:8080,10.8.0.67:8080	...

```
satir-svc 10.8.0.65:8080,10.8.0.69:8080,10.8.0.70:8080 ...
```

4. Enable the sticky sessions for the sakila-app web application. Then, use the curl command to confirm that the sticky sessions are working for the ingr-sakila object.

1. Configure a cookie for the ingr-sakila ingress object.

```
2. [student@workstation ~]$ oc annotate ingress ingr-sakila \
```

```
3. ingress.kubernetes.io/affinity=cookie
```

```
ingress.networking.k8s.io/ingr-sakila annotated
```

4. Use the `curl` command to access the `ingr-sakila` ingress object. The output states the name of the pod that is servicing the request. Notice that the connection is load-balanced between the replicas.

```
5. [student@workstation ~]$ for i in {1..10}; do \  
6.   curl ingr-sakila.apps.ocp4.example.com ; done  
7. Welcome to Red Hat Training, from sakila-app-66947cdd78-x5kpd  
8. Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg  
9. Welcome to Red Hat Training, from sakila-app-66947cdd78-x5kpd
```

...output omitted...

10. Use the `curl` command to save the `ingr-sakila` ingress object cookie to the `/tmp/cookie_jar` file.

```
11. [student@workstation ~]$ curl ingr-sakila.apps.ocp4.example.com \  
12.   -c /tmp/cookie_jar
```

Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg

13. Confirm that the cookie exists in the `/tmp/cookie_jar` file.

```
14. [student@workstation ~]$ cat /tmp/cookie_jar  
15. ...output omitted...
```

```
#HttpOnly_ingr-sakila.apps.ocp4.example.com      FALSE /      FALSE  0  
b9b484110526b4b1b3159860d3aeb04 921e139c5145950d00424bf3b0a46d22
```

16. The cookie provides session stickiness for connections to the `ingr-sakila` route. Use the `curl` command and the cookie in the `/tmp/cookie_jar` file to connect to the `ingr-sakila` route again. Confirm that you are connected to the same pod that handled the request in the previous step.

```
17. [student@workstation ~]$ for i in {1..10}; do \  
18.   curl ingr-sakila.apps.ocp4.example.com -b /tmp/cookie_jar; done  
19. Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg
```

```
20.Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg
21.Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg
```

...output omitted...

22. Use the `curl` command to connect to the `ingr-sakila` route without the cookie. Observe that session stickiness occurs only with the cookie.

```
23.[student@workstation ~]$ for i in {1..10}; do \
24.  curl ingr-sakila.apps.ocp4.example.com ; done
25.Welcome to Red Hat Training, from sakila-app-66947cdd78-x5kpd
26.Welcome to Red Hat Training, from sakila-app-66947cdd78-xrfzg
27.Welcome to Red Hat Training, from sakila-app-66947cdd78-x5kpd
```

...output omitted...

5. Enable the sticky sessions for the `satir-app` web application. Then, use the `curl` command to confirm that sticky sessions are active for the `satir` route.

1. Configure a cookie with the `hello` value for the `satir` route.

```
2.[student@workstation ~]$ oc annotate route satir \
3.  router.openshift.io/cookie_name="hello"
```

`route.route.openshift.io/satir` annotated

4. Use the `curl` command to access the `satir` route. The output states the name of the pod that is servicing the request. Notice that the connection is load-balanced between the three replica pods.

```
5.[student@workstation ~]$ for i in {1..10}; do \
6.  curl satir-web-applications.apps.ocp4.example.com; done
7.Welcome to Red Hat Training, from satir-app-787b7d7858-bdfsh
8.Welcome to Red Hat Training, from satir-app-787b7d7858-gz8xm
9.Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
```

...output omitted...

10. Use the `curl` command to save the `hello` cookie to the `/tmp/cookie_jar` file.

```
11. [student@workstation ~]$ curl satir-web-applications.apps.ocp4.example.com \
12.   -c /tmp/cookie_jar
```

```
Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
```

13. Confirm that the `hello` cookie exists in the `/tmp/cookie_jar` file.

```
14. [student@workstation ~]$ cat /tmp/cookie_jar
15. ...output omitted...
```

```
#HttpOnly_satir-web-applications.apps.ocp4.example.com FALSE / F
ALSE 0 hello b7dd73d32003e513a072e25a32b6c881
```

16. The `hello` cookie provides session stickiness for connections to the `satir` route. Use the `curl` command and the `hello` cookie in the `/tmp/cookie_jar` file to connect to the `satir` route again. Confirm that you are connected to the same pod that handled the request in the previous step.

```
17. [student@workstation ~]$ for i in {1..10}; do \
18.   curl satir-web-applications.apps.ocp4.example.com -b /tmp/cookie_jar; don
   e
19. Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
20. Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
21. Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
```

```
...output omitted...
```

22. Use the `curl` command to connect to the `satir` route without the `hello` cookie. Observe that session stickiness occurs only with the cookie.

```
23. [student@workstation ~]$ for i in {1..10}; do \
24.   curl satir-web-applications.apps.ocp4.example.com; done
25. Welcome to Red Hat Training, from satir-app-787b7d7858-gz8xm
26. Welcome to Red Hat Training, from satir-app-787b7d7858-q7bhj
27. Welcome to Red Hat Training, from satir-app-787b7d7858-bdfsh
```

```
...output omitted...
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish deploy-routes
```

Lab: Deploy Managed and Networked Applications on Kubernetes

Deploy a database server and a web application that connects to that database, and expose the web application to external access.

Outcomes

- Deploy a MySQL database from a container image.
- Deploy a web application from a container image.
- Configure environment variables for a deployment.
- Expose the web application for external access.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the cluster is accessible and that all exercise resources are available. It also creates the `database-applications` project.

```
[student@workstation ~]$ lab start deploy-review
```

Instructions

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

Log in to the OpenShift cluster as the `developer` user with `developer` as the password.

Use the database-applications project for your work.

1. Log in to the OpenShift cluster, and change to the database-applications project.
1. Log in to the OpenShift cluster as the developer user with developer as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful
```

...output omitted...

5. Change to the database-applications project.

```
6. [student@workstation ~]$ oc project database-applications
7. Now using project "database-applications" on server "https://api.ocp4.example.com:6443".
```

...output omitted...

2. Create a MySQL database deployment named `mysql-app` by using the `registry.ocp4.example.com:8443/redhattraining/mysql-app:v1` image, and identify the root cause of the failure.
1. Create the MySQL database deployment.

```
2. [student@workstation ~]$ oc create deployment mysql-app \
3. --image registry.ocp4.example.com:8443/redhattraining/mysql-app:v1
```

`deployment.apps/mysql-app created`

4. Verify the deployment status. The pod name might differ in your output.

```
5. [student@workstation ~]$ oc get pods
6. NAME                                READY  STATUS  ...
7. mysql-app-75dfd58f99-5xfqc         0/1    Error   ...
8. [student@workstation ~]$ oc status
9. ...output omitted...
10. Errors:
11. pod/mysql-app-75dfd58f99-5xfqc is crash-looping
```


12.

1 error, 1 info identified, use 'oc status --suggest' to see details.

13. Identify the root cause of the deployment failure.

14. [student@workstation ~]\$ **oc logs mysql-app-75dfd58f99-5xfqc**

15. ...output omitted...

16. You must either specify the following environment variables:

17. **MYSQL_USER** (regex: '^[a-zA-Z0-9_]+\$')

18. **MYSQL_PASSWORD** (regex: '^[a-zA-Z0-9_~!@#\$\$%^&*()-=<>,.?;:|]+\$')

19. **MYSQL_DATABASE** (regex: '^[a-zA-Z0-9_]+\$')

20. Or the following environment variable:

21. **MYSQL_ROOT_PASSWORD** (regex: '^[a-zA-Z0-9_~!@#\$\$%^&*()-=<>,.?;:|]+\$')

...output omitted...

3. Configure the environment variables for the mysql-app deployment by using the following information:

Field	Value
MYSQL_USER	redhat
MYSQL_PASSWORD	redhat123
MYSQL_DATABASE	world_x

4. Then, execute the following command in the mysql-app deployment pod to load the world_x database:

5. **/bin/bash -c "mysql -uredhat -predhat123 </tmp/world_x.sql"**

1. Update the environment variables for the mysql-app deployment.

2. [student@workstation ~]\$ **oc set env deployment/mysql-app **

3. **MYSQL_USER=redhat MYSQL_PASSWORD=redhat123 MYSQL_DATABASE=world_x**

```
deployment.apps/mysql-app updated
```

4. Verify that the `mysql-app` application pod is in the `RUNNING` state. The pod name might differ in your output.

```
5. [student@workstation ~]$ oc get pods
```

```
6. NAME                                READY   STATUS    ...
```

```
mysql-app-57c44f646-5qt2k    1/1     Running   ...
```

7. Load the `world_x` database.

```
8. [student@workstation ~]$ oc exec -it mysql-app-57c44f646-5qt2k \
```

```
9.   -- /bin/bash -c "mysql -uredhat -predhat123 </tmp/world_x.sql"
```

```
10. mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
[student@workstation ~]$
```

11. Confirm that you can access the MySQL database.

```
[student@workstation ~]$ oc rsh mysql-app-57c44f646-5qt2k
```

```
sh-4.4$ mysql -uredhat -predhat123 world_x
```

```
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
Welcome to the MySQL monitor. Commands end with ; or \g.
```

```
...output omitted...
```

```
mysql>
```

12. Exit the MySQL database, and then exit the container.

```
13. mysql> exit
```

```
14. Bye
```

```
15. sh-4.4$ exit
```

```
exit
```

6. Create a service for the `mysql-app` deployment by using the following information:

Field	Value
Name	<code>mysql-service</code>
Port	<code>3306</code>
Target port	<code>3306</code>

1. Expose the `mysql-app` deployment.

```
2. [student@workstation ~]$ oc expose deployment mysql-app --name mysql-service \
3.   --port 3306 --target-port 3306
```

```
service/mysql-service created
```

4. Verify the service configuration. The endpoint IP address might differ in your output.

```
5. [student@workstation ~]$ oc get services
6. NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
7. mysql-service       ClusterIP     172.30.146.213 <none>         3306/TCP   10s
8. [student@workstation ~]$ oc get endpoints
9. NAME                ENDPOINTS          AGE
```

```
mysql-service  10.8.0.102:3306  19s
```

7. Create a web application deployment named `php-app` by using the `registry.ocp4.example.com:8443/redhattraining/php-webapp:v1` image.

1. Create the web application deployment.

```
2. [student@workstation ~]$ oc create deployment php-app \
3.   --image registry.ocp4.example.com:8443/redhattraining/php-webapp:v1
```

```
deployment.apps/php-app created
```

4. Verify the deployment status. Verify that the php-app application pod is in the RUNNING state.

```
5. [student@workstation ~]$ oc get pods
```

```
6. NAME                READY  STATUS   ...
```

```
7. php-app-725...      1/1    Running  ...
```

```
mysql-app-57c... 1/1    Running  ...
```

```
[student@workstation ~]$ oc status
```

```
...output omitted...
```

```
deployment/php-app deploys registry.ocp4.example.com:8443/redhattraining/ph  
p-webapp:v1
```

```
deployment #1 running for about a minute - 1 pod
```

```
...output omitted...
```

8. Create a service for the php-app deployment by using the following information:

Field	Value
Name	php-svc
Port	8080
Target port	8080

9. Then, create a route named phpapp to expose the web application to external access.

1. Expose the php-app deployment.

```
2. [student@workstation ~]$ oc expose deployment php-app --name php-svc \
```

```
3. --port 8080 --target-port 8080
```

```
service/php-svc exposed
```

4. Verify the service configuration. The endpoint IP address might differ in your output.

```
5. [student@workstation ~]$ oc get services
```

6. NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
7. mysql-service	ClusterIP	172.30.146.213	<none>	3306/TCP	7m47s
8. php-svc	ClusterIP	172.30.228.80	<none>	8080/TCP	4m34s

```
9. [student@workstation ~]$ oc get endpoints
```

10. NAME	ENDPOINTS	AGE
11. mysql-service	10.8.0.102:3306	7m50s

php-svc	10.8.0.107:8080	4m37s
----------------	-----------------	-------

12. Expose the php-svc service.

```
13. [student@workstation ~]$ oc expose service/php-svc --name phpapp
```

```
route.route.openshift.io/phpapp exposed
```

```
[student@workstation ~]$ oc get routes
```

NAME	HOST/PORT	...
phpapp	phpapp-database-applications.apps.ocp4.example.com	...

10. Test the connectivity between the web application and the MySQL database. In a web browser, go to the phpapp-database-applications.apps.ocp4.example.com route, and verify that the application retrieves data from the MySQL database.

1. Go to the phpapp-database-applications.apps.ocp4.example.com route in the web browser.

Connected successfully

1	Kabul	AFG Kabul	{"Population": 1780000}
2	Qandahar	AFG Qandahar	{"Population": 237500}
3	Herat	AFG Herat	{"Population": 186800}
4	Mazar-e-Sharif	AFG Balkh	{"Population": 127800}
5	Amsterdam	NLD Noord-Holland	{"Population": 731200}
6	Rotterdam	NLD Zuid-Holland	{"Population": 593321}
7	Haag	NLD Zuid-Holland	{"Population": 440900}
8	Utrecht	NLD Utrecht	{"Population": 234323}
9	Eindhoven	NLD Noord-Brabant	{"Population": 201843}
10	Tilburg	NLD Noord-Brabant	{"Population": 193238}
11	Groningen	NLD Groningen	{"Population": 172701}
12	Breda	NLD Noord-Brabant	{"Population": 160398}
13	Apeldoorn	NLD Gelderland	{"Population": 153491}

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade deploy-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish deploy-review
```

Summary

- Many resources in Kubernetes and RHOCp create or affect pods.
- Resources are created imperatively or declaratively. The imperative strategy instructs the cluster what to do. The declarative strategy defines the state that the cluster matches.

- The `oc new-app` command creates resources that are determined via heuristics.
- A deployment is the main way to deploy an application in RHOCP.
- The workload API includes several resources to create pods. The choice between resources depends on for how long and how often the pod needs to run.
- A job resource executes a one-time task on the cluster via a pod. The cluster retries the job until it succeeds, or it retries a specified number of attempts.
- Resources are organized into projects and are selected via labels.
- A route connects a public-facing IP address and a DNS hostname to an internal-facing service IP address. Services provide network access between pods, whereas routes provide network access to pods from users and applications outside the RHOCP cluster.

Chapter 5. Manage Storage for Application Configuration and Data

Externalize the Configuration of Applications

Guided Exercise: Externalize the Configuration of Applications

Provision Persistent Data Volumes

Guided Exercise: Provision Persistent Data Volumes

Selecting a Storage Class for an Application

Guided Exercise: Selecting a Storage Class for an Application

Manage Non-shared Storage with Stateful Sets

Guided Exercise: Manage Non-shared Storage with Stateful Sets

Lab: Manage Storage for Application Configuration and Data

Summary

Abstract

Goal	Externalize application configurations in Kubernetes resources and provision storage volumes for persistent data files.
-------------	---

Sections	• Externalize the Configuration of Applications (and Guided Exercise) • Provision Persistent Data Volumes (and Guided Exercise) • Selecting a Storage Class for an Application (and Guided Exercise) • Manage Non-shared Storage with Stateful Sets (and Guided Exercise)
Lab	• Manage Storage for Application Configuration and Data

Externalize the Configuration of Applications

Objectives

- Configure applications by using Kubernetes secrets and configuration maps to initialize environment variables and to provide text and binary configuration files.

Configuring Kubernetes Applications

When an application is run in Kubernetes with a pre-existing image, the application uses the default configuration. This action is valid for testing purposes. However, for production environments, you might need to customize your applications before deploying them.

With Kubernetes, you can use manifests in JSON and YAML formats to specify the intended configuration for each application. You can define the name of the application, labels, the image source, storage, environment variables, and more.

The following snippet shows an example of a YAML manifest file of a deployment:

```
apiVersion: apps/v1

kind: Deployment

metadata:
  name: hello-deployment

spec:
  replicas: 1
```



```
selector:
  matchLabels:
    app: hello-deployment
template:
  metadata:
    labels:
      app: hello-deployment

  spec:
    containers:

    - env:
      - name: ENV_VARIABLE_1
        valueFrom:
          secretKeyRef:
            key: hello
            name: world

    image: quay.io/hello-image:latest
```

API version of the resource.

Deployment resource type.

In this section, you specify the metadata of your application, such as the name.

You can define the general configuration of the resource that is applied to the deployment, such as the number of replicas (pods), the selector label, and the template data.

In this section, you specify the configuration for your application, such as the image name, the container name, ports, environment variables, and more.

You can define the environment variables to configure your application needs.

Sometimes, your application requires using a combination of files. For example, at the time of creation, a database deployment must have preloaded databases and data. You most commonly configure applications by using environment variables, external files, or command-line arguments. This process of configuration externalization ensures that the application is portable across environments when the container image, external files, and environment variables are available in the environment where the application runs.

Kubernetes provides a mechanism to externalize the configuration of your applications by using configuration maps and secrets.

You can use configuration maps to inject containers with configuration data. The *ConfigMap* (configuration map) namespaced objects provide ways to inject configuration data into containers, which helps to maintain platform independence of the containers. These objects can store fine-grained information, such as individual properties, or coarse-grained information, such as entire configuration files or JSON blobs (JSON sections). The information in configuration maps does not require protection.

The following listing shows an example of a configuration map:

```
apiVersion: v1

kind: ConfigMap

metadata:
  name: example-configmap
  namespace: my-app

data:
  example.property.1: hello
  example.property.2: world
  example.property.file: |-
    property.1=value-1
    property.2=value-2
    property.3=value-3

binaryData:
  bar: R0lGODlhAgACAIABA04AAP///yH5BAEHAAEALAAAAACAAIAAAIDRAIFADs=
```

ConfigMap resource type.

Contains the configuration data.

Points to an encoded file in base64 that contains non-UTF-8 data, for example, a binary Java keystore file. Place a key followed by the encoded file.

Applications often require access to sensitive information. For example, a back-end web application requires access to database credentials to query a database. Kubernetes and OpenShift use secrets to hold sensitive information. For example, you can use secrets to store the following types of sensitive information:

- Passwords
- Sensitive configuration files
- Credentials to an external resource, such as an SSH key or OAuth token

The following listing shows an example of a secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: example-secret
  namespace: my-app

type: Opaque

data:
  username: bXl1c2VyCg==
  password: bXlQQDU1Cg==

stringData:
  hostname: myapp.mydomain.com
  secret.properties: |
    property1=valueA
    property2=valueB
```

Specifies the type of secret.

Specifies the encoded string and data.

Specifies the decoded string and data.

A secret is a namespaced object and it can store any type of data. Data in a secret is Base64-encoded, and is not stored in plain text. Secret data is not encrypted; you can decode the secret from Base64 format to access the original data. The following

example shows the decoded values for the username and password objects from the example-secret secret:

```
[user@host ~]$ echo bX11c2VyCg== | base64 --decode
myuser
[user@host ~]$ echo bX1QQDU1Cg== | base64 --decode
myP@55
```

Kubernetes and OpenShift support the following types of secrets:

- Opaque secrets: An opaque secret store key and value pairs that contain arbitrary values, and are not validated to conform to any convention for key names or values.
- Service account tokens: Store a token credential for applications that authenticate to the Kubernetes API.
- Basic authentication secrets: Store the needed credentials for basic authentication. The data parameter of the secret object must contain the user and the password keys that are encoded in the Base64 format.
- SSH keys: Store data that is used for SSH authentication.
- TLS certificates: Store a certificate and a key that are used for TLS.
- Docker configuration secrets: Store the credentials for accessing a container image registry.

When you store information in a specific secret resource type, Kubernetes validates that the data conforms to the type of secret.

NOTE

By default, configuration maps and secrets are not encrypted. To encrypt your secret data at rest, you must encrypt the Etcd database. When enabled, Etcd encrypts the following resources: secrets, configuration maps, routes, OAuth access tokens, and OAuth authorization tokens. Encrypting the Etcd database is outside the scope of the course.

Creating Secrets

If a pod requires access to sensitive information, then create a secret for the information before you deploy the pod. Both the `oc` and `kubectl` command-line

tools provide the `create secret` command. Use one of the following commands to create a secret:

- Create a generic secret that contains key-value pairs from literal values that are typed on the command line:

- `[user@host ~]$ oc create secret generic secret_name \`
- `--from-literal key1=secret1 \`

```
--from-literal key2=secret2
```

- Create a generic secret by using key names that are specified on the command line and values from files:

- `[user@host ~]$ kubectl create secret generic ssh-keys \`
- `--from-file id_rsa=/path-to/id_rsa \`

```
--from-file id_rsa.pub=/path-to/id_rsa.pub
```

- Create a TLS secret that specifies a certificate and the associated key:

- `[user@host ~]$ oc create secret tls secret-tls \`

```
--cert /path-to-certificate --key /path-to-key
```

To create an opaque secret from the web console, click the **Workloads** → **Secrets** menu. Click **Create** and select **Key/value secret**. Complete the form with the key name, and specify the value by writing it in the following section, or by extracting it from a file.

Create key/value secret

Key/value secrets let you inject sensitive data into your application as files or environment variables.

Secret name *

Unique name of the new secret.

Key *

Value

Browse...

Drag and drop file with your value here or browse to upload it.

To create a secret from the web console that stores the credentials for accessing a container image registry, click the **Workloads** → **Secrets** menu. Click **Create** and select **Image pull secret**. Complete the form or upload a configuration file with the secret name, select the authentication type, and add the registry server address, the username, password, and email credentials.

Create image pull secret

Image pull secrets let you authenticate against a private image registry.

Secret name *

Unique name of the new secret.

Authentication type

Registry server address *

For example quay.io or docker.io

Username *

Password *

Email

Creating Configuration Maps

The syntax for creating a configuration map and for creating a secret closely match. You can enter key-value pairs on the command line, or use the content of a file as the value of a specified key. You can use either the `oc` or `kubectl` command-line tools to create a configuration map. The following command shows how to create a configuration map:

```
[user@host ~]$ kubectl create configmap my-config \
--from-literal key1=config1 --from-literal key2=config2
```

You can also use the `cm` shorthand to create a configuration map.

```
[user@host ~]$ oc create cm my-config \
--from-literal key1=config1 --from-literal key2=config2
```

To create a configuration map from the web console, click the **Workloads** → **ConfigMaps** menu. Click **Create ConfigMap** and complete the configuration map by using the form view or the YAML view.

The screenshot shows the 'Create ConfigMap' form in the OpenShift web console. The form is titled 'Name' with a red asterisk, indicating it is a required field. The 'Name' field contains the text 'database'. Below the field is a description: 'A unique name for the ConfigMap within the project'. There is a checkbox labeled 'Immutable' which is currently unchecked. Below the checkbox is a description: 'Immutable, if set to true, ensures that data stored in the ConfigMap cannot be updated'. The 'Data' section is titled 'Data' and has a description: 'Data contains the configuration data that is in UTF-8 range'. To the right of the 'Data' section is a blue link with a minus icon labeled 'Remove key/value'. The 'Key' section is titled 'Key' with a red asterisk. The 'Key' field contains the text 'database'. The 'Value' section is titled 'Value'. It has a large text area for the value, which currently contains the text 'countries'. To the right of the text area is a 'Browse...' button. Below the text area is a description: 'Drag and drop file with your value here or browse to upload it.' At the bottom left of the form is a blue link with a plus icon labeled 'Add key/value'. At the bottom of the form are two buttons: 'Create' and 'Cancel'.

Name *

database

A unique name for the ConfigMap within the project

☐ Immutable

Immutable, if set to true, ensures that data stored in the ConfigMap cannot be updated

Data

Data contains the configuration data that is in UTF-8 range

[- Remove key/value](#)

Key *

database

Value

Browse...

Drag and drop file with your value here or browse to upload it.

countries

[+ Add key/value](#)

Create Cancel

You can use files on each key that you add by clicking **Browse** beside the **Value** field. The **Key** field must be the name of the added file in the **Value** field.

Data
Data contains the configuration data that is in UTF-8 range

Remove key/value

Key *
index.html

Value
index.html

Browse...

Drag and drop file with your value here or browse to upload it.
Click me!<a>

NOTE

Use a binary data key instead of a data key if the file uses the binary format, such as a PNG file.

Using Configuration Maps and Secrets to Initialize Environment Variables

You can use configuration maps to populate individual environment variables that configure your application. Unlike secrets, the information in configuration maps does not require protection. The following listing shows an initialization example of environment variables:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: config-map-example

  namespace: example-app
data:

  database.name: sakila

  database.user: redhat
```

The project where the configuration map resides. ConfigMap objects can be referenced only by pods in the same project.

Initializes the `database.name` variable to the `sakila` value.

Initializes the `database.user` variable to the `redhat` value.

You can then use the configuration map to populate environment variables for your application. The following example shows a pod resource that populates specific environment variables by using a configuration map.

```
apiVersion: v1
kind: Pod
metadata:
  name: config-map-example-pod
  namespace: example-app
spec:
  containers:
    - name: example-container
      image: registry.example.com/mysql-80:1-237
      command: [ "/bin/sh", "-c", "env" ]

      env:
        - name: MYSQL_DATABASE
          valueFrom:
            configMapKeyRef:
              name: config-map-example
              key: database.name
        - name: MYSQL_USER
          valueFrom:
            configMapKeyRef:
              name: config-map-example
```

```
key: database.user
```

```
optional: true
```

The attribute to specify environment variables for the pod.

The name of a pod environment variable where you are populating a key's value.

Name of the ConfigMap object to pull the environment variables from.

The environment variable to pull from the ConfigMap object.

Sets the environment variable as optional. The pod is started even if the specified ConfigMap object and keys do not exist.

The following example shows a pod resource that injects all environment variables from a configuration map:

```
apiVersion: v1
kind: Pod
metadata:
  name: config-map-example-pod2
  namespace: example-app
spec:
  containers:
    - name: example-container
      image: registry.example.com/mysql-80:1-237
      command: [ "/bin/sh", "-c", "env" ]

  envFrom:
    - configMapRef:

      name: config-map-example
  restartPolicy: Never
```

The attribute to pull all environment variables from a ConfigMap object.

The name of the ConfigMap object to pull environment variables from.

You can use secrets with other Kubernetes resources such as pods, deployments, builds, and more. You can specify secret keys or volumes with a mount path to store your secrets. The following snippet shows an example of a pod that populates environment variables with data from the test-secret Kubernetes secret:

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-example-pod
spec:
  containers:
    - name: secret-test-container
      image: busybox
      command: [ "/bin/sh", "-c", "export" ]

      env:
        - name: TEST_SECRET_USERNAME_ENV_VAR

          valueFrom:

            secretKeyRef:

              name: test-secret

              key: username
```

Specifies the environment variables for the pod.

Indicates the source of the environment variables.

The `secretKeyRef` source object of the environment variables.

Name of the secret, which must exist.

The key that is extracted from the secret is the username for authentication.

In contrast with configuration maps, the values in secrets are always encoded (not encrypted), and their access is restricted to fewer authorized users.

Using Secrets and Configuration Maps as Volumes

To expose a secret to a pod, you must first create the secret in the same namespace, or project, as the pod. In the secret, assign each piece of sensitive data to a key. After creation, the secret contains key-value pairs.

The following command creates a generic secret that contains key-value pairs from literal values that are typed on the command line: `user` with the `demo-user` value, and `root_password` with the `zT1kTgk` value.

```
[user@host ~]$ oc create secret generic demo-secret \
--from-literal user=demo-user \
--from-literal root_password=zT1kTgk
```

You can also create a generic secret by specifying key names on the command line and values from files:

```
[user@host ~]$ oc create secret generic demo-secret \
--from-file user=/tmp/demo/user \
--from-file root_password=/tmp/demo/root_password
```

You can mount a secret to a directory within a pod. Kubernetes creates a file for each key in the secret that uses the name of the key. The content of each file is the decoded value of the secret. The following command shows how to mount secrets in a pod:

```
[user@host ~]$ oc set volume deployment/demo \
--add --type secret \
--secret-name demo-secret \
--mount-path /app-secrets
```

Modify the volume configuration in the demo deployment.

Add a new volume from a secret.

Use the demo-secret secret.

Make the secret data available in the /app-secrets directory in the pod. The content of the /app-secrets/user file is demo-user. The content of the /app-secrets/root_password file is zT1KTgk.

To assign a secret as a volume to a deployment from the web console, list the available secrets from the **Workloads** → **Secrets** menu.

Secrets

Create

Filter

Name

Search by name... /

Name	Type	Si...	Created
 builder-dockercfg-xpbl8	kubernetes.io/dockercfg	1	 Oct 16, 2023, 4:27 AM
 builder-token-cwxj4	kubernetes.io/service-account-token	4	 Oct 16, 2023, 4:27 AM
 controller-certs-secret	kubernetes.io/tls	2	 Oct 16, 2023, 4:28 AM
 controller-dockercfg-zm946	kubernetes.io/dockercfg	1	 Oct 16, 2023, 4:27 AM
 controller-token-72r9n	kubernetes.io/service-account-token	4	 Oct 16, 2023, 4:27 AM
 default-dockercfg-fzq8c	kubernetes.io/dockercfg	1	 Oct 16, 2023, 4:27 AM
 default-token-w2684	kubernetes.io/service-account-token	4	 Oct 16, 2023, 4:27 AM
 deployer-dockercfg-4bq4z	kubernetes.io/dockercfg	1	 Oct 16, 2023, 4:27 AM
 deployer-token-ppbsh	kubernetes.io/service-account-token	4	 Oct 16, 2023, 4:27 AM

Select a secret and click **Add Secret to workload**.

Project: metallb-system

Secrets > Secret details

 builder-dockercfg-xpbl8

Add Secret to workload

Actions

Details

YAML

Secret details

Name	Type
builder-dockercfg-xpbl8	kubernetes.io/dockercfg

Select the workload, choose the **Volume** option, and define the mount path for the secret.

Add secret to workload

Add all values from  builder-dockercfg-xpbl8 to a workload as environment variables or a volume.

Add this secret to workload *

 controller ▼

Add secret as *

☐ Environment variables

☒ Volume

Mount path *

/etc/temp

Cancel

Save

Similar to secrets, you must first create a configuration map before a pod can consume it. The configuration map must exist in the same namespace, or project, as the pod. The following command shows how to create a configuration map from an external configuration file:

```
[user@host ~]$ oc create configmap demo-map \
--from-file=config-files/httpd.conf
```

You can similarly add a configuration map as a volume by using the following command:

```
[user@host ~]$ oc set volume deployment/demo \  
--add --type configmap \  
--configmap-name demo-map \  
--mount-path /app-secrets
```

To confirm that the volume is attached to the deployment, use the following command:

```
[user@host ~]$ oc set volume deployment/demo  
demo  
configMap/demo-map as volume-du9in  
mounted at /app-secrets
```

You can also use the `oc set env` command to set application environment variables from either secrets or configuration maps. In some cases, you can modify the names of the keys to match the names of environment variables by using the `--prefix` option. In the following example, the `user` key from the `demo-secret` secret sets the `MYSQL_USER` environment variable, and the `root_password` key from the `demo-secret` secret sets the `MYSQL_ROOT_PASSWORD` environment variable. If the key name from the secret is lowercase, then the corresponding environment variable is converted to uppercase to match the pattern that the `--prefix` option defines.

```
[user@host ~]$ oc set env deployment/demo \  
--from secret/demo-secret --prefix MYSQL_
```

NOTE

You cannot assign configuration maps by using the web console.

Updating Secrets and Configuration Maps

Secrets and configuration maps occasionally require updates. OpenShift provides the `oc extract` command to ensure that you have the latest data. You can save the data to a specific directory by using the `--to` option. Each key in the secret or configuration map creates a file with the same name as the key. The content of each file is the value of the associated key. If you run the `oc extract` command more than one time, then you must use the `--confirm` option to overwrite the

existing files. You can also use the `--confirm` option to create the target directory for the extracted content.

```
[user@host ~]$ oc extract secret/demo-secrets -n demo \
--to /tmp/demo --confirm
[user@host ~]$ ls /tmp/demo/
user  root_password
[user@host ~]$ cat /tmp/demo/root_password
zT1KTgk
[user@host ~]$ echo k8qhcw3m0 > /tmp/demo/root_password
```

After updating the locally saved files, use the `oc set data` command to update the secret or configuration map. For each key that requires an update, specify the name of a key and the associated value. If a file contains the value, then use the `--from-file` option.

```
[user@host ~]$ oc set data secret/demo-secrets -n demo \
--from-file /tmp/demo/root_password
```

You must restart pods that use environment variables for the pods to read the updated secret or configuration map. Pods that use a volume mount to reference secrets or configuration maps receive the updates without a restart by using an eventually consistent approach. By default, the kubelet agent watches for changes to the keys and values that are used in volumes for pods on the node.

The kubelet agent detects changes and propagates the changes to the pods to keep volume data consistent. Despite the automatic updates that Kubernetes provides, a restart of the pod is still required if the software reads configuration data only at startup time.

Deleting Secrets and Configuration Maps

Similar to other Kubernetes resources, you can use the `delete` command to delete secrets and configuration maps that are no longer needed or in use.

```
[user@host ~]$ kubectl delete secret/demo-secrets -n demo
[user@host ~]$ oc delete configmap/demo-map -n demo
```

Guided Exercise: Externalize the Configuration of Applications

Deploy a web server that takes configuration files from a configuration map.

Outcomes

- Create a web application deployment.
- Expose the web application deployment to external access.
- Create a configuration map from two files.
- Mount the configuration map in the web application deployment.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start storage-configs
```

Instructions

1. Log in to the OpenShift web console as the developer user and switch to the **Administrator** perspective.
1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
2. Click **Red Hat Identity Management** and log in as the developer user with developer as the password. If the **Welcome to the Developer Perspective** message is displayed, then click **Skip tour**.
3. From the perspective switcher, select **Administrator**.
2. Create a web application deployment named `webconfig` from the web console. Use the `registry.ocp4.example.com:8443/ubi9/httpd-24:latest` container image.
1. Go to **Workloads** → **Deployments**.
2. Set the project to `storage-configs` and click **Create Deployment**.
3. Use `webconfig` for the deployment name and change the image name to `registry.ocp4.example.com:8443/ubi9/httpd-24:latest`.

Images

Container:  container

☐ Deploy image from an image stream tag

Image Name *

registry.ocp4.example.com:8443/ubi9/httpd-24:latest

Container image name

› [Show advanced image options](#)

4. Click **Create**. Wait for the blue circle to indicate that three pods are running.
3. Expose the web application to external access from the web console. Create a service and a route for the web application. Use the values from the following tables to create the service and the route.

Service field	Service value
Service name	webconfig-svc
App selector	webconfig
Port number	8080
Target port	8080
Route field	Route value

Service field	Service value
Route name	webconfig-rt
Service name	webconfig-svc
Target port	8080

1. Go to **Networking** → **Services** and click **Create Service**.
2. Update the service values from the service table.

Create Service

Create by manually entering YAML or JSON definitions, or by

Alt + F1 Accessibility

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: webconfig-svc
5    namespace: storage-configs
6  spec:
7    selector:
8      app: webconfig
9    ports:
10     - protocol: TCP
11       port: 8080
12       targetPort: 8080
13
```

3. Click **Create**.

Alternatively, you can create the service from the CLI by using the following command:

```
[student@workstation ~]$ oc expose deploy/webconfig \
  --namespace storage-configs --name webconfig-svc \
  --selector app=webconfig \
  --target-port 8080 --port 8080
```

4. Go to **Networking** → **Routes** and click **Create Route**.
5. Create the route by using the values from the route table:

Project: storage-configs ▼

Create Route

Routing is a way to make your application publicly visible

Configure via: ☒ Form view ☐ YAML view

Name *

webconfig-rt

A unique name for the Route within the project

Hostname

Public hostname for the Route. If not specified, a hostname is generated.

Path

Path that the router watches to route traffic to the service.

Service *

 webconfig-svc

Service to route to.

Service weight

A number between 0 and 255 that depicts relative weight compared with other targets.

Target port *

8080 → 8080 (TCP)

Target port for traffic

6. Click **Create**.

Alternatively, you can create the route from the CLI by using the following command:

```
[student@workstation ~]$ oc expose svc/webconfig-svc \
```

```
--namespace storage-configs --name webconfig-rt
```

7. On the **Details** page of the route, click the **Location** link.

Project: storage-configs ▼

Routes > Route details

RT webconfig-rt Actions ▼

Details YAML

Route details

Name webconfig-rt	Location http://webconfig-rt-storage-configs.apps.ocp4.example.com/  
Namespace NS storage-configs	Status  Accepted
Labels No labels Edit 	Host webconfig-rt-storage-configs.apps.ocp4.example.com
Annotations 1 annotation 	Path /
Service S webconfig-svc	Router canonical hostname router-default.apps.ocp4.example.com
Target port 8080	

8. The route link opens a browser tab that displays the default test page of the web server. Do not close the browser tab, because you use it in another step.



Red Hat Enterprise Linux Test Page

This page is used to test the proper operation of the HTTP server after it has been installed. If you can read this page, it means that the HTTP server installed at this site is working properly.

- 9.
4. Create a configuration map that contains the files for the web application by using the web console.
 1. Go to **Workloads** → **ConfigMaps** and click **Create ConfigMap**.
 2. Set the name of the configuration map to `webfiles`.
 3. Add a data key. In the **Data** section, define the `index.html` name as the **Key** value, and click **Browse** in the **Value** field to select the `/home/student/D0180/labs/storage-configs/index.html` file.

Project: storage-configs ▼

Name *

webfiles

A unique name for the ConfigMap within the project

☐ Immutable

Immutable, if set to true, ensures that data stored in the ConfigMap cannot be updated

Data

[Remove key/value](#)

Key *

index.html

Value

index.html [Browse...](#)

Drag and drop file with your value here or browse to upload it.

```
<a href="redhatlogo.png">Click me!<a>
```

4. Add a binary data key. In the **Binary Data** section, click **Add key/value**.
5. Define the redhatlogo.png name as the **Key** value, and click **Browse** in the **Value** field to select the /home/student/D0180/labs/storage-configs/redhatlogo.png file.

Binary Data

➔ Remove key/value

Key *

redhatlogo.png

Value

redhatlogo.png Browse...

Drag and drop file with your value here or browse to upload it.

i Non-printable file detected.

File contains non-printable characters. Preview is not available.

➕ Add key/value

BinaryData contains the binary data that is not in UTF-8 range

Create Cancel

6. Click **Create**.
5. Mount the webfiles configuration map as a volume in the webconfig deployment by using the command line.
1. In a terminal window, log in to the RHOCP cluster as the developer user with developer as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Select the storage-configs project.

```
6. [student@workstation ~]$ oc project storage-configs
```

Now using project "storage-configs" on server "https://api.ocp4.example.com:6443".

7. Mount the webfiles configuration map as a volume.

```
8. [student@workstation ~]$ oc set volume deployment/webconfig \
9. --add --type configmap --configmap-name webfiles \
```

```
10. --name webfiles-vol --mount-path /var/www/html/
```

```
deployment.apps/webconfig volume updated
```

11. Verify that the deployment has three available replicas.

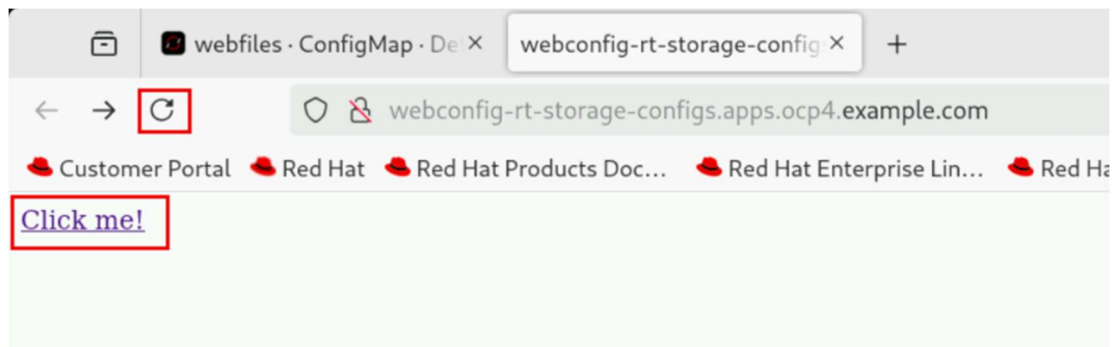
```
12. [student@workstation ~]$ oc get deployment
```

```
13. NAME          READY    UP-TO-DATE    AVAILABLE    AGE
```

```
webconfig    3/3      3             3            3h20m
```

6. Verify that the web application shows the content from the configuration map.

1. Switch to the web browser, and go to the web server tab. Click the reload icon in the web browser.



2. Click the **Click me!** link that the web page displays. The web page shows the Red Hat logo.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish storage-configs
```

Provision Persistent Data Volumes

Objectives

- Provide applications with persistent storage volumes for block and file-based data.

Kubernetes Persistent Storage

Containers have ephemeral storage by default. The lifetime of this ephemeral storage does not extend beyond the life of the individual pod, and this ephemeral storage cannot be shared across pods. When a container is deleted, all the files and data inside it are also deleted. To preserve the files, containers use persistent storage volumes.

Because OpenShift Container Platform uses the Kubernetes persistent volume (PV) framework, cluster administrators can provision persistent storage for a cluster. Developers can use persistent volume claims (PVCs) to request PV resources without specific knowledge of the underlying storage infrastructure.

Two ways exist to provision storage for the cluster: static and dynamic. Static provisioning requires the cluster administrator to create persistent volumes manually. Dynamic provisioning uses storage classes to create the persistent volumes on demand.

Administrators can use storage classes to provide persistent storage. Storage classes describe types of storage for the cluster. Cluster administrators create storage classes to manage storage services or storage tiers of a service. Rather than specifying provisioned storage, PVCs instead refer to a storage class.

Developers use PVCs to add persistent volumes to their applications. Developers need not know details of the storage infrastructure. With static provisioning, developers use previously created PVs, or ask a cluster administrator to manually create persistent volumes for their applications. With dynamic provisioning, developers declare the storage requirements of the application, and the cluster creates a PV to fill the request.

Persistent Volumes

Not all storage is equal. Storage types vary in cost, performance, and reliability. Multiple storage types are usually available for each Kubernetes cluster.

The following list of commonly used storage volume types and their use cases is not exhaustive.

configMap

The `configMap` volume externalizes the application configuration data. This use of the `configMap` resource ensures that the application configuration is portable across environments and can be version-controlled.

emptyDir

An `emptyDir` volume provides a per-pod directory for scratch data. The directory is usually empty after provisioning. `emptyDir` volumes are often required for ephemeral storage.

hostPath

A `hostPath` volume mounts a file or directory from the host node into your pod. To use a `hostPath` volume, the cluster administrator must configure pods to run as privileged. This configuration grants access to other pods in the same node.

Red Hat does not recommend the use of `hostPath` volumes in production. Instead, Red Hat supports `hostPath` mounting for development and testing on a single-node cluster. Although most pods do not need a `hostPath` volume, it does offer a quick option for testing if an application requires it.

iSCSI

Internet Small Computer System Interface (iSCSI) is an IP-based standard that provides block-level access to storage devices. With iSCSI volumes, Kubernetes workloads can consume persistent storage from iSCSI targets.

local

You can use `Local` persistent volumes to access local storage devices, such as a disk or partition, by using the standard PVC interface. `Local` volumes are subject to the availability of the underlying node, and are not suitable for all applications.

NFS

An NFS (Network File System) volume can be accessed from multiple pods at the same time, and thus provides shared data between pods. The NFS volume type is commonly used because of its ability to share data safely. Red Hat recommends to use NFS only for non-production systems.

Volume Access Mode

Persistent volume providers vary in capabilities. A volume uses access modes to specify the modes that it supports. For example, NFS can support multiple read/write clients, but a specific NFS PV might be exported on the server as read-only. OpenShift defines the following access modes, as summarized in the following table:

Table 5.1. Volume Access Modes

Access mode	Abbreviation	Description
ReadWriteOnce	RWO	A single node mounts the volume as read/write. Multiple pods on the node can access the volume.
ReadWriteOncePod	RWOP	A single pod in the cluster mounts the volume as read/write.
ReadOnlyMany	ROX	Many nodes mount the volume as read-only.
ReadWriteMany	RWX	Many nodes mount the volume as read/write.

Developers must select a volume type that supports the required access level by the application. The following table shows some example supported access modes:

Table 5.2. Access Mode Support

Volume type	RWO
configMap	Yes
emptyDir	Yes
hostPath	Yes
iSCSI	Yes
local	Yes

Volume type	RWO
NFS	Yes

Volume Modes

Kubernetes supports two volume modes for persistent volumes: `Filesystem` and `Block`. If the volume mode is not defined for a volume, then Kubernetes assigns the default volume mode, `Filesystem`, to the volume.

OpenShift Container Platform can provision raw block volumes. These volumes do not have a file system, and can provide performance benefits for applications that either write to the disk directly or that implement their own storage service. Raw block volumes are provisioned by specifying `volumeMode: Block` in the PV and PVC specification.

The following table provides examples of storage options with block volume support:

Table 5.3. Block Volume Support

Volume plug-in	Manually provisioned
AWS EBS	Yes
Azure disk	Yes
Cinder	Yes
Fibre channel	Yes
GCP	Yes
iSCSI	Yes
local	Yes
Red Hat OpenShift Data Foundation	Yes
VMware vSphere	Yes

Manually Creating a PV

Administrators use `PersistentVolume` manifest files to manually create persistent volumes. The following example creates a persistent volume from a fiber channel storage device that uses block mode.

```
apiVersion: v1

kind: PersistentVolume
metadata:

  name: block-pv
spec:
  capacity:

    storage: 10Gi
  accessModes:

    - ReadWriteOnce

  volumeMode: Block

  persistentVolumeReclaimPolicy: Retain

  fc:
    targetWWNs: ["50060e801049cfd1"]
    lun: 0
    readOnly: false
```

`PersistentVolume` is the resource type for PVs.

Provide a name for the PV, which subsequent claims use to access the PV.

Specify the amount of storage that is allocated to this volume.

The storage device must support the access mode that the PV specifies.

The `volumeMode` attribute is optional for `Filesystem` volumes, but is required for `Block` volumes.

The `persistentVolumeReclaimPolicy` determines how the cluster handles the PV when the PVC is deleted. Valid options are `Retain` or `Delete`.

The remaining attributes are specific to the storage type. In this example, the `fc` object specifies the Fiber Channel storage type attributes.

If the previous manifest is in a file named `my-fc-volume.yaml`, then log in as an administrator and run the following command to create the PV resource:

```
[user@host ~]$ oc create -f my-fc-volume.yaml
```

To create a persistent volume from the web console, log in as an administrator and go to **Storage** → **PersistentVolumes**.

Persistent Volume Claims

A persistent volume claim (PVC) resource represents a request from an application for storage. A PVC specifies the minimal storage characteristics, such as capacity and access mode. A PVC does not specify a storage technology, such as iSCSI or NFS.

The lifecycle of a PVC is not tied to a pod, but to a namespace. Multiple pods from the same namespace but with potentially different workload controllers can connect to the same PVC. You can also sequentially connect storage to and detach storage from different application pods, to initialize, convert, migrate, or back up data.

Kubernetes matches each PVC to a persistent volume (PV) resource that can satisfy the requirements of the claim. It is not an exact match. A PVC might be bound to a PV with a larger disk size than is requested. A PVC that specifies single access might be bound to a PV that is shareable for multiple concurrent accesses. Rather than enforcing policy, PVCs declare what an application needs, which Kubernetes provides on a best-effort basis.

Creating a PVC

A PVC belongs to a specific project. To create a PVC, developers must specify the access mode and size, among other options. A PVC cannot be shared between projects. Developers use a PVC to access a persistent volume (PV). Persistent volumes are not exclusive to projects, and are accessible across the entire

OpenShift cluster. When a PV binds to a PVC, the PV cannot be bound to another PVC.

To add a volume to your application deployment, create a `PersistentVolumeClaim` resource, and add it to the application as a volume. Create the PVC by using either a Kubernetes manifest or the `oc set volumes` command. In addition to either creating a PVC or using an existing PVC, the `oc set volumes` command can modify a deployment to mount the PVC as a volume within the pod.

To add a volume to an application deployment, developers can use the `oc set volumes` command:

```
[user@host ~]$ oc set volumes deployment/example-application \

--add \

--name example-pv-storage \

--type persistentVolumeClaim \

--claim-mode rwo \

--claim-size 15Gi \

--mount-path /var/lib/example-app \

--claim-name mypvc
```

Specify the name of the deployment that requires the PVC resource.

Setting the `add` option to `true` adds volumes and volume mounts for containers.

The `name` option specifies a volume name. If not specified, a name is autogenerated.

The supported types, for the `add` operation, include `emptyDir`, `hostPath`, `secret`, `configMap`, and `persistentVolumeClaim`. The `claim-mode` option defaults to `ReadWriteOnce`. The valid values are `ReadWriteOnce (RWO)`, `ReadWriteOncePod (RWOP)`, `ReadWriteMany (RWX)`, and `ReadOnlyMany (ROX)`.

Create a claim with the given size in bytes, if specified along with the persistent volume type. The size must use SI notation, for example, 15, 15 G, or 15 Gi.

The `mount-path` option specifies the mount path inside the container.

The `claim-name` option provides the name for the PVC, and is required for the `persistentVolumeClaim` type.

The command creates a PVC resource and adds it to the application as a volume within the pod.

The command updates the deployment for the application with `volumeMounts` and `volumes` specifications.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  ...output omitted...

  namespace: storage-volumes
  ...output omitted...
spec:
  ...output omitted...
  template:
    ...output omitted...
    spec:
      ...output omitted...
      volumeMounts:
        - mountPath: /var/lib/example-app

          name: example-pv-storage
      ...output omitted...
      volumes:
        - name: example-pv-storage
          persistentVolumeClaim:
```

```
    claimName: mypvc
...output omitted...
```

The deployment, which must be in the same namespace as the PVC.

The mount path in the container.

The volume name, which is used to specify the volume that is associated with the mount.

The claim name that is bound to the volume.

The following example specifies a PVC by using a YAML manifest to create a PersistentVolumeClaim API object:

```
---
apiVersion: v1

kind: PersistentVolumeClaim

metadata:

  name: mypvc
  labels:
    app: example-application
spec:
  accessModes:

    - ReadWriteOnce
  resources:
    requests:

      storage: 15Gi
```

PersistentVolumeClaim is the resource type for a PVC.

Use this name in the `claimName` field of the `persistentVolumeClaim` element in the `volumes` section of a deployment manifest.

Specify the access mode that this PVC requests. The storage class provisioner must provide this access mode. If persistent volumes are created statically, then an eligible persistent volume must provide this access mode.

The storage class creates a persistent volume that matches this size request. If persistent volumes are created statically, then an eligible persistent volume must be at least the requested size.

Developers can use the `oc create` command to create the PVC from the manifest file.

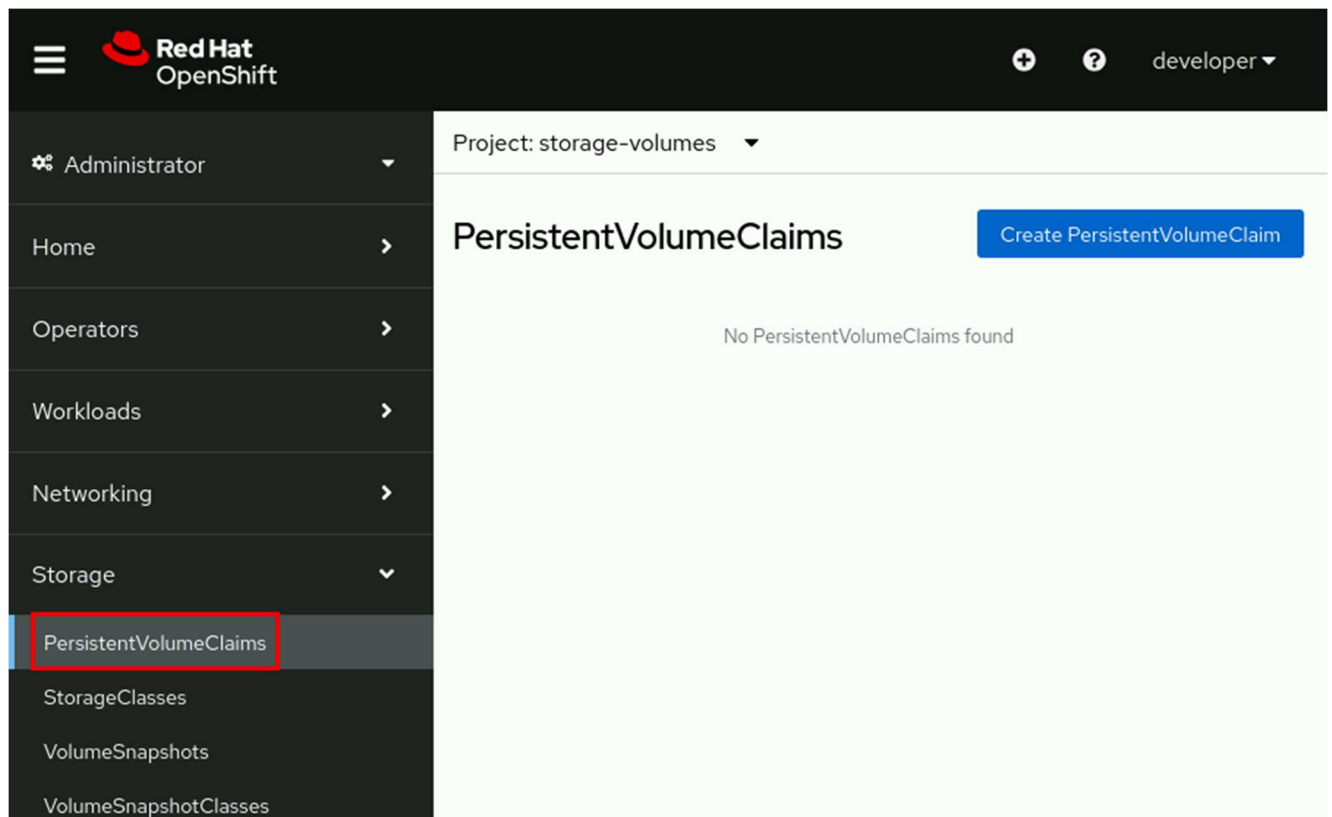
```
[user@host ~]$ oc create -f pvc_file_name.yaml
```

Use `oc get pvc` to view the available PVCs in the current namespace.

```
[user@host ~]$ oc get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	...
mypvc	Bound	pvc-13...	15Gi	RWO	nfs-storage	...

As a developer, to create a persistent volume claim from the web console, go to **Storage** → **PersistentVolumeClaims**.



Click **Create PersistentVolumeClaim** and complete the form by adding the name, the storage class, the size, the access mode, and the volume mode.

Create PersistentVolumeClaim

[Edit YAML](#)

StorageClass

SC nfs-storage ▼

StorageClass for the new claim

PersistentVolumeClaim name *

mypvc

A unique name for the storage claim within the project

Access mode *

Single user (RWO) ▼

Access mode is set by StorageClass and cannot be changed

Size *

–

15

+

GiB ▼

Desired storage capacity

☐ Use label selectors to request storage

PersistentVolume resources that match all label selectors will be considered for binding.

Volume mode *

☒ Filesystem ☐ Block

Create

Cancel

Kubernetes Dynamic Provisioning

PVs are defined by a `PersistentVolume` API object, which is from existing storage in the cluster. The cluster administrator must statically provision some storage types. Alternatively, the Kubernetes persistent volume framework can use a `StorageClass` object to dynamically provision PVs.

To create PVCs, developers specify the storage amount, the required access mode, and a storage class to describe and classify the storage. The control loop in the RHOCP control node watches for new PVCs, and binds the new PVC to an appropriate PV. If an appropriate PV does not exist, then a provisioner for the storage class creates one.

Claims remain unbound indefinitely if a matching volume does not exist or if a volume cannot be created with any available provisioner that services a storage class. Claims are bound when matching volumes become available. For example, a cluster with many manually provisioned 50 GiB volumes would not match a PVC that requests 100 GiB. The PVC can be bound when a 100 GiB PV is added to the cluster.

Developers can run the `oc get storageclass` command to view the storage classes that the cluster provides.

```
[user@host ~]$ oc get storageclass
NAME                                PROVISIONER ...
lvms-vgl                            topolvm.io ...
nfs-storage (default) k8s-sigs.io/nfs-subdir-external-provisioner
```

In the example, the `nfs-storage` storage class is marked as the default storage class. When a default storage class is configured, the PVC must explicitly name any other storage class to use, or can set the `storageClassName` annotation to `""`, to be bound to a PV without a storage class.

The following `oc set volume` command example uses the `claim-class` option to specify a dynamically provisioned PV.

```
[user@host ~]$ oc set volumes deployment/example-application \
--add --name example-pv-storage --type pvc \
```



```
--claim-class nfs-storage --claim-mode rwo --claim-size 15Gi \  
--mount-path /var/lib/example-app --claim-name mypvc
```

NOTE

Because a cluster administrator can change the default storage class, Red Hat recommends that you always specify the storage class when you create a PVC.

PV and PVC Lifecycles

When developers create PVCs, they request a specific amount of storage, an access mode, and a storage class. Kubernetes binds each PVC to an appropriate PV. If an appropriate PV does not exist, then a provisioner for the storage class creates one.

Figure 5.18: PV lifecycle

PVs follow a lifecycle based on their relationship to the PVC.

Available

After a PV is created, it becomes *available* for any PVC to use in the cluster in any namespace.

Bound

A PV that is *bound* to a PVC is also bound to the same namespace as the PVC, and no other PVC can use it.

In Use

You can delete a PVC if no pods actively use it. The *Storage Object in Use Protection* feature prevents the removal of bound PVs and PVCs that pods are actively using. Such removal can result in data loss. Storage Object in Use Protection is enabled by default.

If a user deletes a PVC that a pod actively uses, then the PVC is not removed immediately. PVC removal is postponed until no pods actively use the PVC. Also, if a cluster administrator deletes a PV that is bound to a PVC, then the

PV is not removed immediately. PV removal is postponed until the PV is no longer bound to a PVC.

Released

After the developer deletes the PVC that is bound to a PV, the PV is *released*, and the storage that the PV used can be reclaimed.

Reclaimed

The reclaim policy of a persistent volume tells the cluster what to do with the volume after it is released. A volume's reclaim policy can be Retain or Delete.

Table 5.4. Volume Reclaim Policy

Policy	Description
Retain	Enables manual reclamation of the resource for those volume plug-ins that support it.
Delete	Deletes both the PersistentVolume object from OpenShift Container Platform and the associated storage asset in external infrastructure.

Deleting a Persistent Volume Claim

To delete a volume, developers can use the `oc delete` command to delete the PVC. The storage class reclaims the volume after removing the PVC.

```
[user@host ~]$ oc delete pvc/mypvc
```

Guided Exercise: Provision Persistent Data Volumes

Deploy a MySQL database with persistent storage from a PVC and identify the PV and storage provisioner that back the application.

Outcomes

- Deploy a MySQL database with persistent storage from a PVC.

- Identify the PV that backs the application.
- Identify the storage provisioner that created the PV.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start storage-volumes
```

Instructions

1. Log in to the OpenShift web console as the developer user and switch to the **Administrator** perspective.
1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
2. Click **Red Hat Identity Management** and log in as the developer user with developer as the password. If the **Welcome to the Developer Perspective** message is displayed, then click **Skip tour**.
3. From the perspective switcher, select **Administrator**.
2. Identify the default storage class by going to **Storage** → **StorageClasses**. The `nfs-storage` storage class has the `Default` label and uses the `k8s-sigs.io/nfs-subdir-external-provisioner` provisioner.

Name ▾	Search by name...	/
Name ▴	Provisioner ▴	Reclaim policy ▴
 lvms-vgl	topolvm.io	Delete ▴
 nfs-storage - Default	k8s-sigs.io/nfs-subdir-external-provisioner	Delete ▴

3. Use the `registry.ocp4.example.com:8443/rhel8/mysql-80` container image to create a MySQL deployment named `db-pod`. Use the `storage-volumes` project that the `lab` command created. Add a service for the database.
1. Go to **Workloads** → **Deployments**.
2. Set the project to `storage-volumes` and click **Create Deployment**.
3. Use `db-pod` for the deployment name and change the image name to `registry.ocp4.example.com:8443/rhel8/mysql-80`.

4. Add the required environment variables for the database.

Table 5.5. MySQL Environment Variables

Name
MYSQL_USER
MYSQL_PASSWORD
MYSQL_DATABASE

5. In the **Advanced options** section, click **Scaling** and set the number of replicas to 1.
6. Click **Create**. Wait for the blue circle to indicate that a single pod is running.
7. Create a service for the database. Go to **Networking** → **Services** and click **Create Service**.
8. Update the resource file with the following values:

Table 5.6. Service Fields

Field name
Name
Selector
Port
Target port

Create Service

Create by manually entering YAML or JSON definitions, or by dragging and dropping a file into the editor.

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: db-pod
5    namespace: storage-volumes
6  spec:
7    selector:
8      app: db-pod
9    ports:
10     - protocol: TCP
11       port: 3306
12       targetPort: 3306
13
```

9. Click **Create**.
4. Add to the deployment a 1 GiB, ReadWriteOnce (RWO) PVC named db-pod-pvc. Set the /var/lib/mysql directory as the mount path.
1. Go to **Workloads** → **Deployments**.
2. Click the vertical ellipsis icon (:) in the row for the db-pod deployment, and select **Add storage**.
3. In the Add Storage form, click **Create new claim**.
4. Complete the form with the following values:

Table 5.7. Add PVC Storage Fields

Field name
Persistent volume claim name
Access mode
Size

Field name
Volume mode
Mount path

- Click **Save**.
- On the Deployment details page, scroll to the volumes section, and click the db-pod-pvc link to view the PVC details.

Volumes						
Name	Mount path	SubPath	Type	Permission...	Utilized by	
db-pod-pvc	/var/lib/mysql	No subpath	PVC db-pod-pvc	Read/Write	container	

- To retrieve the name of the PV that is associated with the PVC, go to the **YAML** tab. The name of the PV is available in the `.spec.volumeName` parameter. The name starts with pvc- even though the resource is a PV.

PVC db-pod-pvc Bound

Details

YAML

Events

VolumeSnapshots

```

1 kind: PersistentVolumeClaim
2 apiVersion: v1
3 metadata:
4   name: db-pod-pvc
5   namespace: storage-volumes
6   uid: 8b08e567-b649-4ee9-800e-183c30ec0310
7   resourceVersion: '116011'
8   creationTimestamp: '2025-08-25T08:46:59Z'
9   annotations:
10    pv.kubernetes.io/bind-completed: 'yes'
11    pv.kubernetes.io/bound-by-controller: 'yes'
12    volume.beta.kubernetes.io/storage-provisioner: k8s-sigs.io/nfs-subdir-external
13    volume.kubernetes.io/storage-provisioner: k8s-sigs.io/nfs-subdir-external-prov
14   finalizers:
15    - kubernetes.io/pvc-protection
16   managedFields: ...
59 spec:
60   accessModes:
61    - ReadWriteOnce
62   resources:
63     requests:
64       storage: 1Gi
65   volumeName: pvc-8b08e567-b649-4ee9-800e-183c30ec0310
66   storageClassName: nfs-storage
67   volumeMode: Filesystem

```

The name of the PV is different on your system.

5. Verify that configuring storage changes the deployment resource.
1. Go to **Workloads** → **Deployments**, click the db-pod deployment name, and go to the **YAML** tab.
2. Verify that configuring storage for the deployment results in the volumes and volumeMounts additions.

```
3.kind: Deployment
4.apiVersion: apps/v1
5.
6....output omitted...
7.
8.    volumes:
9.      - name: db-pod-pvc
10.        persistentVolumeClaim:
11.          claimName: db-pod-pvc
12.
13....output omitted...
14.
15.    volumeMounts:
16.      - name: db-pod-pvc
17.        mountPath: /var/lib/mysql
18.
```

```
...output omitted...
```

6. Use a configuration map resource to add initialization data to the database.
1. Review the ~/D0180/labs/storage-volumes/configmap/init-db.sql script that initializes the database. You do not have to change its contents.

```
2.DROP TABLE IF EXISTS `Item`;
3.CREATE TABLE `Item` (`id` BIGINT not null auto_increment primary key, `description` VARCHAR(100), `done` BIT);
4.INSERT INTO `Item` (`id`,`description`,`done`) VALUES (1,'Pick up newspaper', 0);
```

```
INSERT INTO `Item` (`id`,`description`,`done`) VALUES (2,'Buy groceries', 1);
```

5. In a terminal window, log in to the RHOC cluster as the developer user with developer as the password.

```
6. [student@workstation ~]$ oc login -u developer -p developer \
7. https://api.ocp4.example.com:6443
8. Login successful.
```

...output omitted...

9. Set the storage-volumes project as the active project.

```
10. [student@workstation ~]$ oc project storage-volumes
```

...output omitted...

11. Use the contents of the init-db.sql file to create a configuration map named init-db-cm.

```
12. [student@workstation ~]$ oc create configmap init-db-cm \
13. --from-file \
14. ~/D0180/labs/storage-volumes/configmap/init-db.sql
```

configmap/init-db-cm created

15. Add the init-db-cm configuration map as a volume named init-db-volume to the deployment. Specify the volume type as configmap, and set the /var/db/config directory as the mount path.

```
16. [student@workstation ~]$ oc set volumes deployment/db-pod \
17. --add --name init-db-volume --type configmap \
18. --configmap-name init-db-cm --mount-path /var/db/config
```

deployment.apps/db-pod volume updated

19. Start a remote shell session inside the container.

```
20. [student@workstation ~]$ oc rsh deployment/db-pod
```



```
sh-4.4$
```

21. Use the `mysql` client to execute the database script in the `/var/db/config/init-db` volume.

```
22. sh-4.4$ mysql -uuser1 -predhat123 \  
23. items < /var/db/config/init-db.sql  
24. mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
sh-4.4$
```

25. Execute a query to verify the database contents.

```
26. sh-4.4$ mysql -uuser1 -predhat123 items \  
27. -e 'select * from Item;'  
28. mysql: [Warning] Using a password on the command line interface can be insecure.  
29. +----+-----+-----+  
30. | id | description      | done      |  
31. +----+-----+-----+  
32. |  1 | Pick up newspaper | 0x00      |  
33. |  2 | Buy groceries    | 0x01      |  
34. +----+-----+-----+
```

```
sh-4.4$
```

35. Exit the shell session.

```
36. sh-4.4$ exit
```

```
exit
```

7. Delete and then re-create the `db-pod` deployment to verify that the database data persists.

1. Delete the `db-pod` deployment.

```
2. [student@workstation ~]$ oc delete deployment/db-pod
```

```
deployment.apps "db-pod" deleted
```

3. Verify that the PVC still exists without the deployment.

```
4. [student@workstation ~]$ oc get pvc
```

```
5. NAME                STATUS    VOLUME             CAPACITY    ...
```

```
db-pod-pvc    Bound    pvc-8b08...0310    1Gi        ...
```

6. Re-create the db-pod deployment.

```
7. [student@workstation ~]$ oc create deployment db-pod \
```

```
8.   --port 3306 \
```

```
9.   --image registry.ocp4.example.com:8443/rhel8/mysql-80
```

```
deployment.apps/db-pod created
```

10. Add the environment variables.

```
11. [student@workstation ~]$ oc set env deployment/db-pod \
```

```
12.   MYSQL_USER=user1 \
```

```
13.   MYSQL_PASSWORD=redhat123 \
```

```
14.   MYSQL_DATABASE=items
```

```
deployment.apps/db-pod updated
```

15. Use the `oc set volumes` command to attach the existing PVC to the deployment.

```
16. [student@workstation ~]$ oc set volumes deployment/db-pod \
```

```
17.   --add --type pvc \
```

```
18.   --mount-path /var/lib/mysql \
```

```
19.   --name db-pod-vol \
```

```
20.   --claim-name db-pod-pvc
```

```
deployment.apps/db-pod volume updated
```

21. Create a query-db pod by using the `oc run` command and the `registry.ocp4.example.com:8443/redhattraining/do180-dbinit` container image. Use the pod to execute a query against the database service.

It takes up to one minute for the command to display the result.

```
[student@workstation ~]$ oc run query-db -it --rm --image \
  registry.ocp4.example.com:8443/redhattraining/do180-dbinit \
  --restart Never \
  -- /bin/bash -c "mysql -uuser1 -predhat123 --protocol tcp \
  -h db-pod -P3306 items -e 'select * from Item;'"
mysql: [Warning] Using a password on the command line interface can be insecure.

+----+-----+-----+
| id | description      | done      |
+----+-----+-----+
|  1 | Pick up newspaper | 0x00      |
|  2 | Buy groceries    | 0x01      |
+----+-----+-----+
pod "query-db" deleted
```

8. Delete the db-pod deployment and the db-pod-pvc PVC.

1. Delete the db-pod deployment.

```
2. [student@workstation ~]$ oc delete deployment/db-pod
```

```
deployment.apps "db-pod" deleted
```

3. Delete the db-pod-pvc PVC.

```
4. [student@workstation ~]$ oc delete pvc/db-pod-pvc
```

```
persistentvolumeclaim "db-pod-pvc" deleted
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish storage-volumes
```

Selecting a Storage Class for an Application

Objectives

- Match applications with storage classes that provide storage services to satisfy application requirements.

Storage Class Selection

Storage classes describe types of storage for the cluster and provision dynamic storage on demand. The cluster administrator determines the meaning of a storage class, which is also named a *profile* in other storage systems. For example, an administrator can create one storage class for development use and another for production use.

Kubernetes supports multiple storage back ends. The storage options differ in cost, performance, reliability, and function. An administrator can create different storage classes for these options. As a result, developers can select the storage solution that fits the needs of the application. Developers do not need to know the storage infrastructure details.

An administrator selects the default storage class for dynamic provisioning. A default storage class enables Kubernetes to automatically provision a PVC that does not specify a storage class. Because an administrator can change the default storage class, a developer should explicitly set the storage class for an application.

Reclaim Policy

Outside the application function, the developer must also consider the impact of the reclaim policy on storage requirements.

A reclaim policy determines what happens to the data on a PVC after the PVC is deleted. When you are finished with a volume, you can delete the PVC object from

the API, which enables reclamation of the resource. Kubernetes releases the volume when the PVC is deleted, but the volume is not yet available for another claim. The previous claimant's data remains on the volume and must be handled according to the policy. To keep your data, choose a storage class with the `retain` reclaim policy.

By using the `retain` reclaim policy, when you delete a PVC, only the PVC object is deleted from the cluster. The PV that backed the PVC, the physical storage device that the PV used, and your data still exist. To reclaim the storage and to use it in your cluster again, the cluster administrator must take manual steps.

To manually reclaim a PV as a cluster administrator, follow these steps:

1. Delete the PV.

```
[user@host ~]$ oc delete pv pv-name
```

The associated asset in the external storage infrastructure, such as an Amazon Web Services (AWS) Elastic Block Store (EBS), Google Compute Engine (GCE) Persistent Disk, Azure Disk, or Cinder volume, still exists after the PV is deleted.

2. At this point, the cluster administrator can create another PV by using the same storage and data from the previous PV. A developer could then mount the new PV and access the data from the previous PV.
3. Alternatively, the cluster administrator can remove the data on the storage asset and then delete the storage asset.

To automatically delete the PV, the data, and the physical storage for a deleted PVC, you must choose a storage class that uses the `delete` reclaim policy. This reclaim policy automatically reclaims your storage volume when the PVC is deleted.

The `delete` reclaim policy is the default setting for all storage provisioners that adhere to the Kubernetes Container Storage Interface (CSI) standards. If you use a storage class that does not specify a reclaim policy, then the `delete` reclaim policy is used.

For more information about the Kubernetes Container Storage Interface standards, refer to the *Kubernetes CSI Developer Documentation* website at <https://kubernetes-csi.github.io/docs/>

Kubernetes and Application Responsibilities

Kubernetes does not change how an application relates to storage. The application is responsible for working with its storage devices and for ensuring data integrity and consistency.

Kubernetes storage does not prevent an application from inadvisable actions, such as sharing a data volume between two databases that require exclusive access to data.

Because a PVC is a storage device that your Linux host mounts, an improperly configured application could behave unexpectedly. For example, you could have an iSCSI Logical Unit Number (LUN), which is expressed as a `ReadWriteOnce` (RWO) PVC that is not supposed to be shared, and then mount that same PVC on two pods of the same host. Whether this situation is problematic depends on the applications.

Usually, it is fine for two processes on the same host to share a disk. After all, many applications on your personal machine share a local disk. However, nothing prevents one text editor from overwriting and losing all edits from another text editor. The use of Kubernetes storage must come with the same caution.

Single-node access (RWO) and shared access (RWX) do not ensure that files can be shared safely and reliably. RWO means that only one cluster node can read and write to the PVC. Alternatively, with RWX, Kubernetes provides a storage volume that any pod can access for reading or writing.

Use Cases for Storage Classes

The administrator creates storage classes that serve the needs of the developers. A `StorageClass` object defines each storage class, and the object contains information about the storage provider and the capabilities of the storage medium. The provider creates PVs to match the specifications of the storage class. Administrators can create storage classes with various functional levels, based on many factors.

Storage volume modes

A storage class with `block` volume mode support can increase performance for applications that can use raw block devices. Consider using a storage

class with `Filesystem` volume mode support for applications that share files or that provide file access.

Quality of Service (QoS) levels

A Solid State Drive (SSD) provides high speed and support for frequently accessed files. Use a lower cost and a slower hard drive (HDD) for files that are accessed less often.

Administrative support tier

A production-tier storage class can include volumes that are backed up often. In contrast, a development-tier storage class might include volumes that are not configured with a backup schedule.

Storage classes can use a combination of these factors and others to best fit the needs of the developers.

Kubernetes matches PVCs with the best available PV that is not bound to another PVC. The PV must provide the access mode that is specified in the PVC, and the volume must be at least as large as the requested size in the PVC. The supported access modes depend on the capabilities of the storage provider. A PVC can specify additional criteria, such as the name of a storage class. If a PVC cannot find a PV that matches all criteria, then the PVC enters a `pending` state and waits until an appropriate PV becomes available.

PVCs can request a specific storage class by specifying the `storageClassName` attribute. This method of selecting a specific storage class ensures that the storage medium is a good fit for the application requirements. Only PVs of the requested storage class can be bound to the PVC. The cluster administrator can configure dynamic provisioners to service storage classes. The cluster administrator can also create a PV on demand that matches the specifications in the PVC.

Create a Storage Class

The following YAML excerpt describes the definition for a `StorageClass` object. A cluster administrator or a `storage-admin` user creates globally scoped `StorageClass` objects. The following resource shows the parameters for configuring a storage class. This example uses the AWS EBS object definition.

```
apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

  name: io1-gold-storage

  annotations:
    storageclass.kubernetes.io/is-default-class: 'false'
    kubernetes.io/description: Provides RWO and RWOP volumes
...output omitted...

parameters:
  type: io1
  iopsPerGB: "10"
...output omitted...

provisioner: ebs.csi.aws.com

reclaimPolicy: Delete

volumeBindingMode: WaitForFirstConsumer

allowVolumeExpansion: true
```

A required item that specifies the current API version.

A required item that specifies the API object type.

A required item that specifies the name of the storage class.

An optional item that specifies annotations for the storage class.

An optional item that specifies the required parameters for the specific provisioner; this object differs between plug-ins.

A required item that specifies the type of provisioner that is associated with this storage class.

An optional item that specifies the selected reclaim policy for the storage class.

An optional item that specifies the selected volume binding mode for the storage class.

An optional item that specifies the volume expansion setting.

Several attributes, such as the API version, API object type, and annotations, are common for Kubernetes objects, whereas other attributes are specific to storage class objects.

Parameters

Parameters can configure file types, change storage types, enable encryption, enable replication, and so on. Each provisioner has different parameter options. Accepted parameters depend on the storage provisioner. For example, the `io1` value for the `type` parameter, and the `iopsPerGB` parameter, are specific to EBS. When a parameter is omitted, the storage provisioner uses the default value.

Provisioners

The `provisioner` attribute identifies the source of the storage medium plug-in. Provisioners with names that begin with a `kubernetes.io` value are available by default in a Kubernetes cluster.

ReclaimPolicy

The default reclaim policy, `Delete`, automatically reclaims the storage volume when the PVC is deleted. Reclaiming storage in this way can reduce the storage costs. The `Retain` reclaim policy does not delete the storage volume, so that data is not lost if the wrong PVC is deleted. This reclaim policy can result in higher storage costs if space is not manually reclaimed.

VolumeBindingMode

The `volumeBindingMode` attribute determines how volume attachments are handled for a requesting PVC. Using the default `Immediate` volume binding mode creates a PV to match the PVC when the PVC is created. This setting does not wait for the pod to use the PVC, and thus can be inefficient. The `Immediate` binding mode can also cause problems for storage back ends that are topology-constrained or are not globally accessible from nodes in the cluster. PVs are also bound without the knowledge of a pod's scheduling requirements, which might result in unschedulable pods.

By using the `waitForFirstConsumer` mode, the volume is created after the pod that uses the PVC is in use. With this mode, Kubernetes creates PVs that conform to the pod's scheduling constraints, such as resource requirements and selectors.

AllowVolumeExpansion

When set to the `true` value, the storage class specifies that the underlying storage volume can be expanded if more storage is required. Users can resize the volume by editing the corresponding PVC object. This feature can be used only to grow a volume, not to shrink it.

The cluster administrator can use the `create` command to create a storage class from a YAML manifest file. The resulting storage class is not namespaced, and thus is available to all projects in the cluster.

```
[user@host ~]$ oc create -f storage-class-filename.yaml
```

To create a storage class from the web console, log in as an administrator, go to **Storage** → **StorageClasses**, and click **Create StorageClass**. Complete the form or the YAML manifest.

Cluster Storage Classes

Use the `oc get storageclass` command to view the available storage class options in a cluster.

```
[user@host ~]$ oc get storageclass
```

A regular cluster user can view the attributes of a storage class by using the `describe` command. The following example queries the attributes of the storage class with the `lvms-vg1` name.

```
[user@host ~]$ oc describe storageclass lvms-vg1
IsDefaultClass:      No
Annotations:         description=Provides RWO and RWOP volumes
Provisioner:          topolvm.io
Parameters:           csi.storage.k8s.io/fstype=xfss, topolvm.io/device-class=vg1
```

```
AllowVolumeExpansion:  True
MountOptions:           <none>
ReclaimPolicy:          Delete
VolumeBindingMode:      WaitForFirstConsumer
Events:                 <none>
```

The `describe` command can help a developer to decide whether the storage class is a good fit for an application. If none of the storage classes in the cluster are appropriate for the application, then the developer can request the cluster administrator to create a PV with the required features.

Storage Class Usage

Recall that the `oc set volume` command can add a PVC and an associated PV to a deployment. A YAML manifest file can declare the parameters of a PVC independently from the deployment. This method is the preferred option to support repeatability, configuration management, and version control. Use the `storageClassName` attribute to specify the storage class for the PVC.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-block-pvc
spec:
  accessModes:
    - RWO
  volumeMode: Block
  storageClassName: storage-class-name
resources:
  requests:
    storage: 10Gi
```

Use the `create` command to create the resource from the YAML manifest file.

```
[user@host ~]$ oc create -f my-pvc-filename.yaml
```

Use the `--claim-name` option with the `set volume` command to add the existing PVC to a deployment.

```
[user@host ~]$ oc set volume deployment/deployment-name \  
  --add --name my-volume-name \  
  --claim-name my-block-pvc \  
  --mount-path /var/tmp
```

Guided Exercise: Selecting a Storage Class for an Application

Deploy a MySQL database with persistent storage based on block storage by selecting block storage instead of default file storage.

Outcomes

- Create volumes from a storage class.
- Deploy applications with persistent storage.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start storage-classes
```

Instructions

1. In a terminal window, log in to the RHOCP cluster as the `developer` user with `developer` as the password, and switch to the `storage-classes` project.
1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \  
3. https://api.ocp4.example.com:6443  
4. Login successful.
```

...output omitted...

5. Set the `storage-classes` project as the active project.

```
6. [student@workstation ~]$ oc project storage-classes
```

```
...output omitted...
```

2. Examine the available storage classes on the cluster. Identify an appropriate storage class to use for a database application.
1. Use the `oc get` command to retrieve a list of storage classes in the cluster. You can use the `storageclass` short name, `sc`, in the command.

```
2. [student@workstation ~]$ oc get sc
```

```
3. NAME                                PROVISIONER    ...
4. lvms-vg1                            topolvm.io     ...
```

```
nfs-storage (default) k8s-sigs.io/nfs-subdir-external-provisioner ...
```

Because an administrator can change the default storage class, applications must specify a storage class that meets the application requirements.

5. Use the `oc describe` command to view the details of the `lvms-vg1` storage class.

```
6. [student@workstation ~]$ oc describe sc lvms-vg1
```

```
7. Name:                               lvms-vg1
8. IsDefaultClass:                     No
9. Annotations:                        description=Provides RWO and RWOP Filesystem & Block
    volumes
10. Provisioner:                       topolvm.io
11. Parameters:                        csi.storage.k8s.io/fstype=xfs,topolvm.io/device-clas
    s=vg1
12. AllowVolumeExpansion:              True
13. MountOptions:                      <none>
14. ReclaimPolicy:                     Delete
15. VolumeBindingMode:                 WaitForFirstConsumer
```

```
Events:                                <none>
```

The description annotation states that the storage class provides support for block volumes. For some applications, such as databases, block volumes can provide a performance advantage over file system volumes. In the `lvms-vg1` storage class, the `AllowVolumeExpansion` field is set to `True`. With volume expansion, cluster users can edit their PVC objects and specify a new size for the PVC. Kubernetes then uses the storage back end to automatically expand the volume to the requested size. Kubernetes also expands the file system of pods that use the PVC. Enabling volume expansion can help to protect an application from failing due to the data growing too fast. With these features, the `lvms-vg1` storage class is a good choice for the database application.

3. Use the `registry.ocp4.example.com:8443/rhel8/mysql-80` container image to create a MySQL deployment named `db-pod`. Add the missing environment variables for the pod to run.

1. Create the `db-pod` deployment.

```
2. [student@workstation ~]$ oc create deployment db-pod \  
3.   --port 3306 \  
4.   --image registry.ocp4.example.com:8443/rhel8/mysql-80
```

```
deployment.apps/db-pod created
```

5. Add the environment variables.

```
6. [student@workstation ~]$ oc set env deployment/db-pod \  
7.   MYSQL_USER=user1 \  
8.   MYSQL_PASSWORD=redhat123 \  
9.   MYSQL_DATABASE=items
```

```
deployment.apps/db-pod updated
```

10. Verify that the pod is running.

```
11. [student@workstation ~]$ oc get pods
```

12. NAME	READY	STATUS	RESTARTS	AGE
----------	-------	--------	----------	-----

db-pod-b4ccfb74-nn4s5	1/1	Running	0	5s
-----------------------	-----	---------	---	----

13. Expose the db-pod deployment to create a service.

```
14. [student@workstation ~]$ oc expose deployment/db-pod
```

```
service/db-pod exposed
```

4. Add a 1 Gi, RWO PVC named db-pod-pvc to the deployment. Specify the volume name as lvm-storage, and set the /var/lib/mysql directory as the mount path. Use the lvms-vg1 storage class.

1. Use the `oc set volume` command to create a PVC for the deployment.

```
2. [student@workstation ~]$ oc set volumes deployment/db-pod \  
3. --add --name lvm-storage --type pvc --claim-mode rwo \  
4. --claim-size 1Gi --mount-path /var/lib/mysql \  
5. --claim-class lvms-vg1 --claim-name db-pod-pvc
```

```
deployment.apps/db-pod volume updated
```

The `claim-class` option specifies a non-default storage class.

NOTE

The `oc set volumes` command does not have an option to set the volume mode property. Thus, the `oc set volumes` command results in using the default `Filesystem` volume mode. Use a declarative manifest to set the volume mode to `Block`. The use of declarative manifests is out of scope for this course.

6. Use the `oc get pvc` command to view the status of the PVC. Identify the name of the PV, and confirm that the PVC uses the `lvms-vg1` non-default storage class.

```
7. [student@workstation ~]$ oc get pvc
```

8. NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
---------	--------	--------	----------	--------------	--------------

db-pod-pvc	Bound	pvc-72...	1Gi	RWO	lvms-vg1 ...
------------	-------	-----------	-----	-----	--------------

9. Use the `oc describe pvc` command to inspect the details of the db-pod-pvc PVC.

```
10. [student@workstation ~]$ oc describe pvc db-pod-pvc
```

```
11. Name:          db-pod-pvc
12. Namespace:     storage-classes
13. StorageClass:  lvms-vg1
14. Status:        Bound
15. Volume:        pvc-72084a46-3f32-435e-987b-4ad3b9026021
16. Labels:        <none>
17. Annotations:   pv.kubernetes.io/bind-completed: yes
18.                pv.kubernetes.io/bound-by-controller: yes
19.                volume.beta.kubernetes.io/storage-provisioner: topolvm.io
20.                volume.kubernetes.io/selected-node: master01
21.                volume.kubernetes.io/storage-provisioner: topolvm.io
22. Finalizers:     [kubernetes.io/pvc-protection]
23. Capacity:      1Gi
24. Access Modes:   RW0
25. VolumeMode:     Filesystem
26. Used By:        db-pod-568888457d-qmxn9
27. Events:
```

...output omitted...

The Used By attribute confirms that the PVC is bound to a pod.

5. Connect to the database to verify that it is working.

```
6. [student@workstation ~]$ oc run query-db -it --rm \
7.   --image registry.ocp4.example.com:8443/rhel8/mysql-80 \
8.   --restart Never --command \
9.   -- /bin/bash -c "mysql -uuser1 -predhat123 --protocol tcp \
10.  -h db-pod -P3306 items -e 'show databases;'"
11. mysql: [Warning] Using a password on the command line interface can be insecure.
12. +-----+
13. | Database          |
14. +-----+
15. | information_schema |
```



```
16. | items          |
17. | performance_schema |
18. +-----+
```

```
pod "query-db" deleted
```

19. Delete the db-pod deployment and the db-pod-pvc PVC.

1. Delete the db-pod deployment and service.

```
2. [student@workstation ~]$ oc delete deployment db-pod
3. deployment.apps "db-pod" deleted
4. [student@workstation ~]$ oc delete service db-pod
```

```
service "db-pod" deleted
```

5. Verify that the PVC still exists without the deployment.

```
6. [student@workstation ~]$ oc get pvc
7. NAME          STATUS VOLUME      CAPACITY ACCESS MODES STORAGECLASS
```

```
db-pod-pvc Bound  pvc-72... 1Gi          RW0          lvms-vg1 ...
```

8. Delete the db-pod-pvc PVC.

```
9. [student@workstation ~]$ oc delete pvc db-pod-pvc
```

```
persistentvolumeclaim "db-pod-pvc" deleted
```

20. Create a PVC for an application that requires a shared storage volume from the nfs-storage storage class.

1. Create a PVC YAML manifest file named `nfs-pvc.yaml` with the following contents:

```
2. ---
3. apiVersion: v1
4. kind: PersistentVolumeClaim
5. metadata:
6.   name: nfs-pvc
7. spec:
```

```
8.  accessModes:
9.    - ReadWriteOnce
10. resources:
11.   requests:
12.     storage: 1Gi
```

```
storageClassName: nfs-storage
```

13. Create the PVC by using the `oc create -f` command and the YAML manifest file.

```
14. [student@workstation ~]$ oc create -f nfs-pvc.yaml
```

```
persistentvolumeclaim/nfs-pvc created
```

15. Use the `oc describe pvc` command to view the details of the PVC resource.

```
16. [student@workstation ~]$ oc describe pvc nfs-pvc
17. Name:          nfs-pvc
18. Namespace:     storage-classes
19. StorageClass:  nfs-storage
20. Status:        Bound
21. Volume:        pvc-9f462124-d96e-43e5-96a7-f96dd9375579
22. Labels:        <none>
23. Annotations:   pv.kubernetes.io/bind-completed: yes
24.                pv.kubernetes.io/bound-by-controller: yes
25.                volume.beta.kubernetes.io/storage-provisioner: k8s-sigs.io/nfs-subdir-external-provisioner
26.                volume.kubernetes.io/storage-provisioner: k8s-sigs.io/nfs-subdir-external-provisioner
27. Finalizers:     [kubernetes.io/pvc-protection]
28. Capacity:      1Gi
29. Access Modes:   RWX
30. VolumeMode:     Filesystem
31. Used By:        <none>
32. Events:
```

...output omitted...

The `Used By: <none>` attribute shows that no pod is using the PVC.
The `Status: Bound` value and the `Volume` attribute assignment confirm that the volume binding mode of the storage class is set to `Immediate`.

21. Use the `registry.ocp4.example.com:8443/ubi9/httpd-24:1-321` container image to create a web application deployment named `web-pod`.

1. Create the `web-pod` deployment.

```
2. [student@workstation ~]$ oc create deployment web-pod \  
3.   --port 8080 \  
4.   --image registry.ocp4.example.com:8443/ubi9/httpd-24:1-321
```

`deployment.apps/web-pod created`

5. Create a service for the `web-pod` application.

```
6. [student@workstation ~]$ oc expose deployment web-pod
```

`service/web-pod exposed`

7. Expose the service to create a route for the `web-pod` application.
Specify `web-pod.apps.ocp4.example.com` as the hostname.

```
8. [student@workstation ~]$ oc expose svc web-pod \  
9.   --hostname web-pod.apps.ocp4.example.com
```

`route.route.openshift.io/web-pod exposed`

10. View the route that is assigned to the `web-pod` application.

```
11. [student@workstation ~]$ oc get routes  
12. NAME          HOST/PORT          PATH  SERVICES  PORT  ...
```

```
web-pod  web-pod.apps.ocp4.example.com  web-pod  8080  ...
```

13. Use the `curl` command to view the index page of the `web-pod` application.

```
14. [student@workstation ~]$ curl -s\
```

```
15. http://web-pod.apps.ocp4.example.com/ | head
16. <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
17.
18. <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
19.     <head>
20.         <title>Test Page for the HTTP Server on Red Hat Enterprise Linux</title>
```

...output omitted...

22. Add the nfs-pvc PVC to the web-pod deployment.

1. Use the `oc set volume` command to add the PVC to the deployment. Specify the volume name as `nfs-volume`, and set the mount path to the `/var/www/html` directory.

```
2. [student@workstation ~]$ oc set volumes deployment/web-pod \
3.   --add --name nfs-volume \
4.   --claim-name nfs-pvc \
5.   --mount-path /var/www/html
```

deployment.apps/web-pod volume updated

The volume `mount-path` is set to the `/var/www/html` directory. The server uses this path to serve HTML content.

23. Use the `registry.ocp4.example.com:8443/redhattraining/do180-roster` container image to create a custom web application deployment named `app-pod`.

1. Create the `app-pod` deployment and specify port `9090` as the target port.

```
2. [student@workstation ~]$ oc create deployment app-pod \
3.   --port 9090 --image \
4.   registry.ocp4.example.com:8443/redhattraining/do180-roster
```

deployment.apps/app-pod created

5. Create a service for the `app-pod` application.

```
6. [student@workstation ~]$ oc expose deployment app-pod
```

```
service/app-pod exposed
```

7. Expose the service to create a route for the app-pod application. Use app-pod.apps.ocp4.example.com for the hostname.

```
8. [student@workstation ~]$ oc expose svc app-pod \
```

```
9.   --hostname app-pod.apps.ocp4.example.com
```

```
route.route.openshift.io/app-pod exposed
```

24. Add the nfs-pvc PVC to the app-pod application deployment.

1. Use the `oc set volume` command to add the PVC to the deployment. Set the volume name to `nfs-volume` and set the mount path to the `/var/tmp` directory.

```
2. [student@workstation ~]$ oc set volumes deployment/app-pod \
```

```
3.   --add --name nfs-volume \
```

```
4.   --claim-name nfs-pvc \
```

```
5.   --mount-path /var/tmp
```

```
deployment.apps/app-pod volume updated
```

At this point, the `web-pod` and the `app-pod` applications are sharing a PVC. Kubernetes cannot prevent data conflicts between the two applications. In this case, the `app-pod` application is a writer and the `web-pod` application is a reader, and thus they do not conflict. The application implementation, not Kubernetes, prevents data corruption from the two applications that use the same PVC. The RWO access mode does not protect data integrity; a single node can mount the volume as read/write, and pods that share the volume must exist on the same node.

25. Use the `app-pod` application to add content to the shared volume.

1. Open a web browser and go to `http://app-pod.apps.ocp4.example.com`

First Name:

Last Name:

State:

Hobby:

Contact:

save

First: Antone Last: Adamson State: Florinese Hobby: sword-fighting Contact: aadamson@example.com

Delete

push

2. In the form, enter your information and click **save**. The application adds your information to the list after the form.
3. Click **push** to create the `/var/tmp/People.html` file on the shared volume.
26. Verify that the web-pod application displays the `People.html` file from the shared PVC. Open a browser tab and go to `http://web-pod.apps.ocp4.example.com/People.html`

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish storage-classes
```

Manage Non-shared Storage with Stateful Sets

Objectives

- Deploy applications that scale without sharing storage.

Application Clustering

Clustering applications, such as MySQL and Cassandra, typically require persistent storage. This storage maintains the integrity of the data and files that the

application uses. When many applications require persistent storage at the same time, multi-disk provisioning might not be possible because of limited resources.

Shared storage solves this problem by allocating resources from a single device to multiple services.

File Storage

File storage solutions provide a familiar directory structure. Using file storage is ideal when applications generate or consume typical volumes of organized data. Applications that use file-based implementations are prevalent, easy to manage, and provide an affordable storage solution.

File-based solutions are a good fit for data backup, archiving, file sharing, and collaboration services because of their reliability. Most data centers provide file storage solutions, such as a *network-attached storage* (NAS) cluster, for these scenarios.

NAS is a file-based storage architecture that makes stored data accessible to networked devices. NAS gives networks a single access point for storage with built-in security, management, and fault-tolerant capabilities. Networks can run multiple data transfer protocols. Two fundamental protocols are *internet protocol (IP)* and *transmission control protocol (TCP)*.

The files that are transferred by using these protocols can be formatted with one of the following protocols:

- *Network File System* (NFS): This protocol enables remote hosts to mount file systems over a network and interact with those file systems as though they are mounted locally.
- *Server Message Block* (SMB): This protocol implements an application-layer network protocol to access resources on a server, such as file shares and shared printers.

NAS solutions can provide file-based storage to applications within the same data center. This approach applies to the following application architectures:

- Web server content
- File-sharing services
- FTP storage

- Backup archives

These applications take advantage of data reliability and the ease of file sharing that is available by using file storage. Additionally, for file storage data, the OS and file system handle the locking and caching of the files.

Although familiar and prevalent, file storage solutions are not ideal for all application scenarios. One pitfall of file storage is poor handling of large data sets or unstructured data.

Block Storage

Block storage solutions, such as Storage Area Network (SAN) and iSCSI technologies, provide access to raw block devices for application storage. These block devices function as independent storage volumes, such as the physical drives in servers, and typically require formatting and mounting for application access.

Using block storage is ideal when applications require faster access for optimizing computationally heavy data workloads. Applications that use block-level storage implementations gain efficiencies by communicating at the raw device level, instead of relying on operating system layer access.

Block-level approaches enable data distribution on blocks across the storage volume. Blocks also use basic metadata, including a unique identification number for each block of data, for quick retrieval and reassembly of blocks for reading.

SAN and iSCSI technologies provide applications with block-level volumes from network-based storage pools. Using block-level access to storage volumes is common for the following application architectures:

- SQL databases (single-node access)
- Virtual machines (multinode access)
- High-performance data access
- Server-side processing applications
- Multiple block device RAID configurations

Application storage that uses several block devices in a RAID configuration benefits from the data integrity and performance that the various arrays provide.

With Red Hat OpenShift Container Platform, you can create customized storage classes for your applications. With NAS and SAN storage technologies, OpenShift applications can use either the NFS protocol for file-based storage or a block-level protocol for block storage.

Stateful Sets

A stateful application acts according to past states or transactions, which affect the current and future states of the application. Using a stateful application simplifies recovery from failures by starting from a certain point in time.

A stateful set is the representation of a set of pods with consistent identities. These identities are defined as a network with a single stable DNS, hostname, and storage from as many volume claims as the stateful set specifies. A stateful set guarantees that a given network identity maps to the same storage identity.

Deployments represent a set of containers within a pod. Each deployment can have many active replicas, depending on the user specification. These replicas can be scaled up or down, as needed. A replica set is a native Kubernetes API object that ensures that the specified number of pod replicas are running. Deployments are used for stateless applications by default, and they can be used for stateful applications by attaching a persistent volume. All pods in a deployment use the same PVC.

In contrast with deployments, stateful set pods do not share a persistent volume. Instead, each stateful set pod has its own unique persistent volume. Pods are created without a replica set, and each replica records its own transactions. Each replica has its own identifier, which is maintained in any rescheduling. You must configure application-level clustering so that stateful set pods have the same data.

Stateful sets are the best option for applications, such as databases, that require consistent identities and non-shared persistent storage.

Headless Services for Stateful Sets

Stateful sets require a headless service to provide the stable network identity for its pods. Unlike a typical service that gets a single stable IP address (`clusterIP`) for load balancing, a headless service does not have a `clusterIP` address.

You create a headless service by explicitly setting its `clusterIP` field to `None`. For a DNS lookup on a headless service, the DNS server returns the IP addresses of all the individual pods that the service selects. This feature allows clients to connect directly to a specific pod instead of going through a load balancer.

For a `StatefulSet`, this behavior is critical. When you associate a stateful set with a headless service, each pod in the set gets a unique and predictable DNS entry. This DNS entry follows the format `<pod-name>.<service-name>.<namespace>.svc.cluster.local`. For example, a pod named `dbserver-0` that is governed by a headless service named `mysql-headless` can be reached reliably at the `dbserver-0.mysql-headless` FQDN.

This stable identity enables peer discovery and direct communication between pods in a clustered application. These capabilities are crucial for applications such as distributed databases, where specific pods must be addressed directly for operations such as writing or replication.

Working with Stateful Sets

With Kubernetes, you can use manifest files to specify the intended configuration of a stateful set. You can define the name of the application, labels, the image source, storage, environment variables, and more. The following snippet shows an example of a YAML manifest file for a stateful set named `dbserver`:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: dbserver
spec:
  selector:
    matchLabels:
      app: database
  replicas: 3
  serviceName: mysql
```

```
template:
  metadata:
    labels:

      app: database
  spec:
    containers:

      - env:
        - name: MYSQL_USER
          valueFrom:
            secretKeyRef:
              key: user
              name: sakila-cred

        image: registry.ocp4.example.com:8443/redhattraining/mysql-app:v1

        name: database

        ports:
          - containerPort: 3306
            name: database

        volumeMounts:
          - mountPath: /var/lib/mysql
            name: data

        terminationGracePeriodSeconds: 10
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:

        accessModes: [ "ReadWriteOncePod" ]

        storageClassName: "lvms-vg1"
```

```
resources:
  requests:

    storage: 1Gi
```

Name of the stateful set.

Application labels.

Number of replicas.

Name of the service that governs the stateful set. This service must exist before the stateful set.

Environment variables, which can be defined explicitly or by using a secret object.

Image source.

Container name.

Container ports.

Mount path information for the persistent volumes for each replica. Each persistent volume has the same configuration.

The access mode of the persistent volume. The valid values are `ReadWriteOncePod`, `ReadWriteOnce`, `ReadWriteMany`, and `ReadOnlyMany`.

The `ReadWriteOncePod` mode is recommended for production use.

The storage class that the persistent volume uses.

Size of the persistent volume.

NOTE

Stateful sets can be created only by using manifest files. The `oc` and `kubectl` CLIs do not have commands to create stateful sets imperatively.

The following snippet shows an example YAML manifest for a headless service named `mysql` for the `dbserver` stateful set:

```
apiVersion: v1
kind: Service
metadata:

  name: mysql
```

```
labels:
  app: database
spec:
  ports:
  - port: 3306
    name: mysql

  clusterIP: None
  selector:

    app: database
```

The name of the headless service.

Set the `clusterIP` field to `None` to make the service headless.

The service selector must match the labels of the pods that the stateful set created.

You create the `mysql` headless service by using the `oc create` command:

```
[user@host ~]$ oc create -f headless-service.yml
```

You can verify the creation of the service by running the following command:

```
[user@host ~]$ oc get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/mysql	ClusterIP	None	<none>	3306/TCP	5s

You can create the `dbserver` stateful set by using the `oc create` command:

```
[user@host ~]$ oc create -f statefulset-dbserver.yml
```

You can verify the creation of the stateful set by running the following command. In the following example, the stateful set has three replica pods, and all three pods are ready.

```
[user@host ~]$ oc get statefulset
```

NAME	READY	AGE
dbserver	3/3	6s

The stateful set controller assigns each pod a unique and stable name in the `<statefulset-name>-<ordinal-index>` format. The controller starts pods one at a time, sorted by their ordinal index, and it waits until each pod reports being ready before starting the next one.

```
[user@host ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
dbserver-0	1/1	Running	0	85s
dbserver-1	1/1	Running	0	82s
dbserver-2	1/1	Running	0	79s

You can get the service endpoints, which are the pods' IP addresses by running the following command:

```
[user@host ~]$ oc get endpoints
```

NAME	ENDPOINTS	AGE
mysql	10.8.0.45:3306,10.8.0.54:3306,10.8.0.61:3306	2m

You resolve the pods' IP addresses by using the headless service; for example, to resolve the IP address for the `dbserver-0` pod, you use the `dbserver-0.mysql` hostname.

```
[user@host ~]$ oc rsh dbserver-0 curl -v \
  telnet://dbserver-0.mysql:3306
Rebuilt URL to: telnet://dbserver-0.mysql:3306/
Trying 10.8.0.45...
TCP_NODELAY set
Connected to dbserver-0.mysql (10.8.0.45) port 3306 (#0)
```

The stateful set creates three PVCs, one PVC for each replica pod. Each PVC has a unique and stable name in the `data-<statefulset-name>-<ordinal-index>` format.

```
[user@host ~]$ oc get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	...
data-dbserver-0	Bound	pvc-c28...	1Gi	RWO	...
data-dbserver-1	Bound	pvc-ddb...	1Gi	RWO	...
data-dbserver-2	Bound	pvc-830...	1Gi	RWO	...

Each pod attaches its associated PVC. You can check the PVC that is attached to each replica pod in the stateful set by running the following commands:

```
[user@host ~]$ oc describe pod dbserver-0
```

...output omitted...

Volumes:

data:

Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)

ClaimName: **data-dbserver-0**

...output omitted...

```
[user@host ~]$ oc describe pod dbserver-1
```

...output omitted...

Volumes:

data:

Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)

ClaimName: **data-dbserver-1**

...output omitted...

```
[user@host ~]$ oc describe pod dbserver-2
```

...output omitted...

Volumes:

data:

Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)

ClaimName: **data-dbserver-2**

...output omitted...

The data is replicated between the three pods at the application level. For example, in the `dbserver` stateful set, you can configure the first pod as the primary, and the other two pods as replicas. The primary pod handles read and write requests, and the replica pods sync with the primary pod for data replication.

NOTE

Configuring a replicated stateful set application is outside the scope of this course.

You can update the number of replicas of the stateful set by using the `oc scale` command. The following command adds one more pod to the stateful set. A fourth pod is created if the third pod is up and running.

```
[user@host ~]$ oc scale statefulset/dbserver --replicas 4
```

NAME	READY	STATUS	RESTARTS	...
dbserver-0	4/4	Running	0	...

To delete the stateful set, use the `delete statefulset` command:

```
[user@host ~]$ oc delete statefulset dbserver
```

statefulset.apps "dbserver" deleted

The PVCs are not deleted after the execution of the `oc delete statefulset` command:

```
[user@host ~]$ oc get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS	MODES	...
data-dbserver-0	Bound	pvc-c28...	1Gi	RWO		...
data-dbserver-1	Bound	pvc-ddb...	1Gi	RWO		...
data-dbserver-2	Bound	pvc-830...	1Gi	RWO		...

You can create a stateful set from the web console. Go to the **Workloads** → **StatefulSets** menu. Click **Create StatefulSet** and customize the YAML manifest.

Lab: Manage Storage for Application Configuration and Data

Deploy a web application and its database that share database credentials from a secret. The database should use the default storage for the cluster. Also, you deploy a file-sharing application that runs with multiple replicas and shares its storage volume with a file uploader application. The file sharing and file uploader applications take configuration files from a configuration map and should use NFS file storage for shareability. The database should use local storage for increased performance.

Outcomes

- Deploy a database server.
- Deploy a web application.
- Create a secret that contains the database server credentials.
- Create a configuration map that contains an SQL file.
- Add and remove a volume on the database server and the web application.
- Expose the database server and the web application.
- Scale up the web application.
- Mount the configuration map as a volume.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start storage-review
```

Instructions

In this exercise, you work in the `storage-review` project that the `lab` command created. The command also created files in the `/home/student/D0180/labs/storage-review` directory that you use in the exercise.

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

1. Log in to the OpenShift cluster as the developer user with developer as the password, and change to the storage-review project. You use this project for all your work in this exercise.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \  
3. https://api.ocp4.example.com:6443
```

...output omitted...

4. Change to the storage-review project.

```
5. [student@workstation ~]$ oc project storage-review
```

...output omitted...

2. Create a secret named world-cred that contains the following data:

Field	Value
user	redhat
password	redhat123
database	world_x

1. Create a secret that contains the database credentials.

```
2. [student@workstation]$ oc create secret generic world-cred \  
3. --from-literal user=redhat \  
4. --from-literal password=redhat123 \  
5. --from-literal database=world_x
```

secret/world-cred created

6. Confirm the creation of the secret.

```
7. [student@workstation ~]$ oc get secrets world-cred  
8. NAME          TYPE          DATA   AGE
```

```
world-cred    Opaque    3        2m34s
```

3. Create a configuration map named `dbfiles` by using the `~/D0180/labs/storage-review/insertdata.sql` file.
1. Create a configuration map named `dbfiles` by using the `insertdata.sql` file in the `~/D0180/labs/storage-review` directory.

```
2. [student@workstation ~]$ oc create configmap dbfiles \  
3.   --from-file ~/D0180/labs/storage-review/insertdata.sql
```

```
configmap/dbfiles created
```

4. Verify the creation of the configuration map.

```
5. [student@workstation]$ oc get configmaps  
6. NAME      DATA  AGE  
7. dbfiles   1      11s
```

```
...output omitted...
```

4. Create a database server deployment named `dbserver` by using the `registry.ocp4.example.com:8443/redhattraining/mysql-app:v1` container image. Then, set the missing environment variables by using the `world-cred` secret. Use the `MYSQL_` prefix for the variables.
1. Create the database server deployment.

```
2. [student@workstation ~]$ oc create deployment dbserver \  
3.   --image \  
4.   registry.ocp4.example.com:8443/redhattraining/mysql-app:v1
```

```
deployment.apps/dbserver created
```

5. Set the missing environment variables.

```
6. [student@workstation ~]$ oc set env deployment/dbserver \  
7.   --from secret/world-cred --prefix MYSQL_
```

```
deployment.apps/dbserver updated
```

8. Verify that the dbserver pod is in the RUNNING state. The pod name might differ in your output.

```
9. [student@workstation ~]$ oc get pods
```

```
10. NAME                                READY   STATUS   ...
```

```
dbserver-6d5bf5d86c-ptrb2    1/1     Running ...
```

5. Add a volume to the dbserver deployment by using the following information:

Field	Value
Name	dbserver-lvm
Type	persistentVolumeClaim
Claim mod	rwo
Claim size	1Gi
Mount path	/var/lib/mysql
Claim class	lvms-vg1
Claim name	dbserver-lvm-pvc

1. Add a volume to the dbserver deployment.

```
2. [student@workstation ~]$ oc set volume deployment/dbserver \  
3. --add --name dbserver-lvm --type persistentVolumeClaim \  
4. --claim-mode rwo --claim-size 1Gi \  
5. --mount-path /var/lib/mysql --claim-class lvms-vg1 \  
6. --claim-name dbserver-lvm-pvc
```

```
deployment.apps/dbserver volume updated
```

7. Verify the deployment status.

8. [student@workstation ~]\$ **oc get pods**

9. NAME READY STATUS ...

dbserver-5bc6bd5d7b-7z7lv 1/1 Running ...

10. Verify the volume status.

11. [student@workstation ~]\$ **oc get pvc**

12. NAME STATUS VOLUME CAPACITY ...

dbserver-lvm-pvc Bound pvc-2cb8...5025 1Gi ...

6. Create a service for the dbserver deployment by using the following information:

Field	Value
Name	mysql-service
Port	3306
Target port	3306

1. Expose the dbserver deployment.

2. [student@workstation ~]\$ **oc expose deployment dbserver **

3. **--name mysql-service --port 3306 --target-port 3306**

service/mysql-service exposed

4. Verify the service configuration. The endpoint IP address might differ in your output.

5. [student@workstation ~]\$ **oc get services**

6. NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
7. mysql-service ClusterIP 172.30.240.100 <none> 3306/TCP

8. [student@workstation ~]\$ **oc get endpoints**

9. NAME	ENDPOINTS	AGE
mysql-service	10.8.1.36:3306	20s

7. Create a web application deployment named file-sharing by using the registry.ocp4.example.com:8443/redhattraining/php-webapp-mysql:v1 container image. Scale the deployment to two replicas. Then, expose the deployment by using the following information:

Field	Value
Name	file-sharing
Port	8080
Target port	8080

8. Create a route named file-sharing to expose the file-sharing web application to external access. Access the file-sharing route in a web browser to test the connection between the web application and the database server.

1. Create a web application deployment.

```
2. [student@workstation ~]$ oc create deployment file-sharing \
3.   --image registry.ocp4.example.com:8443/redhattraining/php-webapp-mysql:v1
```

```
deployment.apps/file-sharing created
```

4. Verify the deployment status. Verify that the file-sharing application pod is in the RUNNING state. The pod names might differ on your system.

```
5. [student@workstation ~]$ oc get pods
6. NAME                                READY   STATUS    ...
7. dbserver-5bc6bd5d7b-7z7lv          1/1     Running   ...
```

```
file-sharing-789c5948c8-gdrlz    1/1     Running   ...
```

8. Scale the deployment to two replicas.

```
9. [student@workstation ~]$ oc scale deployment file-sharing \
10. --replicas 2
```

```
deployment.apps/file-sharing scaled
```

11. Verify the replica status and retrieve the pod name. The pod names might differ on your system.

```
12. [student@workstation ~]$ oc get pods
13. NAME                                READY   STATUS    ...
14. dbserver-5bc6bd5d7b-7z7lv          1/1     Running   ...
15. file-sharing-789c5948c8-62j9s      1/1     Running   ...
```

```
file-sharing-789c5948c8-gdrlz 1/1     Running   ...
```

16. Expose the file-sharing deployment.

```
17. [student@workstation ~]$ oc expose deployment file-sharing \
18. --name file-sharing --port 8080 --target-port 8080
```

```
service/file-sharing exposed
```

19. Verify the service configuration. The endpoint IP address might differ in your output.

```
20. [student@workstation ~]$ oc get services
21. NAME           TYPE           CLUSTER-IP     EXTERNAL-IP  PORT(S)
22. file-sharing   ClusterIP      172.30.139.210 <none>       8080/TCP
23. mysql-service ClusterIP      172.30.240.100 <none>       3306/TCP
24. [student@workstation ~]$ oc get endpoints
25. NAME           ENDPOINTS                                AGE
26. file-sharing   10.8.1.37:8080,10.8.1.38:8080          14s
```

```
mysql-service 10.8.1.36:3306                                5m23s
```

27. Expose the file-sharing service.

```
28. [student@workstation ~]$ oc expose service/file-sharing
```

29. route.route.openshift.io/file-sharing exposed

30. [student@workstation ~]\$ **oc get routes**

31. NAME HOST/PORT ...

file-sharing file-sharing-storage-review.apps.ocp4.example.com

32. Test the connectivity between the web application and the database server. In a web browser, go to <http://file-sharing-storage-review.apps.ocp4.example.com>, and verify that the Connected successfully message is displayed.

9. Mount the dbfiles configuration map to the file-sharing deployment as a volume named config-map-pvc. Set the mount path to the /home/database-files directory. Then, verify the content of the insertdata.sql file.

1. Mount the dbfiles configuration map to the file-sharing deployment.

2. [student@workstation ~]\$ **oc set volume deployment/file-sharing **

3. **--add --name config-map-pvc --type configmap **

4. **--configmap-name dbfiles **

5. **--mount-path /home/database-files**

deployment.apps/file-sharing volume updated

6. Verify the deployment status.

7. [student@workstation ~]\$ **oc get pods**

8. NAME READY STATUS ...

9. dbserver-5bc6bd5d7b-7z7lv 1/1 Running ...

10. file-sharing-7f77855b7f-949lg 1/1 Running ...

file-sharing-7f77855b7f-9zvzq 1/1 Running ...

11. Verify the content of the /home/database-files/insertdata.sql file.

12. [student@workstation ~]\$ **oc exec -it **

13. **pod/file-sharing-7f77855b7f-949lg -- **

14. **head /home/database-files/insertdata.sql**

15. -- MySQL dump 10.13 Distrib 8.0.19, for osx10.14 (x86_64)

16. --


```
17.-- Host: 127.0.0.1    Database: world_x
18.-- -----
19.-- Server version 8.0.19-debug
```

...output omitted...

10. Add a shared volume to the file-sharing deployment. Use the following information to create the volume:

Field	Value
Name	shared-volume
Type	persistentVolumeClaim
Claim mode	rwo
Claim size	1Gi
Mount path	/home/sharedfiles
Claim class	nfs-storage
Claim name	shared-pvc

11. Next, connect to one of the file-sharing deployment pods, and use the `cp` command to copy the `/home/database-files/insertdata.sql` file to the `/home/sharedfiles` directory. Then, remove the `config-map-pvc` volume from the file-sharing deployment.

1. Add the `shared-volume` volume to the file-sharing deployment.

```
2. [student@workstation ~]$ oc set volume deployment/file-sharing \
3.   --add --name shared-volume --type persistentVolumeClaim \
4.   --claim-mode rwo --claim-size 1Gi \
5.   --mount-path /home/sharedfiles --claim-class nfs-storage \
6.   --claim-name shared-pvc
```

```
deployment.apps/file-sharing volume updated
```

7. Verify the deployment status. Your pod names might differ on your system.

```
8. [student@workstation ~]$ oc get pods
```

9. NAME	READY	STATUS	...
10. dbserver-5bc6bd5d7b-7z7lv	1/1	Running	...
11. file-sharing-65884f75bb-92fxf	1/1	Running	...

file-sharing-65884f75bb-gsghk	1/1	Running	...
-------------------------------	-----	---------	-----

12. Verify the volume status.

```
13. [student@workstation ~]$ oc get pvc
```

14. NAME	STATUS	VOLUME	CAPACITY	...
15. dbserver-lvm-pvc	Bound	pvc-2cb8...5025	1Gi	...

shared-pvc	Bound	pvc-cf2d...de48	1Gi	...
------------	-------	-----------------	-----	-----

16. Copy the /home/database-files/insertdata.sql file to the /home/sharedfiles path.

```
17. [student@workstation ~]$ oc exec -it \
```

```
18. pod/file-sharing-65884f75bb-92fxf -- \
```

```
cp /home/database-files/insertdata.sql /home/sharedfiles/
```

```
[student@workstation ~]$ oc exec -it \
```

```
pod/file-sharing-65884f75bb-92fxf -- \
```

```
ls /home/sharedfiles/
```

```
insertdata.sql
```

19. Remove the config-map-pvc volume from the file-sharing deployment.

```
20. [student@workstation ~]$ oc set volume deployment/file-sharing \
```

```
21. --remove --name=config-map-pvc
```

```
deployment.apps/file-sharing volume updated
```

12. Add the shared-volume PVC to the dbserver deployment. Then, connect to the dbserver deployment pod and verify the content of the /home/sharedfiles/insertdata.sql file.

1. Add the shared-volume volume to the dbserver deployment.

```
2. [student@workstation ~]$ oc set volume deployment/dbserver \  
3.   --add --name shared-volume \  
4.   --claim-name shared-pvc \  
5.   --mount-path /home/sharedfiles
```

```
deployment.apps/dbserver volume updated
```

6. Verify the deployment status. The pod names might differ on your system.

```
7. [student@workstation ~]$ oc get pods  
8. NAME                                READY   STATUS   ...  
9. dbserver-6676fbf5fc-n9hpk          1/1     Running  ...  
10. file-sharing-5fdb44cf57-2hhwj      1/1     Running  ...
```

```
file-sharing-5fdb44cf57-z4n7g    1/1     Running  ...
```

11. Verify the content of the /home/sharedfiles/insertdata.sql file.

```
12. [student@workstation ~]$ oc exec -it \  
13.   pod/dbserver-6676fbf5fc-n9hpk -- \  
14.   head /home/sharedfiles/insertdata.sql  
15. -- MySQL dump 10.13  Distrib 8.0.19, for osx10.14 (x86_64)  
16. --  
17. -- Host: 127.0.0.1    Database: world_x  
18. -- -----  
19. -- Server version 8.0.19-debug
```

```
...output omitted...
```

13. Connect to the database server and execute the /home/sharedfiles/insertdata.sql file to add data to the world_x database. You can execute the file by using the following command:

```
14.mysql -u$MYSQL_USER -p$MYSQL_PASSWORD world_x < \
```

```
/home/sharedfiles/insertdata.sql
```

Then, confirm connectivity between the web application and the database server by accessing the file-sharing route in a web browser.

1. Connect to the database server and execute the `/home/sharedfiles/insertdata.sql` file. Then, exit the database server.

```
[student@workstation ~]$ oc rsh dbserver-6676fbf5fc-n9hpk
sh-4.4$ mysql -u$MYSQL_USER -p$MYSQL_PASSWORD world_x < \
/home/sharedfiles/insertdata.sql

mysql: [Warning] Using a password on the command line interface can be insecure.

sh-4.4$ exit

exit
```

2. Test the connectivity between the web application and the database server. In a web browser, go to `http://file-sharing-storage-review.apps.ocp4.example.com`, and verify that the application retrieves data from the `world_x` database.

Connected successfully

1	Kabul	AFG Kabul	{"Population": 1780000}
2	Qandahar	AFG Qandahar	{"Population": 237500}
3	Herat	AFG Herat	{"Population": 186800}
4	Mazar-e-Sharif	AFG Balkh	{"Population": 127800}
5	Amsterdam	NLD Noord-Holland	{"Population": 731200}
6	Rotterdam	NLD Zuid-Holland	{"Population": 593321}

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade storage-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish storage-review
```

Summary

- Configuration maps are objects that provide mechanisms to inject configuration data into containers.
- The values in secrets are always encoded (not encrypted).
- A PVC resource represents a request from an application for storage, and specifies the essential storage characteristics, such as the capacity and access mode.
- Kubernetes supports two volume modes for persistent volumes: `Filesystem` and `Block`.
- Storage classes are a way to describe types of storage for the cluster and to provision dynamic storage on demand.
- A reclaim policy determines what happens to the data on a PV after the PVC is deleted.
- A storage class with block volume mode support can improve performance for applications that can use raw block devices.
- A stateful set is the representation of a set of pods with consistent identities.
- Each stateful set pod has its own unique PV.

Chapter 6. Configure Applications for Reliability

Application High Availability with Kubernetes

Guided Exercise: Application High Availability with Kubernetes

Application Health Probes

Guided Exercise: Application Health Probes

Reserving Compute Capacity for Applications

Guided Exercise: Reserving Compute Capacity for Applications

Limit Compute Capacity for Applications

Guided Exercise: Limit Compute Capacity for Applications

Application Autoscaling

Guided Exercise: Application Autoscaling

Lab: Configure an Application for Reliability

Quiz: Configure an Application for Reliability

Summary

Abstract

Goal	Configure applications to work with Kubernetes for high availability and resilience.
Sections	• Application High Availability with Kubernetes (and Guided Exercise) • Application Health Probes (and Guided Exercise) • Reserving Compute Capacity for Applications (and Guided Exercise) • Limit Compute Capacity for Applications (and Guided Exercise) • Application Autoscaling (and Guided Exercise)
Lab	• Configure an Application for Reliability

Application High Availability with Kubernetes

Objectives

- Describe how Kubernetes works to keep applications running after failures.

Concepts of Deploying Highly Available Applications

High availability (HA) is a goal of making applications more robust and resistant to runtime failures. Implementing HA techniques decreases the likelihood that an application is completely unavailable to users.

In general, HA can protect an application from failures in the following contexts:

- From itself in the form of application bugs
- From its environment, such as networking issues
- From other applications that exhaust cluster resources

Additionally, HA practices can protect the cluster from applications, such as one with a memory leak.

Writing Reliable Applications

At its core, cluster-level HA tooling mitigates worst-case scenarios. Although HA is not a substitute for fixing application-level issues, it augments developer mitigations. Applications must work with the cluster so that Red Hat OpenShift Container Platform can best handle failure scenarios.

Red Hat OpenShift Container Platform expects the following behaviors from applications:

- Tolerates restarts
- Responds to health probes, such as the startup, readiness, and liveness probes
- Supports multiple simultaneous instances
- Has well-defined and well-behaved resource usage
- Operates with restricted privileges

Although the cluster can run applications that lack the preceding behaviors, applications with these behaviors better use the reliability and HA features that Kubernetes provides.

Most HTTP-based applications provide an endpoint to verify application health. The cluster can be configured to observe this endpoint and mitigate potential issues for the application.

The application is responsible for providing such an endpoint. Developers must decide how the application determines its state.

For example, if an application depends on a database connection, then the application might respond with a healthy status only when the database is reachable. However, not all applications that make database connections need such a check. This decision is at the discretion of the developers.

Kubernetes Application Reliability

If an application pod crashes, then it cannot respond to requests. Depending on the configuration, the cluster can automatically restart the pod. If the application fails without crashing the pod, then the pod continues to run but does not receive requests. The cluster can detect and act on a running but unhealthy pod only with the appropriate health probes.

Kubernetes uses the following HA techniques to improve application reliability:

- Restarting pods: By configuring a restart policy on a pod, the cluster restarts misbehaving instances of an application.
- Probing: By using health probes, the cluster detects when applications cannot respond to requests, and can automatically act to mitigate the issue.
- Horizontal scaling: When the application load changes, the cluster can scale the number of replicas to match the load.

These techniques are explored throughout this chapter.

Restarting Pods

A container in a pod can terminate for various reasons. The `restartPolicy` field of a pod specification controls whether the kubelet restarts the containers in that pod. The possible values are `Always`, `OnFailure`, and `Never`. The default value is `Always`.

Policy	Description
Always	The kubelet always restarts a terminated container. This policy is the default.
OnFailure	The kubelet restarts a terminated container only if it exited with a non-zero exit code.

Policy	Description
Never	The kubelet never restarts a terminated container.

Guided Exercise: Application High Availability with Kubernetes

Simulate different types of application failures and observe how Kubernetes handles them.

Outcomes

- Explore how the `restartPolicy` attribute affects crashing pods.
- Observe the behavior of a slow-starting application that has no configured probes.
- Use a deployment to scale the application, and observe the behavior of a broken pod.

As the `student` user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the following conditions are true:

- The `reliability-ha` project exists.
- The resource files are available in the course directory.
- The classroom registry has the `long-load` container image.

The `long-load` container image contains an application with utility endpoints. These endpoints perform such tasks as crashing the process and toggling the server's health status.

```
[student@workstation ~]$ lab start reliability-ha
```

Instructions

1. As the `developer` user, create a pod from a YAML manifest in the `reliability-ha` project.

1. Log in to the OpenShift cluster as the developer user with developer as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
5.
```

...output omitted...

6. Select the reliability-ha project.

```
7. [student@workstation ~]$ oc project reliability-ha
```

Now using project "reliability-ha" on server "https://api.ocp4.example.com:6443".

8. Go to the lab materials directory and view the contents of the pod definition. In particular, the manifest sets restartPolicy to Always.

```
[student@workstation ~]$ cd D0180/labs/reliability-ha
[student@workstation reliability-ha]$ cat long-load.yaml
apiVersion: v1
kind: Pod
metadata:
  name: long-load
spec:
  containers:
  - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1
    name: long-load
    securityContext:
      allowPrivilegeEscalation: false
  restartPolicy: Always
```

9. Create a pod by using the oc apply command.

```
10. [student@workstation reliability-ha]$ oc apply -f long-load.yaml
```

```
pod/long-load created
```

11. Send a request to the pod to confirm that it is running and responding.

```
12. [student@workstation reliability-ha]$ oc exec long-load -- \
13. curl -s localhost:3000/health
```

```
Ok
```

2. Trigger the pod to crash, and observe that the restartPolicy instructs the cluster to re-create the pod.

1. Observe that the pod is running and did not restart.

```
2. [student@workstation reliability-ha]$ oc get pods
3.
```

4. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

long-load	1/1	Running	0	1m
-----------	-----	---------	---	----

5. Send a request to the /destruct endpoint in the application. This request triggers the process to crash.

```
6. [student@workstation reliability-ha]$ oc exec long-load -- \
7. curl -s localhost:3000/destruct
```

```
command terminated with exit code 52
```

8. Observe that the pod is running and restarted one time.

```
9. [student@workstation reliability-ha]$ oc get pods
10. NAME          READY   STATUS    RESTARTS   AGE
```

long-load	1/1	Running	1 (34s ago)	4m16s
-----------	-----	---------	-------------	-------

11. Delete the long-load pod.

```
12. [student@workstation reliability-ha]$ oc delete pod long-load
```

```
pod "long-load" deleted
```

The cluster does not re-create the pod, because you manually created the pod outside of a workload resource, such as a deployment.

3. Use a restart policy of `Never` to create the pod, and observe that the cluster does not re-create the pod on crashing.

1. Modify the `long-load.yaml` file so that the `restartPolicy` is set to `Never`.

2. ...*output omitted*...

```
restartPolicy: Never
```

3. Create the pod with the updated YAML file.

4. [student@workstation reliability-ha]\$ **oc apply -f long-load.yaml**

```
pod/long-load created
```

5. Send a request to the pod to confirm that the pod is running and that the application is responding.

6. [student@workstation reliability-ha]\$ **oc exec long-load -- **

7. **curl -s localhost:3000/health**

```
Ok
```

8. Send a request to the `/destruct` endpoint in the application to crash it.

9. [student@workstation reliability-ha]\$ **oc exec long-load -- **

10. **curl -s localhost:3000/destruct**

```
command terminated with exit code 52
```

11. Observe that the cluster does not restart the pod and is in an error state.

12. [student@workstation reliability-ha]\$ **oc get pods**

13. NAME READY STATUS RESTARTS AGE

long-load	0/1	Error	0	2m36s
-----------	-----	--------------	---	-------

14. Delete the `long-load` pod.

```
15. [student@workstation reliability-ha]$ oc delete pod long-load
```

```
pod "long-load" deleted
```

4. Because the cluster does not know when the application inside the pod is ready to receive requests, you must add a startup delay to the application. A later exercise covers adding this capability by using probes.

1. Update the `long-load.yaml` file by adding a startup delay and use a restart policy of `Always`. Set the `START_DELAY` variable to 60,000 milliseconds (one minute) so that the file exactly matches the following excerpt:

```
2. ...output omitted...
3. spec:
4.   containers:
5.     - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1
6.       imagePullPolicy: Always
7.       securityContext:
8.         allowPrivilegeEscalation: false
9.       name: long-load
10.      env:
11.        - name: START_DELAY
12.          value: "60000"
```

```
restartPolicy: Always
```

NOTE

Although numbers are a valid YAML type, you must pass environment variables as strings. YAML syntax is also indentation-sensitive.

For these reasons, ensure that your file appears *exactly* as in the preceding example.

13. Apply the YAML file to create the pod and proceed within one minute to the next step.

```
14. [student@workstation reliability-ha]$ oc apply -f long-load.yaml
```

```
pod/long-load created
```

15. Within one minute of pod creation, verify the status of the pod. The status shows as Running even though it is not. Try to send a request to the application, and observe that it fails.

```
16. [student@workstation reliability-ha]$ oc get pods
```

```
17. NAME          READY   STATUS    RESTARTS   AGE
```

```
long-load    1/1     Running   0           16s
```

```
[student@workstation reliability-ha]$ oc exec long-load -- \
```

```
curl -s localhost:3000/health
```

```
app is still starting
```

18. After waiting one minute for the application to start, send another a request to the pod to confirm that it is running and responding.

```
19. [student@workstation reliability-ha]$ oc exec long-load -- \
```

```
20. curl -s localhost:3000/health
```

```
Ok
```

5. Use a deployment to scale up the number of deployed pods. Observe that deleting the pods causes service outages, even though the deployment handles re-creating the pods.
1. Review the long-load-deploy.yaml file, which defines a deployment, service, and route. The deployment creates three replicas of the application pod.

In each pod, the deployment file sets a `START_DELAY` environment variable to 15,000 milliseconds (15 seconds). In each pod, the application responds that it is not ready until after the delay.

```
[student@workstation reliability-ha]$ cat long-load-deploy.yaml
```

```
...output omitted...
```

```
spec:
```

```
  containers:
```

```
    - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1
```

```
      imagePullPolicy: Always
```

```
name: long-load
env:
  - name: START_DELAY
    value: "15000"
...output omitted...
```

2. Start the load test script, which sends a request to the /health API endpoint of the application every two seconds. Leave the script running in a visible terminal window.

```
3. [student@workstation reliability-ha]$ ./load-test.sh
```

```
...output omitted...
```

4. In a new terminal window, apply the ~/D0180/labs/reliability-ha/long-load-deploy.yaml file.

```
5. [student@workstation ~]$ oc apply -f \
6. ~/D0180/labs/reliability-ha/long-load-deploy.yaml
7. deployment.apps/long-load created
8. service/long-load created
```

```
route.route.openshift.io/long-load created
```

9. Watch the output of the load test script as the pods and the application instances start. After a delay, the requests succeed.

```
10. ...output omitted...
11. Ok
12. Ok
13. Ok
```

```
...output omitted...
```

14. By using the /togglesick API endpoint of the application, put one of the three pods into a broken state.

```
15. [student@workstation ~]$ curl \
```

```
16. long-load-reliability-ha.apps.ocp4.example.com/togglesick
```

no output expected

17. Watch the output of the load test script as some requests start failing.
Because of the load balancer, the exact order of the output is random.

```
18. ...output omitted...
19. Ok
20. app is unhealthy
21. app is unhealthy
22. Ok
23. Ok
```

...output omitted...

Press **Ctrl+C** to end the load test script.

24. Return to the `/home/student/` directory.

```
25. [student@workstation reliability-ha]$ cd /home/student/
```

```
[student@workstation ~]$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish reliability-ha
```

Application Health Probes

Objectives

- Describe how Kubernetes uses health probes during deployment, scaling, and failover of applications.

Kubernetes Probes

Health probes are an important part of maintaining a robust cluster. Health probes enable the cluster to determine the status of an application by repeatedly probing it for a response.

A set of health probes affect a cluster's ability to do the following tasks:

- Mitigate crashes by automatically attempting to restart failing pods.
- Manage failover and load balancing by sending requests only to healthy pods.
- Monitor pods by determining whether and when they are failing.
- Scale applications by determining when a new replica is ready to receive requests.

Authoring Probe Endpoints

Application developers code health probe endpoints during application development. These endpoints determine the health and status of the application. For example, a data-driven application might report a successful health probe only if it can connect to the database.

Health probe endpoints must perform quickly because the cluster calls them often. Endpoints must not perform complicated database queries or many network calls.

Probe Types

Kubernetes provides the following types of probes: startup, readiness, and liveness. Depending on the application, you might configure one or more of these types.

Startup Probes

A *startup probe* determines when an application's startup process is complete. Unlike a liveness probe, a startup probe is not called after the probe succeeds. If the startup probe does not succeed after a configurable timeout, then the pod restarts based on its `restartPolicy` value.

Add a startup probe to applications that have a long start time. A startup probe enables the liveness probe to remain short and responsive.

Readiness Probes

A *readiness probe* determines whether the application is ready to serve requests. If the readiness probe fails, then Kubernetes removes the pod's IP address from the service resource to prevent client traffic from reaching the application.

Readiness probes help to detect temporary issues that might affect your applications. For example, the application might be temporarily unavailable when it starts, because it must establish initial network connections, load files in a cache, or perform initial tasks that take time to complete. The application might occasionally need to run long batch jobs, which make it temporarily unavailable to clients.

Kubernetes continues to run the probe even after the application fails a check. If the probe succeeds again, then Kubernetes adds back the pod's IP address to the service resource, and requests are sent to the pod again.

In such cases, the readiness probe addresses a temporary issue and improves application availability.

Liveness Probes

Like a readiness probe, a *liveness probe* is called throughout the lifetime of the application. Liveness probes determine whether the application container is in a healthy state. If an application fails its liveness probe enough times, then the cluster restarts the pod according to its restart policy.

Unlike a startup probe, liveness probes run after the application's initial startup process is complete. Usually, this mitigation is by restarting or re-creating the pod.

Types of Tests

When defining a probe, you must specify one of the following types of test to perform:

HTTP GET

Each time that the probe runs, the cluster sends an HTTP GET request to the specified HTTP endpoint. The test is considered a success if the request responds with an HTTP response code between 200 and 399. Other responses cause the test to fail.

Container command

Each time that the probe runs, the cluster runs the specified command in the container. If the command exits with a status code of 0, then the test succeeds. Other status codes cause the test to fail.

TCP socket

Each time that the probe runs, the cluster attempts to open a socket to the container on the specified port. The test succeeds only if the connection is established.

Timings and Thresholds

All the types of probes include timing variables. The `period seconds` variable defines how often, in seconds, the probe runs. The `failure threshold` defines how many failed attempts are required before the probe itself fails.

For example, a probe with a failure threshold of 3 and period seconds of 5 can fail up to three times before the overall probe fails. With this configuration, an issue can persist for at least 10 seconds before mitigation begins. Running probes too often wastes cluster resources. Balance these values when you configure probes.

Adding Probes by Using a YAML File

Because probes are defined on a pod template, you can add probes to workload resources such as deployments. To add a probe to an existing deployment, update and apply its YAML file or use the `oc edit` command.

The following YAML excerpt defines a deployment pod template with a liveness probe:

```
apiVersion: apps/v1
kind: Deployment
...output omitted...
spec:
...output omitted...
  template:
    spec:
```

```
containers:
- name: web-server
  ...output omitted...

livenessProbe:

  failureThreshold: 6

  periodSeconds: 10

  httpGet:

    path: /health

    port: 3000
```

Defines a liveness probe.

Specifies how many times the probe must fail before mitigating.

Defines how often the probe runs.

Sets the probe as an HTTP request and defines the request port and path.

Specifies the HTTP path to send the request to.

Specifies the port to send the HTTP request over.

Adding Probes via the CLI

Use the `oc set probe` command to add or modify a probe on a resource. The following command adds a readiness probe to a deployment named `front-end`:

```
[user@host ~]$ oc set probe deployment/front-end \

--readiness \

--failure-threshold 6 \

--period-seconds 10 \
```

```
--get-url http://:8080/healthz
```

Defines a readiness probe.

Sets how many times the probe must fail before mitigating.

Sets how often the probe runs.

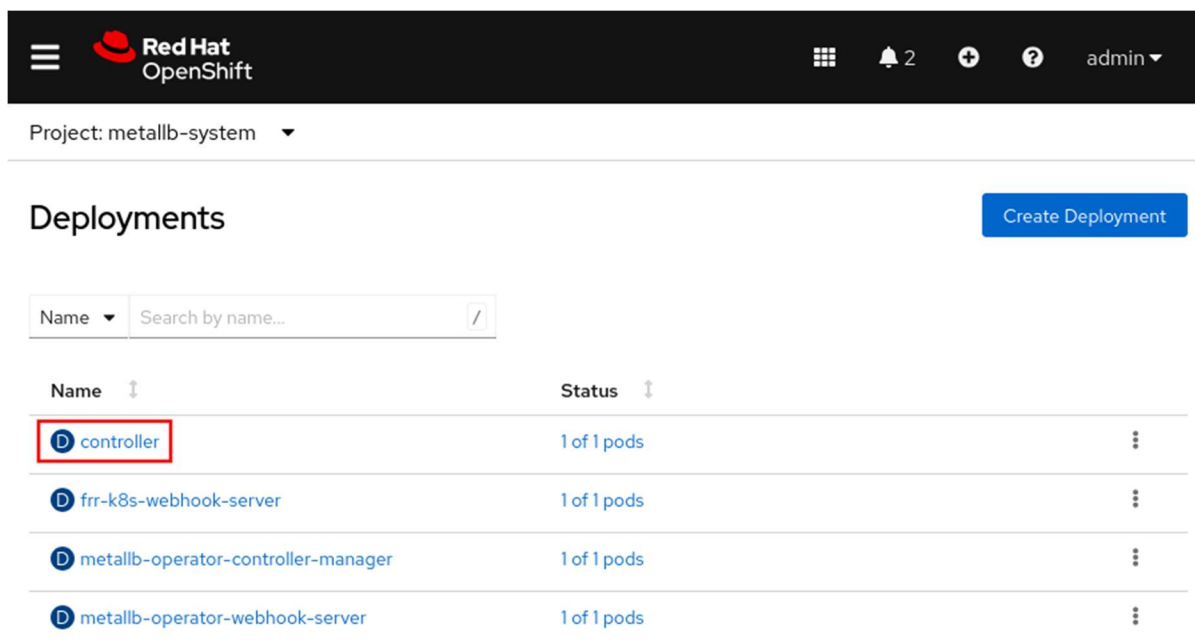
Sets the probe as an HTTP request, and defines the request port and path.

NOTE

The `set probe` command is exclusive to RHOCP and `oc`.

Adding Probes via the Web Console

To add or modify a probe on a deployment from the web console, go to the **Workloads** → **Deployments** menu and select a deployment.



Red Hat OpenShift

Project: metallb-system

Deployments

Create Deployment

Name Search by name...

Name	Status
controller	1 of 1 pods
frr-k8s-webhook-server	1 of 1 pods
metallb-operator-controller-manager	1 of 1 pods
metallb-operator-webhook-server	1 of 1 pods

Click **Actions** and click **Add Health Checks**.

Red Hat OpenShift

Project: metallb-system

Deployments > Deployment details

D controller
Managed by **MLB** metallb

Details Metrics YAML ReplicaSets Pods Environment Events

Deployment details

1 Pod

Name
controller

Update strategy
RollingUpdate

Namespace

Max unavailable

Actions

- Edit Pod count
- Add PodDisruptionBudget
- Pause rollouts
- Restart rollout
- Add Health Checks
- Add storage
- Edit update strategy
- Edit resource limits
- Edit labels
- Edit annotations

Click **Edit probe** to specify the readiness type, the HTTP headers, the path, the port, and more.

Project: metallb-system ▾

Add health checks [Learn more](#)

Health checks for **controller**

Container **controller** ▾

Readiness probe [Edit Probe](#)

A readiness probe checks if the Container is ready to handle requests. A failed readiness probe means that a Container should not receive any traffic from a proxy, even if it's running.

✓ Readiness probe added

Liveness probe [Edit Probe](#)

A liveness probe checks if the Container is still running. If the liveness probe fails the Container is killed.

Guided Exercise: Application Health Probes

Configure health probes in a deployment and verify that network clients are insulated from application failures.

Outcomes

- Assess potential issues with an application that is not configured with health probes.
- Configure startup, liveness, and readiness probes for the application.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the following conditions are true:

- The `reliability-probes` project exists.
- The resource files are available in the course directory.
- The classroom registry has the `long-load` container image.

The registry.ocp4.example.com:8443/redhattraining/long-load:v1 container image contains an application with utility endpoints. These endpoints perform such tasks as crashing the process and toggling the server's health status.

```
[student@workstation ~]$ lab start reliability-probes
```

Instructions

1. As the developer user, deploy the long-load application in the reliability-probes project.
1. Log in to the OpenShift cluster as the developer user with developer as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Set the reliability-probes project as the active project.

```
6. [student@workstation ~]$ oc project reliability-probes
```

Now using project reliability-probes on server "https://api.ocp4.example.com:6443".

7. Apply the long-load-deploy.yaml file to create the pod. Move to the next step within one minute.

```
8. [student@workstation ~]$ oc apply -f \
9. ~/D0180/labs/reliability-probes/long-load-deploy.yaml
10. deployment.apps/long-load created
11. service/long-load created
```

route.route.openshift.io/long-load created

12. Verify that the pods take several minutes to start by sending a request to a pod in the deployment.

```
13. [student@workstation ~]$ oc exec deploy/long-load -- \
```



```
14. curl -s localhost:3000/health
```

```
app is still starting
```

15. Observe that the pods are ready even though the application is not ready.

```
16. [student@workstation ~]$ oc get pods
```

17. NAME	READY	STATUS	RESTARTS	AGE
18. long-load-8564d998cc-579nx	1/1	Running	0	30s
19. long-load-8564d998cc-ttqpg	1/1	Running	0	30s

long-load-8564d998cc-wjtfw	1/1	Running	0	30s
----------------------------	-----	---------	---	-----

2. Add a startup probe to the pods so that the cluster knows when the pods are ready.

1. Modify the deployment in the ~/D0180/labs/reliability-probes/long-load-deploy.yaml YAML file by defining a startup probe. The probe runs every three seconds and triggers a pod as failed after 30 failed attempts. The file should match the following excerpt:

```
2. ...output omitted...
3. spec:
4.   ...output omitted...
5.   template:
6.     ...output omitted...
7.     spec:
8.       containers:
9.       - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1
10.         imagePullPolicy: Always
11.         name: long-load
12.         startupProbe:
13.           failureThreshold: 30
14.           periodSeconds: 3
15.           httpGet:
16.             path: /health
17.             port: 3000
```

18. env:

...output omitted...

19. Apply the updated long-load-deploy.yaml file.

```
20. [student@workstation ~] oc apply -f \
21. ~/D0180/labs/reliability-probes/long-load-deploy.yaml
22. deployment.apps/long-load configured
23. service/long-load unchanged
```

route.route.openshift.io/long-load configured

24. Observe that the pods do not show as ready until the application is ready and the startup probe succeeds. Wait for the three pods to reach the ready state. Press **Ctrl+c** to stop the watch command.

```
25. [student@workstation ~]$ watch oc get pods
26. Every 2.0s: oc get pods
27.
28. NAME                                READY   STATUS    RESTARTS   AGE
29. long-load-785b5b4fc8-7x5ln         1/1     Running   0           90s
30. long-load-785b5b4fc8-f7pdk         1/1     Running   0           90s
```

```
long-load-785b5b4fc8-r2nqj    1/1     Running   0           90s
```

3. Add a liveness probe so that the cluster restarts broken instances of the application.

1. Start the load test script.

```
2. [student@workstation ~]$ ~/D0180/labs/reliability-probes/load-test.sh
3. Ok
4. Ok
5. Ok
```

...output omitted...

Keep the script running in a visible window.

6. In a new terminal window, use the `/togglesick` endpoint to make one of the pods unhealthy. Move to the next step within one minute.

```
7. [student@workstation ~]$ oc exec \  
8.   deploy/long-load -- curl -s localhost:3000/togglesick
```

no output expected

The load test window begins to show app is unhealthy. Because only one pod is unhealthy, the remaining pods still respond with ok.

9. Update the deployment in the `~/D0180/labs/reliability-probes/long-load-deploy.yaml` file to add a liveness probe. The probe runs every three seconds and triggers the pod as failed after three failed attempts. Modify the `spec.template.spec.containers` object in the file to match the following excerpt.

```
10. spec:  
11.   ...output omitted...  
12.   template:  
13.     ...output omitted...  
14.     spec:  
15.       containers:  
16.         - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1  
17.           ...output omitted...  
18.           startupProbe:  
19.             failureThreshold: 30  
20.             periodSeconds: 3  
21.             httpGet:  
22.               path: /health  
23.               port: 3000  
24.           livenessProbe:  
25.             failureThreshold: 3  
26.             periodSeconds: 3  
27.             httpGet:  
28.               path: /health  
29.               port: 3000
```

```
30.          env:
```

```
...output omitted...
```

31. Scale down the deployment to zero replicas.

```
32. [student@workstation ~]$ oc scale deployment/long-load --replicas 0
```

```
deployment.apps/long-load scaled
```

The load test script shows that the application is not available.

NOTE

Red Hat recommends scaling down an application to zero replicas before making multiple changes to a deployment, such as adding health probes, resource requests, or environment variables. Scaling down to zero replicas prevents creating any pods, and enables pods to terminate gracefully after finishing all current requests.

For multiple deployment updates, scaling down to zero prevents a noisy event log from Kubernetes responding to every change, and conserves cluster resources.

33. Apply the updated `long-load-deploy.yaml` file to update the deployment, which triggers the deployment to re-create its pods.

```
34. [student@workstation ~]$ oc apply -f \  
35. ~/D0180/labs/reliability-probes/long-load-deploy.yaml  
36. deployment.apps/long-load configured  
37. service/long-load unchanged
```

```
route.route.openshift.io/long-load unchanged
```

38. Wait for the three new pods to reach the ready state. Press **Ctrl+c** to stop the watch command.

```
39. [student@workstation ~]$ watch oc get pods  
40. Every 2.0s: oc get pods  
41.
```

42. NAME	READY	STATUS	RESTARTS	AGE
43. long-load-785b5b4fc8-8x5ln	1/1	Running	0	70s
44. long-load-785b5b4fc8-f8pdk	1/1	Running	0	70s

long-load-785b5b4fc8-r9nqj	1/1	Running	0	70s
----------------------------	-----	---------	---	-----

45. Wait for the load test window to show ok for all responses, and then toggle one of the pods to be unhealthy.

```
46. [student@workstation ~]$ oc exec \
```

```
47.   deploy/long-load -- curl -s localhost:3000/togglesick
```

no output expected

The load test window might show app is unhealthy several times before the cluster restarts the pod.

48. Observe that the cluster restarts the unhealthy pod after the liveness probe fails. After the pod is restarted, the load test window shows only ok.

```
49. [student@workstation ~]$ watch oc get pods
```

50. NAME	READY	STATUS	RESTARTS	AGE
51. long-load-fbb7468d9-8xm8j	1/1	Running	0	9m42s
52. long-load-fbb7468d9-k66dm	1/1	Running	0	8m38s

long-load-fbb7468d9-ncxkh	0/1	Running	1 (11s ago)	10m
---------------------------	-----	---------	-------------	-----

4. Add a readiness probe so that traffic goes only to pods that are ready and healthy.

1. Scale down the deployment to zero replicas.

```
2. [student@workstation ~]$ oc scale deploy/long-load --replicas 0
```

deployment.apps/long-load scaled

3. Use the `oc set probe` command to add the readiness probe.

```
4. [student@workstation ~]$ oc set probe deploy/long-load \
```

```
5.   --readiness --failure-threshold 1 --period-seconds 3 \
```

```
6. --get-url http://:3000/health
```

```
deployment.apps/long-load probes updated
```

7. Scale up the deployment to three replicas.

```
8. [student@workstation ~]$ oc scale deploy/long-load --replicas 3
```

```
deployment.apps/long-load scaled
```

9. Observe the status of the pods by using the `watch` command.

```
10. [student@workstation ~]$ watch oc get pods
```

11. NAME	READY	STATUS	RESTARTS	AGE
12. long-load-d5794d744-8hqlh	0/1	Running	0	48s
13. long-load-d5794d744-hphgb	0/1	Running	0	48s

long-load-d5794d744-lgkns	0/1	Running	0	48s
---------------------------	-----	---------	---	-----

The command does not immediately finish, but continues to show updates to the pods' status. Leave this command running in a visible window.

14. Wait for the pods to show as ready. Then, in a new terminal window, make one of the pods unhealthy for five seconds by using the `/hiccup` endpoint.

```
15. [student@workstation ~]$ oc exec \
```

```
16.  deploy/long-load -- curl -s localhost:3000/hiccup?time=5
```

no output expected

The pod status window shows that one of the pods is no longer ready.

```
[student@workstation ~]$ watch oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
long-load-d5794d744-8hqlh	1/1	Running	0	83s
long-load-d5794d744-hphgb	0/1	Running	0	83s
long-load-d5794d744-lgkns	1/1	Running	0	83s

After five seconds, the pod is healthy again and shows as ready.

The load test window might show `app` is unhealthy one time before the pod is set as not ready. After the cluster determines that the pod is no longer ready, it stops sending traffic to the pod until either the pod recovers or the liveness probe fails. Because the pod is sick for only five seconds, it is enough time for the readiness probe to fail, but not the liveness probe.

NOTE

Optionally, repeat this step and observe as the temporarily sick pod's status changes.

17. Stop the load test and status commands by pressing **Ctrl+c** in their respective windows.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish reliability-probes
```

Reserving Compute Capacity for Applications

Objectives

- Configure an application with resource requests so Kubernetes can make scheduling decisions.

Kubernetes Pod Scheduling

The Red Hat OpenShift Container Platform pod scheduler determines the placement of new pods onto nodes in the cluster. Appropriate scheduling prevents resource contention and places workloads on nodes that meet their specific requirements. The pod scheduler algorithm follows a three-step process:

Filtering nodes

A pod can define a node selector that matches the labels in the cluster nodes. The scheduler considers only nodes with matching labels as eligible.

Additionally, the scheduler filters the list of running nodes by evaluating each node against a set of predicates. A pod can define resource requests for compute resources such as CPU, memory, and storage. Only nodes with enough available computer resources are eligible.

The filtering step reduces the list of eligible nodes. In some cases, the pod could run on any of the nodes. In other cases, all the nodes are filtered out. The pod cannot be scheduled until a node with the appropriate prerequisites becomes available.

If all nodes are filtered out, then a `FailedScheduling` event is generated for the pod.

Prioritizing nodes

By using multiple priority criteria, the scheduler determines a weighted score for each node. Nodes with higher scores are better candidates to run the pod.

Selecting a node

The scheduler sorts the candidate list according to these scores, and selects the node with the highest score. If multiple nodes have the same high score, then one node is selected in a round-robin fashion. After a host is selected, the cluster generates a `Scheduled` event for the pod.

NOTE

The scheduler is flexible and can be customized for advanced scheduling situations. Customizing the scheduler is outside the scope of this course.

Configuring Compute Resource Requests

For applications that require a specific amount of compute resources, you can define a resource request in the pod definition. The resource requests assign hardware resources for the application deployment.

Resource requests specify the minimum required compute resources to schedule a pod. The scheduler tries to find a node with enough available compute resources to satisfy the pod requests.

In Kubernetes, memory resources are measured in bytes, and CPU resources are measured in CPU units. CPU units are allocated by using millicore units. A millicore is a CPU core, either virtual or physical, that is split into 1000 units. A request value of "1000 m" allocates an entire CPU core to a pod. You can also use fractional values to allocate CPU resources. For example, you can set the CPU resource request to the 0.1 value, which represents 100 millicores (100 m). Likewise, a CPU resource request with the 1.0 value represents an entire CPU or 1000 millicores (1000 m).

You can define resource requests for each container in a deployment resource. If you do not define resources, then the container specification shows a `resources: {}` line.

To specify requests, modify the `resources: {}` section in the deployment manifest. The following example defines a resource request of 100 millicores (100m) of CPU and one gibibyte (1Gi) of memory.

```
...output omitted...
spec:
  containers:
  - image: quay.io/redhattraining/hello-world-nginx:v1.0
    name: hello-world-nginx
    resources:
      requests:
        cpu: "100m"
        memory: "1Gi"
```

If you use the `oc edit` command to modify a deployment, then ensure that you use the correct indentation. Indentation mistakes can result in the editor not saving changes. Alternatively, use the `oc set resources` command to specify resource requests. The following command sets the same requests as the preceding example:

```
[user@host ~]$ oc set resources deployment hello-world-nginx \
```

```
--requests cpu=100m,memory=1gi
```

The `oc set resources` command works with any resource that includes a pod template, such as Deployment and Job resources.

Inspecting Cluster Compute Resources

Cluster administrators can view the available and used compute resources of a node. As a cluster administrator, you can use the `oc describe node` command to determine the compute resource capacity of a node. The command shows the capacity of the node and how much of that capacity is allocatable. It also displays the amount of allocated and requested resources on the node.

```
[user@host ~]$ oc describe node master01
Name:                master01
Roles:               control-plane,master,worker
...output omitted...
Capacity:
  cpu:                8
  ephemeral-storage:  125293548Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             20531668Ki
  pods:               250
Allocatable:
  cpu:                7500m
  ephemeral-storage:  114396791822
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             19389692Ki
  pods:               250
...output omitted...
Non-terminated Pods: (88 in total)
... Name            CPU Requests  CPU Limits   Memory Requests  Memory Limits ...
... ----            -
```

```
... controller-... 10m (0%)      0 (0%)      20Mi (0%)      0 (0%)      ...
... metallb-...   50m (0%)      0 (0%)      20Mi (0%)      0 (0%)      ...
... metallb-...   0 (0%)        0 (0%)      0 (0%)         0 (0%)      ...
```

Allocated resources:

(Total limits may be over 100 percent, i.e., overcommitted.)

Resource	Requests	Limits
-----	-----	-----
cpu	3183m (42%)	1202m (16%)
memory	12717Mi (67%)	1350Mi (7%)

...output omitted...

RHOC cluster administrators can also use the `oc adm top pods` command. This command shows the compute resource usage for each pod in a project. You must include the `--namespace` or `-n` options to specify a project. If you do not specify a project, then the command returns the resource usage for pods in the current project.

The following command displays the resource usage for pods in the `openshift-dns` project:

```
[user@host ~]$ oc adm top pods -n openshift-dns
```

NAME	CPU(cores)	MEMORY(bytes)
dns-default-5kpn5	1m	33Mi
node-resolver-6kdxp	0m	2Mi

Additionally, cluster administrators can use the `oc adm top nodes` command to view the resource usage of cluster nodes. You can include the node name to view the resource usage of a particular node.

```
[user@host ~]$ oc adm top node master01
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
master01	1250m	16%	10268Mi	54%

Guided Exercise: Reserving Compute Capacity for Applications

Configure an application with compute resource requests that allow and prevent successful scheduling and scaling of its pods.

Outcomes

- Inspect how memory resource requests allocate cluster node memory.
- Explore how adjusting resource requests impacts the number of replicas that the cluster can schedule on a node.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the following conditions are true:

- The `reliability-requests` project exists.
- The resource files are available in the course directory.
- The classroom registry has the `registry.ocp4.example.com:8443/redhattraining/long-load:v1` container image.

The `registry.ocp4.example.com:8443/redhattraining/long-load:v1` container image contains an application with utility endpoints. These endpoints perform such tasks as crashing the process and toggling the server's health status.

```
[student@workstation ~]$ lab start reliability-requests
```

Instructions

1. As the `admin` user, deploy the `long-load` application by applying the `long-load-deploy.yaml` file in the `reliability-requests` project.
1. Log in to the OpenShift cluster as the `admin` user with `redhatocp` as the password.

2. `[student@workstation ~]$ oc login -u admin -p redhatocp \`
3. `https://api.ocp4.example.com:6443`
4. Login successful.

...output omitted...

NOTE

In general, use accounts with the least required privileges to perform a task. In the classroom environment, this account is the `developer` user. However, you need cluster administrator privileges to view the cluster node metrics in this exercise.

5. View the total memory request allocation for the node.

```
6. [student@workstation ~]$ oc describe node master01
7. ...output omitted...
8. Allocated resources:
9. (Total limits may be over 100 percent, i.e., overcommitted.)
10. Resource           Requests          Limits
11. -----
12.  cpu                 3158m (42%)      980m (13%)
13.  memory              12667Mi (66%)    1250Mi (6%)
```

...output omitted...

The command output shows that the pods that are currently running on the node requested a total of 12667 MiB of memory. That value might be slightly different on your system.

IMPORTANT

Projects and objects from previous exercises can cause the memory usage from this exercise to mismatch the intended results. Delete any unrelated projects before continuing.

If you still experience issues, then re-create your classroom environment and try this exercise again.

14. Select the `reliability-requests` project.

```
15. [student@workstation ~]$ oc project reliability-requests
```

Now using project "reliability-requests" on server "https://api.ocp4.example.com:6443".

16. Go to the `~/D0180/labs/reliability-requests` directory. Create a deployment, a service, and a route by using the `oc apply` command and the `long-load-deploy.yaml` file.

```
17. [student@workstation ~]$ cd D0180/labs/reliability-requests
18. [student@workstation reliability-requests]$ oc apply -f long-load-deploy.yaml
19. deployment.apps/long-load created
20. service/long-load created
```

```
route.route.openshift.io/long-load created
```

2. Add a resource request to the pod definition and scale the deployment beyond the cluster's capacity.
1. Modify the `long-load-deploy.yaml` file by adding a resource request. The request allocates one gibibyte (1 GiB) to each of the application pods.

```
2. spec:
3.   ...output omitted...
4.   template:
5.     ...output omitted...
6.     spec:
7.       containers:
8.       - image: registry.ocp4.example.com:8443/redhattraining/long-load:v1
9.         resources:
10.          requests:
11.            memory: 1Gi
```

```
...output omitted...
```

12. Apply the YAML file to modify the deployment with the resource request.

```
13. [student@workstation reliability-requests]$ oc apply -f long-load-deploy.yaml
14. deployment.apps/long-load configured
15. service/long-load unchanged
```

```
route.route.openshift.io/long-load unchanged
```

16. Scale the deployment to have 10 replicas.

```
17. [student@workstation reliability-requests]$ oc scale deploy/long-load \
18.   --replicas 10
```

```
deployment.apps/long-load scaled
```

19. Observe that the cluster cannot schedule all pods on the single node. The cluster cannot schedule pods with the Pending status.

```
20. [student@workstation reliability-requests]$ oc get pods
21. NAME                                READY   STATUS    RESTARTS   AGE
22. ...output omitted...
23. long-load-86bb4b79f8-44zwd         0/1     Pending   0           58s
```

```
...output omitted...
```

24. Retrieve the cluster event log, and observe that insufficient memory is the cause of the failed scheduling.

```
25. [student@workstation reliability-requests]$ oc get events \
26.   --field-selector reason="FailedScheduling"
```

```
... pod/long-load-...    0/1 nodes are available: 1 Insufficient memory. ...
```

Alternatively, view the events for a pending pod to see the reason. In the following command, replace the pod name with one of the pending pods in your classroom.

```
[student@workstation reliability-requests]$ oc describe \
  pod/long-load-86bb4b79f8-44zwd
...output omitted...
Events:
...output omitted... 0/1 nodes are available: 1 Insufficient memory. ...
```

27. Observe that the node's requested memory usage is high.

```
28. [student@workstation reliability-requests]$ oc describe node master01
29. ...output omitted...
```

30. Allocated resources:

31. (Total limits may be over 100 percent, i.e., overcommitted.)

32. Resource	Requests	Limits
33. -----	-----	-----
34. cpu	3158m (42%)	980m (13%)
35. memory	18811Mi (99%)	1250Mi (6%)

...output omitted...

The command output shows that the pods from the long-load deployment requested most of the remaining memory from the node. However, not enough memory is available to accommodate the 10 replicas.

3. Reduce the requested memory per pod so that the replicas can run on the node.

1. Manually set the resource request to 250Mi.

2. [student@workstation reliability-requests]\$ **oc set resources deploy/long-load **

3. **--requests memory=150Mi**

deployment.apps/long-load resource requirements updated

4. Delete the pods so that the deployment controller re-creates the pods with the new resource request.

5. [student@workstation reliability-requests]\$ **oc delete pod -l app=long-load**

6. pod "long-load-557b4d94f5-29brx" deleted

...output omitted...

7. Observe that all pods can start with the lowered memory request. Within one minute, the cluster marks the pods as Ready and places them in the Running state, with no pods in the Pending status.

8. [student@workstation reliability-requests]\$ **oc get pods**

9. NAME	READY	STATUS	RESTARTS	AGE
10. long-load-557b4d94f5-68hbb	1/1	Running	0	3m14s
11. long-load-557b4d94f5-bfk7c	1/1	Running	0	3m21s

12. long-load-557b4d94f5-bnpzh	1/1	Running	0	3m21s
13. long-load-557b4d94f5-ctv9	1/1	Running	0	3m21s
14. long-load-557b4d94f5-drg2p	1/1	Running	0	3m14s
15. long-load-557b4d94f5-hwsz6	1/1	Running	0	3m12s
16. long-load-557b4d94f5-k5vqj	1/1	Running	0	3m21s
17. long-load-557b4d94f5-lgstq	1/1	Running	0	3m21s
18. long-load-557b4d94f5-r8hq4	1/1	Running	0	3m21s

long-load-557b4d94f5-xrg7c	1/1	Running	0	3m21s
----------------------------	-----	---------	---	-------

19. Observe that the memory usage of the node is lower.

```
20. [student@workstation reliability-requests]$ oc describe node master01
```

```
21. ...output omitted...
```

```
22. Allocated resources:
```

```
23. (Total limits may be over 100 percent, i.e., overcommitted.)
```

24. Resource	Requests	Limits
25. -----	-----	-----
26. cpu	3158m (42%)	980m (13%)
27. memory	15167Mi (80%)	1250Mi (6%)

```
...output omitted...
```

28. Return to the /home/student/ directory.

```
29. [student@workstation reliability-requests]$ cd /home/student/
```

```
[student@workstation ~]$
```

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish reliability-requests
```

Limit Compute Capacity for Applications

Objectives

- Configure an application with resource limits so Kubernetes can protect other applications from it.

Memory and CPU requests that you define for containers help Red Hat OpenShift Container Platform (RHOCP) to select a compute node to run your pods. However, these resource requests do not restrict the memory and CPU that the containers can use. For example, setting a memory request at 1 GiB does not prevent the container from consuming more memory.

Red Hat recommends that you set the memory and CPU requests to the peak usage of your application. In contrast, by setting lower values, you overcommit the node resources. If all the applications that are running on the node start to use resources above the values that they request, then the compute nodes might run out of memory and CPU.

In addition to requests, you can set memory and CPU *limits* to prevent your applications from consuming too many resources.

Setting Memory Limits

A memory limit specifies the amount of memory that a container can use across all its processes.

As soon as the container reaches the limit, the compute node selects and then kills a process in the container. When that event occurs, RHOCP detects that the application is not working any more, because the main container process is missing, or because the health probes report an error. RHOCP then restarts the container according to the pod `restartPolicy` attribute, which defaults to `Always`.

RHOCP relies on Linux kernel features to implement resource limits, and to kill processes in containers that reach their memory limits:

Control groups (cgroups)

RHOC uses control groups to implement resource limits. Control groups are a Linux kernel mechanism for controlling and monitoring system resources, such as CPU and memory.

Out-of-Memory killer (OOM killer)

When a container reaches its memory limit, the Linux kernel triggers the OOM killer subsystem to select and then kill a process.

You must set a memory limit when the application has a memory usage pattern that you must mitigate, such as when the application has a memory leak. A memory leak is a bug in the application, which occurs when the application uses some memory but does not free it after use. If the leak appears in an infinite service loop, then the application uses more and more memory over time, and can end up consuming all the available memory on the system. For these applications, setting a memory limit prevents them from consuming all the node's memory. The memory limit also enables OpenShift to regularly restart applications to free up their memory when they reach the limit.

To set a memory limit for the container in a pod, use the `oc set resources` command:

```
[user@host ~]$ oc set resources deployment/hello --limits memory=1Gi
```

In addition to the `oc set resources` command, you can define resource limits from a file in the YAML format:

```
apiVersion: apps/v1
kind: Deployment
...output omitted...
spec:
  containers:
  - image: registry.access.redhat.com/ubi9/nginx-120:1-86
    name: hello
    resources:
      requests:
        cpu: 100m
        memory: 500Mi
```

```
limits:
  cpu: 200m
  memory: 1Gi
```

When RHOCP restarts a pod because of an OOM event, it updates the pod's `lastState` attribute, and sets the reason to `OOMKilled`:

```
[user@host ~]$ oc get pod hello-67645f4865-vvr42 -o yaml
...output omitted...
status:
...output omitted...
containerStatuses:
- containerID: cri-o://806b...9fe7
  image: registry.access.redhat.com/ubi9/nginx-120:1-86
  imageID: registry.access.redhat.com/ubi9/nginx-120:1-86@sha256:1403...fd34
  lastState:
    terminated:
      containerID: cri-o://bbc4...9eb2
      exitCode: 137
      finishedAt: "2023-03-08T07:56:06Z"
      reason: OOMKilled
      startedAt: "2023-03-08T07:51:43Z"
  name: hello
  ready: true
  restartCount: 1
...output omitted...
```

To set a memory limit for the container in a pod from the web console, select a deployment, and click **Actions** → **Edit resource limits**.

Actions ▼

Edit Pod count

Add HorizontalPodAutoscaler

Add PodDisruptionBudget

Pause rollouts

Restart rollout

Add Health Checks

Add storage

Edit update strategy

Edit resource limits

Set memory limits by increasing or decreasing the memory on the **Limit** section.

Memory

Request

–

0

+

Mi ▼

The minimum amount of Memory the Container is guaranteed.

Limit

–

0

+

Mi ▼

The maximum amount of Memory the Container is allowed to use when running.

Cancel

Save

Setting CPU Limits

CPU limits work differently from memory limits. When the container reaches the CPU limit, RHOCPS inhibits the container's processes, even if the node has available CPU cycles. The application continues to work, but at a slower pace.

In contrast, if you do not set a CPU limit, then the container can consume as much CPU as is available on the node. If the node's CPU is under pressure, for example because several containers are running CPU-intensive tasks, then the Linux kernel shares the CPU resource between all these containers, according to the CPU requests value for the containers.

You must set a CPU limit when you require a consistent application behavior across clusters and nodes. For example, if the application runs on a node where the CPU is available, then the application can execute at full speed. On the other hand, if the application runs on a node with CPU pressure, then the application executes at a slower pace.

The same behavior can occur between your development and production clusters. Because the two environments might have different node configurations, the application might run differently when you move it from development to production.

NOTE

Clusters can have differences in hardware configuration beyond what limits observe. For example, two clusters' nodes might have CPUs with equal core count and unequal clock speeds.

Requests and limits do not account for these hardware differences. If your clusters differ in such a way, take care that requests and limits are appropriate for both configurations.

By setting a CPU limit, you mitigate the differences between the configuration of the nodes, and you experience a more consistent behavior.

To set a CPU limit for the container in a pod, use the `oc set resources` command:

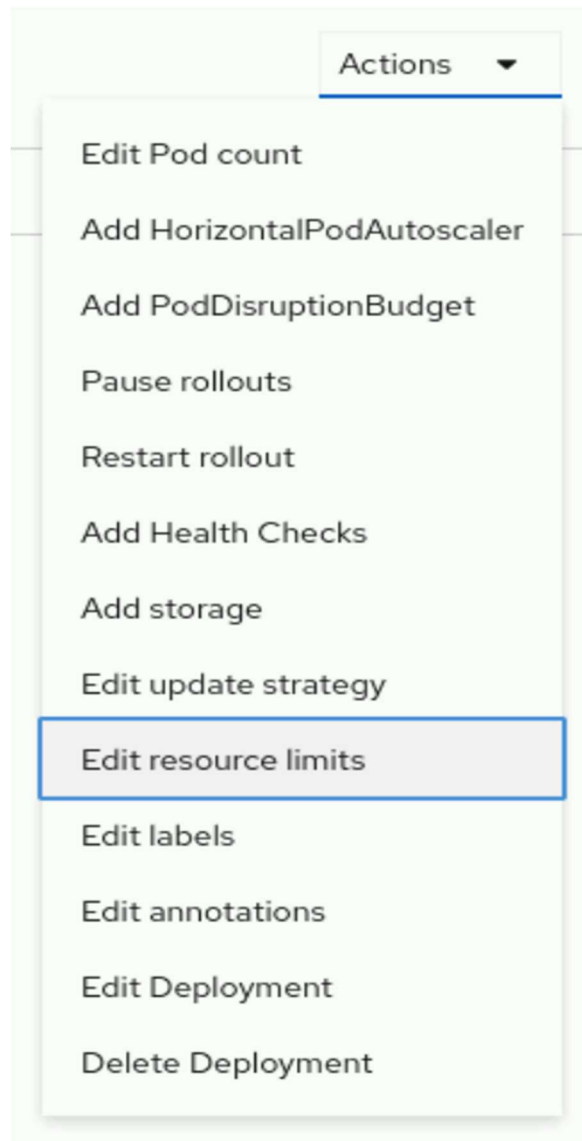
```
[user@host ~]$ oc set resources deployment/hello --limits cpu=200m
```

You can also define CPU limits from a file in the YAML format:

```
apiVersion: apps/v1
kind: Deployment
...output omitted...
spec:
  containers:
  - image: registry.access.redhat.com/ubi9/nginx-120:1-86
    name: hello
    resources:
      requests:
        cpu: 100m
        memory: 500Mi
      limits:
        cpu: 200m
```

memory: 1Gi

To set a CPU limit for the container in a pod from the web console, select a deployment, and click **Actions** → **Edit resource limits**.



Set CPU limits by increasing or decreasing the CPU on the **Limit** section.

CPU

Request

-

0

+

cores ▼

The minimum amount of CPU the Container is guaranteed.

Limit

-

0

+

cores ▼

The maximum amount of CPU the Container is allowed to use when running.

Viewing Requests, Limits, and Actual Usage

By using the RHOCP command-line interface, cluster administrators can view compute usage information on individual nodes. The `oc describe node` command displays detailed information about a node, including information about the pods that are running on the node. For each pod, it shows CPU requests and limits, as well as memory requests and limits. If you do not specify a request or limit, then the pod shows a zero for that column. The command also displays a summary of all the resource requests and limits.

```
[user@host ~]$ oc describe node master01
Name:                master01
Roles:                control-plane,master,worker
...output omitted...
Non-terminated Pods:                (88 in total)
... Name                CPU Requests  CPU Limits  Memory Requests  Memory Limits ...
... ----                -
... controller-... 10m (0%)      0 (0%)      20Mi (0%)        0 (0%)        ...
... metallb-...    50m (0%)      0 (0%)      20Mi (0%)        0 (0%)        ...
... metallb-...    0 (0%)        0 (0%)      0 (0%)           0 (0%)        ...
```

```
...output omitted...
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests          Limits
-----
cpu                 3183m (42%)      1202m (16%)
memory              12717Mi (67%)    1350Mi (7%)
...output omitted...
```

The `oc describe node` command displays requests and limits. The `oc adm top` command shows resource usage. The `oc adm top nodes` command shows the resource usage for nodes in the cluster. You must run these commands as a cluster administrator.

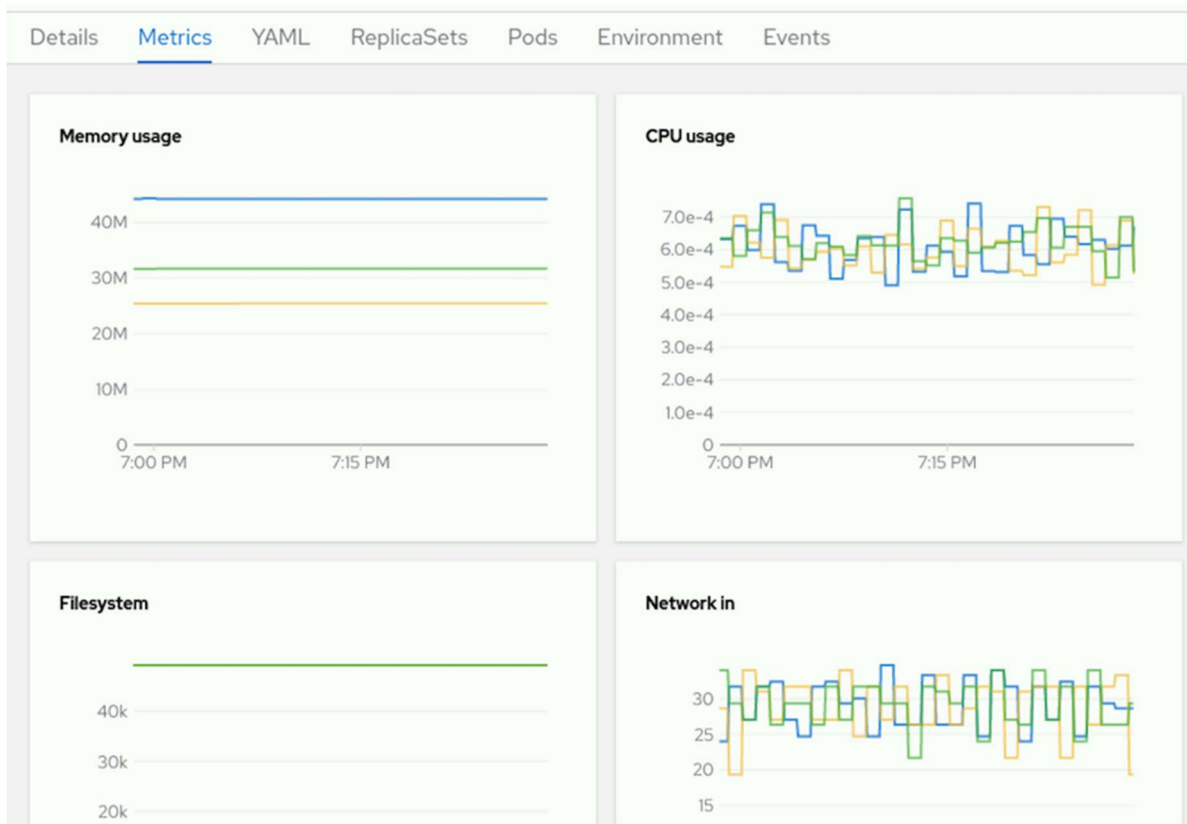
The `oc adm top pods` command shows the resource usage for each pod in a project.

The following command displays the resource usage for the pods in the `openshift-console` project:

```
[user@host ~]$ oc adm top pods -n openshift-console
```

NAME	CPU(cores)	MEMORY(bytes)
console-6689c8589c-bcpb7	1m	48Mi
downloads-79658645bd-hx649	1m	31Mi

To visualize the consumption of resources from the web console, select a deployment, and click the **Metrics** tab. From this tab, you can view the usage for memory, CPU, the file system, and incoming and outgoing traffic.



Guided Exercise: Limit Compute Capacity for Applications

Configure an application with compute resource limits that allow and prevent successful execution of its pods.

Outcomes

You should be able to monitor the memory usage of an application, and set a memory limit for a pod.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `reliability-limits` project and the `/home/student/D0180/labs/reliability-limits/resources.txt` file.

The `resources.txt` file contains some commands that you use during the exercise. You can use the file to copy and paste these commands.

```
[student@workstation ~]$ lab start reliability-limits
```

Instructions

1. Log in to the OpenShift cluster as the developer user with the developer password. Use the reliability-limits project.
1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Set the reliability-limits project as the active project.

```
6. [student@workstation ~]$ oc project reliability-limits
```

...output omitted...

2. Create the leakapp deployment from the ~/D0180/labs/reliability-limits/leakapp.yml file that the lab command prepared. The application has a bug, and leaks 1 MiB of memory every second.
1. Review the ~/D0180/labs/reliability-limits/leakapp.yml resource file. The memory limit is set to 35 MiB. Do not change the file.

```
2. ...output omitted...
3.         resources:
4.         requests:
5.             memory: 20Mi
6.         limits:
```

memory: 35Mi

7. Use the oc apply command to create the application. Ignore the warning message.

```
8. [student@workstation ~]$ oc apply -f \
9. ~/D0180/labs/reliability-limits/leakapp.yml
```

```
deployment.apps/leakapp created
```

10. Wait for the pod to start. You might have to rerun the command several times for the pod to report a Running status. The name of the pod on your system probably differs.

```
11. [student@workstation ~]$ oc get pods
```

12. NAME	READY	STATUS	RESTARTS	AGE
----------	-------	--------	----------	-----

leakapp-99bb64c8d-hk26k	1/1	Running	0	12s
-------------------------	-----	---------	---	-----

3. Watch the pod. OpenShift restarts the pod after 30 seconds.

1. Use the `watch` command to monitor the `oc get pods` command. Wait for OpenShift to restart the pod, and then press **Ctrl+C** to quit the `watch` command.

```
2. [student@workstation ~]$ watch oc get pods
```

```
3. Every 2.0s: oc get pods                               workstation: Thur Sep  4 09:04:47
   2025
```

```
4.
```

5. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

leakapp-99bb64c8d-hk26k	1/1	Running	1 (15s ago)	48s
-------------------------	-----	---------	-------------	-----

6. Retrieve the container status to verify that OpenShift restarted the pod due to an Out-Of-Memory (OOM) event.

```
7. [student@workstation ~]$ oc get pods leakapp-99bb64c8d-hk26k \
```

```
8. -o jsonpath='{.status.containerStatuses[0].lastState}' | jq .
```

```
9. {
```

```
10.   "terminated": {
```

```
11.     "containerID": "cri-o://5800...1d04",
```

```
12.     "exitCode": 137,
```

```
13.     "finishedAt": "2025-09-04T09:04:47Z",
```

```
14.     "reason": "OOMKilled",
```

```
15.     "startedAt": "2025-09-04T09:04:17Z"
```

```
16.   }
```

```
}
```

4. Observe the pod status for a few minutes, until the `CrashLoopBackOff` status is displayed. During this period, OpenShift restarts the pod several times because of the memory leak.

Between each restart, OpenShift sets the pod status to `CrashLoopBackOff`, waits an increasing amount of time between retries, and then restarts the pod. The delay between restarts gives the operator the opportunity to fix the issue.

After various retries, OpenShift finally sets the `CrashLoopBackOff` wait timer to five minutes. During this wait time, the application is not available to your customers.

```
[student@workstation ~]$ watch oc get pods
```

```
Every 2.0s: oc get pods  
7 2025
```

```
workstation: Thur Sep 4 09:04:4
```

NAME	READY	STATUS	RESTARTS	AGE
leakapp-99bb64c8d-hk26k	0/1	CrashLoopBackOff	4 (82s ago)	5m25s

Press **Ctrl+C** to quit the watch command.

5. Fixing the memory leak would resolve the issue. However, it might take some time for the developers to fix the bug. In the meantime, set the memory limit to 600 MiB. With this setting, the pod can run for ten minutes before the application reaches the limit.
1. Use the `oc set resources` command to set the new limit. Ignore the warning message.

2. `[student@workstation ~]$ oc set resources deployment/leakapp \`
3. `--limits memory=600Mi`

```
deployment.apps/leakapp resource requirements updated
```

- Wait for the pod to start. You might have to rerun the command several times for the pod to report a Running status. The name of the pod on your system probably differs.

```
5. [student@workstation ~]$ oc get pods
```

```
6. NAME                                READY   STATUS    RESTARTS   AGE
```

```
leakapp-6bc64dfcd-86fpc    1/1     Running   0           12s
```

- Wait two minutes to verify that OpenShift no longer restarts the pod every 30 seconds.

```
8. [student@workstation ~]$ watch oc get pods
```

```
9. Every 2.0s: oc get pods                                workstation: Thur Sep  4 09:04:47 2025
```

```
10.
```

```
11. NAME                                READY   STATUS    RESTARTS   AGE
```

```
leakapp-6bc64dfcd-86fpc    1/1     Running   0           3m12s
```

Press **Ctrl+C** to quit the watch command.

- Review the memory that the pod consumes. The metrics for the pod are available a few minutes after the pod starts. You might have to rerun the command after the metrics are available, or if the command displays an error message that the metrics are not available. The memory usage on your system can differ.

```
7. [student@workstation ~]$ oc adm top pods
```

```
8. NAME                                CPU(cores)   MEMORY(bytes)
```

```
leakapp-6bc64dfcd-86fpc    1m           133Mi
```

- Optional.** Wait about 10 minutes from the creation time until the application reaches the out of memory error. After this period, OpenShift restarts the pod, because it reached the 600 MiB memory limit.
- Open a new terminal window, and then run the watch command to monitor the `oc adm top pods` command.

```
2. [student@workstation ~]$ watch oc adm top pods
```

```
3. Every 2.0s: oc adm top pods                    workstation: Thur Sep  4 09:04:47 2025
```

```
4.
```

```
5. NAME                                CPU(cores)   MEMORY(bytes)
```

```
leakapp-6bc64dfcd-86fpc    0m           176Mi
```

Leave the command running and do not interrupt it.

NOTE

You might see a message that metrics are not yet available. If so, wait some time and try again.

6. In the first terminal, run the `watch` command to monitor the `oc get pods` command. Watch the output of the `oc adm top pods` command in the second terminal. When the memory usage reaches 600 MiB, the OOM subsystem kills the process inside the container, and OpenShift restarts the pod.

```
7. [student@workstation ~]$ watch oc get pods
```

```
8. Every 2.0s: oc get pods                    workstation: Thur Sep  4 09:04:47 2025
```

```
9.
```

```
10. NAME                                READY   STATUS    RESTARTS   AGE
```

```
leakapp-6bc64dfcd-86fpc    1/1     Running   1 (3s ago)  9m58s
```

Press **Ctrl+C** to quit the `watch` command.

11. Press **Ctrl+C** to quit the `watch` command in the second terminal. Close this second terminal when done.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

Application Autoscaling

Objectives

- Configure a horizontal pod autoscaler for an application.

Kubernetes can autoscale a deployment based on current load on the application pods, by means of the `HorizontalPodAutoscaler` resource type.

Horizontal Pod Autoscaling

In a production environment, the load on an application can fluctuate significantly. Manually scaling the number of application pods to match demand is inefficient and prone to error. Red Hat OpenShift Container Platform (RHOCP) provides an automated solution for this challenge by using the Horizontal Pod Autoscaler (HPA). The HPA automatically scales the number of pods in a replication controller, deployment, replica set, or stateful set based on observed metrics such as CPU utilization.

For an enterprise, effective autoscaling is critical for several reasons:

- **High Availability and Performance:** By automatically adding pods during traffic spikes, the HPA ensures that the application remains responsive and meets its service-level objectives.
- **Cost Optimization:** During periods of low traffic, the HPA scales down the number of pods. Removing the extra pods releases unused resources back to the cluster, to enable other workloads to use them and to optimize overall resource usage.
- **Operational Efficiency:** Automating the scaling process removes the need for manual intervention by operations teams. The automation reduces the risk of human error and enables engineers to focus on other tasks.

The Autoscaling Process in Kubernetes

The HPA is implemented as a control loop that runs inside the Kubernetes controller manager. A `HorizontalPodAutoscaler` resource uses performance metrics

that the RHOCP monitoring stack collects. The monitoring stack comes preinstalled in RHOCP. The autoscaler works in a continuous loop. By default, the controller manager checks HPA resources every 15 seconds. During each check, it performs the following steps:

- The autoscaler retrieves the target metric for scaling from the HPA resource definition.
- For each pod that the HPA resource targets, the autoscaler collects the current value of the metric from the metrics server.
- For each targeted pod, the autoscaler computes the usage percentage, based on the collected metric and the resource requests that are defined in the pod specification.
- The autoscaler computes the average usage across all the targeted pods.
- It establishes a ratio between the target metric value and the current average metric value.
- The autoscaler uses this ratio to make a scaling decision, either to increase or to decrease the number of replicas for the target resource.

Prerequisites for Autoscaling

To autoscale a deployment, you must specify resource requests for the containers in your pods. The HPA uses the requested resource values to calculate the utilization percentage. The autoscaler requires resource requests to calculate utilization and to perform scaling actions.

IMPORTANT

Pods that are created by using the `oc create deployment` command do not define resource requests by default. Using the RHOCP autoscaler might therefore require editing the deployment resources to add requests. Alternatively, you can create custom YAML resource files for your application. You can also add to your project a `LimitRange` resource, which defines default resource requests for pods.

Creating a Horizontal Pod Autoscaler

You can create an HPA resource by using the command line, a YAML manifest, or the web console.

Using the oc autoscale Command

To create a HorizontalPodAutoscaler resource from the command line, use the `oc autoscale` command:

```
[user@host ~]$ oc autoscale deployment/hello \
--min 1 --max 10 --cpu-percent 80
```

This command creates an HPA that targets the `hello` deployment. The HPA maintains the pod count between 1 and 10 replicas. It scales the deployment to keep the average CPU utilization of its pods at or below 80% of their requested CPU.

The maximum and minimum values for the horizontal pod autoscaler resource accommodate bursts of load and avoid overloading the RHOCP cluster. If the load on the application changes too quickly, then it might help to keep several spare pods to cope with sudden bursts of user requests. Conversely, too many pods can use up all cluster capacity and impact other applications that use the same RHOCP cluster.

Using a YAML Manifest

For more advanced configurations and for managing infrastructure as code, you can define an HPA resource in a YAML file.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello
spec:

  minReplicas: 1

  maxReplicas: 10

  metrics:
  - resource:
```

```

    name: cpu
    target:

      averageUtilization: 80
      type: Utilization
    type: Resource

scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: hello

```

The `minReplicas` field specifies the minimum number of pods.

The `maxReplicas` field specifies the maximum number of pods.

The `metrics` array defines the metrics to use for scaling decisions.

The `averageUtilization` field specifies the target average CPU utilization for each pod, and is represented as a percentage. If the global average CPU usage is above that value, then the horizontal pod autoscaler starts new pods. If the global average CPU usage is below that value, then the horizontal pod autoscaler deletes pods.

The `scaleTargetRef` field points to the resource that the HPA manages.

Use the `oc apply` command to create the resource from the file.

```
[user@host ~]$ oc apply -f hello-hpa.yaml
```

The preceding example creates an HPA resource that scales based on CPU usage. Alternatively, it can scale based on memory usage by setting the resource name to `memory`.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello
spec:
  minReplicas: 1

```

```
maxReplicas: 10
metrics:
- resource:

  name: memory
  target:
    averageUtilization: 80
...output omitted...
```

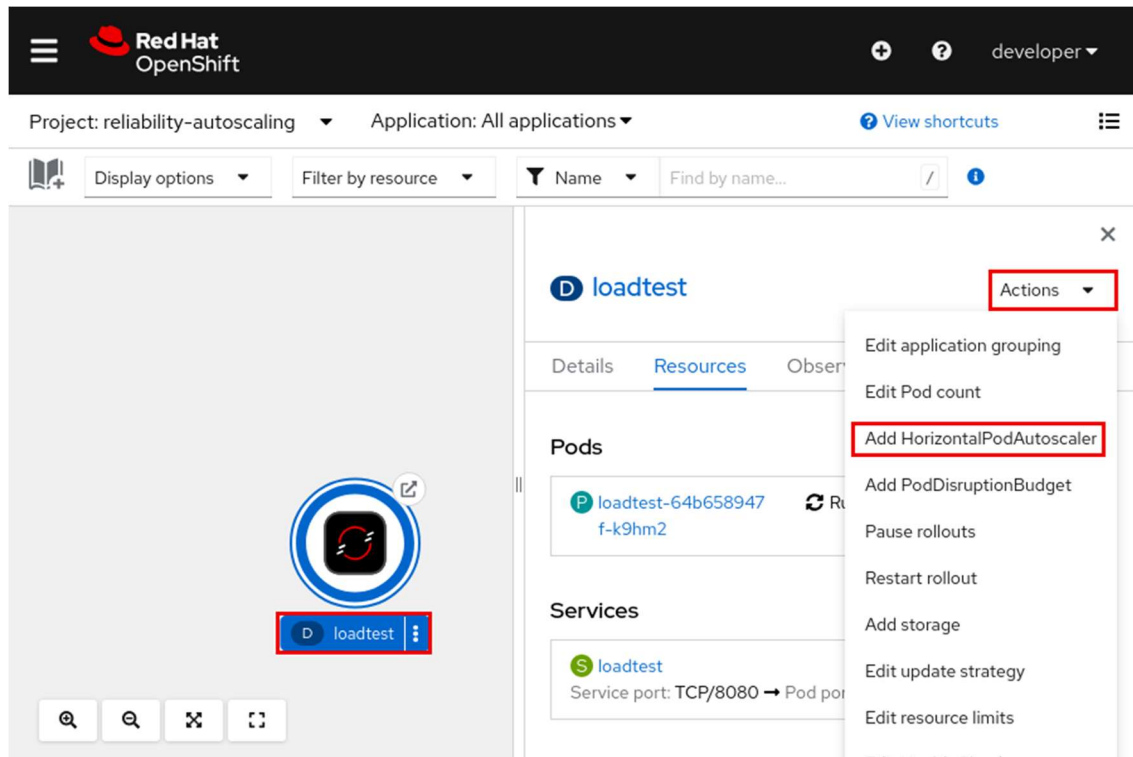
The resource name is set to `memory` to trigger scaling based on memory utilization.

NOTE

Memory-based autoscaling is not suitable for all applications. For example, applications that are based on the Java Virtual Machine (JVM) often claim large memory at startup and do not release it. If an application's memory usage does not correlate with its load, then memory-based autoscaling might not be effective.

Using the Web Console

To create an HPA resource from the web console, in the Developer perspective, select the project and click the **Topology** menu. Click the Deployment for the application to display the details for the deployment. Select **Action** → **Add HorizontalPodAutoscaler** from the menu.



Use the Add Horizontal Pod Autoscaler form or YAML view to set the HPA parameters.



 developer

Project: reliability-autoscaling

Add HorizontalPodAutoscaler

Resource  loadtest

 Memory resource limits must be set if you want to use memory utilization. The HorizontalPodAutoscaler will not have memory metrics until resource limits are set.

Configure via: ☒ Form view ☐ YAML view

Name

Minimum Pods

–

2

+

Maximum Pods

–

20

+

CPU Utilization

%

CPU request and limit must be set before CPU utilization can be set.

Memory Utilization

%

Memory request and limit must be set before Memory utilization can be set.

Save

Cancel

Verifying Horizontal Pod Autoscaler Status

After creating an HPA object, you can monitor its status and activity.

Viewing HPA Objects

To get information about HPA resources in the current project, use the `oc get hpa` command:

```
[user@host ~]$ oc get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hello	Deployment/hello	<unknown>/80%	1	10	1	30s

The horizontal pod autoscaler initially has the value of <unknown> in the TARGETS column. It might take up to five minutes for <unknown> to change and display a percentage for current usage. A persistent value of <unknown> in the TARGETS column might indicate that the target deployment does not define resource requests for the specified metric. The horizontal pod autoscaler scales only pods with resource requests.

Inspecting HPA Details and Events

For detailed information, use the `oc describe hpa` command. This command shows the HPA's configuration, current metrics, and a log of recent scaling events.

```
[user@host ~]$ oc describe hpa hello
```

```
...output omitted...
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulRescale	16m	horizontal-pod-autoscaler	New size: 2; ...
Normal	SuccessfulRescale	14m	horizontal-pod-autoscaler	New size: 4; ...
Normal	SuccessfulRescale	14m	horizontal-pod-autoscaler	New size: 8; ...

```
...output omitted...
```

The Events section is useful for troubleshooting, by providing a chronological record of when and why the HPA scaled up or scaled down the target resource.

Guided Exercise: Application Autoscaling

Configure an autoscaler for an application and then load test that application to observe scaling up.

Outcomes

- Deploy an application.
- Manually scale the application.

- Configure application resource requests.
- Deploy the application with a Horizontal Pod Autoscaler.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the cluster is accessible and that all resources are available for this exercise. It also creates the `reliability-autoscaling` project.

```
[student@workstation ~]$ lab start reliability-autoscaling
```

Instructions

1. Log in to the OpenShift cluster as the `developer` user with `developer` as the password. Use the `reliability-autoscaling` project.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Set the `reliability-autoscaling` project as the active project.

```
6. [student@workstation ~]$ oc project reliability-autoscaling
```

...output omitted...

2. Create the `loadtest` deployment, service, and route. The deployment uses the `registry.ocp4.example.com:8443/redhattraining/loadtest:v1.0` container image, which provides a web application. The web application exposes an API endpoint that creates a CPU-intensive task when queried.

1. Review the `~/D0180/labs/reliability-autoscaling/loadtest.yml` resource file that the `lab` command prepared. The container specification does not include the `resources` section that you use to specify CPU requests and limits. You configure that section in another step. Do not change the file for now.

```
2. apiVersion: v1
```

```

3.kind: List
4.metadata: {}
5.items:
6.  - apiVersion: apps/v1
7.    kind: Deployment
8....output omitted...
9.      spec:
10.        containers:
11.          - image: registry.ocp4.example.com:8443/redhattraining/loadtest:v
12.            1.0
13.            name: loadtest
14.            readinessProbe:
15.              failureThreshold: 3
16.              httpGet:
17.                path: /api/loadtest/v1/healthz
18.                port: 8080
19.                scheme: HTTP
20.              periodSeconds: 10
21.              successThreshold: 1
22.              timeoutSeconds: 1
23.  - apiVersion: v1
24.    kind: Service
25....output omitted...
26.
27.  - apiVersion: route.openshift.io/v1
28.    kind: Route

```

```
...output omitted...
```

29. Use the `oc apply` command to create the application.

```

30. [student@workstation ~]$ oc apply -f \
31. ~/D0180/labs/reliability-autoscaling/loadtest.yml
32. deployment.apps/loadtest created

```

```
33. service/loadtest created
```

```
route.route.openshift.io/loadtest created
```

34. Run the watch command and wait for the pod to report the Running status. The name of the pod is different in your cluster.

```
35. [student@workstation ~]$ watch oc get pods
```

```
36. Every 2.0s: oc get pods
```

```
37.
```

```
38. NAME                                READY   STATUS    RESTARTS   AGE
```

```
loadtest-86d987b9bf-w66x9    1/1     Running   0           40s
```

Press **Ctrl+C** to end the watch command.

3. Configure a horizontal pod autoscaler resource for the loadtest deployment. Set the minimum number of replicas to 2 and the maximum to 20. Set the average CPU usage to 50% of the CPU requests attribute.

The horizontal pod autoscaler does not work, because the loadtest deployment does not specify requests for CPU usage.

1. Use the `oc autoscale` command to create the horizontal pod autoscaler resource.

```
2. [student@workstation ~]$ oc autoscale deployment/loadtest --min 2 --max 20 \
3. --cpu-percent 50
```

```
horizontalpodautoscaler.autoscaling/loadtest autoscaled
```

4. Retrieve the status of the loadtest horizontal pod autoscaler resource. The unknown value in the TARGETS column indicates that OpenShift cannot compute the current CPU usage of the loadtest deployment. The deployment must include the CPU requests attribute for OpenShift to be able to compute the CPU usage.

```
5. [student@workstation ~]$ oc get hpa loadtest
```

6. NAME AGE	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
loadtest 74s	Deployment/loadtest	<unknown>/50%	2	20	2

7. Get more details about the resource status. You might have to rerun the command several times. Wait up to three minutes for the command to report the warning message.

```

8. [student@workstation ~]$ oc describe hpa loadtest
9. Name: loadtest
10. Namespace: reliability-autoscaling
11. ...output omitted...
12. Conditions:
13.   Type          Status Reason          Message
14.   ----          -
15.   AbleToScale    True   SucceededGetScale the HPA controller was able to get the target's current scale
16.   ScalingActive  False  FailedGetResourceMetric the HPA was unable to compute the replica count: failed to get cpu utilization: missing request for cpu
17. Events:
18.   Type          ... Message
19.   ----          ... -
20. ...output omitted...
21.   Warning ... failed to get cpu utilization: missing request for cpu

```

```

...output omitted...

```

22. Delete the horizontal pod autoscaler resource. You re-create the resource in another step, after you fix the loadtest deployment.

```

23. [student@workstation ~]$ oc delete hpa loadtest

```

```

horizontalpodautoscaler.autoscaling "loadtest" deleted

```

24. Delete the loadtest application.

```
25. [student@workstation ~]$ oc delete -f \
26. ~/D0180/labs/reliability-autoscaling/loadtest.yml
27. deployment.apps "loadtest" deleted
28. service "loadtest" deleted
```

```
route.route.openshift.io "loadtest" deleted
```

4. Add a CPU resource section to the ~/D0180/labs/reliability-autoscaling/loadtest.yml file. Redeploy the application from the file.
1. Edit the ~/D0180/labs/reliability-autoscaling/loadtest.yml file, and configure the CPU limits and requests for the loadtest deployment. The pod needs 25 millicores to operate, and must not consume more than 100 millicores.

You can compare your work with the completed ~/D0180/solutions/reliability-autoscaling/loadtest.yml file that the lab command prepared.

```
...output omitted...
spec:
  containers:
    - image: registry.ocp4.example.com:8443/redhattraining/loadtest:v
      name: loadtest
      readinessProbe:
        failureThreshold: 3
        httpGet:
          path: /api/loadtest/v1/healthz
          port: 8080
          scheme: HTTP
        periodSeconds: 10
        successThreshold: 1
        timeoutSeconds: 1
      resources:
        requests:
          cpu: 25m
```

```
limits:
  cpu: 100m
...output omitted...
```

2. Use the `oc apply` command to deploy the application from the file.

```
3. [student@workstation ~]$ oc apply -f \
4. ~/D0180/labs/reliability-autoscaling/loadtest.yml
5. deployment.apps/loadtest created
6. service/loadtest created
```

```
route.route.openshift.io/loadtest created
```

7. Run the `watch` command and wait until the pod is running. The name of the pod is different in your cluster.

```
8. [student@workstation ~]$ watch oc get pods
```

9. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

loadtest-667bdc99-vhc9x	1/1	Running	0	36s
-------------------------	-----	---------	---	-----

Press **Ctrl+C** to end the watch command after the pod displays Running in the STATUS column.

5. Manually scale the `loadtest` deployment by first increasing and then decreasing the number of running pods.

1. Scale up the `loadtest` deployment to five pods.

```
2. [student@workstation ~]$ oc scale deployment/loadtest --replicas 5
```

```
deployment.apps/loadtest scaled
```

3. Run the `watch` command and wait until the five pods are running. The names of the pods are different in your cluster.

```
4. [student@workstation ~]$ watch oc get pods
```

5. NAME	READY	STATUS	RESTARTS	AGE
---------	-------	--------	----------	-----

6. loadtest-667bdc99-5fcvh	1/1	Running	0	43s
----------------------------	-----	---------	---	-----

7. loadtest-667bdc99-dpspr	1/1	Running	0	42s
----------------------------	-----	---------	---	-----

8. loadtest-667bdc99-hkssk	1/1	Running	0	43s
9. loadtest-667bdc99-vhc9x	1/1	Running	0	8m11s

loadtest-667bdc99-z5n9q	1/1	Running	0	43s
-------------------------	-----	---------	---	-----

Press **Ctrl+C** to end the watch command after the pods display Running in the STATUS column.

10. Scale down the loadtest deployment back to one pod.

```
11. [student@workstation ~]$ oc scale deployment/loadtest --replicas 1
```

```
deployment.apps/loadtest scaled
```

12. Run the watch command and wait until the pod is running. The name of the pod is different in your cluster.

```
13. [student@workstation ~]$ watch oc get pods
```

14. NAME	READY	STATUS	RESTARTS	AGE
----------	-------	--------	----------	-----

loadtest-667bdc99-vhc9x	1/1	Running	0	11m
-------------------------	-----	---------	---	-----

Press **Ctrl+C** to end the watch command after the pod displays Running in the STATUS column.

6. Configure a horizontal pod autoscaler resource for the loadtest deployment. Set the minimum number of replicas to 2 and the maximum to 20. Set the average CPU usage to 50% of the CPU request attribute.

1. Use the oc autoscale command to create the horizontal pod autoscaler resource.

```
2. [student@workstation ~]$ oc autoscale deployment/loadtest --min 2 --max 20 \
3. --cpu-percent 50
```

```
horizontalpodautoscaler.autoscaling/loadtest autoscaled
```

4. Open a new terminal window and run the watch command to monitor the oc get hpa loadtest command. Wait up to five minutes for

the loadtest horizontal pod autoscaler to report usage in the TARGETS column.

Notice that the horizontal pod autoscaler scales up the deployment to two replicas, to conform with the minimum number of pods that you configured.

```
[student@workstation ~]$ watch oc get hpa loadtest
```

Every 2.0s: oc get hpa loadtest workstation: Fri Mar 3 06:26:24 2023

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	A
loadtest	Deployment/loadtest	0%/50%	2	20	2	5

Leave the command running, and do not interrupt it.

7. Increase the CPU usage by sending requests to the loadtest application API.

1. Use the `oc get route` command to retrieve the URL of the application.

```
2. [student@workstation ~]$ oc get route loadtest
```

```
3. NAME          HOST/PORT          ...
```

```
loadtest    loadtest-reliability-autoscaling.apps.ocp4.example.com    ...
```

4. Send a request to the application API to simulate additional CPU pressure on the container. Do not wait for the `curl` command to complete, and continue with the exercise. After one minute, the command reports a timeout error that you can ignore.

```
5. [student@workstation ~]$ curl \
```

```
6. loadtest-reliability-autoscaling.apps.ocp4.example.com/api/loadtest/v1/cpu/1
```

```
7. <html><body><h1>504 Gateway Time-out</h1>
```

```
8. The server didn't respond in time.
```

```
</body></html>
```


9. Watch the output of the `oc get hpa loadtest` command in the second terminal. After a minute, the horizontal pod autoscaler detects an increase in the CPU usage and deploys additional pods.

NOTE

The increased activity of the application does not immediately trigger the autoscaler. Wait a few moments if you do not see any changes to the number of replicas.

You might need to run the `curl` command multiple times before the application uses enough CPU to trigger the autoscaler.

The CPU usage and the number of replicas on your system probably differ.

Every 2.0s: oc get hpa loadtest				workstation: Fri Mar 3 07:20:19		
2023						
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	A
GE						
loadtest	Deployment/loadtest	220%/50%	2	20	9	1
6m						

10. Wait five minutes after the `curl` command completes. The `oc get hpa loadtest` command shows that the CPU load decreases.

NOTE

Although the horizontal pod autoscaler resource can be quick to scale up, it is slower to scale down.

Every 2.0s: oc get hpa loadtest				workstation: Fri Mar 3 07:23:11		
2023						
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AG
E						
loadtest	Deployment/loadtest	0%/50%	2	20	9	18
m						

11. **Optional:** Wait for the `loadtest` application to scale down. It takes five additional minutes for the horizontal pod autoscaler to scale down to two replicas.

```
12. Every 2.0s: oc get hpa loadtest          workstation: Fri Mar  3 07:29:12
    2023
```

```
13.
```

14. NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AG
E						

loadtest	Deployment/loadtest	0%/50%	2	20	2	24
m						

15. Press **Ctrl+C** to quit the watch command. Close that second terminal when done.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish reliability-autoscaling
```

Lab: Configure an Application for Reliability

Deploy and troubleshoot a reliable application that defines health probes, compute resource requests, and compute resource limits so it can run N instances per node; and configure a horizontal pod autoscaler that scales to a maximum of N instances.

Outcomes

- Set the resource requests for a deployment.
- Configure a readiness probe.
- Create a horizontal pod autoscaler.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise. This command ensures that the cluster is accessible and that all resources are available for this exercise.

```
[student@workstation ~]$ lab start reliability-review
```

Instructions

In this exercise, you work in the `reliability-review` project that the `lab` command created. The command also deployed the `longload` application in the `reliability-review` project.

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

1. Log in to the OpenShift cluster as the `developer` user with `developer` as the password, and change to the `reliability-review` project. You use this project for all your work in this exercise.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
```

...output omitted...

4. Change to the `reliability-review` project.

```
5. [student@workstation ~]$ oc project reliability-review
```

...output omitted...

2. The `longload` application in the `reliability-review` project fails to start. Diagnose and then fix the issue. The application needs 512 MiB of memory to work.

After you fix the issue, you can confirm that the application works by running the `~/D0180/labs/reliability-review/curl_loop.sh` script that the `lab` command prepared. The script sends requests to the application in a loop. For each request, the script displays the pod name and the application status. Press **Ctrl+C** to end the script.

1. List the pods in the project. The pod is in the `Pending` status. The name of the pod is different in your cluster.

```
2. [student@workstation ~]$ oc get pods
```

3. NAME	READY	STATUS	RESTARTS	AGE
longload-64bf8dd776-b6rkz	0/1	Pending	0	8m1s

4. Retrieve the events for the pod. No compute node has enough memory to accommodate the pod.

```

5. [student@workstation ~]$ oc describe pod longload-64bf8dd776-b6rkz
6. Name:                longload-64bf8dd776-b6rkz
7. Namespace:           reliability-review
8. ...output omitted...
9. Events:
10.  Type      Reason          Age    From          Message
11.  ----      -

```

```

Warning FailedScheduling 8m    default-scheduler 0/1 nodes are available: 1 Insufficient memory.
preemption: 0/1 nodes are available: 1 No preemption victims found for incoming pod.

```

12. Verify that the longload deployment requests 8 GiB of memory by running the following command:

```

13. [student@workstation ~]$ oc get deployment longload -o \
14.   jsonpath='{.spec.template.spec.containers[0].resources.requests.memory}'{"
    \n"}'

```

```
8Gi
```

15. Set the memory requests for the longload deployment to 512 MiB.

```

16. [student@workstation ~]$ oc set resources deployment/longload \
17.   --requests memory=512Mi

```

```
deployment.apps/longload resource requirements updated
```

18. Run the watch command and wait until the longload pod is running. The name of the pod is different in your cluster.

```

19. [student@workstation ~]$ oc get pods

```

20. NAME	READY	STATUS	RESTARTS	AGE
longload-5897c9558f-cx4gt	1/1	Running	0	86s

Press **Ctrl+C** to end the watch command after the pod displays Running in the STATUS column.

21. Run the `~/D0180/labs/reliability-review/curl_loop.sh` script to confirm that the application works. You might get the `curl: (7) Failed to connect ...` message until the application is ready.

```

22. [student@workstation ~]$ ~/D0180/labs/reliability-review/curl_loop.sh
23. 1 curl: (7) Failed to connect to master01.ocp4.example.com port 30372: Conn
    ection refused
24. 2 longload-5897c9558f-cx4gt: app is still starting
25. 3 longload-5897c9558f-cx4gt: app is still starting
26. 4 longload-5897c9558f-cx4gt: app is still starting
27. 5 longload-5897c9558f-cx4gt: Ok
28. 6 longload-5897c9558f-cx4gt: Ok
29. 7 longload-5897c9558f-cx4gt: Ok
30. 8 longload-5897c9558f-cx4gt: Ok

```

...output omitted...

Press **Ctrl+C** to end the script after the application displays the ok message.

3. When the longload application scales up, your customers complain that some requests fail. To replicate the issue, manually scale up the longload application to three replicas, and run the `~/D0180/labs/reliability-review/curl_loop.sh` script at the same time.

The application takes seven seconds to initialize. The application exposes the `/health` API endpoint on HTTP port `3000`. Configure the longload deployment to use this endpoint, to ensure that the application is ready before serving client requests.

1. Open a second terminal window and run the `~/D0180/labs/reliability-review/curl_loop.sh` script.

```
2. [student@workstation ~]$ ~/D0180/labs/reliability-review/curl_loop.sh
3. 1 longload-5897c9558f-cx4gt: Ok
4. 2 longload-5897c9558f-cx4gt: Ok
5. 3 longload-5897c9558f-cx4gt: Ok
6. 4 longload-5897c9558f-cx4gt: Ok
```

...output omitted...

Leave the script running and do not interrupt it.

7. In the first terminal window, scale up the `longload` application to three replicas.

```
8. [student@workstation ~]$ oc scale deployment/longload --replicas 3
```

`deployment.apps/longload scaled`

9. Switch back to the second terminal, and watch the output of the `curl_loop.sh` script. Some requests fail because OpenShift sends requests to the new pods before the application is ready.

```
10. ....output omitted...
11. 22 longload-5897c9558f-cx4gt: Ok
12. 23 longload-5897c9558f-cx4gt: Ok
13. 24 longload-5897c9558f-cx4gt: Ok
14. 25 curl: (7) Failed to connect to master01.ocp4.example.com port 30372: Connection refused
15. 26 curl: (7) Failed to connect to master01.ocp4.example.com port 30372: Connection refused
16. 27 longload-5897c9558f-cx4gt: Ok
17. 28 curl: (7) Failed to connect to master01.ocp4.example.com port 30372: Connection refused
18. 29 longload-5897c9558f-cx4gt: Ok
19. 30 curl: (7) Failed to connect to master01.ocp4.example.com port 30372: Connection refused
20. 31 longload-5897c9558f-tpssf: app is still starting
```

```
21.32 longload-5897c9558f-kkvm5: app is still starting
22.33 longload-5897c9558f-cx4gt: Ok
23.34 longload-5897c9558f-tpssf: app is still starting
24.35 longload-5897c9558f-tpssf: app is still starting
25.36 longload-5897c9558f-tpssf: app is still starting
26.37 longload-5897c9558f-cx4gt: Ok
27.38 longload-5897c9558f-tpssf: app is still starting
28.39 longload-5897c9558f-cx4gt: Ok
29.40 longload-5897c9558f-cx4gt: Ok
```

...output omitted...

Leave the script running and do not interrupt it.

30. Add a readiness probe to the longload deployment.

```
31.[student@workstation ~]$ oc set probe deployment/longload --readiness \
32.  --initial-delay-seconds 7 \
33.  --get-url http://:3000/health
```

deployment.apps/longload probes updated

34. Scale down the longload application back to one pod.

```
35.[student@workstation ~]$ oc scale deployment/longload --replicas 1
```

deployment.apps/longload scaled

If scaling down breaks the `curl_loop.sh` script, then press **Ctrl+c** to stop the script in the second terminal. Then, restart the script.

36. To test your work, scale up the application to three replicas again.

```
37.[student@workstation ~]$ oc scale deployment/longload --replicas 3
```

deployment.apps/longload scaled

38. Watch the output of the `curl_loop.sh` script in the second terminal, and observe that no requests fail.

```
39. ...output omitted...
40. 92 longload-7ddcc9b7fd-72dtm: Ok
41. 93 longload-7ddcc9b7fd-72dtm: Ok
42. 94 longload-7ddcc9b7fd-72dtm: Ok
43. 95 longload-7ddcc9b7fd-q1n95: Ok
44. 96 longload-7ddcc9b7fd-wrxrb: Ok
45. 97 longload-7ddcc9b7fd-q1n95: Ok
46. 98 longload-7ddcc9b7fd-wrxrb: Ok
47. 99 longload-7ddcc9b7fd-72dtm: Ok
```

```
...output omitted...
```

Press **Ctrl+C** to end the script. Do not close the second terminal window.

4. Configure the `longload` application so that it automatically scales up when the average memory usage is above 60% of the memory requests value, and scales down when the usage is below this percentage. The minimum number of replicas must be one, and the maximum must be three. The resource that you create for scaling the application must be named `longload`.

The `lab` command provides the `~/D0180/labs/reliability-review/hpa.yml` resource file as an example. Use the `oc explain` command to learn the valid parameters for the `hpa.spec.metrics.resource.target` attribute. Because the file is incomplete, you must update it first if you choose to use it.

To test your work, use the `oc exec deploy/longload -- curl localhost:3000/leak` command to send an HTTP request to the application `/leak` API endpoint. Each request consumes an additional 480 MiB of memory. To free this memory, you can use the `~/D0180/labs/reliability-review/free.sh` script.

1. Before you create the horizontal pod autoscaler resource, scale down the application to one pod.


```
2. [student@workstation ~]$ oc scale deployment/longload --replicas 1
```

```
deployment.apps/longload scaled
```

3. Edit the ~/D0180/labs/reliability-review/hpa.yml resource file. You can retrieve the parameters for the resource attribute by using the `oc explain hpa.spec.metrics.resource` and `oc explain hpa.spec.metrics.resource.target` commands.

```
4. apiVersion: autoscaling/v2
5. kind: HorizontalPodAutoscaler
6. metadata:
7.   name: longload
8.   labels:
9.     app: longload
10. spec:
11.   maxReplicas: 3
12.   minReplicas: 1
13.   scaleTargetRef:
14.     apiVersion: apps/v1
15.     kind: Deployment
16.     name: longload
17.   metrics:
18.   - type: Resource
19.     resource:
20.       name: memory
21.       target:
22.         type: Utilization
```

```
averageUtilization: 60
```

23. Use the `oc apply` command to deploy the horizontal pod autoscaler.

```
24. [student@workstation ~]$ oc apply -f ~/D0180/labs/reliability-review/hpa.yml
1
```

```
horizontalpodautoscaler.autoscaling/longload created
```

25. In the second terminal, run the `watch oc get hpa longload` command. Wait for the `longload` horizontal pod autoscaler to report usage in the `TARGETS` column. The percentages might be different in your cluster.

26. [student@workstation ~]\$ **watch oc get hpa longload**

27. NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	A
GE						

longload	Deployment/longload	13%/60%	1	3	1	7
5s						

Leave the command running and do not interrupt it.

28. To test your work, run the `oc exec deploy/longload -- curl localhost:3000/leak` command in the first terminal for the application to allocate 480 MiB of memory.

29. [student@workstation ~]\$ **oc exec deploy/longload -- curl -s localhost:3000/leak**

```
longload-7ddcc9b7fd-72dtm: consuming memory!
```

30. In the second terminal, after a few seconds, observe that the `oc get hpa longload` command shows the memory increase. The horizontal pod autoscaler scales up the application to more than one replica. The percentages might be different in your cluster.

31. NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
AGE					

longload	Deployment/longload	145%/60%	1	3	3
5m18s					

32. To test your work, run the `~/D0180/labs/reliability-review/free.sh` script in the first terminal for the application to release the memory. Ensure that the pod that was consuming memory now frees the memory. Execute the `free.sh` script several times if necessary.

```
33. [student@workstation ~]$ ~/D0180/labs/reliability-review/free.sh
```

```
longload-7ddcc9b7fd-72dtm: releasing memory!
```

34. In the second terminal, after ten minutes, observe that the `oc get hpa longload` command shows the memory decrease. The horizontal pod autoscaler scales down the application to one replica. The percentages might be different in your cluster.

35. NAME AGE	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
longload 5m28s	Deployment/longload	12%/60%	1	3	1

Press **Ctrl+C** to end the watch command. Close the second terminal window.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade reliability-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish reliability-review
```

Summary

- A highly available application is resistant to scenarios that might otherwise make it unavailable.
- Kubernetes and RHOCP provide HA features, such as health probes, that help the cluster to route traffic only to working pods.

- Resource requests and limits help to keep cluster node resource usage balanced.
- Horizontal pod autoscalers automatically add or remove replicas based on current resource usage and specified parameters.

Chapter 7. Manage Application Updates

Container Image Identity and Tags

Guided Exercise: Container Image Identity and Tags

Update Application Image and Settings

Guided Exercise: Update Application Image and Settings

Reproducible Deployments with OpenShift Image Streams

Guided Exercise: Reproducible Deployments with OpenShift Image Streams

Automatic Image Updates with OpenShift Image Change Triggers

Guided Exercise: Automatic Image Updates with OpenShift Image Change Triggers

Lab: Manage Application Updates

Summary

Abstract

Goal	Manage reproducible application updates and rollbacks of code and configurations.
Sections	• Container Image Identity and Tags (and Guided Exercise) • Update Application Image and Settings (and Guided Exercise) • Reproducible Deployments with OpenShift Image Streams (and Guided Exercise) • Automatic Image Updates with OpenShift Image Change Triggers (and Guided Exercise)
Lab	• Manage Application Updates

Container Image Identity and Tags

Objectives

- Relate container image tags to their identifier hashes, and identify container images from pods and containers on Kubernetes nodes.

Kubernetes Image Tags

The full name of a container image is composed of several parts. For example, the `registry.redhat.io/rhel9/nginx-124:9.6` image name is composed of the following elements:

- The registry server is `registry.redhat.io`.
- The namespace is `rhel9`.
- The name is `nginx-124`. In this example, the name of the image includes the version of the software, which is Nginx version 1.20.
- The tag, which points to a specific version of the image, is `9.6`. If you omit the tag, then most container tools use the `latest` tag by default.

Multiple tags can refer to the same image version. The following screenshot of the Red Hat Ecosystem Catalog at <https://catalog.redhat.com/software/containers/explore> lists the tags for the `rhel9/nginx-124` image:

Nginx 1.24

rhel9/nginx-124

Builder imageSingle-stream repository

Provided by  **Red Hat**

amd64 ▾

9.6 ▾

9.6-1756959211 1-1756959211 latest 9.6 1

Overview

Description

Nginx is a web server and a reverse proxy server for HTTP, SMTP, POP3 and IMAP protocols, with a strong focus on high concurrency, performance and low memory usage. The container image provides a containerized packaging of the nginx 1.24 daemon. The image can be used as a base image for other

Published

Release category

Generally Available

In this case, the 9.6, latest, and 1 tags point to the same image version. You can use any of these tags to refer to that version.

The latest and 1 tags are *floating tags*, because they can point to different image versions over time. For example, when developers publish a new version of the image, they change the latest tag to point to that new version. They also update the 1 tag to point to the latest release of that version, such as 1-1756959211.

As a user of the image, by specifying a floating tag, you ensure that you always consume the up-to-date image version that corresponds to the tag.

Floating Tag Issues

Vendors, organizations, and developers who publish images manage their tags and establish their own lifecycle for floating tags. They can reassign a floating tag to a new image version without notice.

As a user of the image, you might not notice that the tag that you were using now points to a different image version.

Suppose that you deploy an application on OpenShift and use the `latest` tag for the image. The following series of events might occur:

1. When OpenShift deploys the container, it pulls the image with the `latest` tag from the container registry.
2. Later, the image developer pushes a new version of the image, and reassigns the `latest` tag to that new version.
3. OpenShift relocates the pod to a different cluster node, for example because the original node fails.
4. On that new node, OpenShift pulls the image with the `latest` tag, and thereby retrieves the new image version.
5. Now the OpenShift deployment runs with a new version of the application, without your awareness of that version update.

A similar issue is that when you scale up your deployment, OpenShift starts new pods. On the nodes, OpenShift pulls the `latest` image version for these new pods. As a result, if a new version is available, then your deployment runs with containers that use different versions of the image. Application inconsistencies and unexpected behavior might occur.

To prevent these issues, select an image that is guaranteed not to change over time. You thus gain control over the lifecycle of your application; you can choose when and how OpenShift deploys a new image version.

You can select a static image version in several ways:

- Use a tag that does not change, instead of relying on floating tags.
- Use OpenShift image streams for tight control over the image versions. Another section in this course discusses image streams further.
- Use the *Secure Hash Algorithm* (SHA) image ID instead of a tag when referencing an image version.

The distinction between a floating and non-floating tag is not a technical one, but a convention. A developer is discouraged, although not prevented, from pushing a different image to an existing tag. Thus, you must specify the SHA image ID to guarantee that the referenced container image does not change.

Using an SHA Image ID

Developers assign tags to images. In contrast, an SHA image ID, or *digest*, is a unique identifier that the container registry computes and assigns to images. The SHA ID is an immutable string that refers to a specific image version. Using the SHA ID for identifying an image is the most secure approach.

To refer to an image by its SHA ID, replace `name:tag` with `name@SHA-ID` in the image name. The following example uses the SHA image ID instead of a tag:

```
registry.redhat.io/rhel9/nginx-124@sha256:126903359331eb9f2c87950f952442...
```

To retrieve the SHA image ID from the tag, use the `oc image info` command.

NOTE

A multi-architecture image references images for several CPU architectures. Multi-architecture images include an index that points to the images for different platforms and CPU architectures.

For these images, the `oc image info` command requires you to select an architecture by using the `--filter-by-os` option:

```
[user@host ~]$ oc image info registry.redhat.io/rhel9/nginx-124:9.6
error: the image is a manifest list and contains multiple images - use --filter-by-os
to select from:
```

OS	DIGEST
linux/amd64	sha256:c83789b0c7765e172bee5d36948e63e5db43248d8a3d69a3bb1...
linux/arm64/v8	sha256:dd239a25567a7b3887848b927bf9376fc76c4c4a57290afee55...
linux/ppc64le	sha256:0e9df5ef304708e1244697dc127816f07ece420bc47ea388cf1...
linux/s390x	sha256:0dd7ddd965e11c8a70fbcc66499b72de4e508afb0e4dfca48e8...

The following example displays the SHA ID for the image that the 9.6 tag currently points to:

```
[user@host ~]$ oc image info --filter-by-os linux/amd64 \
```



```
registry.redhat.io/rhel9/nginx-124:9.6
```

```
Name: registry.redhat.io/rhel9/nginx-124:9.6
```

```
Digest: sha256:c83789b0c7765e172bee5d36948e63e5db43248d8a3d69a3bb1...
```

```
Manifest List: sha256:126903359331eb9f2c87950f952442db456750190956b9301d0...
```

```
Media Type: application/vnd.oci.image.manifest.v1+json
```

```
...output omitted...
```

The SHA image ID, or digest, for the specified image tag and architecture.

You can also use the `skopeo inspect` command. The output format differs from the `oc image info` command, although both commands report similar data.

If you use the `oc debug node/node-name` command to connect to a compute node, then you can list the locally available images by running the `crictl images --digests --no-trunc` command. The `--digests` option instructs the command to display the SHA image IDs, and the `--no-trunc` option instructs the command to display the full SHA string. Otherwise, the command displays only the first characters.

```
[user@host ~]$ oc debug node/node-name
```

```
Temporary namespace openshift-debug-csn2p is created for debugging node...
```

```
Starting pod/node-name-debug ...
```

```
To use host binaries, run `chroot /host`
```

```
Pod IP: 192.168.50.10
```

```
If you don't see a command prompt, try pressing enter.
```

```
sh-4.4# chroot /host
```

```
sh-4.4# crictl images --digests --no-trunc \
```

```
registry.redhat.io/rhel9/nginx-124:9.6
```

IMAGE	TAG	DIGEST	IMAGE ID	...
registry.redhat.io/rhel9/nginx-124:	9.6	sha256:c837...71de	2e68...949e	...

The `IMAGE ID` column displays the local image identifier that the container engine assigns to the image. This identifier is not related to the SHA ID.

The container image format relies on SHA-256 hashes to identify several image components, such as the image layers or the image metadata. Because some

commands also report these SHA-256 strings, ensure that you use the SHA-256 hash that corresponds to the SHA image ID. Commands often refer to the SHA image ID as the image digest.

Selecting a Pull Policy

When you deploy an application, OpenShift selects a compute node to run the pod. On that node, OpenShift pulls the image and then starts the container.

By setting the `imagePullPolicy` attribute in the deployment resource, you can control how OpenShift pulls the image.

The following example shows the `myapp` deployment resource:

```
[user@host ~]$ oc get deployment myapp -o yaml
apiVersion: apps/v1
kind: Deployment
...output omitted...
template:
  metadata:
    creationTimestamp: null
    labels:
      app: myapp
  spec:
    containers:
      - image: registry.redhat.io/rhel9/nginx-124:9.6

      imagePullPolicy: IfNotPresent
      name: nginx-124
...output omitted...
```

The image pull policy is set to `IfNotPresent`.

The `imagePullPolicy` attribute can take the following values:

IfNotPresent

If the image is already on the compute node, because another container is using it or because OpenShift pulled the image during a preceding pod run, then OpenShift uses that local image. Otherwise, OpenShift pulls the image from the container registry.

If you use a floating tag in your deployment, and the image with that tag is already on the node, then OpenShift does not pull the image again, even if the floating tag might point to a newer image in the source container registry.

OpenShift sets the `imagePullPolicy` attribute to `IfNotPresent` by default when you use a tag or the SHA ID to identify the image.

Always

OpenShift always verifies whether an updated version of the image is available on the source container registry. To do so, OpenShift retrieves the SHA ID of the image from the registry. If a local image with that same SHA ID is already on the compute node, then OpenShift uses that image. Otherwise, OpenShift pulls the image.

If you use a floating tag in your deployment, and an image with that tag is already on the node, then OpenShift queries the registry anyway to ensure that the tag still points to the same image version. However, if the developer pushed a new version of the image and updated the floating tag, then OpenShift retrieves that new image version.

OpenShift sets the `imagePullPolicy` attribute to `Always` by default when you use the `latest` tag, or when you do not specify a tag.

Never

OpenShift does not pull the image, and expects the image to be already available on the node. Otherwise, the deployment fails.

To use this option, you must prepopulate your compute nodes with the images that you plan to use. You use this mechanism to improve speed or to avoid relying on a container registry for these images.

Pruning Images from Cluster Nodes

When OpenShift deletes a pod from a compute node, it does not remove the associated image. OpenShift can reuse the images without having to pull them again from the remote registry.

Because the images consume disk space on the compute nodes, OpenShift needs to remove, or *prune*, the unused images when disk space becomes sparse. The kubelet process, which runs on the compute nodes, includes a garbage collector that runs every five minutes. If the usage of the file system that stores the images is above 85%, then the garbage collector removes the oldest unused images. Garbage collection stops when the file-system usage drops below 80%.

The references section includes instructions to adjust these default thresholds.

From a compute node, you can run the `crictl imagefsinfo` command to retrieve the name of the file system that stores the images:

```
[user@host ~]$ oc debug node/node-name
Temporary namespace openshift-debug-csn2p is created for debugging node...
Starting pod/node-name-debug ...
To use host binaries, run `chroot /host`
Pod IP: 192.168.50.10
If you don't see a command prompt, try pressing enter.
sh-4.4# chroot /host
sh-4.4# crictl imagefsinfo
{
  "status": {
    "timestamp": "1674465624446958511",
    "fsId": {
      "mountpoint": "/var/lib/containers/storage/overlay-images"
    },
  },
  "usedBytes": {
    "value": "1318560"
  },
  "inodesUsed": {
    "value": "446"
  }
}
```

```
}  
}
```

NOTE

The `oc debug node` and `crictl` commands require cluster administrator privileges.

From the preceding command output, the file system that stores the images is `/var/lib/containers/storage/overlay-images`. The images consume 1318560 bytes of disk space.

From the compute node, you can use the `crictl rmi` command to remove an unused image. However, pruning objects by using the `crictl` command might interfere with the garbage collector and the `kubelet` process.

It is recommended that you rely on the garbage collector to prune unused objects, images, and containers from the compute nodes. The garbage collector is configurable to better fulfill any custom needs. The garbage collector thresholds are configurable via the `KubeletConfig` Custom Resource (CR).

Guided Exercise: Container Image Identity and Tags

Update an application by changing its deployment to reference a newer image tag, and find the hashes of the old and new application images.

Outcomes

Inspect container images, list images of containers that run on compute nodes, and deploy applications by using image tags or SHA IDs.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `updates-ids` project and the `/home/student/D0180/labs/updates-ids/resources.txt` file. The `resources.txt` file contains the name of the images and some commands that you use during the exercise. You can use the file to copy and paste these image names and commands.

```
[student@workstation ~]$ lab start updates-ids
```

Instructions

1. Log in to the OpenShift cluster as the developer user with developer as the password. Use the updates-ids project.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

...output omitted...

5. Set the updates-ids project as the active project.

```
6. [student@workstation ~]$ oc project updates-ids
```

...output omitted...

2. Inspect the two versions of the registry.ocp4.example.com:8443/ubi8/httpd-24 image from the classroom container registry. The classroom setup copied that image from the Red Hat Ecosystem Catalog. The original image is registry.access.redhat.com/ubi8/httpd-24.
1. Use the `oc image info` command to inspect the image version that the 1-209 tag references. Notice the unique SHA ID that identifies the image version.

NOTE

To improve readability, the instructions truncate the SHA-256 strings.

On your system, the commands return the full SHA-256 strings. Also, you must type the full SHA-256 string, to provide such a parameter to a command.

```
[student@workstation ~]$ oc image info \
registry.ocp4.example.com:8443/ubi8/httpd-24:1-209 | head
```

```
Name:          registry.ocp4.example.com:8443/ubi8/httpd-24:1-209
Digest:        sha256:b1e3...f876
...output omitted...
```

2. Inspect the image version that the 1-215 tag references. Notice that the SHA ID, or digest, differs from the preceding image version.

```
3. [student@workstation ~]$ oc image info \
4.   registry.ocp4.example.com:8443/ubi8/httpd-24:1-215 | head
5. Name:          registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
6. Digest:        sha256:91ad...fd83
```

```
...output omitted...
```

7. Use the `skopeo inspect` command to inspect images. The output format differs from the `oc image info` command, although both commands report similar data.

Use the `skopeo login` command to log in to the registry as the developer user with `developer` as the password.

```
[student@workstation ~]$ skopeo login registry.ocp4.example.com:8443 -u dev
eloper
Password: developer
Login Succeeded!
```

8. Use the `skopeo inspect` command to inspect the 1-215 image tag.

```
9. [student@workstation ~]$ skopeo inspect \
10.  docker://registry.ocp4.example.com:8443/ubi8/httpd-24:1-215 | head
11. {
12.   "Name": "registry.ocp4.example.com:8443/ubi8/httpd-24",
13.   "Digest": "sha256:91ad...fd83",
14.   "RepoTags": [
15.     "1-209",
16.     "1-215",
17.     "latest"
```

```
18.    ],
19....output omitted...
```

```
}
```

The `skopeo inspect` command also shows other existing image tags.

3. Deploy an application from the image version that the 1-209 tag references.
1. Use the `oc create deployment` command to deploy the application. Set the name of the deployment to `httpd1`.

```
2. [student@workstation ~]$ oc create deployment httpd1 \
3.   --image registry.ocp4.example.com:8443/ubi8/httpd-24:1-209
```

```
deployment.apps/httpd1 created
```

4. Wait for the pod to start, and then retrieve the name of the cluster node that runs it. You might have to rerun the command several times for the pod to report the `Running` status. The name of the pod on your system probably differs.

```
5. [student@workstation ~]$ oc get pods -o wide
6. NAME                                READY  STATUS   RESTARTS  AGE  IP              NODE
   ...
```

```
httpd1-6dff796d99-pm2x6 1/1    Running  0          19s  10.8.0.104  master0
1 ...
```

7. Retrieve the name of the container that is running inside the pod. The `crictl ps` command that you run in a following step takes the container name as an argument.

```
8. [student@workstation ~]$ oc get deployment httpd1 -o wide
9. NAME      READY  UP-TO-DATE  AVAILABLE  AGE  CONTAINERS  ...
```

```
httpd1  1/1    1          1          1m10s  httpd-24    ...
```

4. Access the cluster node and retrieve the image that the container is using.

1. Log in to the OpenShift cluster as the admin user with redhatocp as the password.

```
2. [student@workstation ~]$ oc login -u admin -p redhatocp
3. Login successful.
```

...output omitted...

4. Use the `oc debug node` command to access the cluster node.

```
5. [student@workstation ~]$ oc debug node/master01
6. Temporary namespace openshift-debug-flz4d is created for debugging node...
7. Starting pod/master01-debug-w9cqk ...
8. To use host binaries, run `chroot /host`
9. Pod IP: 192.168.50.10
10. If you don't see a command prompt, try pressing enter.
```

sh-5.1#

11. In the remote shell, run the `chroot /host` command to use host binaries.

```
12. sh-5.1# chroot /host
```

sh-5.1#

13. Use the `crictl ps` command to confirm that the `httpd-24` container is running. Add the `-o yaml` option to display the container details in YAML format.

```
14. sh-4.4# crictl ps --name httpd-24 -o yaml
15. containers:
16. - annotations:
17. ...output omitted...
18.   image:
19.     annotations: {}
20.     image: registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:b1e3...f876
21. ...output omitted...
22.   imageRef: registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:b1e3...f876
```

23. labels:

24. ...output omitted...

state: **CONTAINER_RUNNING**

Notice that the command refers to the image by its SHA ID, and not by the tag that you specified when you created the deployment resource.

25. Use the `crictl images` command to list the locally available images on the node. The `registry.ocp4.example.com:8443/ubi8/httpd-24:1-209` image is in that list, because the local container engine pulled it when you deployed the `httpd1` application.

NOTE

The `IMAGE ID` column displays the local image identifier that the container engine assigns to the image. This identifier is not related to the SHA image ID that the container registry assigned to the image.

Most `crictl` commands, such as `crictl images` or `crictl rmi`, accept a local image identifier instead of the full image name. For example, you can run the `crictl images 8ee59251acc93` command as a short version of the `crictl images registry.ocp4.example.com:8443/ubi8/httpd-24:1-209` command.

```
sh-5.1# crictl images
```

IMAGE	TAG	IMAGE ID	SIZE
-------	-----	----------	------

...output omitted...

quay.io/openshift-release-dev/ocp-release	<none>	d29424caa281a	517MB
---	--------	---------------	-------

quay.io/openshift-release-dev/ocp-v4.0-art-dev	<none>	42a50bb73e2fb	588MB
--	--------	---------------	-------

quay.io/openshift-release-dev/ocp-v4.0-art-dev	<none>	df18dddad77e7	510MB
--	--------	---------------	-------

quay.io/openshift-release-dev/ocp-v4.0-art-dev	<none>	388587ea693e1	447MB
--	--------	---------------	-------

...output omitted...

registry.ocp4.example.com:8443/ubi8/httpd-24	1-209	8ee59251acc93	461MB
--	-------	---------------	-------

...output omitted...

26. The `crictl images` command does not display the SHA image IDs by default. Rerun the command and add the `--digests` option to display the SHA IDs. Also, add the local image ID to the command to limit the output to the `registry.ocp4.example.com:8443/ubi8/httpd-24:1-209` image.

The command reports only the first characters of the SHA image ID. These characters match the SHA ID of the image that the `httpd-24` container is using. Therefore, the `httpd-24` container is using the expected image.

```
sh-5.1# crictl images --digests 8ee59251acc93
```

IMAGE	TAG	DIGEST	IMAGE ID
...			
registry.ocp4.example.com:8443/ubi8/httpd-24	1-209	b1e3c572516d1	8ee59251acc93 ...

27. Disconnect from the cluster node.

```
28. sh-5.1# exit
29. exit
30. sh-5.1# exit
31. exit
32.
33. Removing debug pod ...
34. Temporary namespace openshift-debug-flz4d was removed.
```

[student@workstation ~]\$

5. Log in as the `developer` user and deploy another application by using the SHA ID of the image as the digest.
1. Log in to the OpenShift cluster as the `developer` user with `developer` as the password.

```
2. [student@workstation ~]$ oc login -u developer -p developer
3. Login successful.
```

...output omitted...

4. Use the `oc image info` command to retrieve the SHA ID of the image version that the `1-209` tag references. Specify the JSON format for the command output. Parse the JSON output with the `jq -r` command to retrieve the value of the `.digest` object.

```
5. [student@workstation ~]$ oc image info \
6.   registry.ocp4.example.com:8443/ubi8/httpd-24:1-209 -o json | \
7.   jq -r .digest
```

```
sha256:b1e3...f876
```

8. Export the SHA ID as the `$IMAGE` environment variable.

```
[student@workstation ~]$ IMAGE=sha256:b1e3...f876
```

9. Use the `oc create deployment` command to deploy the application. Set the name of the deployment to `httpd2`.

```
10. [student@workstation ~]$ oc create deployment httpd2 \
11.   --image registry.ocp4.example.com:8443/ubi8/httpd-24@$IMAGE
```

```
deployment.apps/httpd2 created
```

12. Confirm that the new deployment refers to the image version by its SHA ID.

```
13. [student@workstation ~]$ oc get deployment httpd2 -o wide
14. NAME          READY   ... CONTAINERS   IMAGES ...
```

```
httpd2    1/1     ... httpd-24    registry.../ubi8/httpd-24@sha256:b1e3...f876 ...
```

6. Update the `httpd2` application by using a more recent image version.
1. In the `httpd2` deployment, update the `httpd-24` container to use the image version that the `1-215` tag references.

```
2. [student@workstation ~]$ oc set image deployment/httpd2 \
3.   httpd-24=registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
```

```
deployment.apps/httpd2 image updated
```

4. Confirm that the deployment refers to the new image version.

```
5. [student@workstation ~]$ oc get deployment httpd2 -o wide
```

```
6. NAME          READY    ...    IMAGES ...
```

```
httpd2    1/1      ...    registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
...
```

7. Confirm that the deployment finished redeploying the pod. You might have to rerun the command several times for the pod to report a Running status. The pod names probably differ on your system.

```
8. [student@workstation ~]$ oc get pods
```

```
9. NAME                                READY   STATUS    RESTARTS   AGE
10. httpd1-6dff796d99-pm2x6           1/1     Running   0           118m
```

```
httpd2-998d9b9b9-5859j    1/1     Running   0           21s
```

11. Inspect the pod to confirm that the container is using the new image. Replace the pod name with your own pod from the previous step.

```
12. [student@workstation ~]$ oc get pod httpd2-998d9b9b9-5859j \
```

```
13.   -o jsonpath='{.spec.containers[0].image}'{"\n"}'
```

```
registry.ocp4.example.com:8443/ubi8/httpd-24:1-215
```

7. Add the latest tag to the image version that the 1-209 tag already references. Deploy an application from the image with the latest tag.

1. Use the `skopeo login` command to log in to the classroom container registry as the `developer` user with `developer` as the password.

```
2. [student@workstation ~]$ skopeo login -u developer -p developer \
```

```
3.   registry.ocp4.example.com:8443
```

```
Login Succeeded!
```

4. Use the `skopeo copy` command to add the latest tag to the image.

```
5. [student@workstation ~]$ skopeo copy \  
6.   docker://registry.ocp4.example.com:8443/ubi8/httpd-24:1-209 \  
7.   docker://registry.ocp4.example.com:8443/ubi8/httpd-24:latest  
8. Getting image source signatures  
9. ...output omitted...
```

```
Writing manifest to image destination
```

10. Use the `oc image info` command to confirm that both tags refer to the same image. The two commands report the same SHA image ID, which indicates that the tags point to the same image version.

```
11. [student@workstation ~]$ oc image info \  
12.   registry.ocp4.example.com:8443/ubi8/httpd-24:1-209 | head  
13. Name:           registry.ocp4.example.com:8443/ubi8/httpd-24:1-209  
14. Digest:         sha256:b1e3...f876
```

```
...output omitted...
```

```
[student@workstation ~]$ oc image info \  
   registry.ocp4.example.com:8443/ubi8/httpd-24:latest | head  
Name:           registry.ocp4.example.com:8443/ubi8/httpd-24:latest  
Digest:         sha256:b1e3...f876  
...output omitted...
```

15. Use the `oc create deployment` command to deploy another application. Set the name of the deployment to `httpd3`. To confirm that by default the command selects the `latest` tag, do not provide the tag part in the image name.

```
16. [student@workstation ~]$ oc create deployment httpd3 \  
17.   --image registry.ocp4.example.com:8443/ubi8/httpd-24
```

```
deployment.apps/httpd3 created
```

18. Confirm that the pod is running. You might have to rerun the command several times for the pod to report the `Running` status. The pod names probably differ on your system.

```
19. [student@workstation ~]$ oc get pods
```

20. NAME	READY	STATUS	RESTARTS	AGE
21. httpd1-6dff796d99-pm2x6	1/1	Running	0	150m
22. httpd2-998d9b9b9-5859j	1/1	Running	0	32m

httpd3-85b978d758-fvqdr	1/1	Running	0	42s
--------------------------------	-----	----------------	---	-----

23. Confirm that the pod is using the expected image. Notice that the SHA image ID corresponds to the image that the 1-209 tag references. You retrieved that SHA image ID in a preceding step when you ran the `oc image info` command.

```
24. [student@workstation ~]$ oc describe pod httpd3-85b978d758-fvqdr | \
```

```
25.   grep ID
```

```
26.     Container ID: cri-o://baa3...7c90
```

```
      Image ID:      registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:b1e3.
      ..f876
```

8. Assign the `latest` tag to a different image version. This operation simulates a developer who pushes a new version of an image and assigns the `latest` tag to that new image version.

1. Use the `skopeo copy` command to add the `latest` tag to the image version that the 1-215 tag already references. The command automatically removes the `latest` tag from the earlier image.

```
2. [student@workstation ~]$ skopeo copy \
```

```
3.   docker://registry.ocp4.example.com:8443/ubi8/httpd-24:1-215 \
```

```
4.   docker://registry.ocp4.example.com:8443/ubi8/httpd-24:latest
```

```
5. Getting image source signatures
```

```
6. ...output omitted...
```

```
7. Writing manifest to image destination
```

```
      Storing signatures
```

8. Log out of the classroom container registry.

```
9. [student@workstation ~]$ skopeo logout registry.ocp4.example.com:8443
```

```
Removed login credentials for registry.ocp4.example.com:8443
```

NOTE

The `skopeo logout` command logs out of a specified registry server by deleting the cached credentials in the `${XDG_RUNTIME_DIR}/containers/auth.json` file.

Red Hat recommends removing cached credentials that are no longer required.

10. Even though the `latest` tag is now referencing a different image version, OpenShift does not redeploy the pods that are running with the previous image version.

Rerun the `oc describe pod` command to confirm that the pod still uses the preceding image.

```
[student@workstation ~]$ oc describe pod httpd3-85b978d758-fvqdr | \
grep ID
    Container ID: cri-o://2cee...3a68
    Image ID:      registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:b1e3.
..f876
```

9. Scale the `httpd3` deployment to two pods.
 1. Use the `oc scale` command to add a new pod to the deployment.

```
2. [student@workstation ~]$ oc scale deployment/httpd3 --replicas 2
```

```
deployment.apps/httpd3 scaled
```

3. List the pods to confirm that two pods are running for the `httpd3` deployment. The pod names probably differ on your system.

```
4. [student@workstation ~]$ oc get pods
5. httpd1-6dff796d99-pm2x6    1/1    Running    0        75m
6. httpd2-998d9b9b9-5859j    1/1    Running    0        30m
7. httpd3-85b978d758-f98jh    1/1    Running    0        54s
```



```
httpd3-85b978d758-fvqdr    1/1    Running    0    11m
```

8. Retrieve the SHA image ID for the pod that the deployment initially created. The ID did not change. The container is still using the original image version.

```
9. [student@workstation ~]$ oc describe pod httpd3-85b978d758-fvqdr | \
10.   grep ID
11.     Container ID: cri-o://2cee...3a68
```

```
Image ID:      registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:b1e3.
..f876
```

12. Retrieve the SHA image ID for the additional pod. Notice that the ID is different. The additional pod is using the image that the latest tag is currently referencing.

```
13. [student@workstation ~]$ oc describe pod httpd3-85b978d758-f98jh | \
14.   grep ID
15.     Container ID: cri-o://d254...c893
```

```
Image ID:      registry.ocp4.example.com:8443/ubi8/httpd-24@sha256:91ad.
..fd83
```

The state of the deployment is inconsistent. The two replicated pods use a different image version. Consequently, the scaled application might not behave correctly. Red Hat recommends that you use a less volatile tag than `latest` in production environments, or that you tightly control the tag assignments in your container registry.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish updates-ids
```

Update Application Image and Settings

Objectives

- Update applications with minimal downtime by using deployment strategies.

Application Code, Configuration, and Data

Modern applications loosely couple code, configuration, and data. Configuration files and data are not hardcoded as part of the software. Instead, the software loads the configuration and data from an external source. Externalizing configuration and data enables deploying an application to different environments without requiring a change to the application source code.

OpenShift provides configuration map, secret, and volume resources to store the application configuration and data. The application code is available through container images.

Because OpenShift deploys applications from container images, developers must build a new version of the image when they update the code of their application. Organizations usually use a continuous integration and continuous delivery (CI/CD) pipeline to automatically build the image from the application source code, and then push the resulting image to a container registry.

You use OpenShift resources, such as configuration maps and secrets, to update the application configuration. To control the deployment process of a new image version, use a Deployment object.

Deployment Strategies

Deploying functional application changes or new versions to users is a significant phase of the CI/CD pipelines, where you add value to the development process.

Introducing application changes carries risks, such as downtime during the deployment, bugs, or reduced application performance. You can reduce or mitigate some risks with testing and validation stages in the pipelines.

Application or service downtime can result in lost business, disruption to other services that depend on yours, and violations of the service-level agreements, among others. To reduce downtime and minimize risks in deployments, use a

deployment strategy. A deployment strategy changes or upgrades an application to minimize the impact of those changes.

In OpenShift, you use `Deployment` objects to define deployments and deployment strategies. The `RollingUpdate` and `Recreate` strategies are the main OpenShift deployment strategies.

To select the `RollingUpdate` or `Recreate` strategies, you set the `.spec.strategy.type` property of the `Deployment` object. The following snippet shows a `Deployment` object that uses the `Recreate` strategy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  ...output omitted...
spec:
  progressDeadlineSeconds: 600
  replicas: 10
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: myapp2
  strategy:
    type: Recreate
  template:
    ...output omitted...
```

Rolling Update Strategy

The `RollingUpdate` strategy replaces one instance after another until all instances are replaced. In this strategy, both versions of the application run simultaneously, and the `RollingUpdate` strategy scales down instances of the previous version only when the new version is ready. The main drawback is that this strategy requires compatibility between the versions in the deployment.

The following graphic shows the deployment of a new version of an application by using the `RollingUpdate` strategy:

1. Some application instances run a code version that needs updating (v1). OpenShift scales up a new instance with the updated application version (v2). Because the new instance with version v2 is not ready, only the version v1 instances fulfill customer requests.
2. The instance with v2 is ready and accepts customer requests. OpenShift scales down an instance with version v1, and scales up a new instance with version v2. Both versions of the application fulfill customer requests.
3. The new instance with v2 is ready and accepts customer requests. OpenShift scales down the remaining instance with version v1.
4. No instances remain to replace. The application update was successful and without downtime.

The RollingUpdate strategy supports continuous deployment, and eliminates application downtime during deployments. You can use this strategy if the different versions of your application can run at the same time.

NOTE

The RollingUpdate strategy is the default strategy if you do not specify a strategy on the Deployment objects.

The following snippet shows a Deployment object that uses the RollingUpdate strategy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  ...output omitted...
spec:
  progressDeadlineSeconds: 600
  replicas: 10
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: myapp2
  strategy:
    rollingUpdate:
```

```
    maxSurge: 25%
    maxUnavailable: 50%
    type: RollingUpdate
  template:
...output omitted...
```

Out of the many parameters to configure the `RollingUpdate` strategy, the preceding snippet shows the `maxSurge` and `maxUnavailable` parameters.

During a rolling update, the number of pods for the application varies, because OpenShift starts new pods for the new revision and removes pods from the previous revision. The `maxSurge` parameter indicates how many pods OpenShift can create above the normal number of replicas. The `maxUnavailable` parameter indicates how many pods OpenShift can remove below the normal number of replicas. You can express these parameters as percentages or as the number of pods.

If you do not configure a readiness probe for your deployment, then during a rolling update, OpenShift starts sending client traffic to new pods as soon as they are running. However, the application inside a container might not be immediately ready to accept client requests, which might need to load files to cache, to establish a network connection to a database, or to perform initial tasks that might take time to complete. Consequently, OpenShift redirects client requests to a container that is not yet ready, and these requests fail.

Adding a readiness probe to the deployment prevents OpenShift from sending traffic to new pods that are not ready.

Recreate Strategy

In this strategy, all the instances of an application are stopped first, and are then replaced with new ones. The major drawback of this strategy is that it causes downtime in your services. For a period, no application instances are available to fulfill requests.

The following graphic shows the deployment of a new version of an application that uses the `Recreate` strategy:

1. The application has some instances that run a code version to update (v1).
2. OpenShift scales down the running instances to zero. Scaling down to zero causes application downtime, because no instances are available to fulfill requests.
3. OpenShift scales up new instances with a new version of the application (v2). When the new instances are booting, the downtime continues.
4. Starting the new instances resolves the application outage and completes the Recreate strategy.

The Recreate strategy is not recommended for applications that need high availability, for example, medical systems. You can use this strategy when your application cannot have different simultaneously running code versions. You might also use this strategy to execute data migrations or data transformations before the new code starts.

Rolling out Applications

When you update a `Deployment` object, OpenShift automatically rolls out the application. If you apply several modifications in a row, such as modifying the image version, updating the environment variables, and configuring the readiness probe, then OpenShift rolls out the application for each modification.

To prevent these multiple deployments, pause the rollout, apply all your modifications to the `Deployment` object, and then resume the rollout. OpenShift then performs a single rollout to apply all your modifications to the `Deployment` object.

- Use the `oc rollout pause deployment/myapp` command for the `myapp` deployment:

```
[user@host ~]$ oc rollout pause deployment/myapp
```

- The following commands modify the image, an environment variable, and the readiness probe:

- `[user@host ~]$ oc set image deployment/myapp \`
- `nginx-120=registry.access.redhat.com/ubi9/nginx-120:1-86`
- `[user@host ~]$ oc set env deployment/myapp NGINX_LOG_TO_VOLUME=1`

```
[user@host ~]$ oc set probe deployment/myapp --readiness --get-url http://:8080
```

- Resume the rollout of the `myapp` deployment:

```
[user@host ~]$ oc rollout resume deployment/myapp
```

OpenShift rolls out the application to apply all your modifications to the `Deployment` object.

You can follow a similar process when you create and configure a new deployment:

- Create the deployment, and set the number of replicas to zero. By setting the replicas to zero, OpenShift does not roll out your application, and no pods are running:

```
[user@host ~]$ oc create deployment myapp2 \
```

```
--image registry.access.redhat.com/ubi9/nginx-120:1-86 --replicas 0
```

- Confirm that the deployment has no running pods:

```
[user@host ~]$ oc get deployment/myapp2
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
myapp2	0/0	0	0	9s

- Apply the configuration to the `Deployment` object. The following command adds a readiness probe:

```
[user@host ~]$ oc set probe deployment/myapp2 --readiness --get-url http://:8080
```

- Scale up the deployment and OpenShift rolls out the application:

```
[user@host ~]$ oc scale deployment/myapp2 --replicas 10
```

```
[user@host ~]$ oc get deployment/myapp2
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
myapp2	10/10	10	10	18s

Monitoring Replica Sets

When OpenShift rolls out an application from the `Deployment` object, it creates a `ReplicaSet` object. Replica sets are responsible for creating and monitoring the pods. If a pod fails, then the `ReplicaSet` object deploys a new one.

To deploy pods, replica sets use the pod template definition from the `Deployment` object. OpenShift copies the template definition from the `Deployment` object when it creates the `ReplicaSet` object.

When you update the `Deployment` object, OpenShift does not update the existing `ReplicaSet` object. Instead, it creates another `ReplicaSet` object with the new pod template definition. Then, OpenShift rolls out the application according to the update strategy.

Therefore, several `ReplicaSet` objects for a deployment can exist at the same time on your system. During a rolling update, the earlier and the new `ReplicaSet` objects coexist and coordinate the rollout of the new application version. After the rollout completes, OpenShift keeps the earlier `ReplicaSet` object so that you can roll back if the new application version does not operate correctly.

The following graphic shows a `Deployment` object and two `ReplicaSet` objects. The earlier `ReplicaSet` object for version 1 of the application does not run any pods. The current `ReplicaSet` object for version 2 of the application manages three replicated pods.

Do not directly change or delete `ReplicaSet` objects, because OpenShift manages them through the associated `Deployment` objects.

The `.spec.revisionHistoryLimit` attribute in `Deployment` objects specifies how many `ReplicaSet` objects OpenShift keeps. OpenShift automatically deletes the extra `ReplicaSet` objects. Also, when you delete a `Deployment` object, OpenShift deletes all the associated `ReplicaSet` objects.

Run the `oc get replicaset` command to list the `ReplicaSet` objects. OpenShift uses the `Deployment` object name as a prefix for the `ReplicaSet` objects.

```
[user@host ~]$ oc get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
------	---------	---------	-------	-----

myapp2-574968dd59	0	0	0	3m27s
myapp2-76679885b9	10	10	10	22s
myapp2-786cbf9bc8	0	0	0	114s

The preceding output shows three ReplicaSet objects for the myapp2 deployment. Whenever you modified the myapp2 deployment, OpenShift created a ReplicaSet object. The second object in the list is active and monitors 10 pods. The other ReplicaSet objects do not manage any pods.

During a rolling update, two ReplicaSet objects are active. The earlier ReplicaSet object is scaling down, and at the same time the new ReplicaSet object is scaling up:

```
[user@host ~]$ oc get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
myapp2-574968dd59	0	0	0	13m
myapp2-5fb5766df5	4	4	2	21s
myapp2-76679885b9	8	8	8	10m
myapp2-786cbf9bc8	0	0	0	11m

The new ReplicaSet object already started four pods, but the READY column shows that the readiness probe succeeded for only two pods so far. These two pods are likely to receive client traffic.

The ReplicaSet object already scaled down from 10 to 8 pods.

Managing Rollout

Because OpenShift preserves ReplicaSet objects from earlier deployment versions, you can roll back if you notice that the new version of the application does not work.

Use the `oc rollout undo` command to roll back to the preceding deployment version. The command uses the existing ReplicaSet object for that version to roll back the pods. The command also reverts the Deployment object to the preceding version.

```
[user@host ~]$ oc rollout undo deployment/myapp2
```

Use the `oc rollout status` command to control the rollout process:

```
[user@host ~]$ oc rollout status deployment/myapp2
deployment "myapp2" successfully rolled out
```

If the rollout operation fails, because you specify a wrong container image name or the readiness probe fails, then OpenShift does not automatically roll back your deployment. In this case, run the `oc rollout undo` command to revert to the preceding working configuration.

By default, the `oc rollout undo` command rolls back to the preceding deployment version. If you need to roll back to an earlier revision, then list the available revisions and add the `--to-revision` option to the `oc rollout undo` command.

- Use the `oc rollout history` command to list the available revisions:

```
[user@host ~]$ oc rollout history deployment/myapp2
deployment.apps/myapp2
REVISION  CHANGE-CAUSE
1          <none>
3          <none>
4          <none>
```

```
5          <none>
```

NOTE

The `CHANGE-CAUSE` column provides a user-defined message that describes the revision. You can store the message in the `kubernetes.io/change-cause` deployment annotation after every rollout:

```
[user@host ~]$ oc annotate deployment/myapp2 \
kubernetes.io/change-cause="Image updated to 1-86"
deployment.apps/myapp2 annotated
[user@host ~]$ oc rollout history deployment/myapp2
```

```
deployment.apps/myapp2
REVISION  CHANGE-CAUSE
1          <none>
3          <none>
4          <none>
5          Image updated to 1-86
```

- Add the `--revision` option to the `oc rollout history` command for more details about a specific revision:

- ```
[user@host ~]$ oc rollout history deployment/myapp2 --revision 1
```
- deployment.apps/myapp2 with revision #1
  - Pod Template:
    - Labels: app=myapp2
    - pod-template-hash=574968dd59
  - Containers:
    - nginx-120:
      - Image: registry.access.redhat.com/ubi9/nginx-120:1-86
      - Port: <none>
      - Host Port: <none>
      - Environment: <none>
      - Mounts: <none>

```
Volumes: <none>
```

The `pod-template-hash` attribute is the suffix of the associated `ReplicaSet` object. For example, you can inspect the `ReplicaSet` object for more details by using the `oc describe replicaset myapp2-574968dd59` command.

- Roll back to a specific revision by adding the `--to-revision` option to the `oc rollout undo` command:

```
[user@host ~]$ oc rollout undo deployment/myapp2 --to-revision 1
```

If you use floating tags to refer to container image versions in deployments, then the resulting image when you roll back a deployment might have changed in the container registry. As a result, the image that you run after the rollback might not be the original one that you used.

To prevent this issue, use the OpenShift image streams for referencing images instead of floating tags. A later section in this course discusses the OpenShift image streams in more detail.

## Guided Exercise: Update Application Image and Settings

Update the manifests of a database and a web application while minimizing interruption of service to their users.

### Outcomes

You should be able to pause, update, and resume a deployment, and roll back a failing application.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `updates-rollout-db` project and deploys a MySQL database in that project. It creates the `updates-rollout-web` project and then deploys a web application with 10 replicas.

The command creates the `/home/student/D0180/labs/updates-rollout/resources.txt` file. The `resources.txt` file contains the name of the images and some commands that you use during the exercise. You can use the file to copy and paste these image names and commands.

```
[student@workstation ~]$ lab start updates-rollout
```

### Instructions

1. Log in to the OpenShift cluster as the `developer` user with the `developer` password. Use the `updates-rollout-db` project.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

*...output omitted...*

5. Set the updates-rollout-db project as the active project.

```
6. [student@workstation ~]$ oc project updates-rollout-db
```

*...output omitted...*

2. Review the resources that the lab command created. Confirm that you can connect to the database. The MySQL database uses ephemeral storage.

1. List the Deployment object and confirm that the pod is available. Retrieve the name of the container. You use that information when you update the container image in another step.

```
2. [student@workstation ~]$ oc get deployment -o wide
```

```
3. NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS ...
```

```
mydb 1/1 0 1 17m mysql-80 ...
```

4. List the pods and confirm that the pod is running. The name of the pod on your system probably differs.

```
5. [student@workstation ~]$ oc get pod
```

```
6. NAME READY STATUS RESTARTS AGE
```

```
mydb-5c79866d48-5xzkk 1/1 Running 0 18m
```

7. Retrieve the name of the image that the pod is using. The pod is using the `rhel9/mysql-80` image version 1-224. Replace the pod name with your own from the previous step.

```
8. [student@workstation ~]$ oc get pod mydb-5c79866d48-5xzkk \
```

```
9. -o jsonpath='{.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/rhel9/mysql-80:1-224
```

The classroom setup copied that image from the Red Hat Ecosystem Catalog. The original image is `registry.redhat.io/rhel9/mysql-80`.

10. Confirm that you can connect to the database system by listing the available databases. Run the `mysql` command from inside the pod and connect as the `operator1` user by using `test` as the password.

```
11. [student@workstation ~]$ oc rsh mydb-5c79866d48-5xzkk \
12. mysql --user=operator1 --password=test -e "SHOW DATABASES"
13. mysql: [Warning] Using a password on the command line interface can be insecure.
14. +-----+
15. | Database |
16. +-----+
17. | information_schema |
18. | performance_schema |
19. | quotes |
```

```
+-----+
```

3. You must implement several updates to the `Deployment` object. Pause the deployment to prevent OpenShift from rolling out the application for each modification that you make. After you pause the deployment, change the password for the `operator1` database user, update the container image, and then resume the deployment.

1. Pause the `mydb` deployment.

```
2. [student@workstation ~]$ oc rollout pause deployment/mydb
```

```
deployment.apps/mydb paused
```

3. Change the password of the `operator1` database user to `redhat123`. To change the password, update the `MYSQL_PASSWORD` environment variable in the pod template of the `Deployment` object.

```
4. [student@workstation ~]$ oc set env deployment/mydb MYSQL_PASSWORD=redhat123
```

```
deployment.apps/mydb updated
```

5. Because the Deployment object is paused, confirm that the new password is not yet active. To do so, rerun the `mysql` command by using the current password. The database connection succeeds.

```
6. [student@workstation ~]$ oc rsh mydb-5c79866d48-5xzkk \
7. mysql --user=operator1 --password=test -e "SHOW DATABASES"
8. mysql: [Warning] Using a password on the command line interface can be insecure.
9. +-----+
10. | Database |
11. +-----+
12. | information_schema |
13. | performance_schema |
14. | quotes |
```

```
+-----+
```

15. Update the MySQL container image to the 1-228 version.

```
16. [student@workstation ~]$ oc set image deployment/mydb \
17. mysql-80=registry.ocp4.example.com:8443/rhel9/mysql-80:1-228
```

```
deployment.apps/mydb image updated
```

18. Because the Deployment object is paused, confirm that the pod still uses the 1-224 image version.

```
19. [student@workstation ~]$ oc get pod mydb-5c79866d48-5xzkk \
20. -o jsonpath='{.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/rhel9/mysql-80:1-224
```

21. Resume the `mydb` deployment.

```
22. [student@workstation ~]$ oc rollout resume deployment/mydb
```

```
deployment.apps/mydb resumed
```

23. Confirm that the new rollout completes by waiting for the new pod to be running. The name of the pod on your system probably differs.

```
24. [student@workstation ~]$ oc get pods
```

```
25. NAME READY STATUS RESTARTS AGE
```

```
mydb-dd5dcbddb-rmf85 1/1 Running 0 2m2s
```

4. Verify that OpenShift applied all your modifications to the Deployment object.

1. Retrieve the name of the image that the new pod is using. In the following command, use the name of the new pod as a parameter to the `oc get pod` command. The pod is now using the 1-228 image version.

```
2. [student@workstation ~]$ oc get pod mydb-dd5dcbddb-rmf85 \
```

```
3. -o jsonpath='{.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/rhel9/mysql-80:1-228
```

4. Confirm that you can connect to the database system by using the new password, `redhat123`, for the `operator1` database user.

```
5. [student@workstation ~]$ oc rsh mydb-dd5dcbddb-rmf85 \
```

```
6. mysql --user=operator1 --password=redhat123 -e "SHOW DATABASES"
```

```
7. mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
8. +-----+
```

```
9. | Database |
```

```
10. +-----+
```

```
11. | information_schema |
```

```
12. | performance_schema |
```

```
13. | quotes |
```

```
+-----+
```



5. In the second part of the exercise, you perform a rolling update of a replicated web application. Use the `updates-rollout-web` project and review the resources that the `lab` command created.

1. Set the `updates-rollout-web` project as the active project.

```
2. [student@workstation ~]$ oc project updates-rollout-web
```

```
...output omitted...
```

3. List the Deployment object and confirm that the pods are available. Retrieve the name of the containers. You use that information when you update the container image in another step.

```
4. [student@workstation ~]$ oc get deployment -o wide
```

```
5. NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS ...
```

```
version 10/10 10 10 32m versioned-hello ...
```

6. List the ReplicaSet objects. Because OpenShift did not yet perform rolling updates, only one ReplicaSet object exists. The name of the ReplicaSet object on your system probably differs.

```
7. [student@workstation ~]$ oc get replicaset
```

```
8. NAME DESIRED CURRENT READY AGE
```

```
version-7bfff6b5b4 10 10 10 11m
```

9. Retrieve the name and version of the image that the ReplicaSet object uses to deploy the pods. The pods are using the `redhattraining/versioned-hello` image version `v1.0`.

```
10. [student@workstation ~]$ oc get replicaset version-7bfff6b5b4 \
```

```
11. -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0
```

12. Confirm that the version deployment includes a readiness probe. The probe performs an HTTP GET request on port 8080.

```
13. [student@workstation ~]$ oc get deployment version \
```

```
14. -o jsonpath='{.spec.template.spec.containers[0].readinessProbe}' | jq .
```

```
15. {
16. "failureThreshold": 3,
17. "httpGet": {
18. "path": "/",
19. "port": 8080,
20. "scheme": "HTTP"
21. },
22. "initialDelaySeconds": 3,
23. "periodSeconds": 10,
24. "successThreshold": 1,
25. "timeoutSeconds": 1
```

```
}
```

6. To watch the rolling update that you cause in a following step, open a new terminal window and then run the `~/D0180/labs/updates-rollout/curl_loop.sh` script that the lab command prepared. The script sends web requests to the application in a loop.

1. Open a new terminal.
2. Run the `/home/student/D0180/labs/updates-rollout/curl_loop.sh` script. Leave the script running and do not interrupt it.

```
3. [student@workstation ~]$ /home/student/D0180/labs/updates-rollout/curl_loop.sh
4. Hi!
5. Hi!
6. Hi!
7. Hi!
```

*...output omitted...*

7. Change the container image of the version deployment. The new application version creates a web page with a different message.

1. Switch back to the first terminal window, and then use the `oc set image` command to update the deployment.

```
2. [student@workstation ~]$ oc set image deployment/version \
```

```
3. versioned-hello=registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.1
```

```
deployment.apps/version image updated
```

4. Changing the image caused a rolling update. Watch the output of the `curl_loop.sh` script in the second terminal. If the output of the `curl_loop.sh` is not updating, then press **Ctrl+c** to stop the script in the second terminal. Then, restart the script.

Before the update, only pods that run the `v1.0` version of the application reply. During the rolling updates, both old and new pods are responding. After the update, only pods that run the `v1.1` version of the application reply. The following output probably differs on your system.

```
...output omitted...
Hi!
Hi!
Hi!
Hi!
Hi! v1.1
Hi! v1.1
Hi!
Hi! v1.1
Hi!
Hi! v1.1
Hi! v1.1
Hi! v1.1
Hi! v1.1
Hi! v1.1
Hi! v1.1
Hi! v1.1
...output omitted...
```

Do not stop the script.

8. Confirm that the rollout process is successful. List the `ReplicaSet` objects and verify that the new object uses the new image version.

1. Use the `oc rollout status` command to confirm that the rollout process is successful.

```
2. [student@workstation ~]$ oc rollout status deployment/version
```

```
deployment "version" successfully rolled out
```

3. List the ReplicaSet objects. The initial object scaled down to zero pods. The new ReplicaSet object scaled up to 10 pods. The names of the ReplicaSet objects on your system probably differ.

```
4. [student@workstation ~]$ oc get replicaset
```

| 5. NAME               | DESIRED | CURRENT | READY | AGE |
|-----------------------|---------|---------|-------|-----|
| 6. version-7bfff6b5b4 | 0       | 0       | 0     | 28m |

|                   |    |    |    |       |
|-------------------|----|----|----|-------|
| version-b7fddfc8c | 10 | 10 | 10 | 3m40s |
|-------------------|----|----|----|-------|

7. Retrieve the name and version of the image that the new ReplicaSet object uses. This image provides the new version of the application.

```
8. [student@workstation ~]$ oc get replicaset version-b7fddfc8c \
9. -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.1
```

9. Roll back the version deployment.

1. Use the `oc rollout undo` command to roll back to the initial application version. Ignore the warning message.

```
2. [student@workstation ~]$ oc rollout undo deployment/version
```

```
deployment.apps/version rolled back
```

3. Watch the output of the `curl_loop.sh` script in the second terminal. The pods that run the `v1.0` version of the application are responding again. The following output probably differs on your system.

```
4. ...output omitted...
```

```
5. Hi! v1.1
```

```
6. Hi! v1.1
```

```
7. Hi! v1.1
8. Hi! v1.1
9. Hi!
10. Hi! v1.1
11. Hi!
12. Hi! v1.1
13. Hi! v1.1
14. Hi!
15. Hi!
16. Hi!
```

*...output omitted...*

Press **Ctrl+C** to quit the script. Close that second terminal when done.

17. List the ReplicaSet objects. The initial object scaled up to 10 pods. The object for the new application version scaled down to zero pods.

```
18. [student@workstation ~]$ oc get replicaset
19. NAME DESIRED CURRENT READY AGE
20. version-7bfff6b5b4 10 10 10 52m
```

```
version-b7fddfc8c 0 0 0 27m
```

## Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish updates-rollout
```

# Reproducible Deployments with OpenShift Image Streams

## Objectives

- Ensure reproducibility of application deployments by using image streams and short image names.

## Image Streams

Image streams provide a crucial abstraction layer for managing container images within a Red Hat OpenShift Container Platform (RHOCP) environment. For enterprises, this abstraction is key to establishing secure, consistent, and efficient software supply chains. By decoupling application definitions from the specific location and version of an image, image streams enable robust CI/CD pipelines, simplify image promotion across environments, and enhance security by controlling which images are available for deployment.

The distinction between a floating and a non-floating tag is a convention rather than technical. A developer is discouraged, although not prevented, from pushing a different image to an existing tag. Therefore, a pod that references an image tag is not guaranteed to retrieve the same image when the pod is restarted. Image streams resolve this problem and also provide essential rollback capabilities.

Image streams are one of the main differentiators between OpenShift and upstream Kubernetes. Whereas Kubernetes resources reference container images directly, OpenShift resources, such as build configurations, reference image streams. OpenShift also extends Kubernetes resources, such as Kubernetes Deployments, with annotations that enable them to use image streams.

With image streams, OpenShift can ensure reproducible, stable deployments of containerized applications and also rollbacks of deployments to their latest known-good state.

Image streams provide a stable, short name to reference a container image that is independent of any registry server and container runtime configuration.

## ***Image Stream Tags***

An *image stream* represents one or more sets of container images. Each set, or stream, is identified by an *image stream tag*. Unlike container images in a registry server, which have multiple tags from the same image repository, an image stream can have multiple image stream tags that reference container images from different registry servers and image repositories.

An image stream provides default configurations for a set of image stream tags. Each image stream tag references one stream of container images, and can override most configurations from its associated image stream.

In the following illustration, a deployment that uses the `Image-4` image can be rolled back to using the `Image-3` image, because the `ISTAG-B` image stream tag keeps a history of used images:

An image stream tag stores a copy of the metadata about its current container image, including the SHA image ID. The SHA image ID is a unique identifier that the container registry computes and assigns to images. Storing metadata supports faster search and inspection of container images, because you do not need to reach its source registry server.

You can also configure an image stream tag to store the source image layers in the OpenShift internal container registry, which acts as a local image cache. Storing image layers locally avoids the need to fetch these layers from their source registry server. Consumers of the cached image, such as pods and deployments, reference the internal registry as the source registry of the image.

To better visualize the relationship between image streams and image stream tags, you can explore the `openshift` project that is pre-created in all OpenShift clusters. You can see many image streams in that project, including the `php` image stream:

```
[user@host ~]$ oc get is -n openshift -o name
...output omitted...
imagestream.image.openshift.io/nodejs
imagestream.image.openshift.io/perl
imagestream.image.openshift.io/php
imagestream.image.openshift.io/postgresql
imagestream.image.openshift.io/python
...output omitted...
```

Several tags exist for the `php` image stream, and an image stream tag resource exists for each tag:

```
[user@host ~]$ oc get istag -n openshift | grep php
```

|          |                    |            |
|----------|--------------------|------------|
| 8.0-ubi9 | image-registry ... | 6 days ago |
| 8.0-ubi8 | image-registry ... | 6 days ago |
| 7.4-ubi8 | image-registry ... | 6 days ago |
| 7.3-ubi7 | image-registry ... | 6 days ago |

The `oc describe` command on an image stream shows information from both the image stream and its image stream tags:

```
[user@host ~]$ oc describe is php -n openshift
Name: php
Namespace: openshift
...output omitted...
Tags: 5

8.0-ubi9
 tagged from registry.access.redhat.com/ubi9/php-80:latest
...output omitted...

8.0-ubi8 (latest)
 tagged from registry.access.redhat.com/ubi8/php-80:latest
...output omitted...

7.4-ubi8
 tagged from registry.access.redhat.com/ubi8/php-74:latest
...output omitted...

7.3-ubi7
 tagged from registry.access.redhat.com/ubi7/php-73:latest
...output omitted...
```

In the previous example, each of the php image stream tags refers to a different image name.



## ***Image Names, Tags, and IDs***

The textual name of a container image is a string. Although the image name is sometimes interpreted as consisting of multiple components, such as `registry-host-name/repository-or-organization-or-user-name/image-name:tag-name`, splitting the image name into its components is a matter of convention, not of structure.

An SHA image ID is an SHA-256 hash that uniquely identifies an immutable container image. You cannot modify a container image. Instead, you create a container image with a new ID. When you push a new container image to a registry server, the server associates the existing textual name with the new image ID.

When you start a container from an image name, you download the image that is currently associated with that image name. The image ID behind that name might change at any moment, and the next container that you start might have a different image ID. If the image that is associated with an image name has any issues, and you know only the image name, then you cannot roll back to an earlier image.

OpenShift image stream tags keep a history of the latest image IDs that they fetched from a registry server. The history of image IDs is the stream of images from an image stream tag. You can use the history inside an image stream tag to roll back to a previous image, if for example a new container image causes a deployment error.

Updating a container image in an external registry does not automatically update an image stream tag. The image stream tag keeps the reference to the last image ID that it fetched. The image stream tag behavior is crucial to scaling applications, because it isolates OpenShift from changes that happen on a registry server.

Suppose that you deploy an application from an external registry, and after a few days of testing with some users, you decide to scale its deployment to enable a larger user population. In the meantime, your vendor updates the container image on the external registry. If OpenShift had no image stream tags, then the new pods would get the new container image, which is different from the image on the original pod. Depending on the changes, this new image could cause your application to fail. Because OpenShift stores the image ID of the original image in an image stream tag, it can create pods by using the same image ID and avoid any incompatibility between the original and the updated image.

OpenShift keeps the image ID of the first pod, and ensures that new pods use the same image ID. OpenShift ensures that all pods use the same image.

To better visualize the relationship between an image stream, an image stream tag, an image name, and an image ID, refer to the following `oc describe is` command, which shows the source image and current image ID for each image stream tag:

```
[user@host ~]$ oc describe is php -n openshift
Name: php
Namespace: openshift
...output omitted...

8.0-ubi9
 tagged from registry.access.redhat.com/ubi9/php-80:latest
...output omitted...
 * registry.access.redhat.com/ubi9/php-80@sha256:2b82...f544
 2 days ago

8.0-ubi8 (latest)
 tagged from registry.access.redhat.com/ubi8/php-80:latest
 * registry.access.redhat.com/ubi8/php-80@sha256:2c74...5ef4
 2 days ago
...output omitted...
```

If your OpenShift cluster administrator already updated the `php:8.0-ubi9` image stream tag, the `oc describe is` command shows multiple image IDs for that tag:

```
[user@host ~]$ oc describe is php -n openshift
Name: php
Namespace: openshift
...output omitted...

8.0-ubi9
 tagged from registry.access.redhat.com/ubi9/php-80:latest
...output omitted...
```

```
* registry.access.redhat.com/ubi9/php-80@sha256:2b82...f544
```

2 days ago

```
registry.access.redhat.com/ubi9/php-80@sha256:8840...94f0
```

5 days ago

```
registry.access.redhat.com/ubi9/php-80@sha256:506c...5d90
```

9 days ago

In the previous example, the asterisk (\*) shows which image ID is the current one for each image stream tag. It is usually the latest one to be imported, and the first one that is listed.

When an OpenShift image stream tag references a container image from an external registry, you must explicitly update the image stream tag to get new image IDs from the external registry. By default, OpenShift does not monitor external registries for changes to the image ID that is associated with an image name.

You can configure an image stream tag to check the external registry for updates on a defined schedule. By default, new image stream tags do not check for updated images.

## ***Image Stream Use Cases***

Image streams provide an evolution approach for the container workflows of an organization, and can improve deployment reliability and resilience.

As an example, an organization could start by downloading container images directly from the Red Hat public registry and later set up an enterprise registry as a mirror of those images to reduce bandwidth. OpenShift users would not notice any change, because they still refer to these images by using the same image stream name. Users of the RHEL container tools would notice the change, because those users would need either to change the registry names in their commands, or to change their container engine configurations to search for the local mirror first.

A common enterprise use case for image streams is to manage the promotion of application images across different environments, such as for development, testing, and production. A CI/CD pipeline can build an image and push it to an image stream tag such as `myapp:latest-dev`. After automated tests pass, the same immutable image digest can be tagged into the testing environment's image

stream, for example, as `myapp:qa-stable`. This use of the image stream tags ensures that the same artifact is tested. Finally, after successful QA, the image digest is tagged as `myapp:prod-v1.2`, triggering a production deployment. This process provides traceability and consistency throughout the application lifecycle.

In other scenarios, the registry-agnostic flexibility that an image stream provides can be helpful. Suppose that you start with a database container image with security issues, and the vendor takes too long to update the image with fixes. Later, you find another vendor who provides an alternative container image for the same database, where those security issues are already fixed, and even better, with a history of providing timely updates to them. If those container images have a compatible configuration of environment variables and volumes, then you could change your image stream to point to the image from the alternative vendor.

Red Hat provides hardened, supported container images, such as the MariaDB database, that work mostly as drop-in replacements of container images from popular open source projects. Replacing unreliable image sources with supported Red Hat alternatives, when available, is a preferred use of image streams.

## Creating Image Streams and Tags

In addition to the image streams in the `openshift` project, you can create image streams in your project so that the resources in that project, such as `Deployment` objects, can use them.

Use the `oc create is` command to create an image stream in the current project. The following example creates an image stream named `keycloak`:

```
[user@host ~]$ oc create is keycloak
```

After you create the image stream, you can add image stream tags. A common method is to import an image from a remote registry, which creates the image stream if it does not exist and adds the tag. Use the `oc import-image` command for this purpose. The following example adds the `25.0` tag to the `keycloak` image stream. In this example, the image stream tag refers to the `quay.io/keycloak/keycloak:25.0.2` image from the Quay.io public repository.

```
[user@host ~]$ oc import-image keycloak:25.0 \
--from=quay.io/keycloak/keycloak:25.0.2 --confirm
```

Alternatively, use the `oc tag SOURCE-IMAGE IMAGE-STREAM-TAG` command to add image stream tags to an existing image stream.

```
[user@host ~]$ oc tag quay.io/keycloak/keycloak:25.0.2 keycloak:25.0
```

Repeat the preceding command if you need more image stream tags:

```
[user@host ~]$ oc tag quay.io/keycloak/keycloak:19.0 keycloak:19.0
```

Use the `oc tag` command to update an image stream tag with a new source image reference. The following example changes the `keycloak:25.0` image stream tag to point to the `quay.io/keycloak/keycloak:25.0.3` image:

```
[user@host ~]$ oc tag quay.io/keycloak/keycloak:25.0.3 keycloak:25.0
```

Use the `oc describe is` command to verify that the image stream tag points to the SHA ID of the source image:

```
[user@host ~]$ oc describe is keycloak
Name: keycloak
Namespace: myproject
Created: 5 minutes ago
Labels: <none>
Annotations: openshift.io/image.dockerRepositoryCheck=2023-01-31T11:12:44Z
Image Repository: image-registry.openshift-image-registry.svc:5000/.../keycloak
Image Lookup: local=false
Unique Images: 3
Tags: 2

25.0
 tagged from quay.io/keycloak/keycloak:25.0.3

* quay.io/keycloak/keycloak@sha256:c167...62e9
 47 seconds ago
 quay.io/keycloak/keycloak@sha256:5569...b311
```

5 minutes ago

19.0

tagged from quay.io/keycloak/keycloak:19.0

\* quay.io/keycloak/keycloak@sha256:40cc...ffde

5 minutes ago

## ***Inspecting an Image Stream Definition***

To learn the structure of an image stream, you can inspect its YAML definition.

```
[user@host ~]$ oc get is/php -o yaml -n openshift
apiVersion: image.openshift.io/v1
kind: ImageStream
metadata:
...output omitted...

 name: php

 namespace: openshift
...output omitted...
spec:
 lookupPolicy:

 local: false

 tags:
 - annotations:

 description: Build and run PHP 7.4 applications on UBI 8. For more information
 about using this builder image, including OpenShift considerations, see https
 ://github.com/sclorg/s2i-php-container/blob/master/7.4/README.md.
...output omitted...

 version: "7.4"

 from:
```

```

 kind: DockerImage
 name: registry.ocp4.example.com:8443/ubi8/php-74:latest
 generation: 2
 importPolicy:
 importMode: Legacy
 name: 7.4-ubi8
 referencePolicy:
 type: Local
...output omitted...
status:
 dockerImageRepository: image-registry.openshift-image-registry.svc:5000/openshift/p
hp

 tags:
 - items:
 - created: "2025-09-12T09:34:46Z"
 dockerImageReference: registry.ocp4.example.com:8443/ubi8/php-74@sha256:32cb...
f691
 generation: 2
 image: sha256:32cb...f691
 tag: 7.4-ubi8
 - items:
 - created: "2025-09-12T09:34:46Z"
 dockerImageReference: registry.ocp4.example.com:8443/ubi8/php-80@sha256:0194...
aff6
 generation: 2
 image: sha256:0194...aff6
 tag: 8.0-ubi8
...output omitted...

```

The name of the ImageStream object.

The project where the image stream resides.

The `lookupPolicy` controls whether this image stream can be used to satisfy short image names in pod specifications within the same project.

The `spec.tags` section defines the intended state, and lists each tag and its source image.

The `status.dockerImageRepository` field shows the path to this image stream in the internal RHOC P registry.

The `status.tags` section shows the current state, including the history of image digests for each tag.

The `dockerImageReference` in the `status` section records the immutable digest (SHA ID) of the image that was imported for the tag.

## ***Importing Image Stream Tags Periodically***

When you create an image stream tag, OpenShift configures it with the SHA ID of the source image that you specify. After creation, the image stream tag does not change, even if the developer pushes a new version of the source image.

By using image stream tags, you are in control of the images that your applications are using. If you want to use a new image version, then you must manually update the image stream tag to point to that new version.

However, for some container registries that you trust, or for some specific images, you might prefer the image stream tags to refresh automatically.

For example, Red Hat regularly updates the images from the Red Hat Ecosystem Catalog with bug and security fixes. To benefit from these updates as soon as Red Hat releases them, you can configure your image stream tags to refresh regularly.

OpenShift can periodically verify whether a new image version is available. When OpenShift detects a new version, it automatically updates the image stream tag. To activate that periodic refresh, add the `--scheduled` option to the `oc tag` command.

```
[user@host ~]$ oc tag quay.io/keycloak/keycloak:25.0.3 keycloak:25.0 --scheduled
```

By default, OpenShift verifies the image every 15 minutes. The refresh period is a setting that your cluster administrators can adapt.

## ***Configuring Image Pull-through***

When OpenShift starts a pod that uses an image stream tag, it pulls the corresponding image from the source container registry.



When the image comes from a registry on the internet, pulling the image can take time, or even fail in the event of a network outage. Some public registries have bandwidth throttling rules that can slow down your downloads further.

To mitigate these issues, you can configure your image stream tags to cache the images in the OpenShift internal container registry. The first time that OpenShift pulls the image, it downloads the image from the source repository and then stores the image in its internal registry. After that initial pull, OpenShift retrieves the image from the internal registry.

To activate image pull-through, add the `--reference-policy local` option to the `oc tag` command.

```
[user@host ~]$ oc tag quay.io/keycloak/keycloak:25.0.3 keycloak:25.0 \
--reference-policy local
```

## Using Image Streams in Deployments

When you create a `Deployment` object, you can specify an image stream instead of a container image from a registry. Using an image stream in Kubernetes workload resources, such as deployments, requires preparation:

- Create the image stream object in the same project as the `Deployment` object.
- Enable the local lookup policy in the image stream object.
- In the `Deployment` object, reference the image stream tag by its name, such as `keycloak:25.0`, and not by the full image name from the source registry.

## ***Enabling the Local Lookup Policy***

When you use an image stream in a `Deployment` object, OpenShift looks for that image stream in the current project. However, OpenShift considers only the image streams with an enabled local lookup policy.

Use the `oc set image-lookup` command to enable the local lookup policy for an image stream:

```
[user@host ~]$ oc set image-lookup keycloak
```

Use the `oc describe is` command to verify that the policy is active:

```
[user@host ~]$ oc describe is keycloak
Name: keycloak
Namespace: myproject
Created: 3 hours ago
Labels: <none>
Annotations: openshift.io/image.dockerRepositoryCheck=2023-01-31T11:12:44Z
Image Repository: image-registry.openshift-image-registry.svc:5000/.../keycloak
Image Lookup: local=true
Unique Images: 3
Tags: 2
...output omitted...
```

You can also retrieve the local lookup policy status for all the image streams in the current project by running the `oc set image-lookup` command without parameters:

```
[user@host ~]$ oc set image-lookup
NAME LOCAL
keycloak true
zabbix-agent false
nagios false
```

To disable the local lookup policy, add the `--enabled=false` option to the `oc set image-lookup` command:

```
[user@host ~]$ oc set image-lookup keycloak --enabled=false
```

## ***Configuring Image Streams in Deployments***

When you create a Deployment object by using the `oc create deployment` command, use the `--image` option to specify the image stream tag:

```
[user@host ~]$ oc create deployment mykeycloak --image keycloak:25.0
```

When you use a short name, OpenShift looks for a matching image stream in the current project. OpenShift considers only the image streams with an enabled local lookup policy. If it does not find an image stream, then OpenShift looks for a regular container image in the allowed container registries. The reference documentation at the end of this lecture describes how to configure these allowed registries.

You can also use image streams with other Kubernetes workload resources:

- Job objects, which you can create by using the following command:

```
[user@host ~]$ oc create job NAME --image IMAGE-STREAM-TAG -- COMMAND
```

- CronJob objects, which you can create by using the following command:

```
[user@host ~]$ oc create cronjob NAME --image IMAGE-STREAM-TAG \
```

```
--schedule CRON-SYNTAX -- COMMAND
```

- Pod objects, which you can create by using the following command:

```
[user@host ~]$ oc run NAME --image IMAGE-STREAM-TAG
```

Another section in this course discusses how changing an image stream tag can automatically roll out the associated deployments. The *Automatic Image Updates with OpenShift Image Change Triggers* chapter in this course discusses how updating an image stream tag triggers automatic rollouts of associated deployments.

## Guided Exercise: Reproducible Deployments with OpenShift Image Streams

Deploy an application that references container images indirectly by using image streams.

### Outcomes

You should be able to create image streams and image stream tags, and deploy applications that use image stream tags.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `updates-imagestreams` project and the `/home/student/D0180/labs/updates-imagestreams/resources.txt` file. The `resources.txt` file contains the name of the images and some commands that you use during the exercise. You can use the file to copy and paste these image names and commands.

```
[student@workstation ~]$ lab start updates-imagestreams
```

## Instructions

1. Log in to the OpenShift cluster as the `developer` user with the `developer` password. Use the `updates-imagestreams` project.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

*...output omitted...*

5. Set the `updates-imagestreams` project as the active project.

```
6. [student@workstation ~]$ oc project updates-imagestreams
```

*...output omitted...*

2. Create the `versioned-hello` image stream and the `v1.0` image stream tag from the `registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0` image.

1. Use the `oc create is` command to create the image stream.

```
2. [student@workstation ~]$ oc create is versioned-hello
```

`imagestream.image.openshift.io/versioned-hello created`

3. Use the `oc tag` command to create the image stream tag.

```
4. [student@workstation ~]$ oc tag \
```

5. `registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0 \`
6. `versioned-hello:v1.0`

```
Tag versioned-hello:v1.0 set to registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0.
```

3. Enable image stream resolution for the `versioned-hello` image stream so that Kubernetes resources in the current project can use it.
1. Use the `oc set image-lookup` command to enable image lookup resolution.

2. `[student@workstation ~]$ oc set image-lookup versioned-hello`

```
imagestream.image.openshift.io/versioned-hello image lookup updated
```

3. Run the `oc set image-lookup` command without any arguments to verify your work.

4. `[student@workstation ~]$ oc set image-lookup`

5. NAME LOCAL

```
versioned-hello true
```

4. Review the image stream and confirm that the image stream tag refers to the source image by its SHA ID. Verify that the source image in the `registry.ocp4.example.com:8443` registry has the same SHA ID.
1. Retrieve the details of the `versioned-hello` image stream.

## NOTE

To improve readability, the instructions truncate the SHA-256 strings.

```
[student@workstation ~]$ oc describe is versioned-hello
Name: versioned-hello
Namespace: updates-imagestreams
Created: 7 minutes ago
...output omitted...

v1.0
```

```
tagged from registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0
```

```
* registry.ocp4.example.com:8443/.../versioned-hello@sha256:66e0...105e
```

```
7 minutes ago
```

2. Use the `oc image info` command to query the image from the classroom container registry. The SHA image ID is the same as the one from the image stream tag.

```
3. [student@workstation ~]$ oc image info \
```

```
4. registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0 | head
```

```
5. Name: registry.ocp4.example.com:8443/redhattraining/versioned-hello:v1.0
```

```
6. Digest: sha256:66e0...105e
```

```
7. Media Type: application/vnd.docker.distribution.manifest.v2+json
```

```
...output omitted...
```

5. Create a deployment named `version` that uses the `versioned-hello:v1.0` image stream tag.

1. Use the `oc create deployment` command to create the object.

```
2. [student@workstation ~]$ oc create deployment version --image versioned-hello:v1.0
```

```
deployment.apps/version created
```

3. Wait for the pod to start. You might have to rerun the command several times for the pod to report a `Running` status. The name of the pod on your system probably differs.

```
4. [student@workstation ~]$ oc get pods
```

| 5. NAME | READY | STATUS | RESTARTS | AGE |
|---------|-------|--------|----------|-----|
|---------|-------|--------|----------|-----|

|                          |     |         |   |       |
|--------------------------|-----|---------|---|-------|
| version-744bf7694b-bzhd2 | 1/1 | Running | 0 | 2m11s |
|--------------------------|-----|---------|---|-------|

6. Confirm that both the deployment and the pod refer to the image by its SHA ID.

1. Retrieve the image that the deployment uses. The deployment refers to the image from the source registry by its SHA ID. The v1.0 image stream tag also points to that SHA image ID.

```
2. [student@workstation ~]$ oc get deployment -o wide
3. ... IMAGES ...
```

```
... registry.ocp4.example.com:8443/.../versioned-hello@sha256:66e0...105e .
..
```

4. Retrieve the image that the pod is using. The pod is also referring to the image by its SHA ID.

```
5. [student@workstation ~]$ oc get pod version-744bf7694b-bzhd2 \
6. -o jsonpath='{.spec.containers[0].image}{"\n"}'
```

```
registry.ocp4.example.com:8443/redhattraining/versioned-hello@sha256:66e0...
.105e
```

## Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish updates-imagestreams
```

# Automatic Image Updates with OpenShift Image Change Triggers

## Objectives

- Ensure automatic update of application pods by using image streams with Kubernetes workload resources.

## Using Triggers to Manage Images

Image stream tags record the SHA ID of the source container image. Thus, an image stream tag always points to an immutable image.

If a new version of the source image becomes available, then you can change the image stream tag to point to that new image. However, a Deployment object that uses the image stream tag does not roll out automatically. For an automatic rollout, you must configure the Deployment object with an image trigger.

If you update an image stream tag to point to a new image version, and you notice that this version does not work as expected, then you can revert the image stream tag. Deployment objects for which you configured a trigger automatically roll back to that previous image.

Other Kubernetes workloads also support image triggers, such as Pod, CronJob, and Job objects.

## Configuring Image Triggers for Deployments

Before you can configure image triggers for a Deployment object, ensure that the Deployment object is using image stream tags for its containers:

- Create the image stream object in the same project as the Deployment object.
- Enable the local lookup policy in the image stream object by using the `oc set image-lookup` command.
- In the Deployment object, reference the image stream tags by their names, such as `keycloak:20`, and not by the full image names from the source registry.

Image triggers apply at the container level. If your Deployment object includes several containers, then you can specify a trigger for each one. Before you can set triggers, retrieve the container names:

```
[user@host ~]$ oc get deployment mykeycloak -o wide
```

| NAME       | READY | UP-TO-DATE | AVAILABLE | AGE | CONTAINERS | ... |
|------------|-------|------------|-----------|-----|------------|-----|
| mykeycloak | 0/1   | 1          | 0         | 6s  | keycloak   | ... |

Use the `oc set triggers` command to configure an image trigger for the container inside the Deployment object. Use the `--from-image` option to specify the image stream tag to watch.

```
[user@host ~]$ oc set triggers deployment/mykeycloak --from-image keycloak:20 \
```



## `--containers keycloak`

To provide automatic image rollout for Deployment objects, OpenShift adds the `image.openshift.io/triggers` annotation to store the configuration in JSON format.

The following example retrieves the content of the `image.openshift.io/triggers` annotation, and then uses the `jq` command to display the configuration in a more readable format:

```
[user@host ~]$ oc get deployment mykeycloak \
 -o jsonpath='{.metadata.annotations.image\.openshift\.io/triggers}' | jq .
[
 {
 "from": {
 "kind": "ImageStreamTag",
 "name": "keycloak:20"
 },
 "fieldPath": "spec.template.spec.containers[?(@.name==\"keycloak\")].image"
 }
]
```

The `fieldPath` attribute is a JSONPath expression that OpenShift uses to locate the attribute that stores the container image name. OpenShift updates that attribute with the new image name and SHA ID whenever the image stream tag changes.

For a more concise view, use the `oc set triggers` command with the name of the Deployment object as an argument:

```
[user@host ~]$ oc set triggers deployment/mykeycloak
```

| NAME                   | TYPE   | VALUE                  | AUTO |
|------------------------|--------|------------------------|------|
| deployments/mykeycloak | config |                        | true |
| deployments/mykeycloak | image  | keycloak:20 (keycloak) | true |

OpenShift uses the configuration trigger to roll out the deployment whenever you change its configuration, such as to update environment variables or to configure the readiness probe. OpenShift watches the `keycloak:20` image stream tag that the `keycloak` container uses.

The `true` value under the `AUTO` column indicates that the trigger is enabled.

You can disable the configuration trigger by using the `oc rollout pause` command, and you can re-enable it by using the `oc rollout resume` command.

You can disable the image trigger by adding the `--manual` option to the `oc set triggers` command:

```
[user@host ~]$ oc set triggers deployment/mykeycloak --manual \
--from-image keycloak:20 --containers keycloak
```

You re-enable the trigger by using the `--auto` option:

```
[user@host ~]$ oc set triggers deployment/mykeycloak --auto \
--from-image keycloak:20 --containers keycloak
```

You can remove the triggers from all the containers in the `Deployment` object by adding the `--remove-all` option to the command:

```
[user@host ~]$ oc set triggers deployment/mykeycloak --remove-all
```

## ***Rolling out Deployments***

A `Deployment` object with an image trigger automatically rolls out when the image stream tag changes.

The image stream tag might change because you manually update it to point to a new version of the source image. The image stream tag might also change automatically if you configure it for periodic refresh, by adding the `--scheduled` option to the `oc tag` command. When the image stream tag automatically changes, all the `Deployment` objects with a trigger that refers to that image stream tag also roll out.

## ***Rolling Back Deployments***

To roll back the Deployment objects, you revert the image stream tag. By reverting the image stream tag, OpenShift rolls out the Deployment object to use the previous image that the image stream tag is pointing to again.

### Managing Image Stream Tags

You can create image streams and image stream tags in several ways. The following three commands perform the same operation. They all create the keycloak image stream if it does not exist, and then create the keycloak:20.0.2 image stream tag:

```
[user@host ~]$ oc create istag keycloak:20.0.2 \
 --from-image quay.io/keycloak/keycloak:20.0.2

[user@host ~]$ oc import-image keycloak:20.0.2 \
 --from quay.io/keycloak/keycloak:20.0.2 --confirm

[user@host ~]$ oc tag quay.io/keycloak/keycloak:20.0.2 keycloak:20.0.2
```

You can rerun the `oc import-image` and `oc tag` commands to update the image stream tag from the source image. If the source image changes, then the commands update the image stream tag to point to that new version. However, you can use the `oc create istag` command only for the initial creation of the image stream tag. You cannot update tags by using the `oc create istag` command.

The `--help` option shows more details about these commands.

You can create several image stream tags that point to the same image. The following command creates the `keycloak:20` image stream tag, which points to the same image as the `keycloak:20.0.2` image stream tag. In this example, the `keycloak:20` image stream tag is an alias for the `keycloak:20.0.2` image stream tag.

```
[user@host ~]$ oc tag --alias keycloak:20.0.2 keycloak:20
```

The `oc describe` command reports that both tags point to the same image:

```
[user@host ~]$ oc describe is keycloak
```

```
Name: keycloak
```

```
Namespace: myproject
```

```
...output omitted...
```

### 20.0.2 (20)

```
tagged from quay.io/keycloak/keycloak:20.0.2
```

```
* quay.io/keycloak/keycloak@sha256:5569...b311
```

```
3 minutes ago
```

Using aliases for image tags is a similar concept to the use of floating tags for container images. Suppose that a new image version is available in the Quay.io repository. You can create an image stream tag for that new image:

```
[user@host ~]$ oc create istag keycloak:20.0.3 \
```

```
--from-image quay.io/keycloak/keycloak:20.0.3
```

```
imagestreamtag.image.openshift.io/keycloak:20.0.3 created
```

```
[user@host ~]$ oc describe is keycloak
```

```
Name: keycloak
```

```
Namespace: myproject
```

```
...output omitted...
```

### 20.0.3

```
tagged from quay.io/keycloak/keycloak:20.0.3
```

```
* quay.io/keycloak/keycloak@sha256:c167...62e9
```

```
36 seconds ago
```

### 20.0.2 (20)

```
tagged from quay.io/keycloak/keycloak:20.0.2
```

```
* quay.io/keycloak/keycloak@sha256:5569...b311
```

About an hour ago

The `keycloak:20` image stream tag does not change. Therefore, the `Deployment` objects that use that tag do not roll out.

After testing the new image, you can move the `keycloak:20` tag to point to the new image stream tag:

```
[user@host ~]$ oc tag --alias keycloak:20.0.3 keycloak:20
```

Tag `keycloak:20` set up to track `keycloak:20.0.3`.

```
[user@host ~]$ oc describe is keycloak
```

Name: keycloak

Namespace: myproject

...output omitted...

#### 20.0.3 (20)

tagged from `quay.io/keycloak/keycloak:20.0.3`

\* `quay.io/keycloak/keycloak@sha256:c167...62e9`

10 minutes ago

#### 20.0.2

tagged from `quay.io/keycloak/keycloak:20.0.2`

\* `quay.io/keycloak/keycloak@sha256:5569...b311`

About an hour ago

Because the `keycloak:20` image stream tag points to a new image, OpenShift rolls out all the `Deployment` objects that use that tag. These new deployments that use the new image should be tested to ensure that they perform as expected. If the new application does not work as expected, you can roll back the deployments by resetting the `keycloak:20` tag to the previous image stream tag, as shown here:

```
[user@host ~]$ oc tag --alias keycloak:20.0.2 keycloak:20
```

By providing a level of abstraction, image streams give you control over managing the container images that you use in your OpenShift cluster.

## Guided Exercise: Automatic Image Updates with OpenShift Image Change Triggers

Update an application that references container images indirectly through image streams.

### Outcomes

- Add an image trigger to a deployment.
- Modify an image stream tag to point to a new image.
- Watch the rollout of the application.
- Roll back a deployment to the previous image.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `updates-triggers` project and deploys a web application with 10 replicas.

The command creates the `/home/student/D0180/labs/updates-triggers/resources.txt` file. The `resources.txt` file contains the name of the images and some commands that you use during the exercise. You can use the file to copy and paste these image names and commands.

```
[student@workstation ~]$ lab start updates-triggers
```

### Instructions

1. Log in to the OpenShift cluster as the `developer` user with the `developer` password. Use the `updates-triggers` project.
1. Log in to the OpenShift cluster.

2. `[student@workstation ~]$ oc login -u developer -p developer \`
3. `https://api.ocp4.example.com:6443`
4. Login successful.

```
...output omitted...
```

5. Set the updates-triggers project as the active project.

```
6. [student@workstation ~]$ oc project updates-triggers
```

```
...output omitted...
```

2. Inspect the versioned-hello image stream that the lab command created.

1. Verify that the lab command enabled the local lookup policy for the versioned-hello image stream.

```
2. [student@workstation ~]$ oc set image-lookup
```

```
3. NAME LOCAL
```

```
versioned-hello true
```

4. Verify that the lab command created the versioned-hello:1 image stream tag. The image stream tag refers to the image in the classroom registry by its SHA ID.

## NOTE

To improve readability, the instructions truncate the SHA-256 strings.

On your system, the commands return the full SHA-256 strings. Also, you must type the full SHA-256 string, to provide such a parameter to a command.

```
[student@workstation ~]$ oc get istag
```

```
NAME IMAGE REFERENCE ...
```

```
versioned-hello:1 ...:8443/redhattraining/versioned-hello@sha256:66e0...105e ...
```

5. Verify that the lab command created the versioned-hello:1 image stream tag from the registry.ocp4.example.com:8443/redhattraining/versioned-hello:1-123 image.

```
6. [student@workstation ~]$ oc get istag versioned-hello:1 \
```

```
7. -o jsonpath='{.tag.from.name}{"\n"}'
```

```
registry.ocp4.example.com:8443/redhattraining/versioned-hello:1-123
```

3. Inspect the Deployment object that the `lab` command created. Verify that the application is available from outside the cluster.

1. List the Deployment objects. The version deployment retrieved the SHA image ID from the `versioned-hello:1` image stream tag.

The Deployment object includes a container named `versioned-hello`. You use that information in a later step, when you configure the trigger.

```
2. [student@workstation ~]$ oc get deployment -o wide
```

```
3. NAME READY ... CONTAINERS IMAGES ...
```

```
version 10/10 ... versioned-hello .../versioned-hello:1 ...
```

4. Open a new terminal.

5. Run the `/home/student/D0180/labs/updates-triggers/curl_loop.sh` script that the `lab` command prepared. The script sends web requests to the application in a loop. Leave the script running and do not interrupt it.

```
6. [student@workstation ~]$ /home/student/D0180/labs/updates-triggers/curl_loop.sh
```

```
7. Hi!
```

```
8. Hi!
```

```
9. Hi!
```

```
10. Hi!
```

```
...output omitted...
```

4. Add an image trigger to the Deployment object.

1. Return to the first terminal window, and then use the `oc set triggers` command to add the trigger for the `versioned-hello:1` image stream tag to the `versioned-hello` container.

```
2. [student@workstation ~]$ oc set triggers deployment/version \
```

```
3. --from-image versioned-hello:1 --containers versioned-hello
```

```
deployment.apps/version triggers updated
```



4. Review the definition of the trigger from the `image.openshift.io/triggers` annotation.

```
5. [student@workstation ~]$ oc get deployment version \
6. -o jsonpath='{.metadata.annotations.image\.openshift\.io/triggers}' | jq .
7. [
8. {
9. "from": {
10. "kind": "ImageStreamTag",
11. "name": "versioned-hello:1"
12. },
13. "fieldPath": "spec.template.spec.containers[?(@.name==\"versioned-hello
14. \")].image"
```

```
]

```

5. Update the `versioned-hello:1` image stream tag to point to the `1-125` tag of the `registry.ocp4.example.com:8443/redhattraining/versioned-hello` image. Watch the output of the `curl_loop.sh` script to verify that the Deployment object automatically rolls out.

1. Use the `oc tag` command to update the `versioned-hello:1` tag.

```
2. [student@workstation ~]$ oc tag \
3. registry.ocp4.example.com:8443/redhattraining/versioned-hello:1-125 \
4. versioned-hello:1
```

```
Tag versioned-hello:1 set to registry.ocp4.example.com:8443/redhattraining/
versioned-hello:1-125.
```

5. Changing the image stream tag triggered a rolling update. Watch the output of the `curl_loop.sh` script in the second terminal.

Before the update, only pods that use the earlier version of the image reply. During the rolling updates, both old and new pods respond. After the update, only pods that run the latest version of the image reply. The following output probably differs on your system, and after the rollout completes, shows only the `v1.1` version.

```
...output omitted...
Hi!
Hi!
Hi!
Hi!
Hi! v1.1
Hi! v1.1
Hi!
Hi! v1.1
Hi!
Hi! v1.1
Hi! v1.1
Hi! v1.1
Hi! v1.1
...output omitted...
```

Do not stop the script.

6. Inspect the Deployment object and the image stream.

1. List the version deployment and notice that the image changed.

```
2. [student@workstation ~]$ oc get deployment version -o wide
```

```
3. NAME READY ... CONTAINERS IMAGES ...
```

```
version 10/10 ... versioned-hello .../versioned-hello@sha256:834d...fcb4
...
```

4. Display the details of the versioned-hello image stream. The versioned-hello:1 image stream tag points to the image with the same SHA ID as in the Deployment object.

Notice that the preceding image is still available. In the following step, you roll back to that image by specifying its SHA ID.

```
[student@workstation ~]$ oc describe is versioned-hello
```

```
Name: versioned-hello
```

```
Namespace: updates-triggers
```

...output omitted...

1

tagged from registry.ocp4.example.com:8443/redhattraining/versioned-hello:1-125

\* registry.ocp4.example.com:8443/.../versioned-hello@sha256:834d...fcb4

6 minutes ago

registry.ocp4.example.com:8443/.../versioned-hello@sha256:66e0...105e

37 minutes ago

7. Roll back the Deployment object by reverting the versioned-hello:1 image stream tag.

1. Use the `oc tag` command. For the source image, copy and paste the old image name and the SHA ID from the output of the preceding command.

2. [student@workstation ~]\$ `oc tag \`

3. `registry.ocp4.example.com:8443/redhattraining/versioned-hello@sha256:66e0...105e \`

4. `versioned-hello:1`

Tag versioned-hello:1 set to registry.ocp4.example.com:8443/redhattraining/versioned-hello@sha256:66e0...105e.

5. Watch the output of the `curl_loop.sh` script in the second terminal. The pods that run the `v1.0` version of the application are responding again. The following output probably differs on your system, and after the rollout completes, shows only the `v1.0` version.

6. ....output omitted...

7. Hi! v1.1

8. Hi! v1.1

9. Hi! v1.1

10. Hi! v1.1

11. Hi!

12. Hi! v1.1

13. Hi! v1.1

```
14. Hi! v1.1
15. Hi!
16. Hi! v1.1
17. Hi! v1.1
18. Hi!
19. Hi!
20. Hi!
```

*...output omitted...*

Press **Ctrl+C** to end the script execution. Close the second terminal.

## Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish updates-triggers
```

## Lab: Manage Application Updates

Update two live applications to their latest releases as identified by non-floating tags.

### Outcomes

You should be able to configure `Deployment` objects with images and triggers, and configure image stream tags and aliases.

As the `student` user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. It also creates the `updates-review` project and deploys two applications, `app1` and `app2`, in that project.

The command creates the `/home/student/D0180/labs/updates-review/resources.txt` file. The `resources.txt` file contains the name of the images that you use during the exercise. You can use the file to copy and paste these image names.

```
[student@workstation ~]$ lab start updates-review
```

## Instructions

The API URL of your OpenShift cluster is `https://api.ocp4.example.com:6443`, and the `oc` command is already installed on your workstation machine.

Log in to the OpenShift cluster as the `developer` user with the `developer` password. Use the `updates-review` project for your work.

1. Your team created the `app1` deployment in the `updates-review` project from the `registry.ocp4.example.com:8443/redhattraining/php-ssl:latest` container image. Recently, a developer in your organization pushed a new version of the image and then reassigned the `latest` tag to that version.

Reconfigure the `app1` deployment to use the `1-222` static tag instead of the `latest` floating tag, to prevent accidental redeployment of your application with untested image versions that your developers can publish at any time.

1. Log in to the OpenShift cluster.

```
2. [student@workstation ~]$ oc login -u developer -p developer \
3. https://api.ocp4.example.com:6443
4. Login successful.
```

*...output omitted...*

5. Set the `updates-review` project as the active project.

```
6. [student@workstation ~]$ oc project updates-review
```

*...output omitted...*

7. Verify that the app1 deployment uses the latest tag. Retrieve the container name.

```
8. [student@workstation ~]$ oc get deployment/app1 -o wide
```

```
9. NAME READY ... CONTAINERS IMAGES ...
```

```
app1 1/1 ... php-ssl registry...:8443/redhattraining/php-ssl:latest
...
```

10. In the Deployment object, change the image to registry.ocp4.example.com:8443/redhattraining/php-ssl:1-222.

```
11. [student@workstation ~]$ oc set image deployment/app1 \
```

```
12. php-ssl=registry.ocp4.example.com:8443/redhattraining/php-ssl:1-222
```

```
deployment.apps/app1 image updated
```

13. Verify your work.

```
14. [student@workstation ~]$ oc get deployment/app1 -o wide
```

```
15. NAME READY ... CONTAINERS IMAGES ...
```

```
app1 1/1 ... php-ssl registry...:8443/redhattraining/php-ssl:1-222
...
```

2. The app2 deployment is using the php-ssl:1 image stream tag, which is an alias for the php-ssl:1-222 image stream tag.

Enable image triggering for the app2 deployment, so that whenever the php-ssl:1 image stream tag changes, OpenShift rolls out the application. You test your configuration in a later step, when you reassign the php-ssl:1 alias to a new image stream tag.

1. Retrieve the container name from the Deployment object.

```
2. [student@workstation ~]$ oc get deployment/app2 -o wide
```

```
3. NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS ...
```

```
app2 1/1 1 1 21m php-ssl ...
```

4. Add the image trigger to the Deployment object.

```
5. [student@workstation ~]$ oc set triggers deployment/app2 \
6. --from-image php-ssl:1 --containers php-ssl
```

```
deployment.apps/app2 triggers updated
```

## 7. Verify your work.

```
8. [student@workstation ~]$ oc set triggers deployment/app2
```

| 9. NAME              | TYPE   | VALUE | AUTO |
|----------------------|--------|-------|------|
| 10. deployments/app2 | config |       | true |

```
deployments/app2 image php-ssl:1 (php-ssl) true
```

3. A new image version, registry.ocp4.example.com:8443/redhattraining/php-ssl:1-234, is available in the container registry. Your QA team tested and approved that version. It is ready for production.

Create the php-ssl:1-234 image stream tag that points to the new image. Move the php-ssl:1 image stream tag alias to the new php-ssl:1-234 image stream tag. Verify that the app2 application redeploys.

1. Create the php-ssl:1-234 image stream tag.

```
2. [student@workstation ~]$ oc create istag php-ssl:1-234 \
3. --from-image registry.ocp4.example.com:8443/redhattraining/php-ssl:1-234
```

```
imagestreamtag.image.openshift.io/php-ssl:1-234 created
```

4. Move the php-ssl:1 alias to the new php-ssl:1-234 image stream tag.

```
5. [student@workstation ~]$ oc tag --alias php-ssl:1-234 php-ssl:1
```

```
Tag php-ssl:1 set up to track php-ssl:1-234.
```

6. Verify that the app2 application rolls out. The names of the replica sets on your system probably differ.

```
7. [student@workstation ~]$ oc describe deployment/app2
```

|               |                |
|---------------|----------------|
| 8. Name:      | app2           |
| 9. Namespace: | updates-review |

```
10. ...output omitted...
```

```
11. Events:
```

```
12. Type ... Age Message
```

```
13. ---- ---- -
```

```
14. Normal ... 33m Scaled up replica set app2-7dd589f6d5 to 1
```

```
15. Normal ... 3m30s Scaled up replica set app2-7bf5b7787 to 1
```

```
Normal ... 3m28s Scaled down replica set app2-7dd589f6d5 to 0 from 1
```

## Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade updates-review
```

## Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish updates-review
```

## Summary

- You reference container images by tags or by SHA IDs. Image developers assign tags to images. Container registries compute and assign SHA IDs to images.
- Deployment objects have an `imagePullPolicy` attribute that specifies how compute nodes pull the image from the registry.
- A deployment strategy is a way of changing or upgrading your application to minimize the impact of those changes.
- Deployment objects support the rolling update and the re-create deployment strategies.



- OpenShift image stream and image stream tag resources provide stable references to container images.
- Kubernetes workload resources, such as Deployments and Jobs, can use image streams. You must create the image streams in the same project as the Kubernetes resources, and you must enable the local lookup policy in the image streams to use them.
- You can configure image monitoring in deployments so that OpenShift rolls out the application whenever the image stream tag changes.

## Chapter 8. Comprehensive Review

### Comprehensive Review

#### Lab: Deploy Web Applications

#### Lab: Troubleshoot and Scale Applications

### Abstract

|                 |                                                                                                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Goal</b>     | Review tasks from <i>Red Hat OpenShift Administration I: Operating a Production Cluster</i> .                                                                          |
| <b>Sections</b> | <ul style="list-style-type: none"> <li>• Comprehensive Review</li> <li>• Deploy Web Applications (Lab)</li> <li>• Troubleshoot and Scale Applications (Lab)</li> </ul> |
| <b>Lab</b>      | <ul style="list-style-type: none"> <li>• Deploy Web Applications</li> <li>• Troubleshoot and Scale Applications</li> </ul>                                             |

## Comprehensive Review

### Objectives

After completing this section, you should have reviewed and refreshed the knowledge and skills that you learned in *Red Hat OpenShift Administration I: Operating a Production Cluster*.

## *Reviewing Red Hat OpenShift Administration I: Operating a Production Cluster*

Before beginning the comprehensive review for this course, you should be comfortable with the topics covered in each chapter. Do not hesitate to ask the instructor for extra guidance or clarification on these topics.

### ***Chapter 1, Introduction to Kubernetes and OpenShift***

Identify the main Kubernetes cluster services and OpenShift platform services and monitor them by using the web console.

### ***Chapter 2, Kubernetes and OpenShift Command-line Interfaces and APIs***

Access an OpenShift cluster by using the command line and query its Kubernetes API resources to assess the health of a cluster.

### ***Chapter 3, Run Applications as Containers and Pods***

Run and troubleshoot containerized applications as unmanaged Kubernetes pods.

### ***Chapter 4, Deploy Managed and Networked Applications on Kubernetes***

Deploy applications and expose them to network access from inside and outside a Kubernetes cluster.

### ***Chapter 5, Manage Storage for Application Configuration and Data***

Externalize application configurations in Kubernetes resources and provision storage volumes for persistent data files.

### ***Chapter 6, Configure Applications for Reliability***

Configure applications to work with Kubernetes for high availability and resilience.

## ***Chapter 7, Manage Application Updates***

Manage reproducible application updates and rollbacks of code and configurations.

### **Lab: Deploy Web Applications**

Use image streams with Kubernetes workload resources to ensure reproducibility of application deployments.

Configure applications by using Kubernetes secrets to initialize environment variables.

Provide applications with persistent storage volumes.

Expose applications to clients outside the cluster.

#### **Outcomes**

You should be able to create and configure OpenShift and Kubernetes resources, such as projects, secrets, deployments, persistent volumes, services, and routes.

#### **NOTE**

The allocated time for this activity is 35 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. The command also creates the `/home/student/D00018L/labs/compreview-deploy/resources.txt` file. The `resources.txt` file contains the URLs of your OpenShift cluster and the image names that you use in the exercise. You can use the file to copy and paste these URLs and image names.

```
[student@workstation ~]$ lab start compreview-deploy
```

#### **Specifications**

You are a developer who is responsible for deploying the Famous Authors web application, which your company created to list quotations from famous authors for people to use as references. This application consists of a UI and a MySQL database to store the quotations.

- The UI and MySQL database require project workloads that reference the database image by the `mysql8:1` short name.
  - The name of the project must be `review`.
- The application must be a database that matches the following criteria:
  - A database image stream named `mysql8:1` that points to the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-228` container image.

## NOTE

The database image name and its source registry might change in the future, you need to isolate the workload from that change.

- MySQL database parameters that are stored in a secret named `dbparams` with the `MYSQL_` prefix for each variable name.

| Name     | Value     |
|----------|-----------|
| user     | operator1 |
| password | redhat123 |
| database | quotesdb  |

- The data is preserved with no more than 2 GiB of disk space by using `/var/lib/mysql` as the working directory, and the `lvms-vg1` storage class.
- The database must be automatically deployed whenever the source container in the `mysql8:1` resource changes. To test your configuration, you can use the `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` container image.
- The UI is an HTML application in the `registry.ocp4.example.com:8443/redhattraining/famous-quotes:2-42` image.

- The front end of the application requires using environment variables from the same secret that the database uses.
  - The configuration of the environment variables must match the following criteria:

| Environment variable name | Value                                     |
|---------------------------|-------------------------------------------|
| QUOTES_USER               | The user key from the dbparams secret     |
| QUOTES_PASSWORD           | The password key from the dbparams secret |
| QUOTES_DATABASE           | The database key from the dbparams secret |
| QUOTES_HOSTNAME           | quotesdb                                  |

- The application must be accessible from outside the cluster by using the `http://frontend-review.apps.ocp4.example.com` route.
  - The application service must listen on port 8000.

## WARNING

Before the resulting configuration is ready for production, you must configure probes and resource limits. Another comprehensive review exercise covers these subjects.

## NOTE

The password for the developer user is `developer`, and the password for the admin user is `redhatocp`, although you do not need administrator privileges to complete this exercise.

1. Use the `oc create istag NAME --from-image IMAGE-URL` command to create an image stream and an image stream tag.
2. Use the `oc set image-lookup IMAGE-NAME` command to enable image lookup resolution.
3. Use the `oc create secret generic NAME --from-literal user=operator1 --from-literal password=redhat123 --from-literal database=quotesdb` command to create a generic secret.

4. Use the `oc create deployment NAME --image IMAGE-NAME --replicas 0` command to create a deployment.
5. Use the `oc set triggers` command to add the trigger for the `mysql8:1` image stream tag to the `mysql8` container.
6. Use the `oc set env deployment/NAME --from secret/NAME --prefix MYSQL_` command to add environment variables to a deployment from a secret.
7. Use the `oc set volumes deployment/NAME --add --claim-class lvms-vg1 --claim-size 2Gi --mount-path PATH` to add a persistent volume to a deployment.
8. Use the `oc expose deployment NAME --port 3306` command to create a service for a deployment.
9. Use the `oc expose service SERVICE-NAME` command to create a route.
10. Use the `curl` command to test the application.

## Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

## NOTE

The grading scripts provide remediation hints to help you identify what content you need to revisit to successfully complete the tasks.

```
[student@workstation ~]$ lab grade compreview-deploy
```

Evaluate the time allocated for this activity against the time it took you to complete it. If you used more time than the allocated, then you must revisit the course content again, or practice more.

## Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-deploy
```

## Lab: Troubleshoot and Scale Applications

Navigate the OpenShift web console to identify CPU-consuming workloads.

Troubleshoot and fix a failed MySQL pod.

Manually scale an application.

Configure health probes.

## Outcomes

You should be able to troubleshoot malfunctioning workloads, configure deployments, and scale applications.

## NOTE

The allocated time for this activity is 20 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

As the `student` user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that all resources are available for this exercise. The command also creates the `compreview-scale` project and deploys some applications in that project.

The command creates the `/home/student/D00018L/labs/compreview-scale/resources.txt` file. The `resources.txt` file contains the URLs of your OpenShift cluster and the name of the images that you use during the exercise. You can use the file to copy and paste these URLs and image names.

```
[student@workstation ~]$ lab start compreview-scale
```

## Specifications

You are an administrator of an OpenShift cluster, and you must remediate the Famous Authors application. The application returns a list of quotations from famous authors, and it consists of a UI and a MySQL database to store the quotations. The application is broken for now, and is missing some configuration to be ready for production.

A pod in the cluster is consuming excessive CPU and is interfering with other tasks. You need to identify the pod and remove its workload.

The application uses two Kubernetes `Deployment` objects. The `frontend` deployment provides the application web pages and relies on the `quotesdb` deployment that runs a MySQL database. The `lab` command already created the services and routes that connect the application components and that make the application available from outside the cluster.

- The database requires the following parameters and criteria:

| Name          | Value     |
|---------------|-----------|
| Username      | operator1 |
| Password      | redhat123 |
| Database name | quotes    |

- Your security team validated a new version of the MySQL container image that fixes a security issue. The new container image is `registry.ocp4.example.com:8443/rhel9/mysql-80:1-237`. The classroom setup copied the image from the Red Hat Ecosystem Catalog. The original image is `registry.redhat.io/rhel9/mysql-80:1-237`.
- The database must redeploy.
- The database requires probes to detect when the database is ready to accept requests, and to verify regularly the status of the database.
- The deployment needs 200 millicores of CPU and 256 MiB of memory to run, and you must restrict its CPU usage to 500 millicores and its memory usage to 1 GiB.
- The UI requires the following criteria:
  - The UI requires probes: One probe to detect when the web application is ready to accept requests on port 8000 to the `/status` path, and a second probe that regularly verifies the status of the web front end on port 8000 to the `/env` path.
  - Requires 200 millicores of CPU and 256 MiB of memory, and a limit of 500 millicores of CPU usage and 512 MiB memory usage.
  - High availability of the application.



## NOTE

The password for the developer user is developer, and the password for the admin user is redhatocp.

1. Go to **Observe** → **Dashboards** and select the **Kubernetes / Compute Resources / Namespace (Workloads)** dashboard to identify the deployment that consumes excessive CPU.
2. Use the `oc set env deployment/NAME MYSQL_USER=operator1 MYSQL_PASSWORD=redhat123 MYSQL_DATABASE=quotes` command to add the missing environment variables to the quotesdb deployment.
3. Use the `oc set image deployment/NAME mysql-80=registry.ocp4.example.com:8443/rhel9/mysql-80:1-237` command to update the MySQL container image for the quotesdb deployment.
4. Use the `oc set probe deployment/NAME --readiness --mysqladmin ping` command to add a readiness probe to the quotesdb deployment that runs the mysqladmin ping command.
5. Use the `oc set probe deployment/NAME --liveness --mysqladmin ping` command to add a liveness probe to the quotesdb deployment that runs the mysqladmin ping command.
6. Use the `oc set resources deployment/quotesdb --requests cpu=200m,memory=256Mi --limits cpu=500m,memory=1Gi` command to set the CPU request to 200 millicores, the memory request to 256 MiB, the CPU limit to 500 millicores, and the memory limit to 1 GiB for the quotesdb deployment.
7. Use the `oc set probe deployment/NAME --readiness --get-url http://:8000/status` command to add a readiness probe to the frontend deployment that tests the /status path on HTTP port 8000.
8. Use the `oc set probe deployment/NAME --liveness --get-url http://:8000/env` command to add a readiness probe to the frontend deployment that tests the /env path on HTTP port 8000.
9. Use the `oc scale deployment/NAME --replicas 3` command to scale the frontend deployment to three pods.
10. Use the `curl http://frontend-compreview-scale.apps.ocp4.example.com` command to test the application.

## Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

## NOTE

The grading scripts provide remediation hints to help you identify what content you need to revisit to successfully complete the tasks.

```
[student@workstation ~]$ lab grade compreview-scale
```

Evaluate the time allocated for this activity against the time it took you to complete it. If you used more time than the allocated, then you must revisit the course content again, or practice more.

## Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-scale
```

---